

.NET Multi-platform App UI documentation

.NET Multi-platform App UI (.NET MAUI) lets you build native apps using a .NET cross-platform UI toolkit that targets the mobile and desktop form factors on Android, iOS, macOS, Windows, and Tizen.



OVERVIEW
[What is .NET MAUI?](#)



OVERVIEW
[Supported platforms](#)



DOWNLOAD
[Installation](#)



TUTORIAL
[Build your first app](#)



TRAINING
[Build mobile and desktop apps with .NET MAUI](#)



ARCHITECTURE
[Enterprise application patterns using .NET MAUI](#)



SAMPLE
[Find a sample](#)



WHAT'S NEW
[What's new?](#)

.NET MAUI common tasks

Upgrade from Xamarin

Run your Xamarin projects on .NET after an upgrade process.

 [Upgrade from Xamarin to .NET](#)

Learn about the app lifecycle

Understand how apps start and go to the background.

 [App lifecycle](#)

Use XAML & data-binding

Show a list of data

Use the MVVM pattern to show data in your UI.

 [Get started with XAML](#)

Use the CollectionView to scroll and select data.

 [Present lists of data](#)

Customize UIs for specific platforms

Make the most effective use of space in your app UI.

 [Customize UI appearance based on the platform](#)

Access a REST web service

Integrate and interact with web services using HTTP and JSON

 [Consume a REST-based web service](#)

Access device features

Use cross-platform APIs for common device features.

 [Platform integration](#)

Deploy apps to devices

Upload to test devices and app stores.

 [Deployment](#)

.NET MAUI guides

Get started

-  [What is .NET MAUI?](#)
-  [Supported platforms](#)
-  [Installation](#)
-  [Build your first app](#)
-  [Create a .NET MAUI app](#)
-  [Upgrade your app with MVVM](#)
-  [Learning resources](#)
-  [Troubleshooting](#)

Migrate from Xamarin

-  [Upgrade from Xamarin to .NET](#)
-  [Upgrade Xamarin native projects](#)
-  [Upgrade with the .NET Upgrade Assistant](#)
-  [Upgrade to a multi-project app](#)
-  [Upgrade to a single project app](#)
-  [Layout behavior changes](#)
-  [Migrate app properties data](#)
-  [Migrate secure storage data](#)

Fundamentals

-  [Accessibility](#)
-  [App lifecycle](#)
-  [Data binding](#)
-  [Data templates](#)
-  [Localization](#)
-  [Resource dictionaries](#)
-  [Shell](#)

User interface

-  [Animation](#)
-  [Brushes](#)
-  [Controls](#)
-  [Graphics](#)
-  [Handlers](#)
-  [Layouts](#)
-  [Pages](#)

 [Single project](#)

 [Styles](#)

XAML

-  [Overview](#)
-  [Fundamentals](#)
-  [Compilation](#)
-  [Generics](#)
-  [Markup extensions](#)
-  [Namespaces](#)
-  [Pass arguments](#)
-  [XAML hot reload](#)

Platform integration

-  [Configure multi-targeting](#)
-  [Customize UI appearance](#)
-  [Device information](#)
-  [Device sensors](#)
-  [Invoke platform code](#)
-  [Native embedding](#)
-  [Permissions](#)
-  [Preferences](#)

Data & cloud services

-  [Build a .NET MAUI app with Azure Mobile Apps](#)
-  [Add authentication with Azure Mobile Apps](#)
-  [Offline data sync with Azure Mobile Apps](#)
-  [Send push notifications using Azure Notification Hubs](#)
-  [Consume a REST-based web service](#)
-  [Connect to local web services](#)
-  [Local databases](#)

Deployment

-  [Overview](#)
-  [Hot restart](#)
-  [Improve app performance](#)
-  [Native AOT](#)
-  [Publish Android apps](#)
-  [Publish iOS apps](#)
-  [Publish Mac Catalyst apps](#)
-  [Publish Windows apps](#)
-  [Trimming](#)
-  [Wireless deployment](#)

.NET MAUI Blazor Hybrid apps

Build a .NET MAUI Blazor Hybrid app

Build and run your first .NET MAUI Blazor Hybrid app in Visual Studio.

Blazor Hybrid routing and navigation

Understand how to manage request routing and navigation in Blazor Hybrid apps.

Build a mobile and desktop app with Blazor Hybrid and .NET MAUI

Build a cross-platform hybrid app with Blazor, .NET MAUI, and C#.

.NET MAUI community resources

[.NET MAUI Community Toolkit](#)

The .NET MAUI Community Toolkit is a collection of reusable elements for .NET MAUI app development.

[.NET Community Toolkit](#)

The .NET Community Toolkit is a collection of helpers and APIs that are agnostic of any UI platform.

[Windows Community Toolkit](#)

The Windows Community Toolkit is a collection of controls that simplifies common developer tasks for...

API reference

[.NET MAUI API reference](#)

API reference documentation for .NET MAUI.

[.NET API reference](#)

API reference documentation for .NET.

[.NET Platform Extensions API reference](#)

API reference documentation for .NET Platform Extensions.

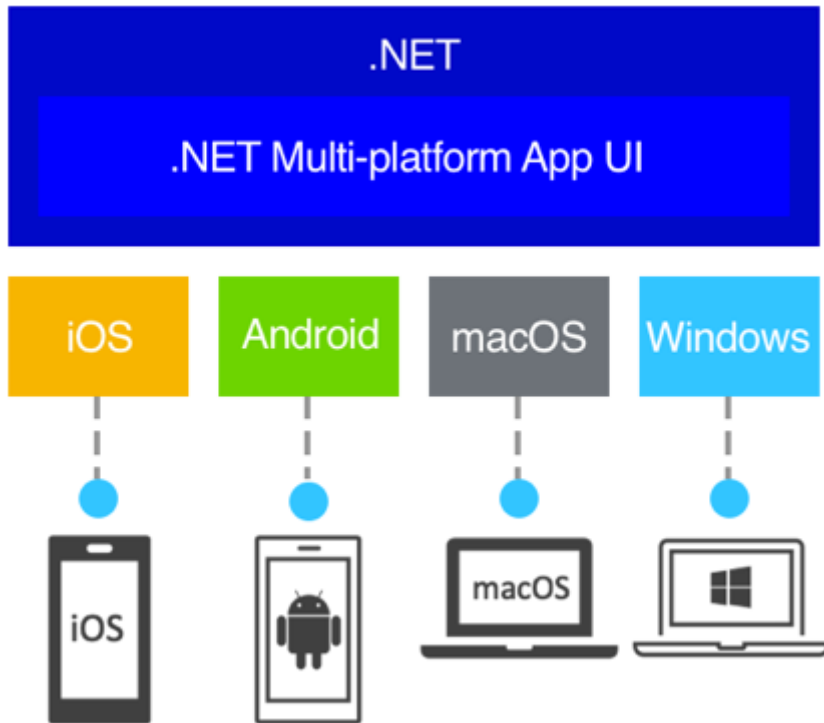
[Blogs](#) - [Microsoft Q&A](#) - [Developer Community](#) - [Release notes](#)

What is .NET MAUI?

Article • 06/21/2024

.NET Multi-platform App UI (.NET MAUI) is a cross-platform framework for creating native mobile and desktop apps with C# and XAML.

Using .NET MAUI, you can develop apps that can run on Android, iOS, macOS, and Windows from a single shared code-base.



.NET MAUI is open-source and is the evolution of Xamarin.Forms, extended from mobile to desktop scenarios, with UI controls rebuilt from the ground up for performance and extensibility. If you've previously used Xamarin.Forms to build cross-platform user interfaces, you'll notice many similarities with .NET MAUI. However, there are also some differences. Using .NET MAUI, you can create multi-platform apps using a single project, but you can add platform-specific source code and resources if necessary. One of the key aims of .NET MAUI is to enable you to implement as much of your app logic and UI layout as possible in a single code-base.

Who .NET MAUI is for

.NET MAUI is for developers who want to:

- Write cross-platform apps in XAML and C#, from a single shared code-base in Visual Studio.

- Share UI layout and design across platforms.
- Share code, tests, and business logic across platforms.

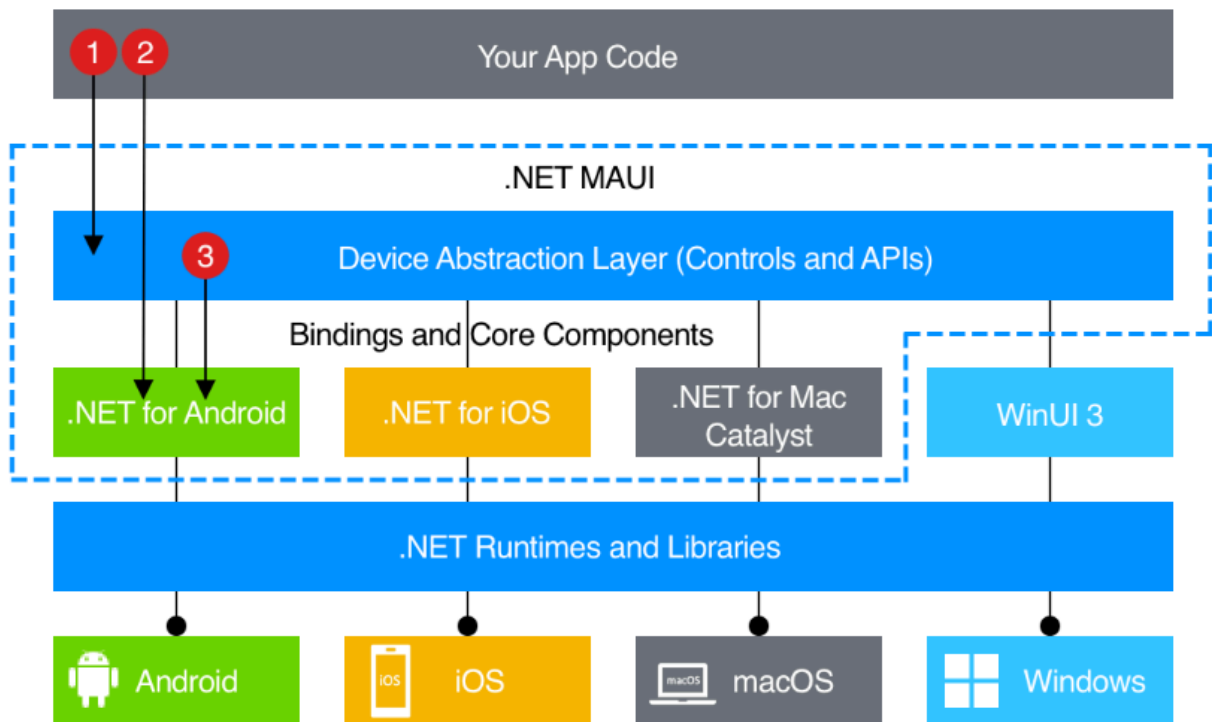
How .NET MAUI works

.NET MAUI unifies Android, iOS, macOS, and Windows APIs into a single API that allows a write-once run-anywhere developer experience, while additionally providing deep access to every aspect of each native platform.

.NET 6 or greater provides a series of platform-specific frameworks for creating apps: .NET for Android, .NET for iOS, .NET for Mac Catalyst, and Windows UI 3 (WinUI 3) library. These frameworks all have access to the same .NET Base Class Library (BCL). This library abstracts the details of the underlying platform away from your code. The BCL depends on the .NET runtime to provide the execution environment for your code. For Android, iOS, and macOS, the environment is implemented by Mono, an implementation of the .NET runtime. On Windows, .NET CoreCLR provides the execution environment.

While the BCL enables apps running on different platforms to share common business logic, the various platforms have different ways of defining the user interface for an app, and they provide varying models for specifying how the elements of a user interface communicate and interoperate. You can craft the UI for each platform separately using the appropriate platform-specific framework (.NET for Android, .NET for iOS, .NET for Mac Catalyst, or WinUI 3), but this approach then requires you to maintain a code-base for each individual family of devices.

.NET MAUI provides a single framework for building the UIs for mobile and desktop apps. The following diagram shows a high-level view of the architecture of a .NET MAUI app:



In a .NET MAUI app, you write code that primarily interacts with the .NET MAUI controls and API layer (1). This layer then directly consumes the native platform APIs (3). In addition, app code may directly exercise platform APIs (2), if required.

.NET MAUI apps can be written on PC or Mac, and compile into native app packages:

- Android apps built using .NET MAUI compile from C# into an intermediate language (IL) which is then just-in-time (JIT) compiled to a native assembly when the app launches.
- iOS apps built using .NET MAUI are fully ahead-of-time (AOT) compiled from C# into native ARM assembly code.
- macOS apps built using .NET MAUI use Mac Catalyst, a solution from Apple that brings your iOS app built with UIKit to the desktop, and augments it with additional AppKit and platform APIs as required.
- Windows apps built using .NET MAUI use Windows UI 3 (WinUI 3) library to create native apps that target the Windows desktop. For more information about WinUI 3, see [Windows UI Library](#).

📌 Note

Building apps for iOS and macOS requires a Mac.

What .NET MAUI provides

.NET MAUI provides a collection of controls that can be used to display data, initiate actions, indicate activity, display collections, pick data, and more. In addition to a collection of controls, .NET MAUI also provides:

- An elaborate layout engine for designing pages.
- Multiple page types for creating rich navigation types, like drawers.
- Support for data-binding, for more elegant and maintainable development patterns.
- The ability to customize handlers to enhance the way in which UI elements are presented.
- Cross-platform APIs for accessing native device features. These APIs enable apps to access device features such as the GPS, the accelerometer, and battery and network states. For more information, see [Cross-platform APIs for device features](#).
- Cross-platform graphics functionality, that provides a drawing canvas that supports drawing and painting shapes and images, compositing operations, and graphical object transforms.
- A single project system that uses multi-targeting to target Android, iOS, macOS, and Windows. For more information, see [.NET MAUI Single project](#).
- .NET hot reload, so that you can modify both your XAML and your managed source code while the app is running, then observe the result of your modifications without rebuilding the app. For more information, see [.NET hot reload](#).

Cross-platform APIs for device features

.NET MAUI provides cross-platform APIs for native device features. Examples of functionality provided by .NET MAUI for accessing device features includes:

- Access to sensors, such as the accelerometer, compass, and gyroscope on devices.
- Ability to check the device's network connectivity state, and detect changes.
- Provide information about the device the app is running on.
- Copy and paste text to the system clipboard, between apps.
- Pick single or multiple files from the device.
- Store data securely as key/value pairs.
- Utilize built-in text-to-speech engines to read text from the device.
- Initiate browser-based authentication flows that listen for a callback to a specific app registered URL.

Single project

.NET MAUI single project takes the platform-specific development experiences you typically encounter while developing apps and abstracts them into a single shared

project that can target Android, iOS, macOS, and Windows.

.NET MAUI single project provides a simplified and consistent cross-platform development experience, regardless of the platforms being targeted. .NET MAUI single project provides the following features:

- A single shared project that can target Android, iOS, macOS, and Windows.
- A simplified debug target selection for running your .NET MAUI apps.
- Shared resource files within the single project.
- A single app manifest that specifies the app title, id, and version.
- Access to platform-specific APIs and tools when required.
- A single cross-platform app entry point.

.NET MAUI single project is enabled using multi-targeting and the use of SDK-style projects. For more information about .NET MAUI single project, see [.NET MAUI single project](#).

Hot reload

.NET MAUI includes support for .NET hot reload, which enables you to modify your managed source code while the app is running, without the need to manually pause or hit a breakpoint. Then, your code edits can be applied to your running app without recompilation.

.NET MAUI also includes support for XAML hot reload, which enables you to save your XAML files and see the changes reflected in your running app without recompilation. In addition, your navigation state and data will be maintained, enabling you to quickly iterate on your UI without losing your place in the app.

Supported platforms for .NET MAUI apps

Article • 10/04/2024

.NET Multi-platform App UI (.NET MAUI) apps can be written for the following platforms:

- Android 5.0 (API 21) or higher is required.
- iOS 12.2 or higher is required.
- macOS 12 or higher, using Mac Catalyst.
- Windows 11 and Windows 10 version 1809 or higher, using [Windows UI Library \(WinUI\) 3](#).

.NET MAUI Blazor apps have the following additional platform requirements:

- Android 7.0 (API 24) or higher is required.
- iOS 14 or higher is required.
- macOS 12 or higher, using Mac Catalyst.

.NET MAUI Blazor apps also require an updated platform specific WebView control. For more information, see [Blazor supported platforms](#).

Important

For information about the version of Xcode, the Android SDK and JDK, and the Windows App SDK, that's required for a specific version of .NET MAUI, see [Release versions](#) .

.NET MAUI apps for Android, iOS, and Windows can be built in Visual Studio. However, a networked Mac is required for iOS development.

On Windows, .NET MAUI apps for Android, iOS, and Windows can also be built with Visual Studio Code when using the .NET MAUI extension. On macOS, .NET MAUI apps for Android, iOS, and macOS can be built in Visual Studio Code when using the .NET MAUI extension.

Additional platform support

.NET MAUI also includes Tizen support, which is provided by Samsung.

Installation

Article • 11/12/2024

Developing native, cross-platform .NET Multi-platform App UI (.NET MAUI) apps requires Visual Studio 2022 17.12 or greater, or the latest Visual Studio Code with the .NET MAUI extension.

Visual Studio

To start developing native, cross-platform .NET MAUI apps on Windows, install Visual Studio 2022 17.12 or greater by following the [installation](#) steps.

Prerequisites

- Visual Studio 2022 17.12 or greater. For information about supported operating systems, hardware, supported languages, and additional requirements and guidance, see [Visual Studio 2022 System Requirements](#).

To build, sign, and deploy .NET MAUI apps for iOS, you'll also need:

- A Mac that is compatible with Xcode. For more information, see Apple's [minimum requirements documentation](#).
- A specific version of Xcode, which depends on the version of .NET MAUI that you're using. For information, see [Release versions](#).
- An [Apple ID](#) and paid [Apple Developer Program](#) enrollment. An Apple ID is required to deploy apps to devices, and to submit apps to the Apple Store.

Alternatively, to deploy debug builds of your app directly from Windows to your iOS device with [hot restart](#), you'll need:

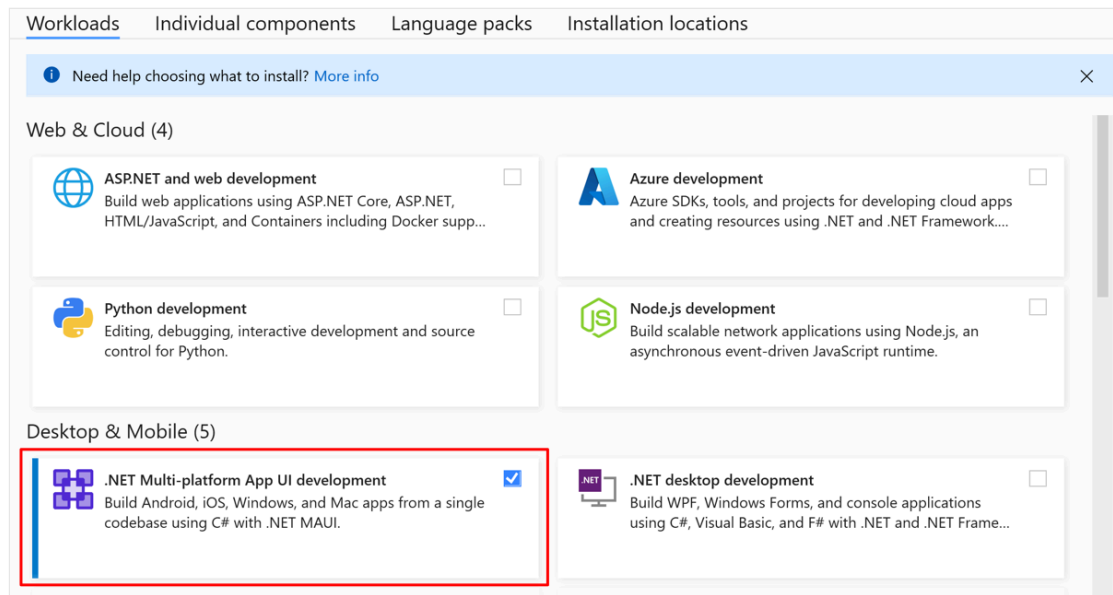
- An [Apple Developer account](#) and paid [Apple Developer Program](#) enrollment.

Installation

1. To create .NET MAUI apps, you'll need the latest version of Visual Studio 2022:

- [Download Visual Studio 2022 Community](#)
- [Download Visual Studio 2022 Professional](#)
- [Download Visual Studio 2022 Enterprise](#)

2. Either install Visual Studio, or modify your existing installation, and install the .NET Multi-platform App UI development workload with its default optional installation options:



Next steps

To learn how to create and run your first .NET MAUI app in Visual Studio 2022 on Windows, or Visual Studio Code, click the button below.

[Build your first app](#)

Build your first app

Article • 11/12/2024

In this tutorial, you'll learn how to create and run your first .NET Multi-platform App UI (.NET MAUI) app in Visual Studio 2022 on Windows or Visual Studio Code on Windows, macOS, or Linux. This will help to ensure that your development environment is correctly set up.

Visual Studio

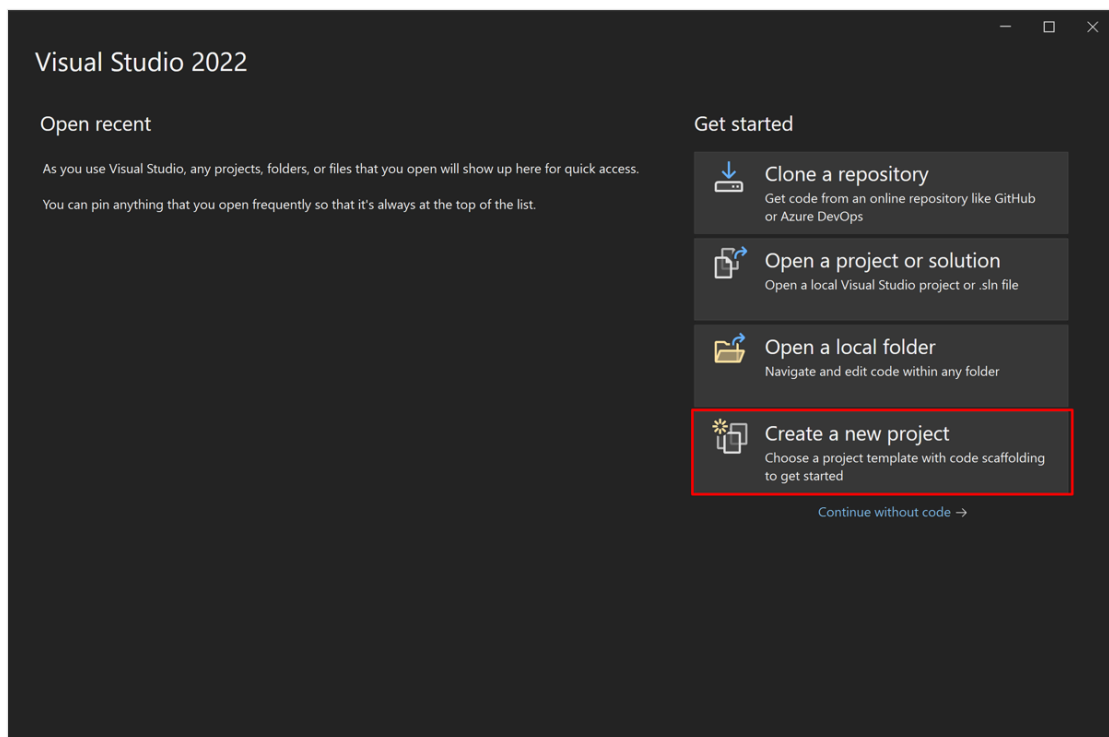
Prerequisites

- Visual Studio 2022 17.12 or greater, with the .NET Multi-platform App UI workload installed. For more information, see [Installation](#).

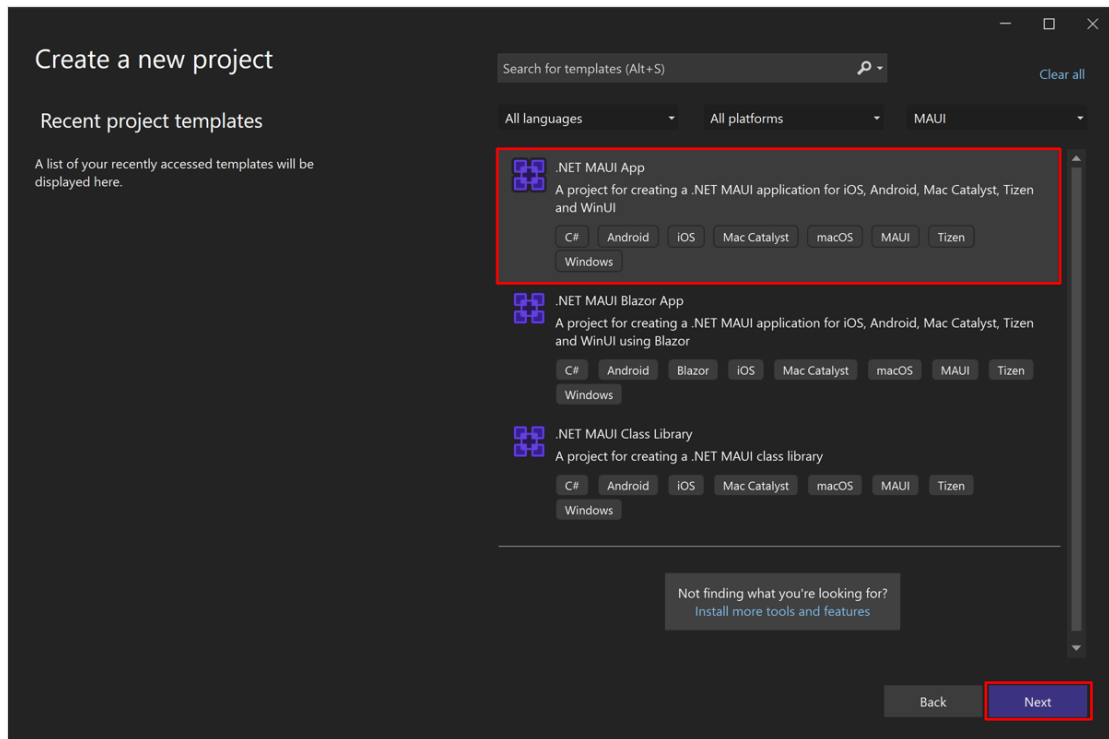
Create an app

In this tutorial, you'll create your first .NET MAUI app in Visual Studio 2022 and run it on Windows:

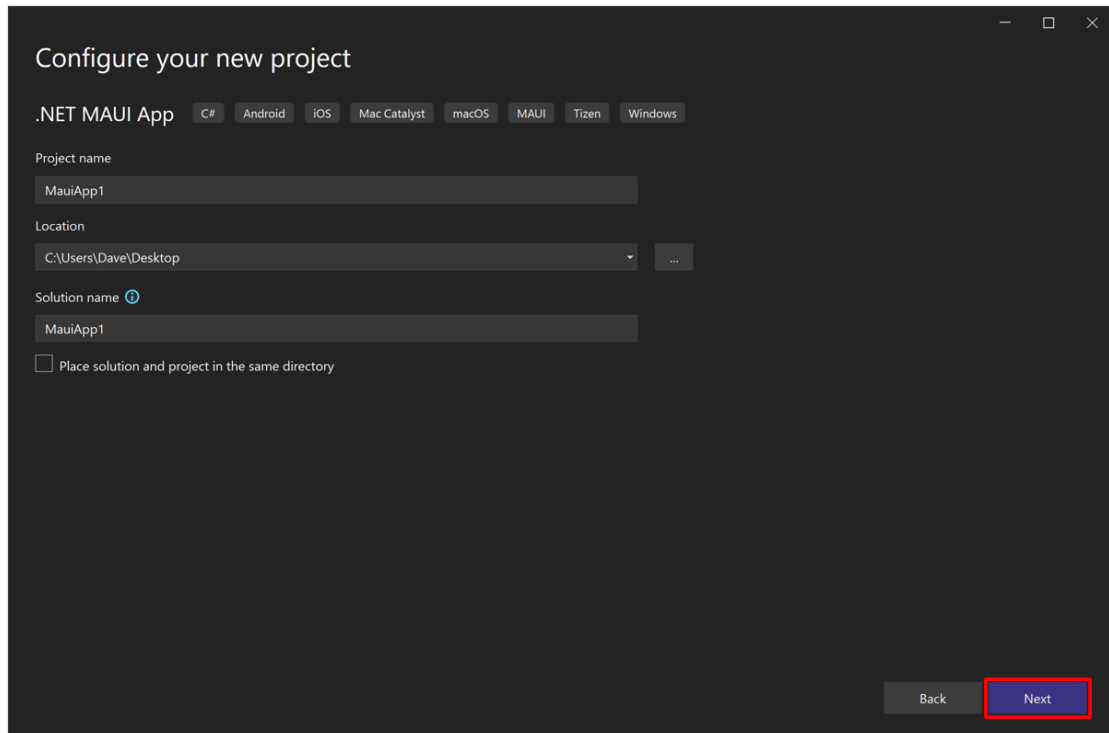
1. Launch Visual Studio 2022. In the start window, click **Create a new project** to create a new project:



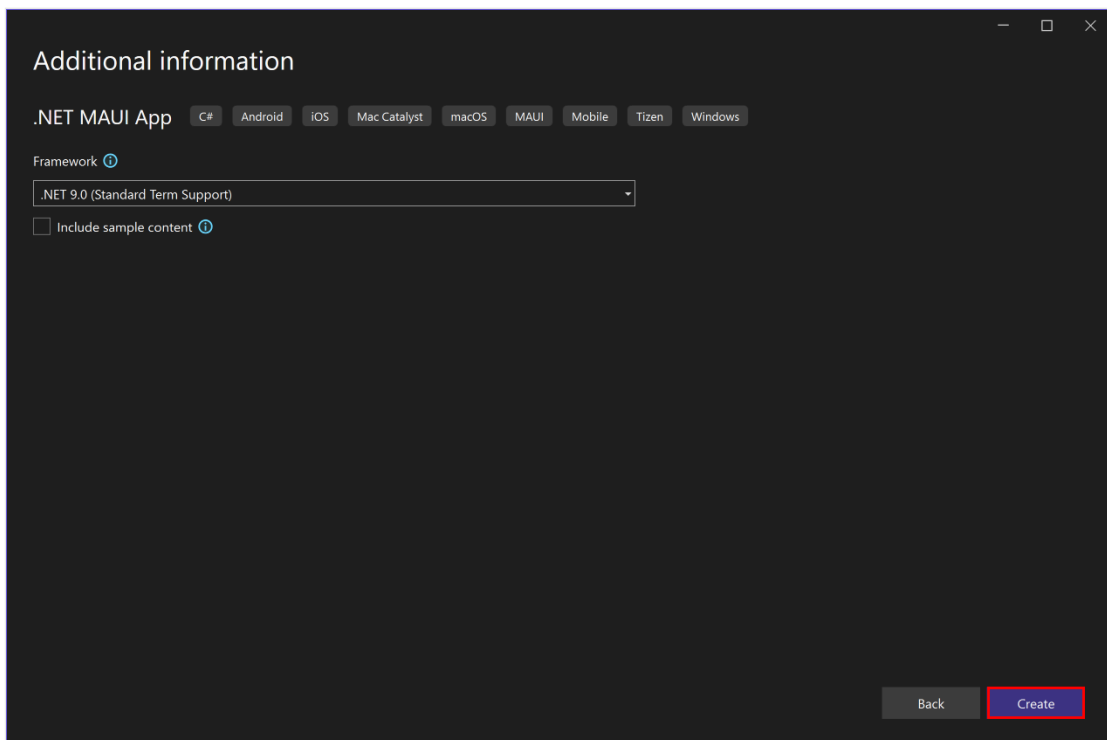
2. In the **Create a new project** window, select **MAUI** in the **All project types** drop-down, select the **.NET MAUI App** template, and click the **Next** button:



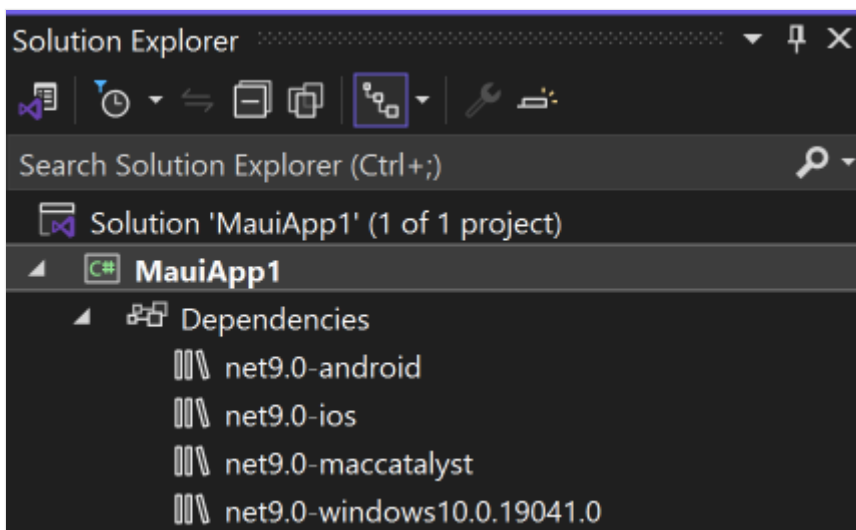
3. In the **Configure your new project** window, name your project, choose a suitable location for it, and click the **Next** button:



4. In the **Additional information** window, choose the version of .NET that you'd like to target and click the **Create** button:



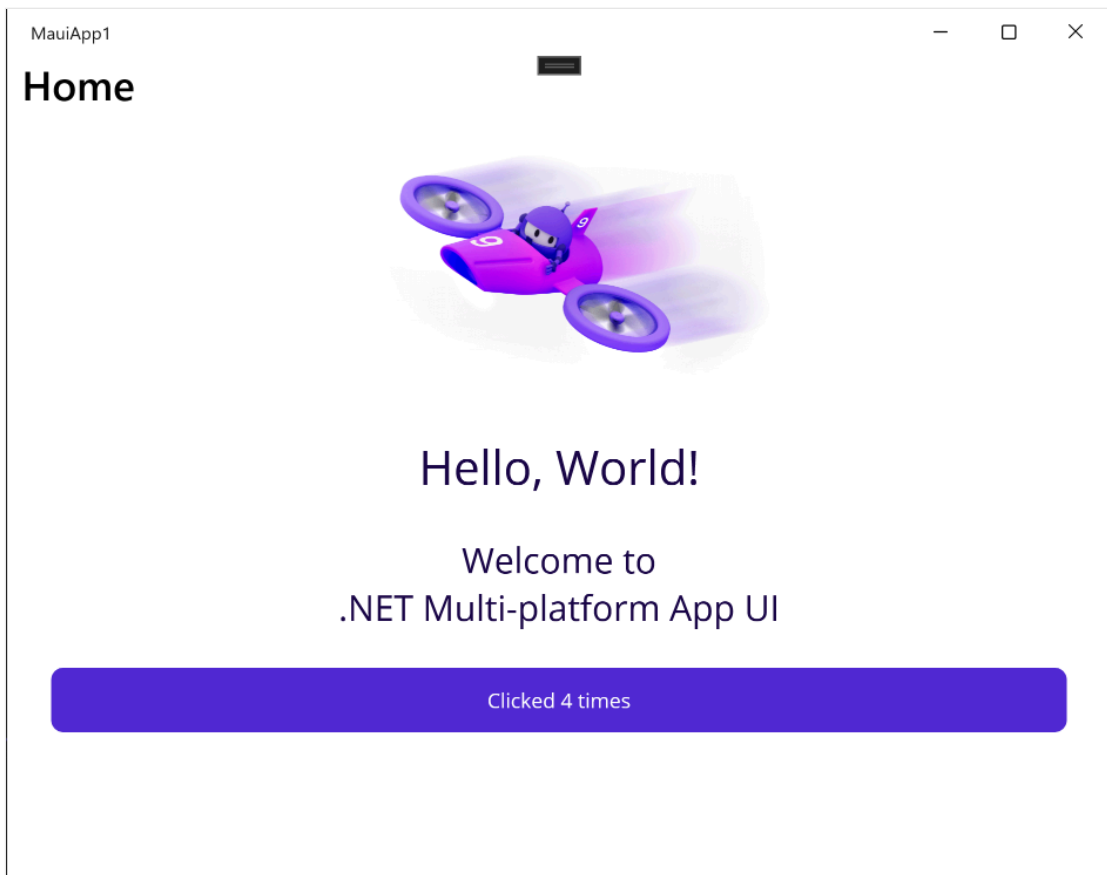
5. Wait for the project to be created and its dependencies to be restored:



6. In the Visual Studio toolbar, press the **Windows Machine** button to build and run the app:



7. In the running app, press the **Click me** button several times and observe that the count of the number of button clicks is incremented:



Troubleshooting

If your app fails to compile, review [Troubleshooting known issues](#), which may have a solution to your problem.

Next steps

In this tutorial, you've learned how to create and run your first .NET Multi-platform App UI (.NET MAUI) app.

To learn the fundamentals of building an app with .NET MAUI, see [Create a .NET MAUI app](#). Alternatively, for a full .NET MAUI training course, see [Build mobile and desktop apps with .NET MAUI](#).

Resources for learning .NET MAUI

Article • 09/21/2022

There are many different resources available for you to use to learn .NET Multi-platform App UI (.NET MAUI). There are Microsoft Learn modules, workshops, videos, and podcasts. Each varies in its depth and the topics it covers.

- [Build mobile and desktop apps with .NET MAUI](#)

Learn how to use .NET MAUI to build apps that run on mobile devices and the desktop using C# and Visual Studio. You'll learn the fundamentals of building an app with .NET MAUI and more advanced topics such as local data storage and invoking REST-based web services.

- [.NET MAUI for beginners](#) ↗

Follow a short video series that teaches you how to get started with .NET MAUI and Visual Studio, to build your very first cross-platform desktop and mobile app.

- [.NET MAUI workshop](#) ↗

Learn how to build a .NET MAUI app that displays a list of monkeys from around the world. You'll start by building the business logic backend that retrieves JSON-encoded data from a REST-based endpoint. You then use .NET MAUI to find the closest monkey to you, and show the monkey on a map. You'll also examine how to display data using different approaches, and then finally fully theme the app.

- [Enterprise application patterns using .NET MAUI](#)

This book provides real world solutions for addressing challenges faced when building an enterprise app using .NET MAUI. The book covers topics such as:

- Model-View-ViewModel (MVVM) pattern
- Dependency injection
- Navigation
- Configuration
- Loose-coupling of components
- Additional enterprise concerns

The content of this book is helpful for anyone looking to build a new app or looking to solve the problems of apps that evolve over time.

- [.NET MAUI samples](#)

Download and explore the code of different example .NET MAUI apps.

- [.NET MAUI podcast](#) 

Keep up with the latest news in the world of mobile and desktop development with the official .NET MAUI podcast.

Android emulator

Article • 01/25/2024

The Android emulator, which is produced by Google, simulates Android devices on your computer so that you can test your apps on a variety of devices and Android API levels, without needing to have each physical device. When you deploy and test your app on the emulator you select a pre-configured or custom virtual device, known as an Android Virtual Device (AVD) that simulates a physical Android device such as a Pixel phone. Alternatively, you can deploy your app to a physical device. For more information about deploying your app to physical device, see [Set up Android device for debugging](#).

The Android emulator is installed by Visual Studio, and an AVD created, when you build your first .NET Multi-platform App UI (.NET MAUI) app for Android. For more information, see [Build your first app](#).

If hardware acceleration isn't available or enabled, the emulator will run very slowly. You can significantly improve the performance of the emulator by enabling hardware acceleration and by using virtual device images that are appropriate for your processor architecture. For more information, see [How to enable hardware acceleration with Android emulators](#).

Physical Android devices are emulated by AVDs, which specify the Android version and hardware characteristics of the simulated device. To test an app, create AVDs that model each device your app is designed to run on. Each AVD runs as an independent device with its own storage for user data, and SD card data. The emulator stores the user data, and SD card data in a folder specific to the AVD. When you launch the emulator, it loads the user data and SD card data from the AVD folder. For information about managing virtual devices, see [Managing virtual devices with the Android Device Manager](#). For information about editing the configuration of a virtual device, see [Editing Android virtual device properties](#).

After you've created an AVD you can launch the emulator and debug an app in it. For more information, see [Debug on the Android emulator](#). Once an app is installed on an AVD, you can run it from the device as you would run any app on a device.

In addition, the emulator provides versatile networking capabilities that can be used for different purposes, including connecting to an emulator running on a Mac from inside a Windows virtual machine (VM). For more information, see [Connect to an Android emulator on a Mac from inside a Windows virtual machine](#).

For information about diagnosing emulator issues, see [Android emulator troubleshooting](#).

Use the emulator

When the Android emulator is running, you use your computer mouse pointer to mimic your finger on the touch screen, select menu items and input fields, and click buttons and controls. You can type in the emulator by using your computer keyboard or by using the onscreen keyboard that appears in the emulator.

Common actions can be performed in the emulator via its panel on the right hand side:

 Expand table

Action	Icon	Description
Close		Close the emulator.
Minimize		Minimize the emulator window.
Power		Click to turn the screen on/off. Click and hold to turn the device on/off.
Volume up		Increase the volume.
Volume down		Decrease the volume.
Rotate left		Rotate the screen 90 degrees left.
Rotate right		Rotate the screen 90 degrees right.
Take screenshot		Click to take a screenshot of the device.
Enter zoom mode		Click to change the cursor to the zoom icon. Click again to exit zoom mode.
Back		Return to the previous screen, or close an options menu, dialog, onscreen keyboard, or Notifications panel.
Home		Return to the Home screen.
Overview (recent apps)		Click to open a list of apps you've worked with recently. To open an app, tap its thumbnail.
More		Click for more features and settings.

The following gestures can be simulated in the emulator:

 Expand table

Gesture	Description
Tap	Press and release the primary mouse button.
Double tap	Double-click and then release the primary mouse button.
Drag	Press and hold the primary mouse button, move the item, and then release.
Touch and hold	Press the primary mouse button, hold, and release.
Pinch	Press the CTRL key on Windows, or the Command key on macOS, to display a pinch gesture multi-touch interface. The mouse acts as the first finger, which can be moved by dragging the cursor. The second finger is across from the anchor point. Pressing the left mouse button simulates touching both points, and releasing simulates removing both points.
Swipe	Press and hold the primary mouse button, swipe, and then release.
Vertical swipe	Open a vertical menu on the screen and use the mouse scroll wheel to scroll through the menu items.

Advanced emulator scenarios

You might want to test your app on a virtual device using more than just basic touch gestures. For example, you might want to simulate different network conditions. The following table lists where to find more information about advanced Android emulator scenarios:

 Expand table

Scenarios	More information
Use the camera	Camera support  on developer.android.com
Install and add files	Install and add files  on developer.android.com
Use Wi-Fi	Wi-Fi  on developer.android.com
Use extended controls	Extended controls, settings, and help  on developer.android.com
Use snapshots	Snapshots  on developer.android.com

Scenarios	More information
Use the emulator from the command line	Start the emulator from the command line ↗ on developer.android.com
Send console commands	Send emulator console commands ↗ on developer.android.com
Set up networking	Set up Android Emulator networking ↗ on developer.android.com
Configure hardware acceleration	Configure hardware acceleration for the Android Emulator ↗ on developer.android.com
Android emulator tools	Comparison of Android Emulator tools ↗ on developer.android.com

Limitations

The Android Emulator doesn't include virtual hardware for the following:

- Bluetooth
- NFC
- SD card insert/eject
- Device-attached headphones
- USB

How to enable hardware acceleration with Android emulators (Hyper-V & AEHD)

Article • 08/27/2024

With Visual Studio, you can easily test and debug your .NET MAUI app for Android in emulators for situations where an Android device isn't available. However, if hardware acceleration isn't available or enabled, the emulator will run very slowly. You can significantly improve the performance of the emulator by enabling hardware acceleration and by using virtual device images that are appropriate for your processor architecture. For more information, see [Configure hardware acceleration for the Android Emulator](#) on developer.android.com.

The emulator provides versatile networking capabilities that can be used for different purposes, including connecting to an emulator running on a Mac from inside a Windows virtual machine (VM). For more information, see [Connect to an Android emulator on a Mac from inside a Windows virtual machine](#).

On macOS, the Android emulator uses the built-in Hypervisor.Framework. For more information, see [Hypervisor](#) on developer.apple.com.

Accelerate Android emulators on Windows

The following virtualization technologies are available for accelerating the Android emulator on Windows:

- The Windows Hypervisor Platform (WHPX). [Hyper-V](#) is a virtualization feature of Windows that makes it possible to run virtualized computer systems on a physical host computer.
- The Android Emulator hypervisor driver (AEHD).

ⓘ Note

The Intel Hardware Accelerated Execution Manager (HAXM) is deprecated from emulator 33.x.x.x, and has been replaced by AEHD on Intel processors. For information about using HAXM on emulator 32.x.x.x and lower, see [Configure VM acceleration using Intel HAXM on Windows](#) on developer.android.com.

For the best experience on Windows, it's recommended you use WHPX to accelerate the Android emulator. If WHPX isn't available on your computer, then AEHD can be used. The Android emulator automatically uses hardware acceleration if the following criteria are met:

- Hardware acceleration is available and enabled on your development computer.
- The emulator is running a system image created for an **x86-64** or **x86**-based virtual device.

Important

A Virtual Machine (VM) accelerated emulator can run inside another VM, including Microsoft Dev Box, provided that nested virtualization is enabled in the VM.

For information about launching and debugging with the Android emulator, see [Debugging on the Android Emulator](#).

Accelerate with Hyper-V

Before enabling Hyper-V, read the following section to verify that your computer supports Hyper-V.

Verify support for Hyper-V

Hyper-V runs on the Windows Hypervisor Platform. To use the Android emulator with Hyper-V, your computer must meet the following criteria to support the Windows Hypervisor Platform:

- Your computer hardware must meet the following requirements:
 - Intel processors with support for Virtualization Technology (VT-x), Extended Page Tables (EPT), and Unrestricted Guest (UG) features. VT-x must be enabled in your computer's BIOS.
 - AMD processors: AMD Ryzen processor recommended. Virtualization or SVM must be enabled in your computer's BIOS.
- Your computer must be running the Enterprise, Pro, or Education versions of Windows 11 or Windows 10 Version 1909 or later.

To verify that your computer hardware and software is compatible with Hyper-V, open a command prompt and type the following command:

```
Windows Command Prompt
```

systeminfo

If all listed Hyper-V requirements have a value of **Yes**, then your computer can support Hyper-V. For example:

```
Hyper-V Requirements:      VM Monitor Mode Extensions: Yes
                           Virtualization Enabled In Firmware: Yes
                           Second Level Address Translation: Yes
                           Data Execution Prevention Available: Yes
```

If the Hyper-V result indicates that a hypervisor is currently running, Hyper-V is already enabled.

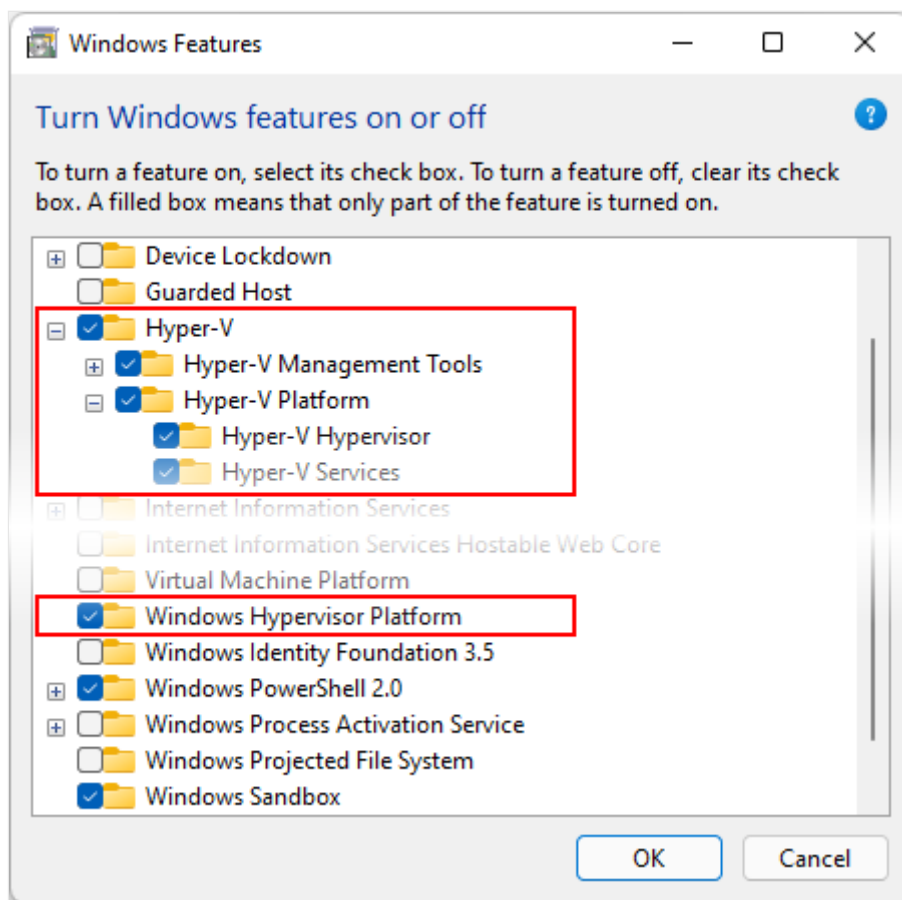
Important

If Windows is running inside a virtual machine, nested virtualization must be enabled in the host hypervisor.

Enable Hyper-V acceleration in Windows

If your computer meets the above criteria, use the following steps to accelerate the Android emulator with Hyper-V:

1. Enter **windows features** in the Windows search box and select **Turn Windows features on or off** in the search results. In the **Windows Features** dialog, enable both **Hyper-V** and **Windows Hypervisor Platform**:



After making these changes, reboot your computer.

Important

On Windows 10 October 2018 Update (RS5) and higher, you only need to enable Hyper-V, as it will use Windows Hypervisor Platform (WHPX) automatically.

2. Make sure that the virtual device you [created in the Android Device Manager](#) is an **x86-64** or **x86**-based system image. If you use an Arm-based system image, the virtual device won't be accelerated and will run slowly.

After Hyper-V is enabled, you'll be able to run your accelerated Android emulator.

Accelerate with AEHD

If your computer doesn't support Hyper-V, you should use AEHD to accelerate the Android emulator. Before you can install and use AEHD, read the following section to verify that your computer supports AEHD.

Verify support for AEHD

Your computer must meet the following criteria to support AEHD:

- An Intel or AMD processor with virtualization extension, which must be enabled in your BIOS.
- 64-bit Windows 11 or Windows 10.
- Hyper-V must be turned off.

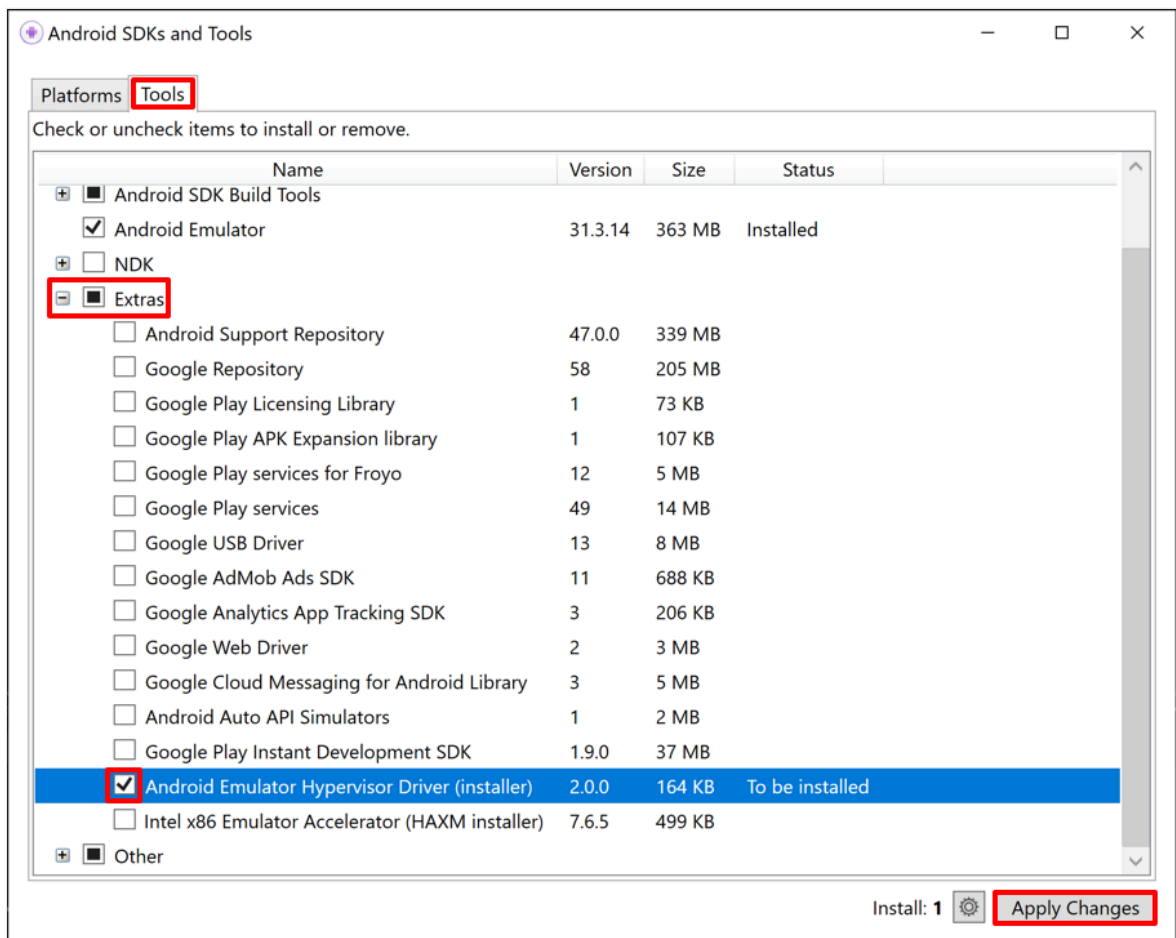
ⓘ Note

Several features in Windows enable Hyper-V implicitly. For more information, see [Double-check when disabling Hyper-V](#) [↗] on developer.android.com.

Enable AEHD acceleration in Windows

If your computer meets the above criteria, use the following steps to accelerate the Android emulator with AEHD:

1. In Visual Studio, select the **Tools > Android > Android SDK Manager...** menu item.
2. In the **Android SDKs and Tools** window, select the **Tools** tab.
3. In the **Tools** tab, expand **Extras**, tick the checkbox for the **Android Emulator Hypervisor Driver (installer)** item, and then select the **Apply Changes** button:



ⓘ Note

Alternatively, AEHD can be downloaded and installed from [GitHub](#). After unpacking the driver package, run `silent_install.bat` at a command line with administrator privileges.

4. Make sure that the virtual device you [created in the Android Device Manager](#) is an **x86-64** or **x86**-based system image. If you use an Arm-based system image, the virtual device won't be accelerated and will run slowly.

AEHD 2.1 and higher

After installation, confirm that the driver is operating correctly using the following command:

```
Windows Command Prompt
```

```
sc query aehd
```

If the driver is operating correctly, the status message will include the following information:


```
SERVICE_NAME: aehd
...
STATE           : 4  RUNNING
...
```

The following error message means that the virtualization extension isn't enabled in your BIOS or that Hyper-V isn't disabled:

```
SERVICE_NAME: aehd
...
STATE           : 1  STOPPED
WIN32_EXIT_CODE  : 4294967201 (0xffffffffa1)
...
```

After AEHD is installed and running, you'll be able to run your accelerated Android emulator.

AEHD 2.0 and lower

After installation, confirm that the driver is operating correctly using the following command:

Windows Command Prompt

```
sc query gvm
```

If the driver is operating correctly, the status message will include the following information:

```
SERVICE_NAME: gvm
...
STATE           : 4  RUNNING
...
```

The following error message means that the virtualization extension isn't enabled in your BIOS or that Hyper-V isn't disabled:

```
SERVICE_NAME: gvm
...
STATE          : 1  STOPPED
WIN32_EXIT_CODE : 4294967201 (0xffffffffa1)
...
```

After AEHD is installed and running, you'll be able to run your accelerated Android emulator.

Uninstall AEHD

To uninstall AEHD, use the following commands at a command line with administrator privileges:

- AEHD 2.1 and higher

Windows Command Prompt

```
sc stop aehd
sc delete aehd
```

- AEHD 2.0 and lower

Windows Command Prompt

```
sc stop gvm
sc delete gvm
```

Important

Shut down any x86 emulators before uninstalling AEHD for AMD.

Troubleshoot

For help with troubleshooting hardware acceleration issues, see the Android emulator [Troubleshooting](#) guide.

Managing virtual devices with the Android Device Manager

Article • 08/27/2024

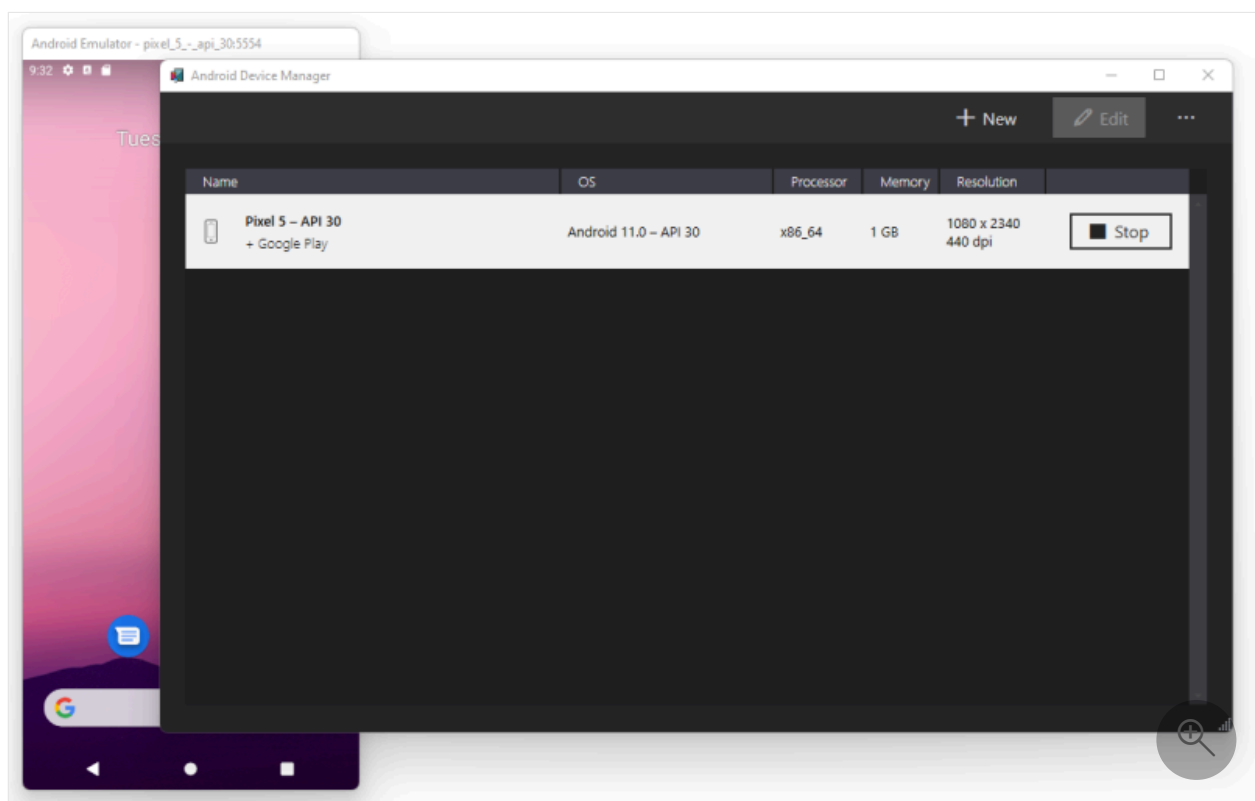
This article explains how to use the Android Device Manager to create and configure Android Virtual Devices (AVDs) that emulate physical Android devices. You can use these virtual devices to run and test your app without having to rely on a physical device. For more information about Android virtual devices, see [Create and manage virtual devices](#) on developer.android.com.

Important

Enable hardware acceleration for the Android devices. For more information, see [Hardware Acceleration for Emulator Performance](#).

Android Device Manager on Windows

You use the Android Device Manager to create and configure an Android Virtual Devices (AVD) that run in the [Android Emulator](#). Each AVD is an emulator configuration that simulates a physical Android device. This makes it possible to run and test your app in a variety of configurations that simulate different physical Android devices.

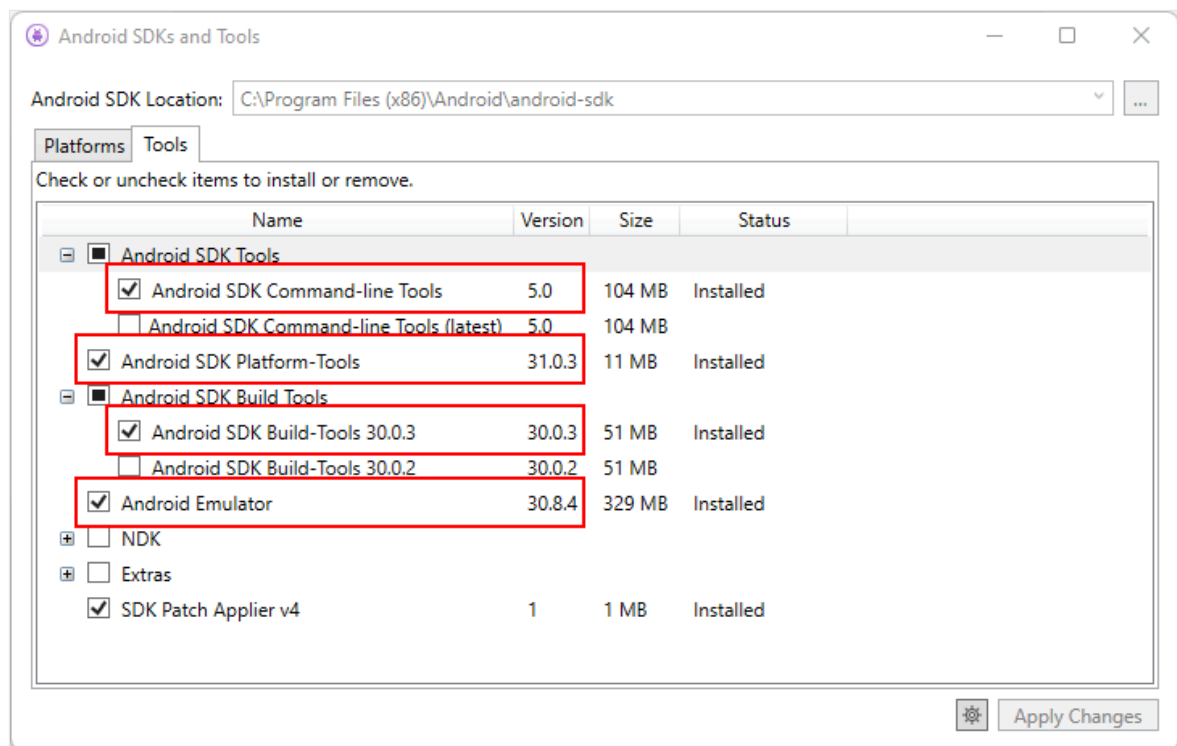


Requirements

To use the Android Device Manager, you'll need the following items:

- Visual Studio 2022: Community, Professional, and Enterprise editions are supported.
- The **Android SDK API Level 30** or later. Be sure to install the Android SDK at its default location if it isn't already installed: *C:\Program Files (x86)\Android\android-sdk*.
- The following packages must be installed:
 - **Android SDK Tools 5.0** or later
 - **Android SDK Platform-Tools 31.0.3** or later
 - **Android SDK Build-Tools 30.0.2** or later
 - **Android Emulator 30.8.4** or later

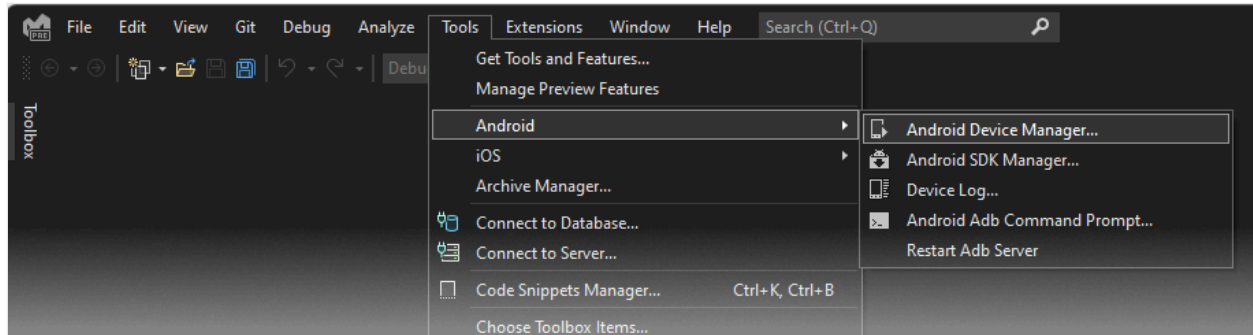
These packages should be displayed with **Installed** status as seen in the following screenshot:



When you install the **.NET Multi-Platform App UI development** workload in Visual Studio, everything is installed for you. For more information on setting up .NET MAUI with Visual Studio, see [Build your first app](#).

Open the device manager

Open the Android Device Manager in Visual Studio from the **Tools** menu by pressing **Tools > Android > Android Device Manager**:



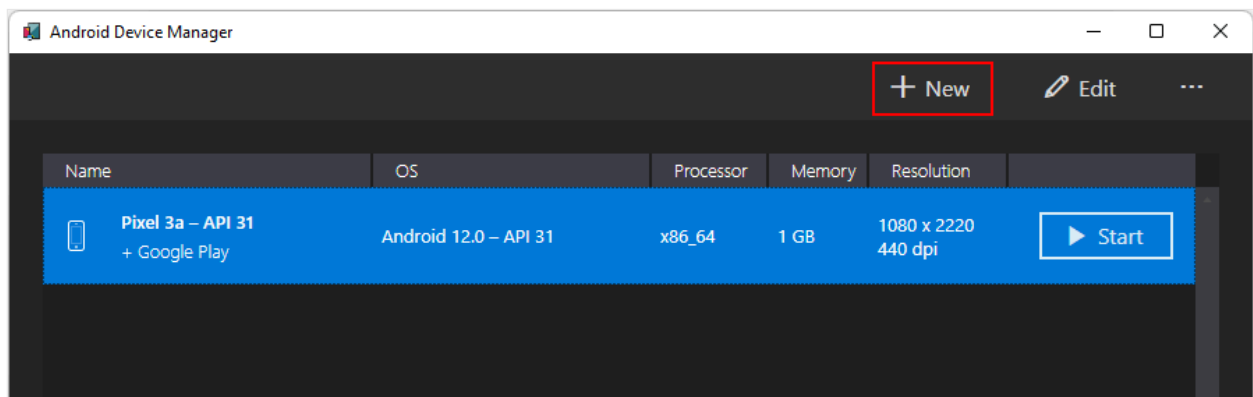
Main screen

When you run the Android Device Manager, it presents a screen that displays all currently configured virtual devices. For each virtual device, the **Name**, **OS** (Android Version), **Processor**, **Memory** size, and screen **Resolution** are displayed:

When you select a device in the list, the **Start** button appears on the right. Press the **Start** button to launch the emulator with this virtual device. If the emulator is running with the selected virtual device, the **Start** button changes to a **Stop** button that you can use to halt the emulator.

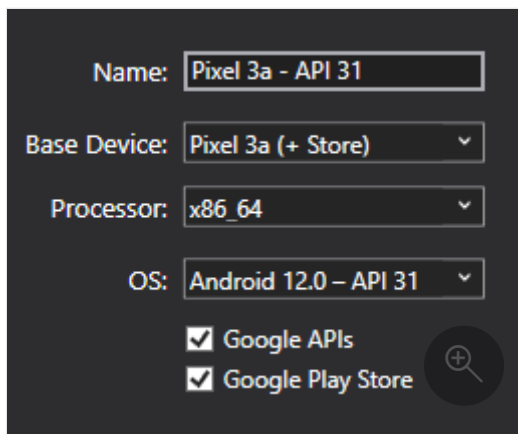
Create a new device

To create a new device, press the **New** button:



The **New Device** window is displayed. To configure the device, follow these steps:

1. Give the device a new name. In the following example, the new device is named *Pixel 3a - API 31*.



2. Select a physical device to emulate by selecting a device in the **Base Device** box.
3. Select a processor type for this virtual device with the **Processor** box.

It's recommended that you choose `x86_64` and enable [hardware acceleration](#).

4. Select the Android version (API level) with the **OS** box.

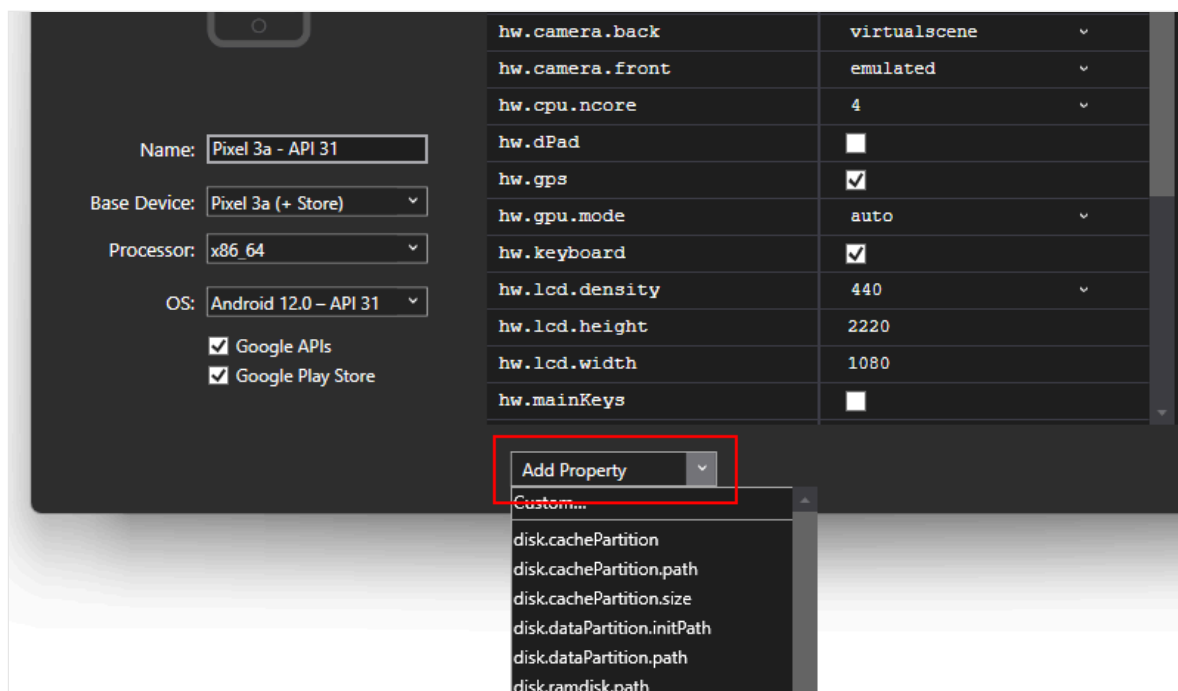
If you select an Android API level that has not yet been installed, the Device Manager will display **A new device will be downloaded** message at the bottom of the screen – it will download and install the necessary files as it creates the new virtual device.

5. If you want to include Google Play Services APIs in your virtual device, select the **Google APIs** option. To include the Google Play Store app on the virtual device, select the **Google Play Store** option

⚠ Note

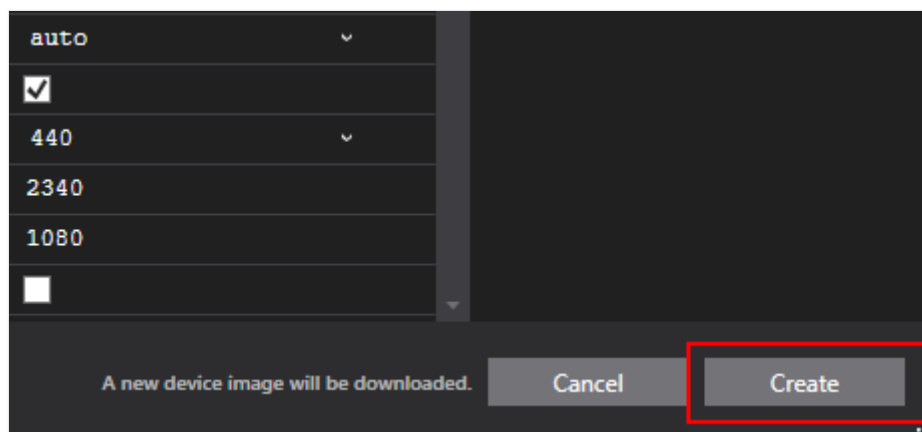
Google Play Store images are available only for some base device types such as Pixel, Pixel 2, Pixel 3, and Nexus 5. This is indicated by the text `(+ Store)` in the image name.

6. Use the property list to change some of the most commonly modified properties. To make changes to properties, see [Editing Android Virtual Device Properties](#).
7. Add any additional properties that you need to explicitly set with the **Add Property** box at the bottom of the window:



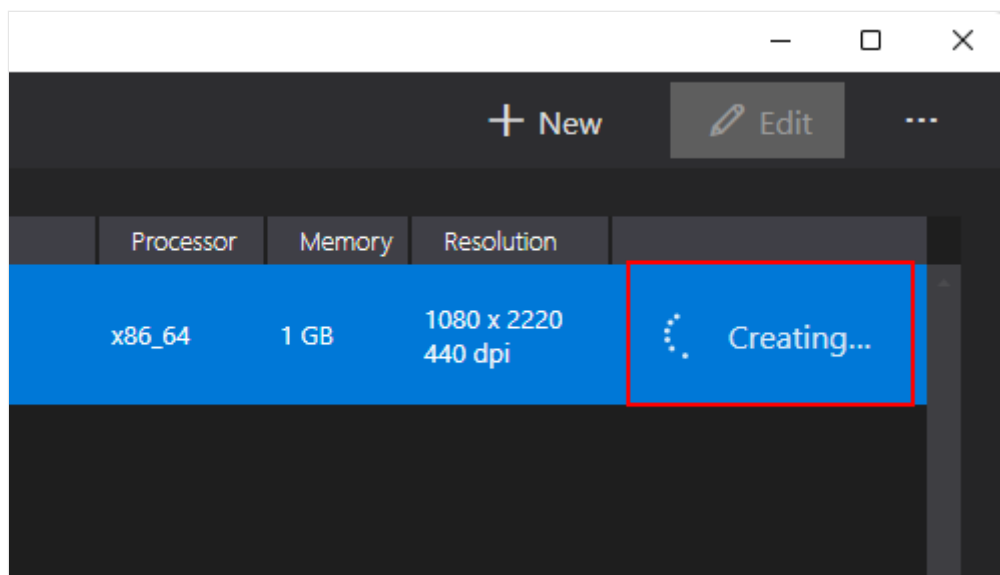
You can also define a custom property by selecting **Custom....**

- Press the **Create** button to create the new device:



You might get a **License Acceptance** screen when you create the device. Select **Accept** if you agree to the license terms.

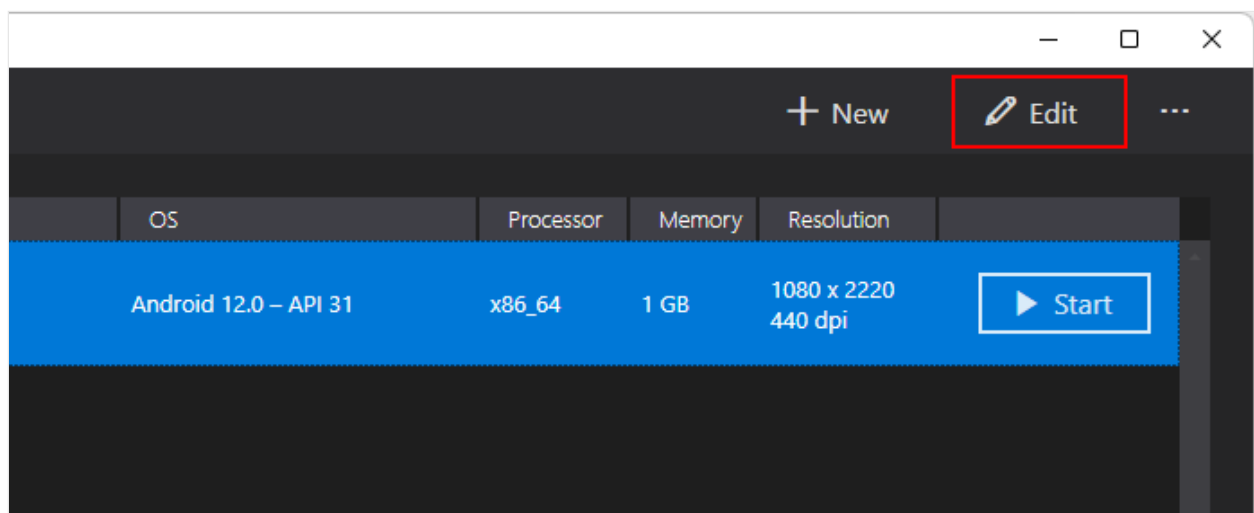
- The Android Device Manager adds the new device to the list of installed virtual devices while displaying a **Creating** progress indicator during device creation:



10. When the creation process is complete, the new device is shown in the list of installed virtual devices with a **Start** button, ready to launch

Edit device

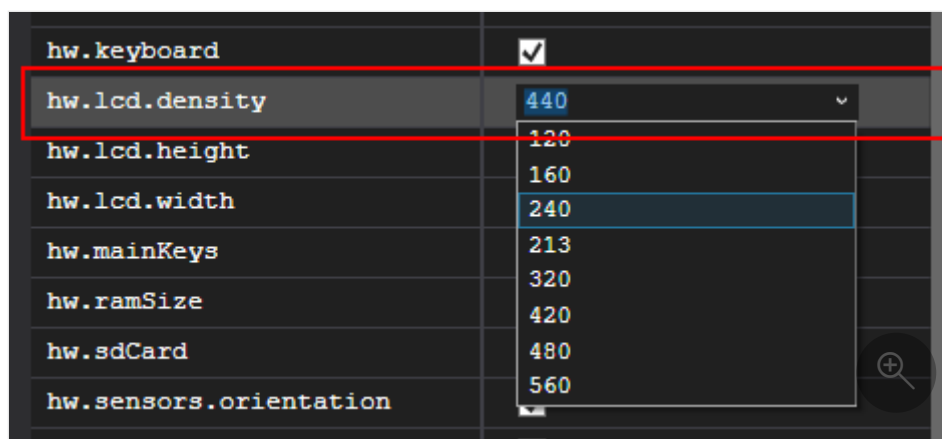
To edit an existing virtual device, select the device and then press the **Edit** button:



Pressing **Edit** displays the **Device Editor** window for the selected virtual device.

The **Device Editor** window lists the properties of the virtual device under the **Property** column, with the corresponding values of each property in the **Value** column. When you select a property, a detailed description of that property is displayed on the right.

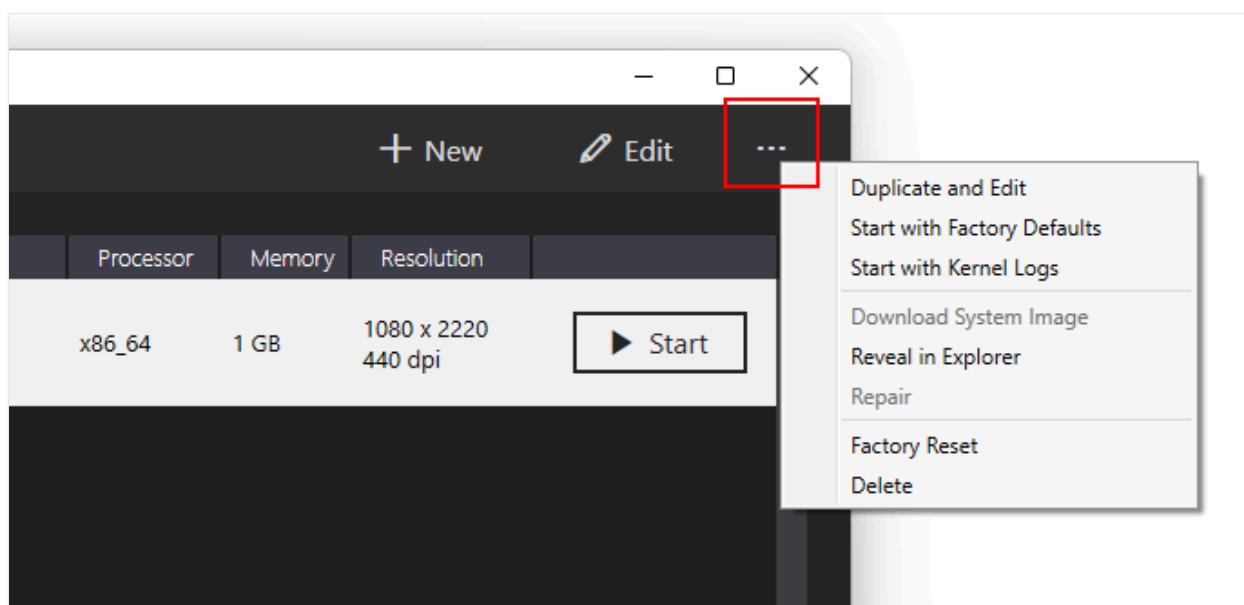
To change a property, edit its value in the **Value** column. For example, in the following screenshot the `hw.lcd.density` property is being changed to **240**:



After you've made the necessary configuration changes, press the **Save** button. For more information about changing virtual device properties, see [Editing Android Virtual Device Properties](#).

Additional options

Additional options for working with devices are available from the **Additional Options** (...) pull-down menu:



The additional options menu contains the following items:

- **Duplicate and Edit** – Duplicates the currently selected device and opens it in the **New Device** screen with a new name that's similar to the existing device. For example, selecting **Pixel 3a - API 31** and pressing **Duplicate and Edit** appends a counter to the name: **Pixel 3a - API 31 (1)**.
- **Start with Factory Defaults** – Starts the device with a cold boot.
- **Start with Kernel Logs** – Starts the emulator and opens up kernel logs directory.

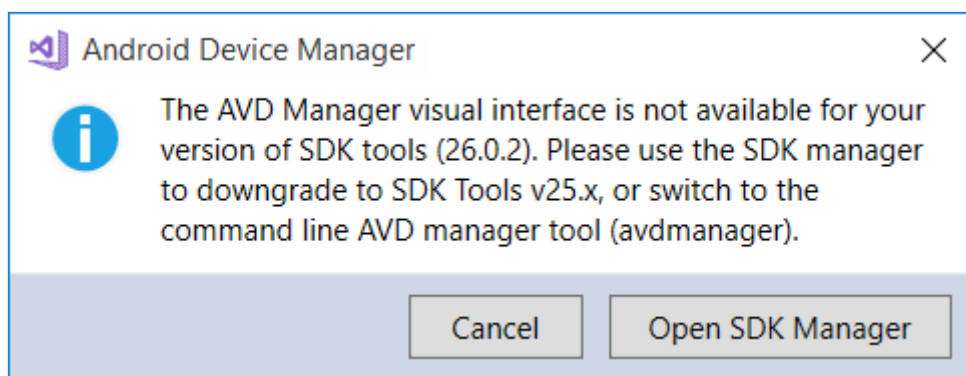
- **Download System Image** – Downloads the Android OS system image for the device, if it's not already downloaded.
- **Reveal in Explorer** – Opens Windows Explorer and navigates to the folder that holds the files for the virtual device.
- **Repair** – Initiates a repair on the device.
- **Factory Reset** – Resets the selected device to its default settings, erasing any user changes made to the internal state of the device while it was running. This action also erases the current **Fast Boot** snapshot if it exists. This change doesn't alter modifications that you make to the virtual device during creation and editing. A dialog box will appear with the reminder that this reset cannot be undone – press **Factory Reset** to confirm the reset.
- **Delete** – Permanently deletes the selected virtual device. A dialog box will appear with the reminder that deleting a device cannot be undone. Press **Delete** if you are certain that you want to delete the device.

Troubleshooting

The following sections explain how to diagnose and work around problems that may occur when using the Android Device Manager to configure virtual devices.

Wrong version of Android SDK Tools

If you have the wrong Android SDK tools installed, installed, you may see this error dialog on launch:



If you see that error dialog, press **Open SDK Manager** to open the Android SDK Manager. In the Android SDK Manager, go to the **Tools** tab and install the following packages:

- **Android SDK Command-line Tools 5.0** or later

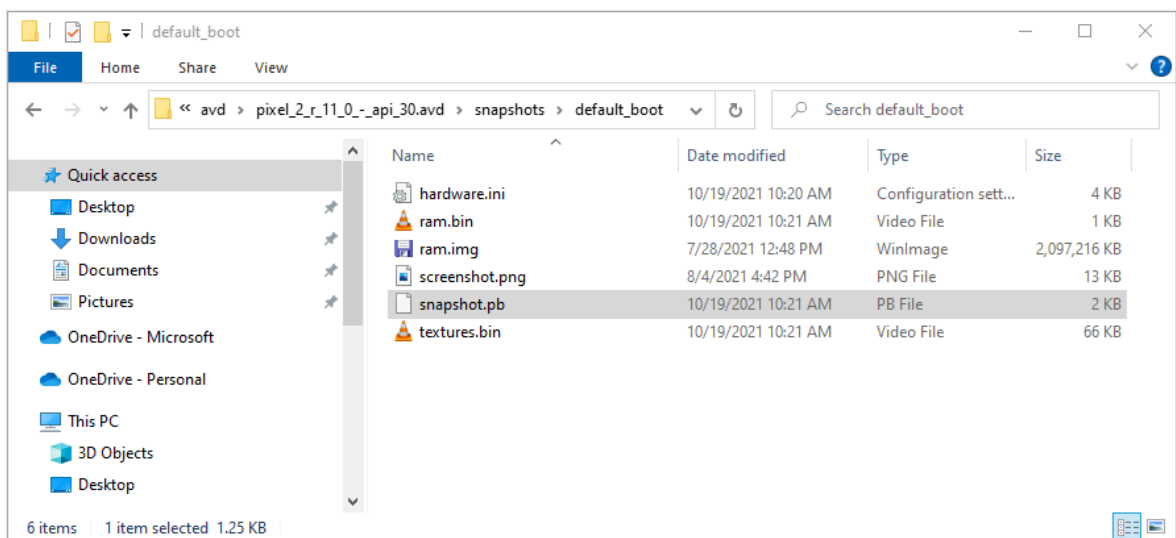
- Android SDK Platform-Tools 31.0.3 or later
- Android SDK Build-Tools 30.0.3 or later

Snapshot disables Wi-Fi on Android Oreo

If you've an AVD configured for Android Oreo with simulated Wi-Fi access, restarting the AVD after a snapshot may cause Wi-Fi access to become disabled.

To work around this problem,

1. Open the **Android Device Manager**.
2. Select the AVD in the Android Device Manager.
3. From the **Additional Options (...)** menu, select **Reveal in Explorer**.
4. Navigate to the **snapshots > default_boot** folder.
5. Delete the *snapshot.pb* file:



6. Restart the AVD.

After these changes are made, the AVD will restart in a state that allows Wi-Fi to work again.

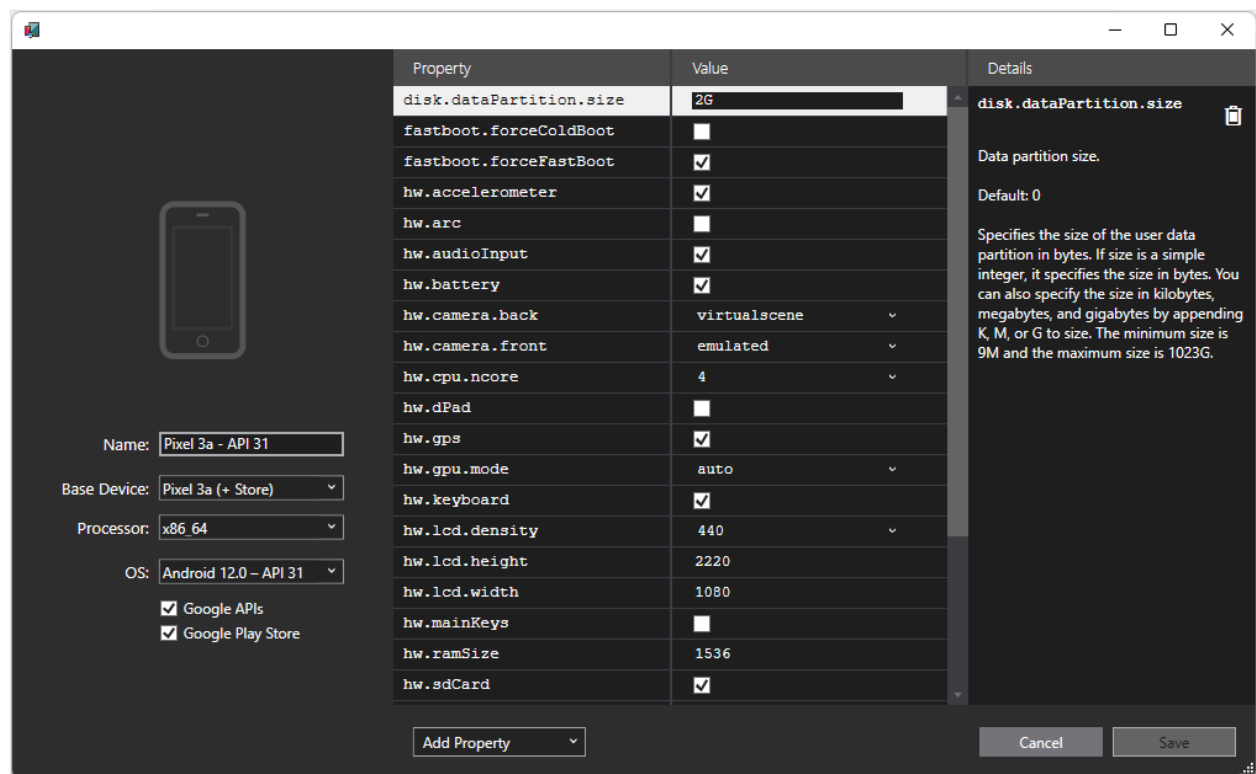
Editing Android virtual device properties

Article • 12/20/2023

This article explains how to use the Android Device Manager (AVD) to edit the profile properties of an Android virtual device. For more information about Android virtual devices, see [Create and manage virtual devices](https://developer.android.com/studio/run/create-virtual-device) on developer.android.com.

Android Device Manager on Windows

The **Android Device Manager** supports the editing of individual Android virtual device profile properties. The **New Device** and **Device Edit** screens list the properties of the virtual device in the first column, with the corresponding values of each property in the second column (as seen in this example):



When you select a property, a detailed description of that property is displayed on the right. You can modify hardware profile properties and AVD properties. Hardware profile properties (such as `hw.ramSize` and `hw.accelerometer`) describe the physical characteristics of the emulated device. These characteristics include screen size, the amount of available RAM, whether or not an accelerometer is present. AVD properties specify the operation of the AVD when it runs. For example, AVD properties can be configured to specify how the AVD uses your development computer's graphics card for rendering.

You can change properties by using the following guidelines:

- To change a boolean property, click the check mark to the right of the boolean property:

<code>fastboot.forceFastBoot</code>	☒
<code>hw.accelerometer</code>	☒
<code>hw.arc</code>	☐

- To change an *enum* (enumerated) property, click the down-arrow to the right of the property and choose a new value.

<code>hw.battery</code>	☒
<code>hw.camera.back</code>	virtualscene
<code>hw.camera.front</code>	emulated
<code>hw.cpu.ncore</code>	none
<code>hw.dPad</code>	webcam0
	virtualscene

- To change a string or integer property, double-click the current string or integer setting in the value column and enter a new value.

<code>hw.battery</code>	☒
<code>hw.camera.back</code>	virtualscene
<code>hw.camera.front</code>	emulated
<code>hw.cpu.ncore</code>	none
<code>hw.dPad</code>	webcam0
	virtualscene

The following table provides a detailed explanation of the properties listed in the **New Device** and **Device Editor** screens:

[Expand table](#)

Property	Description	Options
<code>abi.type</code>	ABI type – Specifies the ABI (application binary interface) type of the emulated device. The x86 option is for the instruction set commonly referred to as "x86" or "IA-32." The x86_64 option is for the 64-bit x86 instruction set. The armeabi-v7a option is for the ARM instruction set with v7-a ARM extensions. The arm64-v8a option is for the ARM instruction set that supports AArch64.	x86, x86_64, armeabi-v7a, arm64-v8a

Property	Description	Options
<code>disk.cachePartition</code>	Cache partition – Determines whether the emulated device uses a /cache partition on the device. The /cache partition (which is initially empty) is the location where Android stores frequently accessed data and app components. If set to no , the emulator doesn't use a /cache partition and the other <code>disk.cache</code> settings are ignored.	yes, no
<code>disk.cachePartition.path</code>	Cache partition path – Specifies a cache partition image file on your development computer. The emulator uses this file for the /cache partition. Enter an absolute path or a path relative to the emulator's data directory. If not set, the emulator creates an empty temporary file called cache.img on your development computer. If the file doesn't exist, it's created as an empty file. This option is ignored if <code>disk.cachePartition</code> is set to no .	
<code>disk.cachePartition.size</code>	Cache partition size – The size of the cache partition file (in bytes). Normally you don't need to set this option unless the app will download very large files that are larger than the default cache size of 66 megabytes. This option is ignored if <code>disk.cachePartition</code> is set to no . If this value is an integer, it specifies the size in bytes. You can also specify the size in kilobytes, megabytes, and gigabytes by appending K , M , or G to the value. The minimum size is 9M and the maximum size is 1023G .	
<code>disk.dataPartition.initPath</code>	Initial path to the data partition – Specifies the initial contents of the data partition. After wiping user data, the emulator copies the contents of the specified file to user data (by default, userdata-qemu.img) instead of using userdata.img as the initial version.	
<code>disk.dataPartition.path</code>	Path to the data partition – Specifies the user data partition file. To configure a persistent user data file, enter a filename and a path on your development computer. If the file doesn't exist, the emulator creates an image from the default file userdata.img , stores it in the filename specified by <code>disk.dataPartition.path</code> , and persists user data to it when the emulator shuts down. If you don't specify a path, the default file	

Property	Description	Options
	is named userdata-qemu.img . The special value <temp> causes the emulator to create and use a temporary file. If <code>disk.dataPartition.initPath</code> is set, its content is copied to the <code>disk.dataPartition.path</code> file at boot time. This option can't be left blank.	
<code>disk.dataPartition.size</code>	Data partition size – Specifies the size of the user data partition in bytes. If this value is an integer, it specifies the size in bytes. You can also specify the size in kilobytes, megabytes, and gigabytes by appending K , M , or G to the value. The minimum size is 9M and the maximum size is 1023G .	
<code>disk.ramdisk.path</code>	Ramdisk path – Path to the boot partition (ramdisk) image. The ramdisk image is a subset of the system image that's loaded by the kernel before the system image is mounted. The ramdisk image typically contains boot-time binaries and initialization scripts. If this option isn't specified, the default is ramdisk.img in the emulator system directory.	
<code>disk.snapStorage.path</code>	Snapshot storage path – Path to the snapshot storage file where all snapshots are stored. All snapshots made during execution are saved to this file. Only snapshots that are saved to this file can be restored during the emulator run. If this option isn't specified, the default is snapshots.img in the emulator data directory.	
<code>disk.systemPartition.initPath</code>	System partition init path – Path to the read-only copy of the system image file; specifically, the partition containing the system libraries and data corresponding to the API level and any variant. If this path isn't specified, the default is system.img in the emulator system directory.	
<code>disk.systemPartition.path</code>	System partition path – Path to the read/write system partition image. If this path isn't set, a temporary file is created and initialized from the contents of the file specified by <code>disk.systemPartition.initPath</code> .	
<code>disk.systemPartition.size</code>	System partition size – The ideal size of the system partition (in bytes). The size is ignored if the actual system partition image is larger than this setting; otherwise, it specifies the maximum	

Property	Description	Options
	size that the system partition file can grow to. If this value is an integer, it specifies the size in bytes. You can also specify the size in kilobytes, megabytes, and gigabytes by appending K , M , or G to the value. The minimum size is 9M and the maximum size is 1023G .	
<code>hw.accelerometer</code>	Accelerometer – Determines whether the emulated device contains an accelerometer sensor. The accelerometer helps the device determine orientation (used for autorotation). The accelerometer reports the acceleration of the device along three sensor axes.	yes, no
<code>hw.audioInput</code>	Audio recording support – Determines whether the emulated device can record audio.	yes, no
<code>hw.audioOutput</code>	Audio playback support – Determines whether the emulated device can play audio.	yes, no
<code>hw.battery</code>	Battery support – Determines whether the emulated device can run on a battery.	yes, no
<code>hw.camera</code>	Camera support – Determines whether the emulated device has a camera.	yes, no
<code>hw.camera.back</code>	Back-facing camera – Configures the back-facing camera (the lens faces away from the user). If you're using a webcam on your development computer to simulate the back-facing camera on the emulated device, this value must be set to <code>webcamn</code> , where n selects the webcam (if you have only one webcam, choose webcam0). If set to <code>emulated</code> , the emulator simulates the camera in software. To disable the back-facing camera, set this value to <code>none</code> . If you enable the back-facing camera, be sure to also enable <code>hw.camera</code> .	emulated, none, webcam0
<code>hw.camera.front</code>	Front-facing camera – Configures the front-facing camera (the lens faces towards the user). If you're using a webcam on your development computer to simulate the front-facing camera on the emulated device, this value must be set to <code>webcamn</code> , where n selects the webcam (if you have only one webcam, choose webcam0). If set to <code>emulated</code> , the emulator simulates a camera in software. To disable the front-facing camera, set	emulated, none, webcam0

Property	Description	Options
	this value to <code>none</code> . If you enable the front-facing camera, be sure to also enable <code>hw.camera</code> .	
<code>hw.camera.maxHorizontalPixels</code>	Maximum horizontal camera pixels – Configures the maximum horizontal resolution of the emulated device's camera (in pixels).	
<code>hw.camera.maxVerticalPixels</code>	Maximum vertical camera pixels – Configures the maximum vertical resolution of the emulated device's camera (in pixels).	
<code>hw.cpu.arch</code>	CPU architecture – The CPU architecture to be emulated by the virtual device. To simulate an ARM CPU, select arm for 32-bit or select arm64 for a 64-bit ARM CPU. Keep in mind that ARM-based virtual devices run much slower than devices that are x86-based because hardware acceleration isn't available for ARM.	x86, x86_64, arm, arm64
<code>hw.cpu.model</code>	CPU model – This value is normally left unset (it will be set to a value that is derived from <code>hw.cpu.arch</code> if it isn't explicitly set). However, it can be set to an emulator-specific string for experimental use.	
<code>hw.dPad</code>	DPad keys – Determines whether the emulated device supports directional pad (DPad) keys. A DPad typically has four keys to indicate directional control.	yes, no
<code>hw.gps</code>	GPS support – Determines whether the emulated device has a GPS (Global Positioning System) receiver.	yes, no
<code>hw.gpu.enabled</code>	GPU emulation – Determines whether the emulated device supports GPU emulation. When enabled, GPU emulation uses Open GL for Embedded Systems (OpenGL ES) for rendering both 2D and 3D graphics on the screen, and the associated GPU Emulation Mode setting determines how the GPU emulation is implemented.	yes, no
<code>hw.gpu.mode</code>	GPU emulation mode – Determines how GPU emulation is implemented by the emulator. If you select auto, the emulator chooses hardware and software acceleration based on your development computer setup. If you select host, the emulator uses your development computer's	auto, host, mesa, angle, swiftshader, off

Property	Description	Options
	graphics processor to perform GPU emulation for faster rendering. If your GPU isn't compatible with the emulator and you are on Windows, you can try angle instead of host. The angle mode uses DirectX to provide performance similar to host. If you select mesa, the emulator uses the Mesa 3D software library to render graphics. Select mesa if you have problems rendering via your development computer's graphics processor. The swiftshader mode can be used to render graphics in software with slightly less performance than using your computer's GPU. The off option (disable graphics hardware emulation) is a deprecated option that can cause improper rendering for some items and is therefore not recommended.	
<code>hw.gsmModem</code>	GSM modem support – Determines whether the emulated device includes a modem that supports the GSM (Global System for Mobile Communications) telephony radio system.	yes, no
<code>hw.initialOrientation</code>	Initial screen orientation – Configures the initial orientation of the screen on the emulated device (portrait or landscape mode). In portrait mode, the screen is taller than it is wide. In landscape mode, the screen is wider than it is tall. When running the emulated device, you can change the orientation if both portrait and landscape are supported in the device profile.	portrait, landscape
<code>hw.keyboard</code>	Keyboard support – Determines whether the emulated device supports a QWERTY keyboard.	yes, no
<code>hw.keyboard.charmap</code>	Keyboard charmap name – The name of the hardware charmap for this device. NOTE: This should always be the default qwerty2 unless you've modified the system image accordingly. This name is sent to the kernel at boot time. Using an incorrect name will result in an unusable virtual device.	
<code>hw.keyboard.lid</code>	Keyboard lid support – If keyboard support is enabled, this setting determines whether the QWERTY keyboard can be closed/hidden or opened/visible. This setting is ignored if <code>hw.keyboard</code> is set to false. NOTE: the default	yes, no

Property	Description	Options
	value is false if the emulated device targets API level 12 or higher.	
<code>hw.lcd.backlight</code>	LCD backlight – Determines whether an LCD backlight is simulated by the emulated device.	yes, no
<code>hw.lcd.density</code>	LCD density – The density of the emulated LCD display, measured in density-independent pixels, or dp (dp is a virtual pixel unit). When the setting is 160 dp, each dp corresponds to one physical pixel. At runtime, Android uses this value to select and scale the appropriate resources/assets for correct display rendering.	120, 160, 240, 213, 320
<code>hw.lcd.depth</code>	LCD color depth – The color bit-depth of the emulated framebuffer that holds the bitmap for driving the LCD display. This value can be 16 bits (65,536 possible colors) or 32 bits (16,777,216 colors plus transparency). The 32-bit setting can make the emulator run slightly slower but with better color accuracy.	16, 32
<code>hw.lcd.height</code>	LCD pixel height – The number of pixels that make up the vertical dimension of the emulated LCD display.	
<code>hw.lcd.width</code>	LCD pixel width – The number of pixels that make up the horizontal dimension of the emulated LCD display.	
<code>hw.mainKeys</code>	Hardware Back/Home keys – Determines whether the emulated device supports hardware Back and Home navigation buttons. You can set this value to yes if the buttons are implemented only in software. If <code>hw.mainKeys</code> is set to yes , the emulator doesn't display navigation buttons on the screen, but you can use the emulator side panel to "press" these buttons.	yes, no
<code>hw.ramSize</code>	Device RAM Size – The amount of physical RAM on the emulated device, in megabytes. The default value is computed from the screen size or the skin version. Increasing the size can provide faster emulator operation, but at the expense of demanding more resources from your development computer.	
<code>hw.screen</code>	Touch screen type – Defines the type of screen on the emulated device. A multi-touch screen	touch, multi-touch,

Property	Description	Options
	can track two or more fingers on the touch interface. A touch screen can detect only single-finger touch events. A no-touch screen doesn't detect touch events.	no-touch
<code>hw.sdCard</code>	SDCard support – Determines whether the emulated device supports insertion and removal of virtual SD (Secure Digital) cards. The emulator uses mountable disk images stored on your development computer to simulate the partitions of actual SD card devices (see <code>hw.sdCard.path</code>).	yes, no
<code>sdcard.size</code>	SDCard size – Specifies the size of the virtual SD card file at the location specified by <code>hw.sdCard.path</code> , available on the device (in bytes). If this value is an integer, it specifies the size in bytes. You can also specify the size in kilobytes, megabytes, and gigabytes by appending K , M , or G to the value. The minimum size is 9M and the maximum size is 1023G .	
<code>hw.sdCard.path</code>	SDCard Image Path – Specifies the filename and path to an SD card partition image file on your development computer. For example, this path could be set to C:\sd\sdcard.img on Windows.	
<code>hw.sensors.magnetic_field</code>	Magnetic Field Sensor – Determines whether the emulated device supports a magnetic field sensor. The magnetic field sensor (also known as magnetometer) reports the ambient geomagnetic field as measured along three sensor axes. Enable this setting for apps that need access to a compass reading. For example, a navigation app might use this sensor to detect which direction the user faces.	yes, no
<code>hw.sensors.orientation</code>	Orientation Sensor – Determines whether the emulated device provides orientation sensor values. The orientation sensor measures degrees of rotation that a device makes around all three physical axes (x, y, z). The orientation sensor was deprecated as of Android 2.2 (API level 8).	yes, no
<code>hw.sensors.proximity</code>	Proximity Sensor – Determines whether the emulated device supports a proximity sensor. This sensor measures the proximity of an object relative to the view screen of a device. This	yes, no

Property	Description	Options
	sensor is typically used to determine whether a handset is being held up to a person's ear.	
<code>hw.sensors.temperature</code>	Temperature Sensor – Determines whether the emulated device supports a temperature sensor. This sensor measures the temperature of the device in degrees Celsius (°C).	yes, no
<code>hw.touchScreen</code>	Touch-screen support – Determines whether the emulated device supports a touch screen. The touch screen is used for direct manipulation of objects on the screen.	yes, no
<code>hw.trackBall</code>	Trackball support – Determines whether the emulated device supports a trackball.	yes, no
<code>hw.useext4</code>	EXT4 file system support – Determines whether the emulated device uses the Linux EXT4 file system for partitions. Because the file system type is now autodetected, this option is deprecated and ignored.	no
<code>kernel.newDeviceNaming</code>	Kernel new device naming – Used to specify whether the kernel requires a new device naming scheme. A new naming scheme is typically used with Linux 3.10 kernels and later. If set to autodetect , the emulator automatically detects whether the kernel requires a new device naming scheme.	autodetect, yes, no
<code>kernel.parameters</code>	Kernel parameters – Specifies the string of Linux kernel boot parameters. By default, this setting is left blank.	
<code>kernel.path</code>	Kernel path – Specifies the path to the Linux kernel. If this path isn't specified, the emulator looks in the emulator system directory for kernel-ranchu.	
<code>kernel.supportsYaffs2</code>	YAFFS2 partition support – Determines whether the kernel supports YAFFS2 (Yet Another Flash File System 2) partitions. Typically, this support applies only to kernels before Linux 3.10. If set to autodetect , the emulator automatically detects whether the kernel can mount YAFFS2 file systems.	autodetect, yes, no
<code>skin.name</code>	Skin name – The name for an Android emulator skin. A skin is a collection of files that defines the	

Property	Description	Options
	visual and control elements of an emulator display; it describes what the window of the AVD will look like on your development computer. A skin describes screen size, buttons, and the overall design, but it doesn't affect the operation of your app.	
<code>skin.path</code>	Skin path – Path to the directory that contains the emulator skin files specified in <code>skin.name</code> . This directory contains <code>hardware.ini</code> layout files, and image files for the display elements of the skin.	
<code>skin.dynamic</code>	Skin dynamic – Whether or not the skin is dynamic. The emulator skin is a dynamic skin if the emulator is to construct a skin of a given size based on a specified width and height.	no

For more information about these properties, see [Hardware Profile Properties](#).

Debug on the Android Emulator

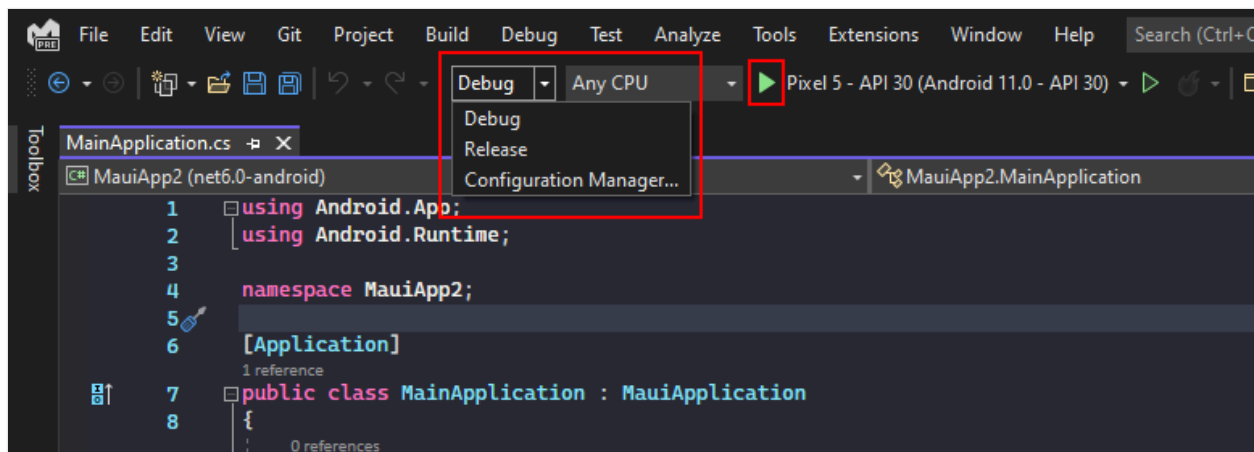
Article • 08/27/2024

The Android Emulator, installed as part of the **.NET Multi-Platform App UI development** workload, can be run in various configurations to simulate different Android devices. Each one of these configurations is created as a *virtual device*. In this article, you'll learn how to launch the emulator from Visual Studio and run your app in a virtual device. For more information about how to create and configure a virtual device, see [Managing virtual devices with the Android Device Manager](#).

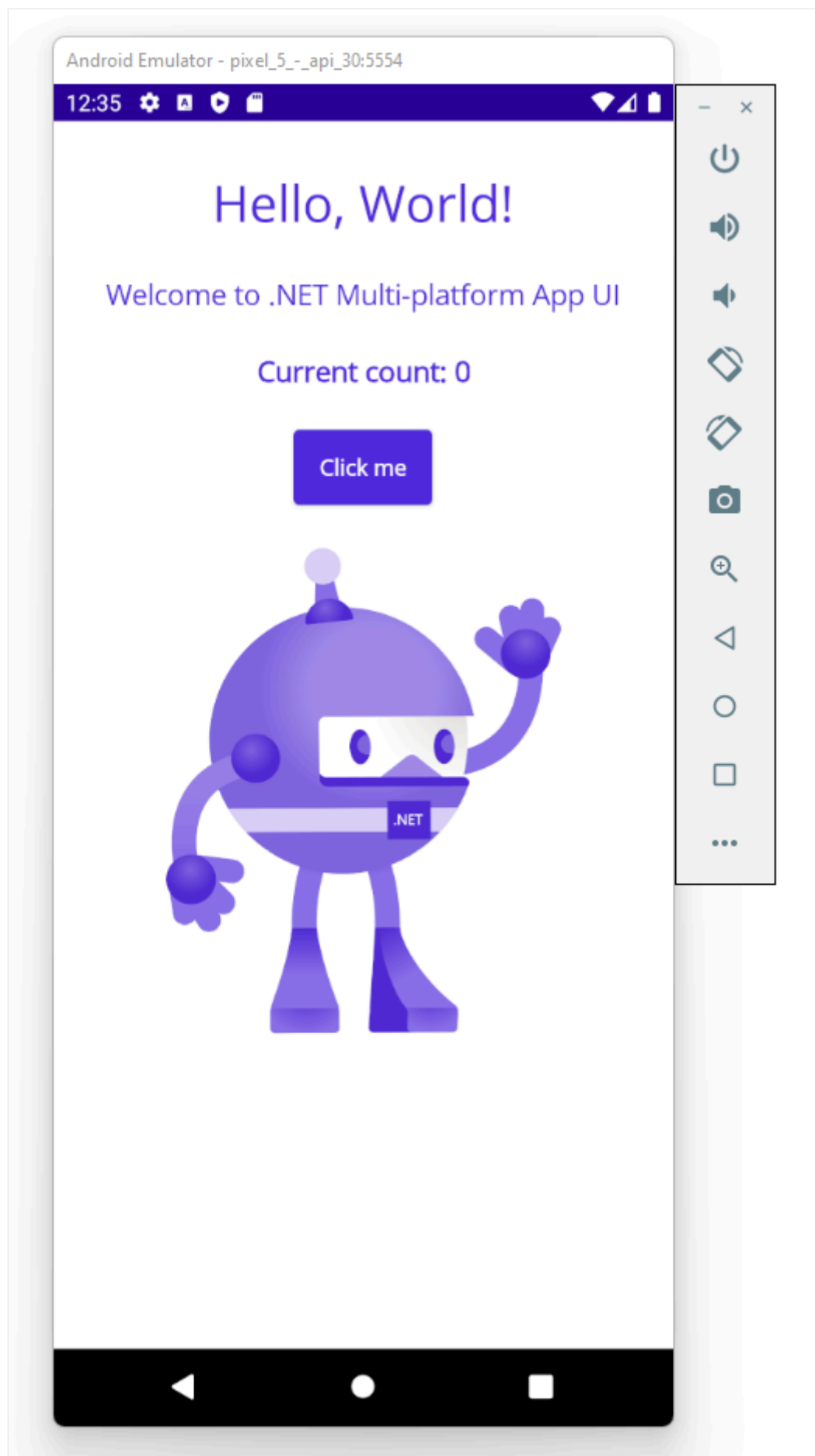
Launching the Emulator

Near the top of Visual Studio, there's the **Solution Configurations** drop-down menu that can be used to select **Debug** or **Release** mode. Choosing **Debug** causes the debugger to attach to the application process running inside the emulator after the app starts. Choosing **Release** mode disables the debugger. When in release mode, you'll need to rely on app logging for debugging.

After you've chosen a virtual device from the **Debug Target** device drop-down menu, select either **Debug** or **Release** mode, then select the Play button to run the application:



After the emulator starts, Visual Studio deploys the app to the virtual device. An example screenshot of the Android Emulator is displayed below. In this example, the emulator is running the .NET MAUI template app.



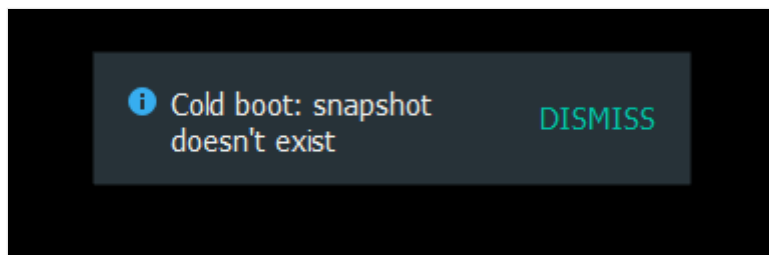
When you're finished debugging and running your app, you can leave the emulator running. The first time a .NET MAUI app is run in the emulator, the .NET MAUI shared runtime for the targeted API level is installed, followed by the app. The runtime

installation may take a few moments to install. If you leave the emulator running, later debugging sessions start faster as the runtime is already present on the device. If the device is restarted, the runtime will be redeployed to the device.

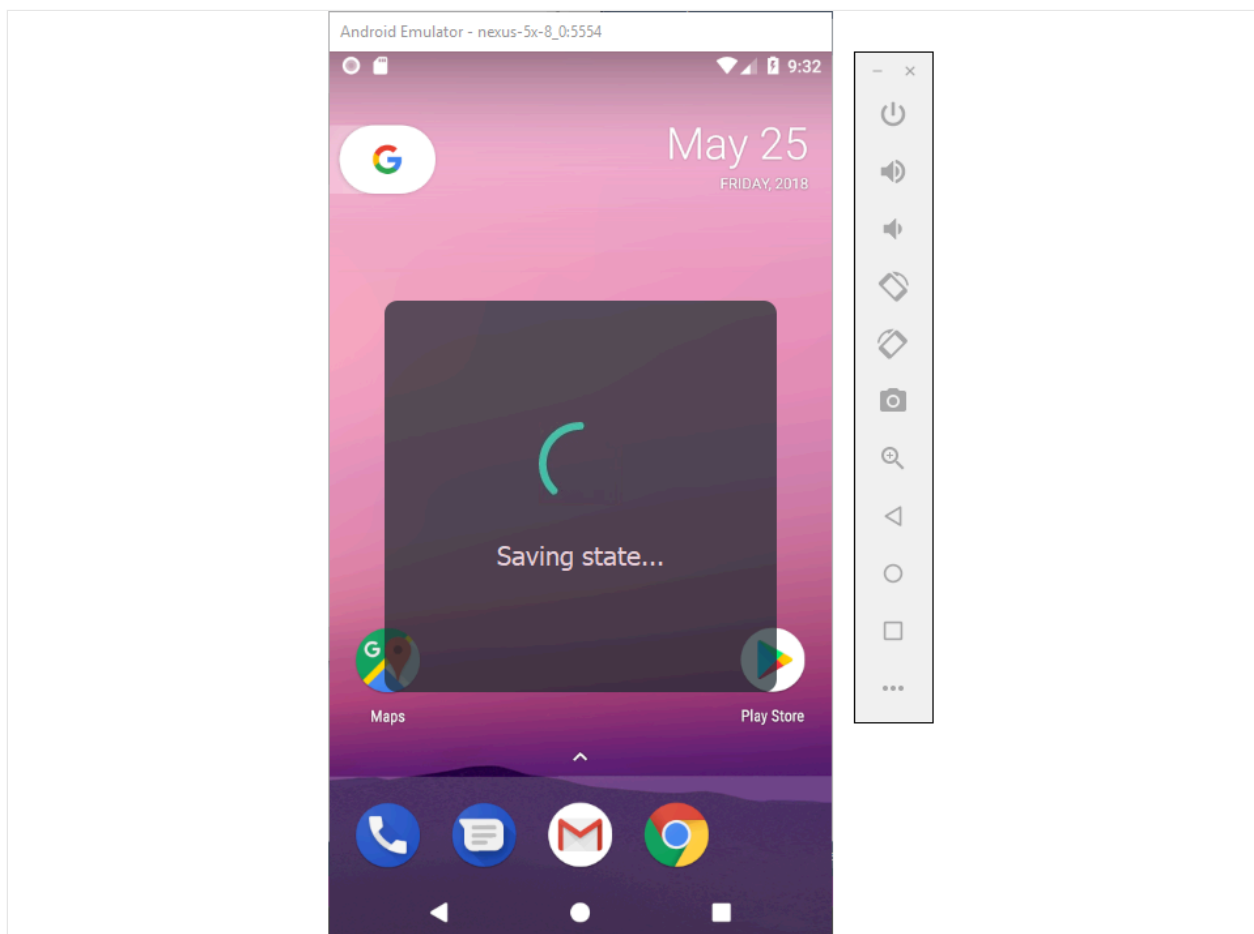
Fast boot

The Android Emulator includes a feature named Fast Boot which is enabled by default. This feature is configured by each device's emulator settings. With this feature enabled, a snapshot of the virtual device is saved when the emulator is closed. The snapshot is quickly restored the next time the device is started.

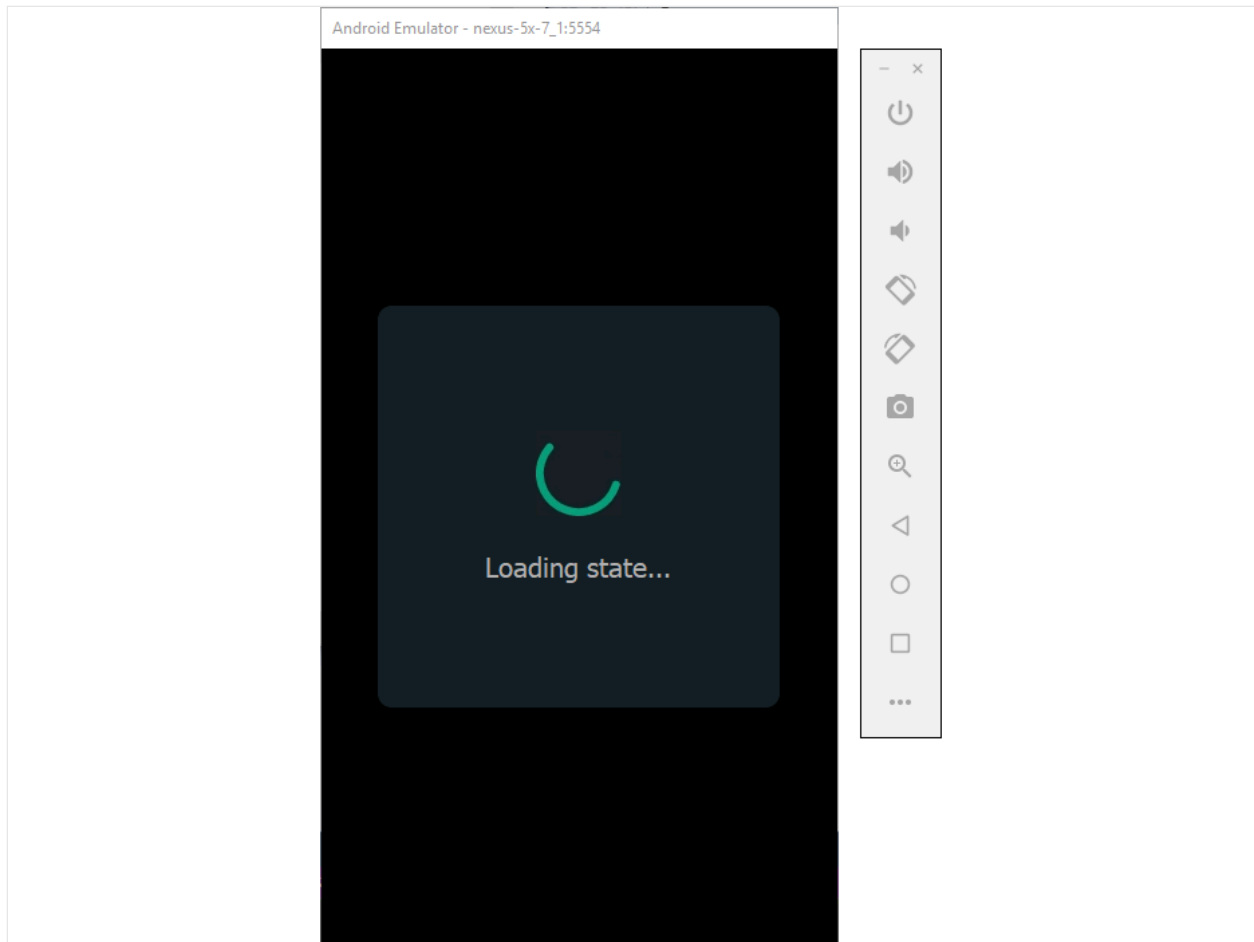
The first time a virtual device is started, a cold boot of the virtual device takes place without a speed improvement because a snapshot hasn't yet been created:



When you exit out of the emulator, Fast Boot saves the state of the emulator in a snapshot:



The next time the virtual device starts, it loads much faster because the emulator simply restores the state at which you closed the emulator.



Troubleshooting

For tips and workarounds for common emulator problems, see [Android Emulator Troubleshooting](#).

For more information about using the Android Emulator, see the following Android Developer articles:

- [Navigating on the Screen](#) [↗](#)
- [Performing Basic Tasks in the Emulator](#) [↗](#)
- [Working with Extended Controls, Settings, and Help](#) [↗](#)
- [Run the emulator with Quick Boot](#) [↗](#)

Connect to an Android emulator on a Mac from inside a Windows virtual machine

Article • 08/27/2024

The Android emulator provides versatile networking capabilities that can be used for different purposes, including connecting to an emulator running on a Mac from inside a Windows virtual machine (VM). This is useful when running Windows in Parallels on a Mac. In this scenario it's required to use the emulator on the Mac since it can't run in Parallels. For information about installing the Android emulator on a Mac, see [Installation](#).

❗ Important

The address 127.0.0.1 on your development machine corresponds to the emulator's loopback interface.

There are two main approaches for connecting to an Android emulator on a Mac from inside a Windows VM:

1. Using `nc` to perform packet forwarding. For more information, see [Use nc to perform packet forwarding](#).
2. Using `ssh` port forwarding. For more information, see [Use ssh port forwarding](#).

In both cases, the Android Debug Bridge (ADB) is used to connect to the emulator. ADB is a command-line tool that's bundled with the Android SDK Platform Tools package, that lets you communicate with a device. The `adb` command facilitates a variety of device actions, including connecting to devices. For more information about `adb`, see [Android Debug Bridge \(adb\)](#) [↗] on developer.android.com.

Use nc to perform packet forwarding

To connect to the Android Emulator running on a Mac from a Windows VM, by using packet forwarding, use the following steps:

1. On your Mac, start the Android emulator.
2. On your Mac, open **Terminal**.

3. In **Terminal**, kill the `adb` server:

```
zsh  
  
adb kill-server
```

4. In **Terminal**, identify the ports used by the emulator:

```
zsh  
  
lsof -iTCP -sTCP:LISTEN -P | grep 'emulator\|qemu'  
  
emulator6 94105 macuser 20u IPv4 0xa8dacfb1d4a1b51f 0t0 TCP  
localhost:5555 (LISTEN)  
emulator6 94105 macuser 21u IPv4 0xa8dacfb1d845a51f 0t0 TCP  
localhost:5554 (LISTEN)
```

The emulator uses a pair of sequential TCP ports - an even-numbered port for console connections, and an odd-numbered port for `adb` connections. In the output above the emulator listens for `adb` on port 5555 and listens for console connections on port 5554.

5. In **Terminal**, use `nc` to forward inbound TCP packets received externally on port 5555 (or any other port) to the odd-numbered port on the loopback interface, and to forward the outbound packets back the other way:

```
zsh  
  
cd /tmp  
mkfifo backpipe  
nc -kl 5555 0<backpipe | nc 127.0.0.1 5555 > backpipe
```

In this example, `127.0.0.1 5555` denotes the odd-numbered port on the loopback interface.

Provided that the `nc` command keeps running in a Terminal window, packets will be forwarded as expected. Once you've finished using the emulator, you can stop `nc` performing packet forwarding by pressing `CTRL+C` in the Terminal window.

6. In your Windows VM, open **Command Prompt**.

7. In **Command Prompt**, connect to the emulator:

```
Windows Command Prompt
```

```
adb connect ip-address-of-the-mac:5555
```

Replace `ip-address-of-the-mac` in this example with the IP address of your Mac, and 5555 with the port you've used from the previous step.

⚠ Note

Command line access to `adb` can be obtained in Visual Studio through the **Tools > Android > Android Adb Command Prompt...** menu item.

When the connection is completed Visual Studio will display the emulator as a debug target under **Android Local Devices**, which you can use to deploy your app to the emulator.

Use ssh port forwarding

Provided that **Remote Login** is enabled on your Mac, you can use `ssh` port forwarding to connect to the emulator.

📌 Important

`ssh` port forwarding requires you to have installed a SSH client in your Windows VM. One option is to install [Git for Windows](#). The `ssh` command will then be available in the **Git Bash** command prompt.

To connect to the Android Emulator running on a Mac from a Windows virtual machine, by using `ssh` port forwarding, use the following steps:

1. On your Mac, start the Android emulator.
2. On your Mac, open **Terminal**.
3. In **Terminal**, kill the `adb` server:

```
zsh
```

```
adb kill-server
```

4. In **Terminal**, identify the ports used by the emulator:

```
zsh
```

```
lsof -iTCP -sTCP:LISTEN -P | grep 'emulator\|qemu'
```

```
emulator6 94105 macuser 20u IPv4 0xa8dacfb1d4a1b51f 0t0 TCP
localhost:5555 (LISTEN)
emulator6 94105 macuser 21u IPv4 0xa8dacfb1d845a51f 0t0 TCP
localhost:5554 (LISTEN)
```

The emulator uses a pair of sequential TCP ports - an even-numbered port for console connections, and an odd-numbered port for `adb` connections. In the output above the emulator listens for `adb` on port 5555 and listens for console connections on port 5554.

5. In your Windows VM, open **Command Prompt**.
6. In **Command Prompt**, run `ssh` to setup two-way port forwarding between a local port on Windows and the odd-numbered emulator port on the Mac's loopback interface:

```
Windows Command Prompt
```

```
ssh -L localhost:15555:127.0.0.1:5555 mac-username@ip-address-of-the-
mac
```

In this example, `localhost:15555` denotes the local port on Windows and `127.0.0.1 5555` denotes the odd-numbered port on the loopback interface.

Replace `mac-username` with your Mac username, which can be obtained with the `whoami` command, and `ip-address-of-the-mac` with the IP address of your Mac.

7. In **Command Prompt**, connect to the emulator using the local port:

```
Windows Command Prompt
```

```
adb connect localhost:15555
```

In this example, `localhost:15555` denotes the local port on Windows

⚠ Note

Command line access to `adb` can be obtained in Visual Studio through the **Tools > Android > Android Adb Command Prompt...** menu item.

When the connection is completed Visual Studio will display the emulator as a debug target under **Android Local Devices**, which you can use to deploy your app to the emulator.

Android emulator troubleshooting

Article • 08/27/2024

This article describes the most common warning messages and issues that occur while configuring and running the Android Emulator. Also, it describes solutions for resolving these errors and various troubleshooting tips to help you diagnose emulator problems. For more information, see [Troubleshoot known issues with Android Emulator](#) on developer.android.com.

Deployment issues on Windows

Some error messages may be displayed by the emulator when you deploy your app. The most common errors and solutions are explained here.

Deployment errors

If you see an error about a failure to install the APK on the emulator or a failure to run the Android Debug Bridge (**adb**), verify that the Android SDK can connect to your emulator. To verify emulator connectivity, use the following steps:

1. Launch the emulator from the **Android Device Manager** (select your virtual device and select **Start**).
2. Open a command prompt and go to the folder where **adb** is installed. If the Android SDK is installed at its default location, **adb** is located at `C:\Program Files (x86)\Android\android-sdk\platform-tools\adb.exe`; if not, modify this path for the location of the Android SDK on your computer.
3. Type the following command:

```
Console
```

```
adb devices
```

4. If the emulator is accessible from the Android SDK, the emulator should appear in the list of attached devices. For example:

```
Console
```

```
List of devices attached
emulator-5554    device
```


5. If the emulator doesn't appear in this list, start the **Android SDK Manager**, apply all updates, then try launching the emulator again.

MMIO access error

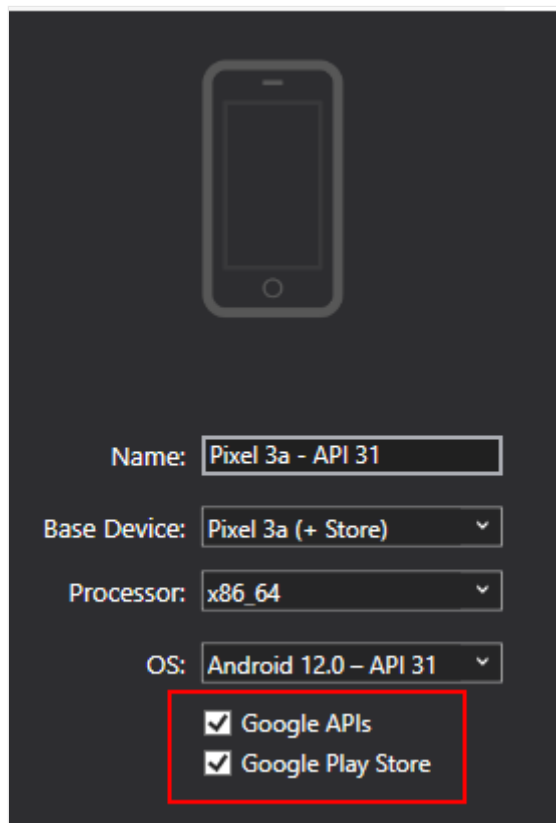
If the message **An MMIO access error has occurred** is displayed, restart the emulator.

Missing Google Play Services

If the emulated Android device doesn't have Google Play Services or Google Play Store installed, you probably created a virtual device that excluded these packages. When you [create a virtual device](#), be sure to select one or both of the following options:

- **Google APIs**—includes Google Play Services in the virtual device.
- **Google Play Store**—includes Google Play Store in the virtual device.

For example, this virtual device will include Google Play Services and Google Play Store:



ⓘ Note

Google Play Store images are available only for some base device types such as Pixel, Pixel 2, Nexus 5, and Nexus 5X.

Performance issues

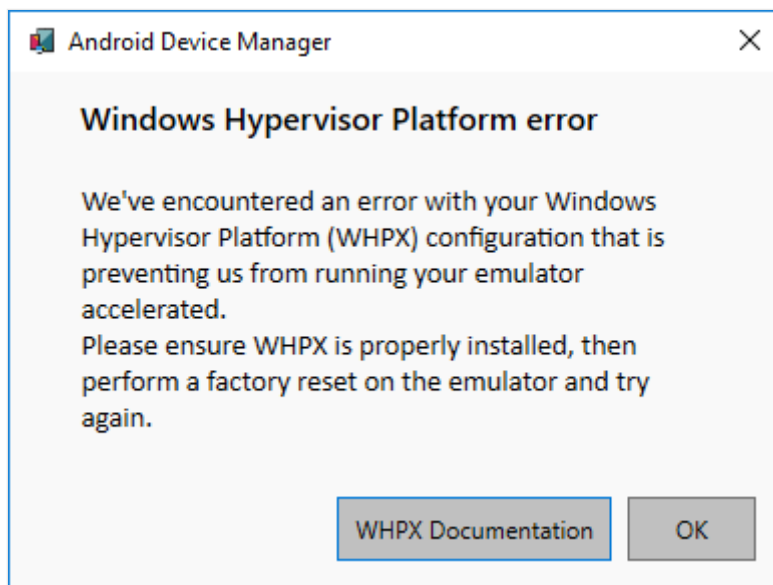
Performance issues are typically caused by one of the following problems:

- The emulator is running without hardware acceleration.
- The virtual device running in the emulator using an Arm-based image.

The following sections cover these scenarios in more detail.

Hardware acceleration isn't enabled

When you start a virtual device, and you don't have hardware acceleration enabled, the Device Manager displays an error dialog similar to the following image:



To fix this error, follow the troubleshooting steps in the [Hardware acceleration issues](#) section.

Hardware acceleration issues

When using hardware acceleration, you may run into configuration problems or conflicts with other software on your computer. The first step in troubleshooting is verifying that hardware acceleration is enabled. You can use the Android's SDK to check this setting. Open a command prompt and enter the following command:

Windows Command Prompt

```
"C:\Program Files (x86)\Android\android-sdk\emulator\emulator-check.exe"  
accel
```

This command assumes that the Android SDK is installed at the default location of *C:\Program Files (x86)\Android\android-sdk*. If the Android SDK is installed elsewhere, modify the preceding command to the correct location.

Tip

Make sure the Android Emulator is up to date. From Visual Studio, press **Tools > Android > Android SDK Manager**. Select the **Tools** tab and see if the **Android Emulator** entry has an update available.

Incorrect BIOS settings

If the BIOS hasn't been configured properly to support hardware acceleration, a message similar to the following example will be displayed when you run the **emulator-check.exe accel** command:

Windows Command Prompt

VT feature disabled in BIOS/UEFI

To correct this problem, reboot into your computer's BIOS and enable the following options:

- Virtualization Technology (may have a different label depending on motherboard manufacturer).
- Hardware Enforced Data Execution Prevention.

If problems still occur because of issues related to Hyper-V, see the following section.

Hyper-V issues

In some cases, enabling both **Hyper-V** and **Windows Hypervisor Platform** in the **Turn Windows features on or off** dialog may not properly enable Hyper-V. To verify that Hyper-V is enabled, use the following steps:

1. Enter **PowerShell** in the Windows search box.
2. Right-click **Windows PowerShell** in the search results and select **Run as administrator**.
3. In the PowerShell console, enter the following command:

PowerShell

```
Get-WindowsOptionalFeature -FeatureName Microsoft-Hyper-V-All -Online
```

If Hyper-V isn't enabled, a message similar to the following example will be displayed to indicate that the state of Hyper-V is **Disabled**:

Windows Command Prompt

```
FeatureName       : Microsoft-Hyper-V-All
DisplayName       : Hyper-V
Description       : Provides services and management tools for creating
                  and running virtual machines and their resources.
RestartRequired  : Possible
State            : Disabled
CustomProperties  :
```

4. In the PowerShell console, enter the following command:

PowerShell

```
Get-WindowsOptionalFeature -FeatureName HypervisorPlatform -Online
```

If the Hypervisor isn't enabled, a message similar to the following example will be displayed to indicate that the state of HypervisorPlatform is **Disabled**:

Windows Command Prompt

```
FeatureName       : HypervisorPlatform
DisplayName       : Windows Hypervisor Platform
Description       : Enables virtualization software to run on the
                  Windows hypervisor
RestartRequired  : Possible
State            : Disabled
CustomProperties  :
```

If Hyper-V or HypervisorPlatform aren't enabled, use the following PowerShell commands to enable them:

PowerShell

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
Enable-WindowsOptionalFeature -Online -FeatureName HypervisorPlatform -All
```

After these commands complete, reboot.

For more information about enabling Hyper-V (including techniques for enabling Hyper-V using the Deployment Image Servicing and Management tool), see [Install Hyper-V](#).

More troubleshooting tips

The following suggestions are often helpful in diagnosing Android emulator issues.

Starting the emulator from the command line

If the emulator isn't already running, you can start it from the command line (rather than from within Visual Studio) to view its output. Typically, Android emulator AVD images are stored at the following location: `%userprofile%\android\avd`.

You can launch the emulator with an AVD image from this location by passing in the folder name of the AVD. For example, this command launches an AVD named **Pixel_API_27**:

Windows Command Prompt

```
"C:\Program Files (x86)\Android\android-sdk\emulator\emulator.exe" -  
partition-size 2000 -no-boot-anim -verbose -feature  
WindowsHypervisorPlatform -avd pixel_5_-_api_30 -prop  
monodroid.avdname=pixel_5_-_api_30
```

This command assumes that the Android SDK is installed at the default location of `C:\Program Files (x86)\Android\android-sdk`. If the Android SDK is installed elsewhere, modify the preceding command to the correct location.

Viewing Device Manager logs

Often you can diagnose emulator problems by viewing the Device Manager logs. These logs are written to the following location:

`%userprofile%\AppData\Local\Xamarin\Logs\16.0`.

You can view each *DeviceManager.log* file by using a text editor.

Set up Android device for debugging

Article • 09/30/2024

While the [Android emulator](#) is a great way to rapidly develop and test your app, you'll want to test your apps on a real Android device. To run on a device, you'll need to enable developer mode on the device and connect it to your computer. For more information, see [Run apps on a hardware device](#) on developer.android.com.

Important

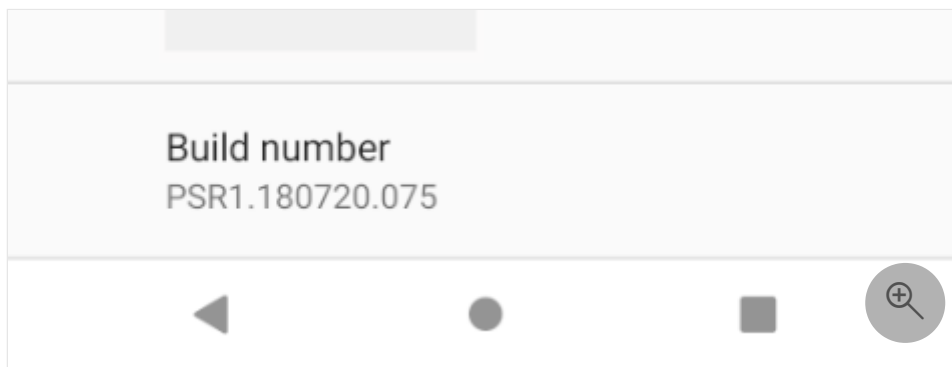
The steps in this article are written generically, to work on as many devices as possible. If you can't find these settings on your device, consult your device manufacturer's documentation.

Enable developer mode on the device

A device must enable Developer mode in order to deploy and test an Android app. Developer mode is enabled by following these steps:

1. Go to the **Settings** screen.
2. Select **About phone**.
3. Tap **Build Number** seven times until **You are now a developer!** is visible.

Depending on the UI your device is running, the **About phone** option may be in a different location. Consult your device documentation if you can't find **About phone**.



Enable USB debugging

After enabling developer mode on your device, enable USB debugging by following these steps:

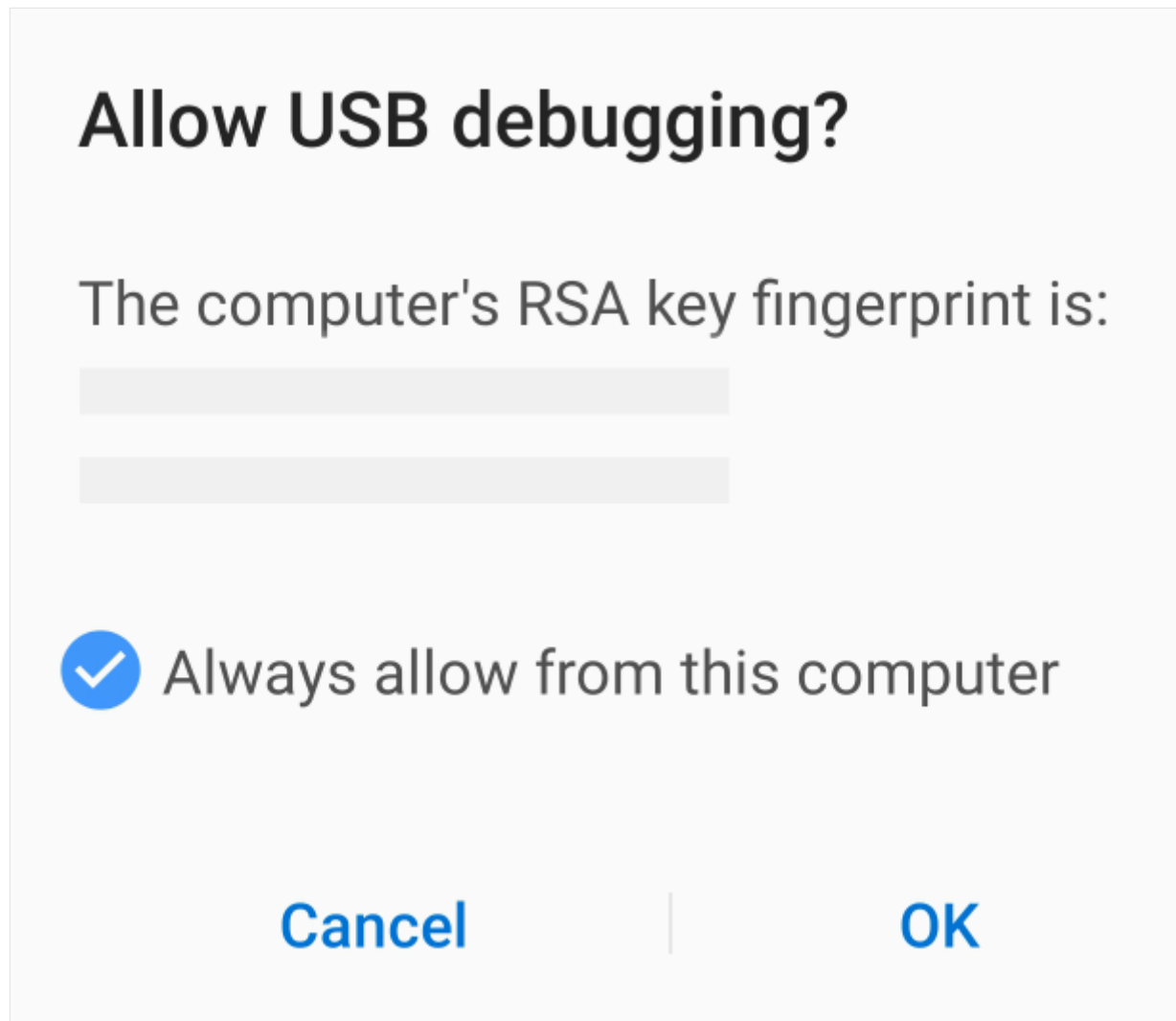
1. Go to the **Settings** screen.
2. Select **Developer options**.
3. Turn on the **USB debugging** option.

Depending on the UI your device is running, the **USB debugging** option may be in a different location. Consult your device documentation if you can't find **USB debugging**.

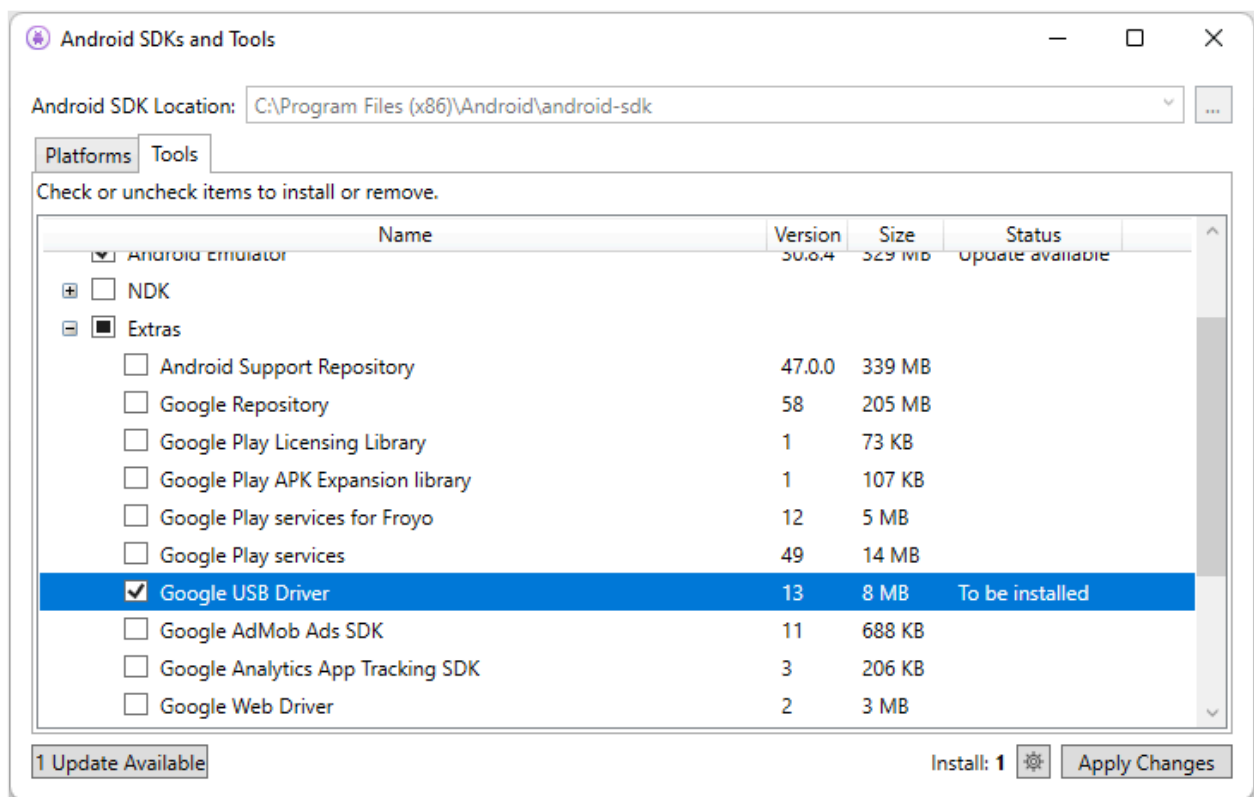
Connect the device to the computer

The final step is to connect the device to the computer. The easiest and most reliable way is to do so over USB.

You'll receive a prompt to trust the computer on your device if you haven't used it for debugging before. You can also check **Always allow from this computer** to prevent requiring this prompt each time you connect the device.



If your computer isn't recognizing the device when it's plugged in, try installing a driver for the device. Consult your device manufacturer's support documentation. You can also try installing the Google USB Driver through the Android SDK Manager:



Enable WiFi debugging

It's possible to debug an android device over WiFi, without keeping the device physically connected to the computer. This technique requires more effort, but could be useful when the device is too far from the computer to remain constantly plugged-in via a cable.

Connecting over WiFi

By default, the Android Debug Bridge (adb) is configured to communicate with an Android device via USB. It's possible to reconfigure it to use TCP/IP instead of USB. To do this, both the device and the computer must be on the same WiFi network.

ⓘ Note

The Google USB driver is required to perform `adb` debugging on Windows with Google devices. For more information, see [Get the Google USB Driver](https://developer.android.com/studio/run/win-usb) on developer.android.com. Windows drivers for all other devices are provided by the respective hardware manufacturer. For more information, see [Install OEM USB drivers](https://developer.android.com/studio/run/win-usb) on developer.android.com.

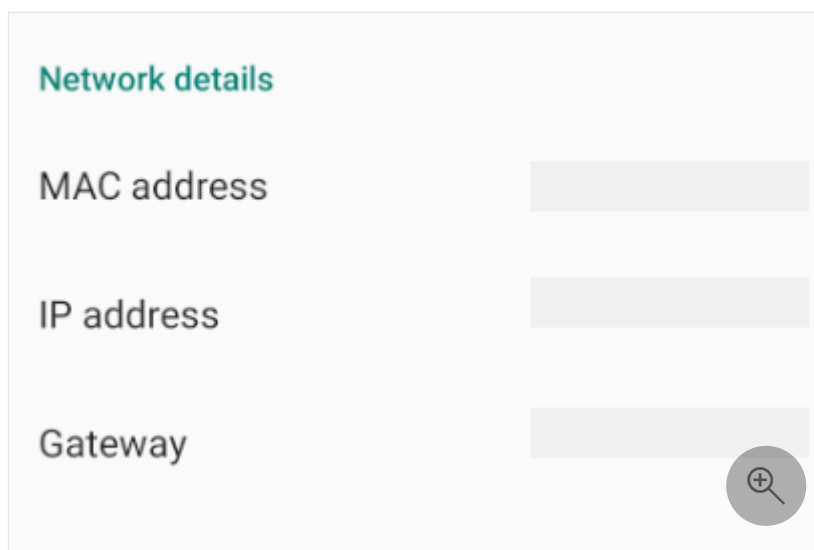
First, enable Wireless debugging on your Android device:

1. Follow the steps in the [Enable developer mode on the device](#) section.
2. Follow the steps in the [Enable USB debugging](#) section.
3. Go to the **Settings** screen.
4. Select **Developer options**.
5. Turn on the **Wireless debugging** option.

Depending on the UI your device is running, the **Wireless debugging** option may be in a different location. Consult your device documentation if you can't find **Wireless debugging**.

Next, use adb to connect to your device, first through a USB connection:

1. Determine the IP address of your Android device. One way to find out the IP address is to look under **Settings > Network & internet > Wi-Fi**, then tap on the WiFi network that the device is connected to, and then tap on **Advanced**. This will open a drop-down showing information about the network connection, similar to what is seen in the screenshot below:



On some versions of Android the IP address won't be listed there but can be found instead under **Settings > About phone > Status**.

2. In Visual Studio, open the adb command prompt by selecting the menu option: **Tools > Android > Android Adb Command Prompt....**
3. In the command prompt, use the `adb tcpip` command to tell the device to listen to TCP/IP connections on port 5555.

command

```
adb tcpip 5555
```

4. Disconnect the USB cable from your device.

5. Connect to the device's IP address with port 5555:

```
command
```

```
adb connect 192.168.1.28:5555
```

When this command finishes, the Android device is connected to the computer via WiFi.

When you're finished debugging via WiFi, you can reset ADB back to USB mode with the following command:

```
command
```

```
adb usb
```

To see the devices connected to the computer, use the `adb devices` command:

```
command
```

```
adb devices
```

Configure on-device developer options

The Settings app on Android includes a screen called **Developer options** where you can configure system behaviors that help you profile and debug your app performance. For more information, see [Configure on-device developer options](#) on developer.android.com.

Build an iOS app with .NET CLI

Article • 09/30/2024

In this tutorial, you'll learn how to create and run a .NET Multi-platform App UI (.NET MAUI) app on iOS using .NET Command Line Interface (CLI) on macOS:

1. To create .NET MAUI apps, you'll need to download and run the [installer](#) for the latest .NET runtime. You'll also need to download and install the version of Xcode that's required by the version of .NET MAUI that you're using. For information, see [Release versions](#).
2. On your Mac, open **Terminal** and check that you have the latest .NET runtime installed:

```
zsh  
  
dotnet --version
```

3. In **Terminal**, install the latest public build of .NET MAUI:

```
zsh  
  
sudo dotnet workload install maui --source  
https://api.nuget.org/v3/index.json
```

This command will install the latest released version of .NET MAUI, including the required platform SDKs.

4. In **Terminal**, create a new .NET MAUI app using .NET CLI:

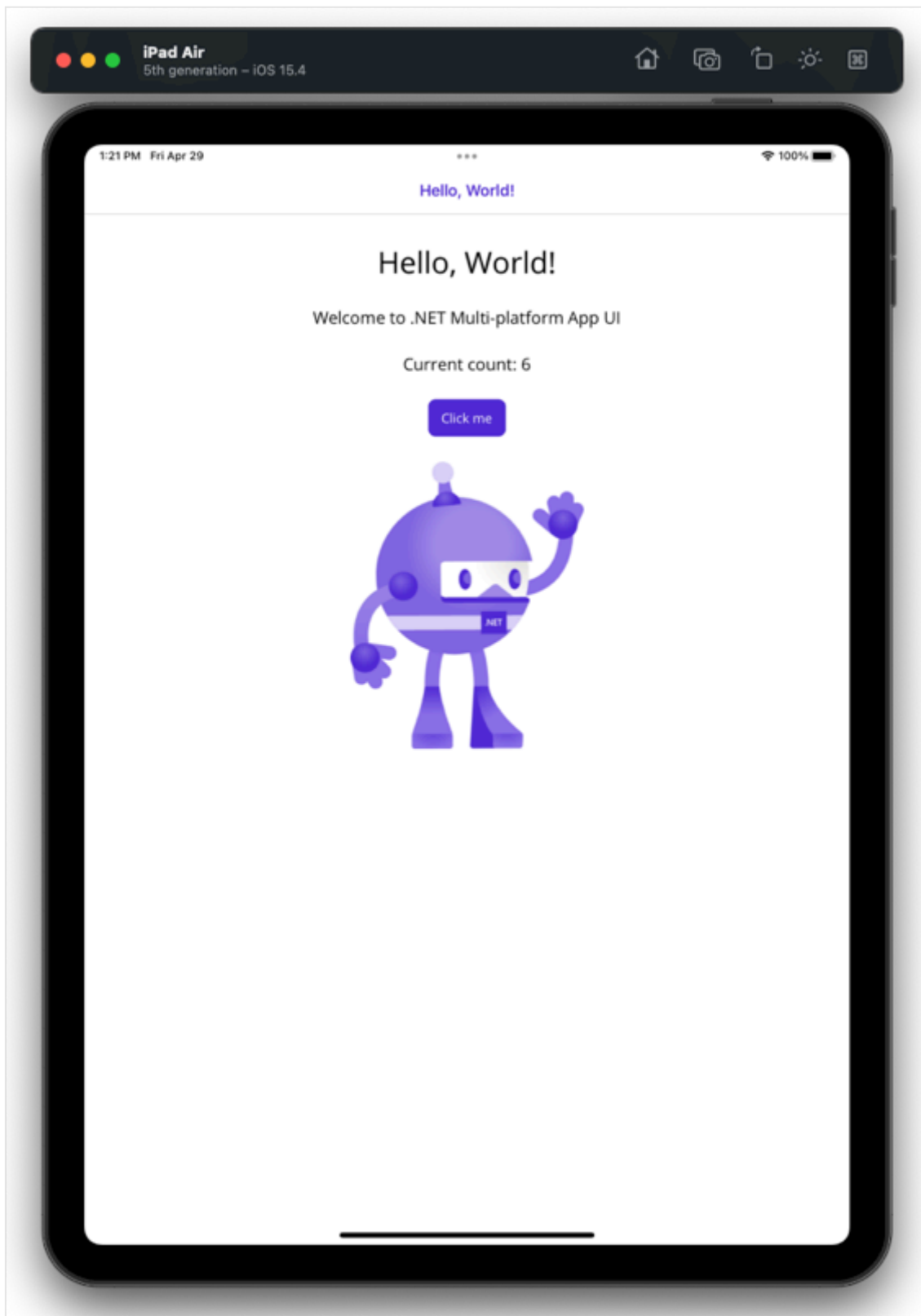
```
zsh  
  
dotnet new maui -n "MyMauiApp"
```

5. In **Terminal**, change directory to *MyMauiApp*, and build and run the app:

```
zsh  
  
cd MyMauiApp  
dotnet build -t:Run -f net8.0-ios
```

The `dotnet build` command will restore the project the dependencies, build the app, and launch it in the default simulator.

6. In the default simulator, press the **Click me** button several times and observe that the count of the number of button clicks is incremented.



Build with a specific version of Xcode

If you have multiple versions of Xcode installed on your Mac, it's possible to specify which Xcode version should be used when building your app. There are a number of

approaches that can be used to accomplish this, but the recommended approach is to set the `MD_APPLE_SDK_ROOT` environment variable to the path of the Xcode version.

⚠ Warning

Using `xcode-select -s` to set the version of Xcode to use isn't recommended.

To set the `MD_APPLE_SDK_ROOT` environment variable for the duration of the current terminal session:

1. Open the **Terminal** application.
2. Type the following command, substituting in your version of Xcode, and press Enter:

```
zsh

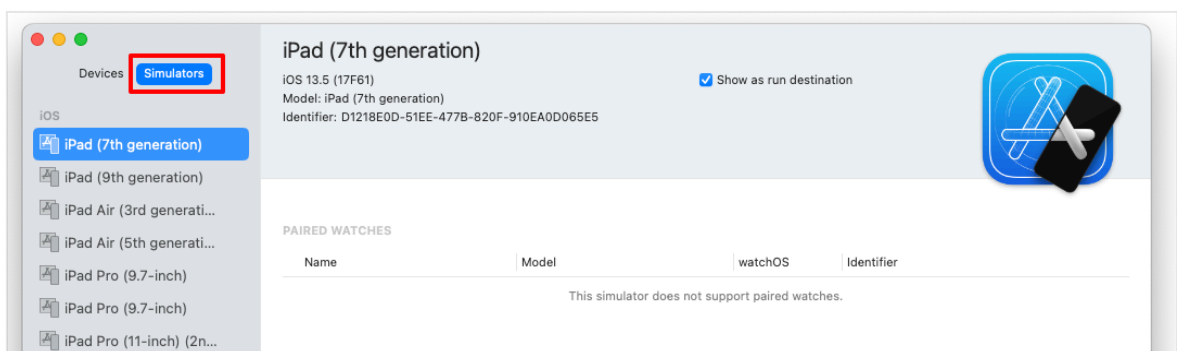
export MD_APPLE_SDK_ROOT=/Applications/Xcode_14.1.0.app
```

If you want to set this environment variable permanently, you can add the `export` command to your shell profile, such as `.zprofile`.

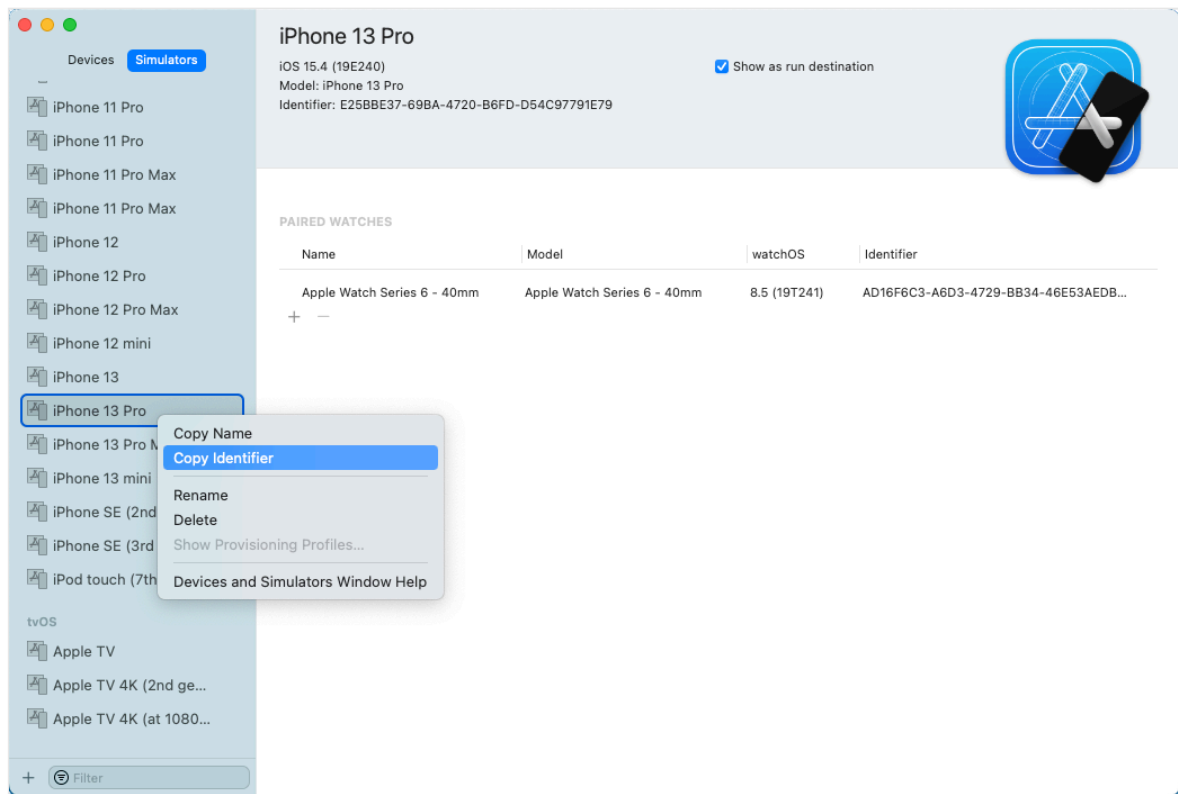
Launch the app on a specific simulator

A .NET MAUI iOS app can be launched on a specific iOS simulator from a Mac by providing its unique device id (UDID):

1. On your Mac, open **Xcode**, select the **Windows > Devices and Simulators** menu item, and then the **Simulators** tab.



2. Right-click on your chosen simulator, and select **Copy Identifier** to copy the UDID to the clipboard.



Alternatively, you can retrieve a list of UDID values by executing the `simctl list` command:

```
zsh
```

```
/Applications/Xcode.app/Contents/Developer/usr/bin/simctl list
```

3. In **Terminal**, build the app and run it on your chosen simulator by specifying the `_DeviceName` MSBuild property using the `-p` [MSBuild option](#):

```
zsh
```

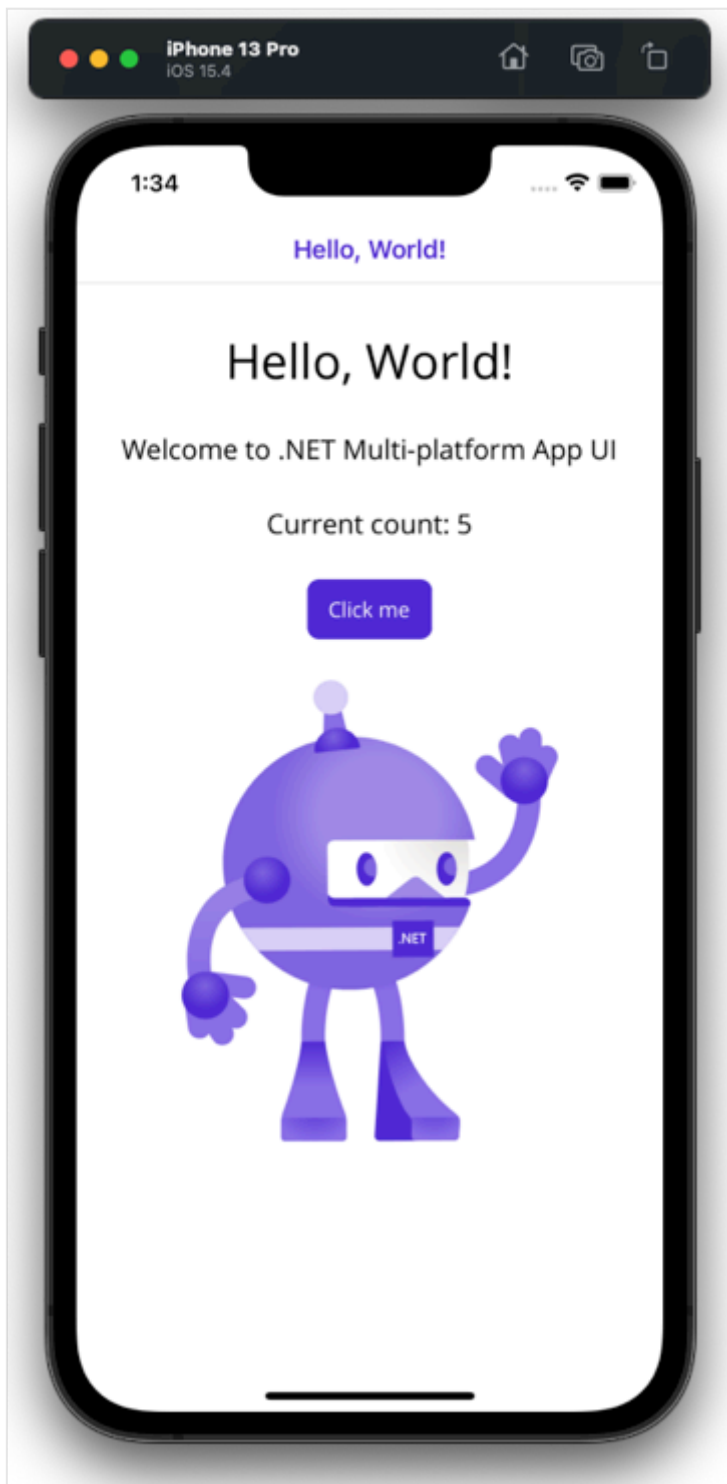
```
dotnet build -t:Run -f net8.0-ios -  
p:_DeviceName=:v2:udid=MY_SPECIFIC_UDID
```

For example, use the following command to build the app and run it on the iPhone 13 Pro simulator:

```
zsh
```

```
dotnet build -t:Run -f net8.0-ios -p:_DeviceName=:v2:udid=E25BBE37-  
69BA-4720-B6FD-D54C97791E79
```

4. In your chosen simulator, press the **Click me** button several times and observe that the count of the number of button clicks is incremented.

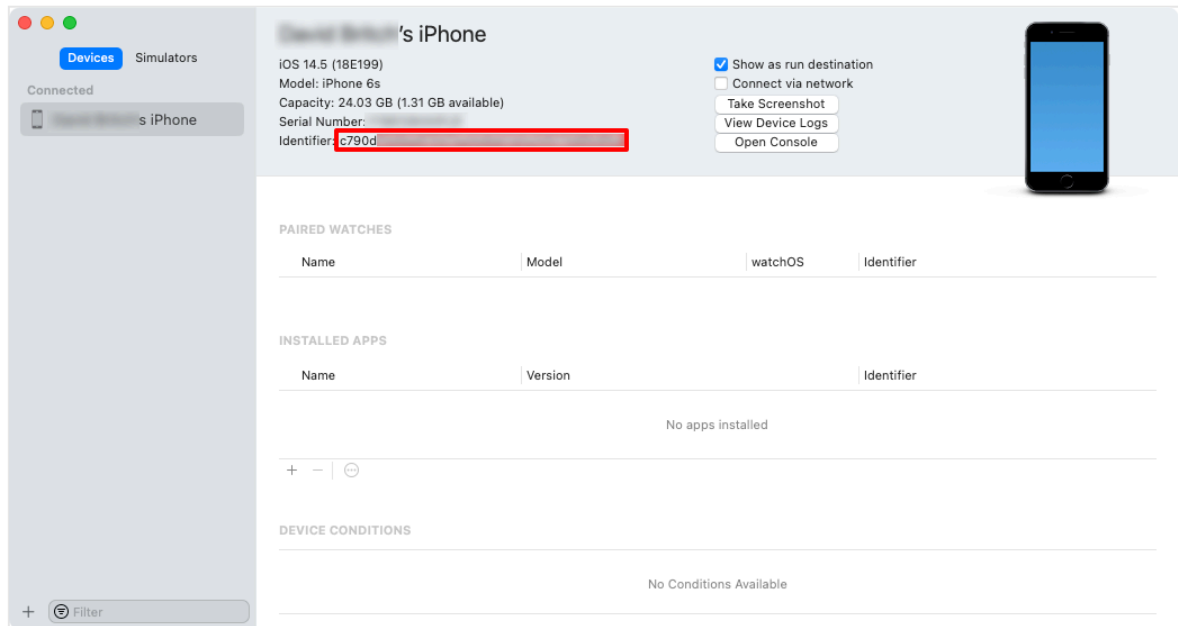


Launch the app on a device

A device must be provisioned before you can deploy an iOS app to it. For more information, see [Device provisioning for iOS](#). Once a device has been provisioned, a .NET MAUI iOS app can be launched on the device from a Mac by providing its unique device id (UDID):

1. Connect your device to your local Mac with a USB cable.
2. Open **Xcode**, and navigate to **Window > Devices and Simulators**.

3. In **Xcode**, select the **Devices** tab, and select the device from the list of connected devices.
4. In **Xcode**, copy the **Identifier** value to the clipboard:



Alternatively, right-click on your device and select **Copy Identifier** to copy the UDID to the clipboard.

5. In **Terminal**, build the app and run it on your chosen device by specifying the `_DeviceName` MSBuild property using the `-p` [MSBuild option](#):

```
zsh
```

```
dotnet build -t:Run -f net8.0-ios -p:RuntimeIdentifier=ios-arm64 -  
p:_DeviceName=MY_SPECIFIC_UDID
```

Replace "MY_SPECIFIC_UDID" with the device identifier you copied to the clipboard.

Pair to Mac for iOS development

Article • 08/30/2024

Building native iOS applications using .NET Multi-platform App UI (.NET MAUI) requires access to Apple's build tools, which only run on a Mac. Because of this, Visual Studio 2022 must connect to a network-accessible Mac to build .NET MAUI iOS apps.

Visual Studio 2022's Pair to Mac feature discovers, connects to, authenticates with, and remembers Mac build hosts so that you can work productively on Windows.

Pair to Mac enables the following software development workflow:

- You can write .NET MAUI iOS code in Visual Studio 2022.
- Visual Studio 2022 opens a network connection to a Mac build host and uses the build tools on that machine to compile and sign the iOS app.
- There's no need to run a separate application on the Mac – Visual Studio 2022 invokes Mac builds securely over SSH.
- Visual Studio 2022 is notified of changes as soon as they happen. For example, when an iOS device is plugged into the Mac or becomes available on the network, the iOS Toolbar updates instantly.
- Multiple instances of Visual Studio 2022 can connect to the Mac simultaneously.
- It's possible to use the Windows command-line to build iOS apps.

ⓘ Note

Before following the instructions in this article, on a Mac, [install Xcode](#). Then manually open Xcode, after installation, so that it can add additional components. In addition, you should also install either the latest [Visual Studio 2022 for Mac](#) or [Mono](#). In addition, if you have a Mac computer with Apple silicon please ensure that Rosetta is installed.

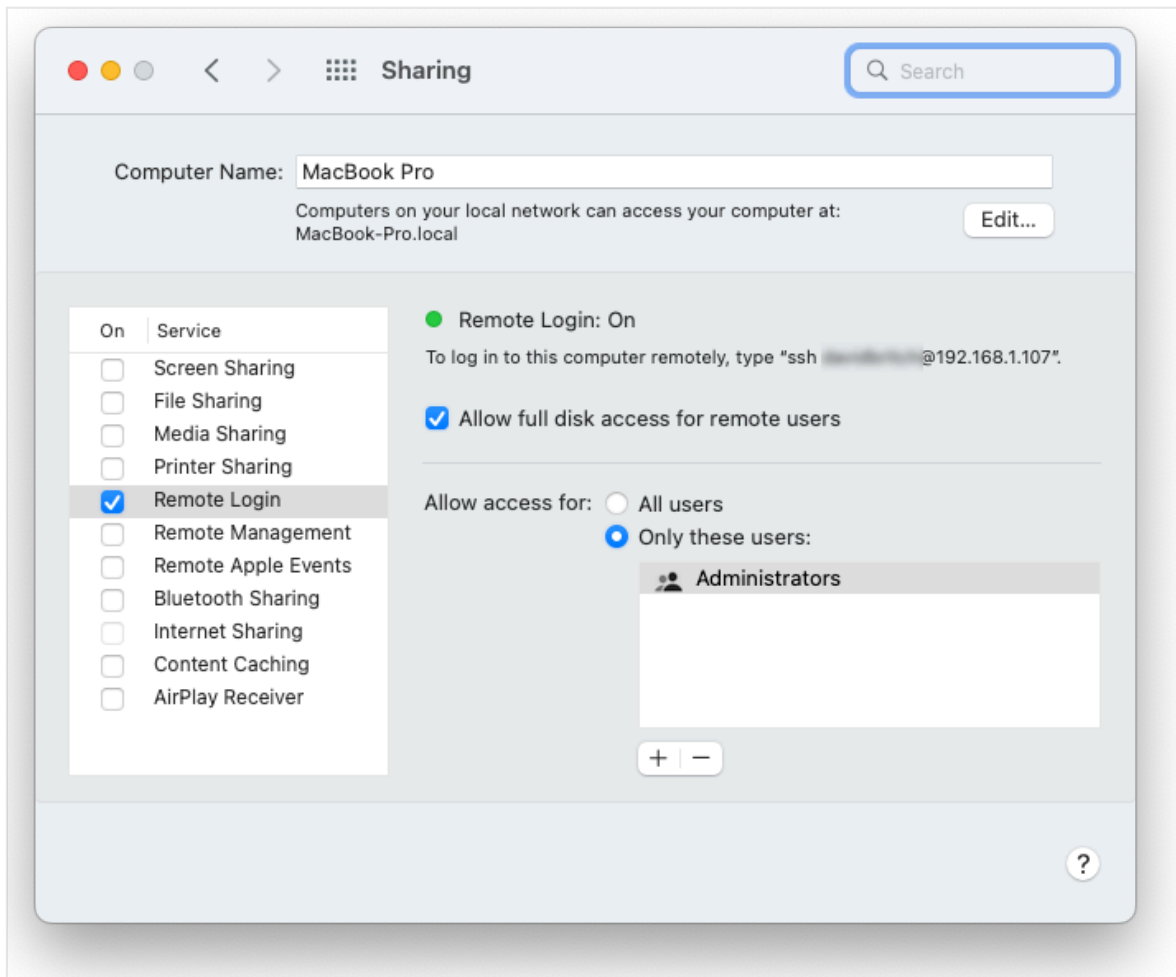
If you would prefer not to install Visual Studio 2022 for Mac, Visual Studio 2022 can automatically configure the Mac build host. However, you must still install and run Xcode, and install Mono.

Enable remote login on the Mac

To set up the Mac build host, first enable remote login:

1. On the Mac, open **System Preferences** and go to the **Sharing** pane.

2. Check **Remote Login** in the **Service** list.



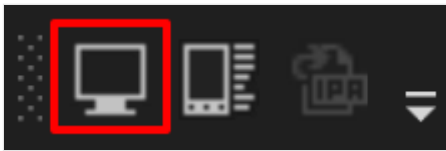
Make sure that it's configured to allow access for **All users**, or that your Mac username or group is included in the list of allowed users.

3. If prompted, configure the macOS firewall. If you have set the macOS firewall to block incoming connections, you may need to allow `mono-sgen` to receive incoming connections. An alert appears to prompt you if so.
4. If it's on the same network as the Windows machine, the Mac should now be discoverable by Visual Studio 2022. If the Mac is still not discoverable, try [manually adding a Mac](#).

Connect to the Mac from Visual Studio 2022

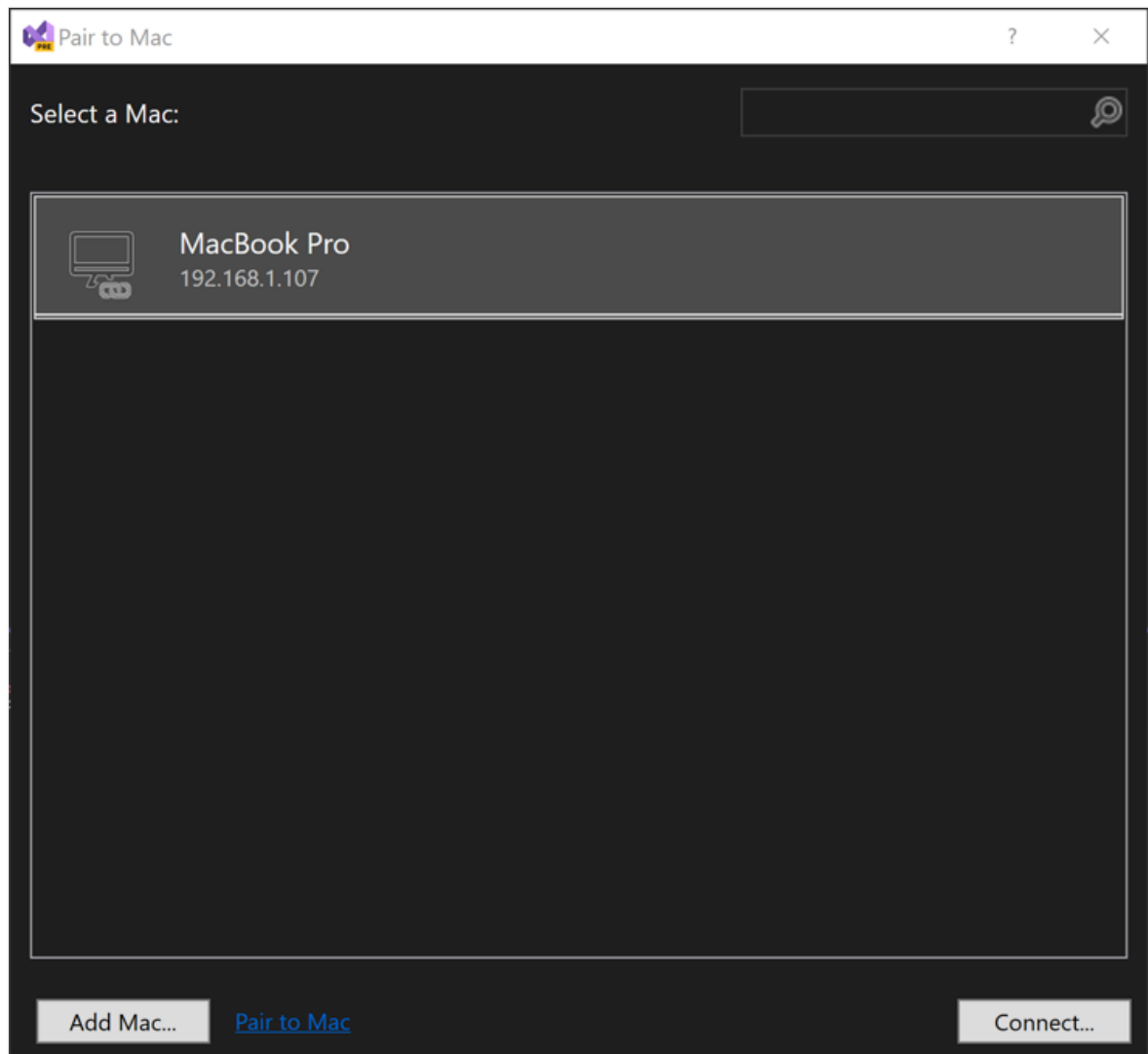
After enabling remote login on the Mac, connect Visual Studio 2022 to the Mac:

1. In Visual Studio 2022, open an existing .NET MAUI project or create a new one.
2. Open the **Pair to Mac** dialog with the **Pair to Mac** button iOS toolbar:

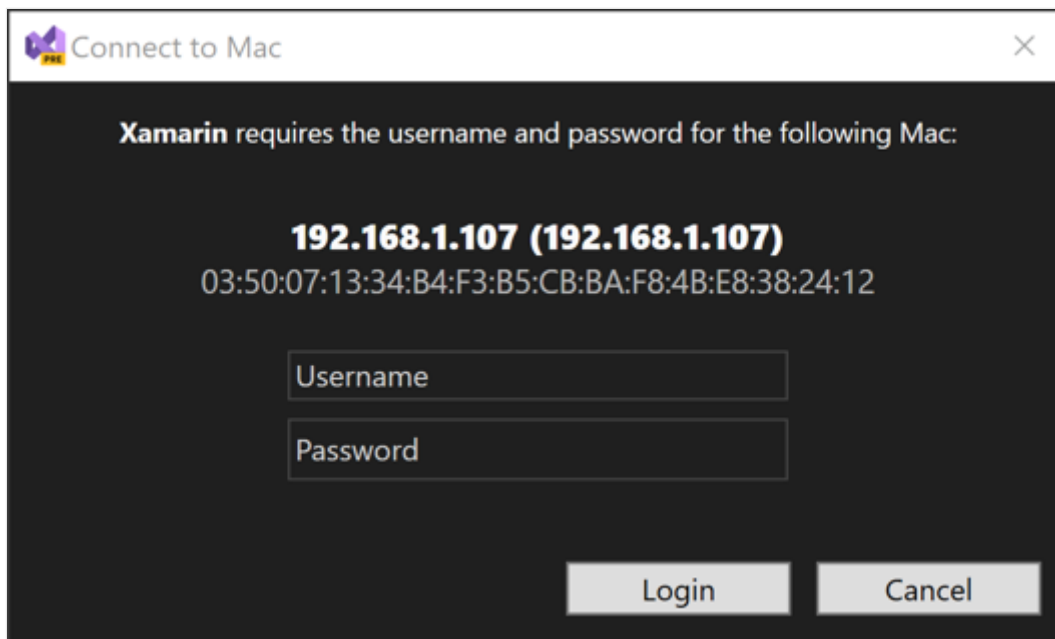


Alternatively, select **Tools > iOS > Pair to Mac**.

The **Pair to Mac** dialog displays a list of all previously connected and currently available Mac build hosts:



3. Select a Mac in the list and select **Connect**.
4. Enter your username and password. The first time you connect to any particular Mac, you're prompted to enter your username and password for that machine:



 Tip

When logging in, use your system username.

Pair to Mac uses these credentials to create a new SSH connection to the Mac. If it succeeds, a key is added to the **authorized_keys** file on the Mac. Subsequent connections to the same Mac will log in automatically.

5. Pair to Mac automatically configures the Mac. Visual Studio 2022 installs or updates pre-requisites on a connected Mac build host as needed. However, Xcode must still be installed manually.
6. Examine the connection status icon. When Visual Studio 2022 is connected to a Mac, that Mac's item in the **Pair to Mac** dialog displays an icon indicating that it's currently connected:



There can be only one connected Mac at a time.

 Tip

Right-clicking any Mac in the **Pair to Mac** list brings up a context menu that allows you to **Connect...**, **Forget this Mac**, or **Disconnect**:



If you choose **Forget this Mac**, your credentials for the selected Mac will be forgotten. To reconnect to that Mac, you will need to re-enter your username and password.

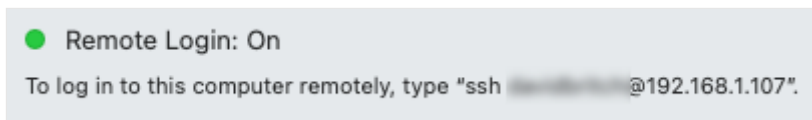
If you've successfully paired to a Mac build host, you're ready to build .NET MAUI iOS apps in Visual Studio 2022. For more information, see [Build your first app](#).

If you haven't been able to pair a Mac, try [manually adding a Mac](#).

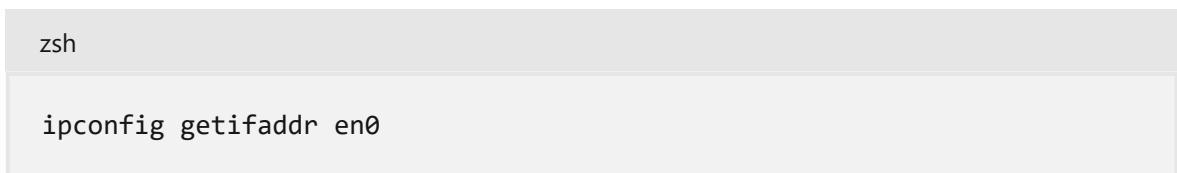
Manually add a Mac

If you don't see a specific Mac listed in the **Pair to Mac** dialog, add it manually:

1. Open **System Preferences > Sharing > Remote Login** on your Mac to locate your Mac's IP address:



Alternatively, use the command line. In **Terminal**, issue the following command:

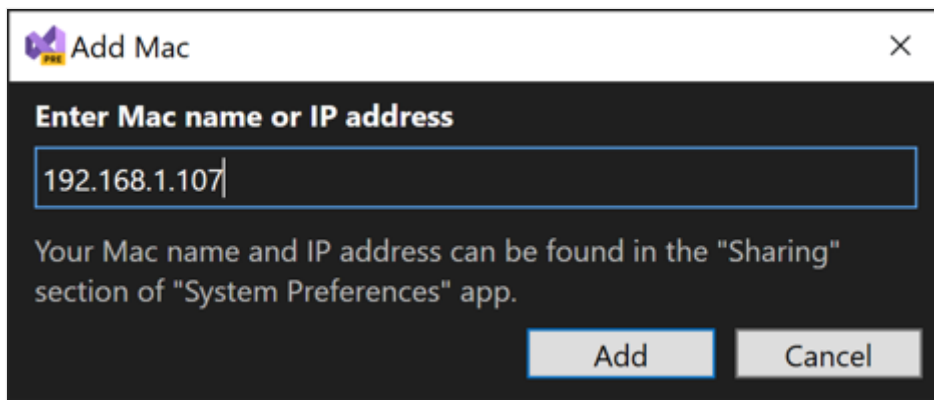


Depending on your network configuration, you may need to use an interface name other than `en0`, for example, `en1` or `en2`.

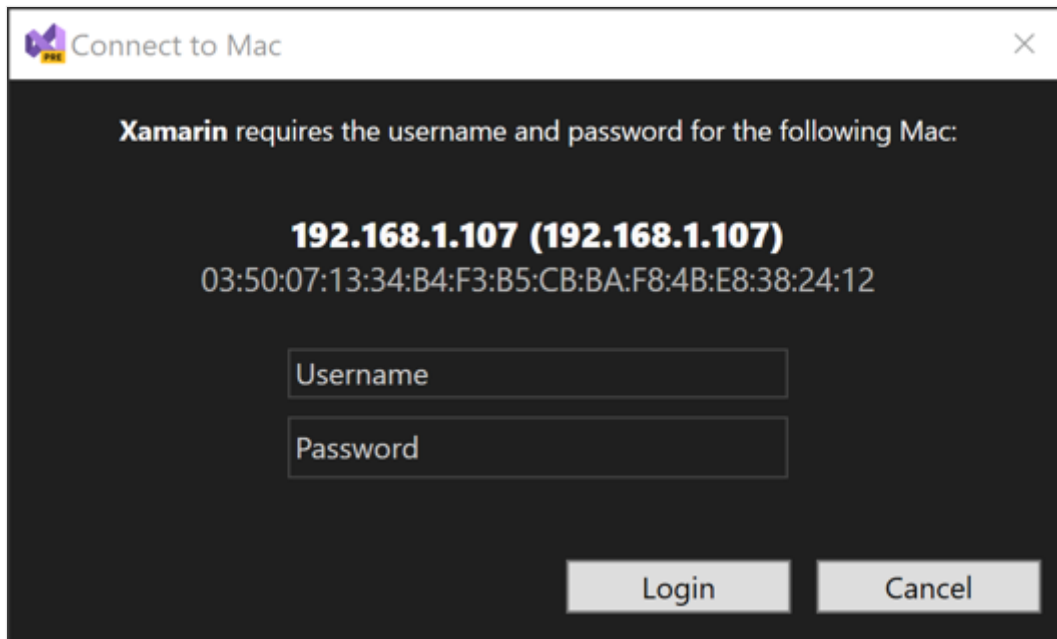
2. In Visual Studio 2022's **Pair to Mac** dialog, select **Add Mac...**:



3. Enter the Mac's IP address and select **Add**:



4. Enter your username and password for the Mac:



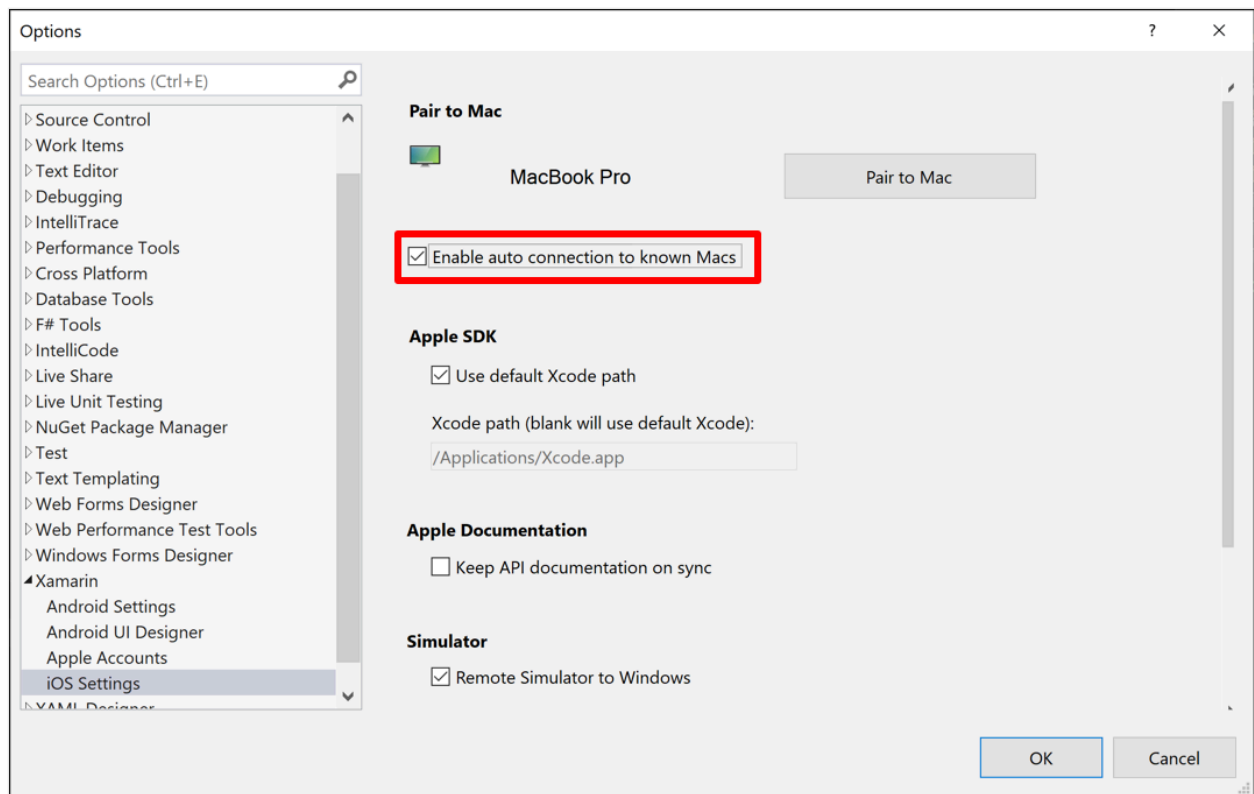
Tip

When logging in, use your system username.

5. Select **Login** to connect Visual Studio 2022 to the Mac over SSH and add it to the list of known machines.

Enable automatic connection to known Macs

By default, a connection to previously paired Macs won't be established when Visual Studio starts. However, automatic connection to known Macs can be enabled in Visual Studio by navigating to **Tools > Options > Xamarin > iOS Settings** and ensuring that **Enable auto connection to known Macs** is checked:



After restarting Visual Studio, it will automatically connect to known Macs on each launch.

Automatic Mac provisioning

Pair to Mac automatically provisions a Mac with the software necessary for building .NET MAUI iOS apps. This includes .NET and various Xcode-related tools (but not Xcode itself).

Important

- Pair to Mac can't install Xcode. You must manually install it on the Mac build host. It's required for .NET MAUI iOS development.
- Automatic Mac provisioning requires that remote login is enabled on the Mac, and the Mac must be network-accessible to the Windows machine.
- Automatic Mac provisioning requires sufficient free space on the Mac to install .NET.

In addition, Pair to Mac performs required software installations and updates to the Mac, when Visual Studio 2022 connects to it.

Xcode tools and license

Pair to Mac will also check to determine whether Xcode has been installed and its license accepted. While Pair to Mac doesn't install Xcode, it does prompt for license acceptance.

In addition, Pair to Mac will install or update various packages distributed with Xcode. The installation of these packages happens quickly and without a prompt.

Troubleshooting automatic Mac provisioning

If you encounter any trouble using automatic Mac provisioning, take a look at the Visual Studio 2022 IDE logs, stored in %LOCALAPPDATA%\Xamarin\Logs\17.0. These logs may contain error messages to help you better diagnose the failure or get support.

Build iOS apps from the Windows command-line

Pair to Mac supports building .NET MAUI apps from the command line. Navigate to the folder that holds the source of your .NET MAUI iOS app and execute the following command:

dotnet

```
dotnet build -f net8.0-ios -p:ServerAddress={macOS build host IP address} -  
p:ServerUser={macOS username} -p:ServerPassword={macOS password} -  
p:TcpPort=58181 -p:_DotNetRootRemoteDirectory=/Users/{macOS  
username}/Library/Caches/Xamarin/XMA/SDKs/dotnet/
```

The parameters passed to `dotnet` in the above example are:

- `ServerAddress` – the IP address of the Mac build host.
- `ServerUser` – the username to use when logging in to the Mac build host. Use your system username rather than your full name.
- `ServerPassword` – the password to use when logging in to the Mac build host.
- `_DotNetRootRemoteDirectory` - the folder on the Mac build host that contains the .NET SDK.

The first time Pair to Mac logs in to a Mac build host from either Visual Studio 2022 or the command-line, it sets up SSH keys. With these keys, future logins won't require a username or password. Newly created keys are stored in %LOCALAPPDATA%\Xamarin\MonoTouch.

If the `ServerPassword` parameter is omitted from a command-line build invocation, Pair to Mac attempts to log in to the Mac build host using the saved SSH keys.

For more information about building iOS apps from the Windows command-line, see [Publish an iOS app using the command line](#).

Remote iOS Simulator for Windows

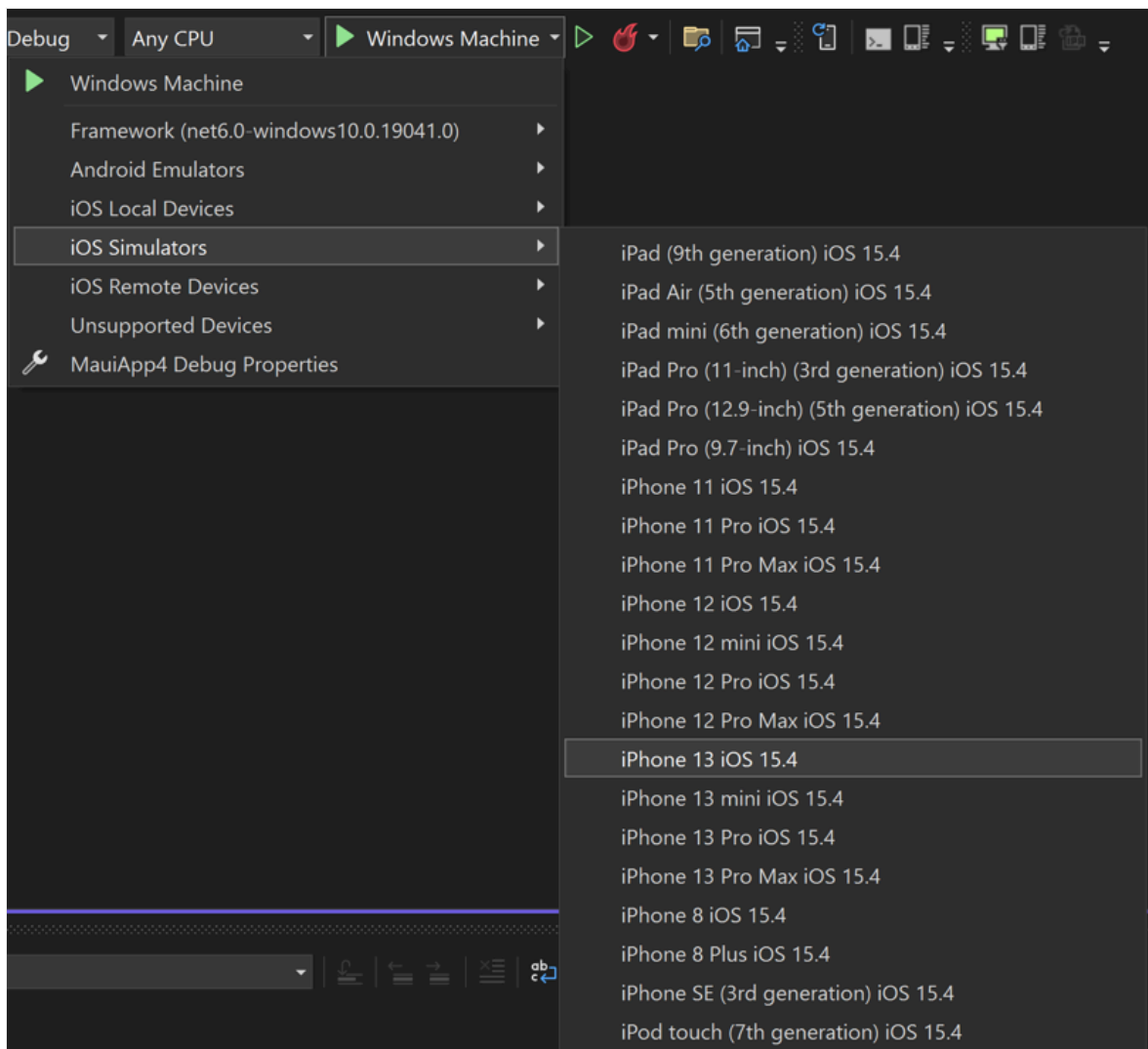
Article • 12/10/2024

The remote iOS Simulator for Windows allows you to test your apps on an iOS simulator displayed in Windows alongside Visual Studio 2022.

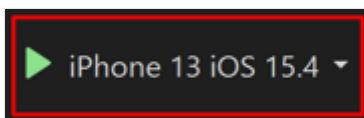
Get started

The remote iOS Simulator for Windows is installed automatically as part of the .NET Multi-platform App UI development workload in Visual Studio 2022. To use it, follow these steps:

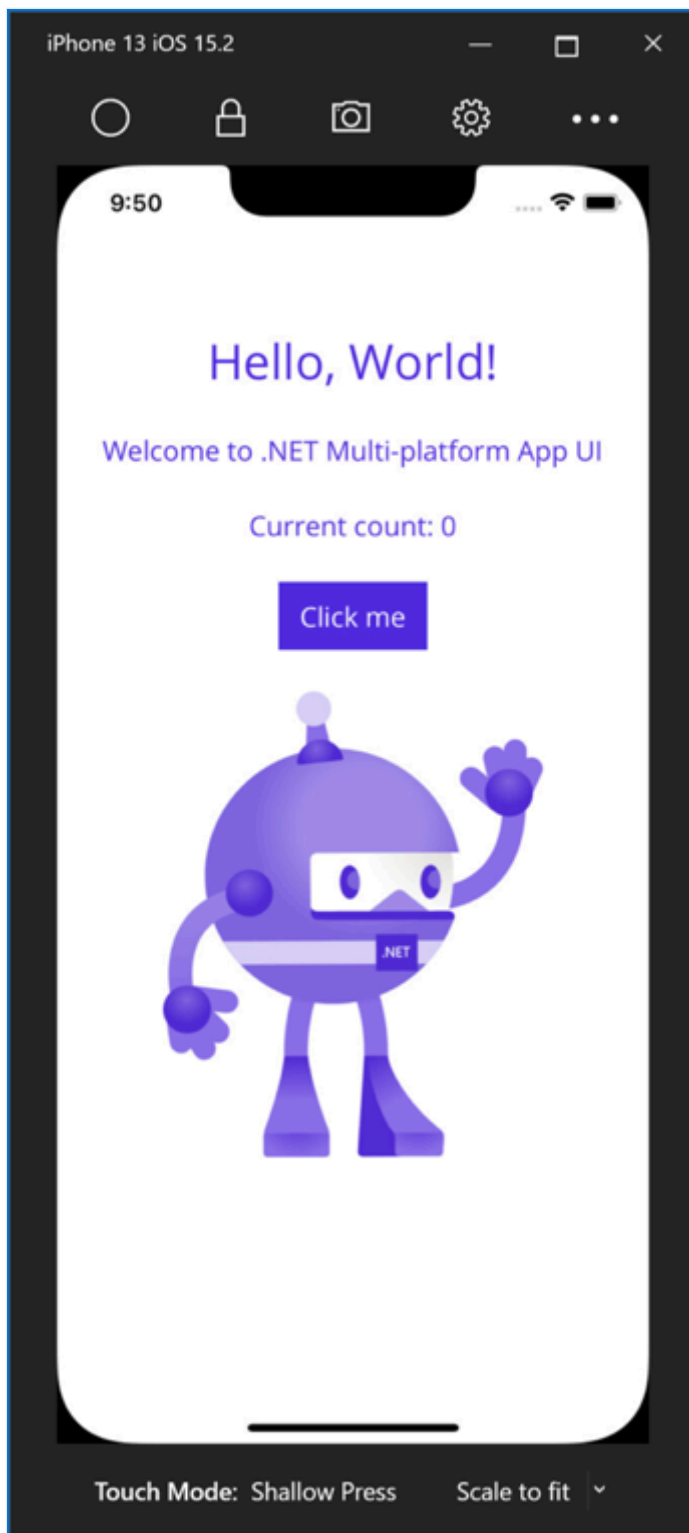
1. Launch Visual Studio 2022 and create or load a .NET MAUI app project.
2. In Visual Studio 2022, pair the IDE to a Mac Build host if you haven't previously. For more information, see [Pair to Mac for iOS development](#).
3. In the Visual Studio toolbar, use the **Debug Target** drop-down to select **iOS Simulators** and then a specific iOS simulator:



4. In the Visual Studio toolbar, press the green Start button for your chosen iOS simulator:

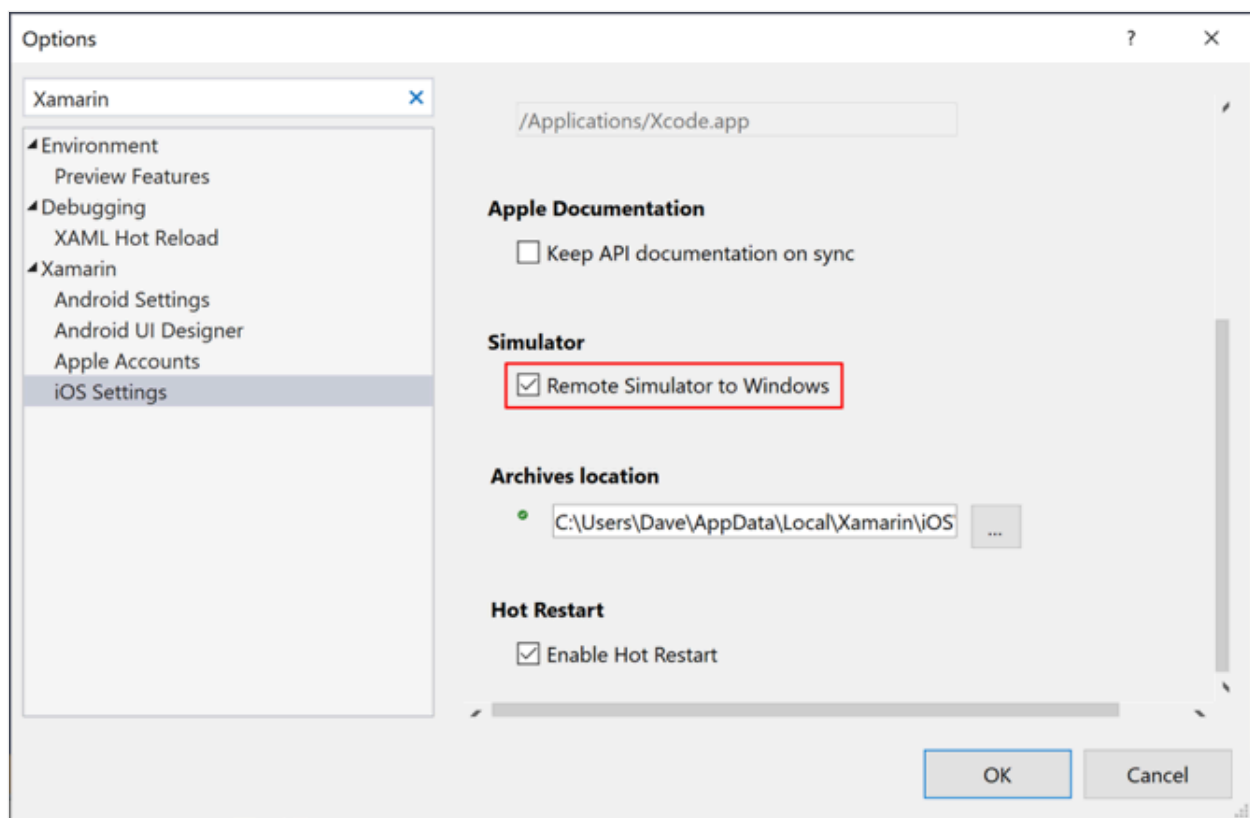


Visual Studio will build the app, start the remote iOS simulator for Windows, and deploy the app to the simulator:



Enable the remote iOS simulator for Windows

The remote iOS simulator for Windows is enabled by default. However, if it's been previously disabled it can be enabled in Visual Studio by navigating to **Tools > Options > Xamarin > iOS Settings** and ensuring that **Remote Simulator to Windows** is checked:

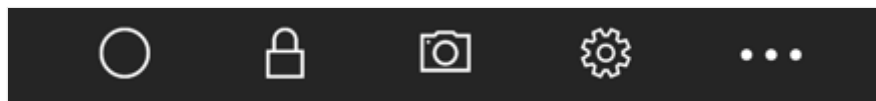


! Note

When the remote simulator is disabled in Visual Studio, debugging a .NET MAUI iOS app will open the iOS Simulator on the connected Mac build host.

Simulator window toolbar

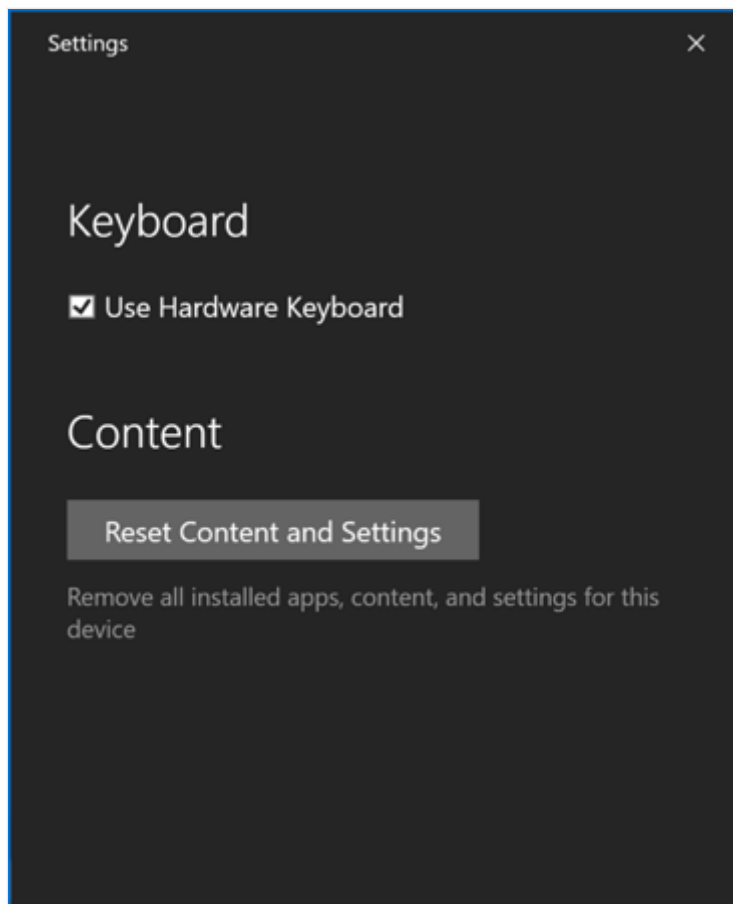
The toolbar at the top of the simulator's window displays five buttons:



The buttons are as follows:

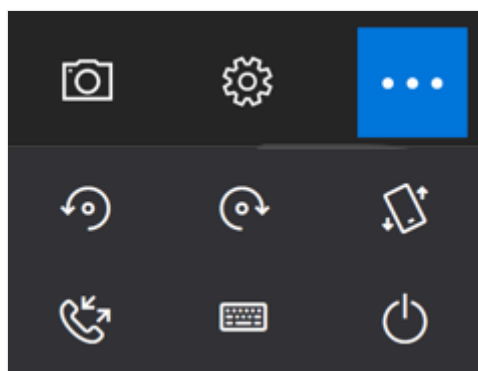
- **Home** – simulates the home button on an iOS device.
- **Lock** – locks the simulator (swipe to unlock).
- **Take Screenshot** – saves a screenshot of the simulator to `\Users\{User}\Pictures\Xamarin\iOS Simulator`.
- **Settings** – displays keyboard and other settings.
- **Other options** – displays various simulator options such as rotation, and shake gesture.

Clicking the toolbar's **Settings** button (the gear icon) opens the **Settings** window:



These settings allow you to enable the hardware keyboard and reset the content and settings for the simulator.

Clicking the toolbar's **Other options** button (the ellipsis icon) reveals additional buttons such as rotation, shake gestures, and rebooting:



ⓘ Note

Right-clicking anywhere in the remote iOS simulator window will display all the toolbar buttons as a context menu.

Touchscreen support

Many Windows computers have touch screens. Since the remote iOS Simulator for Windows supports touch interactions, you can test your app with the same pinch, swipe, and multi-finger touch gestures that you use with physical iOS devices.

Similarly, the remote iOS Simulator for Windows treats Windows Stylus input as Apple Pencil input.

Sound handling

Sounds played by the simulator will come from the host Mac's speakers. iOS sounds are not heard on Windows.

Troubleshooting

In some circumstances, an Xcode configuration problem can result in the remote iOS Simulator for Windows getting stuck in a Connecting to Mac...Checking Server...Connected... loop. When this occurs, you need to remove and reset the Simulators on your Mac build host:

- Ensure that Xamarin Mac Agent (XMA) and Xcode aren't running.
- Delete your `~/Library/Developer/CoreSimulator/Devices` folder.
- Run `killall -9 com.apple.CoreSimulator.CoreSimulatorService`.
- Run `xcrun simctl list devices`.

In some cases, Xcode may have generated simulator devices that can also cause the remote iOS Simulator for Windows to be unable to read its properties correctly as it attempts to verify which ones can be deployed to, causing a boot loop.

You can work around this by removing and creating new simulators which you can deploy to:

- Ensure that Xamarin Mac Agent (XMA) and Xcode aren't running.
- Run `xcrun simctl list | grep -w "Shutdown" | grep -o "([-A-Z0-9]*)" | sed 's/[\\(\\)]//g' | xargs -I uuid xcrun simctl delete uuid`.
- Run `xcrun simctl list devices`. You should see no devices listed.
- Open Xcode on your connected Mac.
- Open **Windows > Devices & Simulators**.
- Select **Simulators**.
- Create a new simulator you wish to deploy to.

Logs

If you experience issues with the remote iOS Simulator, you can view the logs in the following locations:

- **Mac** – `~/Library/Logs/Xamarin/Simulator.Server`
- **Windows** – `%LOCALAPPDATA%\Xamarin\Logs\Xamarin.Simulator`

Build a Mac Catalyst app with .NET CLI

Article • 03/19/2024

In this tutorial, you'll learn how to create and run a .NET Multi-platform App UI (.NET MAUI) app on Mac Catalyst using .NET Command Line Interface (CLI) on macOS:

1. To create .NET MAUI apps, you'll need to download and run the [installer](#) for the latest .NET runtime. You'll also need to download and install the version of Xcode that's required by the version of .NET MAUI that you're using. For information, see [Release versions](#).
2. On your Mac, open **Terminal** and check that you have the latest .NET runtime installed:

```
zsh  
  
dotnet --version
```

3. In **Terminal**, install the latest public build of .NET MAUI:

```
zsh  
  
sudo dotnet workload install maui --source  
https://api.nuget.org/v3/index.json
```

This command will install the latest released version of .NET MAUI, including the required platform SDKs.

4. In **Terminal**, create a new .NET MAUI app using .NET CLI:

```
zsh  
  
dotnet new maui -n "MyMauiApp"
```

5. In **Terminal**, change directory to *MyMauiApp*, and build and run the app:

```
zsh  
  
cd MyMauiApp  
dotnet build -t:Run -f net8.0-maccatalyst
```

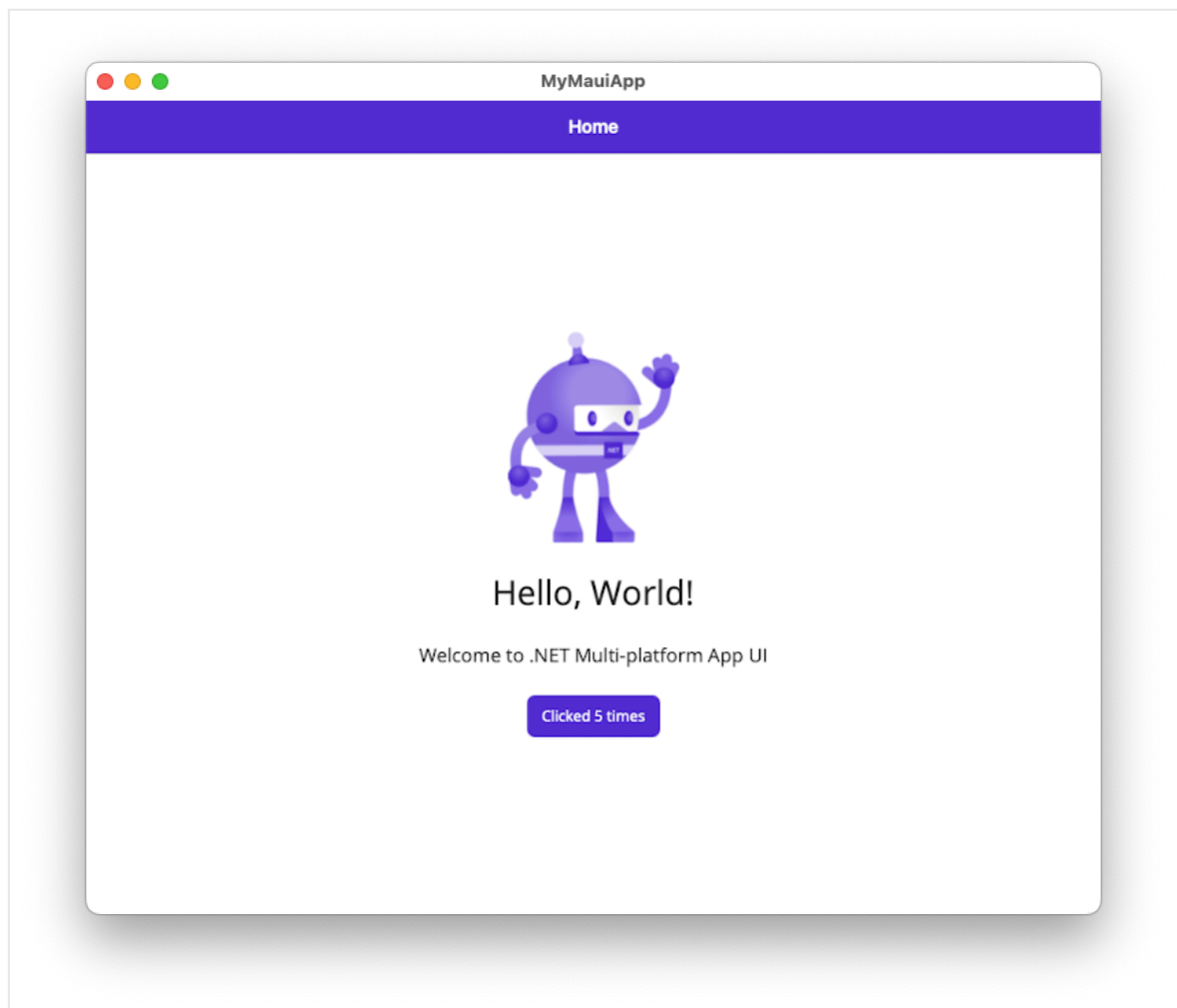
The `dotnet build` command will restore the project dependencies, build the app, and launch it.

If you see a build error and a warning that the Xcode app bundle could not be found, you may need to run the following command:

```
zsh
```

```
xcode-select --reset
```

6. In the running app, press the **Click me** button several times and observe that the count of the number of button clicks is incremented.



Build with a specific version of Xcode

If you have multiple versions of Xcode installed on your Mac, it's possible to specify which Xcode version should be used when building your app. There are a number of approaches that can be used to accomplish this, but the recommended approach is to set the `MD_APPLE_SDK_ROOT` environment variable to the path of the Xcode version.

 **Warning**

Using `xcode-select -s` to set the version of Xcode to use isn't recommended.

To set the `MD_APPLE_SDK_ROOT` environment variable for the duration of the current terminal session:

1. Open the **Terminal** application.
2. Type the following command, substituting in your version of Xcode, and press Enter:

```
zsh
```

```
export MD_APPLE_SDK_ROOT=/Applications/Xcode_14.1.0.app
```

If you want to set this environment variable permanently, you can add the `export` command to your shell profile, such as `.zprofile`.

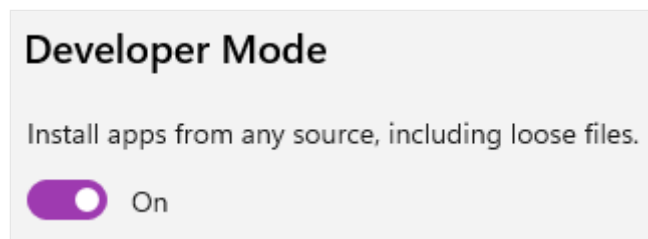
Deploy and debug your .NET MAUI app on Windows

Article • 11/14/2023

You can use your local Windows development computer to deploy and debug a .NET Multi-platform App UI (.NET MAUI) app. This article describes how to configure Windows to debug a .NET MAUI app.

Configure Windows

You must enable Developer Mode in Windows. Both Windows 10 and Windows 11 are supported.



Windows 11

Developer Mode is enabled in **Settings** app, under **Privacy & security > For developers**. To enable Developer Mode in Windows 11:

1. Open the Start Menu.
2. Type **Developer settings** in the search box and select it.
3. Turn on **Developer Mode**.
4. If you receive a warning message about Developer Mode, read it, and select **Yes** if you understand the warning.

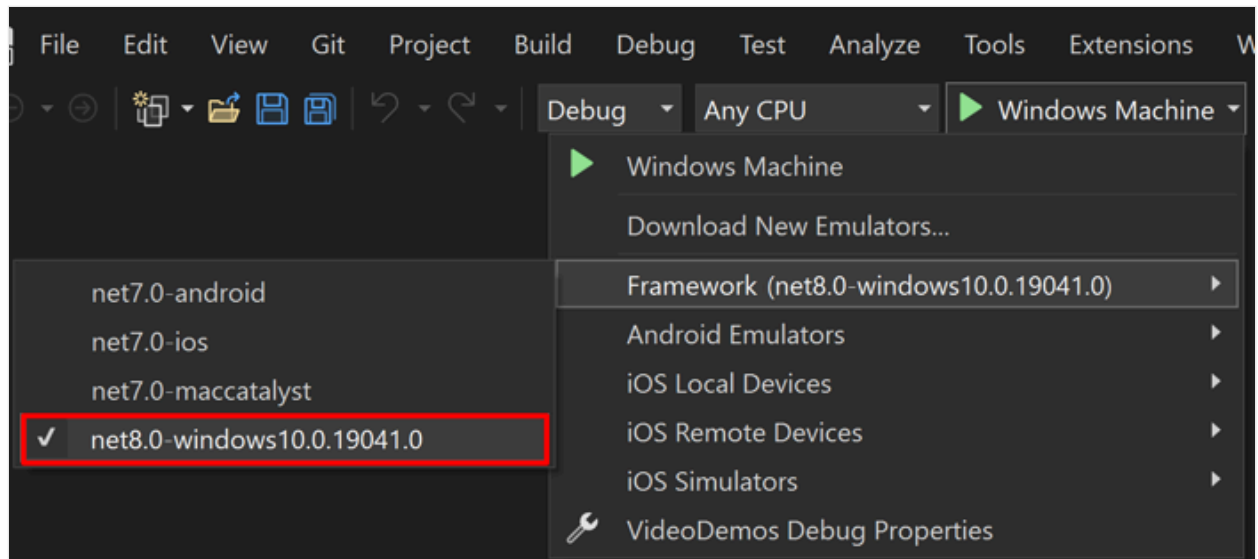
Windows 10

Developer Mode is enabled in **Settings** app, under **Update & Security > For developers**. To enable Developer Mode in Windows 10:

1. Open the Start Menu.
2. Search for **Developer settings**, select it.
3. Turn on **Developer Mode**.
4. If you receive a warning message about Developer Mode, read it, and select **Yes** if you understand the warning.

Target Windows

In Visual Studio, set the **Debug Target** to **Framework (...)** > **net8.0-windows**. There is a version number in the item entry, which may or may not match the following screenshot:



Upgrade from Xamarin to .NET

Article • 08/30/2024

 Important

[Microsoft support for Xamarin](#)  will end on May 1, 2024 for all Xamarin SDKs including Xamarin.Forms. Help us improve your upgrade experience from Xamarin to .NET MAUI by completing this [short survey](#) .

Xamarin projects can run on .NET after completing an upgrade process. The following table lists the Xamarin project types that can be upgraded to .NET:

 **Expand table**

Project type	Upgrade	Guide
Xamarin.Android		Upgrade Xamarin native projects
Xamarin.iOS		Upgrade Xamarin native projects
Xamarin.Mac		Upgrade Xamarin native projects
Xamarin.tvOS		Upgrade Xamarin native projects
Xamarin.Forms		Upgrade a Xamarin.Forms app to a multi-project .NET MAUI app Upgrade a Xamarin.Forms app to a single project .NET MAUI app
Xamarin.Forms UWP		Xamarin.Forms UWP project migration
iOS App Extensions		Upgrade Xamarin native projects
Android Wear		Upgrade Xamarin native projects
Android Binding Library		Xamarin.Android binding project migration
iOS Binding Library		Xamarin.iOS binding project migration
SpriteKit		Upgrade Xamarin native projects
SceneKit		Upgrade Xamarin native projects
Metal		Upgrade Xamarin native projects
OpenGL	 (iOS)	Removed from iOS since OpenTK isn't available

Project type	Upgrade	Guide
Xamarin.watchOS	✗	Recommendation: bundle Swift extensions with .NET for iOS apps

Important

To upgrade an app from Xamarin to .NET:

- All projects **do** need to become SDK-style.
- Projects **don't** need to be rewritten.
- Multi-project solutions **don't** need to become a multi-targeted single project.

To upgrade your Xamarin native projects to .NET, you'll first have to update the projects to be SDK-style projects and then update your dependencies to .NET 8. For more information, see [Upgrade Xamarin.Android, Xamarin.iOS, and Xamarin.Mac projects to .NET](#).

The .NET Upgrade Assistant is a command-line tool that can help you upgrade multi-project Xamarin.Forms apps to multi-project .NET Multi-platform App UI (.NET MAUI) apps. After running the tool, in most cases the app will require additional effort to complete the upgrade. For more information, see [Upgrade a Xamarin.Forms app to a .NET MAUI app with the .NET Upgrade Assistant](#).

You can also manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app with a two-step process:

1. Upgrade your Xamarin native projects, in your Xamarin.Forms solution, to .NET. For more information, see [Upgrade Xamarin.Android, Xamarin.iOS, and Xamarin.Mac apps to .NET](#). In addition, you can upgrade your Xamarin.Forms UWP project to a .NET MAUI WinUI 3 project. For more information, see [Xamarin.Forms UWP project migration](#).
2. Upgrade your Xamarin.Forms library project to .NET Multi-platform App UI (.NET MAUI). For more information, see [Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app](#).

Alternatively, you can manually upgrade a Xamarin.Forms app to a single-project .NET MAUI app. For more information, see [Manually upgrade a Xamarin.Forms app to a single project .NET MAUI app](#).

Upgrade Xamarin.Android, Xamarin.iOS, and Xamarin.Mac projects to .NET

Article • 05/07/2024

To upgrade your Xamarin native projects to .NET, you must:

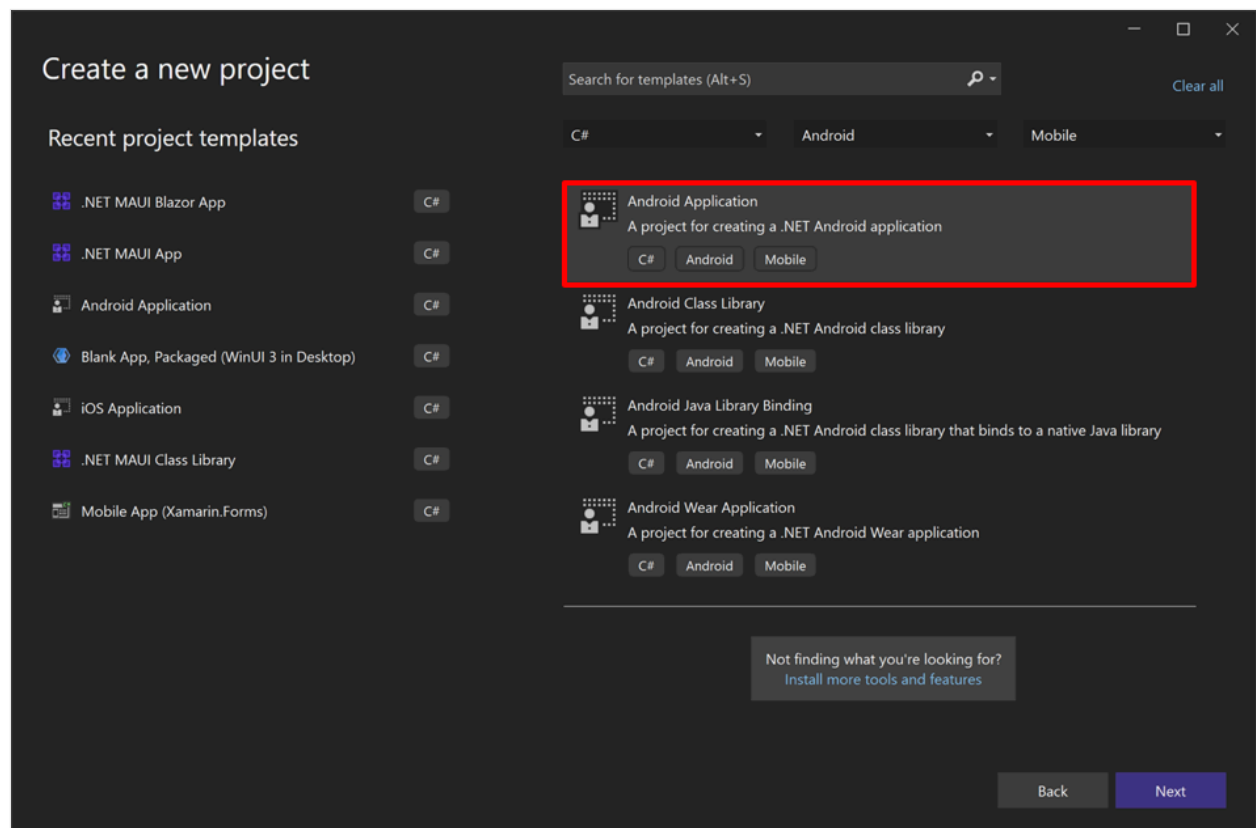
- ✓ Update your project file to be SDK-style.
- ✓ Update or replace incompatible dependencies with .NET 8 versions.
- ✓ Compile and test your app.

For most projects you won't need to change namespaces or undertake other rewrites.

To simplify the upgrade process, we recommend creating a new .NET project of the same type and name as your Xamarin native project, and then copy in your code. This is the approach outlined below.

Create a new project

In Visual Studio, create a new .NET project of the same type and name as your Xamarin native project. For example, to upgrade from Xamarin.Android to .NET for Android select the **Android Application** project template:



The new project should be given the same project and package name as your existing project, and should be placed in a new folder. Opening the project file will confirm that you have a .NET SDK-style project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    <SupportedOSPlatformVersion>21</SupportedOSPlatformVersion>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <ApplicationId>com.companyname.AndroidApp2</ApplicationId>
    <ApplicationVersion>1</ApplicationVersion>
    <ApplicationDisplayVersion>1.0</ApplicationDisplayVersion>
  </PropertyGroup>
</Project>
```

Important

The target framework moniker (TFM) is what denotes the project as using .NET, in this case .NET 8. Valid TFMs for equivalent Xamarin native projects are net8.0-android, net8.0-ios, net8.0-macos, and net8.0-tvos. For information about target frameworks in SDK-style projects, see [Target frameworks in SDK-style projects](#).

Launch the app to confirm that your development environment can build the app.

Merge files

Copy your code and resource files from the folders of your Xamarin native project to identical folders within your new app. You should overwrite any files of the same name.

If you have other library projects, you should add them to your new solution and [add project references](#) to them from your new .NET project.

You'll also need to copy some project properties from your Xamarin native project to your new .NET project, for settings like conditional compilation arguments and code signing. Opening the projects side-by-side in separate Visual Studio instances will enable you to compare the project properties. Alternatively, you can migrate the settings by editing the new project file directly. For more information, see [Xamarin.Android project migration](#) and [Xamarin Apple project migration](#).

Update dependencies

Generally, Xamarin native NuGet packages are not compatible with .NET 8 unless they have been recompiled using .NET TFMs. However, .NET for Android apps can use NuGet packages targeting the `monoandroid` and `monoandroidXX.X` frameworks.

You can confirm a package is .NET 8 compatible by looking at the **Frameworks** tab on [NuGet](#) for the package you're using, and checking that it lists one of the compatible frameworks shown in the following table:

 Expand table

Compatible frameworks	Incompatible frameworks
net8.0-android, monoandroid, monoandroidXX.X	
net8.0-ios	monotouch, xamarinios, xamarinios10
net8.0-macos	monomac, xamarinmac, xamarinmac20
net8.0-tvos	xamarintvos
	xamarinwatchos

 Note

.NET Standard libraries that have no dependencies on the incompatible frameworks listed above are still compatible with .NET 8.

If a package on [NuGet](#) indicates compatibility with any of the compatible frameworks above, regardless of also including incompatible frameworks, then the package is compatible. Compatible NuGet packages can be added to your .NET native project using the NuGet package manager in Visual Studio.

If you can't find a .NET 8 compatible version of a NuGet package you should:

- Recompile the package with .NET TFMs, if you own the code.
- Look for a preview release of a .NET 8 version of the package.
- Replace the dependency with a .NET 8 compatible alternative.

For information about migrating Xamarin.Essentials code in a .NET for Android or .NET for iOS app, see [Migrate Xamarin.Essentials code in .NET for Android and .NET for iOS apps](#).

Compile and troubleshoot

Once your dependencies are resolved and your code and resource files are added to your .NET native project, you should build your project. Any errors will guide you towards next steps.

Tip

- Delete all *bin* and *obj* folders from all projects before opening and building projects in Visual Studio, particularly when changing .NET versions.
- Delete the *Resource.designer.cs* generated file from the Android project.

Xamarin.Android project migration

Article • 06/19/2024

A .NET 8 project for a .NET for Android app is similar to the following example:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    <OutputType>Exe</OutputType>
  </PropertyGroup>
</Project>
```

For a library project, omit the `$(OutputType)` property completely or specify `Library` as the property value.

.NET configuration files

There's no support for [configuration files](#) such as `Foo.dll.config` or `Foo.exe.config` in .NET for Android projects. `<dllmap>` [↗](#) configuration elements aren't supported in .NET Core at all, and other element types for compatibility packages like [System.Configuration.ConfigurationManager](#) [↗](#) have never been supported in Android projects.

Changes to MSBuild properties

The `$(AndroidSupportedAbis)` property shouldn't be used:

XML

```
<PropertyGroup>
  <!-- Used in Xamarin.Android projects -->
  <AndroidSupportedAbis>armeabi-v7a;arm64-
v8a;x86;x86_64</AndroidSupportedAbis>
</PropertyGroup>
```

Instead, the `$(AndroidSupportedAbis)` property should be replaced with .NET runtime identifiers:

XML

```
<PropertyGroup>
  <!-- Used in .NET for Android projects -->
  <RuntimeIdentifiers>android-arm;android-arm64;android-x86;android-
x64</RuntimeIdentifiers>
</PropertyGroup>
```

For more information about runtime identifiers, see [.NET RID Catalog](#).

The following table shows other MSBuild properties that have changed in .NET for Android:

 Expand table

Property	Comments
<code>\$(AndroidUseIntermediateDesignerFile)</code>	<code>True</code> by default.
<code>\$(AndroidBoundExceptionType)</code>	<code>System</code> by default. This property alters the types of exceptions thrown from various methods to better align with .NET semantics, at the cost of compatibility with Xamarin.Android. For more information, see Some of the new wrapped Java exceptions use BCL exceptions that differ from the related BCL types .
<code>\$(AndroidClassParser)</code>	<code>class-parse</code> by default. <code>jar2xml</code> isn't supported.
<code>\$(AndroidDexTool)</code>	<code>d8</code> by default. <code>dx</code> isn't supported.
<code>\$(AndroidCodegenTarget)</code>	<code>XAJavaInterop1</code> by default. <code>XamarinAndroid</code> isn't supported.
<code>\$(AndroidManifest)</code>	Defaults to <code>AndroidManifest.xml</code> in the root of projects because <code>Properties\AssemblyInfo.cs</code> is no longer used in SDK-style projects. <code>Properties\AndroidManifest.xml</code> will also be detected and used if it exists, to ease migration.
<code>\$(DebugType)</code>	<code>portable</code> by default. <code>full</code> and <code>pdbonly</code> aren't supported.
<code>\$(MonoSymbolArchive)</code>	<code>False</code> , since <code>mono-symbolicate</code> isn't supported.

In addition, if Java binding is enabled with `@(InputJar)`, `@(EmbeddedJar)`, or `@(LibraryProjectZip)`, then the `$(AllowUnsafeBlocks)` property defaults to `True`.

Note

Referencing an Android Wear project from an Android app isn't supported.

Changes to AndroidManifest.xml

In Xamarin.Android, Java, and Kotlin Android projects, the `<uses-sdk/>` element denotes the minimum Android version your app supports, as well as the target Android version your app is compiled against:

XML

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    package="com.companyname.myapp">
    <uses-sdk android:minSdkVersion="21" android:targetSdkVersion="33" />
    <application android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:theme="@style/AppTheme" />
</manifest>
```

For more information about the `<uses-sdk/>` element, see the [Android documentation](#).

In .NET 8 Android apps, there are MSBuild properties to set these values. Using the MSBuild properties has other benefits. In most cases the `<uses-sdk/>` element should be removed in favor of values in your project's `.csproj` file:

XML

```
<Project>
  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    <SupportedOSPlatformVersion>21</SupportedOSPlatformVersion>
  </PropertyGroup>
</Project>
```

In this example, `net8.0-android` is shorthand for `net8.0-android34.0`. Future versions of .NET will track the latest Android version available at the time of the .NET release.

`TargetFramework` maps to `android:targetSdkVersion`. At build time, this value will automatically be included in the `<uses-sdk/>` element for you. The benefit of using `TargetFramework` in this way is that you're given the matching C# binding for Android API 34 for `net8.0-android34.0`. Android releases independently of the .NET release cycle,

so we have the flexibility to opt into `net8.0-android35.0` when a binding is available for the next Android release.

Similarly, `SupportedOSPlatformVersion` maps to `android:minSdkVersion`. At build time, this value will automatically be included in the `<uses-sdk/>` element for you. Android APIs are decorated with the [SupportedOSPlatformAttribute](#) so that you get build warnings for calling APIs that are only available for some of the Android versions your app can run on:

```
error CA1416: This call site is reachable on 'Android' 21.0 and later.  
`ConnectivityManager.ActiveNetwork` is only supported on: 'Android' 23.0 and  
later.
```

To safely use this API, you can declare a higher `SupportedOSPlatformVersion` in your project or use the [IsAndroidVersionAtLeast](#) API at runtime:

```
C#  
  
if (OperatingSystem.IsAndroidVersionAtLeast(23))  
{  
    // Use the API here  
}
```

Default file inclusion

Default .NET for Android related file globbing behavior is defined in `AutoImport.props`. This behavior can be disabled for Android items by setting `$(EnableDefaultAndroidItems)` to `false`, or all default item inclusion behavior can be disabled by setting `$(EnableDefaultItems)` to `false`. For more information, see [Workload props files](#).

Runtime behavior

There are behavioral changes to the `String.IndexOf()` method in .NET 5+ on different platforms. For more information, see [.NET globalization and ICU](#).

Linker

.NET 9 has new settings for the linker:

- `<TrimMode>Full</TrimMode>`, which performs full trimming.

For more information, see [Trimming options](#).

In .NET for Android projects by default, `Debug` builds don't use the linker, and `Release` builds set `PublishTrimmed=true` and `TrimMode=partial`.

Ahead-of-Time compilation

`$(RunAOTCompilation)` is the new MSBuild property for enabling Ahead-of-Time (AOT) compilation. This is the same property used for [Blazor WASM](#). The `$(AotAssemblies)` property also enables AOT, in order to help with migration from Xamarin.Android projects to .NET for Android projects. However, this property was deprecated in .NET 7.

Release builds default to the following AOT property values:

XML

```
<PropertyGroup Condition="'$(Configuration)' == 'Release'">
  <RunAOTCompilation>true</RunAOTCompilation>
  <AndroidEnableProfiledAot>true</AndroidEnableProfiledAot>
</PropertyGroup>
```

This is the behavior when the `$(RunAOTCompilation)` and `$(AndroidEnableProfiledAot)` properties are unset, and chooses the optimal settings for startup time and app size.

To disable AOT, you need to explicitly set the `$(RunAOTCompilation)` and `$(AndroidEnableProfiledAot)` properties to `false`:

XML

```
<PropertyGroup Condition="'$(Configuration)' == 'Release'">
  <RunAOTCompilation>>false</RunAOTCompilation>
  <AndroidEnableProfiledAot>>false</AndroidEnableProfiledAot>
</PropertyGroup>
```

Supported encodings

If your Xamarin.Android app uses certain international codesets, they have to be specified explicitly in your project file using the `Mandroid118n` MSBuild property, so that the linker can include supporting resources. For more information about this build property, see [MAAndroid118n](#).

However, the `Mandroid118n` MSBuild property isn't supported in .NET for Android apps. Instead, support is provided by the [System.Text.Encoding.CodePages](#) NuGet package. For more information, see [CodePagesEncodingProvider](#).

.NET CLI

.NET for Android supports using .NET command-line interface (.NET CLI) to create, build, publish, and run Android apps.

dotnet new

`dotnet new` can be used to create new .NET for Android projects and items using project templates and item templates that are named following the patterns and naming of existing .NET templates:

 Expand table

Template	Short Name	Language	Tags
Android Activity template	android-activity	C#	Android
Android Java Library Binding	android-bindinglib	C#	Android
Android Layout template	android-layout	C#	Android
Android Class library	androidlib	C#	Android
Android Application	android	C#	Android

The following examples show using `dotnet new` to create different types of .NET for Android projects:

```
.NET CLI

dotnet new android --output MyAndroidApp --packageName com.mycompany.myandroidapp
dotnet new androidlib --output MyAndroidLibrary
dotnet new android-bindinglib --output MyJavaBinding
```

Once .NET for Android projects have been created, item templates can be used to add items to the projects:

```
.NET CLI
```

```
dotnet new android-activity --name LoginActivity --namespace MyAndroidApp
dotnet new android-layout --name MyLayout --output Resources/layout
```

dotnet build & publish

For .NET for Android, `dotnet build` produces a runnable app. This means creating an `.apk` or `.aab` file during the build process, and reordering MSBuild tasks from the .NET SDK so that they run during the build. Therefore, .NET for Android does the following during a build:

- Run `aapt` to generate `Resource.designer.cs` and potentially emit build errors for issues in `@(AndroidResource)` files.
- Compile C# code.
- Run the [ILLink](#) [↗](#) MSBuild target for linking.
- Generate java stubs, and `AndroidManifest.xml`.
- Compile java code via `javac`.
- Convert java code to `.dex` via `d8/r8`.
- Create an `.apk` or `.aab` and sign it.

`dotnet publish` is reserved for publishing an app for Google Play and other distribution mechanisms such as ad-hoc. It also signs the `.apk` or `.aab` with different keys.

ⓘ Note

Behavior inside IDEs will differ. The `Build` target will not produce an `.apk` file if `$(BuildingInsideVisualStudio)` is `true`. IDEs will call the `Install` target for deployment, which will produce the `.apk` file. This behavior matches Xamarin.Android.

dotnet run

`dotnet run` can be used to launch apps on a device or emulator via the `--project` argument:

.NET CLI

```
dotnet run --project HelloAndroid.csproj
```

Alternatively, you could use the `Run` MSBuild target:

.NET CLI

```
dotnet build HelloAndroid.csproj -t:Run
```

See also

- [Binding projects](#) 

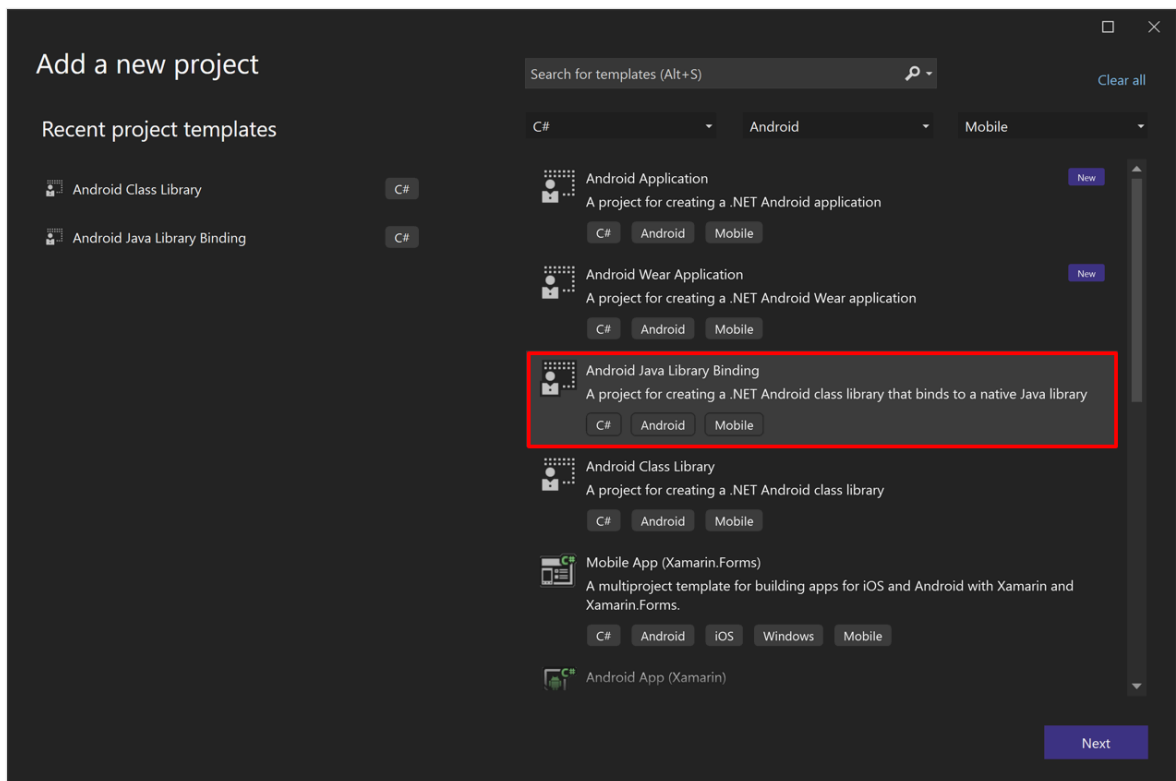
Xamarin.Android binding project migration

Article • 05/07/2024

In .NET, there's no concept of an Android binding project as a separate project type. Any of the MSBuild item groups or build actions that work in Xamarin.Android binding projects are supported through a .NET for Android app or library.

To migrate a Xamarin.Android binding library to a .NET for Android class library:

1. In Visual Studio, create a new Android Java Library Binding project with the same name as your Xamarin.Android binding project:



Opening the project file will confirm that you have a .NET SDK-style project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    <SupportedOSPlatformVersion>21</SupportedOSPlatformVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

⚠ Note

The project file for an Android binding library is identical to the project file for an Android class library.

2. Add your Java Archive (JAR) or Android Archive (AAR) to the project and ensure that its build action is set to **AndroidLibrary**.
3. Copy any transforms or additions from your Xamarin.Android bindings library.

Unsupported legacy options

The following legacy options are no longer supported. The supported alternatives have been available for several years, and the smoothest migration option is to update and test your current projects with these options before migrating them to .NET.

AndroidClassParser

`jar2xml` is no longer a valid option for the `$(AndroidClassParser)` property. `class-parse` is now the default and only supported option.

`class-parse` takes advantage of many new modern features not available in `jar2xml`, such as:

- Automatic parameter names for class methods (if your Java code is compiled with `javac -parameters`).
- Kotlin support.
- Static/default interface member (DIM) support.
- Java Nullable reference type (NRT) annotations support.

AndroidCodegenTarget

`XamarinAndroid` is no longer a valid option for the `$(AndroidCodegenTarget)` property. `XAJavaInterop1` is now the default and only supported option.

If you have hand-bound code in your `Additions` files that interacts with the generated binding code, it may need to be updated to be compatible with `XAJavaInterop1`.

Default file inclusion

Given the following file structure:

```
Transforms/  
    Metadata.xml  
foo.jar
```

`Transforms\*.xml` files are automatically included as a `@(TransformFile)` item, and `.jar/.aar` files are automatically included as a `@(AndroidLibrary)` item. This will bind C# types for the Java types found in `foo.jar` using the metadata fixups from `Transforms\Metadata.xml`.

Default Android related file globbing behavior is defined in [AutoImport.props](#). This behavior can be disabled for Android items by setting the `$(EnableDefaultAndroidItems)` property to `false`, or all default item inclusion behavior can be disabled by setting the `$(EnableDefaultItems)` property to `false`.

Undesired `.jar` or `.aar` files could be included with the default wildcards. For example, the following C# compiler errors results from a `AndroidStudio\gradle\wrapper\gradle-wrapper.jar` file being unintentionally bound:

```
Org.Gradle.Cli.AbstractCommandLineConverter.cs(11,89): error CS0535:  
'Download' does not implement interface member 'IDownload.Download(Uri,  
File)'  
Org.Gradle.Wrapper.Download.cs(10,60): error CS0535:  
'AbstractCommandLineConverter' does not implement interface member  
'ICommandLineConverter.Convert(ParsedCommandLine, Object)'
```

To solve this issue, you can remove the specific file in your project file:

XML

```
<ItemGroup>  
    <AndroidLibrary Remove="AndroidStudio\gradle\wrapper\gradle-wrapper.jar"  
/>  
</ItemGroup>
```

Alternatively, you could exclude all files within a folder:

XML

```
<AndroidLibrary Remove="AndroidStudio\**\*" />
```

New item group names

`<AndroidLibrary>` is now the recommended item group to use for all `.jar` and `.aar` files. In Xamarin.Android, the following items groups were used, which can instead use item metadata to achieve the same result:

[Expand table](#)

Legacy Item Group	New Item Group	Item Metadata	Legacy Project Type
<code>AndroidAarLibrary</code>	<code>AndroidLibrary</code>	<code>Bind="false"</code>	Application
<code>AndroidJavaLibrary</code>	<code>AndroidLibrary</code>	<code>Bind="false"</code>	Application or class library
<code>EmbeddedJar</code>	<code>AndroidLibrary</code>	n/a	Binding project
<code>EmbeddedReferenceJar</code>	<code>AndroidLibrary</code>	<code>Bind="false"</code>	Binding project
<code>InputJar</code>	<code>AndroidLibrary</code>	<code>Pack="false"</code>	Binding project
<code>LibraryProjectZip</code>	<code>AndroidLibrary</code>	n/a	Binding project

Consider a `.aar` or `.jar` file, in which you aren't interested in including a C# binding. This is common for cases where you have Java or Kotlin dependencies that you don't need to call from C#. In this case, you can set the `Bind` metadata to `false`. By default, the file is picked up by the default wildcards. You can also use the `Update` attribute to set the `Bind` metadata:

XML

```
<ItemGroup>
  <AndroidLibrary Update="foo.jar" Bind="false">
</ItemGroup>
```

In an Android class library project, this would redistribute the `.jar` file inside the resulting NuGet package as is. In an Android application project, this would include the `.jar` file in the resulting `.apk` or `.aab` file. Neither would include a C# binding for this Java library.

Embedded JAR/AAR files

In Xamarin.Android, the Java `.jar` or `.aar` was often embedded into the binding `.dll` as an Embedded Resource. However, this led to slow builds, as each `.dll` must be opened and scanned for Java code. If found, it must be extracted to disk to be used.

In .NET, Java code is no longer embedded in the `.dll`. The app build process will automatically include any `.jar` or `.aar` files it finds in the same directory as a referenced `.dll`.

If a project references a binding via `<PackageReference>` or `<ProjectReference>` then everything works and no additional considerations are required. However, if a project references a binding via `<Reference>`, the `.jar/.aar` must be located next to the `.dll`. That is, for the following reference:

XML

```
<Reference Include="MyBinding.dll" />
```

A directory like the one in the following example won't work:

```
lib/  
  MyBinding.dll
```

Instead, the directory must also contain the native code:

```
lib/  
  MyBinding.dll  
  mybinding.jar
```

Migration considerations

There are several new features set by default to help produce bindings that better match their Java counterparts. However, if you are migrating an existing binding project, these features may create bindings that are not API compatible with your existing bindings. To maintain compatibility, you may wish to disable or modify these new features.

Interface constants

Traditionally, C# has not allowed constants to be declared in an `interface`, which is a common pattern in Java:

Java

```
public interface Foo {  
    public static int BAR = 1;  
}
```

This pattern was previously supported by creating an alternative `class` that contains the constants:

C#

```
public abstract class Foo : Java.Lang.Object  
{  
    public static int Bar = 1;  
}
```

With C# 8, these constants are placed on the `interface`:

C#

```
public interface IFoo  
{  
    public static int Bar = 1;  
}
```

However, this means that the alternative class that existing code may depend on is no longer generated.

Setting the `$(AndroidBoundInterfacesContainConstants)` property to `false` in your project file will revert to the legacy behavior.

Nested interface types

Traditionally, C# has not allowed nested types to be declared in an `interface`, which is allowed in Java:

Java

```
public interface Foo {  
    public class Bar { }  
}
```

This pattern was supported by moving the nested type to a top-level type with a generated name composed of the interface and nested type name:

C#

```
public interface IFoo { }

public class IFooBar : Java.Lang.Object { }
```

With C# 8, nested types can be placed in the `interface`:

C#

```
public interface IFoo
{
    public class Bar : Java.Lang.Object { }
}
```

However, this means that the top-level class that existing code may depend on is no longer generated.

Setting the `$(AndroidBoundInterfacesContainTypes)` property to `false` in your project file will revert to the legacy behavior.

If you wish to use a hybrid approach, for example, to keep existing nested types moved to a top-level type, but allow any future nested types to remain nested, you can specify this at the `interface` level using `metadata` to set the `unnest` attribute. Setting it to `true` will result in "un-nesting" any nested types (legacy behavior):

XML

```
<attr path="/api/package[@name='my.package']/interface[@name='Foo']"
name="unnest">true</attr>
```

Setting it to `false` will result in nested types remaining nested in the `interface` (.NET behavior):

XML

```
<attr path="/api/package[@name='my.package']/interface[@name='Foo']"
name="unnest">>false</attr>
```

Using this approach, you could leave the `$(AndroidBoundInterfacesContainTypes)` property as `true` and set `unnest` to `true` for every `interface` with nested types that

you currently have. These will always remain top-level types, while any new nested types introduced later will be nested.

Static and default interface members (DIM)

Traditionally, C# has not allowed interfaces to contain `static` members and `default` methods:

Java

```
public interface Foo {  
    public static void Bar () { ... }  
    public default void Baz () { ... }  
}
```

Static members on interfaces has been supported by moving them to a sibling `class`:

C#

```
public interface IFoo { }  
  
public class Foo  
{  
    public static void Bar () { ... }  
}
```

`default` interface methods have traditionally not been bound, since they aren't required and there wasn't a C# construct to support them.

With C# 8, `static` and `default` members are supported on interfaces, mirroring the Java interface:

C#

```
public interface IFoo  
{  
    public static void Bar () { ... }  
    public default void Baz () { ... }  
}
```

However, this means the alternative sibling `class` containing `static` members will no longer be generated.

Setting the `$AndroidBoundInterfacesContainStaticAndDefaultInterfaceMethods` property to `false` in your project file will revert to the legacy behavior.

Nullable reference types

Support for Nullable Reference Types (NRT) was added in Xamarin.Android 11.0. NRT support can be enabled using the standard .NET mechanism:

XML

```
<PropertyGroup>
  <Nullable>enable</Nullable>
</PropertyGroup>
```

Because the default for .NET is `disable`, the same applies for Xamarin.Android projects.

Resource.designer.cs

In Xamarin.Android, Java binding projects didn't support generating a `Resource.designer.cs` file. Since binding projects are just class libraries in .NET, this file will be generated. This could be a breaking change when migrating existing projects.

One example of a failure from this change, is if your binding generates a class named `Resource` in the root namespace:

```
error CS0101: The namespace 'MyBinding' already contains a definition for
'Resource'
```

Or in the case of AndroidX, there are project files with `-` in the name such as `androidx.window/window-extensions.csproj`. This results in the root namespace `window-extensions` and invalid C# in `Resource.designer.cs`:

```
error CS0116: A namespace cannot directly contain members such as fields,
methods or statements
error CS1514: { expected
error CS1022: Type or namespace definition, or end-of-file expected
```

To disable `Resource.designer.cs` generation, set the `$(AndroidGenerateResourceDesigner)` property to `false` in your project file:

XML

```
<PropertyGroup>  
  <AndroidGenerateResourceDesigner>>false</AndroidGenerateResourceDesigner>  
</PropertyGroup>
```

Xamarin Apple project migration

Article • 05/07/2024

A .NET 8 project for a .NET for iOS app is similar to the following example:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-ios</TargetFramework>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>true</ImplicitUsings>
    <SupportedOSPlatformVersion>13.0</SupportedOSPlatformVersion>
  </PropertyGroup>
</Project>
```

For a library project, omit the `$(OutputType)` property completely or specify `Library` as the property value.

Changes to MSBuild properties

The following table shows how to map properties in Xamarin Apple projects to .NET projects:

 Expand table

Property	Description	.NET	Project conversion
MtouchExtraArgs	Additional arguments to mtouch.	Some arguments are still applicable, some arguments aren't.	Copy
MtouchArch	App architecture(s).	N/A	Convert to <code>RuntimeIdentifier</code> . For more information, see Convert to RuntimeIdentifier .
XamMacArch	App architecture(s).	N/A	Convert to <code>RuntimeIdentifier</code> . For more information, see Convert to RuntimeIdentifier .

Property	Description	.NET	Project conversion
<code>HttpClientHandler</code>	The default <code>HttpClientHandler</code> .	<code>UseNativeHttpHandler</code>	Convert to <code>UseNativeHttpHandler</code> . For more information, see Convert to UseNativeHttpHandler .
<code>MtouchHttpClientHandler</code>	The default <code>MtouchHttpClientHandler</code> .	<code>UseNativeHttpHandler</code>	Convert to <code>UseNativeHttpHandler</code> . For more information, see Convert to UseNativeHttpHandler .
<code>EnableCodeSigning</code>	Specifies if code signing is enabled.		Copy
<code>CodeSigningKey</code>	Specifies the code signing key.		Rename to <code>CodesignKey</code>
<code>CodesignKey</code>	Specifies the code signing key.		Copy
<code>CodesignProvision</code>	Specifies the provisioning profile.		Copy
<code>CodesignEntitlements</code>	The path to the entitlements file.		Copy
<code>CodesignExtraArgs</code>	Extra code signing arguments.		Copy
<code>PackageSigningKey</code>	Specifies the code signing key to sign the package.		Copy
<code>PackagingExtraArgs</code>	Specifies the extra arguments to the packaging tool.		Copy
<code>ProductDefinition</code>	The path to the product definition file to use when packaging.		Copy
<code>MtouchEnableSGenConc</code>			Rename to <code>EnableSGenConc</code> .
<code>EnableSGenConc</code>			Copy

Convert to RuntimeIdentifier

The following table shows how to convert the `MtouchArch` and `XamMacArch` properties to the `RuntimeIdentifier` property, or the `RuntimeIdentifiers` property, when migrating a Xamarin.iOS project to .NET for iOS:

[Expand table](#)

Value	RuntimeIdentifier	RuntimeIdentifiers
ARMv7	ios-arm	
ARMv7s	ios-arm	
ARMv7+ARMv7s	ios-arm	
ARM64	ios-arm64	
ARMv7+ARM64		ios-arm;ios-arm64
ARMv7+ARMv7s+ARM64		ios-arm;ios-arm64
x86_64	iossimulator-x64	
i386	iossimulator-x86	
x86_64+i386		iossimulator-x86;iossimulator-x64

Important

If you have multiple runtime identifiers, use the `RuntimeIdentifiers` property rather than the `RuntimeIdentifier` property.

The following table shows how to convert the `MtouchArch` and `XamMacArch` properties to the `RuntimeIdentifier` property when migrating a Xamarin.Mac project to .NET for macOS+:

[Expand table](#)

Property	RuntimeIdentifier
x86_64	osx-x64

The following table shows how to convert the `MtouchArch` and `XamMacArch` properties to the `RuntimeIdentifier` property when migrating a Xamarin.tvOS project to .NET for tvOS:

[Expand table](#)

Property	RuntimeIdentifier
ARM64	tvos-arm64
x86_64	tvossimulator-x64

For more information about the `RuntimeIdentifier` property, see [RuntimeIdentifier](#). For more information about runtime identifiers, see [.NET RID Catalog](#).

Convert to UseNativeHttpHandler

The following table shows how to convert the `HttpClientHandler` and `MtouchHttpClientHandler` properties to the `UseNativeHttpHandler` property when migrating a Xamarin Apple project to .NET 8:

[Expand table](#)

Value	UseNativeHttpHandler
HttpClientHandler	false
NSURLSessionHandler	<i>don't set</i>
CFNetworkHandler	<i>don't set</i>

Changes to other items

The following table shows how to map other items in Xamarin Apple projects to .NET projects:

[Expand table](#)

Item	Description	.NET	Project conversion
<code>LinkDescription</code>	Additional XML files to the managed linker.	Identical	Copy

Changes to Info.plist

Some values have moved from *Info.plist* to the project file.

MinimumOSVersion and LSMinimumSystemVersion

The `MinimumOSVersion` and `LSMinimumSystemVersion` properties should be converted to the `SupportedOSPlatformVersion` property in .NET 8 projects. For more information, see [Ensure MinimumOSVersion is consistent with SupportedOSPlatformVersion](#).

See also

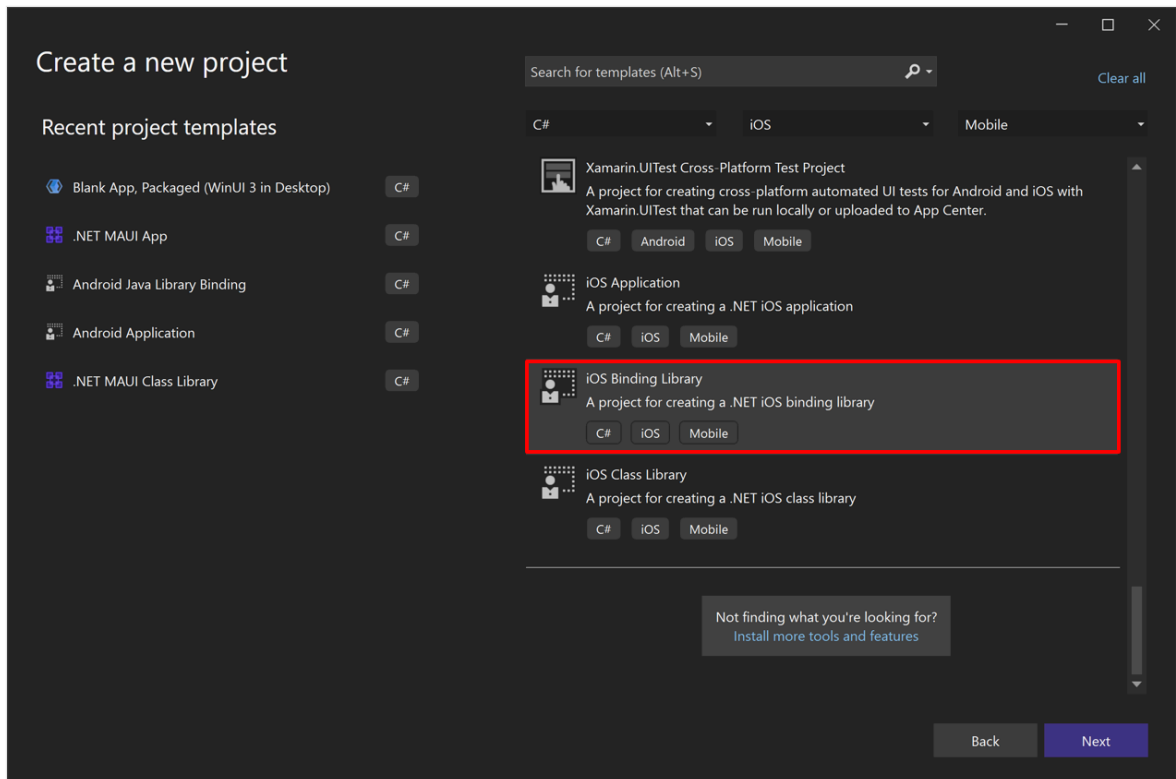
- [Project file properties](#)

Xamarin.iOS binding project migration

Article • 05/07/2024

To migrate a Xamarin.iOS binding library to a .NET for iOS binding library:

1. In Visual Studio, create a new iOS Binding Library project with the same name as your Xamarin.iOS binding project:



Open the project file to confirm that you have a .NET SDK-style project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-ios</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>true</ImplicitUsings>
    <IsBindingProject>true</IsBindingProject>
  </PropertyGroup>

  <ItemGroup>
    <ObjcBindingApiDefinition Include="ApiDefinition.cs" />
    <ObjcBindingCoreSource Include="StructsAndEnums.cs" />
  </ItemGroup>
</Project>
```

ⓘ Note

The project file for an iOS binding library is similar to the project file for an iOS class library.

2. Add your native library, or native framework, to the project as a native reference. Then update the binding project file with the `<NativeReference>` item from your Xamarin.iOS binding project:

XML

```
<ItemGroup>
  <NativeReference Include="libXMBindingLibraryUniversal.a">
    <Kind>Static</Kind>
    <ForceLoad>True</ForceLoad>
  </NativeReference>
</ItemGroup>
```

3. Copy the API definition from your Xamarin.iOS binding project to your .NET for iOS binding project.
4. Copy any additional required code from your Xamarin.iOS binding project to your .NET for iOS binding project.
5. In your .NET for iOS binding project, replace any unsupported APIs with their .NET for iOS equivalents. For more information, see [Unsupported APIs](#).

Unsupported APIs

The following Xamarin.iOS APIs have changed.

System.nint and System.nuint

The `System.nint` and `System.nuint` types aren't available in .NET for iOS. Instead, they're replaced with the `nint` and `nuint` types, which map to `System.IntPtr` and `System.UIntPtr` respectively. Code that uses the `System.nint` and `System.nuint` types won't compile and should be replaced with the `nint` and `nuint` types. In addition, code that overloads on `System.IntPtr` and `nint`, and `System.UIntPtr` and `nuint` won't compile. In this scenario, rename or remove one of the overloads.

System.nfloat

The `System.nfloat` type isn't available in .NET for iOS. Instead, it's replaced with the `System.Runtime.InteropServices.NFloat` type. Modify any code that refers to

`System.nfloat` to use `nfloat` or the fully qualified type name.

NSObject.Handle type change

In .NET for iOS, the `NSObject.Handle` and `INativeObject.Handle` properties have changed type from `System.IntPtr` to a `ObjCRuntime.NativeHandle` struct. This means that many parameters and return values will need to change type. For example:

- All objects that inherit from `Foundation.NSObject` that have `ClassHandle` or `Handle` properties of type `IntPtr` should change their property type to `ObjCRuntime.NativeHandle`.
- All objects that inherit from `Foundation.NSObject` or `ObjCRuntime.DisposableObject` that have constructors that previously took a `System.IntPtr`, or a `System.IntPtr` and a `bool`, should change the `IntPtr` references to `ObjCRuntime.NativeHandle`.

There are implicit conversions between `System.IntPtr` and `ObjCRuntime.NativeHandle`, so after making this type change most code should compile without further changes.

Model.AutoGeneratedName

The `Model.AutoGeneratedName` property isn't available in .NET for iOS. In Xamarin.iOS, this property specified whether the Objective-C type name for the model should be automatically generated. In .NET for iOS, autogenerating the Objective-C type name from the model is the default and only behavior that's available.

Migrate Xamarin.Essentials code in .NET for Android and .NET for iOS apps

Article • 05/07/2024

Xamarin.Essentials is a fundamental library for nearly every Xamarin app, and its functionality is now part of .NET Multi-platform App UI (.NET MAUI).

The following steps outline the process to use .NET MAUI's native device functionality, formerly known as Xamarin.Essentials, in a .NET for Android or .NET for iOS app:

1. Remove the Xamarin.Essentials NuGet package from your .NET for Android or .NET for iOS app.
2. Set the `$(UseMauiEssentials)` build property to `true` in your project file. For more information, see [Modify your project file](#).
3. Initialize the "essentials" functionality by calling the `Platform.Init` method. For more information, see [Initialize the platform](#).
4. Perform additional setup, if necessary. For more information, see [Perform additional setup](#).
5. Add using directives for the required functionality. For more information, see [Add using directives](#).

Important

No action is required to use Xamarin.Essentials in a .NET MAUI app, other than removing references to the `Xamarin.Essentials` namespace, because .NET MAUI already includes the functionality from Xamarin.Essentials.

Modify your project file

To use .NET MAUI's native device functionality in a .NET for Android or .NET for iOS app, modify your project file and set the `$(UseMauiEssentials)` build property to `true`.

Android

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    ...
```

```
<UseMauiEssentials>true</UseMauiEssentials>
</PropertyGroup>
</Project>
```

Initialize the platform

Android

In any [Activity](#) that's launched, you must call the `Platform.Init` method, which is in the `Microsoft.Maui.ApplicationModel` namespace, from the `OnCreate` method:

C#

```
using Android.Content.PM;
using Android.Runtime;
using Microsoft.Maui.ApplicationModel;

namespace MyAndroidApp;

[Activity(Label = "@string/app_name", MainLauncher = true)]
public class MainActivity : Activity
{
    protected override async void OnCreate(Bundle? savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
        Platform.Init(this, savedInstanceState);
        // ...
    }
}
```

The `Platform.Init` method requires an [Application](#) argument, or a [Activity](#) argument and a [Bundle](#) argument.

Perform additional setup

The static `Platform` class contains platform-specific helpers.

Android

 Expand table

Member	Purpose
<code>ActivityStateChanged</code>	An event that's raised when any Activity's state changes.
<code>AppContext</code>	A property that gets the <code>Context</code> object that represents the current app context.
<code>CurrentActivity</code>	A property that gets the current <code>Activity</code> object that represents the current activity.
<code>Intent</code>	A static class that contains the <code>ActionAppAction</code> string, which is the identifier for the <code>Intent</code> used by app actions.
<code>OnNewIntent</code>	Pass an <code>Intent</code> from an activity's overridden method, when invoking an app action.
<code>OnResume</code>	Pass an <code>Activity</code> from an activity's overridden method, when an <code>Activity</code> is resumed as part of invoking an app action.
<code>OnRequestPermissionsResult</code>	Pass permission request results from an activity's overridden method, for handling internal permission requests.
<code>WaitForActivityAsync</code>	Wait for an <code>Activity</code> to be created or become active.

To access the current `Context` or `Activity` for the running app:

C#

```
var context = Platform.AppContext;

// Current Activity or null if not initialized or not started.
var activity = Platform.CurrentActivity;
```

If there's a situation where the `Activity` is needed, but the app hasn't fully started, call the `WaitForActivityAsync` method:

C#

```
var activity = await Platform.WaitForActivityAsync();
```

To handle runtime permission requests, override the `OnRequestPermissionsResult` method in every `Activity` and call the `Platform.OnRequestPermissionsResult` method from it:

C#

```
public override void OnRequestPermissionsResult(int requestCode,
string[] permissions, Permission[] grantResults)
```



```
{
    Platform.OnRequestPermissionsResult(requestCode, permissions,
grantResults);
    base.OnRequestPermissionsResult(requestCode, permissions,
grantResults);
}
```

In addition to getting the current [Activity](#), you can also register for lifecycle events:

C#

```
protected override void OnCreate(Bundle bundle)
{
    base.OnCreate(bundle);
    Platform.Init(this, bundle);
    Platform.ActivityStateChanged += Platform_ActivityStateChanged;
}

protected override void OnDestroy()
{
    base.OnDestroy();
    Platform.ActivityStateChanged -= Platform_ActivityStateChanged;
}

void Platform_ActivityStateChanged(object sender,
ActivityStateChangedEventArgs e) =>
    Toast.MakeText(this, e.State.ToString(), ToastLength.Short).Show();
```

Activity states are:

- Created
- Resumed
- Paused
- Destroyed
- SaveInstanceState
- Started
- Stopped

Add using directives

The implicit `global using` directives for .NET for iOS and .NET for Android don't include the namespaces for .NET MAUI's native device functionality. Therefore, `using` directives for the `Xamarin.Essentials` namespace should be replaced with `using` directives for the namespace that contains the required functionality:

Namespace	Purpose
Microsoft.Maui.ApplicationModel	Application model functionality, including app actions, permissions, and version tracking.
Microsoft.Maui.ApplicationModel.Communication	Communication functionality, including contacts, email, and networking.
Microsoft.Maui.Devices	Device functionality, including battery, sensors, flashlight, and haptic feedback.
Microsoft.Maui.Media	Media functionality, including media picking, and text-to-speech.
Microsoft.Maui.ApplicationModel.DataTransfer	Sharing functionality, including the clipboard, and file sharing.
Microsoft.Maui.Storage	Storage functionality, including file picking, and secure storage.

For more information about the functionality in each namespace, see [Platform integration](#).

Upgrade a Xamarin.Forms app to a .NET MAUI app with the .NET Upgrade Assistant

Article • 01/31/2024

The .NET Upgrade Assistant helps you upgrade Xamarin.Forms projects to .NET Multi-platform App UI (.NET MAUI) by converting the solution's project file and by performing common code updates. Specifically, the tool will:

- Convert the Xamarin.Forms class library project, Xamarin.iOS project, and Xamarin.Android project to SDK-style projects.
- Update the target framework in project files to net8.0-android and net8.0-ios, as required.
- Set `<UseMaui>true</UseMaui>` in project files.
- Add additional project properties, and remove project properties that aren't required.
- Add and remove specific NuGet packages:
 - Remove the Xamarin.Forms and Xamarin.Essentials NuGet packages.
 - Replace the Xamarin.CommunityToolkit NuGet package with the .NET MAUI Community Toolkit NuGet package.
 - Replace Xamarin.Forms compatible versions of the SkiaSharp NuGet packages with .NET MAUI compatible versions, if used.
- Remove references to the `Xamarin.Essentials` namespace, and replace the `Xamarin.Forms` namespace with the `Microsoft.Maui` and `Microsoft.Maui.Controls` namespaces.

After you run the tool, additional effort will be required to complete the migration.

ⓘ Note

The .NET Upgrade Assistant for .NET MAUI doesn't support upgrading UWP projects, iOS extension projects, or binding projects.

For more information about .NET Upgrade Assistant, including the other app types it can convert, see [Overview of the .NET Upgrade Assistant](#).

Get started

.NET Upgrade Assistant is available as a Visual Studio extension on Windows, and as a CLI tool for Windows and Mac. To use it, your Xamarin.Forms project must use Xamarin.Forms 4.8 or higher. However, for best success we recommend that your Xamarin.Forms project uses Xamarin.Forms 5.0, and .NET Standard 2.0 or higher.

Important

The .NET Upgrade Assistant for .NET MAUI is still under development. Please [file feedback](#)  with an 'area:Maui' label so we can continue to improve this tool.

.NET Upgrade Assistant will prompt you for the type of upgrade to perform:

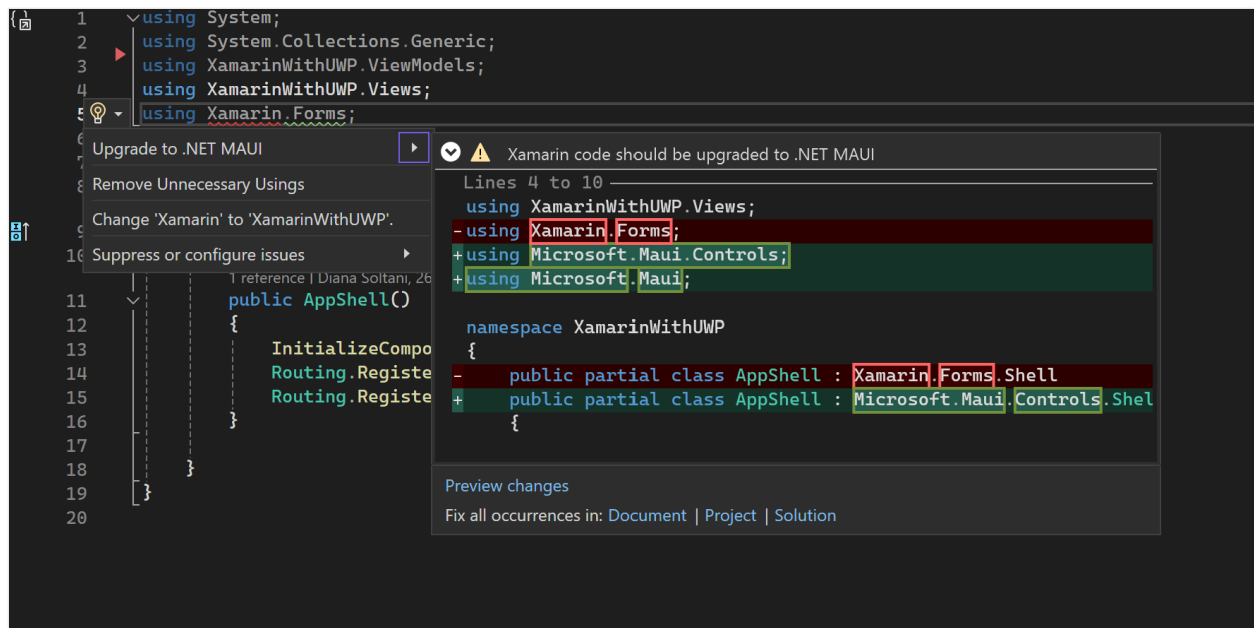
- **In-place:** This option upgrades your project without making a copy.
- **Side-by-side:** This option copies your project and upgrades the copy, leaving the original project alone. Currently, a .NET MAUI side-by-side upgrade requires the creation of new projects to upgrade each project head. Attempting to upgrade a project head into an existing project will likely cause errors and is not a supported experience at this time.

Before upgrading your Xamarin.Forms app to .NET MAUI, you should first update your Xamarin.Forms app to use Xamarin.Forms 5 and ensure that it still runs correctly. In addition, you should update the dependencies that your app uses to the latest versions.

This will help to simplify the rest of the migration process, as it will minimize the API differences between Xamarin.Forms and .NET MAUI, and will ensure that you are using .NET compatible versions of your dependencies if they exist.

Quick actions in Visual Studio

.NET Upgrade Assistant has a quick action for converting Xamarin.Forms namespaces to the appropriate .NET MAUI namespaces, that's displayed as a light bulb:



The quick action will make the necessary replacements across the file.

Note

Quick action light bulbs will appear as you migrate the files from your Xamarin.Forms project into a .NET MAUI project.

Installation

Visual Studio

Visual Studio 2022 version 17.6.0 or later is highly recommended to run the upgrade assistant. To install the upgrade assistant extension:

1. Open Visual Studio and select the **Extensions > Manage Extensions** menu item.
2. In the **Manage Extensions** dialog, search for and download the .NET Upgrade Assistant.
3. Once the extension has been downloaded, close Visual Studio. The extension will then be installed.

Extensions are updated automatically when a new version is available on Visual Studio Marketplace. For more information, see [Automatic extension updates](#).

Run the upgrade assistant

Visual Studio

Right-click on the project in Solution Explorer and select **Upgrade**.

Next steps

[Manual migration](#)

Upgrade manually to a multi-project app

Article • 06/03/2024

A Xamarin.Forms app can be manually upgraded to a multi-project .NET Multi-platform App UI (.NET MAUI) app with two approaches:

- By creating a new .NET MAUI app using the multi-project template.

With this approach you create a new multi-project .NET MAUI app, and then migrate the code and resources from your Xamarin.Forms app to the multi-project .NET MAUI app. For more information, see [Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app with the project template](#).

- By migrating a Xamarin.Forms library project to a .NET MAUI library project.

With this approach you create a new .NET MAUI library project that replaces your Xamarin.Forms library project, and then update your Xamarin.Forms platform projects to be SDK-style projects and reference the .NET MAUI library project. You'll then need to enable .NET MAUI support in each platform project, update each platform project's entry point class, and then configure the bootstrapping of your .NET MAUI app. For more information, see [Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app](#).

Using either multi-project template approach for your .NET MAUI app can more easily provide one-to-one mappings as you incrementally upgrade individual components of your project.

Important

Before upgrading your Xamarin.Forms app to .NET MAUI, you should first update your Xamarin.Forms app to use Xamarin.Forms 5 and ensure that it still runs correctly. In addition, you should update the dependencies that your app uses to the latest versions. This will help to simplify the rest of the migration process, as it will minimize the API differences between Xamarin.Forms and .NET MAUI, and will ensure that you are using .NET compatible versions of your dependencies if they exist.

Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app with the project template

Article • 06/03/2024

Upgrading a multi-project Xamarin.Forms app to a multi-project .NET Multi-platform App UI (.NET MAUI) app can be accomplished by creating a new multi-project .NET MAUI app and then migrating the code and resources from your Xamarin.Forms apps to the multi-project .NET MAUI app. There are then additional steps to take advantage of changes in .NET MAUI.

To manually migrate a multi-project Xamarin.Forms app to a multi-project .NET MAUI library app, you must:

- ✓ Update your Xamarin.Forms app to use Xamarin.Forms 5.
- ✓ Update the app's dependencies to the latest versions.
- ✓ Ensure the app still works.
- ✓ Update namespaces.
- ✓ Address any API changes.
- ✓ Upgrade or replace incompatible dependencies with .NET 8 versions.
- ✓ Compile and test your app.

To simplify the upgrade process, you should create a new multi-project .NET MAUI app of the same name as your Xamarin.Forms app, and then copy in your code, configuration, and resources. This is the approach outlined below.

Update your Xamarin.Forms app

Before upgrading your Xamarin.Forms app to .NET MAUI, you should first update your Xamarin.Forms app to use Xamarin.Forms 5 and ensure that it still runs correctly. In addition, you should update the dependencies that your app uses to the latest versions.

This will help to simplify the rest of the migration process, as it will minimize the API differences between Xamarin.Forms and .NET MAUI, and will ensure that you are using .NET compatible versions of your dependencies if they exist.

Create a new project

In Visual Studio, create a new .NET MAUI multi-project app of the same name as your Xamarin.Forms project. The multi-project template provides a .NET MAUI app for Android, iOS, Mac Catalyst, and WinUI with multiple, separate app project.

ⓘ Note

You can remove the the app projects for platforms you don't support.

Opening the project file for the class library will confirm that you have a .NET SDK-style project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFrameworks>net8.0;net8.0-android;net8.0-ios;net8.0-maccatalyst</TargetFramework>
    <TargetFrameworks Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net8.0-windows10.0.19041.0</TargetFrameworks>
    <SingleProject>true</SingleProject>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseMaui>true</UseMaui>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Opening a specific platform project should also confirm that you have similar properties set including versioning with a reference to the library project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net8.0-android</TargetFramework>
    <SupportedOSPlatformVersion>21.0</SupportedOSPlatformVersion>
    <OutputType>Exe</OutputType>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UseMaui>true</UseMaui>
  </PropertyGroup>

  <!--
  ...
  -->

  <ItemGroup>
    <ProjectReference Include="..\MauiApp.1\MauiApp.1.csproj" />
  </ItemGroup>

</Project>
```

Copy files to the .NET MAUI app

All of the cross-platform code from your Xamarin.Forms library project should be copied into your .NET MAUI library project in identically named folders and files.

Custom renderers can either be reused in a .NET MAUI app, or migrated to a .NET MAUI handler. For more information, see [Reuse custom renderers in .NET MAUI](#) and [Migrate a Xamarin.Forms custom renderer to a .NET MAUI handler](#).

Effects can be reused in a .NET MAUI app. For more information, see [Reuse effects](#).

If there are platform specific services that need to be migrated to .NET MAUI, use the [AddTransient\(IServiceCollection, Type\)](#) method to add a transient service of the specified type to the specified [IServiceCollection](#).

Any platform code, resources, and configuration should then be copied to your multi-project .NET MAUI app.

Namespace changes

Namespaces have changed in the move from Xamarin.Forms to .NET MAUI, and Xamarin.Essentials features are now part of .NET MAUI. To make namespace updates, perform a find and replace for the following namespaces:

 Expand table

Xamarin.Forms namespace	.NET MAUI namespace(s)
<code>Xamarin.Forms</code>	Microsoft.Maui and Microsoft.Maui.Controls
<code>Xamarin.Forms.DualScreen</code>	Microsoft.Maui.Controls.Foldable
<code>Xamarin.Forms.Maps</code>	Microsoft.Maui.Controls.Maps and Microsoft.Maui.Maps
<code>Xamarin.Forms.PlatformConfiguration</code>	Microsoft.Maui.Controls.PlatformConfiguration
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific.AppCompat</code>	Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific.AppCompat
<code>Xamarin.Forms.PlatformConfiguration.TizenSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.TizenSpecific
<code>Xamarin.Forms.PlatformConfiguration.WindowsSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.WindowsSpecific
<code>Xamarin.Forms.PlatformConfiguration.iOSSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.iOSSpecific
<code>Xamarin.Forms.Shapes</code>	Microsoft.Maui.Controls.Shapes
<code>Xamarin.Forms.StyleSheets</code>	Microsoft.Maui.Controls.StyleSheets
<code>Xamarin.Forms.Xaml</code>	Microsoft.Maui.Controls.Xaml

.NET MAUI projects make use of implicit `global using` directives. This feature enables you to remove `using` directives for the `Xamarin.Essentials` namespace, without having to replace them with the equivalent .NET MAUI namespaces.

In addition, the default XAML namespace has changed from `http://xamarin.com/schemas/2014/forms` in Xamarin.Forms to `http://schemas.microsoft.com/dotnet/2021/maui` in .NET MAUI. Therefore, you should replace all occurrences of `xmlns="http://xamarin.com/schemas/2014/forms"` with `xmlns="http://schemas.microsoft.com/dotnet/2021/maui"`.

! Note

You can quickly update your `Xamarin.Forms` namespaces to `Microsoft.Maui` by using [Quick actions in Visual Studio](#), provided that you have [Upgrade Assistant](#) installed.

API changes

Some APIs have changed in the move from Xamarin.Forms to .NET MAUI. This is multiple reasons including removing duplicate functionality caused by Xamarin.Essentials becoming part of .NET MAUI, and ensuring that APIs follow .NET naming guidelines. The following sections discuss these changes.

Color changes

In Xamarin.Forms, the `Xamarin.Forms.Color` struct lets you construct `Color` objects using `double` values, and provides named colors, such as `Xamarin.Forms.Color.AliceBlue`. In .NET MAUI, this functionality has been separated into the `Microsoft.Maui.Graphics.Color` class, and the `Microsoft.Maui.Graphics.Colors` class.

The `Microsoft.Maui.Graphics.Color` class, in the `Microsoft.Maui.Graphics` namespace, lets you construct `Color` objects using `float` values, `byte` values, and `int` values. The `Microsoft.Maui.Graphics.Colors` class, which is also in the `Microsoft.Maui.Graphics` namespace, largely provides the same named colors. For example, use `Colors.AliceBlue` to specify the `AliceBlue` color.

The following table shows the API changes between the `Xamarin.Forms.Color` struct and the `Microsoft.Maui.Graphics.Color` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Color.R</code>	<code>Microsoft.Maui.Graphics.Color.Red</code>	
<code>Xamarin.Forms.Color.G</code>	<code>Microsoft.Maui.Graphics.Color.Green</code>	
<code>Xamarin.Forms.Color.B</code>	<code>Microsoft.Maui.Graphics.Color.Blue</code>	
<code>Xamarin.Forms.Color.A</code>	<code>Microsoft.Maui.Graphics.Color.Alpha</code>	
<code>Xamarin.Forms.Color.Hue</code>	<code>Microsoft.Maui.Graphics.Color.GetHue</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Saturation</code>	<code>Microsoft.Maui.Graphics.Color.GetSaturation</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Luminosity</code>	<code>Microsoft.Maui.Graphics.Color.GetLuminosity</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Default</code>		No .NET MAUI equivalent. Instead, <code>Microsoft.Maui.Graphics.Color</code> objects default to <code>null</code> .

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Color.Accent</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Color.FromHex</code>	Microsoft.Maui.Graphics.Color.FromArgb	Microsoft.Maui.Graphics.Color.FromHex is obsolete and will be removed in a future release.

In addition, all of the numeric values in a [Microsoft.Maui.Graphics.Color](#) are `float`, rather than `double` as used in `Xamarin.Forms.Color`.

ⓘ Note

Unlike `Xamarin.Forms`, a [Microsoft.Maui.Graphics.Color](#) doesn't have an implicit conversion to [System.Drawing.Color](#).

Layout changes

The following table lists the layout APIs that have been removed in the move from `Xamarin.Forms` to .NET MAUI:

🔗 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.AbsoluteLayout</code> <code>out.IAbsoluteList<T>.Add</code>		The <code>Add</code> overload that accepts 3 arguments isn't present in .NET MAUI.
<code>Xamarin.Forms.Grid.IGridLayout<T>.AddHorizontal</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Grid.IGridLayout<T>.AddVertical</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.RelativeLayout</code> <code>out</code>	Microsoft.Maui.Controls.Compatibility.RelativeLayout	In .NET MAUI, <code>RelativeLayout</code> only exists as a compatibility control for users migrating from <code>Xamarin.Forms</code> . Use Grid instead, or add the <code>xmlns</code> for the compatibility namespace.

In addition, adding children to a layout in code in `Xamarin.Forms` is accomplished by adding the children to the layout's `Children` collection:

C#

```
Grid grid = new Grid();
grid.Children.Add(new Label { Text = "Hello world" });
```

In .NET MAUI, the [Children](#) collection is for internal use by .NET MAUI and shouldn't be manipulated directly. Therefore, in code children should be added directly to the layout:

C#

```
Grid grid = new Grid();
grid.Add(new Label { Text = "Hello world" });
```

Important

Any `Add` layout extension methods, such as [GridExtensions.Add](#), are invoked on the layout rather than the layouts [Children](#) collection.

You may notice when running your upgraded .NET MAUI app that layout behavior is different. For more information, see [Layout behavior changes from Xamarin.Forms](#).

Custom layout changes

The process for creating a custom layout in Xamarin.Forms involves creating a class that derives from `Layout<View>`, and overriding the `VisualElement.OnMeasure` and `Layout.LayoutChildren` methods. For more information, see [Create a custom layout in Xamarin.Forms](#).

In .NET MAUI, the layout classes derive from the abstract [Layout](#) class. This class delegates cross-platform layout and measurement to a layout manager class. Each layout manager class implements the [ILayoutManager](#) interface, which specifies that [Measure](#) and [ArrangeChildren](#) implementations must be provided:

- The [Measure](#) implementation calls [IView.Measure](#) on each view in the layout, and returns the total size of the layout given the constraints.
- The [ArrangeChildren](#) implementation determines where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view with its appropriate bounds. The return value is the actual size of the layout.

For more information, see [Custom layouts](#).

Device changes

Xamarin.Forms has a `Xamarin.Forms.Device` class that helps you to interact with the device and platform the app is running on. The equivalent class in .NET MAUI, [Microsoft.Maui.Controls.Device](#), is deprecated and its functionality is replaced by multiple types.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Device` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.Android</code>	Microsoft.Maui.Devices.DevicePlatform.Android	
<code>Xamarin.Forms.Device.iOS</code>	Microsoft.Maui.Devices.DevicePlatform.iOS	

Xamarin.Forms API	.NET MAUI API	Comments
	OS	
Xamarin.Forms.Device.GTK		No .NET MAUI equivalent.
Xamarin.Forms.Device.macOS		No .NET MAUI equivalent. Instead, use Microsoft.Maui.Devices.DevicePlatform.MacCatalyst .
Xamarin.Forms.Device.Tizen	Microsoft.Maui.Devices.DevicePlatform.Tizen	
Xamarin.Forms.Device.UWP	Microsoft.Maui.Devices.DevicePlatform.WinUI	
Xamarin.Forms.Device.WPF		No .NET MAUI equivalent.
Xamarin.Forms.Device.Flags		No .NET MAUI equivalent.
Xamarin.Forms.Device.FlowDirection	Microsoft.Maui.ApplicationModel.Application.RequestedLayoutDirection	
Xamarin.Forms.Device.Idiom	Microsoft.Maui.Devices.DeviceInfo.Idiom	
Xamarin.Forms.Device.IsInvokeRequired	Microsoft.Maui.Dispatching.Dispatcher.IsDispatchRequired	
Xamarin.Forms.Device.OS	Microsoft.Maui.Devices.DeviceInfo.Platform	
Xamarin.Forms.Device.RuntimePlatform	Microsoft.Maui.Devices.DeviceInfo.Platform	
Xamarin.Forms.Device.BeginInvokeOnMainThread	Microsoft.Maui.ApplicationModel.MainThread.BeginInvokeOnMainThread	
Xamarin.Forms.Device.GetMainThreadSynchronizationContextAsync	Microsoft.Maui.ApplicationModel.MainThread.GetMainThreadSynchronizationContextAsync	
Xamarin.Forms.Device.GetNamedColor		No .NET MAUI equivalent.
Xamarin.Forms.Device.GetNamedSize		No .NET MAUI equivalent.
Xamarin.Forms.Device.Invalidated	Microsoft.Maui.Controls.VisualElement.InvalidateMeasure	
Xamarin.Forms.Device.InvokeOnMainThreadAsync	Microsoft.Maui.ApplicationModel.MainThread.InvokeOnMainThreadAsync	
Xamarin.Forms.Device.OnPlatform	Microsoft.Maui.Devices.DeviceInfo.Platform	
Xamarin.Forms.Device.OpenUri	Microsoft.Maui.ApplicationModel.Launcher.OpenAsync	

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.SetFlags</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.SetFlowDirection</code>	Microsoft.Maui.Controls.Window.FlowDirection	
<code>Xamarin.Forms.Device.StartTimer</code>	Microsoft.Maui.Dispatching.DispatcherExtensions.StartTimer or Microsoft.Maui.Dispatching.Dispatcher.DispatchDelayed	

Map changes

In Xamarin.Forms, the `Map` control and associated types are in the `Xamarin.Forms.Maps` namespace. In .NET MAUI, this functionality has moved to the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Maps](#) namespaces. Some properties have been renamed and some types have been replaced with equivalent types from Xamarin.Essentials.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Maps` namespace:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Maps.Map.HasScrollEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsScrollEnabled	
<code>Xamarin.Forms.Maps.Map.HasZoomEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsZoomEnabled	
<code>Xamarin.Forms.Maps.Map.TrafficEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsTrafficEnabled	
<code>Xamarin.Forms.Maps.Map.MoveToLastRegionOnLayoutChange</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Maps.Pin.Id</code>	Microsoft.Maui.Controls.Maps.Pin.MarkerId	
<code>Xamarin.Forms.Maps.Pin.Position</code>	Microsoft.Maui.Controls.Maps.Pin.Location	
<code>Xamarin.Forms.Maps.MapClickedEventArgs.Position</code>	Microsoft.Maui.Controls.Maps.MapClickedEventArgs.Location	
<code>Xamarin.Forms.Maps.Position</code>	Microsoft.Maui.Devices.Sensors.Location	Members of type <code>Xamarin.Forms.Maps.Position</code> have changed to the Microsoft.Maui.Devices.Sensors.Location type.
<code>Xamarin.Forms.Maps.Geocoder</code>	Microsoft.Maui.Devices.Sensors.Geocoding	Members of type <code>Xamarin.Forms.Maps.Geocoder</code> have changed to the Microsoft.Maui.Devices.Sensors.Geocoding type.

.NET MAUI has two `Map` types - [Microsoft.Maui.Controls.Maps.Map](#) and [Microsoft.Maui.ApplicationModel.Map](#). Because the [Microsoft.Maui.ApplicationModel](#) namespace is one of .NET MAUI's `global using` directives, when using the [Microsoft.Maui.Controls.Maps.Map](#) control from code you'll have to fully qualify your `Map` usage or use a [using alias](#).

In XAML, an `xmlns` namespace definition should be added for the [Map](#) control. While this isn't required, it prevents a collision between the `Polygon` and `Polyline` types, which exist in both the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Controls.Shapes](#) namespaces. For more information, see [Display a map](#).

Other changes

A small number of other APIs have been consolidated in the move from Xamarin.Forms to .NET MAUI. The following table shows these changes:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Application.Properties</code>	Microsoft.Maui.Storage.Preferences	
<code>Xamarin.Forms.Button.Image</code>	Microsoft.Maui.Controls.Button.ImageSource	
<code>Xamarin.Forms.Frame.OutlineColor</code>	Microsoft.Maui.Controls.Frame.BorderColor	
<code>Xamarin.Forms.IQueryAttributable.ApplyQueryAttributes</code>	Microsoft.Maui.Controls.IQueryAttributable.ApplyQueryAttributes	In Xamarin.Forms, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, string></code> argument. In .NET MAUI, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, object></code> argument.
<code>Xamarin.Forms.MenuItem.Icon</code>	Microsoft.Maui.Controls.MenuItem.IconImageSource	<code>Xamarin.Forms.MenuItem.Icon</code> is the base class for <code>Xamarin.Forms.ToolbarItem</code> , and so <code>ToolbarItem.Icon</code> becomes <code>ToolbarItem.IconImageSource</code> .
<code>Xamarin.Forms.OrientationStateTrigger.Orientation</code>	Microsoft.Maui.Controls.OrientationStateTrigger.Orientation	In Xamarin.Forms, the <code>OrientationStateTrigger.Orientation</code> property is of type <code>Xamarin.Forms.Internals.DeviceOrientation</code> . In .NET MAUI, the <code>OrientationStateTrigger.Orientation</code> property is of type DisplayOrientation .
<code>Xamarin.Forms.OSAppTheme</code>	Microsoft.Maui.ApplicationModel.AppTheme	
<code>Xamarin.Forms.Span.ForegroundColor</code>	Microsoft.Maui.Controls.Span.TextColor	
<code>Xamarin.Forms.ToolbarItem.Name</code>	Microsoft.Maui.Controls.MenuItem.Text	Microsoft.Maui.Controls.MenuItem.Text is the base class for Microsoft.Maui.Controls.ToolbarItem , and so <code>ToolbarItem.Name</code> b

Xamarin.Forms API	.NET MAUI API	Comments
		comes from <code>ToolbarItem.Text</code> .

In addition, in Xamarin.Forms, the `Page.OnAppearing` override is called on Android when an app is backgrounded and then brought to the foreground. However, this override isn't called on iOS and Windows in the same scenario. In .NET MAUI, the `OnAppearing()` override isn't called on any platforms when an app is backgrounded and then brought to the foreground. Instead, you should listen to lifecycle events on `Window` to be notified when an app returns to the foreground. For more information, see [.NET MAUI windows](#).

Native forms changes

[Native forms](#) in Xamarin.Forms has become native embedding in .NET MAUI, and uses a different initialization approach and different extension methods to convert cross-platform controls to their native types. For more information, see [Native embedding](#).

AssemblyInfo changes

Properties that are typically set in an `AssemblyInfo.cs` file are now available in your SDK-style project. We recommend migrating them from `AssemblyInfo.cs` to your project file in every project, and removing the `AssemblyInfo.cs` file.

Optionally, you can keep the `AssemblyInfo.cs` file and set the `GenerateAssemblyInfo` property in your project file to `false`:

XML

```
<PropertyGroup>
  <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
</PropertyGroup>
```

For more information about the `GenerateAssemblyInfo` property, see [GenerateAssemblyInfo](#).

Update app dependencies

Generally, Xamarin.Forms NuGet packages are not compatible with .NET 8 unless they have been recompiled using .NET target framework monikers (TFMs). However, Android apps can use NuGet packages targeting the `monoandroid` and `monoandroidXX.X` frameworks.

You can confirm a package is .NET 8 compatible by looking at the **Frameworks** tab on [NuGet](#) for the package you're using, and checking that it lists one of the compatible frameworks shown in the following table:

 Expand table

Compatible frameworks	Incompatible frameworks
net8.0-android, monoandroid, monoandroidXX.X	
net8.0-ios	monotouch, xamarinios, xamarinios10
net8.0-macos	monomac, xamarinmac, xamarinmac20
net8.0-tvos	xamarintvos
	xamarinwatchos

 Note

.NET Standard libraries that have no dependencies on the incompatible frameworks listed above are still compatible with .NET 8.

If a package on [NuGet](#) indicates compatibility with any of the compatible frameworks above, regardless of also including incompatible frameworks, then the package is compatible. Compatible NuGet packages can be added to your .NET MAUI library project using the NuGet package manager in Visual Studio.

If you can't find a .NET 8 compatible version of a NuGet package you should:

- Recompile the package with .NET TFMs, if you own the code.
- Look for a preview release of a .NET 8 version of the package.
- Replace the dependency with a .NET 8 compatible alternative.

Compile and troubleshoot

Once your dependencies are resolved, you should build your project. Any errors will guide you towards next steps.

 Tip

- Delete all *bin* and *obj* folders from all projects before opening and building projects in Visual Studio, particularly when changing .NET versions.
- Delete the *Resource.designer.cs* generated file from the Android project.

The following table provides guidance for overcoming common build or runtime issues:

 **Expand table**

Issue	Tip
<code>Xamarin.*</code> namespace doesn't exist.	Update the namespace to its .NET MAUI equivalent. For more information, see Namespace changes .

Issue	Tip
API doesn't exist.	Update the API usage to its .NET MAUI equivalent. For more information, see API changes .
App won't deploy.	Ensure that the required platform project is set to deploy in Visual Studio's Configuration Manager.
App won't launch.	Update each platform project's entry point class, and the app entry point. For more information, see Bootstrap your migrated app .
CollectionView doesn't scroll.	Check the container layout and the measured size of the CollectionView . By default the control will take up as much space as the container allows. A Grid constrains children at its own size. However a StackLayout enables children to take up space beyond its bounds.
Pop-up is displayed under the page on iOS.	In Xamarin.Forms, all pop-ups on iOS are <code>UIWindow</code> instances but in .NET MAUI pop-ups are displayed by locating the current presenting <code>ViewController</code> and displaying the pop-up with <code>PresentViewControllerAsync</code> . In plugins such as Mopups, to ensure that your pop-ups are correctly displayed you should call DisplayAlert , DisplayActionSheet , or DisplayPromptAsync from the ContentPage that's used inside the <code>Mopup</code> popup.
BoxView not appearing.	The default size of a BoxView in Xamarin.Forms is 40x40. The default size of a BoxView in .NET MAUI is 0x0. Set <code>WidthRequest</code> and <code>HeightRequest</code> to 40.
Layout is missing padding, margin, or spacing.	Add default values to your project based on the .NET MAUI style resource. For more information, see Default value changes from Xamarin.Forms .
Custom layout doesn't work.	Custom layout code needs updating to work in .NET MAUI. For more information, see Custom layout changes .
Custom renderer doesn't work.	Renderer code needs updating to work in .NET MAUI. For more information, see Use custom renderers in .NET MAUI .
Effect doesn't work.	Effect code needs updating to work in .NET MAUI. For more information, see Use effects in .NET MAUI .
SkiaSharp code doesn't work.	SkiaSharp code needs minor updates to work in .NET MAUI. For more information, see Reuse SkiaSharp code in .NET MAUI .
Can't access previously created app properties data.	Migrate the app properties data to .NET MAUI preferences. For more information, see Migrate data from the Xamarin.Forms app properties dictionary to .NET MAUI preferences .
Can't access previously created secure storage data.	Migrate the secure storage data to .NET MAUI. For more information, see Migrate from Xamarin.Essentials secure storage to .NET MAUI secure storage .
Can't access previously created version tracking data.	Migrate the version tracking data to .NET MAUI. For more information, see Migrate version tracking data from a Xamarin.Forms app to a .NET MAUI app .

See also

- [Porting from .NET Framework to .NET](#)
- [.NET Upgrade Assistant](#)

Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app

Article • 04/04/2024

Upgrading a multi-project Xamarin.Forms app to a multi-project .NET Multi-platform App UI (.NET MAUI) app follows the same steps as a Xamarin.Android and Xamarin.iOS project, with additional steps to take advantage of changes in .NET MAUI.

This article describes how to manually migrate a Xamarin.Forms library project to a .NET MAUI library project. Before you do this, you must update your Xamarin.Forms platform projects to be SDK-style projects. SDK-style projects are the same project format used by all .NET workloads, and compared to many Xamarin projects are much less verbose. For information about updating your app projects, see [Upgrade Xamarin.Android, Xamarin.iOS, and Xamarin.Mac projects to .NET](#), [Xamarin.Android project migration](#), [Xamarin Apple project migration](#), and [Xamarin.Forms UWP project migration](#).

To migrate a Xamarin.Forms library project to a .NET MAUI library project, you must:

- ✓ Update your Xamarin.Forms app to use Xamarin.Forms 5.
- ✓ Update the app's dependencies to the latest versions.
- ✓ Ensure the app still works.
- ✓ Update your project file to be SDK-style.
- ✓ Update namespaces.
- ✓ Address any API changes.
- ✓ Configure .NET MAUI.
- ✓ Upgrade or replace incompatible dependencies with .NET 8 versions.
- ✓ Compile and test your app.

To simplify the upgrade process, you should create a new .NET MAUI library project of the same name as your Xamarin.Forms library project, and then copy in your code, configuration, and resources. This is the approach outlined below.

Update your Xamarin.Forms app

Before upgrading your Xamarin.Forms app to .NET MAUI, you should first update your Xamarin.Forms app to use Xamarin.Forms 5 and ensure that it still runs correctly. In addition, you should update the dependencies that your app uses to the latest versions.

This will help to simplify the rest of the migration process, as it will minimize the API differences between Xamarin.Forms and .NET MAUI, and will ensure that you are using .NET compatible versions of your dependencies if they exist.

Create a new project

In Visual Studio, create a new .NET MAUI class library project of the same name as your Xamarin.Forms library project. This project will host the code from your Xamarin.Forms library project. Opening the

project file will confirm that you have a .NET SDK-style project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFrameworks>net8.0;net8.0-android;net8.0-ios;net8.0-
maccatalyst</TargetFrameworks>
    <TargetFrameworks
Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net8.0-
windows10.0.19041.0</TargetFrameworks>
    <!-- Uncomment to also build the tizen app. You will need to install tizen by
following this: https://github.com/Samsung/Tizen.NET -->
    <!-- <TargetFrameworks>$(TargetFrameworks);net8.0-tizen</TargetFrameworks> -->
    <UseMaui>true</UseMaui>
    <SingleProject>true</SingleProject>
    <ImplicitUsings>enable</ImplicitUsings>

    <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'ios'">11.0</SupportedOSPlatformVersion>
    <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'maccatalyst'">13.1</SupportedOSPlatformVersion>
    <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'android'">21.0</SupportedOSPlatformVersion>
    <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'windows'">10.0.17763.0</SupportedOSPlatformVersion>
    <TargetPlatformMinVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'windows'">10.0.17763.0</TargetPlatformMinVersion>
    <SupportedOSPlatformVersion
Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)')) ==
'tizen'">6.5</SupportedOSPlatformVersion>
  </PropertyGroup>

</Project>
```

In your platform projects, add a reference to this new library project. Then copy your Xamarin.Forms library files into the .NET MAUI library project.

Namespace changes

Namespaces have changed in the move from Xamarin.Forms to .NET MAUI, and Xamarin.Essentials features are now part of .NET MAUI. To make namespace updates, perform a find and replace for the following namespaces:

 Expand table

Xamarin.Forms namespace	.NET MAUI namespace(s)
Xamarin.Forms	Microsoft.Maui and Microsoft.Maui.Controls

Xamarin.Forms namespace	.NET MAUI namespace(s)
<code>Xamarin.Forms.DualScreen</code>	<code>Microsoft.Maui.Controls.Foldable</code>
<code>Xamarin.Forms.Maps</code>	<code>Microsoft.Maui.Controls.Maps</code> and <code>Microsoft.Maui.Maps</code>
<code>Xamarin.Forms.PlatformConfiguration</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration</code>
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific</code>
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific.AppCompat</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific.AppCompat</code>
<code>Xamarin.Forms.PlatformConfiguration.TizenSpecific</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration.TizenSpecific</code>
<code>Xamarin.Forms.PlatformConfiguration.WindowsSpecific</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration.WindowsSpecific</code>
<code>Xamarin.Forms.PlatformConfiguration.iOSSpecific</code>	<code>Microsoft.Maui.Controls.PlatformConfiguration.iOSSpecific</code>
<code>Xamarin.Forms.Shapes</code>	<code>Microsoft.Maui.Controls.Shapes</code>
<code>Xamarin.Forms.StyleSheets</code>	<code>Microsoft.Maui.Controls.StyleSheets</code>
<code>Xamarin.Forms.Xaml</code>	<code>Microsoft.Maui.Controls.Xaml</code>

.NET MAUI projects make use of implicit `global using` directives. This feature enables you to remove `using` directives for the `Xamarin.Essentials` namespace, without having to replace them with the equivalent .NET MAUI namespaces.

In addition, the default XAML namespace has changed from `http://xamarin.com/schemas/2014/forms` in `Xamarin.Forms` to `http://schemas.microsoft.com/dotnet/2021/maui` in .NET MAUI. Therefore, you should replace all occurrences of `xmlns="http://xamarin.com/schemas/2014/forms"` with `xmlns="http://schemas.microsoft.com/dotnet/2021/maui"`.

ⓘ Note

You can quickly update your `Xamarin.Forms` namespaces to `Microsoft.Maui` by using [Quick actions in Visual Studio](#), provided that you have [Upgrade Assistant](#) installed.

API changes

Some APIs have changed in the move from `Xamarin.Forms` to .NET MAUI. This is multiple reasons including removing duplicate functionality caused by `Xamarin.Essentials` becoming part of .NET MAUI, and ensuring that APIs follow .NET naming guidelines. The following sections discuss these changes.

Color changes

In Xamarin.Forms, the `Xamarin.Forms.Color` struct lets you construct `Color` objects using `double` values, and provides named colors, such as `Xamarin.Forms.Color.AliceBlue`. In .NET MAUI, this functionality has been separated into the `Microsoft.Maui.Graphics.Color` class, and the `Microsoft.Maui.Graphics.Colors` class.

The `Microsoft.Maui.Graphics.Color` class, in the `Microsoft.Maui.Graphics` namespace, lets you construct `Color` objects using `float` values, `byte` values, and `int` values. The `Microsoft.Maui.Graphics.Colors` class, which is also in the `Microsoft.Maui.Graphics` namespace, largely provides the same named colors. For example, use `Colors.AliceBlue` to specify the `AliceBlue` color.

The following table shows the API changes between the `Xamarin.Forms.Color` struct and the `Microsoft.Maui.Graphics.Color` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Color.R</code>	<code>Microsoft.Maui.Graphics.Color.Red</code>	
<code>Xamarin.Forms.Color.G</code>	<code>Microsoft.Maui.Graphics.Color.Green</code>	
<code>Xamarin.Forms.Color.B</code>	<code>Microsoft.Maui.Graphics.Color.Blue</code>	
<code>Xamarin.Forms.Color.A</code>	<code>Microsoft.Maui.Graphics.Color.Alpha</code>	
<code>Xamarin.Forms.Color.Hue</code>	<code>Microsoft.Maui.Graphics.Color.GetHue</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Saturation</code>	<code>Microsoft.Maui.Graphics.Color.GetSaturation</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Luminosity</code>	<code>Microsoft.Maui.Graphics.Color.GetLuminosity</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Default</code>		No .NET MAUI equivalent. Instead, <code>Microsoft.Maui.Graphics.Color</code> objects default to <code>null</code> .
<code>Xamarin.Forms.Color.Accent</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Color.FromHex</code>	<code>Microsoft.Maui.Graphics.Color.FromArgb</code>	<code>Microsoft.Maui.Graphics.Color.FromHex</code> is obsolete and will be removed in a future release.

In addition, all of the numeric values in a `Microsoft.Maui.Graphics.Color` are `float`, rather than `double` as used in `Xamarin.Forms.Color`.

 **Note**

Unlike Xamarin.Forms, a `Microsoft.Maui.Graphics.Color` doesn't have an implicit conversion to `System.Drawing.Color`.

Layout changes

The following table lists the layout APIs that have been removed in the move from Xamarin.Forms to .NET MAUI:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.AbsoluteLayout</code> <code>out.IAbsoluteList<T>.Add</code>		The <code>Add</code> overload that accepts 3 arguments isn't present in .NET MAUI.
<code>Xamarin.Forms.Grid.IGridList<T>.AddHorizontal</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Grid.IGridList<T>.AddVertical</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.RelativeLayout</code> <code>out</code>	Microsoft.Maui.Controls.Compatibility.RelativeLayout	In .NET MAUI, <code>RelativeLayout</code> only exists as a compatibility control for users migrating from Xamarin.Forms. Use Grid instead, or add the <code>xmlns</code> for the compatibility namespace.

In addition, adding children to a layout in code in Xamarin.Forms is accomplished by adding the children to the layout's `Children` collection:

C#

```
Grid grid = new Grid();
grid.Children.Add(new Label { Text = "Hello world" });
```

In .NET MAUI, the `Children` collection is for internal use by .NET MAUI and shouldn't be manipulated directly. Therefore, in code children should be added directly to the layout:

C#

```
Grid grid = new Grid();
grid.Add(new Label { Text = "Hello world" });
```

 Important

Any `Add` layout extension methods, such as [GridExtensions.Add](#), are invoked on the layout rather than the layouts `Children` collection.

You may notice when running your upgraded .NET MAUI app that layout behavior is different. For more information, see [Layout behavior changes from Xamarin.Forms](#).

Custom layout changes

The process for creating a custom layout in Xamarin.Forms involves creating a class that derives from `Layout<View>`, and overriding the `VisualElement.OnMeasure` and `Layout.LayoutChildren` methods. For

more information, see [Create a custom layout in Xamarin.Forms](#).

In .NET MAUI, the layout classes derive from the abstract [Layout](#) class. This class delegates cross-platform layout and measurement to a layout manager class. Each layout manager class implements the [ILayoutManager](#) interface, which specifies that [Measure](#) and [ArrangeChildren](#) implementations must be provided:

- The [Measure](#) implementation calls [IView.Measure](#) on each view in the layout, and returns the total size of the layout given the constraints.
- The [ArrangeChildren](#) implementation determines where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view with its appropriate bounds. The return value is the actual size of the layout.

For more information, see [Custom layouts](#).

Device changes

Xamarin.Forms has a `Xamarin.Forms.Device` class that helps you to interact with the device and platform the app is running on. The equivalent class in .NET MAUI, [Microsoft.Maui.Controls.Device](#), is deprecated and its functionality is replaced by multiple types.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Device` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.Android</code>	Microsoft.Maui.Devices.DevicePlatform.Android	
<code>Xamarin.Forms.Device.iOS</code>	Microsoft.Maui.Devices.DevicePlatform.iOS	
<code>Xamarin.Forms.Device.GTK</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.macOS</code>		No .NET MAUI equivalent. Instead, use Microsoft.Maui.Devices.DevicePlatform.MacCatalyst .
<code>Xamarin.Forms.Device.Tizen</code>	Microsoft.Maui.Devices.DevicePlatform.Tizen	
<code>Xamarin.Forms.Device.UWP</code>	Microsoft.Maui.Devices.DevicePlatform.Windows	
<code>Xamarin.Forms.Device.WPF</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.Flags</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.FlowDirection</code>	Microsoft.Maui.ApplicationModel.Application.RequestedLayoutDirection	
<code>Xamarin.Forms.Device.Idiom</code>	Microsoft.Maui.Devices.DeviceInfo.Idiom	

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.IsInvokeRequired</code>	Microsoft.Maui.Dispatching.Dispatcher.IsDispatchRequired	
<code>Xamarin.Forms.Device.OS</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.RuntimePlatform</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.BeginInvokeOnMainThread</code>	Microsoft.Maui.ApplicationModel.MainThread.BeginInvokeOnMainThread	
<code>Xamarin.Forms.Device.GetMainThreadSynchronizationContextAsync</code>	Microsoft.Maui.ApplicationModel.MainThread.GetMainThreadSynchronizationContextAsync	
<code>Xamarin.Forms.Device.GetNamedColor</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.GetNamedSize</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.Invalidated</code>	Microsoft.Maui.Controls.VisualElement.InvalidateMeasure	
<code>Xamarin.Forms.Device.InvokeOnMainThreadAsync</code>	Microsoft.Maui.ApplicationModel.MainThread.InvokeOnMainThreadAsync	
<code>Xamarin.Forms.Device.OnPlatform</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.OpenUri</code>	Microsoft.Maui.ApplicationModel.Launcher.OpenAsync	
<code>Xamarin.Forms.Device.SetFlags</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.SetFlowDirection</code>	Microsoft.Maui.Controls.Window.FlowDirection	
<code>Xamarin.Forms.Device.StartTimer</code>	Microsoft.Maui.Dispatching.DispatcherExtensions.StartTimer or Microsoft.Maui.Dispatching.Dispatcher.DispatchDelayed	

Map changes

In Xamarin.Forms, the `Map` control and associated types are in the `Xamarin.Forms.Maps` namespace. In .NET MAUI, this functionality has moved to the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Maps](#) namespaces. Some properties have been renamed and some types have been replaced with equivalent types from Xamarin.Essentials.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Maps` namespace:

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Maps.Map.HasScrollEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsScrollEnabled	
<code>Xamarin.Forms.Maps.Map.HasZoomEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsZoomEnabled	
<code>Xamarin.Forms.Maps.Map.TrafficEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsTrafficEnabled	
<code>Xamarin.Forms.Maps.Map.MoveToLastRegionOnLayoutChange</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Maps.Pin.Id</code>	Microsoft.Maui.Controls.Maps.Pin.MarkerId	
<code>Xamarin.Forms.Maps.Pin.Position</code>	Microsoft.Maui.Controls.Maps.Pin.Location	
<code>Xamarin.Forms.Maps.MapClickedEventArgs.Position</code>	Microsoft.Maui.Controls.Maps.MapClickedEventArgs.Location	
<code>Xamarin.Forms.Maps.Position</code>	Microsoft.Maui.Devices.Sensors.Location	Members of type <code>Xamarin.Forms.Maps.Position</code> have changed to the Microsoft.Maui.Devices.Sensors.Location type.
<code>Xamarin.Forms.Maps.Geocoder</code>	Microsoft.Maui.Devices.Sensors.Geocoding	Members of type <code>Xamarin.Forms.Maps.Geocoder</code> have changed to the Microsoft.Maui.Devices.Sensors.Geocoding type.

.NET MAUI has two `Map` types - [Microsoft.Maui.Controls.Maps.Map](#) and [Microsoft.Maui.ApplicationModel.Map](#). Because the [Microsoft.Maui.ApplicationModel](#) namespace is one of .NET MAUI's `global using` directives, when using the [Microsoft.Maui.Controls.Maps.Map](#) control from code you'll have to fully qualify your `Map` usage or use a [using alias](#).

In XAML, an `xmlns` namespace definition should be added for the [Map](#) control. While this isn't required, it prevents a collision between the `Polygon` and `Polyline` types, which exist in both the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Controls.Shapes](#) namespaces. For more information, see [Display a map](#).

Other changes

A small number of other APIs have been consolidated in the move from Xamarin.Forms to .NET MAUI. The following table shows these changes:

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Application.Properties</code>	Microsoft.Maui.Storage.Preferences	
<code>Xamarin.Forms.Button.Image</code>	Microsoft.Maui.Controls.Button.ImageSource	
<code>Xamarin.Forms.Frame.OutlineColor</code>	Microsoft.Maui.Controls.Frame.BorderColor	
<code>Xamarin.Forms.IQueryAttributes.ApplyQueryAttributes</code>	Microsoft.Maui.Controls.IQueryAttribute.ApplyQueryAttributes	In Xamarin.Forms, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, string></code> argument. In .NET MAUI, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, object></code> argument.
<code>Xamarin.Forms.MenuItem.Icon</code>	Microsoft.Maui.Controls.MenuItem.IconImageSource	<code>Xamarin.Forms.MenuItem.Icon</code> is the base class for <code>Xamarin.Forms.ToolbarItem</code> , and so <code>ToolbarItem.Icon</code> becomes <code>ToolbarItem.IconImageSource</code> .
<code>Xamarin.Forms.OrientationStateTrigger.Orientation</code>	Microsoft.Maui.Controls.OrientationStateTrigger.Orientation	In Xamarin.Forms, the <code>OrientationStateTrigger.Orientation</code> property is of type <code>Xamarin.Forms.Internals.DeviceOrientation</code> . In .NET MAUI, the <code>OrientationStateTrigger.Orientation</code> property is of type DisplayOrientation .
<code>Xamarin.Forms.OSAppTheme</code>	Microsoft.Maui.ApplicationModel.AppTheme	
<code>Xamarin.Forms.Span.ForegroundColor</code>	Microsoft.Maui.Controls.Span.TextColor	
<code>Xamarin.Forms.ToolbarItem.Name</code>	Microsoft.Maui.Controls.MenuItem.Text	Microsoft.Maui.Controls.MenuItem.Text is the base class for Microsoft.Maui.Controls.ToolbarItem , and so <code>ToolbarItem.Name</code> becomes <code>ToolbarItem.Text</code> .

In addition, in Xamarin.Forms, the `Page.OnAppearing` override is called on Android when an app is backgrounded and then brought to the foreground. However, this override isn't called on iOS and Windows in the same scenario. In .NET MAUI, the [OnAppearing\(\)](#) override isn't called on any platforms when an app is backgrounded and then brought to the foreground. Instead, you should listen to lifecycle events on [Window](#) to be notified when an app returns to the foreground. For more information, see [.NET MAUI windows](#).

Native forms changes

[Native forms](#) in Xamarin.Forms has become native embedding in .NET MAUI, and uses a different initialization approach and different extension methods to convert cross-platform controls to their native types. For more information, see [Native embedding](#).

Bootstrap your migrated app

When manually updating a Xamarin.Forms app to .NET MAUI you will need to enable .NET MAUI support in each platform project, update each platform project's entry point class, and then configure the bootstrapping of your .NET MAUI app.

Enable .NET MAUI in platform projects

Before you update each platform project's entry point class, you must first enable .NET MAUI support. This can be achieved by setting the `$(UseMaui)` build property to `true` in each platform project:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    ...
    <UseMaui>true</UseMaui>
  </PropertyGroup>
</Project>
```

Important

You must add `<UseMaui>true</UseMaui>` to your project file to enable .NET MAUI support. In addition, ensure you've added `<EnableDefaultMauiItems>false</EnableDefaultMauiItems>` to your WinUI project file. This will stop you receiving build errors about the `InitializeComponent` method already being defined.

Add package references

In .NET 8, .NET MAUI ships as a .NET workload and multiple NuGet packages. The advantage of this approach is that it enables you to easily pin your projects to specific versions, while also enabling you to easily preview unreleased or experimental builds.

You should add the following explicit package references to an `<ItemGroup>` in each project file:

XML

```
<PackageReference Include="Microsoft.Maui.Controls" Version="$(MauiVersion)" />
<PackageReference Include="Microsoft.Maui.Controls.Compatibility"
  Version="$(MauiVersion)" />
```

The `$(MauiVersion)` variable is referenced from the version of .NET MAUI you've installed. You can override this by adding the `$(MauiVersion)` build property to each project file:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <UseMaui>True</UseMaui>
    <MauiVersion>8.0.3</MauiVersion>
```

```
</PropertyGroup>
</Project>
```

Android project configuration

In your .NET MAUI Android project, update the `MainApplication` class to match the code below:

```
C#

using System;
using Android.App;
using Android.Runtime;
using Microsoft.Maui;
using Microsoft.Maui.Hosting;
using YOUR_MAU_CLASS_LIB_HERE;

namespace YOUR_NAMESPACE_HERE.Droid
{
    [Application]
    public class MainApplication : MauiApplication
    {
        public MainApplication(IntPtr handle, JniHandleOwnership ownership) :
        base(handle, ownership)
        {
        }

        protected override MauiApp CreateMauiApp() => MauiProgram.CreateMauiApp();
    }
}
```

Also update the `MainActivity` class to inherit from `MauiAppCompatActivity`:

```
C#

using System;
using Microsoft.Maui;
using Android.App;
using Android.Content.PM;
using Android.Runtime;
using Android.OS;

namespace YOUR_NAMESPACE_HERE.Droid
{
    [Activity(Label = "MyTitle", Icon = "@mipmap/icon", Theme = "@style/MainTheme",
    MainLauncher = true, ConfigurationChanges = ConfigChanges.ScreenSize |
    ConfigChanges.Orientation | ConfigChanges.UiMode | ConfigChanges.ScreenLayout |
    ConfigChanges.SmallestScreenSize)]
    public class MainActivity : MauiAppCompatActivity
    {
        protected override void OnCreate(Bundle savedInstanceState)
        {
            base.OnCreate(savedInstanceState);
        }
    }
}
```

Then, update your manifest file to specify that the `minSdkVersion` is 21, which is the minimum Android SDK version required by .NET MAUI. This can be achieved by modifying the `<uses-sdk />` node, which is a child of the `<manifest>` node:

XML

```
<uses-sdk android:minSdkVersion="21" android:targetSdkVersion="32" />
```

iOS project configuration

In your .NET MAUI iOS project, update the `AppDelegate` class to inherit from `MauiUIApplicationDelegate`:

C#

```
using System;
using System.Collections.Generic;
using System.Linq;
using Microsoft.Maui;
using Foundation;
using UIKit;
using YOUR_MAUI_CLASS_LIB_HERE;

namespace YOUR_NAMESPACE_HERE.iOS
{
    [Register("AppDelegate")]
    public partial class AppDelegate : MauiUIApplicationDelegate
    {
        protected override MauiApp CreateMauiApp() => MauiProgram.CreateMauiApp();
    }
}
```

Then, update *Info.plist* so that `MinimumOSVersion` is 11.0, which is the minimum iOS version required by .NET MAUI.

UWP project configuration

In your .NET MAUI WinUI 3 project, update *App.xaml* to match the code below:

XAML

```
<?xml version="1.0" encoding="utf-8"?>
<maui:MauiWinUIApplication
    x:Class="YOUR_NAMESPACE_HERE.WinUI.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:maui="using:Microsoft.Maui"
    xmlns:local="using:YOUR_NAMESPACE_HERE.WinUI">
    <maui:MauiWinUIApplication.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <XamlControlsResources xmlns="using:Microsoft.UI.Xaml.Controls" />
                <!-- Other merged dictionaries here -->
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </maui:MauiWinUIApplication.Resources>
</maui:MauiWinUIApplication>
```

```
<!-- Other app resources here -->
</ResourceDictionary>
</maui:MauiWinUIApplication.Resources>
</maui:MauiWinUIApplication>
```

ⓘ Note

If your project included resources in your existing *App.xaml* you should migrate them to the new version of the file.

Also, update *App.xaml.cs* to match the code below:

C#

```
using Microsoft.Maui;
using Microsoft.Maui.Hosting;
using YOUR_MAU_CLASS_LIB_HERE;

namespace YOUR_NAMESPACE_HERE.WinUI;

public partial class App : MauiWinUIApplication
{
    public App()
    {
        InitializeComponent();
    }

    protected override MauiApp CreateMauiApp() => MauiProgram.CreateMauiApp();
}
```

ⓘ Note

If your project included business logic in *App.xaml.cs* you should migrate that logic to the new version of the file.

Then add a *launchSettings.json* file to the *Properties* folder of the project, and add the following JSON to the file:

JSON

```
{
  "profiles": {
    "Windows Machine": {
      "commandName": "MsixPackage",
      "nativeDebugging": true
    }
  }
}
```

App entry point

.NET MAUI apps have a single cross-platform app entry point. Each platform entry point calls a `CreateMauiApp` method on the static `MauiProgram` class, and returns a [MauiApp](#).

Therefore, add a new class named `MauiProgram` that contains the following code:

```
C#

namespace YOUR_NAMESPACE_HERE;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>();

        return builder.Build();
    }
}
```

⚠ Note

For Xamarin.Forms UWP projects, the `App` reference in `builder.UseMauiApp<App>()` can be found in the *MainPage.xaml.cs* file.

If there are platform specific services that need to be migrated to .NET MAUI, use the [AddTransient\(IServiceCollection, Type\)](#) method to add a transient service of the specified type to the specified `IServiceCollection`.

⚠ Note

You can quickly update your `Xamarin.Forms` namespaces to `Microsoft.Maui` by using [Quick actions in Visual Studio](#), provided that you have [Upgrade Assistant](#) installed.

AssemblyInfo changes

Properties that are typically set in an *AssemblyInfo.cs* file are now available in your SDK-style project. We recommend migrating them from *AssemblyInfo.cs* to your project file in every project, and removing the *AssemblyInfo.cs* file.

Optionally, you can keep the *AssemblyInfo.cs* file and set the `GenerateAssemblyInfo` property in your project file to `false`:

```
XML

<PropertyGroup>
  <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
</PropertyGroup>
```

For more information about the `GenerateAssemblyInfo` property, see [GenerateAssemblyInfo](#).

Update app dependencies

Generally, Xamarin.Forms NuGet packages are not compatible with .NET 8 unless they have been recompiled using .NET target framework monikers (TFMs). However, Android apps can use NuGet packages targeting the `monoandroid` and `monoandroidXX.X` frameworks.

You can confirm a package is .NET 8 compatible by looking at the **Frameworks** tab on [NuGet](#) for the package you're using, and checking that it lists one of the compatible frameworks shown in the following table:

 Expand table

Compatible frameworks	Incompatible frameworks
net8.0-android, monoandroid, monoandroidXX.X	
net8.0-ios	monotouch, xamarinios, xamarinios10
net8.0-macos	monomac, xamarinmac, xamarinmac20
net8.0-tvos	xamarintvos
	xamarinwatchos

 Note

.NET Standard libraries that have no dependencies on the incompatible frameworks listed above are still compatible with .NET 8.

If a package on [NuGet](#) indicates compatibility with any of the compatible frameworks above, regardless of also including incompatible frameworks, then the package is compatible. Compatible NuGet packages can be added to your .NET MAUI library project using the NuGet package manager in Visual Studio.

If you can't find a .NET 8 compatible version of a NuGet package you should:

- Recompile the package with .NET TFMs, if you own the code.
- Look for a preview release of a .NET 8 version of the package.
- Replace the dependency with a .NET 8 compatible alternative.

Compile and troubleshoot

Once your dependencies are resolved, you should build your project. Any errors will guide you towards next steps.

 Tip

- Delete all *bin* and *obj* folders from all projects before opening and building projects in Visual Studio, particularly when changing .NET versions.
- Delete the *Resource.designer.cs* generated file from the Android project.

The following table provides guidance for overcoming common build or runtime issues:

 Expand table

Issue	Tip
Xamarin.* namespace doesn't exist.	Update the namespace to its .NET MAUI equivalent. For more information, see Namespace changes .
API doesn't exist.	Update the API usage to its .NET MAUI equivalent. For more information, see API changes .
App won't deploy.	Ensure that the required platform project is set to deploy in Visual Studio's Configuration Manager.
App won't launch.	Update each platform project's entry point class, and the app entry point. For more information, see Bootstrap your migrated app .
CollectionView doesn't scroll.	Check the container layout and the measured size of the CollectionView . By default the control will take up as much space as the container allows. A Grid constrains children at its own size. However a StackLayout enables children to take up space beyond its bounds.
Pop-up is displayed under the page on iOS.	In Xamarin.Forms, all pop-ups on iOS are <code>UIWindow</code> instances but in .NET MAUI pop-ups are displayed by locating the current presenting <code>ViewController</code> and displaying the pop-up with <code>PresentViewControllerAsync</code> . In plugins such as Mopups, to ensure that your pop-ups are correctly displayed you should call DisplayAlert , DisplayActionSheet , or DisplayPromptAsync from the ContentPage that's used inside the <code>Mopup</code> popup.
BoxView not appearing.	The default size of a BoxView in Xamarin.Forms is 40x40. The default size of a BoxView in .NET MAUI is 0x0. Set <code>WidthRequest</code> and <code>HeightRequest</code> to 40.
Layout is missing padding, margin, or spacing.	Add default values to your project based on the .NET MAUI style resource. For more information, see Default value changes from Xamarin.Forms .
Custom layout doesn't work.	Custom layout code needs updating to work in .NET MAUI. For more information, see Custom layout changes .
Custom renderer doesn't work.	Renderer code needs updating to work in .NET MAUI. For more information, see Use custom renderers in .NET MAUI .
Effect doesn't work.	Effect code needs updating to work in .NET MAUI. For more information, see Use effects in .NET MAUI .
SkiaSharp code doesn't work.	SkiaSharp code needs minor updates to work in .NET MAUI. For more information, see Reuse SkiaSharp code in .NET MAUI .
Can't access previously created app properties data.	Migrate the app properties data to .NET MAUI preferences. For more information, see Migrate data from the Xamarin.Forms app properties dictionary to .NET MAUI preferences .

Issue	Tip
Can't access previously created secure storage data.	Migrate the secure storage data to .NET MAUI. For more information, see Migrate from Xamarin.Essentials secure storage to .NET MAUI secure storage .
Can't access previously created version tracking data.	Migrate the version tracking data to .NET MAUI. For more information, see Migrate version tracking data from a Xamarin.Forms app to a .NET MAUI app .

See also

- [Porting from .NET Framework to .NET](#)
- [.NET Upgrade Assistant](#)

Manually upgrade a Xamarin.Forms app to a single project .NET MAUI app

Article • 05/07/2024

To migrate a Xamarin.Forms app to a single project .NET Multi-platform App UI (.NET MAUI) app, you must:

- ✓ Update your Xamarin.Forms app to use Xamarin.Forms 5.
- ✓ Update the app's dependencies to the latest versions.
- ✓ Ensure the app still works.
- ✓ Create a .NET MAUI app.
- ✓ Copy code and configuration from the Xamarin.Forms app to the .NET MAUI app.
- ✓ Copy resources from your Xamarin.Forms app to the .NET MAUI app.
- ✓ Update namespaces.
- ✓ Address any API changes.
- ✓ Upgrade or replace incompatible dependencies with .NET 8 versions.
- ✓ Compile and test your app.

To simplify the upgrade process, you should create a new .NET MAUI app of the same name as your Xamarin.Forms app, and then copy in your code, configuration, and resources. This is the approach outlined below.

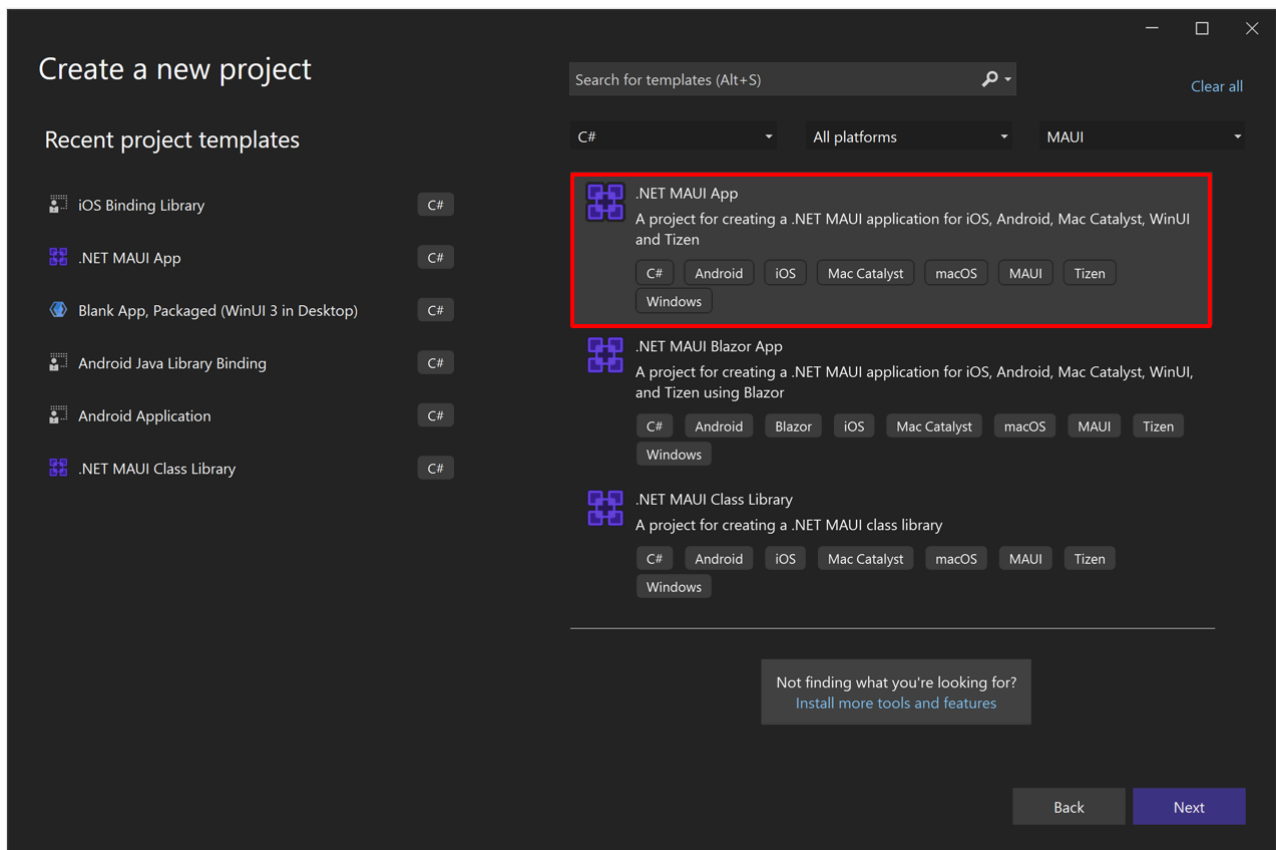
Update your Xamarin.Forms app

Before upgrading your Xamarin.Forms app to .NET MAUI, you should first update your Xamarin.Forms app to use Xamarin.Forms 5 and ensure that it still runs correctly. In addition, you should update the dependencies that your app uses to the latest versions.

This will help to simplify the rest of the migration process, as it will minimize the API differences between Xamarin.Forms and .NET MAUI, and will ensure that you are using .NET compatible versions of your dependencies if they exist.

Create a .NET MAUI app

In Visual Studio, create a new .NET MAUI app using the same name as your Xamarin.Forms app:



Opening the project file will confirm that you have a .NET SDK-style project.

Copy code to the .NET MAUI app

All of the cross-platform code from your `Xamarin.Forms` library project should be copied into your .NET MAUI app project in identically named folders and files.

Custom renderers can either be reused in a .NET MAUI app, or migrated to a .NET MAUI handler. For more information, see [Reuse custom renderers in .NET MAUI](#) and [Migrate a Xamarin.Forms custom renderer to a .NET MAUI handler](#).

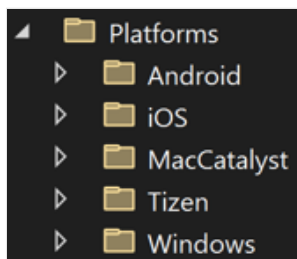
Effects can be reused in a .NET MAUI app. For more information, see [Reuse effects](#).

ⓘ Note

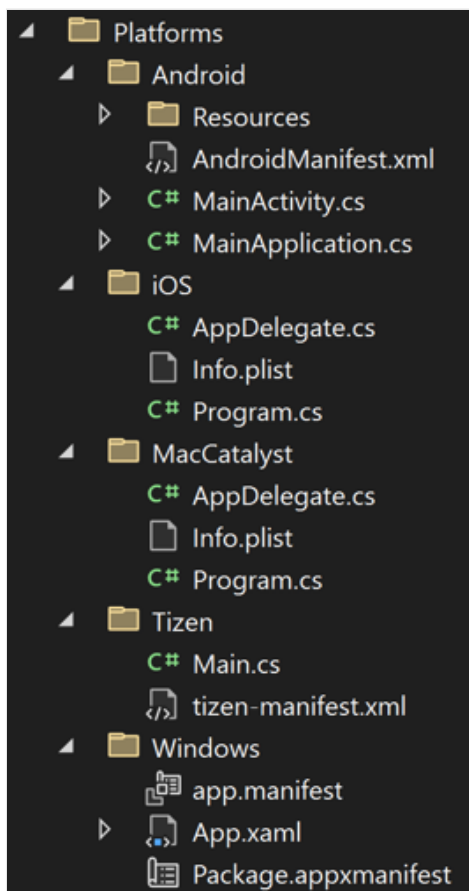
You can quickly update your `Xamarin.Forms` namespaces to `Microsoft.Maui` by using [Quick actions in Visual Studio](#), provided that you have [Upgrade Assistant](#) installed.

Platform-specific code

A .NET MAUI app project contains a *Platforms* folder, with each child folder representing a platform that .NET MAUI can target:



The folders for each platform contain platform-specific resources, and code that starts the app on each platform:



Code, and their containing folders, from your Xamarin.Forms head projects should be copied to these folders:

- Code from your Xamarin.Forms Android head project should be copied to the *Platform\Android* folder of your .NET MAUI app project. In addition, copy any custom code from your Xamarin.Forms `MainActivity` and `MainApplication` classes to the same classes in your .NET MAUI app project.
- Code from your Xamarin.Forms iOS head project should be copied to the *Platforms\iOS* folder of your .NET MAUI app project. In addition, copy any custom code from your Xamarin.Forms `AppDelegate` class to the same class in your .NET MAUI app project.

ⓘ Note

For a list of breaking changes in .NET for iOS, see [Breaking changes in .NET for iOS](#).

- Code from your Xamarin.Forms UWP head project should be copied to the *Platforms\Windows* folder of your .NET MAUI app project. In addition, copy any custom code from your Xamarin.Forms `App` class to the same class in your .NET MAUI app project.

At build time, the build system only includes the code from each folder when building for that specific platform. For example, when you build for Android the files in the *Platforms\Android* folder will be built into the app package, but the files in the other *Platforms* folders won't be. This approach uses multi-targeting to target multiple platforms from a single project. .NET MAUI apps can also be multi-targeted based on your own filename and folder criteria. This enables you to structure your .NET MAUI app project so that you don't have to place your platform code into child-folders of the *Platforms* folder. For more information, see [Configure multi-targeting](#).

Copy configuration to the .NET MAUI app

Each platform uses its own native app manifest file to specify information such as the app title, ID, version, and more. .NET MAUI single project enables you to specify this common app data in a single location in the project file.

To specify the shared app manifest data for a project, open the shortcut menu for the project in **Solution Explorer**, and then choose **Properties**. The app title, ID, and version can then be specified in **MAUI Shared > General**:

The screenshot shows the 'MAUI Shared' properties window with the 'General' tab selected. The left sidebar lists project properties: Application, Build, Package, Code Analysis, Debug, Resources, MAUI Shared (expanded), General (selected), Android, and iOS. The main area displays the following settings:

Property	Description	Value
Application Title	The display name of the application.	MauiApp1
Application ID	The identifier of the application in reverse domain name format e.g. com.microsoft.maui.	com.companyname.mauiapp1
Application ID (GUID)	The identifier of the application in GUID format e.g. DA583B0B-9202-42A7-B68C-87225195B0D2.	442A62EB-F23C-4286-AF14-D7551EF08428
Application Display Version	The display version of the application. This should be at most a three part version number e.g. 1.0.0.	1.0
Application Version	The version of the application. This should be a single digit integer e.g. 1.	1

At build time the shared app manifest data is merged with platform-specific data in the native app manifest file, to produce the manifest file for the app package. For more information, see [Project configuration in .NET MAUI - MAUI Shared](#).

The remaining data from your Xamarin.Forms app manifests should be copied to your .NET MAUI app manifest:

- On Android, copy any additional data from the *AndroidManifest.xml* file in your Xamarin.Forms Android head project, to the *Platforms\Android\AndroidManifest.xml* file in your .NET MAUI app project.
- On iOS, copy any additional data from the *Info.plist* file in your Xamarin.Forms iOS head project, to the *Platforms\iOS\Info.plist* file in your .NET MAUI app project. In addition, copy the *Entitlements.plist* file in your Xamarin.Forms iOS head project to the *Platforms\iOS* folder in your .NET MAUI app project.
- On Windows, copy additional data from the *Package.appxmanifest* file in your Xamarin.Forms UWP head project, to the *Platforms\Windows\Package.appxmanifest* file in your .NET MAUI app project.

Copy resources to the .NET MAUI app

.NET MAUI single project enables resource files to be stored in a single location while being consumed on each platform. This includes fonts, images, the app icon, the splash screen, raw assets, and CSS files for styling .NET MAUI apps.

Resource files should typically be placed in the *Resources* folder of your .NET MAUI app project, or child folders of the *Resources* folder, and must have their build action set correctly. The following table shows the build actions for each resource file type:

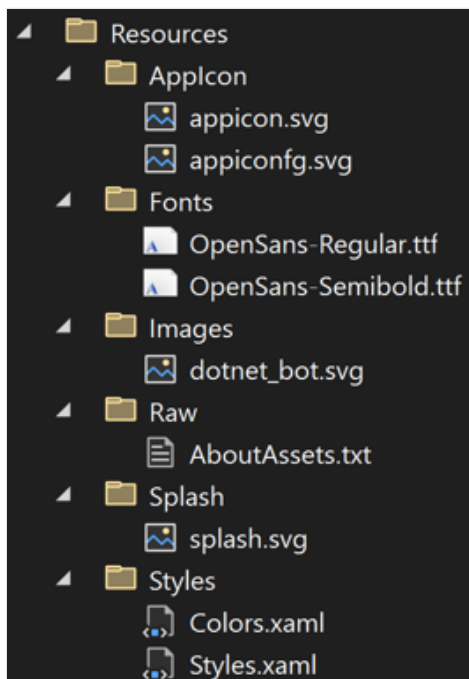
 Expand table

Resource	Build action
App icon	MauIcon
Fonts	MauFont
Images	MauImage
Splash screen	MauSplashScreen
Raw assets	MauAsset
CSS files	MauCss

Note

XAML files are also stored in your .NET MAUI app project, and are automatically assigned the **MauXaml** build action. However, only XAML resource dictionaries will typically be placed in the *Resources* folder of the app project.

The following screenshot shows a typical *Resources* folder containing child-folders for each resource type:



The build action for a resource file will be correctly set, if the resource has been added to the correct *Resources* child folder.

Important

Platform-specific resources will override their shared resource counterparts. For example, if you have an Android-specific image located at *Platforms\Android\Resources\drawable-xhdpi\logo.png*, and you also provide a shared *Resources\Images\logo.svg* image, the Scalable Vector Graphics (SVG) file will be used to generate the required Android images, except for the XHDPI image that already exists as a platform-specific image.

App icons

Your Xamarin.Forms app icon should be added to your .NET MAUI app project by dragging the image into the *Resources\Applcon* folder of the project, where its build action will automatically be set to **MauiIcon**. At build time, the app icon is resized to the correct sizes for the target platform and device. App icons are resized to multiple resolutions because they have multiple uses, including being used to represent the app on the device, and in the app store.

For more information, see [Add an app icon to a .NET MAUI app project](#).

Splash screen

If your Xamarin.Forms app has a splash screen, it should be added to your .NET MAUI app project by dragging the image into the *Resources\Splash* folder of the project, where its build action will automatically be set to **MauiSplashScreen**. At build time, the splash screen image is resized to the correct size for the target platform and device.

For more information, see [Add a splash screen to a .NET MAUI app project](#).

Images

Devices have a range of screen sizes and densities and each platform has functionality for displaying density-dependent images. In Xamarin.Forms, density-dependent images are typically placed in head projects and adopt a platform-specific naming convention. There are two approaches that can be taken to migrate these images to .NET MAUI.

The recommended approach is to copy the highest resolution version of each image from your Xamarin.Forms solution to your .NET MAUI app project by dragging it into the *Resources\Images* folder of the project, where its build action will automatically be set to **MauiImage**. It will also be necessary to set the `BaseSize` attribute of each bitmap image, to ensure that resizing occurs. This eliminates the need to have multiple versions of each image, on each platform. At build time, any images will then be resized into multiple density-dependent images that meet platform requirements. For more information, see [Add images to a .NET MAUI app project](#).

Alternatively, you could copy density-dependent images from your Xamarin.Forms solution to identically named folders in the *Platforms\{Platform}* folder of your .NET MAUI app project, and set their build actions to the build actions that are used in your Xamarin.Forms solution. The following table lists example image locations for a Xamarin.Forms solution, and their equivalent location in a .NET MAUI app project:

 Expand table

Xamarin.Forms image location	.NET MAUI image location	.NET MAUI platform image build action
<i>{MyApp.Android}\Resources\drawable-xhdpi\image.png</i>	<i>Platforms\Android\Resources\drawable-xhdpi\image.png</i>	AndroidResource
<i>{MyApp.iOS}\image.jpg</i>	<i>*Platforms\iOS\Resources\image.jpg</i>	BundleResource
<i>{MyApp.UWP}\Assets\Images\image.gif</i>	<i>*Platforms\Windows\Assets\Images\image.gif</i>	Content

Provided that you've adopted the same image naming convention as used in your Xamarin.Forms solution, the appropriate image will be chosen at runtime based on the device's capabilities. The disadvantage of this approach is that you still have multiple versions of each image on each platform.

Fonts

Any fonts from your Xamarin.Forms solution can be added to your .NET MAUI solution by dragging them into the *Resources\Fonts* folder of your .NET MAUI app project, where their build action will automatically be set to **MauiFont**.

For more information, see [Fonts](#).

CSS files

Any CSS files from your Xamarin.Forms solution can be added to your .NET MAUI solution by dragging them into an identically named folder, and setting their build action to **MauiCss** in the **Properties**

window.

For more information about using CSS files in a .NET MAUI app, see [Style apps using Cascading Style Sheets](#).

Raw assets

Any raw asset files, such as HTML, JSON, and video, should be copied from your Xamarin.Forms solution to your .NET MAUI app project by dragging them into the *Resources\Raw* folder of your project, where their build action will automatically be set to **MauiAsset**.

Localized resources

In a .NET MAUI app, strings are localized using the same approach as in a Xamarin.Forms app. Therefore, your .NET resource files (*.resx*) should be copied from your Xamarin.Forms solution to an identically named folder in your .NET MAUI solution. Then, the neutral language of your .NET MAUI app must be specified. For more information, see [Specify the app's neutral language](#).

ⓘ Note

.NET resource files don't have to be placed in the *Resources* folder of your .NET MAUI app project.

In a .NET MAUI app, images are localized using the same approach as in a Xamarin.Forms app. Therefore, your localized images, and the folders in which they reside, should be copied from your Xamarin.Forms solution to your .NET MAUI app project:

- On Android, the root folder in your .NET MAUI app project for localized images is *Platforms\Android\Resources*.
- On iOS, the root folder in your .NET MAUI app project for localized images is *Platforms\iOS\Resources*.
- On Windows, the root folder in your .NET MAUI app project for localized images is *Platforms\Windows\Assets\Images*.

Localized images should have their build actions set to the build actions that are used in your Xamarin.Forms solution. For more information, see [Localize images](#).

In a .NET MAUI app, app names are localized using the same approach as in a Xamarin.Forms app:

- On Android, the localized app name can be stored using a folder-based naming convention in the *Platforms\Android\Resources* folder. App name localization folders and files should be copied to this folder from your Xamarin.Forms solution.
- On iOS, the localized app name is stored using a folder-based naming convention in the *Platforms\iOS\Resources* folder. App name localization folders and files should be copied to this folder from your Xamarin.Forms solution.
- On Windows, the localized app name is stored in the app package manifest.

For more information, see [Localize the app name](#). For more information about .NET MAUI app localization, see [Localization](#).

Namespace changes

Namespaces have changed in the move from Xamarin.Forms to .NET MAUI, and Xamarin.Essentials features are now part of .NET MAUI. To make namespace updates, perform a find and replace for the following namespaces:

 Expand table

Xamarin.Forms namespace	.NET MAUI namespace(s)
<code>Xamarin.Forms</code>	Microsoft.Maui and Microsoft.Maui.Controls
<code>Xamarin.Forms.DualScreen</code>	Microsoft.Maui.Controls.Foldable
<code>Xamarin.Forms.Maps</code>	Microsoft.Maui.Controls.Maps and Microsoft.Maui.Maps
<code>Xamarin.Forms.PlatformConfiguration</code>	Microsoft.Maui.Controls.PlatformConfiguration
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific
<code>Xamarin.Forms.PlatformConfiguration.AndroidSpecific.AppCompat</code>	Microsoft.Maui.Controls.PlatformConfiguration.AndroidSpecific.AppCompat
<code>Xamarin.Forms.PlatformConfiguration.TizenSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.TizenSpecific
<code>Xamarin.Forms.PlatformConfiguration.WindowsSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.WindowsSpecific
<code>Xamarin.Forms.PlatformConfiguration.iOSSpecific</code>	Microsoft.Maui.Controls.PlatformConfiguration.iOSSpecific
<code>Xamarin.Forms.Shapes</code>	Microsoft.Maui.Controls.Shapes
<code>Xamarin.Forms.StyleSheets</code>	Microsoft.Maui.Controls.StyleSheets
<code>Xamarin.Forms.Xaml</code>	Microsoft.Maui.Controls.Xaml

.NET MAUI projects make use of implicit `global using` directives. This feature enables you to remove `using` directives for the `Xamarin.Essentials` namespace, without having to replace them with the equivalent .NET MAUI namespaces.

In addition, the default XAML namespace has changed from `http://xamarin.com/schemas/2014/forms` in Xamarin.Forms to `http://schemas.microsoft.com/dotnet/2021/maui` in .NET MAUI. Therefore, you should replace all occurrences of `xmlns="http://xamarin.com/schemas/2014/forms"` with `xmlns="http://schemas.microsoft.com/dotnet/2021/maui"`.

Note

You can quickly update your `Xamarin.Forms` namespaces to `Microsoft.Maui` by using [Quick actions in Visual Studio](#), provided that you have [Upgrade Assistant](#) installed.

API changes

Some APIs have changed in the move from Xamarin.Forms to .NET MAUI. This is multiple reasons including removing duplicate functionality caused by Xamarin.Essentials becoming part of .NET MAUI, and ensuring that APIs follow .NET naming guidelines. The following sections discuss these changes.

Color changes

In Xamarin.Forms, the `Xamarin.Forms.Color` struct lets you construct `Color` objects using `double` values, and provides named colors, such as `Xamarin.Forms.Color.AliceBlue`. In .NET MAUI, this functionality has been separated into the `Microsoft.Maui.Graphics.Color` class, and the `Microsoft.Maui.Graphics.Colors` class.

The `Microsoft.Maui.Graphics.Color` class, in the `Microsoft.Maui.Graphics` namespace, lets you construct `Color` objects using `float` values, `byte` values, and `int` values. The `Microsoft.Maui.Graphics.Colors` class, which is also in the `Microsoft.Maui.Graphics` namespace, largely provides the same named colors. For example, use `Colors.AliceBlue` to specify the `AliceBlue` color.

The following table shows the API changes between the `Xamarin.Forms.Color` struct and the `Microsoft.Maui.Graphics.Color` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Color.R</code>	<code>Microsoft.Maui.Graphics.Color.Red</code>	
<code>Xamarin.Forms.Color.G</code>	<code>Microsoft.Maui.Graphics.Color.Green</code>	
<code>Xamarin.Forms.Color.B</code>	<code>Microsoft.Maui.Graphics.Color.Blue</code>	
<code>Xamarin.Forms.Color.A</code>	<code>Microsoft.Maui.Graphics.Color.Alpha</code>	
<code>Xamarin.Forms.Color.Hue</code>	<code>Microsoft.Maui.Graphics.Color.GetHue</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Saturation</code>	<code>Microsoft.Maui.Graphics.Color.GetSaturation</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Luminosity</code>	<code>Microsoft.Maui.Graphics.Color.GetLuminosity</code>	Xamarin.Forms property replaced with a method in .NET MAUI.
<code>Xamarin.Forms.Color.Default</code>		No .NET MAUI equivalent. Instead, <code>Microsoft.Maui.Graphics.Color</code> objects default to <code>null</code> .
<code>Xamarin.Forms.Color.Accent</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Color.FromHex</code>	<code>Microsoft.Maui.Graphics.Color.FromArgb</code>	<code>Microsoft.Maui.Graphics.Color.FromHex</code> is obsolete and will be removed in a future release.

In addition, all of the numeric values in a `Microsoft.Maui.Graphics.Color` are `float`, rather than `double` as used in `Xamarin.Forms.Color`.

 **Note**

Unlike `Xamarin.Forms`, a `Microsoft.Maui.Graphics.Color` doesn't have an implicit conversion to `System.Drawing.Color`.

Layout changes

The following table lists the layout APIs that have been removed in the move from `Xamarin.Forms` to `.NET MAUI`:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.AbsoluteLayout</code> <code>out.IAbsoluteList<T>.Add</code>		The <code>Add</code> overload that accepts 3 arguments isn't present in <code>.NET MAUI</code> .
<code>Xamarin.Forms.Grid.IGridList<T>.AddHorizontal</code>		No <code>.NET MAUI</code> equivalent.
<code>Xamarin.Forms.Grid.IGridList<T>.AddVertical</code>		No <code>.NET MAUI</code> equivalent.
<code>Xamarin.Forms.RelativeLayout</code> <code>out</code>	Microsoft.Maui.Controls.Compatibility.RelativeLayout	In <code>.NET MAUI</code> , <code>RelativeLayout</code> only exists as a compatibility control for users migrating from <code>Xamarin.Forms</code> . Use <code>Grid</code> instead, or add the <code>xmlns</code> for the compatibility namespace.

In addition, adding children to a layout in code in `Xamarin.Forms` is accomplished by adding the children to the layout's `Children` collection:

C#

```
Grid grid = new Grid();
grid.Children.Add(new Label { Text = "Hello world" });
```

In `.NET MAUI`, the `Children` collection is for internal use by `.NET MAUI` and shouldn't be manipulated directly. Therefore, in code children should be added directly to the layout:

C#

```
Grid grid = new Grid();
grid.Add(new Label { Text = "Hello world" });
```

 **Important**

Any `Add` layout extension methods, such as [GridExtensions.Add](#), are invoked on the layout rather than the layouts [Children](#) collection.

You may notice when running your upgraded .NET MAUI app that layout behavior is different. For more information, see [Layout behavior changes from Xamarin.Forms](#).

Custom layout changes

The process for creating a custom layout in Xamarin.Forms involves creating a class that derives from `Layout<View>`, and overriding the `VisualElement.OnMeasure` and `Layout.LayoutChildren` methods. For more information, see [Create a custom layout in Xamarin.Forms](#).

In .NET MAUI, the layout classes derive from the abstract [Layout](#) class. This class delegates cross-platform layout and measurement to a layout manager class. Each layout manager class implements the [ILayoutManager](#) interface, which specifies that [Measure](#) and [ArrangeChildren](#) implementations must be provided:

- The [Measure](#) implementation calls [IView.Measure](#) on each view in the layout, and returns the total size of the layout given the constraints.
- The [ArrangeChildren](#) implementation determines where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view with its appropriate bounds. The return value is the actual size of the layout.

For more information, see [Custom layouts](#).

Device changes

Xamarin.Forms has a `Xamarin.Forms.Device` class that helps you to interact with the device and platform the app is running on. The equivalent class in .NET MAUI, [Microsoft.Maui.Controls.Device](#), is deprecated and its functionality is replaced by multiple types.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Device` class:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.Android</code>	Microsoft.Maui.Devices.DevicePlatform.Android	
<code>Xamarin.Forms.Device.iOS</code>	Microsoft.Maui.Devices.DevicePlatform.iOS	
<code>Xamarin.Forms.Device.GTK</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.macOS</code>		No .NET MAUI equivalent. Instead, use Microsoft.Maui.Devices.DevicePlatform.MacCatalyst .

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Device.Tizen</code>	Microsoft.Maui.Devices.DevicePlatform.Tizen	
<code>Xamarin.Forms.Device.UWP</code>	Microsoft.Maui.Devices.DevicePlatform.WinUI	
<code>Xamarin.Forms.Device.WPF</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.Flags</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.FlowDirection</code>	Microsoft.Maui.ApplicationModel.Application.RequestedLayoutDirection	
<code>Xamarin.Forms.Device.Idiom</code>	Microsoft.Maui.Devices.DeviceInfo.Idiom	
<code>Xamarin.Forms.Device.IsInvokeRequired</code>	Microsoft.Maui.Dispatching.Dispatcher.IsDispatchRequired	
<code>Xamarin.Forms.Device.OS</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.RuntimePlatform</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.BeginInvokeOnMainThread</code>	Microsoft.Maui.ApplicationModel.MainThread.BeginInvokeOnMainThread	
<code>Xamarin.Forms.Device.GetMainThreadSynchronizationContextAsync</code>	Microsoft.Maui.ApplicationModel.MainThread.GetMainThreadSynchronizationContextAsync	
<code>Xamarin.Forms.Device.GetNamedColor</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.GetNamedSize</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.Invalidate</code>	Microsoft.Maui.Controls.VisualElement.InvalidateMeasure	
<code>Xamarin.Forms.Device.InvokeOnMainThreadAsync</code>	Microsoft.Maui.ApplicationModel.MainThread.InvokeOnMainThreadAsync	
<code>Xamarin.Forms.Device.OnPlatform</code>	Microsoft.Maui.Devices.DeviceInfo.Platform	
<code>Xamarin.Forms.Device.OpenUri</code>	Microsoft.Maui.ApplicationModel.Launcher.OpenAsync	
<code>Xamarin.Forms.Device.SetFlags</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Device.SetFlowDirection</code>	Microsoft.Maui.Controls.Window.FlowDirection	
<code>Xamarin.Forms.Device.StartTimer</code>	Microsoft.Maui.Dispatching.DispatcherExtensions.StartTimer or Microsoft.Maui.Dispatching.DispatcherExtensions.StartTimer	

Xamarin.Forms API	.NET MAUI API	Comments
	patching.Dispatcher.DispatchDelayed	

Map changes

In Xamarin.Forms, the `Map` control and associated types are in the `Xamarin.Forms.Maps` namespace. In .NET MAUI, this functionality has moved to the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Maps](#) namespaces. Some properties have been renamed and some types have been replaced with equivalent types from Xamarin.Essentials.

The following table shows the .NET MAUI replacements for the functionality in the `Xamarin.Forms.Maps` namespace:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comment
<code>Xamarin.Forms.Maps.Map.HasScrollEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsScrollEnabled	
<code>Xamarin.Forms.Maps.Map.HasZoomEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsZoomEnabled	
<code>Xamarin.Forms.Maps.Map.TrafficEnabled</code>	Microsoft.Maui.Controls.Maps.Map.IsTrafficEnabled	
<code>Xamarin.Forms.Maps.Map.MoveToLastRegionOnLayoutChange</code>		No .NET MAUI equivalent.
<code>Xamarin.Forms.Maps.Pin.Id</code>	Microsoft.Maui.Controls.Maps.Pin.MarkerId	
<code>Xamarin.Forms.Maps.Pin.Position</code>	Microsoft.Maui.Controls.Maps.Pin.Location	
<code>Xamarin.Forms.Maps.MapClickedEventArgs.Position</code>	Microsoft.Maui.Controls.Maps.MapClickedEventArgs.Location	
<code>Xamarin.Forms.Maps.Position</code>	Microsoft.Maui.Devices.Sensors.Location	Members of type <code>Xamarin.Forms.Maps.Position</code> have changed to the Microsoft.Maui.Devices.Sensors.Location type.
<code>Xamarin.Forms.Maps.Geocoder</code>	Microsoft.Maui.Devices.Sensors.Geocoding	Members of type <code>Xamarin.Forms.Maps.Geocoder</code> have changed to the Microsoft.Maui.Devices.Sensors.Geocoding type.

.NET MAUI has two `Map` types - [Microsoft.Maui.Controls.Maps.Map](#) and [Microsoft.Maui.ApplicationModel.Map](#). Because the [Microsoft.Maui.ApplicationModel](#) namespace is one of .NET MAUI's `global using` directives, when using the [Microsoft.Maui.Controls.Maps.Map](#) control from code you'll have to fully qualify your `Map` usage or use a [using alias](#).

In XAML, an `xmlns` namespace definition should be added for the [Map](#) control. While this isn't required, it prevents a collision between the `Polygon` and `Polyline` types, which exist in both the [Microsoft.Maui.Controls.Maps](#) and [Microsoft.Maui.Controls.Shapes](#) namespaces. For more information, see [Display a map](#).

Other changes

A small number of other APIs have been consolidated in the move from Xamarin.Forms to .NET MAUI. The following table shows these changes:

 Expand table

Xamarin.Forms API	.NET MAUI API	Comments
<code>Xamarin.Forms.Application.Properties</code>	Microsoft.Maui.Storage.Preferences	
<code>Xamarin.Forms.Button.Image</code>	Microsoft.Maui.Controls.Button.ImageSource	
<code>Xamarin.Forms.Frame.OutlineColor</code>	Microsoft.Maui.Controls.Frame.BorderColor	
<code>Xamarin.Forms.IQueryAttributable.ApplyQueryAttributes</code>	Microsoft.Maui.Controls.IQueryAttributable.ApplyQueryAttributes	In Xamarin.Forms, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, string></code> argument. In .NET MAUI, the <code>ApplyQueryAttributes</code> method accepts an <code>IDictionary<string, object></code> argument.
<code>Xamarin.Forms.MenuItem.Icon</code>	Microsoft.Maui.Controls.MenuItem.IconImageSource	<code>Xamarin.Forms.MenuItem.Icon</code> is the base class for <code>Xamarin.Forms.ToolbarItem</code> , and so <code>ToolbarItem.Icon</code> becomes <code>ToolbarItem.IconImageSource</code> .
<code>Xamarin.Forms.OrientationStateTrigger.Orientation</code>	Microsoft.Maui.Controls.OrientationStateTrigger.Orientation	In Xamarin.Forms, the <code>OrientationStateTrigger.Orientation</code> property is of type <code>Xamarin.Forms.Internals.DeviceOrientation</code> . In .NET MAUI, the <code>OrientationStateTrigger.Orientation</code> property is of type DisplayOrientation .
<code>Xamarin.Forms.OSAppTheme</code>	Microsoft.Maui.ApplicationModel.AppTheme	
<code>Xamarin.Forms.Span.ForegroundColor</code>	Microsoft.Maui.Controls.Span.TextColor	
<code>Xamarin.Forms.ToolbarItem.Name</code>	Microsoft.Maui.Controls.MenuItem.Text	Microsoft.Maui.Controls.MenuItem.Text is the base class for Microsoft.Maui.Controls.ToolbarItem , and so <code>ToolbarItem.Name</code> becomes <code>ToolbarItem.Text</code> .

In addition, in Xamarin.Forms, the `Page.OnAppearing` override is called on Android when an app is backgrounded and then brought to the foreground. However, this override isn't called on iOS and Windows in the same scenario. In .NET MAUI, the [OnAppearing\(\)](#) override isn't called on any platforms when an app is backgrounded and then brought to the foreground. Instead, you should listen to

lifecycle events on [Window](#) to be notified when an app returns to the foreground. For more information, see [.NET MAUI windows](#).

Native forms changes

[Native forms](#) in Xamarin.Forms has become native embedding in .NET MAUI, and uses a different initialization approach and different extension methods to convert cross-platform controls to their native types. For more information, see [Native embedding](#).

AssemblyInfo changes

Properties that are typically set in an *AssemblyInfo.cs* file are now available in your SDK-style project. We recommend migrating them from *AssemblyInfo.cs* to your project file in every project, and removing the *AssemblyInfo.cs* file.

Optionally, you can keep the *AssemblyInfo.cs* file and set the `GenerateAssemblyInfo` property in your project file to `false`:

```
XML

<PropertyGroup>
  <GenerateAssemblyInfo>false</GenerateAssemblyInfo>
</PropertyGroup>
```

For more information about the `GenerateAssemblyInfo` property, see [GenerateAssemblyInfo](#).

Update app dependencies

Generally, Xamarin.Forms NuGet packages are not compatible with .NET 8 unless they have been recompiled using .NET target framework monikers (TFMs). However, Android apps can use NuGet packages targeting the `monoandroid` and `monoandroidXX.X` frameworks.

You can confirm a package is .NET 8 compatible by looking at the **Frameworks** tab on [NuGet](#) for the package you're using, and checking that it lists one of the compatible frameworks shown in the following table:

 Expand table

Compatible frameworks	Incompatible frameworks
net8.0-android, monoandroid, monoandroidXX.X	
net8.0-ios	monotouch, xamarinios, xamarinios10
net8.0-macos	monomac, xamarinmac, xamarinmac20
net8.0-tvos	xamarintvos
	xamarinwatchos

⚠ Note

.NET Standard libraries that have no dependencies on the incompatible frameworks listed above are still compatible with .NET 8.

If a package on [NuGet](#) indicates compatibility with any of the compatible frameworks above, regardless of also including incompatible frameworks, then the package is compatible. Compatible NuGet packages can be added to your .NET MAUI library project using the NuGet package manager in Visual Studio.

If you can't find a .NET 8 compatible version of a NuGet package you should:

- Recompile the package with .NET TFMs, if you own the code.
- Look for a preview release of a .NET 8 version of the package.
- Replace the dependency with a .NET 8 compatible alternative.

Compile and troubleshoot

Once your dependencies are resolved, you should build your project. Any errors will guide you towards next steps.

💡 Tip

- Delete all *bin* and *obj* folders from all projects before opening and building projects in Visual Studio, particularly when changing .NET versions.
- Delete the *Resource.designer.cs* generated file from the Android project.

The following table provides guidance for overcoming common build or runtime issues:

[Expand table](#)

Issue	Tip
Xamarin.* namespace doesn't exist.	Update the namespace to its .NET MAUI equivalent. For more information, see Namespace changes .
API doesn't exist.	Update the API usage to its .NET MAUI equivalent. For more information, see API changes .
App won't deploy.	Ensure that the required platform project is set to deploy in Visual Studio's Configuration Manager.
App won't launch.	Update each platform project's entry point class, and the app entry point. For more information, see Bootstrap your migrated app .
CollectionView doesn't scroll.	Check the container layout and the measured size of the CollectionView . By default the control will take up as much space as the container allows. A Grid constrains children at its own size. However a StackLayout enables children to take up space beyond its bounds.

Issue	Tip
Pop-up is displayed under the page on iOS.	In Xamarin.Forms, all pop-ups on iOS are <code>UIWindow</code> instances but in .NET MAUI pop-ups are displayed by locating the current presenting <code>ViewController</code> and displaying the pop-up with <code>PresentViewControllerAsync</code> . In plugins such as Mopups, to ensure that your pop-ups are correctly displayed you should call DisplayAlert , DisplayActionSheet , or DisplayPromptAsync from the <code>ContentPage</code> that's used inside the <code>Mopup</code> popup.
BoxView not appearing.	The default size of a BoxView in Xamarin.Forms is 40x40. The default size of a BoxView in .NET MAUI is 0x0. Set <code>WidthRequest</code> and <code>HeightRequest</code> to 40.
Layout is missing padding, margin, or spacing.	Add default values to your project based on the .NET MAUI style resource. For more information, see Default value changes from Xamarin.Forms .
Custom layout doesn't work.	Custom layout code needs updating to work in .NET MAUI. For more information, see Custom layout changes .
Custom renderer doesn't work.	Renderer code needs updating to work in .NET MAUI. For more information, see Use custom renderers in .NET MAUI .
Effect doesn't work.	Effect code needs updating to work in .NET MAUI. For more information, see Use effects in .NET MAUI .
SkiaSharp code doesn't work.	SkiaSharp code needs minor updates to work in .NET MAUI. For more information, see Reuse SkiaSharp code in .NET MAUI .
Can't access previously created app properties data.	Migrate the app properties data to .NET MAUI preferences. For more information, see Migrate data from the Xamarin.Forms app properties dictionary to .NET MAUI preferences .
Can't access previously created secure storage data.	Migrate the secure storage data to .NET MAUI. For more information, see Migrate from Xamarin.Essentials secure storage to .NET MAUI secure storage .
Can't access previously created version tracking data.	Migrate the version tracking data to .NET MAUI. For more information, see Migrate version tracking data from a Xamarin.Forms app to a .NET MAUI app .

Xamarin.Forms UWP project migration

Article • 12/29/2023

To update your Xamarin.Forms UWP project to a WinUI 3 project, you should:

- ✓ Update your project file to be SDK-style.
- ✓ Update namespaces
- ✓ Address any API changes
- ✓ Update or replace incompatible dependencies with .NET 8 versions.
- ✓ Compile and test your app.

Update to an SDK-style project file

Your existing Xamarin.Forms UWP project can be updated to an SDK-style WinUI 3 project in place. An SDK-style project for a .NET MAUI WinUI 3 app is similar to the following example:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType> <!-- in Xamarin.Forms this was
AppContainerExe -->
    <TargetFramework>net8.0-windows10.0.19041.0</TargetFramework>
    <TargetPlatformMinVersion>10.0.17763.0</TargetPlatformMinVersion>
    <RootNamespace>YOUR_NAMESPACE_HERE.WinUI</RootNamespace>
    <ApplicationManifest>app.manifest</ApplicationManifest>
    <Platforms>x86;x64;ARM64</Platforms>
    <RuntimeIdentifiers>win10-x86;win10-x64;win10-arm64</RuntimeIdentifiers>
    <UseWinUI>true</UseWinUI>
    <EnableMsixTooling>true</EnableMsixTooling>
    <UseMaui>true</UseMaui>
    <!-- We do not want XAML files to be processed as .NET MAUI XAML -->
    <EnableDefaultMauiItems>>false</EnableDefaultMauiItems>
  </PropertyGroup>
  ...
</Project>
```

❗ Important

The target framework moniker (TFM) is what denotes the project as using .NET, in this case .NET 8. For information about target frameworks in SDK-style projects, see [Target frameworks in SDK-style projects](#).

You must add `<UseMaui>true</UseMaui>` to your project file to enable .NET MAUI support. In addition, ensure you've added `<EnableDefaultMauiItems>false</EnableDefaultMauiItems>` to the project file. This will stop you receiving build errors about the `InitializeComponent` method already being defined.

Changes to MSBuild properties

While upgrading your project, it's recommended to remove the following UWP MSBuild properties from your project file:

- `WindowsXamlEnableOverview`
- `AppxPackageSigningEnabled`
- `GenerateAssemblyInfo`

You'll also need to ensure that the platform architectures in the target project are specified with the following .NET runtime identifiers:

XML

```
<PropertyGroup>
  <!-- Used in .NET MAUI WinUI projects -->
  <Platforms>x86;x64;ARM64</Platforms>
</PropertyGroup>
```

For more information about runtime identifiers, see [.NET RID Catalog](#).

Namespace changes

There are differences in the names of namespaces between UWP and WinUI 3. In many cases it's as easy as changing a namespace name and then your code will compile. For example, you'll need to replace the `Windows.UI.Xaml` namespace with the `Microsoft.UI.Xaml` namespace. Similarly, you'll need to replace the `Windows.UI.Xaml.Controls` namespace with the `Microsoft.UI.Xaml.Controls` namespace.

Other times, the mapping takes a bit more work, and in rare cases requires a change in approach. For more information, see [Mapping UWP APIs and libraries to the Windows App SDK](#).

API changes

You'll need to address any API changes that may affect your app. For example, some types, methods, and properties may have been renamed, deprecated or removed. For information on what's supported when upgrading your UWP project to WinUI 3, see [What's supported when migrating from UWP to WinUI 3](#). For information about mapping UWP features and APIs to WinUI 3, see [Mapping UWP features to the Windows App SDK](#) and [Mapping UWP APIs and libraries to the Windows App SDK](#).

Your project can be made compatible with earlier OS versions by setting the `$(SupportedOSPlatformVersion)` property in your project file:

XML

```
<PropertyGroup>
  <SupportedOSPlatformVersion
    Condition="$([MSBuild]::GetTargetPlatformIdentifier('$(TargetFramework)'))
    == 'windows'">10.0.19041.0</SupportedOSPlatformVersion>
</PropertyGroup>
```

The `$(SupportedOSPlatformVersion)` property indicates the minimum OS version required to run your app or library. If you don't explicitly specify this minimum runtime OS version in your project, it defaults to the platform version from the Target Framework Moniker (TFM).

If your project only targets Windows, it's sufficient to omit the platform checking condition and set the `$(SupportedOSPlatformVersion)` property directly:

XML

```
<PropertyGroup>
  <SupportedOSPlatformVersion>10.0.19041.0</SupportedOSPlatformVersion>
</PropertyGroup>
```

For more information about the `$(SupportedOSPlatformVersion)` property, see [Support older OS versions](#).

For an app to run correctly on an older OS version, it can't call APIs that don't exist on that version of the OS. However, you can add guards around calls to newer APIs so they are only called when running on a version of the OS that supports them. This can be achieved with the [IsWindowsVersionAtLeast](#) method:

C#

```
if (OperatingSystem.IsWindowsVersionAtLeast(10))
{
```

```
// Use the API here  
}
```

Remove files

The following files, which are present in Xamarin.Forms UWP projects, don't exist in WinUI 3 projects:

- *MainPage.xaml* and *MainPage.xaml.cs*
- *AssemblyInfo.cs*
- *Default.rd.xml*

Therefore, you should remove these files if they're in your WinUI 3 project. Any required business logic contained in these files should be moved elsewhere.

Changes to Package.appxmanifest

The following changes must be made to your migrated project's *Package.appxmanifest* file:

1. Set the application entry point to `$targetentrypoint$`. For more information, see [Target entry point](#).
2. Add the `runFullTrust` capability. For more information, see [Run full trust capability](#).
3. Add the `Windows.Universal` and `Windows.Desktop` target device families. For more information, see [Universal target device family](#) and [Desktop target device family](#).

Making these changes fixes common deployment errors for your app on Windows.

For an example of a compliant *Package.appxmanifest* file, see [Package.appxmanifest](#).

Runtime behavior

There are behavioral changes to the `String.IndexOf()` method in .NET 5+ on different platforms. For more information, see [.NET globalization and ICU](#).

Next steps

[Bootstrap your app](#)

See also

- [Manually upgrade a Xamarin.Forms app to a multi-project .NET MAUI app](#)
- [Manually upgrade a Xamarin.Forms app to a single project .NET MAUI app](#)

Layout behavior changes from Xamarin.Forms

Article • 07/18/2024

You may notice when running your upgraded .NET Multi-platform App UI (.NET MAUI) app that layout behavior is different. Some of this is the result of changes to layout spacing values. For more information, see [Default value changes from Xamarin.Forms](#).

The following table shows additional behavior changes between layouts in Xamarin.Forms and .NET MAUI:

 Expand table

Layout	Xamarin.Forms	.NET MAUI	Recommendation
All	In certain cases sizing requests aren't honored.	Sizing requests are honored.	
Grid	Columns and rows can be inferred from XAML.	Columns and rows must be explicitly declared.	Add ColumnDefinitions and RowDefinitions .
HorizontalStackLayout		<code>*AndExpand</code> has no effect.	
RelativeLayout		Requires the compatibility namespace.	Use Grid instead, or add the <code>xmlns</code> for the compatibility namespace.
StackLayout	Children can fill space in the stacking direction.	Children are stacked and will go beyond available space.	If you need child views to fill space, change to a Grid .
VerticalStackLayout		<code>*AndExpand</code> has no effect.	

.NET MAUI controls generally honour explicit size requests. If you ask a control to be 200 device-independent units wide, then .NET MAUI will make that control 200 units wide, even if the control's container is only 100 units wide.

Default layout value changes from Xamarin.Forms

Xamarin.Forms uses arbitrary default values for some property values, such as padding, margins, and spacing. .NET MAUI changes these arbitrary property values to zero.

To preserve the Xamarin.Forms default values in projects that don't set explicit values, add implicit styles to your project. For more information about implicit styles, see [Implicit styles](#).

ⓘ Note

The .NET MAUI project template includes resource dictionaries that provide default styles for most controls. It's recommended you take a similar approach in your apps, by modifying or inheriting from these [resource dictionaries](#).

The following table lists the layout property values that have changed between Xamarin.Forms and .NET MAUI:

 Expand table

Property	Xamarin.Forms value	.NET MAUI value
Grid.ColumnSpacing	6	0
Grid.RowSpacing	6	0
StackLayout.Spacing	6	0

The following styles retain the Xamarin.Forms defaults:

XML

```
<!-- Forms defaults -->
<Style TargetType="Grid">
  <Setter Property="ColumnSpacing" Value="6"/>
  <Setter Property="RowSpacing" Value="6"/>
</Style>
<Style TargetType="StackLayout">
  <Setter Property="Spacing" Value="6"/>
</Style>
<Style TargetType="Frame">
  <Setter Property="Padding" Value="{OnPlatform 20,iOS=19}"/>
</Style>
```

Frame

[Frame](#) has been replaced in .NET MAUI by [Border](#). However, it's included to ease migration from Xamarin.Forms. .NET MAUI layout correctly measures [Frame](#) `Padding` across all platforms, whereas Xamarin.Forms had some discrepancies across platforms. This may result in your apps not looking the same in .NET MAUI. The example above accounts for this if you're using default values.

Grid

The biggest change in [Grid](#) behavior between Xamarin.Forms and .NET MAUI is that grids don't automatically add missing rows and columns for you. For example, in Xamarin.Forms you could add controls to a [Grid](#) without specifying their row behavior:

XML

```
<Grid>
  <Label Text="Hello"/>
  <Label Grid.Row="1" Text="World"/>
</Grid>
```

In Xamarin.Forms, despite not declaring that the [Grid](#) contains two rows, a second row would be automatically added for you. .NET MAUI doesn't do this. Instead, you have to explicitly specify how many rows are in the [Grid](#) with the `RowDefinitions` property.

❗ Important

By default, .NET MAUI creates a [Grid](#) with one column and one row. Therefore, it's not necessary to set the `ColumnDefinitions` and `RowDefinitions` properties if this is your intention.

In Xamarin.Forms, when a `Label` is in a column where the width of its `ColumnDefinition` is set to `Auto`, line breaks such as word wrap and tail truncation implicitly occur. In .NET MAUI, in this scenario line breaks don't implicitly occur because the column expands past the width of the screen to accommodate the child's content. If you want the [Label](#) to wrap at the edge of the [Grid](#) you should set the appropriate `ColumnDefinition` to either `*` or a value.

StackLayout

There are several differences between the stack layouts in .NET MAUI ([StackLayout](#), [VerticalStackLayout](#), and [HorizontalStackLayout](#)) and the `StackLayout` in Xamarin.Forms.

The main difference is that .NET MAUI stack layouts are very simple. They stack their child views in a single direction until all of them have been stacked. They will keep going until the last child has been stacked, even if that takes them beyond the available space in the stacking direction. Therefore, .NET MAUI stack layouts arrange controls in a particular direction. They do not subdivide a space. This is completely different to the Xamarin.Forms `StackLayout`, which changes its layout behavior based on circumstances and the presence of any `*AndExpand` layout options, such as `FillAndExpand` or `CenterAndExpand`. The Xamarin.Forms `StackLayout` sometimes subdivides the space, expanding to or stopping at the edge of its container. In other cases, it expands beyond its container.

The new stack layouts in .NET MAUI, [HorizontalStackLayout](#) and [VerticalStackLayout](#), do not recognize the `*AndExpand` layout options. If they encounter a child with such layout options, they simply treat it as if the `AndExpand` wasn't there. For example, `FillAndExpand` becomes `Fill`. However, for simplicity of migration from Xamarin.Forms, the .NET MAUI [StackLayout](#) does honor the `*AndExpand` layout options, although they've been marked as obsolete. To avoid warnings about using obsolete members, you should convert your layouts that use `*AndExpand` layout options to the appropriate layout type. This can be achieved as follows:

1. If your layout is anything other than a [StackLayout](#), remove all uses of `AndExpand`. Just as in Xamarin.Forms, in .NET MAUI the `AndExpand` layout options have no effect on any layout other than [StackLayout](#).
2. Remove any `AndExpand` properties which are orthogonal to the stacking direction. For example, if you have a [StackLayout](#) with an `Orientation` of `Vertical`, and it has a child with a `HorizontalAlignment="CenterAndExpand"` - that layout options has no effect and can be removed.
3. If you have any remaining `AndExpand` properties on a [StackLayout](#), you should convert that [StackLayout](#) to a [Grid](#). A [Grid](#) is designed to subdivide a space, and will provide the layout that `AndExpand` provided in Xamarin.Forms. The following example shows a Xamarin.Forms `StackLayout` that uses an `AndExpand` property:

XAML

```
<StackLayout>
  <Label Text="Hello world!"/>
```

```
<Image VerticalOptions="FillAndExpand" Source="dotnetbot.png"/>
</StackLayout>
```

This can be converted to a [Grid](#) in .NET MAUI:

XAML

```
<Grid RowDefinitions="Auto, *">
    <Label Text="Hello world!"/>
    <Image Grid.Row="1" Source="dotnetbot.png"/>
</Grid>
```

When performing this conversion, anything that was marked `AndExpand` in the `StackLayout` should go in its own row or column with a size of `*` in the [Grid](#).

❗ Important

A [StackLayout](#) continues in its stacking direction until it runs out of content. It does not subdivide its container along that axis. If you want to limit your content to a constrained space in a direction, you should use another layout such as a [Grid](#).

RelativeLayout

Use of [RelativeLayout](#) is not recommended in .NET MAUI. Instead, use a [Grid](#) wherever possible.

If you absolutely require a [RelativeLayout](#), it can be found in the `Microsoft.Maui.Controls.Compatibility` namespace:

XML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:compat="clr-namespace:Microsoft.Maui.Controls.Compatibility;assembly=Microsoft.Maui.Controls"
    x:Class="MyMauiApp.MyPage"
    Title="MyPage">
    <compat:RelativeLayout>
        <!-- Your code goes here -->
    </compat:RelativeLayout>
</ContentPage>
```


ScrollView

While [ScrollView](#) often isn't considered to be a layout, it can be thought of as a layout as it's used to scroll its child content. In Xamarin.Forms, `ScrollView` doesn't behave consistently when stacking. It has some arbitrary limits on minimum size that depend partially on its content, and it will sometimes compress to enable other items to fit on the page inside a `StackLayout` in ways that are inconsistent and sometimes surprising.

In .NET MAUI, the [ScrollView](#) expands to whatever size it wants to be unless otherwise constrained. This means that inside of a [VerticalStackLayout](#), which can expand infinitely, a [ScrollView](#) will expand to its full content height and doesn't scroll. This behavior can be confusing if you're a Xamarin.Forms user.

Reuse custom renderers in .NET MAUI

Article • 04/13/2023

 [Browse the sample](#)

While there are many benefits to using .NET Multi-platform App UI (.NET MAUI) handlers to customize and create controls, it's still possible to use Xamarin.Forms custom renderers in .NET MAUI apps. For more information about custom renderers, see [Xamarin.Forms custom renderers](#).

Shimmed renderers

.NET MAUI provides shimmed renderers that enable easy re-use of Xamarin.Forms custom renderers, provided that the renderer derives from `FrameRenderer`, `ListViewRenderer`, `ShellRenderer` on iOS and Android, `TableViewController`, and `VisualElementRenderer`.

The process for migrating a Xamarin.Forms custom renderer that derives from `FrameRenderer`, `ListViewRenderer`, `ShellRenderer`, `TableViewController`, and `VisualElementRenderer` to a .NET MAUI shimmed renderer is to:

1. Add the custom renderer code into the appropriate location in your .NET MAUI project(s). For more information, see [Add the code](#).
2. Modify the `using` directives and remove `ExportRenderer` attributes. For more information, see [Modify using directives and other code](#).
3. Register the renderers. For more information, see [Register renderers](#).
4. Consume the renderers. For more information, see [Consume the custom renderers](#).

To demonstrate using custom renderers in .NET MAUI, consider a Xamarin.Forms control named `PressableView`. This control exposes `Pressed` and `Released` events based on platform-specific gestures. The custom renderer implementation is composed of 3 files:

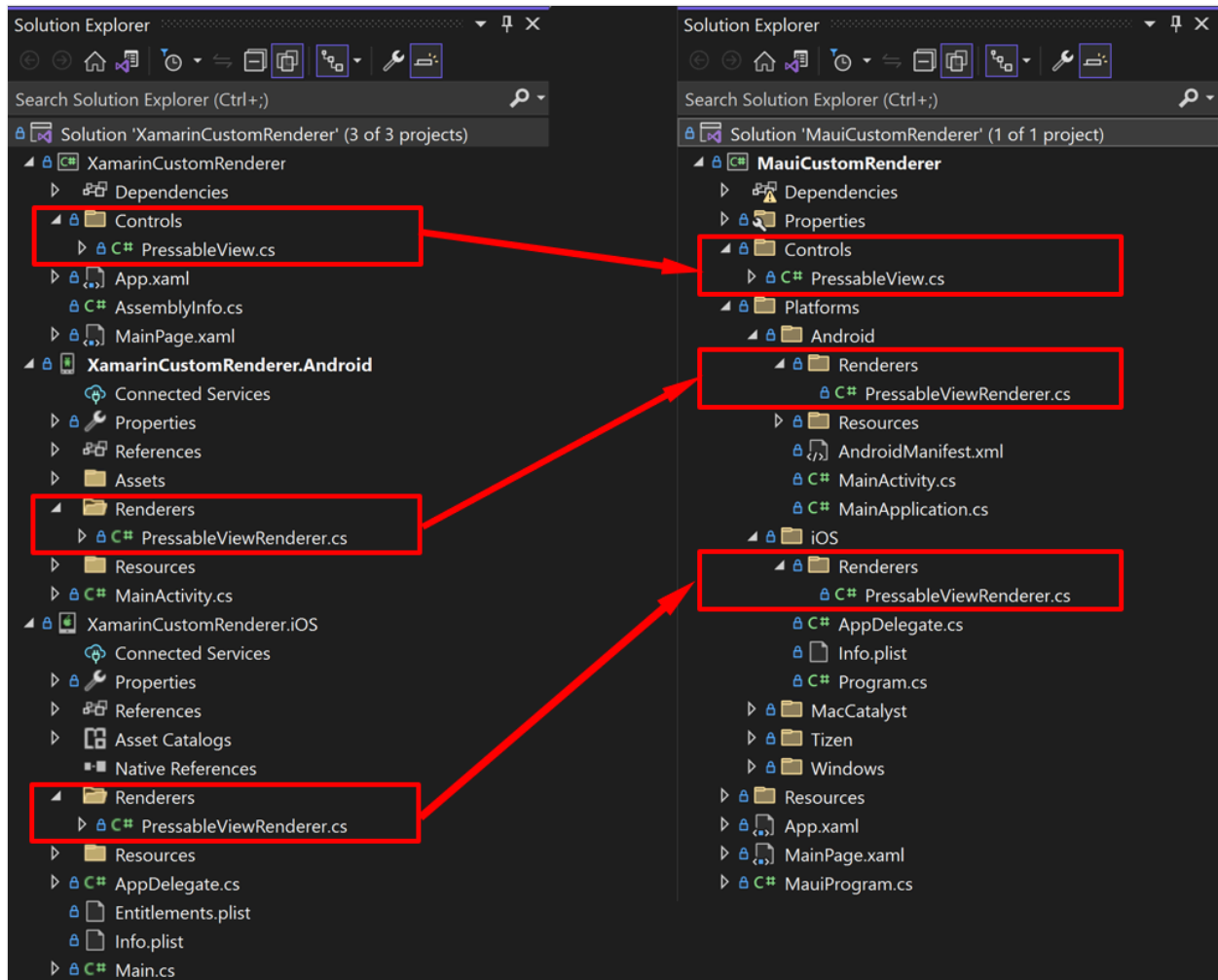
- `PressableView.cs` - the cross-platform class that extends `ContentView`.
- `PressableViewRenderer.cs` - the Android implementation, which derives from `VisualElementRenderer`.
- `PressableViewRenderer.cs` - the iOS implementation, which derives from `VisualElementRenderer`.

 **Note**

An alternative to using a Xamarin.Forms custom renderer in .NET MAUI is to migrate the custom renderer to a .NET MAUI handler. For more information, see [Migrate a Xamarin.Forms custom renderer to a .NET MAUI handler](#).

Add the code

If you're using a .NET MAUI multi-targeted project, the cross-platform file can be moved to anywhere outside the *Platforms* folder, and the platform-specific implementation files should be moved to the corresponding *Platform* folder:



If your solution has separate projects per-platform, then you should move the platform-specific implementation files into the corresponding projects.

Modify using directives and other code

Any reference to the `Xamarin.Forms.*` namespaces need to be removed, and then you can resolve the related types to `Microsoft.Maui.*`. This needs to occur in all files you've added to the .NET MAUI project(s).

You should also remove any `ExportRenderer` attributes as they won't be needed in .NET MAUI. For example, the following should be removed:

C#

```
[assembly: ExportRenderer(typeof(PressableView),
    typeof(PressableViewRenderer))]
```

Register renderers

The cross-platform control and its renderers must be registered with an app before it can be consumed. This should occur in the `CreateMauiApp` method in the `MauiProgram` class in your app project, which is the cross-platform entry point for the app:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            })
            .ConfigureMauiHandlers((handlers) =>
            {
                #if ANDROID
                    handlers.AddHandler(typeof(PressableView),
                        typeof(XamarinCustomRenderer.Droid.Renderers.PressableViewRenderer));
                #elif IOS
                    handlers.AddHandler(typeof(PressableView),
                        typeof(XamarinCustomRenderer.iOS.Renderers.PressableViewRenderer));
                #endif
            });

        return builder.Build();
    }
}
```

The renderers are registered with the `ConfigureMauiHandlers` and `AddHandler` method. This first argument to the `AddHandler` method is the cross-platform control type, with the second argument being its renderer type.

Important

Only renderers that derive from `FrameRenderer`, `ListViewRenderer`, `NavigationRenderer` on iOS, `ShellRenderer` on iOS and Android, `TabbedRenderer` on iOS, `TableViewController`, and `VisualElementRenderer` can be registered with the [AddHandler](#) method.

Consume the custom renderers

The custom renderer can be consumed in a .NET MAUI app as a custom control:

XML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-namespace:XamarinCustomRenderer.Controls"
             x:Class="MauiCustomRenderer.MainPage">
    <Grid BackgroundColor="#f1f1f1">
        <controls:PressableView Pressed="Handle_Pressed"
                               Released="Handle_Released"
                               HorizontalOptions="Center"
                               VerticalOptions="Center">
            <Grid BackgroundColor="#202020"
                  HorizontalOptions="Center"
                  VerticalOptions="Center">
                <Label Text="Press Me"
                      FontSize="16"
                      TextColor="White"
                      Margin="24,20"
                      HorizontalTextAlignment="Center" />
            </Grid>
        </controls:PressableView>
    </Grid>
</ContentPage>
```

Migrate a Xamarin.Forms custom renderer to a .NET MAUI handler

Article • 10/31/2024

In Xamarin.Forms, custom renderers can be used to customize the appearance and behavior of a control, and create new cross-platform controls. Each custom renderer has a reference to the cross-platform control and often relies on `INotifyPropertyChanged` to send property change notifications. Rather than using custom renderers, .NET Multi-platform App UI (.NET MAUI) introduces a new concept called a *handler*.

Handlers offer many performance improvements over custom renderers. In Xamarin.Forms, the `ViewRenderer` class creates a parent element. For example, on Android, a `ViewGroup` is created which is used for auxiliary positioning tasks. In .NET MAUI, the `ViewHandler<TVirtualView,TPlatformView>` class doesn't create a parent element, which helps to reduce the size of the visual hierarchy and improve your app's performance. Handlers also decouple platform controls from the framework. The platform control only needs to handle the needs of the framework. This is not only more efficient, but it's much easier to extend or override when required. Handlers are also suitable for reuse by other frameworks such as [Comet](#) and [Fabulous](#). For more information about handlers, see [Handlers](#).

In Xamarin.Forms, the `OnElementChanged` method in a custom renderer creates the platform control, initializes default values, subscribes to events, and handles the Xamarin.Forms element the renderer was attached to (`OldElement`) and the element that the renderer is attached to (`NewElement`). In addition, a single `OnElementPropertyChanged` method defines the operations to invoke when a property change occurs in the cross-platform control. .NET MAUI simplifies this approach, so that every property change is handled by a separate method, and so that code to create the platform control, perform control setup, and perform control cleanup, is separated into distinct methods.

The process for migrating a Xamarin.Forms custom control that's backed by custom renderers on each platform to a .NET MAUI custom control that's backed by a handler on each platform is as follows:

1. Create a class for the cross-platform control, which provides the control's public API. For more information, see [Create the cross-platform control](#).
2. Create a `partial` handler class. For more information, see [Create the handler](#).
3. In the handler class, create a `PropertyMapper` dictionary, which defines the Actions to take when cross-platform property changes occur. For more information, see [Create the property mapper](#).

4. Create `partial` handler classes for each platform that create the native views that implement the cross-platform control. For more information, see [Create the platform controls](#).
5. Register the handler using the [ConfigureMauiHandlers](#) and [AddHandler](#) methods in your app's `MauiProgram` class. For more information, see [Register the handler](#).

Then, the cross-platform control can be consumed. For more information, see [Consume the cross-platform control](#).

Alternatively, custom renderers that customize Xamarin.Forms controls can be converted so that they modify .NET MAUI handlers. For more information, see [Customize controls with handlers](#).

Create the cross-platform control

To create a cross-platform control, you should create a class that derives from [View](#):

C#

```
namespace MyMauiControl.Controls
{
    public class CustomEntry : View
    {
        public static readonly BindableProperty TextProperty =
            BindableProperty.Create(nameof(Text), typeof(string),
            typeof(CustomEntry), null);

        public static readonly BindableProperty TextColorProperty =
            BindableProperty.Create(nameof(TextColor), typeof(Color),
            typeof(CustomEntry), null);

        public string Text
        {
            get { return (string)GetValue(TextProperty); }
            set { SetValue(TextProperty, value); }
        }

        public Color TextColor
        {
            get { return (Color)GetValue(TextColorProperty); }
            set { SetValue(TextColorProperty, value); }
        }
    }
}
```

The control should provide a public API that will be accessed by its handler, and control consumers. Cross-platform controls should derive from [View](#), which represents a visual

element that's used to place layouts and views on the screen.

Create the handler

After creating your cross-platform control, you should create a `partial` class for your handler:

C#

```
#if IOS || MACCATALYST
using PlatformView = Microsoft.Maui.Platform.MauiTextField;
#elif ANDROID
using PlatformView = AndroidX.AppCompat.Widget.AppCompatEditText;
#elif WINDOWS
using PlatformView = Microsoft.UI.Xaml.Controls.TextBox;
#elif (NETSTANDARD || !PLATFORM) || (NET6_0_OR_GREATER && !IOS && !ANDROID)
using PlatformView = System.Object;
#endif
using MyMauiControl.Controls;
using Microsoft.Maui.Handlers;

namespace MyMauiControl.Handlers
{
    public partial class CustomEntryHandler
    {
    }
}
```

The handler class is a partial class whose implementation will be completed on each platform with an additional partial class.

The conditional `using` statements define the `PlatformView` type on each platform. The final conditional `using` statement defines `PlatformView` to be equal to `System.Object`. This is necessary so that the `PlatformView` type can be used within the handler for usage across all platforms. The alternative would be to have to define the `PlatformView` property once per platform, using conditional compilation.

Create the property mapper

Each handler typically provides a *property mapper*, which defines what Actions to take when a property change occurs in the cross-platform control. The `PropertyMapper` type is a `Dictionary` that maps the cross-platform control's properties to their associated Actions.

❗ Note

The property mapper is the replacement for the `OnElementPropertyChanged` method in a Xamarin.Forms custom renderer.

`PropertyMapper` is defined in .NET MAUI's `ViewHandler<TVirtualView,TPlatformView>` class, and requires two generic arguments to be supplied:

- The class for the cross-platform control, which derives from `View`.
- The class for the handler.

The following code example shows the `CustomEntryHandler` class extended with the `PropertyMapper` definition:

C#

```
public partial class CustomEntryHandler
{
    public static PropertyMapper<CustomEntry, CustomEntryHandler>
    PropertyMapper = new PropertyMapper<CustomEntry, CustomEntryHandler>
    (ViewHandler.ViewMapper)
    {
        [nameof(CustomEntry.Text)] = MapText,
        [nameof(CustomEntry.TextColor)] = MapTextColor
    };

    public CustomEntryHandler() : base(PropertyMapper)
    {
    }
}
```

The `PropertyMapper` is a `Dictionary` whose key is a `string` and whose value is a generic `Action`. The `string` represents the cross-platform control's property name, and the `Action` represents a `static` method that requires the handler and cross-platform control as arguments. For example, the signature of the `MapText` method is `public static void MapText(CustomEntryHandler handler, CustomEntry view)`.

Each platform handler must provide implementations of the Actions, which manipulate the native view APIs. This ensures that when a property is set on a cross-platform control, the underlying native view will be updated as required. The advantage of this approach is that it allows for easy cross-platform control customization, because the property mapper can be modified by cross-platform control consumers without subclassing. For more information, see [Customize controls with handlers](#).

Create the platform controls

After creating the mappers for your handler, you must provide handler implementations on all platforms. This can be accomplished by adding partial class handler implementations in the child folders of the *Platforms* folder. Alternatively you could configure your project to support filename-based multi-targeting, or folder-based multi-targeting, or both.

Filename-based multi-targeting is configured by adding the following XML to the project file, as children of the `<Project>` node:

XML

```
<!-- Android -->
<ItemGroup Condition="$(TargetFramework.StartsWith('net8.0-android')) != true">
  <Compile Remove="**\*.Android.cs" />
  <None Include="**\*.Android.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>

<!-- iOS and Mac Catalyst -->
<ItemGroup Condition="$(TargetFramework.StartsWith('net8.0-ios')) != true
AND $(TargetFramework.StartsWith('net8.0-maccatalyst')) != true">
  <Compile Remove="**\*.MaciOS.cs" />
  <None Include="**\*.MaciOS.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>

<!-- Windows -->
<ItemGroup Condition="$(TargetFramework.Contains('-windows')) != true ">
  <Compile Remove="**\*.Windows.cs" />
  <None Include="**\*.Windows.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>
```

For more information about configuring multi-targeting, see [Configure multi-targeting](#).

Each platform handler class should be a partial class and derive from the `ViewHandler<TVirtualView,TPlatformView>` class, which requires two type arguments:

- The class for the cross-platform control, which derives from [View](#).
- The type of the native view that implements the cross-platform control on the platform. This should be identical to the type of the `PlatformView` property in the handler.

The [ViewHandler<TVirtualView,TPlatformView>](#) class provides [VirtualView](#) and [PlatformView](#) properties. The [VirtualView](#) property is used to access the cross-platform control from its handler. The [PlatformView](#) property, is used to access the native view on each platform that implements the cross-platform control.

Each of the platform handler implementations should override the following methods:

- [CreatePlatformView](#), which should create and return the native view that implements the cross-platform control.
- [ConnectHandler](#), which should perform any native view setup, such as initializing the native view and performing event subscriptions.
- [DisconnectHandler](#), which should perform any native view cleanup, such as unsubscribing from events and disposing objects. This method is automatically invoked by .NET MAUI by default, although this behavior can be changed. For more information, see [Control handler disconnection](#).

ⓘ Note

The [CreatePlatformView](#), [ConnectHandler](#), and [DisconnectHandler](#) overrides are the replacements for the [OnElementChanged](#) method in a Xamarin.Forms custom renderer.

Each platform handler should also implement the Actions that are defined in the mapper dictionaries. In addition, each platform handler should also provide code, as required, to implement the functionality of the cross-platform control on the platform. Alternatively, for more complex controls this can be provided by an additional type.

The following example shows the [CustomEntryHandler](#) implementation on Android:

C#

```
#nullable enable
using AndroidX.AppCompat.Widget;
using Microsoft.Maui.Handlers;
using Microsoft.Maui.Platform;
using MyMauiControl.Controls;

namespace MyMauiControl.Handlers
{
    public partial class CustomEntryHandler : ViewHandler<CustomEntry,
        AppCompatEditText>
    {
        protected override AppCompatEditText CreatePlatformView() => new
            AppCompatEditText(Context);
    }
}
```

```

        protected override void ConnectHandler(AppCompatEditText
platformView)
        {
            base.ConnectHandler(platformView);

            // Perform any control setup here
        }

        protected override void DisconnectHandler(AppCompatEditText
platformView)
        {
            // Perform any native view cleanup here
            platformView.Dispose();
            base.DisconnectHandler(platformView);
        }

        public static void MapText(CustomEntryHandler handler, CustomEntry
view)
        {
            handler.PlatformView.Text = view.Text;

            handler.PlatformView?.SetSelection(handler.PlatformView?.Text?.Length ?? 0);
        }

        public static void MapTextColor(CustomEntryHandler handler,
CustomEntry view)
        {
            handler.PlatformView?.SetTextColor(view.TextColor.ToPlatform());
        }
    }
}

```

`CustomEntryHandler` derives from the `ViewHandler<TVirtualView,TPlatformView>` class, with the generic `CustomEntry` argument specifying the cross-platform control type, and the `AppCompatEditText` argument specifying the type of native control.

The `CreatePlatformView` override creates and returns an `AppCompatEditText` object. The `ConnectHandler` override is the location to perform any required native view setup. The `DisconnectHandler` override is the location to perform any native view cleanup, and so calls the `Dispose` method on the `AppCompatEditText` instance.

The handler also implements the Actions defined in the property mapper dictionary. Each Action is executed in response to a property changing on the cross-platform control, and is a `static` method that requires handler and cross-platform control instances as arguments. In each case, the Action calls methods defined on the native control.

Register the handler

A custom control and its handler must be registered with an app, before it can be consumed. This should occur in the `CreateMauiApp` method in the `MauiProgram` class in your app project, which is the cross-platform entry point for the app:

C#

```
using Microsoft.Extensions.Logging;
using MyMauiControl.Controls;
using MyMauiControl.Handlers;

namespace MyMauiControl;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            })
            .ConfigureMauiHandlers(handlers =>
            {
                handlers.AddHandler(typeof(CustomEntry),
                typeof(CustomEntryHandler));
            });

        #if DEBUG
            builder.Logging.AddDebug();
        #endif

        return builder.Build();
    }
}
```

The handler is registered with the `ConfigureMauiHandlers` and `AddHandler` method. The first argument to the `AddHandler` method is the cross-platform control type, with the second argument being its handler type.

ⓘ Note

This registration approach avoids Xamarin.Forms' assembly scanning, which is slow and expensive.

Consume the cross-platform control

After registering the handler with your app, the cross-platform control can then be consumed:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-namespace:MyMauiControl.Controls"
             x:Class="MyMauiControl.MainPage">
    <Grid>
        <controls:CustomEntry Text="Hello world"
                             TextColor="Blue" />
    </Grid>
</ContentPage>
```

Control handler disconnection

Each platform's handler implementation overrides the [DisconnectHandler](#) implementation, which is used to perform native view cleanup such as unsubscribing from events and disposing objects. By default, handlers automatically disconnect from their controls when possible, such as when navigating backwards in an app.

In some scenarios you might want to control when a handler disconnects from its control, which can be achieved with the [HandlerProperties.DisconnectPolicy](#) attached property. This property requires a [HandlerDisconnectPolicy](#) argument, with the enumeration defining the following values:

- `Automatic`, which indicates that the handler will be disconnected automatically. This is the default value of the [HandlerProperties.DisconnectPolicy](#) attached property.
- `Manual`, which indicates that the handler will have to be disconnected manually by invoking the [DisconnectHandler\(\)](#) implementation.

The following example shows setting the [HandlerProperties.DisconnectPolicy](#) attached property:

XAML

```
<controls:CustomEntry x:Name="customEntry"
                     Text="Hello world"
                     TextColor="Blue"
                     HandlerProperties.DisconnectPolicy="Manual" />
```

When setting the [HandlerProperties.DisconnectPolicy](#) attached property to `Manual` you must invoke the handler's [DisconnectHandler](#) implementation yourself from a suitable location in your app's lifecycle. This can be achieved by invoking

```
customEntry.Handler?.DisconnectHandler();
```

In addition, there's a [DisconnectHandlers](#) extension method that disconnects handlers from a given [IView](#):

C#

```
video.DisconnectHandlers();
```

When disconnecting, the [DisconnectHandlers](#) method will propagate down the control tree until it completes or arrives at a control that has set a manual policy.

See also

- [Create a custom control using handler](#)

Reuse effects in .NET MAUI

Article • 05/03/2024

While there are many benefits to using .NET Multi-platform App UI (.NET MAUI) handlers to customize controls, it's still possible to use Xamarin.Forms effects in .NET MAUI apps. For more information about effects, see [Xamarin.Forms effects](#).

The process for migrating a Xamarin.Forms effect to .NET MAUI is to:

1. Remove the effect attributes from your effect classes. For more information, see [Remove effect attributes](#).
2. Remove the effect `using` directives. For more information, see [Remove using directives](#).
3. Add the effect code into the appropriate location in your .NET MAUI app project. For more information, see [Add the effect code](#).
4. Register the effect. For more information, see [Register the effect](#).
5. Consume your .NET MAUI effect. For more information, see [Consume the effect](#).

Remove effect attributes

Any `ResolutionGroupNameAttribute` and `ExportEffectAttribute` attributes should be removed from your effect classes.

Remove using directives

Any references to the `Xamarin.Forms` and `Xamarin.Forms.Platform.*` namespaces should be removed from your effect classes.

Add the effect code

If you're using a .NET MAUI multi-targeted project, your effect code should be combined into a single file and placed outside the *Platforms* folder. This requires you to combine your [RoutingEffect](#) implementation and [PlatformEffect](#) implementations into a single file, using conditional compilation around platform code. However, if your solution has separate projects per-platform, then you should move the platform-specific effect files into the corresponding projects.

In .NET MAUI, the [RoutingEffect](#) class is in the `Microsoft.Maui.Controls` namespace. This namespace is one of .NET MAUI's implicit `global using` directives, and so you don't

need to add a `using` directive for it. However, the `PlatformEffect` class is in the `Microsoft.Maui.Controls.Platform` namespace, for which you must add a `using` directive.

The following code example shows a `FocusRoutingEffect` class and its platform implementations combined into a single file:

C#

```
using Microsoft.Maui.Controls.Platform;

namespace MyMauiApp.Effects;

internal class FocusRoutingEffect : RoutingEffect
{
}

#if ANDROID
internal class FocusPlatformEffect : PlatformEffect
{
    protected override void OnAttached()
    {
        // Customize the control here
    }

    protected override void OnDetached()
    {
        // Cleanup the control customization here
    }
}
#elif IOS
internal class FocusPlatformEffect : PlatformEffect
{
    protected override void OnAttached()
    {
        // Customize the control here
    }

    protected override void OnDetached()
    {
        // Cleanup the control customization here
    }
}
#elif WINDOWS
internal class FocusPlatformEffect : PlatformEffect
{
    protected override void OnAttached()
    {
        // Customize the control here
    }

    protected override void OnDetached()
    {
    }
}
```

```

    {
        // Cleanup the control customization here
    }
}
#endif

```

Register the effect

In your .NET MAUI app project, open *MauiProgram.cs* and call the [ConfigureEffects](#) method on the [MauiApplicationBuilder](#) object in the `CreateMauiApp` method:

```

C#

public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();
    builder
        .UseMauiApp<App>()
        .ConfigureEffects(effects =>
        {
            effects.Add<FocusRoutingEffect, FocusPlatformEffect>();
        });

    return builder.Build();
}

```

The effect is registered with the [ConfigureEffects](#) method, whose `configureDelegate` registers the [PlatformEffect](#) implementation against its [RoutingEffect](#) implementation.

Consume the effect

The effect can be consumed in a .NET MAUI app by adding it to the [Effects](#) collection of a control:

```

XAML

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:MyMauiApp.Effects"
              x:Class="MyMauiApp.MainPage">
    <VerticalStackLayout>
        <Entry Text="Enter your text">
            <Entry.Effects>
                <local:FocusRoutingEffect />
            </Entry.Effects>
        </Entry>
    </VerticalStackLayout>
</ContentPage>

```

```
</VerticalStackLayout>  
</ContentPage>
```

Reuse SkiaSharp code in .NET MAUI

Article • 01/10/2024

SkiaSharp is a 2D graphics system for .NET and C# that draws 2D vector graphics, bitmaps, and text. It's powered by the open-source Skia graphics engine that's used extensively in Google products. You can reuse SkiaSharp code from your Xamarin.Forms apps in your .NET Multi-platform App UI (.NET MAUI) apps with some minor updates.

To reuse your SkiaSharp code from a Xamarin.Forms app in a .NET MAUI app, you must:

- ✓ Remove the Xamarin.Forms SkiaSharp NuGet packages from your project, and add the .NET MAUI SkiaSharp NuGet packages to your project.
- ✓ Update namespaces.
- ✓ Initialize SkiaSharp.

Add NuGets

SkiaSharp for .NET MAUI is packaged as a series of NuGet packages. After you've migrated your Xamarin.Forms app to a .NET MAUI app, you should remove all the existing SkiaSharp NuGet packages from your app. Then, use the NuGet package manager to search for the [SkiaSharp.Views.Maui.Controls](#) NuGet package and add it to your project. This will also install dependent SkiaSharp packages.

Update namespaces

Xamarin.Forms apps that use SkiaSharp typically use types from the [SkiaSharp](#) namespace and the [SkiaSharp.Views.Forms](#) namespace. In SkiaSharp for .NET MAUI, you'll continue to use the [SkiaSharp](#) namespace but the types that were in the [SkiaSharp.Views.Forms](#) namespace have moved into the [SkiaSharp.Views.Maui](#) and [SkiaSharp.Views.Maui.Controls](#) namespaces.

The following table shows the namespaces you'll need to use to build your SkiaSharp code in a .NET MAUI app:

 Expand table

.NET MAUI namespace	Details
SkiaSharp	Contains all the SkiaSharp classes, structures, and enumerations.

.NET MAUI namespace	Details
SkiaSharp.Views.Maui	Contains types to support touch interactions, and event arguments.
SkiaSharp.Views.Maui.Controls	Contains the SKCanvasView class, which derives from the .NET MAUI View class and hosts your SkiaSharp graphics output. Also contains different <code>ImageSource</code> classes.
SkiaSharp.Views.Maui.Controls.Hosting	Contains the UseSkiaSharp method that's used to initialize SkiaSharp in your .NET MAUI app. For more information, see Initialize SkiaSharp .

Initialize SkiaSharp

Initialize SkiaSharp in your app by calling the [UseSkiaSharp](#) method on the [MauiAppBuilder](#) object in your `MauiProgram` class:

C#

```
using Microsoft.Extensions.Logging;
using SkiaSharp.Views.Maui.Controls.Hosting;

namespace MyMauiApp;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .UseSkiaSharp()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            });

        #if DEBUG
        builder.Logging.AddDebug();
        #endif

        return builder.Build();
    }
}
```

 Note

Calling the UseSkiaSharp method requires you to add a `using` directive for the `SkiaSharp.Views.Maui.Controls.Hosting` namespace.

Migrate data from the Xamarin.Forms app properties dictionary to .NET MAUI preferences

Article • 05/03/2024

Xamarin.Forms has a `Properties` dictionary that can be used to store data, and which is accessed using the `Application.Current.Properties` property. This dictionary uses a `string` key and stores an `object` value. The values in the dictionary are saved to the device when an app is paused or shut down, and loaded when an app is restarted or returns from the background. For more information about the properties dictionary, see [Properties dictionary](#).

When migrating a Xamarin.Forms app that stores data in the app properties dictionary to .NET MAUI, you should migrate this data to .NET MAUI preferences. This can be accomplished with the `LegacyApplication` class, and helper classes, which is presented in this article. This class enables your .NET MAUI app on Android, iOS, and Windows, to read data from the app properties dictionary that was created with a previous Xamarin.Forms version of your app. For more information about .NET MAUI preferences, see [Preferences](#).

Important

There's no API to access the app properties dictionary in .NET MAUI.

Access legacy app properties data

The following code shows the `LegacyApplication` class, which provides access to the app properties data created by your Xamarin.Forms app:

Note

To use this code, add it to a class named `LegacyApplication` in your .NET MAUI app project.

C#

```

namespace MigrationHelpers;

public class LegacyApplication
{
    readonly PropertiesDeserializer deserializer;
    Task<IDictionary<string, object>>? propertiesTask;

    static LegacyApplication? current;
    public static LegacyApplication? Current
    {
        get
        {
            current ??=
            (LegacyApplication)Activator.CreateInstance(typeof(LegacyApplication));
            return current;
        }
    }

    public LegacyApplication()
    {
        deserializer = new PropertiesDeserializer();
    }

    public IDictionary<string, object> Properties
    {
        get
        {
            propertiesTask ??= GetPropertiesAsync();
            return propertiesTask.Result;
        }
    }

    async Task<IDictionary<string, object>> GetPropertiesAsync()
    {
        IDictionary<string, object> properties = await
        deserializer.DeserializePropertiesAsync().ConfigureAwait(false);
        properties ??= new Dictionary<string, object>(4);
        return properties;
    }
}

```

Android

On Android, the `LegacyApplication` class uses the `PropertiesDeserializer` class to deserialize data from the app properties dictionary file. The following code shows the `PropertiesDeserializer` class:

ⓘ Note

To use this code, add it to a class named `PropertiesDeserializer` in the `Platforms\Android` folder of your .NET MAUI app project.

C#

```
using System.Diagnostics;
using System.IO.IsolatedStorage;
using System.Runtime.Serialization;
using System.Xml;

namespace MigrationHelpers;

public class PropertiesDeserializer
{
    const string PropertyStoreFile = "PropertyStore.forms";

    public Task<IDictionary<string, object>> DeserializePropertiesAsync()
    {
        // Deserialize property dictionary to local storage
        return Task.Run(() =>
        {
            using (IsolatedStorageFile store =
                IsolatedStorageFile.GetUserStoreForApplication())
            {
                if (!store.FileExists(PropertyStoreFile))
                    return null;

                using (IsolatedStorageFileStream stream =
                    store.OpenFile(PropertyStoreFile, FileMode.Open, FileAccess.Read))
                using (XmlDictionaryReader reader =
                    XmlDictionaryReader.CreateBinaryReader(stream,
                    XmlDictionaryReaderQuotas.Max))
                {
                    if (stream.Length == 0)
                        return null;

                    try
                    {
                        var dcs = new
                            DataContractSerializer(typeof(Dictionary<string, object>));
                        return (IDictionary<string,
                            object>)dcs.ReadObject(reader);
                    }
                    catch (Exception e)
                    {
                        Debug.WriteLine("Could not deserialize properties: "
                            + e.Message);
                        Console.WriteLine($"PropertyStore Exception while
                            reading Application properties: {e}");
                    }
                }
            }
        })
        return null;
    }
}
```

```
    });  
  }  
}
```

iOS

On iOS, the `LegacyApplication` class uses the `PropertiesDeserializer` class to deserialize data from the app properties dictionary file. The following code shows the `PropertiesDeserializer` class:

ⓘ Note

To use this code, add it to a class named `PropertiesDeserializer` in the *Platforms\iOS* folder of your .NET MAUI app project.

C#

```
using System.Diagnostics;  
using System.IO.IsolatedStorage;  
using System.Runtime.Serialization;  
using System.Xml;  
  
namespace MigrationHelpers;  
  
public class PropertiesDeserializer  
{  
    const string PropertyStoreFile = "PropertyStore.forms";  
  
    public Task<IDictionary<string, object>> DeserializePropertiesAsync()  
    {  
        // Deserialize property dictionary to local storage  
        return Task.Run(() =>  
        {  
            using (var store =  
IsolatedStorageFile.GetUserStoreForApplication())  
                using (var stream = store.OpenFile(PropertyStoreFile,  
System.IO.FileMode.OpenOrCreate))  
                    using (var reader =  
XmlDictionaryReader.CreateBinaryReader(stream,  
XmlDictionaryReaderQuotas.Max))  
                        {  
                            if (stream.Length == 0)  
                                return null;  
  
                            try  
                            {  
                                var dcs = new  
DataContractSerializer(typeof(Dictionary<string, object>));  

```

```

        return (IDictionary<string,
object>)dcs.ReadObject(reader);
    }
    catch (Exception e)
    {
        Debug.WriteLine("Could not deserialize properties: " +
e.Message);
        Console.WriteLine($"PropertyStore Exception while
reading Application properties: {e}");
    }
}
return null;
});
}
}

```

Windows

On Windows, the `LegacyApplication` class uses the `PropertiesDeserializer` class to deserialize data from the app properties dictionary file. The following code shows the `PropertiesDeserializer` class:

⚠ Note

To use this code, add it to a class named `PropertiesDeserializer` in the *Platforms\Windows* folder of your .NET MAUI app project.

C#

```

using System.Diagnostics;
using System.Runtime.Serialization;
using Windows.Storage;

namespace MigrationHelpers;

public class PropertiesDeserializer
{
    const string PropertyStoreFile = "PropertyStore.forms";

    public async Task<IDictionary<string, object>>
DeserializePropertiesAsync()
    {
        try
        {
            StorageFile file = await
ApplicationData.Current.RoamingFolder.GetFileAsync(PropertyStoreFile).DontSyn
c();
            using (Stream stream = (await

```

```

file.OpenReadAsync().DontSync()).AsStreamForRead())
{
    if (stream.Length == 0)
        return new Dictionary<string, object>(4);

    try
    {
        var serializer = new
DataContractSerializer(typeof(IDictionary<string, object>));
        return (IDictionary<string,
object>)serializer.ReadObject(stream);
    }
    catch (Exception e)
    {
        Debug.WriteLine("Could not deserialize properties: " +
e.Message);
        Console.WriteLine($"PropertyStore Exception while
reading Application properties: {e}");
    }
    return null;
}
}
catch (FileNotFoundException)
{
    return new Dictionary<string, object>(4);
}
}
}

```

This Windows version of the `PropertiesDeserializer` class requires the `DontSync` extension method. The following code shows this extension method:

ⓘ Note

To use this code, add it to a class named `Extensions` in the *Platforms\Windows* folder of your .NET MAUI app project.

C#

```

using System.Runtime.CompilerServices;
using Windows.Foundation;

namespace MigrationHelpers;

internal static class Extensions
{
    public static ConfiguredTaskAwaitable<T> DontSync<T>(this
IAsyncOperation<T> self)
    {
        return self.AsTask().ConfigureAwait(false);
    }
}

```

```
}  
}
```

Consume legacy app property data

The `LegacyApplication` class can be used to consume data from the app properties dictionary, on Android, iOS, and Windows, that was created with a previous Xamarin.Forms version of your app:

C#

```
#if ANDROID || IOS || WINDOWS  
using MigrationHelpers;  
...  
  
int id;  
if (LegacyApplication.Current.Properties.ContainsKey("id"))  
{  
    id = (int)LegacyApplication.Current.Properties["id"];  
    Preferences.Set("id", id);  
}  
#endif
```

This example shows using the `LegacyApplication` class to read a value from the app properties dictionary, and then write the value to .NET MAUI preferences.

Important

Always check for the presence of the key in the app properties dictionary before accessing it, to prevent unexpected errors.

Migrate from Xamarin.Essentials secure storage to .NET MAUI secure storage

Article • 11/13/2023

Xamarin.Essentials and .NET Multi-platform App UI (.NET MAUI) both have a `SecureStorage` class that helps you securely store simple key/value pairs. However, there are implementation differences between the `SecureStorage` class in Xamarin.Essentials and .NET MAUI:

 Expand table

Platform	Xamarin.Essentials	.NET MAUI
Android	The Android KeyStore is used to store the cipher key used to encrypt a value before it's saved into a shared preferences object with a name of {your-app-package-id}.xamarinessentials.	Data is encrypted with the <code>EncryptedSharedPreferences</code> class, which wraps the <code>SharedPreferences</code> class, and automatically encrypts keys and values. The name used is {your-app-package-id}.microsoft.maui.essentials.preferences.
iOS	KeyChain is used to store values securely. The <code>SecRecord</code> used to store values has a <code>Service</code> value set to {your-app-package-id}.xamarinessentials.	KeyChain is used to store values securely. The <code>SecRecord</code> used to store values has a <code>Service</code> value set to {your-app-package-id}.microsoft.maui.essentials.preferences.

For more information about the `SecureStorage` class in Xamarin.Essentials, see [Xamarin.Essentials: Secure storage](#). For more information about the `SecureStorage` class in .NET MAUI, see [Secure storage](#).

When migrating a Xamarin.Forms app that uses the `SecureStorage` class to .NET MAUI, you must deal with these implementation differences to provide users with a smooth upgrade experience. This article describes how you can use the `LegacySecureStorage` class and helper classes to deal with the implementation differences. The `LegacySecureStorage` class enables your .NET MAUI app on Android and iOS to read secure storage data that was created with a previous Xamarin.Forms version of your app.

Access legacy secure storage data

The following code shows the `LegacySecureStorage` class, which provides the secure storage implementation from Xamarin.Essentials:

ⓘ Note

To use this code, add it to a class named `LegacySecureStorage` in your .NET MAUI app project.

C#

```
#nullable enable
#if ANDROID || IOS

namespace MigrationHelpers;

public class LegacySecureStorage
{
    internal static readonly string Alias = $"{AppInfo.PackageName}.xamarinessentials";

    public static Task<string> GetAsync(string key)
    {
        if (string.IsNullOrEmpty(key))
            throw new ArgumentNullException(nameof(key));

        string result = string.Empty;

#if ANDROID
        object locker = new object();
        string? encVal = Preferences.Get(key, null, Alias);

        if (!string.IsNullOrEmpty(encVal))
        {
            byte[] encData = Convert.FromBase64String(encVal);
            lock (locker)
            {
                AndroidKeyStore keyStore = new
                AndroidKeyStore(Platform.AppContext, Alias, false);
                result = keyStore.Decrypt(encData);
            }
        }
#elif IOS
        KeyChain keyChain = new KeyChain();
        result = keyChain.ValueForKey(key, Alias);
#endif
        return Task.FromResult(result);
    }

    public static bool Remove(string key)
    {
        bool result = false;
    }
}
```

```

#if ANDROID
    Preferences.Remove(key, Alias);
    result = true;
#elif IOS
    KeyChain keyChain = new KeyChain();
    result = keyChain.Remove(key, Alias);
#endif
    return result;
}

public static void RemoveAll()
{
#if ANDROID
    Preferences.Clear(Alias);
#elif IOS
    KeyChain keyChain = new KeyChain();
    keyChain.RemoveAll(Alias);
#endif
}
}
#endif

```

Android

On Android, the `LegacySecureStorage` class uses the `AndroidKeyStore` class to store the cipher key used to encrypt a value before it's saved into a shared preferences object with a name of {your-app-package-id}.xamarinessentials. The following code shows the `AndroidKeyStore` class:

ⓘ Note

To use this code, add it to a class named `AndroidKeyStore` in the *Platforms\Android* folder of your .NET MAUI app project.

C#

```

using Android.Content;
using Android.OS;
using Android.Runtime;
using Android.Security;
using Android.Security.KeyStore;
using Java.Security;
using Javax.Crypto;
using Javax.Crypto.Spec;
using System.Text;

namespace MigrationHelpers;

```



```

class AndroidKeyStore
{
    const string androidKeyStore = "AndroidKeyStore"; // this is an Android
const value
    const string aesAlgorithm = "AES";
    const string cipherTransformationAsymmetric = "RSA/ECB/PKCS1Padding";
    const string cipherTransformationSymmetric = "AES/GCM/NoPadding";
    const string prefsMasterKey = "SecureStorageKey";
    const int initializationVectorLen = 12; // Android supports an IV of 12
for AES/GCM

    internal AndroidKeyStore(Context context, string keystoreAlias, bool
alwaysUseAsymmetricKeyStorage)
    {
        alwaysUseAsymmetricKey = alwaysUseAsymmetricKeyStorage;
        appContext = context;
        alias = keystoreAlias;

        keyStore = KeyStore.GetInstance(androidKeyStore);
        keyStore.Load(null);
    }

    readonly Context appContext;
    readonly string alias;
    readonly bool alwaysUseAsymmetricKey;
    readonly string useSymmetricPreferenceKey = "essentials_use_symmetric";

    KeyStore keyStore;
    bool useSymmetric = false;

    ISecretKey GetKey()
    {
        // check to see if we need to get our key from past-versions or
newer versions.
        // we want to use symmetric if we are >= 23 or we didn't set it
previously.
        var hasApiLevel = Build.VERSION.SdkInt >= BuildVersionCodes.M;

        useSymmetric = Preferences.Get(useSymmetricPreferenceKey,
hasApiLevel, alias);

        // If >= API 23 we can use the KeyStore's symmetric key
        if (useSymmetric && !alwaysUseAsymmetricKey)
            return GetSymmetricKey();

        // NOTE: KeyStore in < API 23 can only store asymmetric keys
        // specifically, only RSA/ECB/PKCS1Padding
        // So we will wrap our symmetric AES key we just generated
        // with this and save the encrypted/wrapped key out to
        // preferences for future use.
        // ECB should be fine in this case as the AES key should be
        // contained in one block.

        // Get the asymmetric key pair

```

```

var keyPair = GetAsymmetricKeyPair();

var existingKeyStr = Preferences.Get(prefsMasterKey, null, alias);

if (!string.IsNullOrEmpty(existingKeyStr))
{
    try
    {
        var wrappedKey = Convert.FromBase64String(existingKeyStr);

        var unwrappedKey = UnwrapKey(wrappedKey, keyPair.Private);
        var kp = unwrappedKey.JavaCast<ISecretKey>();

        return kp;
    }
    catch (InvalidKeyException ikEx)
    {
        System.Diagnostics.Debug.WriteLine($"Unable to unwrap key: Invalid Key. This may be caused by system backup or upgrades. All secure storage items will now be removed. {ikEx.Message}");
    }
    catch (IllegalBlockSizeException ibsEx)
    {
        System.Diagnostics.Debug.WriteLine($"Unable to unwrap key: Illegal Block Size. This may be caused by system backup or upgrades. All secure storage items will now be removed. {ibsEx.Message}");
    }
    catch (BadPaddingException paddingEx)
    {
        System.Diagnostics.Debug.WriteLine($"Unable to unwrap key: Bad Padding. This may be caused by system backup or upgrades. All secure storage items will now be removed. {paddingEx.Message}");
    }
    LegacySecureStorage.RemoveAll();
}

var keyGenerator = KeyGenerator.GetInstance(aesAlgorithm);
var defSymmetricKey = keyGenerator.GenerateKey();

var newWrappedKey = WrapKey(defSymmetricKey, keyPair.Public);

Preferences.Set(prefsMasterKey,
Convert.ToBase64String(newWrappedKey), alias);

return defSymmetricKey;
}

// API 23+ Only
#pragma warning disable CA1416
ISecretKey GetSymmetricKey()
{
    Preferences.Set(useSymmetricPreferenceKey, true, alias);

    var existingKey = keyStore.GetKey(alias, null);

```

```

        if (existingKey != null)
        {
            var existingSecretKey = existingKey.JavaCast<ISecretKey>();
            return existingSecretKey;
        }

        var keyGenerator =
KeyGenerator.GetInstance(KeyProperties.KeyAlgorithmAes, androidKeyStore);
        var builder = new KeyGenParameterSpec.Builder(alias,
KeyStorePurpose.Encrypt | KeyStorePurpose.Decrypt)
            .SetBlockModes(KeyProperties.BlockModeGcm)
            .SetEncryptionPadding(KeyProperties.EncryptionPaddingNone)
            .SetRandomizedEncryptionRequired(false);

        keyGenerator.Init(builder.Build());

        return keyGenerator.GenerateKey();
    }
#pragma warning restore CA1416

    KeyPair GetAsymmetricKeyPair()
    {
        // set that we generated keys on pre-m device.
        Preferences.Set(useSymmetricPreferenceKey, false, alias);

        var asymmetricAlias = $"{alias}.asymmetric";

        var privateKey = keyStore.GetKey(asymmetricAlias,
null)?.JavaCast<IPrivateKey>();
        var publicKey = keyStore.GetCertificate(asymmetricAlias)?.PublicKey;

        // Return the existing key if found
        if (privateKey != null && publicKey != null)
            return new KeyPair(publicKey, privateKey);

        var originalLocale = Java.Util.Locale.Default;
        try
        {
            // Force to english for known bug in date parsing:
            // https://issuetracker.google.com/issues/37095309
            SetLocale(Java.Util.Locale.English);

            // Otherwise we create a new key
#pragma warning disable CA1416
            var generator =
KeyPairGenerator.GetInstance(KeyProperties.KeyAlgorithmRsa,
androidKeyStore);
#pragma warning restore CA1416

            var end = DateTime.UtcNow.AddYears(20);
            var startDate = new Java.Util.Date();
#pragma warning disable CS0618 // Type or member is obsolete
            var endDate = new Java.Util.Date(end.Year, end.Month, end.Day);
#pragma warning restore CS0618 // Type or member is obsolete

```

```

#pragma warning disable CS0618
    var builder = new
    KeyPairGeneratorSpec.Builder(Platform.AppContext)
        .SetAlias(asymmetricAlias)
        .SetSerialNumber(Java.Math.BigInteger.One)
        .SetSubject(new Javax.Security.Auth.X500.X500Principal($"CN=
{asymmetricAlias} CA Certificate"))
        .SetStartDate(startDate)
        .SetEndDate(endDate);

    generator.Initialize(builder.Build());
#pragma warning restore CS0618

    return generator.GenerateKeyPair();
}
finally
{
    SetLocale(originalLocale);
}

byte[] WrapKey(IKey keyToWrap, IKey withKey)
{
    var cipher = Cipher.GetInstance(cipherTransformationAsymmetric);
    cipher.Init(CipherMode.WrapMode, withKey);
    return cipher.Wrap(keyToWrap);
}

#pragma warning disable CA1416
IKey UnwrapKey(byte[] wrappedData, IKey withKey)
{
    var cipher = Cipher.GetInstance(cipherTransformationAsymmetric);
    cipher.Init(CipherMode.UnwrapMode, withKey);
    var unwrapped = cipher.Unwrap(wrappedData,
    KeyProperties.KeyAlgorithmAes, KeyType.SecretKey);
    return unwrapped;
}
#pragma warning restore CA1416

internal string Decrypt(byte[] data)
{
    if (data.Length < initializationVectorLen)
        return null;

    var key = GetKey();

    // IV will be the first 16 bytes of the encrypted data
    var iv = new byte[initializationVectorLen];
    Buffer.BlockCopy(data, 0, iv, 0, initializationVectorLen);

    Cipher cipher;

    // Attempt to use GCMParameterSpec by default
    try
    {

```

```

        cipher = Cipher.GetInstance(cipherTransformationSymmetric);
        cipher.Init(CipherMode.DecryptMode, key, new
GCMPParameterSpec(128, iv));
    }
    catch (InvalidAlgorithmParameterException)
    {
        // If we encounter this error, it's likely an old bouncycastle
provider version
        // is being used which does not recognize GCMParameterSpec, but
should work
        // with IvParameterSpec, however we only do this as a last
effort since other
        // implementations will error if you use IvParameterSpec when
GCMParameterSpec
        // is recognized and expected.
        cipher = Cipher.GetInstance(cipherTransformationSymmetric);
        cipher.Init(CipherMode.DecryptMode, key, new
IvParameterSpec(iv));
    }

    // Decrypt starting after the first 16 bytes from the IV
    var decryptedData = cipher.DoFinal(data, initializationVectorLen,
data.Length - initializationVectorLen);

    return Encoding.UTF8.GetString(decryptedData);
}

internal void SetLocale(Java.Util.Locale locale)
{
    Java.Util.Locale.Default = locale;
    var resources = appContext.Resources;
    var config = resources.Configuration;

    if (Build.VERSION.SdkInt >= BuildVersionCodes.N)
        config.SetLocale(locale);
    else
#pragma warning disable CS0618 // Type or member is obsolete
        config.Locale = locale;
#pragma warning restore CS0618 // Type or member is obsolete

#pragma warning disable CS0618 // Type or member is obsolete
    resources.UpdateConfiguration(config, resources.DisplayMetrics);
#pragma warning restore CS0618 // Type or member is obsolete
}
}

```

The [Android KeyStore](#) is used to store the cipher key used to encrypt the value before it's saved into a [Shared Preferences](#) file with a name of *{your-app-package-id}.xamarinessentials*. The key (not a cryptographic key, the *key* to the *value*) used in the shared preferences file is an *MD5 Hash* of the key passed into the `SecureStorage` APIs.

On API 23+, an **AES** key is obtained from the Android KeyStore and used with an **AES/GCM/NoPadding** cipher to encrypt the value before it's stored in the shared preferences file. On API 22 and lower, the Android KeyStore only supports storing **RSA** keys, which is used with an **RSA/ECB/PKCS1Padding** cipher to encrypt an **AES** key (randomly generated at runtime) and stored in the shared preferences file under the key *SecureStorageKey*, if one hasn't already been generated.

iOS

On iOS, the `LegacySecureStorage` class uses the `KeyChain` class to store values securely. The `SecRecord` used to store values has a `Service` value set to {your-app-package-id}.xamarin.essentials. The following code shows the `KeyChain` class:

ⓘ Note

To use this code, add it to a class named `KeyChain` in the *Platforms\iOS* folder of your .NET MAUI app project.

C#

```
using Foundation;
using Security;

namespace MigrationHelpers;

class KeyChain
{
    SecRecord ExistingRecordForKey(string key, string service)
    {
        return new SecRecord(SecKind.GenericPassword)
        {
            Account = key,
            Service = service
        };
    }

    internal string ValueForKey(string key, string service)
    {
        using (var record = ExistingRecordForKey(key, service))
        using (var match = SecKeyChain.QueryAsRecord(record, out var
resultCode))
        {
            if (resultCode == SecStatusCode.Success)
                return NSString.FromData(match.ValueData,
NSStringEncoding.UTF8);
            else
                return null;
        }
    }
}
```

```

    }
}

internal bool Remove(string key, string service)
{
    using (var record = ExistingRecordForKey(key, service))
    using (var match = SecKeyChain.QueryAsRecord(record, out var
resultCode))
    {
        if (resultCode == SecStatusCode.Success)
        {
            RemoveRecord(record);
            return true;
        }
    }
    return false;
}

internal void RemoveAll(string service)
{
    using (var query = new SecRecord(SecKind.GenericPassword) { Service
= service })
    {
        SecKeyChain.Remove(query);
    }
}

bool RemoveRecord(SecRecord record)
{
    var result = SecKeyChain.Remove(record);
    if (result != SecStatusCode.Success && result !=
SecStatusCode.ItemNotFound)
        throw new Exception($"Error removing record: {result}");

    return true;
}
}

```

To use this code, you must have an *Entitlements.plist* file for your iOS app with the Keychain entitlement set:

XML

```

<key>keychain-access-groups</key>
<array>
    <string>$(AppIdentifierPrefix)$(CFBundleIdentifier)</string>
</array>

```

You must also ensure that the *Entitlements.plist* file is set as the Custom Entitlements field in the Bundle Signing settings for your app. For more information, see [iOS Entitlements](#).

Consume legacy secure storage data

The `LegacySecureStorage` class can be used to consume legacy secure storage data, on Android and iOS, that was created with a previous Xamarin.Forms version of your app:

C#

```
#if ANDROID || IOS
using MigrationHelpers;
...

string username = await LegacySecureStorage.GetAsync("username");
bool result = LegacySecureStorage.Remove("username");
await SecureStorage.SetAsync("username", username);
#endif
```

The example shows using the `LegacySecureStorage` class to read and remove a value from legacy secure storage, and then write the value to .NET MAUI secure storage.

Migrate version tracking data from a Xamarin.Forms app to a .NET MAUI app

Article • 05/03/2024

Xamarin.Essentials and .NET Multi-platform App UI (.NET MAUI) both have a `VersionTracking` class that lets you check the app's version and build numbers, along with additional information such as if it's the first time the app was launched. However, in Xamarin.Essentials the version tracking data is stored in a platform-specific preferences container with a name of `{your-app-package-id}.xamarin.essentials.versiontracking`, while in .NET MAUI the version tracking data is stored in a platform-specific preferences container with a name of `{your-app-package-id}.microsoft.maui.essentials.versiontracking`.

When migrating a Xamarin.Forms app that uses the `VersionTracking` class to .NET MAUI, you must deal with this preferences container naming difference to provide users with a smooth upgrade experience. This article describes how you can use the `LegacyVersionTracking` class and helper classes to deal with the preferences container. The `LegacyVersionTracking` class enables your .NET MAUI app on Android, iOS, and Windows, to read version tracking data that was created with a previous Xamarin.Forms version of your app.

Important

For the `LegacyVersionTracking` class to work correctly, your .NET MAUI app must have a higher version number than the version number of your Xamarin.Forms app. The version number can be set in your .NET MAUI app's project file with the `$(ApplicationVersion)` and `$(ApplicationDisplayVersion)` build properties.

For more information about the `VersionTracking` class in Xamarin.Essentials, see [Xamarin.Essentials: Version tracking](#). For more information about the `VersionTracking` class in .NET MAUI, see [Version tracking](#).

Access legacy version tracking data

The following code shows the `LegacyVersionTracking` class, which provides access to the version tracking data created by your Xamarin.Forms app:

❗ Note

To use this code, add it to a class named `LegacyVersionTracking` in your .NET MAUI app project.

C#

```
namespace MigrationHelpers;

public static class LegacyVersionTracking
{
    const string versionsKey = "VersionTracking.Versions";
    const string buildsKey = "VersionTracking.Builds";

    static readonly string sharedName =
        LegacyPreferences.GetPrivatePreferencesSharedName("versiontracking");

    static Dictionary<string, List<string>> versionTrail;
    static string LastInstalledVersion =>
        versionTrail[versionsKey].LastOrDefault();
    static string LastInstalledBuild =>
        versionTrail[buildsKey].LastOrDefault();

    public static string VersionsKey => versionsKey;
    public static string BuildsKey => buildsKey;
    public static string SharedName => sharedName;
    public static bool IsFirstLaunchEver { get; private set; }
    public static bool IsFirstLaunchForCurrentVersion { get; private set; }
    public static bool IsFirstLaunchForCurrentBuild { get; private set; }
    public static string CurrentVersion => AppInfo.VersionString;
    public static string CurrentBuild => AppInfo.BuildString;
    public static string PreviousVersion => GetPrevious(versionsKey);
    public static string PreviousBuild => GetPrevious(buildsKey);
    public static string FirstInstalledVersion =>
        versionTrail[versionsKey].FirstOrDefault();
    public static string FirstInstalledBuild =>
        versionTrail[buildsKey].FirstOrDefault();
    public static IEnumerable<string> VersionHistory =>
        versionTrail[versionsKey].ToArray();
    public static IEnumerable<string> BuildHistory =>
        versionTrail[buildsKey].ToArray();
    public static bool IsFirstLaunchForVersion(string version) =>
        CurrentVersion == version && IsFirstLaunchForCurrentVersion;
    public static bool IsFirstLaunchForBuild(string build) => CurrentBuild
        == build && IsFirstLaunchForCurrentBuild;

    static LegacyVersionTracking()
    {
        InitVersionTracking();
    }

    internal static void InitVersionTracking()
```

```

{
    IsFirstLaunchEver = !LegacyPreferences.ContainsKey(versionsKey,
sharedName) || !LegacyPreferences.ContainsKey(buildsKey, sharedName);
    if (IsFirstLaunchEver)
    {
        versionTrail = new Dictionary<string, List<string>>
        {
            { versionsKey, new List<string>() },
            { buildsKey, new List<string>() }
        };
    }
    else
    {
        versionTrail = new Dictionary<string, List<string>>
        {
            { versionsKey, ReadHistory(versionsKey).ToList() },
            { buildsKey, ReadHistory(buildsKey).ToList() }
        };
    }

    IsFirstLaunchForCurrentVersion =
!versionTrail[versionsKey].Contains(CurrentVersion) || CurrentVersion !=
LastInstalledVersion;
    if (IsFirstLaunchForCurrentVersion)
    {
        // Avoid duplicates and move current version to end of list if
already present
        versionTrail[versionsKey].RemoveAll(v => v == CurrentVersion);
        versionTrail[versionsKey].Add(CurrentVersion);
    }

    IsFirstLaunchForCurrentBuild =
!versionTrail[buildsKey].Contains(CurrentBuild) || CurrentBuild !=
LastInstalledBuild;
    if (IsFirstLaunchForCurrentBuild)
    {
        // Avoid duplicates and move current build to end of list if
already present
        versionTrail[buildsKey].RemoveAll(b => b == CurrentBuild);
        versionTrail[buildsKey].Add(CurrentBuild);
    }
}

static string GetPrevious(string key)
{
    var trail = versionTrail[key];
    return (trail.Count >= 2) ? trail[trail.Count - 2] : null;
}

static string[] ReadHistory(string key) => LegacyPreferences.Get(key,
null, sharedName)?.Split(new[] { '|' },
StringSplitOptions.RemoveEmptyEntries) ?? new string[0];
}

```

The `LegacyVersionTracking` class uses the `LegacyPreferences` class, which provides access to version tracking data stored by the Xamarin.Essentials `Preferences` class from your Xamarin.Forms app:

ⓘ Note

To use this code, add it to a class named `LegacyPreferences` in your .NET MAUI app project.

C#

```
#if ANDROID || IOS || WINDOWS
namespace MigrationHelpers;

public static partial class LegacyPreferences
{
    internal static string GetPrivatePreferencesSharedName(string feature)
    => $"{AppInfo.PackageName}.xamarinessentials.{feature}";

    public static bool ContainsKey(string key, string sharedName) =>
PlatformContainsKey(key, sharedName);
    public static void Remove(string key, string sharedName) =>
PlatformRemove(key, sharedName);
    public static string Get(string key, string defaultValue, string
sharedName) => PlatformGet<string>(key, defaultValue, sharedName);
}
#endif
```

The `LegacyPreferences` class is a `partial` class whose remaining implementation is platform-specific.

Android

On Android, the `LegacyPreferences` class provides the preferences container implementation that retrieves data from shared preferences. The following code shows the `LegacyPreferences` class:

ⓘ Note

To use this code, add it to a class named `LegacyPreferences` in the *Platforms\Android* folder of your .NET MAUI app project.

C#

```

using System.Globalization;
using Android.Content;
using Android.Preferences;
using Application = Android.App.Application;

namespace MigrationHelpers;

public static partial class LegacyPreferences
{
    static readonly object locker = new object();

    static bool PlatformContainsKey(string key, string sharedName)
    {
        lock (locker)
        {
            using (var sharedPreferences = GetSharedPreferences(sharedName))
            {
                return sharedPreferences.Contains(key);
            }
        }
    }

    static void PlatformRemove(string key, string sharedName)
    {
        lock (locker)
        {
            using (var sharedPreferences = GetSharedPreferences(sharedName))
            using (var editor = sharedPreferences.Edit())
            {
                editor.Remove(key).Apply();
            }
        }
    }

    static T PlatformGet<T>(string key, T defaultValue, string sharedName)
    {
        lock (locker)
        {
            object value = null;
            using (var sharedPreferences = GetSharedPreferences(sharedName))
            {
                if (defaultValue == null)
                {
                    value = sharedPreferences.GetString(key, null);
                }
                else
                {
                    switch (defaultValue)
                    {
                        case int i:
                            value = sharedPreferences.GetInt(key, i);
                            break;
                        case bool b:
                            value = sharedPreferences.GetBoolean(key, b);
                    }
                }
            }
        }
    }
}

```

```

        break;
    case long l:
        value = sharedPreferences.GetLong(key, l);
        break;
    case double d:
        var savedDouble =
sharedPreferences.GetString(key, null);
        if (string.IsNullOrEmpty(savedDouble))
        {
            value = defaultValue;
        }
        else
        {
            if (!double.TryParse(savedDouble,
NumberStyles.Number | NumberStyles.AllowExponent,
CultureInfo.InvariantCulture, out var outDouble))
            {
                var maxString =
Convert.ToString(double.MaxValue, CultureInfo.InvariantCulture);
                outDouble =
savedDouble.Equals(maxString) ? double.MaxValue : double.MinValue;
            }

            value = outDouble;
        }
        break;
    case float f:
        value = sharedPreferences.GetFloat(key, f);
        break;
    case string s:
        // the case when the string is not null
        value = sharedPreferences.GetString(key, s);
        break;
    }
}

return (T)value;
}
}

static ISharedPreferences GetSharedPreferences(string sharedName)
{
    var context = Application.Context;

    return string.IsNullOrEmpty(sharedName) ?
#pragma warning disable CS0618 // Type or member is obsolete
        PreferenceManager.DefaultSharedPreferences(context) :
#pragma warning restore CS0618 // Type or member is obsolete
        context.GetSharedPreferences(sharedName,
FileCreationMode.Private);
}
}

```

iOS

On iOS, the `LegacyPreferences` class provides the preferences container implementation that retrieves data from `NSUserDefaults`. The following code shows the

`LegacyPreferences` class:

⚠ Note

To use this code, add it to a class named `LegacyPreferences` in the *Platforms\iOS* folder of your .NET MAUI app project.

C#

```
using Foundation;
using System.Globalization;

namespace MigrationHelpers;

public static partial class LegacyPreferences
{
    static readonly object locker = new object();

    static bool PlatformContainsKey(string key, string sharedName)
    {
        lock (locker)
        {
            return GetUserDefaults(sharedName)[key] != null;
        }
    }

    static void PlatformRemove(string key, string sharedName)
    {
        lock (locker)
        {
            using (var userDefaults = GetUserDefaults(sharedName))
            {
                if (userDefaults[key] != null)
                    userDefaults.RemoveObject(key);
            }
        }
    }

    static T PlatformGet<T>(string key, T defaultValue, string sharedName)
    {
        object value = null;

        lock (locker)
        {
            using (var userDefaults = GetUserDefaults(sharedName))
            {
```

```

        if (userDefaults[key] == null)
            return defaultValue;

        switch (defaultValue)
        {
            case int i:
                value = (int)(nint)userDefaults.IntForKey(key);
                break;
            case bool b:
                value = userDefaults.BoolForKey(key);
                break;
            case long l:
                var savedLong = userDefaults.StringForKey(key);
                value = Convert.ToInt64(savedLong,
CultureInfo.InvariantCulture);
                break;
            case double d:
                value = userDefaults.DoubleForKey(key);
                break;
            case float f:
                value = userDefaults.FloatForKey(key);
                break;
            case string s:
                // the case when the string is not null
                value = userDefaults.StringForKey(key);
                break;
            default:
                // the case when the string is null
                if (typeof(T) == typeof(string))
                    value = userDefaults.StringForKey(key);
                break;
        }
    }

    return (T)value;
}

static NSUserDefaults GetUserDefaults(string sharedName)
{
    if (!string.IsNullOrEmpty(sharedName))
        return new NSUserDefaults(sharedName,
NSUserDefaultsType.SuiteName);
    else
        return NSUserDefaults.StandardUserDefaults;
}
}

```

Windows

On Windows, the `LegacyVersionTracking` class provides the preferences container implementation that retrieves data from [ApplicationDataContainer](#). The following code

shows the `LegacyPreferences` class:

⚠ Note

To use this code, add it to a class named `LegacyPreferences` in the *Platforms\Windows* folder of your .NET MAUI app project.

C#

```
using Windows.Storage;

namespace MigrationHelpers;

public static partial class LegacyPreferences
{
    static readonly object locker = new object();

    static bool PlatformContainsKey(string key, string sharedName)
    {
        lock (locker)
        {
            var appDataContainer = GetApplicationDataContainer(sharedName);
            return appDataContainer.Values.ContainsKey(key);
        }
    }

    static void PlatformRemove(string key, string sharedName)
    {
        lock (locker)
        {
            var appDataContainer = GetApplicationDataContainer(sharedName);
            if (appDataContainer.Values.ContainsKey(key))
                appDataContainer.Values.Remove(key);
        }
    }

    static T PlatformGet<T>(string key, T defaultValue, string sharedName)
    {
        lock (locker)
        {
            var appDataContainer = GetApplicationDataContainer(sharedName);
            if (appDataContainer.Values.ContainsKey(key))
            {
                var tempValue = appDataContainer.Values[key];
                if (tempValue != null)
                    return (T)tempValue;
            }
        }

        return defaultValue;
    }
}
```

```

    static ApplicationDataContainer GetApplicationDataContainer(string
sharedName)
    {
        var localSettings = ApplicationData.Current.LocalSettings;
        if (string.IsNullOrEmpty(sharedName))
            return localSettings;

        if (!localSettings.Containers.ContainsKey(sharedName))
            localSettings.CreateContainer(sharedName,
ApplicationDataCreateDisposition.Always);

        return localSettings.Containers[sharedName];
    }
}

```

Consume legacy version tracking data

The `LegacyVersionTracking` class can be used to retrieve version tracking data on Android, iOS, and Windows that was created with a previous Xamarin.Forms version of your app:

C#

```

#if ANDROID || IOS || WINDOWS
using MigrationHelpers;
...

string isFirstLaunchEver =
LegacyVersionTracking.IsFirstLaunchEver.ToString();
string currentVersionIsFirst =
LegacyVersionTracking.IsFirstLaunchForCurrentVersion.ToString();
string currentBuildIsFirst =
LegacyVersionTracking.IsFirstLaunchForCurrentBuild.ToString();
string currentVersion = LegacyVersionTracking.CurrentVersion.ToString();
string currentBuild = LegacyVersionTracking.CurrentBuild.ToString();
string firstInstalledVer =
LegacyVersionTracking.FirstInstalledVersion.ToString();
string firstInstalledBuild =
LegacyVersionTracking.FirstInstalledBuild.ToString();
string versionHistory = String.Join(',',
LegacyVersionTracking.VersionHistory);
string buildHistory = String.Join(',', LegacyVersionTracking.BuildHistory);
string previousVersion = LegacyVersionTracking.PreviousVersion?.ToString()
?? "none";
string previousBuild = LegacyVersionTracking.PreviousBuild?.ToString() ??
"none";
#endif

```

This example shows using the `LegacyVersionTracking` class to read legacy version tracking data. However, this data can't be assigned to .NET MAUI's `VersionTracking` class, because its properties can't be set. Instead, the data can be written to .NET MAUI preferences with the `WriteHistory` method:

C#

```
void WriteHistory(string key, IEnumerable<string> history)
{
    Preferences.Default.Set(key, string.Join("|", history), $"
{AppInfo.Current.PackageName}.microsoft.maui.essentials.versiontracking");
}

#if ANDROID || IOS || WINDOWS
WriteHistory(LegacyVersionTracking.VersionsKey,
LegacyVersionTracking.VersionHistory);
WriteHistory(LegacyVersionTracking.BuildsKey,
LegacyVersionTracking.BuildHistory);
#endif
```

Once the legacy version tracking data has been written to .NET MAUI preferences, it can be consumed by .NET MAUI's `VersionTracking` class:

C#

```
var mauiVersionHistory = VersionTracking.Default.VersionHistory;
var mauiBuildHistory = VersionTracking.Default.BuildHistory;
```

The legacy version tracking data can then be removed from the device:

C#

```
#if ANDROID || IOS || WINDOWS
LegacyPreferences.Remove(LegacyVersionTracking.VersionsKey,
LegacyVersionTracking.SharedName);
LegacyPreferences.Remove(LegacyVersionTracking.BuildsKey,
LegacyVersionTracking.SharedName);
#endif
```

Migrate Azure Notification Hub code from Xamarin.Forms to .NET MAUI

Article • 07/18/2024

In Xamarin.Forms apps, the [Xamarin.Azure.NotificationHubs.iOS](#) and [Xamarin.Azure.NotificationHubs.Android](#) NuGet packages could be used to register for push notifications with Azure Notification Hubs. However, these NuGet packages are no longer supported and you must take an alternative approach to performing device installation with an Azure Notification Hub from a .NET Multi-platform App UI (.NET MAUI) app.

One possible approach is to use the Azure Notification Hubs REST APIs to manage installations and send notifications. This can be accomplished from a backend service, or directly from a device. For more information, see [How to use the Notification Hubs REST Interface](#).

Alternatively, a .NET wrapper around the REST APIs is available via the [Microsoft.Azure.NotificationHubs](#) NuGet package. This NuGet package can be used to handle device installation for a .NET MAUI app, and to initiate a push notification. For more information on this approach, see [Registration management from a backend](#).

Important

Specific Azure Notification Hub SDKs aren't provided for .NET for Android, .NET for iOS, and .NET MAUI. Instead, a [.NET SDK](#) can be used by apps built with .NET.

For information on using [Azure Notification Hubs](#) to send push notifications to a .NET MAUI app targeting Android and iOS, see [Send push notifications to .NET MAUI apps using Azure Notification Hubs via a backend service](#).

Firebase Cloud Messaging support

Google deprecated the Firebase Cloud Messaging (FCM) legacy API for HTTP and XMPP on June 20, 2023, and apps that use this API should migrate to the HTTP v1 API at the earliest opportunity. For more information, see [Migrate from legacy FCM APIs to HTTP v1](#) on [firebase.google.com](#).

If you have users who've already registered for push notifications via the legacy FCM API they'll need migrating to the FCMv1 API. When you send a notification through FCMv1,

the notification will only be sent to devices registered through the FCMv1 API. Therefore, it's recommended to migrate to FCMv1 and ensure that all new device registrations occur through FCMv1. For existing registrations, you'll need to re-register all active device tokens through FCMv1.

Azure Notification Hubs provide support for the FCMv1 API. For more information, see [Azure Notification Hubs and Google Firebase Cloud Messaging migration](#).

XAML

Article • 07/25/2023

The eXtensible Application Markup Language (XAML) is an XML-based language that's an alternative to programming code for instantiating and initializing objects, and organizing those objects in parent-child hierarchies.

XAML allows developers to define user interfaces in .NET Multi-platform App UI (.NET MAUI) apps using markup rather than code. XAML is not required in a .NET MAUI app, but it is the recommended approach to developing your UI because it's often more succinct, more visually coherent, and has tooling support. XAML is also well suited for use with the Model-View-ViewModel (MVVM) pattern, where XAML defines the view that is linked to viewmodel code through XAML-based data bindings.

Within a XAML file, you can define user interfaces using all the .NET MAUI views, layouts, and pages, as well as custom classes. The XAML file can be either compiled or embedded in the app package. Either way, the XAML is parsed at build time to locate named objects, and at runtime the objects represented by the XAML are instantiated and initialized.

XAML has several advantages over equivalent code:

- XAML is often more succinct and readable than equivalent code.
- The parent-child hierarchy inherent in XML allows XAML to mimic with greater visual clarity the parent-child hierarchy of user-interface objects.

There are also disadvantages, mostly related to limitations that are intrinsic to markup languages:

- XAML cannot contain code. All event handlers must be defined in a code file.
- XAML cannot contain loops for repetitive processing. However there are controls that display collections of data, such as [ListView](#) and [CollectionView](#).
- XAML cannot contain conditional processing. However, a data-binding can reference a code-based binding converter that effectively allows some conditional processing.
- XAML generally cannot instantiate classes that do not define a parameterless constructor, although this restriction can sometimes be overcome.
- XAML generally cannot call methods, although this restriction can sometimes be overcome.

There is no visual designer for producing XAML in .NET MAUI apps. All XAML must be hand-written, but you can use XAML hot reload to view your UI as you edit it.

XAML is basically XML, but XAML has some unique syntax features. The most important are:

- Property elements
- Attached properties
- Markup extensions

These features are *not* XML extensions. XAML is entirely legal XML. But these XAML syntax features use XML in unique ways.

Get started with XAML

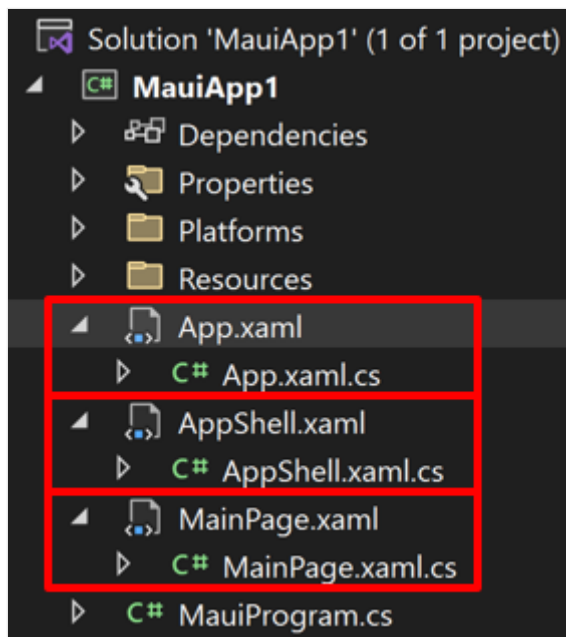
Article • 08/30/2024

 [Browse the sample](#)

In a .NET Multi-platform App UI (.NET MAUI) app, XAML is mostly used to define the visual contents of a page and works together with a C# code-behind file. The code-behind file provides code support for the markup. Together, these two files contribute to a new class definition that includes child views and property initialization. Within the XAML file, classes and properties are referenced with XML elements and attributes, and links between the markup and code are established.

Anatomy of a XAML file

A new .NET MAUI app contains three XAML files, and their associated code-behind files:



The first file pairing is *App.xaml*, a XAML file, and *App.xaml.cs*, a C# code-behind file associated with the XAML file. Both *App.xaml* and *App.xaml.cs* contribute to a class named `App` that derives from `Application`. The second file pairing is *AppShell.xaml* and *AppShell.xaml.cs*, which contribute to a class named `AppShell` that derives from `Shell`. Most other classes with XAML files contribute to a class that derives from `ContentPage`, and define the UI of a page. This is true of the *MainPage.xaml* and *MainPage.xaml.cs* files.

The *MainPage.xaml* file has the following structure:

XAML


```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MyMauiApp.MainPage">
    ...
</ContentPage>
```

The two XML namespace (`xmlns`) declarations refer to URIs on microsoft.com. However, there's no content at these URIs, and they basically function as version identifiers.

The first XML namespace declaration means that tags defined within the XAML file with no prefix refer to classes in .NET MAUI, for example `ContentPage`. The second namespace declaration defines a prefix of `x`. This is used for several elements and attributes that are intrinsic to XAML itself and which are supported by other implementations of XAML. However, these elements and attributes are slightly different depending on the year embedded in the URI. .NET MAUI supports the [2009 XAML specification](#).

At the end of the first tag, the `x` prefix is used for an attribute named `Class`. Because the use of this `x` prefix is virtually universal for the XAML namespace, XAML attributes such as `Class` are almost always referred to as `x:Class`. The `x:Class` attribute specifies a fully qualified .NET class name: the `MainPage` class in the `MyMauiApp` namespace. This means that this XAML file defines a new class named `MainPage` in the `MyMauiApp` namespace that derives from `ContentPage` (the tag in which the `x:Class` attribute appears).

The `x:Class` attribute can only appear in the root element of a XAML file to define a derived C# class. This is the only new class defined in the XAML file. Everything else that appears in a XAML file is instead simply instantiated from existing classes and initialized.

The *MainPage.xaml.cs* file looks similar to this:

```
C#

namespace MyMauiApp;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
    }
}
```

The `MainPage` class derives from `ContentPage`, and is a [partial class](#) definition.

When Visual Studio builds the project, a source generator generates new C# source that contains the definition of the `InitializeComponent` method that's called from the `MainPage` constructor and adds it to the compilation object.

At runtime, code in the `MauiProgram` class bootstraps the app and executes the `App` class constructor, which instantiates `AppShell`. The `AppShell` class instantiates the first page of the app to be displayed, which is `MainPage`. The `MainPage` constructor calls `InitializeComponent`, which initializes all the objects defined in the XAML file, connects them all together in parent-child relationships, attaches event handlers defined in code to events set in the XAML file, and sets the resultant tree of objects as the content of the page.

ⓘ Note

The `AppShell` class uses .NET MAUI Shell to set the first page of the app to be displayed. However, Shell is beyond the scope of this introduction to XAML. For more information, see [.NET MAUI Shell](#).

Set page content

A `ContentPage` should contain a single child, that can be a view or a layout with child views. The child of the `ContentPage` is automatically set as the value of the `ContentPage.Content` property.

The following example shows a `ContentPage` containing a `Label`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.HelloXamlPage"
             Title="Hello XAML Page">
    <Label Text="Hello, XAML!"
           VerticalOptions="Center"
           HorizontalTextAlignment="Center"
           Rotation="-15"
           FontSize="18"
           FontAttributes="Bold"
           TextColor="Blue" />
</ContentPage>
```

From the example above the relationship between classes, properties, and XML should be evident. A .NET MAUI class (such as `ContentPage` or `Label`) appears in the XAML file

as an XML element. Properties of that class—including `Title` on `ContentPage` and seven properties of `Label` usually appear as XML attributes.

Many shortcuts exist to set the values of these properties. Some properties are basic data types. For example, the `Title` and `Text` properties are of type `string`, and `Rotation` is of type `double`. The `HorizontalTextAlignment` property is of type `TextAlignment`, which is an enumeration. For a property of any enumeration type, all you need to supply is a member name.

For properties of more complex types, however, converters are used for parsing the XAML. These are classes in .NET MAUI that derive from `TypeConverter`. For the example above, several .NET MAUI converters are automatically applied to convert string values to their correct type:

- `LayoutOptionsConverter` for the `VerticalOptions` property. This converter converts the names of public static fields of the `LayoutOptions` structure to values of type `LayoutOptions`.
- `ColorTypeConverter` for the `TextColor` property. This converter converts the names of public static fields of the `Colors` class or hexadecimal RGB values, with or without an alpha channel.

Page navigation

When you run a .NET MAUI app, the `MainPage` is typically displayed. To see a different page you can either set that as the new startup page in the `AppShell.xaml` file, or navigate to the new page from `MainPage`.

To implement navigation, in the `MainPage.xaml.cs` constructor, you can create a simple `Button` and use the event handler to navigate to `HelloXamlPage`:

C#

```
public MainPage()
{
    InitializeComponent();

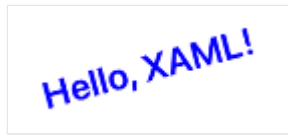
    Button button = new Button
    {
        Text = "Navigate!",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.Center
    };

    button.Clicked += async (sender, args) =>
    {
```

```
        await Navigation.PushAsync(new HelloXamlPage());
    };

    Content = button;
}
```

When you compile and deploy the new version of this app, a button appears on the screen. Pressing it navigates to `HelloXamlPage`:



You can navigate back to `MainPage` using the navigation bar that appears on each platform.

ⓘ Note

An alternative to this navigation model is to use .NET MAUI Shell. For more information, see [.NET MAUI Shell overview](#).

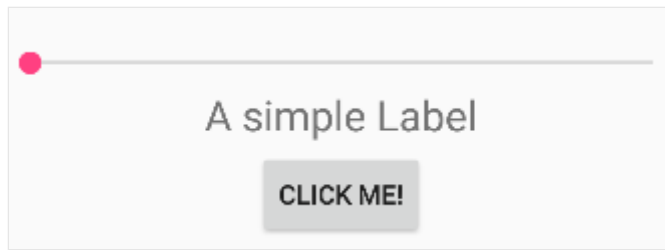
XAML and code interactions

The child of most `ContentPage` derivatives is a layout, such as a `StackLayout` or a `Grid`, and the layout can contain multiple children. In XAML, these parent-child relationships are established with normal XML hierarchy:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.XamlPlusCodePage"
             Title="XAML + Code Page">
    <StackLayout>
        <Slider VerticalOptions="Center" />
        <Label Text="A simple Label"
              FontSize="18"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
        <Button Text="Click Me!"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

This XAML file is syntactically complete, and produces the following UI:



However, while you can interact with the [Slider](#) and [Button](#), the UI isn't updated. The [Slider](#) should cause the [Label](#) to display the current value, and the [Button](#) should do something.

Displaying a [Slider](#) value using a [Label](#) can be achieved entirely in XAML with a *data binding*. However, it's useful to see the code solution first. Even so, handling the [Button](#) click definitely requires code. This means that the code-behind file for `XamlPlusCodePage` must contain handlers for the `ValueChanged` event of the [Slider](#) and the `Clicked` event of the [Button](#):

C#

```
namespace.XamlSamples
{
    public partial class XamlPlusCodePage
    {
        public XamlPlusCodePage()
        {
            InitializeComponent();
        }

        void OnSliderValueChanged(object sender, ValueChangedEventArgs args)
        {
            valueLabel.Text = args.NewValue.ToString("F3");
        }

        async void OnButtonClicked(object sender, EventArgs args)
        {
            Button button = (Button)sender;
            await DisplayAlert("Clicked!", "The button labeled '" +
            button.Text + "' has been clicked", "OK");
        }
    }
}
```

Back in the XAML file, the [Slider](#) and [Button](#) tags need to include attributes for the `ValueChanged` and `Clicked` events that reference these handlers:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="XamlSamples.XamlPlusCodePage"
              Title="XAML + Code Page">
    <StackLayout>
        <Slider VerticalOptions="Center"
                ValueChanged="OnSliderValueChanged" />
        <Label x:Name="valueLabel"
                Text="A simple Label"
                FontSize="18"
                HorizontalOptions="Center"
                VerticalOptions="Center" />
        <Button Text="Click Me!"
                HorizontalOptions="Center"
                VerticalOptions="Center"
                Clicked="OnButtonClicked" />
    </StackLayout>
</ContentPage>

```

Notice that assigning a handler to an event has the same syntax as assigning a value to a property. In addition, for the `ValueChanged` event handler of the `Slider` to use the `Label` to display the current value, the handler needs to reference that object from code. Therefore, the `Label` needs a name, which is specified with the `x:Name` attribute. The `x` prefix of the `x:Name` attribute indicates that this attribute is intrinsic to XAML. The name you assign to the `x:Name` attribute has the same rules as C# variable names. For example, it must begin with a letter or underscore and contain no embedded spaces.

The `ValueChanged` event handler can now set the `Label` to display the new `Slider` value, which is available from the event arguments:

C#

```

void OnSliderValueChanged(object sender, ValueChangedEventArgs args)
{
    valueLabel.Text = args.NewValue.ToString("F3");
}

```

Alternatively, the handler could obtain the `Slider` object that is generating this event from the `sender` argument and obtain the `Value` property from that:

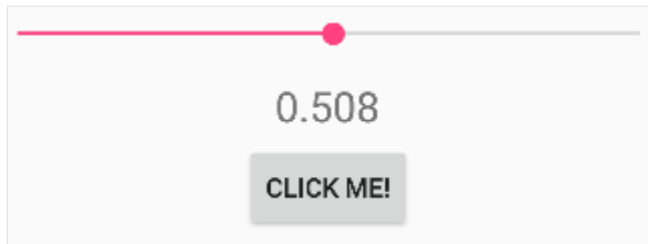
C#

```

void OnSliderValueChanged(object sender, ValueChangedEventArgs args)
{
    valueLabel.Text = ((Slider)sender).Value.ToString("F3");
}

```

The result is that any manipulation of the [Slider](#) causes its value to be displayed in the [Label](#):



In the example above the [Button](#) simulates a response to a `Clicked` event by displaying an alert with the `Text` of the button. Therefore, the event handler can cast the `sender` argument to a [Button](#) and then access its properties:

C#

```
async void OnButtonClicked(object sender, EventArgs args)
{
    Button button = (Button)sender;
    await DisplayAlert("Clicked!", "The button labeled '" + button.Text + "'
has been clicked", "OK");
}
```

The `OnButtonClicked` method is defined as `async` because the [DisplayAlert](#) method is asynchronous and should be prefaced with the `await` operator, which returns when the method completes. Because this method obtains the [Button](#) firing the event from the `sender` argument, the same handler could be used for multiple buttons.

Next steps

XAML is mostly designed for instantiating and initializing objects. But often, properties must be set to complex objects that cannot easily be represented as XML strings, and sometimes properties defined by one class must be set on a child class. These two needs require the essential XAML syntax features of *property elements* and *attached properties*.

Essential XAML syntax

Essential XAML syntax

Article • 08/30/2024

 [Browse the sample](#)

XAML is mostly designed for instantiating and initializing objects. But often, properties must be set to complex objects that cannot easily be represented as XML strings, and sometimes properties defined by one class must be set on a child class. These two needs require the essential XAML syntax features of *property elements* and *attached properties*.

Property elements

In .NET Multi-platform App UI (.NET MAUI) XAML, properties of classes are normally set as XML attributes:

XAML

```
<Label Text="Hello, XAML!"
      VerticalOptions="Center"
      FontAttributes="Bold"
      FontSize="18"
      TextColor="Aqua" />
```

However, there is an alternative way to set a property in XAML:

XAML

```
<Label Text="Hello, XAML!"
      VerticalOptions="Center"
      FontAttributes="Bold"
      FontSize="18">
  <Label.TextColor>
    Aqua
  </Label.TextColor>
</Label>
```

These two examples that specify the `TextColor` property are functionally equivalent, and enable the introduction of some basic terminology:

- `Label` is an *object element*. It is a .NET MAUI object expressed as an XML element.
- `Text`, `VerticalOptions`, `FontAttributes` and `FontSize` are *property attributes*. They are .NET MAUI properties expressed as XML attributes.

- In the second example, `TextColor` has become a *property element*. It is a .NET MAUI property expressed as an XML element.

ⓘ Note

In a property element, the value of the property is always defined as the content between the property-element start and end tags.

Property-element syntax can also be used on more than one property of an object:

XAML

```
<Label Text="Hello, XAML!"
      VerticalOptions="Center">
  <Label.FontAttributes>
    Bold
  </Label.FontAttributes>
  <Label.FontSize>
    Large
  </Label.FontSize>
  <Label.TextColor>
    Aqua
  </Label.TextColor>
</Label>
```

While property-element syntax might seem unnecessary, it's essential when the value of a property is too complex to be expressed as a simple string. Within the property-element tags you can instantiate another object and set its properties. For example, the [Grid](#) layout has properties named `RowDefinitions` and `ColumnDefinitions`, which are of type `RowDefinitionCollection` and `ColumnDefinitionCollection` respectively. These types are collections of `RowDefinition` and `ColumnDefinition` objects, and you typically use property element syntax to set them:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.GridDemoPage"
             Title="Grid Demo Page">
  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="*" />
      <RowDefinition Height="100" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="Auto" />
```

```

        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="100" />
    </Grid.ColumnDefinitions>

    ...
</Grid>
</ContentPage>

```

Attached properties

In the previous example you saw that the [Grid](#) requires property elements for the `RowDefinitions` and `ColumnDefinitions` collections to define the rows and columns. This suggests that there must be a technique for indicating the row and column where each child of the [Grid](#) resides.

Within the tag for each child of the [Grid](#) you specify the row and column of that child using the `Grid.Row` and `Grid.Column` attributes, which have default values of 0. You can also indicate if a child spans more than one row or column with the `Grid.RowSpan` and `Grid.ColumnSpan` attributes, which have default values of 1.

The following example demonstrates placing children within a [Grid](#):

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="XamlSamples.GridDemoPage"
    Title="Grid Demo Page">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="100" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto" />
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="100" />
        </Grid.ColumnDefinitions>

        <Label Text="Autosized cell"
            TextColor="White"
            BackgroundColor="Blue" />
        <BoxView Color="Silver"
            Grid.Column="1" />
        <BoxView Color="Teal"
            Grid.Row="1" />
        <Label Text="Leftover space"
            Grid.Row="1" Grid.Column="1"
            TextColor="Purple"

```

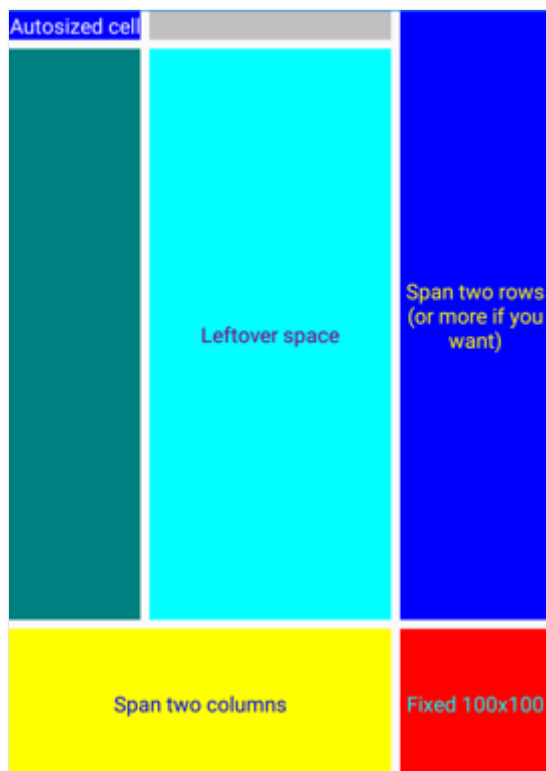
```

        BackgroundColor="Aqua"
        HorizontalTextAlignment="Center"
        VerticalTextAlignment="Center" />
<Label Text="Span two rows (or more if you want)"
        Grid.Column="2" Grid.RowSpan="2"
        TextColor="Yellow"
        BackgroundColor="Blue"
        HorizontalTextAlignment="Center"
        VerticalTextAlignment="Center" />
<Label Text="Span two columns"
        Grid.Row="2" Grid.ColumnSpan="2"
        TextColor="Blue"
        BackgroundColor="Yellow"
        HorizontalTextAlignment="Center"
        VerticalTextAlignment="Center" />
<Label Text="Fixed 100x100"
        Grid.Row="2" Grid.Column="2"
        TextColor="Aqua"
        BackgroundColor="Red"
        HorizontalTextAlignment="Center"
        VerticalTextAlignment="Center" />

</Grid>
</ContentPage>

```

This XAML results in the following layout:



The `Grid.Row`, `Grid.Column`, `Grid.RowSpan`, and `Grid.ColumnSpan` attributes appear to be properties of the `Grid` class, but this class doesn't define anything named `Row`, `Column`, `RowSpan`, or `ColumnSpan`. Instead, the `Grid` class defines four bindable properties named

`RowProperty`, `ColumnProperty`, `RowSpanProperty`, and `ColumnSpanProperty`, that are special types of bindable properties known as *attached properties*. They are defined by the `Grid` class but set on children of the `Grid`.

❗ Note

When you wish to use these attached properties in code, the `Grid` class provides static methods named `GetRow`, `SetRow`, `GetColumn`, `SetColumn`, `GetRowSpan`, `SetRowSpan`, `GetColumnSpan`, and `SetColumnSpan`.

Attached properties are recognizable in XAML as attributes containing both a class and a property name separated by a period. They are called *attached properties* because they are defined by one class (in this case, `Grid`) but attached to other objects (in this case, children of the `Grid`). During layout, the `Grid` can interrogate the values of these attached properties to know where to place each child.

Content properties

In the previous example, the `Grid` object was set to the `Content` property of the `ContentPage`. However, the `Content` property wasn't referenced in the XAML but can be:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.XamlPlusCodePage"
             Title="XAML + Code Page">
    <ContentPage.Content>
        <Grid>
            ...
        </Grid>
    </ContentPage.Content>
</ContentPage>
```

The `Content` property isn't required in XAML because elements defined for use in .NET MAUI XAML are allowed to have one property specified as the `ContentProperty` attribute on the class:

C#

```
[ContentProperty("Content")]
public class ContentPage : TemplatedPage
{
```

```
...  
}
```

Any property specified as the [ContentProperty](#) of a class means that the property-element tags for the property are not required. Therefore, the example above specifies that any XAML content that appears between the start and end [ContentPage](#) tags is assigned to the `Content` property.

Many classes also have [ContentProperty](#) attribute definitions. For example, the content property of [Label](#) is `Text`.

Platform differences

.NET MAUI apps can customize UI appearance on a per-platform basis. This can be achieved in XAML using the `OnPlatform` and `On` classes:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="...">  
    <ContentPage.Padding>  
        <OnPlatform x:TypeArguments="Thickness">  
            <On Platform="iOS" Value="0,20,0,0" />  
            <On Platform="Android" Value="10,20,20,10" />  
        </OnPlatform>  
    </ContentPage.Padding>  
    ...  
</ContentPage>
```

`OnPlatform` is a generic class and so you need to specify the generic type argument, in this case, `Thickness`, which is the type of `Padding` property. This is achieved with the `x:TypeArguments` XAML attribute. The `OnPlatform` class defines a `Default` property that can be set to a value that will be applied to all platforms:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="...">  
    <ContentPage.Padding>  
        <OnPlatform x:TypeArguments="Thickness" Default="20">  
            <On Platform="iOS" Value="0,20,0,0" />  
            <On Platform="Android" Value="10,20,20,10" />  
        </OnPlatform>  
    </ContentPage.Padding>
```

```
...  
</ContentPage>
```

In this example, the `Padding` property is set to different values on iOS and Android, with the other platforms being set to the default value.

The `OnPlatform` class also defines a `Platforms` property, which is an `IList` of `On` objects. Each `On` object can set the `Platform` and `Value` property to define the `Thickness` value for a specific platform. In addition, the `Platform` property of `On` is of type `IList<string>`, so you can include multiple platforms if the values are the same:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="...">  
    <ContentPage.Padding>  
        <OnPlatform x:TypeArguments="Thickness" Default="20">  
            <On Platform="iOS, Android" Value="10,20,20,10" />  
        </OnPlatform>  
    </ContentPage.Padding>  
    ...  
</ContentPage>
```

This is the standard way to set a platform-dependent `Padding` property in XAML.

ⓘ Note

If the `Value` property of an `On` object can't be represented by a single string, you can define property elements for it.

For more information, see [Customize UI appearance based on the platform](#).

Next steps

.NET MAUI XAML markup extensions enable properties to be set to objects or values that are referenced indirectly from other sources. XAML markup extensions are particularly important for sharing objects, and referencing constants used throughout an app.

XAML markup extensions

XAML markup extensions

Article • 08/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) XAML markup extensions enable properties to be set to objects or values that are referenced indirectly from other sources. XAML markup extensions are particularly important for sharing objects, and referencing constants used throughout an app, but they find their greatest utility in data bindings.

Typically, you use XAML to set properties of an object to explicit values, such as a string, a number, an enumeration member, or a string that is converted to a value behind the scenes. Sometimes, however, properties must instead reference values defined somewhere else, or which might require a little processing by code at runtime. For these purposes, XAML *markup extensions* are available.

XAML markup extensions are so named because they are backed by code in classes that implement [IMarkupExtension](#). It's also possible to write your own custom markup extensions.

In many cases, XAML markup extensions are instantly recognizable in XAML files because they appear as attribute values delimited by curly braces, { and }, but sometimes markup extensions also appear in markup as conventional elements.

Important

Markup extensions can have properties, but they are not set like XML attributes. In a markup extension, property settings are separated by commas, and no quotation marks appear within the curly braces.

Shared resources

Some XAML pages contain several views with properties set to the same values. For example, many of the property settings for these [Button](#) objects are the same:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="XamlSamples.SharedResourcesPage"
              Title="Shared Resources Page">
    <StackLayout>
```

```

<Button Text="Do this!"
        HorizontalOptions="Center"
        VerticalOptions="Center"
        BorderWidth="3"
        Rotation="-15"
        TextColor="Red"
        FontSize="24" />
<Button Text="Do that!"
        HorizontalOptions="Center"
        VerticalOptions="Center"
        BorderWidth="3"
        Rotation="-15"
        TextColor="Red"
        FontSize="24" />
<Button Text="Do the other thing!"
        HorizontalOptions="Center"
        VerticalOptions="Center"
        BorderWidth="3"
        Rotation="-15"
        TextColor="Red"
        FontSize="24" />
</StackLayout>
</ContentPage>

```

If one of these properties needs to be changed, you might prefer to make the change just once rather than three times. If this were code, you'd likely be using constants and static read-only objects to help keep such values consistent and easy to modify.

In XAML, one popular solution is to store such values or objects in a *resource dictionary*. The `VisualElement` class defines a property named `Resources` of type `ResourceDictionary`, which is a dictionary with keys of type `string` and values of type `object`. You can put objects into this dictionary and then reference them from markup, all in XAML.

To use a resource dictionary on a page, include a pair of `Resources` property-element tags at the top of the page, and add resources within these tags. Objects and values of various types can be added to the resource dictionary. These types must be instantiable. They can't be abstract classes, for example. These types must also have a public parameterless constructor. Each item requires a dictionary key specified with the `x:Key` attribute:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.SharedResourcesPage"
             Title="Shared Resources Page">
  <ContentPage.Resources>
    <LayoutOptions x:Key="horzOptions"

```



```

        Alignment="Center" />
    <LayoutOptions x:Key="vertOptions"
        Alignment="Center" />
</ContentPage.Resources>
...
</ContentPage>

```

In this example, the two resources are values of the structure type `LayoutOptions`, and each has a unique key and one or two properties set. In code and markup, it's much more common to use the static fields of `LayoutOptions`, but here it's more convenient to set the properties.

ⓘ Note

Optional [ResourceDictionary](#) tags can be included as the child of the `Resources` tags.

The resources can then be consumed by the `Button` objects, by using the `StaticResource` XAML markup extension to set their `HorizontalOptions` and `VerticalOptions` properties:

XAML

```

<Button Text="Do this!"
    HorizontalOptions="{StaticResource horzOptions}"
    VerticalOptions="{StaticResource vertOptions}"
    BorderWidth="3"
    Rotation="-15"
    TextColor="Red"
    FontSize="24" />

```

The `StaticResource` markup extension is always delimited with curly braces, and includes the dictionary key. The name `StaticResource` distinguishes it from `DynamicResource`, which .NET MAUI also supports. `DynamicResource` is for dictionary keys associated with values that might change at runtime, while `StaticResource` accesses elements from the dictionary just once when the elements on the page are constructed. Whenever the XAML parser encounters a `StaticResource` markup extension, it searches up the visual tree and uses the first `ResourceDictionary` it encounters containing that key.

It's necessary to store doubles in the dictionary for the `BorderWidth`, `Rotation`, and `FontSize` properties. XAML conveniently defines tags for common data types like `x:Double` and `x:Int32`:

XAML

```

<ContentPage.Resources>
    <LayoutOptions x:Key="horzOptions"
        Alignment="Center" />
    <LayoutOptions x:Key="vertOptions"
        Alignment="Center" />
    <x:Double x:Key="borderWidth">3</x:Double>
    <x:Double x:Key="rotationAngle">-15</x:Double>
    <x:Double x:Key="fontSize">24</x:Double>
</ContentPage.Resources>

```

These additional three resources can be referenced in the same way as the `LayoutOptions` values:

XAML

```

<Button Text="Do this!"
    HorizontalOptions="{StaticResource horzOptions}"
    VerticalOptions="{StaticResource vertOptions}"
    BorderWidth="{StaticResource borderWidth}"
    Rotation="{StaticResource rotationAngle}"
    TextColor="Red"
    FontSize="{StaticResource fontSize}" />

```

For resources of type `Color`, you can use the same string representations that you use when directly assigning attributes of these types. Type converters included in .NET MAUI are invoked when the resource is created. It's also possible to use the `OnPlatform` class within the resource dictionary to define different values for the platforms. The following example uses this class for setting different text colors:

XAML

```

<OnPlatform x:Key="textColor"
    x:TypeArguments="Color">
    <On Platform="iOS" Value="Red" />
    <On Platform="Android" Value="Aqua" />
</OnPlatform>

```

The `OnPlatform` resource gets an `x:Key` attribute because it's an object in the dictionary, and an `x:TypeArguments` attribute because it's a generic class. The `iOS`, and `Android` attributes are converted to `Color` values when the object is initialized.

The following example shows the three buttons accessing six shared values:

XAML

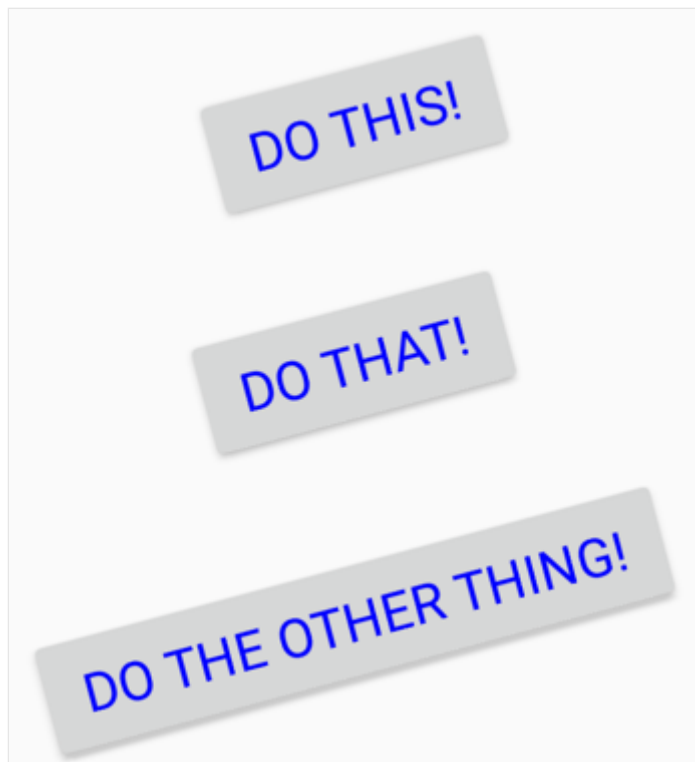
```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="XamlSamples.SharedResourcesPage"
              Title="Shared Resources Page">
  <ContentPage.Resources>
    <LayoutOptions x:Key="horzOptions"
                  Alignment="Center" />
    <LayoutOptions x:Key="vertOptions"
                  Alignment="Center" />
    <x:Double x:Key="borderWidth">3</x:Double>
    <x:Double x:Key="rotationAngle">-15</x:Double>
    <x:Double x:Key="fontSize">24</x:Double>
    <OnPlatform x:Key="textColor"
                x:TypeArguments="Color">
      <On Platform="iOS" Value="Red" />
      <On Platform="Android" Value="Aqua" />
      <On Platform="WinUI" Value="#80FF80" />
    </OnPlatform>
  </ContentPage.Resources>

  <StackLayout>
    <Button Text="Do this!"
            HorizontalOptions="{StaticResource horzOptions}"
            VerticalOptions="{StaticResource vertOptions}"
            BorderWidth="{StaticResource borderWidth}"
            Rotation="{StaticResource rotationAngle}"
            TextColor="{StaticResource textColor}"
            FontSize="{StaticResource fontSize}" />
    <Button Text="Do that!"
            HorizontalOptions="{StaticResource horzOptions}"
            VerticalOptions="{StaticResource vertOptions}"
            BorderWidth="{StaticResource borderWidth}"
            Rotation="{StaticResource rotationAngle}"
            TextColor="{StaticResource textColor}"
            FontSize="{StaticResource fontSize}" />
    <Button Text="Do the other thing!"
            HorizontalOptions="{StaticResource horzOptions}"
            VerticalOptions="{StaticResource vertOptions}"
            BorderWidth="{StaticResource borderWidth}"
            Rotation="{StaticResource rotationAngle}"
            TextColor="{StaticResource textColor}"
            FontSize="{StaticResource fontSize}" />
  </StackLayout>
</ContentPage>

```

The following screenshot verifies the consistent styling:



Although it's common to define the `Resources` collection at the top of the page, you can have `Resources` collections on other elements on the page. For example, the following example shows resources added to a [StackLayout](#):

XAML

```
<StackLayout>
  <StackLayout.Resources>
    <Color x:Key="textColor">Blue</Color>
  </StackLayout.Resources>
  ...
</StackLayout>
```

One of the most common types of objects stored in resource dictionaries is the .NET MAUI [Style](#), which defines a collection of property settings. For more information about styles, see [Style apps using XAML](#).

📌 Note

The purpose of a resource dictionary is to share objects. Therefore, it doesn't make sense to put controls such as a [Label](#) or [Button](#) in a resource dictionary. Visual elements can't be shared because the same instance can't appear twice on a page.

x:Static Markup Extension

In addition to the [StaticResource](#) markup extension, there's also an `x:Static` markup extension. However, while [StaticResource](#) returns an object from a resource dictionary, `x:Static` accesses a public static field, a public static property, a public constant field, or an enumeration member.

❗ Note

The [StaticResource](#) markup extension is supported by XAML implementations that define a resource dictionary, while `x:Static` is an intrinsic part of XAML, as the `x` prefix reveals.

The following example demonstrates how `x:Static` can explicitly reference static fields and enumeration members:

XAML

```
<Label Text="Hello, XAML!"
        VerticalOptions="{x:Static LayoutOptions.Start}"
        HorizontalTextAlignment="{x:Static TextAlignment.Center}"
        TextColor="{x:Static Colors.Aqua}" />
```

The main use of the `x:Static` markup extension is in referencing static fields or properties from your own code. For example, here's an `AppConstants` class that contains some static fields that you might want to use on multiple pages throughout an app:

C#

```
namespace.XamlSamples
{
    static class AppConstants
    {
        public static readonly Color BackgroundColor = Colors.Aqua;
        public static readonly Color ForegroundColor = Colors.Brown;
    }
}
```

To reference the static fields of this class in a XAML file, you need to use an XML namespace declaration to indicate where this file is located. Each additional XML namespace declaration defines a new prefix. To access classes local to the root app namespace, such as `AppConstants`, you could use the prefix `local`. The namespace declaration must indicate the CLR (Common Language Runtime) namespace name, also known as the .NET namespace name, which is the name that appears in a C# `namespace` definition or in a `using` directive:

C#

```
xmlns:local="clr-namespace:XamlSamples"
```

You can also define XML namespace declarations for .NET namespaces. For example, here's a `sys` prefix for the standard .NET `System` namespace, which is in the `netstandard` assembly. Because this is another assembly, you must also specify the assembly name, in this case `netstandard`:

C#

```
xmlns:sys="clr-namespace:System;assembly=netstandard"
```

❗ Note

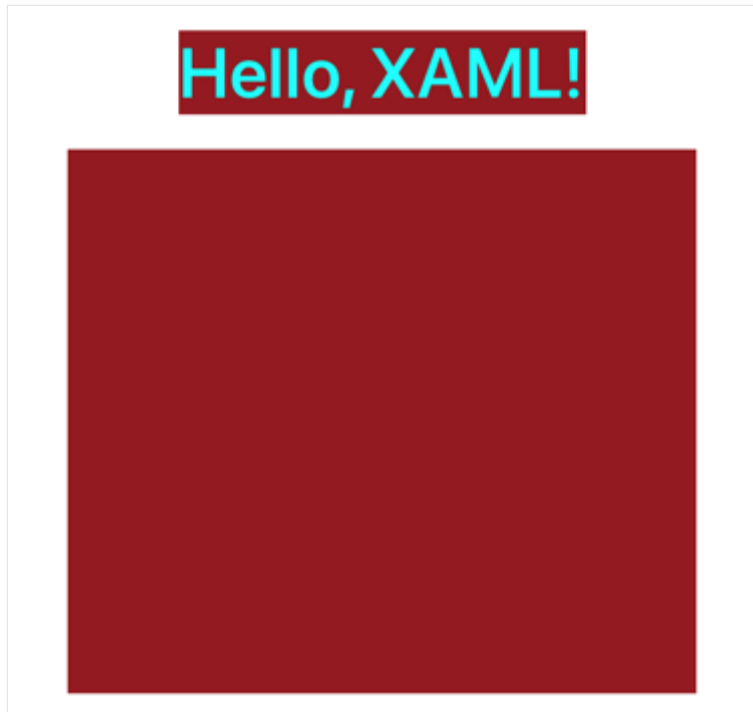
The keyword `clr-namespace` is followed by a colon and then the .NET namespace name, followed by a semicolon, the keyword `assembly`, an equal sign, and the assembly name.

The static fields can then be consumed after declaring the XML namespace:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:XamlSamples"
  xmlns:sys="clr-namespace:System;assembly=netstandard"
  x:Class="XamlSamples.StaticConstantsPage"
  Title="Static Constants Page"
  Padding="5,25,5,0">
  <StackLayout>
    <Label Text="Hello, XAML!"
      TextColor="{x:Static local:AppConstants.BackgroundColor}"
      BackgroundColor="{x:Static
local:AppConstants.ForegroundColor}"
      FontAttributes="Bold"
      FontSize="30"
      HorizontalOptions="Center" />
    <BoxView WidthRequest="{x:Static sys:Math.PI}"
      HeightRequest="{x:Static sys:Math.E}"
      Color="{x:Static local:AppConstants.ForegroundColor}"
      HorizontalOptions="Center"
      VerticalOptions="CenterAndExpand"
      Scale="100" />
  </StackLayout>
</ContentPage>
```

In this example, the [BoxView](#) dimensions are set to `Math.PI` and `Math.E`, but scaled by a factor of 100:



Other markup extensions

Several markup extensions are intrinsic to XAML and supported in .NET MAUI XAML. Some of these are not used very often but are essential when you need them:

- If a property has a non- `null` value by default but you want to set it to `null`, set it to the `{x:Null}` markup extension.
- If a property is of type `Type`, you can assign it to a `Type` object using the markup extension `{x:Type someClass}`.
- You can define arrays in XAML using the `x:Array` markup extension. This markup extension has a required attribute named `Type` that indicates the type of the elements in the array.

For more information about XAML markup extensions, see [Consume XAML markup extensions](#).

Next steps

.NET MAUI data bindings allow properties of two objects to be linked so that a change in one causes a change in the other.

Data binding basics

Data binding basics

Article • 08/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) data bindings allow properties of two objects to be linked so that a change in one causes a change in the other. This is a very valuable tool, and while data bindings can be defined entirely in code, XAML provides shortcuts and convenience.

Data bindings

Data bindings connect properties of two objects, called the *source* and the *target*. In code, two steps are required:

1. The `BindingContext` property of the target object must be set to the source object,
2. The `SetBinding` method (often used in conjunction with the `Binding` class) must be called on the target object to bind a property of that object to a property of the source object.

The target property must be a bindable property, which means that the target object must derive from `BindableObject`. A property of `Label`, such as `Text`, is associated with the bindable property `TextProperty`.

In XAML, you must also perform the same two steps that are required in code, except that the `Binding` markup extension takes the place of the `SetBinding` call and the `Binding` class. However, when you define data bindings in XAML, there are multiple ways to set the `BindingContext` of the target object. Sometimes it's set from the code-behind file, sometimes using a `StaticResource` or `x:Static` markup extension, and sometimes as the content of `BindingContext` property-element tags.

View-to-view bindings

You can define data bindings to link properties of two views on the same page. In this case, you set the `BindingContext` of the target object using the `x:Reference` markup extension.

The following example contains a `Slider` and two `Label` views, one of which is rotated by the `Slider` value and another which displays the `Slider` value:


```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="XamlSamples.SliderBindingsPage"
              Title="Slider Bindings Page">
    <StackLayout>
        <Label Text="ROTATION"
               BindingContext="{x:Reference slider}"
               Rotation="{Binding Path=Value}"
               FontAttributes="Bold"
               FontSize="18"
               HorizontalOptions="Center"
               VerticalOptions="Center" />
        <Slider x:Name="slider"
                Maximum="360"
                VerticalOptions="Center" />
        <Label BindingContext="{x:Reference slider}"
               Text="{Binding Value, StringFormat='The angle is {0:F0}
degrees'}"
               FontAttributes="Bold"
               FontSize="18"
               HorizontalOptions="Center"
               VerticalOptions="Center" />
    </StackLayout>
</ContentPage>

```

The `Slider` contains an `x:Name` attribute that is referenced by the two `Label` views using the `x:Reference` markup extension. The `x:Reference` binding extension defines a property named `Name` to set to the name of the referenced element, in this case `slider`. However, the `ReferenceExtension` class that defines the `x:Reference` markup extension also defines a `ContentProperty` attribute for `Name`, which means that it isn't explicitly required.

The `Binding` markup extension itself can have several properties, just like the `BindingBase` and `Binding` class. The `ContentProperty` for `Binding` is `Path`, but the "Path=" part of the markup extension can be omitted if the path is the first item in the `Binding` markup extension.

The second `Binding` markup extension sets the `StringFormat` property. In .NET MAUI, bindings do not perform any implicit type conversions, and if you need to display a non-string object as a string you must provide a type converter or use `StringFormat`.

❗ Important

Formatting strings must be placed in single quotation marks.

Binding mode

A single view can have data bindings on several of its properties. However, each view can have only one `BindingContext`, so multiple data bindings on that view must all reference properties of the same object.

The solution to this and other problems involves the `Mode` property, which is set to a member of the `BindingMode` enumeration:

- `Default`
- `OneWay` — values are transferred from the source to the target
- `OneWayToSource` — values are transferred from the target to the source
- `TwoWay` — values are transferred both ways between source and target
- `OneTime` — data goes from source to target, but only when the `BindingContext` changes

The following example demonstrates one common use of the `OneWayToSource` and `TwoWay` binding modes:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="XamlSamples.SliderTransformsPage"
             Padding="5"
             Title="Slider Transforms Page">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="Auto" />
        </Grid.ColumnDefinitions>

        <!-- Scaled and rotated Label -->
        <Label x:Name="label"
              Text="TEXT"
              HorizontalOptions="Center"
              VerticalOptions="CenterAndExpand" />

        <!-- Slider and identifying Label for Scale -->
        <Slider x:Name="scaleSlider"
              BindingContext="{x:Reference label}"
```

```

        Grid.Row="1" Grid.Column="0"
        Maximum="10"
        Value="{Binding Scale, Mode=TwoWay}" />
<Label BindingContext="{x:Reference scaleSlider}"
        Text="{Binding Value, StringFormat='Scale = {0:F1}}'"
        Grid.Row="1" Grid.Column="1"
        VerticalTextAlignment="Center" />

<!-- Slider and identifying Label for Rotation -->
<Slider x:Name="rotationSlider"
        BindingContext="{x:Reference label}"
        Grid.Row="2" Grid.Column="0"
        Maximum="360"
        Value="{Binding Rotation, Mode=OneWayToSource}" />
<Label BindingContext="{x:Reference rotationSlider}"
        Text="{Binding Value, StringFormat='Rotation = {0:F0}}'"
        Grid.Row="2" Grid.Column="1"
        VerticalTextAlignment="Center" />

<!-- Slider and identifying Label for RotationX -->
<Slider x:Name="rotationXSlider"
        BindingContext="{x:Reference label}"
        Grid.Row="3" Grid.Column="0"
        Maximum="360"
        Value="{Binding RotationX, Mode=OneWayToSource}" />
<Label BindingContext="{x:Reference rotationXSlider}"
        Text="{Binding Value, StringFormat='RotationX = {0:F0}}'"
        Grid.Row="3" Grid.Column="1"
        VerticalTextAlignment="Center" />

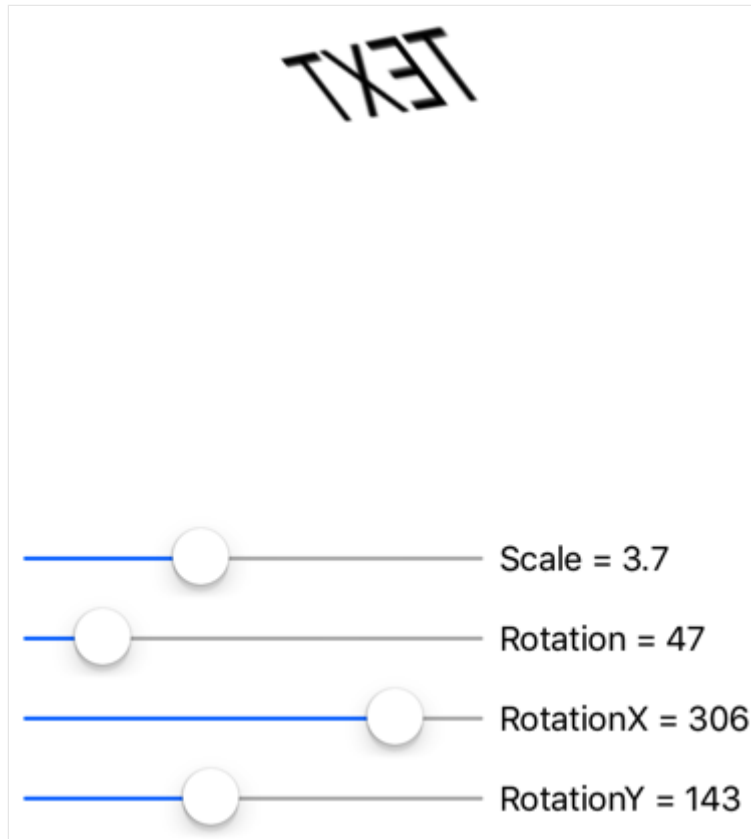
<!-- Slider and identifying Label for RotationY -->
<Slider x:Name="rotationYSlider"
        BindingContext="{x:Reference label}"
        Grid.Row="4" Grid.Column="0"
        Maximum="360"
        Value="{Binding RotationY, Mode=OneWayToSource}" />
<Label BindingContext="{x:Reference rotationYSlider}"
        Text="{Binding Value, StringFormat='RotationY = {0:F0}}'"
        Grid.Row="4" Grid.Column="1"
        VerticalTextAlignment="Center" />
</Grid>
</ContentPage>

```

In this example, four `Slider` views are intended to control the `Scale`, `Rotate`, `RotateX`, and `RotateY` properties of a `Label`. At first, it seems as if these four properties of the `Label` should be data-binding targets because each is being set by a `Slider`. However, the `BindingContext` of `Label` can be only one object, and there are four different sliders. For that reason, the `BindingContext` of each of the four sliders is set to the `Label`, and the bindings are set on the `value` properties of the sliders. By using the `OneWayToSource` and

`TwoWay` modes, these `Value` properties can set the source properties, which are the `Scale`, `Rotate`, `RotateX`, and `RotateY` properties of the `Label`.

The bindings on three of the `Slider` views are `OneWayToSource`, meaning that the `Slider` value causes a change in the property of its `BindingContext`, which is the `Label` named `label`. These three `Slider` views cause changes to the `Rotate`, `RotateX`, and `RotateY` properties of the `Label`:



However, the binding for the `Scale` property is `TwoWay`. This is because the `Scale` property has a default value of 1, and using a `TwoWay` binding causes the `Slider` initial value to be set at 1 rather than 0. If that binding were `OneWayToSource`, the `Scale` property would initially be set to 0 from the `Slider` default value. The `Label` would not be visible.

ⓘ Note

The `VisualElement` class also has `ScaleX` and `ScaleY` properties, which scale the `VisualElement` on the x-axis and y-axis respectively.

Bindings and collections

`ListView` defines an `ItemsSource` property of type `IEnumerable`, and it displays the items in that collection. These items can be objects of any type. By default, `ListView` uses the `ToString` method of each item to display that item. Sometimes this is just what you want, but in many cases, `ToString` returns only the fully-qualified class name of the object.

However, the items in the `ListView` collection can be displayed any way you want through the use of a *template*, which involves a class that derives from `Cell`. The template is cloned for every item in the `ListView`, and data bindings that have been set on the template are transferred to the individual clones. Custom cells can be created for items using the `ViewCell` class.

`ListView` can display a list of every named color that's available in .NET MAUI, with the help of the `NamedColor` class:

C#

```
using System.Reflection;
using System.Text;

namespace XamlSamples
{
    public class NamedColor
    {
        public string Name { get; private set; }
        public string FriendlyName { get; private set; }
        public Color Color { get; private set; }

        // Expose the Color fields as properties
        public float Red => Color.Red;
        public float Green => Color.Green;
        public float Blue => Color.Blue;

        public static IEnumerable<NamedColor> All { get; private set; }

        static NamedColor()
        {
            List<NamedColor> all = new List<NamedColor>();
            StringBuilder stringBuilder = new StringBuilder();

            // Loop through the public static fields of the Color structure.
            foreach (FieldInfo fieldInfo in
typeof(Colors).GetRuntimeFields())
            {
                if (fieldInfo.IsPublic &&
                    fieldInfo.IsStatic &&
                    fieldInfo.FieldType == typeof(Color))
                {
                    // Convert the name to a friendly name.
                    string name = fieldInfo.Name;
```

```

        stringBuilder.Clear();
        int index = 0;

        foreach (char ch in name)
        {
            if (index != 0 && Char.IsUpper(ch))
            {
                stringBuilder.Append(' ');
            }
            stringBuilder.Append(ch);
            index++;
        }

        // Instantiate a NamedColor object.
        NamedColor namedColor = new NamedColor
        {
            Name = name,
            FriendlyName = stringBuilder.ToString(),
            Color = (Color)fieldInfo.GetValue(null)
        };

        // Add it to the collection.
        all.Add(namedColor);
    }
}
all.TrimExcess();
All = all;
}
}
}

```

Each `NamedColor` object has `Name` and `FriendlyName` properties of type `string`, a `Color` property of type `Color`, and `Red`, `Green`, and `Blue` properties. In addition, the `NamedColor` static constructor creates an `IEnumerable<NamedColor>` collection that contains `NamedColor` objects corresponding to the fields of type `Color` in the `Colors` class, and assigns it to its public static `All` property.

Setting the static `NamedColor.All` property to the `ItemsSource` of a `ListView` can be achieved using the `x:Static` markup extension:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:XamlSamples;assembly=XamlSamples"
    x:Class="XamlSamples.ListViewDemoPage"
    Title="ListView Demo Page">
    <ListView ItemsSource="{x:Static local:NamedColor.All}" />
</ContentPage>

```

The result establishes that the items are of type `XamlSamples.NamedColor`:

[illegible]

To define a template for the items, the `ItemTemplate` should be set to a [DataTemplate](#) that references a [ViewCell](#). The [ViewCell](#) should define a layout of one or more views to display each item:

XAML

```
<ListView ItemsSource="{x:Static local:NamedColor.All}">
  <ListView.ItemTemplate>
    <DataTemplate>
      <ViewCell>
        <Label Text="{Binding FriendlyName}" />
      </ViewCell>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

Note

The binding source for cells, and children of cells, is the `ListView.ItemsSource` collection.

In this example, the `Label` element is set to the `View` property of the `ViewCell`. The `ViewCell.View` tags are not needed because the `View` property is the content property of `ViewCell`. This XAML displays the `FriendlyName` property of each `NamedColor` object:

Orange
Pale Goldenrod
Pale Green
Pale Turquoise
Pale Violet Red
Papaya Whip
Peach Puff
Peru
Pink
Plum
Powder Blue
Purple
Red
Rosy Brown
Royal Blue
Saddle Brown
Salmon
Sandy Brown
Sea Green
Sea Shell
Sienna
Silver
Sky Blue
Slate Blue
Slate Gray
Steel Blue

The item template can be expanded to display more information and the actual color:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:XamlSamples"
              x:Class="XamlSamples.ListViewDemoPage"
              Title="ListView Demo Page">
    <ContentPage.Resources>
        <x:Double x:Key="boxSize">50</x:Double>
        <x:Int32 x:Key="rowHeight">60</x:Int32>
        <local:FloatToIntConverter x:Key="intConverter" />
    </ContentPage.Resources>
```



```

<ListView ItemsSource="{x:Static local:NamedColor.All}"
    RowHeight="{StaticResource rowHeight}">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                <StackLayout Padding="5, 5, 0, 5"
                    Orientation="Horizontal"
                    Spacing="15">
                    <BoxView WidthRequest="{StaticResource boxSize}"
                        HeightRequest="{StaticResource boxSize}"
                        Color="{Binding Color}" />
                    <StackLayout Padding="5, 0, 0, 0"
                        VerticalOptions="Center">
                        <Label Text="{Binding FriendlyName}"
                            FontAttributes="Bold"
                            FontSize="14" />
                        <StackLayout Orientation="Horizontal"
                            Spacing="0">
                            <Label Text="{Binding Red,
                                Converter=
{StaticResource intConverter},
ConverterParameter=255,
                                StringFormat='R=
{0:X2}}'" />
                            <Label Text="{Binding Green,
                                Converter=
{StaticResource intConverter},
ConverterParameter=255,
                                StringFormat=', G=
{0:X2}}'" />
                            <Label Text="{Binding Blue,
                                Converter=
{StaticResource intConverter},
ConverterParameter=255,
                                StringFormat=', B=
{0:X2}}'" />
                        </StackLayout>
                    </StackLayout>
                </StackLayout>
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
</ContentPage>

```

Binding value converters

The previous XAML example displays the individual `Red`, `Green`, and `Blue` properties of each `NamedColor`. These properties are of type `float` and range from 0 to 1. If you want to display the hexadecimal values, you can't simply use `StringFormat` with an "X2" formatting specification. That only works for integers and besides, the `float` values need to be multiplied by 255.

This issue can be solved with a *value converter*, also called a *binding converter*. This is a class that implements the `IValueConverter` interface, which means it has two methods named `Convert` and `ConvertBack`. The `Convert` method is called when a value is transferred from source to target. The `ConvertBack` method is called for transfers from target to source in `OneWayToSource` or `TwoWay` bindings:

C#

```
using System.Globalization;

namespace XamlSamples
{
    public class FloatToIntConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object
parameter, CultureInfo culture)
        {
            float multiplier;

            if (!float.TryParse(parameter as string, out multiplier))
                multiplier = 1;

            return (int)Math.Round(multiplier * (float)value);
        }

        public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
        {
            float divider;

            if (!float.TryParse(parameter as string, out divider))
                divider = 1;

            return ((float)(int)value) / divider;
        }
    }
}
```

⚠ Note

The `ConvertBack` method does not play a role in this example because the bindings are only one way from source to target.

A binding references a binding converter with the `Converter` property. A binding converter can also accept a parameter specified with the `ConverterParameter` property. For some versatility, this is how the multiplier is specified. The binding converter checks the converter parameter for a valid `float` value.

The converter is instantiated in the page's resource dictionary so it can be shared among multiple bindings:

XAML

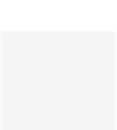
```
<local:FloatToIntConverter x:Key="intConverter" />
```

Three data bindings reference this single instance:

XAML

```
<Label Text="{Binding Red,
                      Converter={StaticResource intConverter},
                      ConverterParameter=255,
                      StringFormat='R={0:X2}'}" />
```

The item template displays the color, its friendly name, and its RGB values:

	R=00, G=80, B=80
	Thistle R=D8, G=BF, B=D8
	Tomato R=FF, G=63, B=47
	Transparent R=FF, G=FF, B=FF
	Turquoise R=40, G=E0, B=D0
	Violet R=EE, G=82, B=EE
	Wheat R=F5, G=DE, B=B3
	White R=FF, G=FF, B=FF
	White Smoke R=F5, G=F5, B=F5

The [ListView](#) can handle changes that dynamically occur in the underlying data, but only if you take certain steps. If the collection of items assigned to the `ItemsSource` property of the [ListView](#) changes during runtime, use an [ObservableCollection<T>](#) class for these items. [ObservableCollection<T>](#) implements the `INotifyCollectionChanged` interface, and [ListView](#) will install a handler for the `CollectionChanged` event.

If properties of the items themselves change during runtime, then the items in the collection should implement the `INotifyPropertyChanged` interface and signal changes to property values using the `PropertyChanged` event.

Next steps

Data bindings provide a powerful mechanism for linking properties between two objects within a page, or between visual objects and underlying data. But when the application begins working with data sources, a popular app architectural pattern begins to emerge as a useful paradigm.

Data binding and MVVM

Article • 08/30/2024

 [Browse the sample](#)

The Model-View-ViewModel (MVVM) pattern enforces a separation between three software layers — the XAML user interface, called the view, the underlying data, called the model, and an intermediary between the view and the model, called the viewmodel. The view and the viewmodel are often connected through data bindings defined in XAML. The `BindingContext` for the view is usually an instance of the viewmodel.

Important

.NET Multi-platform App UI (.NET MAUI) marshals binding updates to the UI thread. When using MVVM this enables you to update data-bound viewmodel properties from any thread, with .NET MAUI's binding engine bringing the updates to the UI thread.

There are multiple approaches to implementing the MVVM pattern, and this article focuses on a simple approach. It uses views and viewmodels, but not models, to focus on the data binding between the two layers. For a detailed explanation of using the MVVM pattern in .NET MAUI, see [Model-View-ViewModel \(MVVM\) in Enterprise Application Patterns using .NET MAUI](#). For a tutorial that helps you to implement the MVVM pattern, see [Upgrade your app with MVVM concepts](#).

Simple MVVM

In [XAML markup extensions](#) you saw how to define a new XML namespace declaration to allow a XAML file to reference classes in other assemblies. The following example uses the `x:Static` markup extension to obtain the current date and time from the static `DateTime.Now` property in the `System` namespace:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:sys="clr-namespace:System;assembly=netstandard"
             x:Class="XamlSamples.OneShotDateTimePage"
             Title="One-Shot DateTime Page">

    <VerticalStackLayout BindingContext="{x:Static sys:DateTime.Now}"
```

```

        Spacing="25" Padding="30,0"
        VerticalOptions="Center"
HorizontalOptions="Center">

    <Label Text="{Binding Year, StringFormat='The year is {0}}'" />
    <Label Text="{Binding StringFormat='The month is {0:MMM}}'" />
    <Label Text="{Binding Day, StringFormat='The day is {0}}'" />
    <Label Text="{Binding StringFormat='The time is {0:T}}'" />

</VerticalStackLayout>

</ContentPage>

```

In this example, the retrieved `DateTime` value is set as the `BindingContext` on a `StackLayout`. When you set the `BindingContext` on an element, it is inherited by all the children of that element. This means that all the children of the `StackLayout` have the same `BindingContext`, and they can contain bindings to properties of that object:

The year is 2017
The month is October
The day is 25
The time is 11:58:23 AM

However, the problem is that the date and time are set once when the page is constructed and initialized, and never change.

⚠ Warning

In a class that derives from `BindableObject`, only properties of type `BindableProperty` are bindable. For example, `VisualElement.IsLoaded` and `Element.Parent` aren't bindable.

A XAML page can display a clock that always shows the current time, but it requires additional code. The MVVM pattern is a natural choice for .NET MAUI apps when data binding from properties between visual objects and the underlying data. When thinking in terms of MVVM, the model and viewmodel are classes written entirely in code. The view is often a XAML file that references properties defined in the viewmodel through data bindings. In MVVM, a model is ignorant of the viewmodel, and a viewmodel is

ignorant of the view. However, often you tailor the types exposed by the viewmodel to the types associated with the UI.

ⓘ Note

In simple examples of MVVM, such as those shown here, often there is no model at all, and the pattern involves just a view and viewmodel linked with data bindings.

The following example shows a viewmodel for a clock, with a single property named `DateTime` that's updated every second:

C#

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace XamlSamples;

class ClockViewModel: INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    private DateTime _dateTime;
    private Timer _timer;

    public DateTime DateTime
    {
        get => _dateTime;
        set
        {
            if (_dateTime != value)
            {
                _dateTime = value;
                OnPropertyChanged(); // reports this property
            }
        }
    }

    public ClockViewModel()
    {
        this.DateTime = DateTime.Now;

        // Update the DateTime property every second.
        _timer = new Timer(new TimerCallback((s) => this.DateTime =
DateTime.Now),
                                null, TimeSpan.Zero, TimeSpan.FromSeconds(1));
    }

    ~ClockViewModel() =>
        _timer.Dispose();
}
```



```
public void OnPropertyChanged([CallerMemberName] string name = "") =>
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}
```

Viewmodels typically implement the `INotifyPropertyChanged` interface, which provides the ability for a class to raise the `PropertyChanged` event whenever one of its properties changes. The data binding mechanism in .NET MAUI attaches a handler to this `PropertyChanged` event so it can be notified when a property changes and keep the target updated with the new value. In the previous code example, the `OnPropertyChanged` method handles raising the event while automatically determining the property source name: `DateTime`.

The following example shows XAML that consumes `ClockViewModel`:

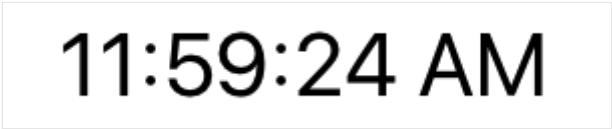
XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:XamlSamples"
             x:Class="XamlSamples.ClockPage"
             Title="Clock Page">
    <ContentPage.BindingContext>
        <local:ClockViewModel />
    </ContentPage.BindingContext>

    <Label Text="{Binding DateTime, StringFormat='{0:T}'}"
           FontSize="18"
           HorizontalOptions="Center"
           VerticalOptions="Center" />
</ContentPage>
```

In this example, `ClockViewModel` is set to the `BindingContext` of the `ContentPage` using property element tags. Alternatively, the code-behind file could instantiate the viewmodel.

The `Binding` markup extension on the `Text` property of the `Label` formats the `DateTime` property. The following screenshot shows the result:



11:59:24 AM

In addition, it's possible to access individual properties of the `DateTime` property of the viewmodel by separating the properties with periods:

XAML

```
<Label Text="{Binding DateTime.Second, StringFormat='{0}'}" ... >
```

Interactive MVVM

MVVM is often used with two-way data bindings for an interactive view based on an underlying data model.

The following example shows the `HslViewModel` that converts a `Color` value into `Hue`, `Saturation`, and `Luminosity` values, and back again:

C#

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace XamlSamples;

class HslViewModel: INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    private float _hue, _saturation, _luminosity;
    private Color _color;

    public float Hue
    {
        get => _hue;
        set
        {
            if (_hue != value)
                Color = Color.FromHsla(value, _saturation, _luminosity);
        }
    }

    public float Saturation
    {
        get => _saturation;
        set
        {
            if (_saturation != value)
                Color = Color.FromHsla(_hue, value, _luminosity);
        }
    }

    public float Luminosity
    {
        get => _luminosity;
        set
        {
            if (_luminosity != value)
```

```

        Color = Color.FromHsla(_hue, _saturation, value);
    }
}

public Color Color
{
    get => _color;
    set
    {
        if (_color != value)
        {
            _color = value;
            _hue = _color.GetHue();
            _saturation = _color.GetSaturation();
            _luminosity = _color.GetLuminosity();

            OnPropertyChanged("Hue");
            OnPropertyChanged("Saturation");
            OnPropertyChanged("Luminosity");
            OnPropertyChanged(); // reports this property
        }
    }
}

public void OnPropertyChanged([CallerMemberName] string name = "") =>
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

In this example, changes to the `Hue`, `Saturation`, and `Luminosity` properties cause the `Color` property to change, and changes to the `Color` property causes the other three properties to change. This might seem like an infinite loop, except that the viewmodel doesn't invoke the `PropertyChanged` event unless the property has changed.

The following XAML example contains a `BoxView` whose `Color` property is bound to the `Color` property of the viewmodel, and three `Slider` and three `Label` views bound to the `Hue`, `Saturation`, and `Luminosity` properties:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:XamlSamples"
    x:Class="XamlSamples.HslColorScrollPage"
    Title="HSL Color Scroll Page">
    <ContentPage.BindingContext>
        <local:HslViewModel Color="Aqua" />
    </ContentPage.BindingContext>

    <VerticalStackLayout Padding="10, 0, 10, 30">
        <BoxView Color="{Binding Color}"
            HeightRequest="100"

```

```

        WidthRequest="100"
        HorizontalOptions="Center" />
<Label Text="{Binding Hue, StringFormat='Hue = {0:F2}}'"
        HorizontalOptions="Center" />
<Slider Value="{Binding Hue}"
        Margin="20,0,20,0" />
<Label Text="{Binding Saturation, StringFormat='Saturation =
{0:F2}}'"
        HorizontalOptions="Center" />
<Slider Value="{Binding Saturation}"
        Margin="20,0,20,0" />
<Label Text="{Binding Luminosity, StringFormat='Luminosity =
{0:F2}}'"
        HorizontalOptions="Center" />
<Slider Value="{Binding Luminosity}"
        Margin="20,0,20,0" />
</VerticalStackLayout>
</ContentPage>

```

The binding on each `Label` is the default `OneWay`. It only needs to display the value. However, the default binding on each `Slider` is `TwoWay`. This allows the `Slider` to be initialized from the viewmodel. When the viewmodel is instantiated it's `Color` property is set to `Aqua`. A change in a `Slider` sets a new value for the property in the viewmodel, which then calculates a new color:



Commanding

Sometimes an app has needs that go beyond property bindings by requiring the user to initiate commands that affect something in the viewmodel. These commands are generally signaled by button clicks or finger taps, and traditionally they are processed in the code-behind file in a handler for the `Clicked` event of the `Button` or the `Tapped` event of a `TapGestureRecognizer`.

The commanding interface provides an alternative approach to implementing commands that is much better suited to the MVVM architecture. The viewmodel can contain commands, which are methods that are executed in reaction to a specific activity in the view such as a `Button` click. Data bindings are defined between these commands and the `Button`.

To allow a data binding between a `Button` and a viewmodel, the `Button` defines two properties:

- `Command` of type `System.Windows.Input.ICommand`
- `CommandParameter` of type `Object`

❗ Note

Many other controls also define `Command` and `CommandParameter` properties.

The `ICommand` interface is defined in the `System.Windows.Input` namespace, and consists of two methods and one event:

- `void Execute(object arg)`
- `bool CanExecute(object arg)`
- `event EventHandler CanExecuteChanged`

The viewmodel can define properties of type `ICommand`. You can then bind these properties to the `Command` property of each `Button` or other element, or perhaps a custom view that implements this interface. You can optionally set the `CommandParameter` property to identify individual `Button` objects (or other elements) that are bound to this viewmodel property. Internally, the `Button` calls the `Execute` method whenever the user taps the `Button`, passing to the `Execute` method its `CommandParameter`.

The `CanExecute` method and `CanExecuteChanged` event are used for cases where a `Button` tap might be currently invalid, in which case the `Button` should disable itself. The `Button` calls `CanExecute` when the `Command` property is first set and whenever the `CanExecuteChanged` event is raised. If `CanExecute` returns `false`, the `Button` disables itself and doesn't generate `Execute` calls.

You can use the `Command` or `Command<T>` class included in .NET MAUI to implement the `ICommand` interface. These two classes define several constructors plus a `ChangeCanExecute` method that the viewmodel can call to force the `Command` object to raise the `CanExecuteChanged` event.

The following example shows a viewmodel for a simple keypad that is intended for entering telephone numbers:

C#

```
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows.Input;

namespace XamlSamples;
```

```

class KeypadViewModel: INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    private string _inputString = "";
    private string _displayText = "";
    private char[] _specialChars = { '*', '#' };

    public ICommand AddCharCommand { get; private set; }
    public ICommand DeleteCharCommand { get; private set; }

    public string InputString
    {
        get => _inputString;
        private set
        {
            if (_inputString != value)
            {
                _inputString = value;
                OnPropertyChanged();
                DisplayText = FormatText(_inputString);

                // Perhaps the delete button must be enabled/disabled.
                ((Command)DeleteCharCommand).ChangeCanExecute();
            }
        }
    }

    public string DisplayText
    {
        get => _displayText;
        private set
        {
            if (_displayText != value)
            {
                _displayText = value;
                OnPropertyChanged();
            }
        }
    }

    public KeypadViewModel()
    {
        // Command to add the key to the input string
        AddCharCommand = new Command<string>((key) => InputString += key);

        // Command to delete a character from the input string when allowed
        DeleteCharCommand =
            new Command(
                // Command will strip a character from the input string
                () => InputString = InputString.Substring(0,
InputString.Length - 1),

                // CanExecute is processed here to return true when there's
something to delete
    }
}

```

```

        () => InputString.Length > 0
    );
}

string FormatText(string str)
{
    bool hasNonNumbers = str.IndexOfAny(_specialChars) != -1;
    string formatted = str;

    // Format the string based on the type of data and the length
    if (hasNonNumbers || str.Length < 4 || str.Length > 10)
    {
        // Special characters exist, or the string is too small or large
        for special formatting
        // Do nothing
    }

    else if (str.Length < 8)
        formatted = string.Format("{0}-{1}", str.Substring(0, 3),
str.Substring(3));

    else
        formatted = string.Format("({0}) {1}-{2}", str.Substring(0, 3),
str.Substring(3, 3), str.Substring(6));

    return formatted;
}

public void OnPropertyChanged([CallerMemberName] string name = "") =>
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

In this example, the `Execute` and `CanExecute` methods for the commands are defined as lambda functions in the constructor. The viewmodel assumes that the `AddCharCommand` property is bound to the `Command` property of several buttons (or anything other controls that have a command interface), each of which is identified by the `CommandParameter`. These buttons add characters to an `InputString` property, which is then formatted as a phone number for the `DisplayText` property. There's also a second property of type `ICommand` named `DeleteCharCommand`. This is bound to a back-spacing button, but the button should be disabled if there are no characters to delete.

The following example shows the XAML that consumes the `KeypadViewModel`:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:XamlSamples"
    x:Class="XamlSamples.KeypadPage"

```



```

        Title="Keypad Page">
<ContentPage.BindingContext>
    <local:KeypadViewModel />
</ContentPage.BindingContext>

<Grid HorizontalOptions="Center" VerticalOptions="Center">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="80" />
        <ColumnDefinition Width="80" />
        <ColumnDefinition Width="80" />
    </Grid.ColumnDefinitions>

    <Label Text="{Binding DisplayText}"
        Margin="0,0,10,0" FontSize="20"
LineBreakMode="HeadTruncation"
        VerticalTextAlignment="Center" HorizontalTextAlignment="End"
        Grid.ColumnSpan="2" />

    <Button Text="⌫" Command="{Binding DeleteCharCommand}"
Grid.Column="2"/>

    <Button Text="1" Command="{Binding AddCharCommand}"
CommandParameter="1" Grid.Row="1" />
    <Button Text="2" Command="{Binding AddCharCommand}"
CommandParameter="2" Grid.Row="1" Grid.Column="1" />
    <Button Text="3" Command="{Binding AddCharCommand}"
CommandParameter="3" Grid.Row="1" Grid.Column="2" />

    <Button Text="4" Command="{Binding AddCharCommand}"
CommandParameter="4" Grid.Row="2" />
    <Button Text="5" Command="{Binding AddCharCommand}"
CommandParameter="5" Grid.Row="2" Grid.Column="1" />
    <Button Text="6" Command="{Binding AddCharCommand}"
CommandParameter="6" Grid.Row="2" Grid.Column="2" />

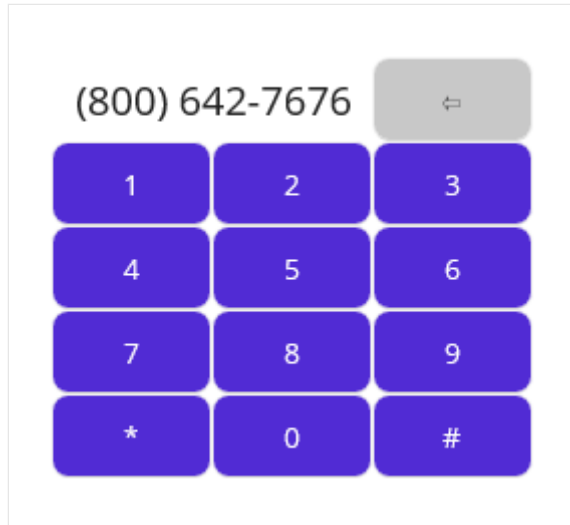
    <Button Text="7" Command="{Binding AddCharCommand}"
CommandParameter="7" Grid.Row="3" />
    <Button Text="8" Command="{Binding AddCharCommand}"
CommandParameter="8" Grid.Row="3" Grid.Column="1" />
    <Button Text="9" Command="{Binding AddCharCommand}"
CommandParameter="9" Grid.Row="3" Grid.Column="2" />

    <Button Text="*" Command="{Binding AddCharCommand}"
CommandParameter="*" Grid.Row="4" />
    <Button Text="0" Command="{Binding AddCharCommand}"
CommandParameter="0" Grid.Row="4" Grid.Column="1" />
    <Button Text="#" Command="{Binding AddCharCommand}"
CommandParameter="#" Grid.Row="4" Grid.Column="2" />

```

```
</Grid>
</ContentPage>
```

In this example, the `Command` property of the first [Button](#) that is bound to the `DeleteCharCommand`. The other buttons are bound to the `AddCharCommand` with a `CommandParameter` that's the same as the character that appears on the [Button](#):



XAML compilation

Article • 06/29/2023

.NET Multi-platform App UI (.NET MAUI) XAML is compiled directly into intermediate language (IL) with the XAML compiler (XAMLC). XAML compilation offers a number of benefits:

- It performs compile-time checking of XAML, notifying you of any errors.
- It removes some of the load and instantiation time for XAML elements.
- It helps to reduce the file size of the final assembly by no longer including .xaml files.

XAML compilation is enabled by default in .NET MAUI apps. For apps built using the debug configuration, XAML compilation provides compile-time validation of XAML, but does not convert the XAML to IL in the assembly. Instead, XAML files are included as embedded resources in the app package, and evaluated at runtime. For apps built using the release configuration, XAML compilation provides compile-time validation of XAML, and converts the XAML to IL that's written to the assembly. However, XAML compilation behavior can be overridden in both configurations with the [XamlCompilationAttribute](#) class.

Important

Compiled bindings can be enabled to improve data binding performance in .NET MAUI applications. For more information, see [Compiled Bindings](#).

Disable XAML compilation

XAML compilation can be disabled by passing `XamlCompilationOptions.Skip` to the [XamlCompilationAttribute](#):

C#

```
[assembly: XamlCompilation(XamlCompilationOptions.Skip)]
```

In this example, XAML compilation is disabled within the assembly, with XAML errors being reported at runtime rather than compile-time.

XAML compilation can also be disabled at the type level:

C#

```
[XamlCompilation (XamlCompilationOptions.Skip)]  
public partial class MyPage : ContentPage  
{  
    ...  
}
```

In this example, XAML compilation is disabled only for the `MyPage` class.

Warning

Disabling XAML compilation is not recommended because XAML is then parsed and interpreted at runtime, which will reduce app performance.

Enable XAML compilation

Because XAML compilation is enabled by default in .NET MAUI apps, removing any `XamlCompilation(XamlCompilationOptions.Skip)` statements will ensure that XAML compilation is enabled.

Alternatively, XAML compilation can be forcibly enabled by passing `XamlCompilationOptions.Compile` to the [XamlCompilationAttribute](#):

C#

```
[assembly: XamlCompilation(XamlCompilationOptions.Compile)]
```

In this example, XAML compilation is enabled for all of the XAML contained within the assembly, with XAML errors being reported at compile-time rather than runtime.

XAML compilation can also be enabled at the type level:

C#

```
[XamlCompilation (XamlCompilationOptions.Compile)]  
public partial class MyPage : ContentPage  
{  
    ...  
}
```

In this example, XAML compilation is enabled only for the `MyPage` class.

Class modifiers

Article • 11/14/2023

The .NET Multi-platform App UI (.NET MAUI) `x:ClassModifier` attribute specifies the access level for a generated class in an assembly, provided that `x:Class` is specified on the same XAML element. This enables types to be hidden in libraries, so that they aren't accessible to consumers.

Valid values of the `x:ClassModifier` attribute are:

- `Public` – specifies that the generated class has no access restrictions.
- `Internal` – specifies that the generated class is accessible only within types in the same assembly.
- `NotPublic` – identical to `Internal`.

The value of the `x:ClassModifier` attribute must align with the access level of the type in its code-behind file.

By default, if the value of the attribute isn't set, the generated class will be `public`.

ⓘ Note

The value of the attribute can use any casing, as it will be converted to lowercase by .NET MAUI.

The following XAML shows an example of setting the attribute:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MyMauiApp.MainPage"
             x:ClassModifier="NotPublic">
    ...
</ContentPage>
```

For information about specifying the access level for XAML fields, see [Field modifiers](#).

Field modifiers

Article • 05/14/2024

The .NET Multi-platform App UI (.NET MAUI) `x:FieldModifier` attribute specifies the access level for generated fields for named XAML elements.

Valid values of the `x:FieldModifier` attribute are:

- `Private` – specifies that the generated field for the XAML element is accessible only within the body of the class in which it is declared.
- `Public` – specifies that the generated field for the XAML element has no access restrictions.
- `Protected` – specifies that the generated field for the XAML element is accessible within its class and by derived class instances.
- `Internal` – specifies that the generated field for the XAML element is accessible only within types in the same assembly.
- `NotPublic` – identical to `Internal`.

By default, if the value of the attribute isn't set, the generated field for the element will be `private`.

ⓘ Note

The value of the attribute can use any casing, as it will be converted to lowercase by .NET MAUI.

The following conditions must be met for an `x:FieldModifier` attribute to be processed:

- The top-level XAML element must be a valid `x:Class`.
- The current XAML element has an `x:Name` specified.

The following XAML shows examples of setting the attribute:

XAML

```
<Label x:Name="privateLabel" />
<Label x:Name="internalLabel" x:FieldModifier="NotPublic" />
<Label x:Name="publicLabel" x:FieldModifier="Public" />
```

ⓘ Important

The `x:FieldModifier` attribute cannot be used to specify the access level of a .NET MAUI XAML class. For information about specifying the access level of a .NET MAUI XAML class, see [Class modifiers](#).

Generics

Article • 04/03/2023

.NET Multi-platform App UI (.NET MAUI) XAML provides support for consuming generic CLR types by specifying the generic constraints as type arguments. This support is provided by the `x:TypeArguments` directive, which passes the constraining type arguments of a generic to the constructor of the generic type.

Type arguments are specified as a string, and are typically prefixed, such as `sys:String` and `sys:Int32`. Prefixing is required because the typical types of CLR generic constraints come from libraries that are not mapped to the default .NET MAUI namespaces. However, the XAML 2009 built-in types such as `x:String` and `x:Int32`, can also be specified as type arguments, where `x` is the XAML language namespace for XAML 2009. For more information about the XAML 2009 built-in types, see [XAML 2009 Language Primitives](#).

❗ Important

Defining generic classes in .NET MAUI XAML, with the `x:TypeArguments` directive, is unsupported.

Multiple type arguments can be specified by using a comma delimiter. In addition, if a generic constraint uses generic types, the nested constraint type arguments should be contained in parentheses.

⚠ Note

The `x:Type` markup extension supplies a Common Language Runtime (CLR) type reference for a generic type, and has a similar function to the `typeof` operator in C#. For more information, see [x:Type markup extension](#).

Single primitive type argument

A single primitive type argument can be specified as a prefixed string argument using the `x:TypeArguments` directive:

XAML


```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:scg="clr-
namespace:System.Collections.Generic;assembly=netstandard"
             ...>
    <CollectionView>
        <CollectionView.ItemsSource>
            <scg:List x:TypeArguments="x:String">
                <x:String>Baboon</x:String>
                <x:String>Capuchin Monkey</x:String>
                <x:String>Blue Monkey</x:String>
                <x:String>Squirrel Monkey</x:String>
                <x:String>Golden Lion Tamarin</x:String>
                <x:String>Howler Monkey</x:String>
                <x:String>Japanese Macaque</x:String>
            </scg:List>
        </CollectionView.ItemsSource>
    </CollectionView>
</ContentPage>

```

In this example, `System.Collections.Generic` is defined as the `scg` XAML namespace. The `CollectionView.ItemsSource` property is set to a `List<T>` that's instantiated with a `string` type argument, using the XAML 2009 built-in `x:String` type. The `List<string>` collection is initialized with multiple `string` items.

Alternatively, but equivalently, the `List<T>` collection can be instantiated with the CLR `String` type:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:scg="clr-
namespace:System.Collections.Generic;assembly=netstandard"
             xmlns:sys="clr-namespace:System;assembly=netstandard"
             ...>
    <CollectionView>
        <CollectionView.ItemsSource>
            <scg:List x:TypeArguments="sys:String">
                <sys:String>Baboon</sys:String>
                <sys:String>Capuchin Monkey</sys:String>
                <sys:String>Blue Monkey</sys:String>
                <sys:String>Squirrel Monkey</sys:String>
                <sys:String>Golden Lion Tamarin</sys:String>
                <sys:String>Howler Monkey</sys:String>
                <sys:String>Japanese Macaque</sys:String>
            </scg:List>
        </CollectionView.ItemsSource>
    </CollectionView>
</ContentPage>

```

```
</CollectionView>
</ContentPage>
```

Single object type argument

A single object type argument can be specified as a prefixed string argument using the `x:TypeArguments` directive:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:models="clr-namespace:GenericsDemo.Models"
             xmlns:scg="clr-namespace:System.Collections.Generic;assembly=netstandard"
             ...>
    <CollectionView>
        <CollectionView.ItemsSource>
            <scg:List x:TypeArguments="models:Monkey">
                <models:Monkey Name="Baboon"
                               Location="Africa and Asia"

ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/Papio_anubis_%28Serengeti%2C_2009%29.jpg/200px-Papio_anubis_%28Serengeti%2C_2009%29.jpg" />
                <models:Monkey Name="Capuchin Monkey"
                               Location="Central and South America"

ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/4/40/Capuchin_Costa_Rica.jpg/200px-Capuchin_Costa_Rica.jpg" />
                <models:Monkey Name="Blue Monkey"
                               Location="Central and East Africa"

ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/8/83/BlueMonkey.jpg/220px-BlueMonkey.jpg" />
            </scg:List>
        </CollectionView.ItemsSource>
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Grid Padding="10">
                    <Grid.RowDefinitions>
                        <RowDefinition Height="Auto" />
                        <RowDefinition Height="Auto" />
                    </Grid.RowDefinitions>
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto" />
                        <ColumnDefinition Width="Auto" />
                    </Grid.ColumnDefinitions>
                    <Image Grid.RowSpan="2"
                           Source="{Binding ImageUrl}"
                           Aspect="AspectFill"

```

```

        HeightRequest="60"
        WidthRequest="60" />
<Label Grid.Column="1"
        Text="{Binding Name}"
        FontAttributes="Bold" />
<Label Grid.Row="1"
        Grid.Column="1"
        Text="{Binding Location}"
        FontAttributes="Italic"
        VerticalOptions="End" />
    </Grid>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</ContentPage>

```

In this example, `GenericsDemo.Models` is defined as the `models` XAML namespace, and `System.Collections.Generic` is defined as the `scg` XAML namespace. The `CollectionView.ItemsSource` property is set to a `List<T>` that's instantiated with a `Monkey` type argument. The `List<Monkey>` collection is initialized with multiple `Monkey` items, and a [DataTemplate](#) that defines the appearance of each `Monkey` object is set as the `ItemTemplate` of the [CollectionView](#).

Multiple type arguments

Multiple type arguments can be specified as prefixed string arguments, delimited by a comma, using the `x:TypeArguments` directive. When a generic constraint uses generic types, the nested constraint type arguments are contained in parentheses:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:models="clr-namespace:GenericsDemo.Models"
        xmlns:scg="clr-
namespace:System.Collections.Generic;assembly=netstandard"
        ...>
    <CollectionView>
        <CollectionView.ItemsSource>
            <scg:List
x:TypeArguments="scg:KeyValuePair(x:String,models:Monkey)">
                <scg:KeyValuePair x:TypeArguments="x:String,models:Monkey">
                    <x:Arguments>
                        <x:String>Baboon</x:String>
                        <models:Monkey Location="Africa and Asia"
ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/f/fc/Papio_an
ubis_%28Serengeti%2C_2009%29.jpg/200px-

```

```

Papio_anubis_%28Serengeti%2C_2009%29.jpg" />
    </x:Arguments>
</scg:KeyValuePair>
<scg:KeyValuePair x:TypeArguments="x:String,models:Monkey">
    <x:Arguments>
        <x:String>Capuchin Monkey</x:String>
        <models:Monkey Location="Central and South America"

ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/4/40/Capuchin
_Costa_Rica.jpg/200px-Capuchin_Costa_Rica.jpg" />
    </x:Arguments>
</scg:KeyValuePair>
<scg:KeyValuePair x:TypeArguments="x:String,models:Monkey">
    <x:Arguments>
        <x:String>Blue Monkey</x:String>
        <models:Monkey Location="Central and East Africa"

ImageUrl="https://upload.wikimedia.org/wikipedia/commons/thumb/8/83/BlueMonk
ey.jpg/220px-BlueMonkey.jpg" />
    </x:Arguments>
</scg:KeyValuePair>
</scg:List>
</CollectionView.ItemsSource>
<CollectionView.ItemTemplate>
    <DataTemplate>
        <Grid Padding="10">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
            </Grid.RowDefinitions>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="Auto" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>
            <Image Grid.RowSpan="2"
                Source="{Binding Value.ImageUrl}"
                Aspect="AspectFill"
                HeightRequest="60"
                WidthRequest="60" />
            <Label Grid.Column="1"
                Text="{Binding Key}"
                FontAttributes="Bold" />
            <Label Grid.Row="1"
                Grid.Column="1"
                Text="{Binding Value.Location}"
                FontAttributes="Italic"
                VerticalOptions="End" />
        </Grid>
    </DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</ContentPage>

```

In this example, `GenericsDemo.Models` is defined as the `models` XAML namespace, and `System.Collections.Generic` is defined as the `scg` XAML namespace. The `CollectionView.ItemsSource` property is set to a `List<T>` that's instantiated with a `KeyValuePair<TKey, TValue>` constraint, with the inner constraint type arguments `string` and `Monkey`. The `List<KeyValuePair<string, Monkey>>` collection is initialized with multiple `KeyValuePair` items, using the non-default `KeyValuePair` constructor, and a [DataTemplate](#) that defines the appearance of each `Monkey` object is set as the `ItemTemplate` of the [CollectionView](#). For information on passing arguments to a non-default constructor, see [Pass constructor arguments](#).

Consume XAML markup extensions

Article • 11/14/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) XAML markup extensions help enhance the power and flexibility of XAML by allowing element attributes to be set from a variety of sources.

For example, you typically set the `Color` property of `BoxView` like this:

XAML

```
<BoxView Color="Blue" />
```

However, you might prefer instead to set the `Color` attribute from a value stored in a resource dictionary, or from the value of a static property of a class that you've created, or from a property of type `Color` of another element on the page, or constructed from separate hue, saturation, and luminosity values. All these options are possible using XAML markup extensions.

A markup extension is a different way to express an attribute of an element. .NET MAUI XAML markup extensions are usually identifiable by an attribute value that is enclosed in curly braces:

XAML

```
<BoxView Color="{StaticResource themeColor}" />
```

Any attribute value in curly braces is *always* a XAML markup extension. However, XAML markup extensions can also be referenced without the use of curly braces.

Note

Several XAML markup extensions are part of the XAML 2009 specification. These appear in XAML files with the customary `x` namespace prefix, and are commonly referred to with this prefix.

In addition to the markup extensions discussed in this article, the following markup extensions are included in .NET MAUI and discussed in other articles:

- [AppThemeBinding](#) - specifies a resource to be consumed based on the current system theme. For more information, see [AppThemeBinding markup extension](#).
- [Binding](#) - establishes a link between properties of two objects. For more information, see [Data binding](#).
- [DynamicResource](#) - responds to changes in objects in a resource dictionary. For more information, see [Dynamic styles](#).
- [FontImage](#) - displays a font icon in any view that can display an [ImageSource](#). For more information, see [Load a font icon](#).
- [OnIdiom](#) - customizes UI appearance based on the idiom of the device the application is running on. For more information, see [Customize UI appearance based on the device idiom](#).
- [OnPlatform](#) - customizes UI appearance on a per-platform basis. For more information, see [Customize UI appearance based on the platform](#).
- [RelativeSource](#) - sets the binding source relative to the position of the binding target. For more information, see [Relative bindings](#).
- [StaticResource](#) - references objects from a resource dictionary. For more information, see [Resource dictionaries](#).
- [TemplateBinding](#) - performs data binding from a control template. For more information, see [Control templates](#).

x:Static markup extension

The `x:Static` markup extension is supported by the [StaticExtension](#) class. The class has a single property named `Member` of type `string` that you set to the name of a public constant, static property, static field, or enumeration member.

One way to use `x:Static` is to first define a class with some constants or static variables, such as this `AppConstants` class:

C#

```
static class AppConstants
{
    public static double NormalFontSize = 18;
}
```

The following XAML demonstrates the most verbose approach to instantiating the [StaticExtension](#) class between `Label.FontSize` property-element tags:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:sys="clr-namespace:System;assembly=netstandard"
             xmlns:local="clr-namespace:MarkupExtensions"
             x:Class="MarkupExtensions.StaticDemoPage"
             Title="x:Static Demo">
    <StackLayout Margin="10, 0">
        <Label Text="Label No. 1">
            <Label.FontSize>
                <x:StaticExtension
Member="local:AppConstants.NormalFontSize" />
            </Label.FontSize>
        </Label>
        ...
    </StackLayout>
</ContentPage>

```

The XAML parser also allows the `StaticExtension` class to be abbreviated as `x:Static`:

XAML

```

<Label Text="Label No. 2">
    <Label.FontSize>
        <x:Static Member="local:AppConstants.NormalFontSize" />
    </Label.FontSize>
</Label>

```

This syntax can be simplified even further by putting the `StaticExtension` class and the member setting in curly braces. The resulting expression is set directly to the `FontSize` attribute:

XAML

```

<Label Text="Label No. 3"
      FontSize="{x:StaticExtension
Member=local:AppConstants.NormalFontSize}" />

```

In this example, there are *no* quotation marks within the curly braces. The `Member` property of `StaticExtension` is no longer an XML attribute. It is instead part of the expression for the markup extension.

Just as you can abbreviate `x:StaticExtension` to `x:Static` when you use it as an object element, you can also abbreviate it in the expression within curly braces:

XAML


```
<Label Text="Label No. 4"
        FontSize="{x:Static Member=local:AppConstants.NormalFontSize}" />
```

The `StaticExtension` class has a `ContentProperty` attribute referencing the property `Member`, which marks this property as the class's default content property. For XAML markup extensions expressed with curly braces, you can eliminate the `Member=` part of the expression:

XAML

```
<Label Text="Label No. 5"
        FontSize="{x:Static local:AppConstants.NormalFontSize}" />
```

This is the most common form of the `x:Static` markup extension.

The root tag of the XAML example also contains an XML namespace declaration for the .NET `System` namespace. This allows the `Label` font size to be set to the static field `Math.PI`. That results in rather small text, so the `Scale` property is set to `Math.E`:

XAML

```
<Label Text="&#x03C0; &#x00D7; E sized text"
        FontSize="{x:Static sys:Math.PI}"
        Scale="{x:Static sys:Math.E}"
        HorizontalOptions="Center" />
```

The following screenshot shows the XAML output:



Label No. 1
Label No. 2
Label No. 3
Label No. 4
Label No. 5

π × E sized text

x:Reference markup extension

The `x:Reference` markup extension is supported by the `ReferenceExtension` class. The class has a single property named `Name` of type `string` that you set to the name of an element on the page that has been given a name with `x:Name`. This `Name` property is the content property of `ReferenceExtension`, so `Name=` is not required when `x:Reference`

appears in curly braces. The `x:Reference` markup extension is used exclusively with data bindings. For more information about data bindings, see [Data binding](#).

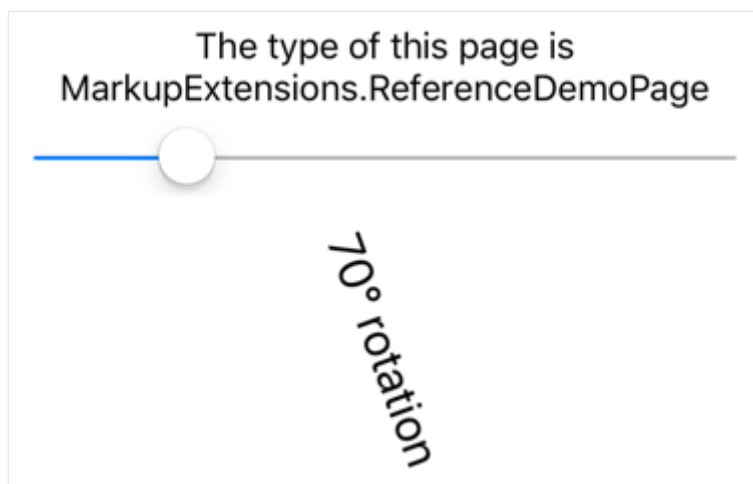
The following XAML example shows two uses of `x:Reference` with data bindings, the first where it's used to set the `Source` property of the `Binding` object, and the second where it's used to set the `BindingContext` property for two data bindings:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MarkupExtensions.ReferenceDemoPage"
             x:Name="page"
             Title="x:Reference Demo">
    <StackLayout Margin="10, 0">
        <Label Text="{Binding Source={x:Reference page},
                           StringFormat='The type of this page is {0}'}"
               FontSize="18"
               VerticalOptions="Center"
               HorizontalTextAlignment="Center" />
        <Slider x:Name="slider"
                Maximum="360"
                VerticalOptions="Center" />
        <Label BindingContext="{x:Reference slider}"
               Text="{Binding Value, StringFormat='{0:F0}&#x00B0;
rotation'}"
               Rotation="{Binding Value}"
               FontSize="24"
               HorizontalOptions="Center"
               VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

In this example, both `x:Reference` expressions use the abbreviated version of the [ReferenceExtension](#) class name and eliminate the `Name=` part of the expression. In the first example, the `x:Reference` markup extension is embedded in the `Binding` markup extension and the `Source` and `StringFormat` properties are separated by commas.

The following screenshot shows the XAML output:



x:Type markup extension

The `x:Type` markup extension is the XAML equivalent of the C# `typeof` keyword. It's supported by the `TypeExtension` class, which defines a property named `TypeName` of type `string` that should be set to a class or structure name. The `x:Type` markup extension returns the `Type` object of that class or structure. `TypeName` is the content property of `TypeExtension`, so `TypeName=` is not required when `x:Type` appears with curly braces.

The `x:Type` markup extension is commonly used with the `x:Array` markup extension. For more information, see [x:Array markup extension](#).

The following XAML example demonstrates using the `x:Type` markup extension to instantiate .NET MAUI objects and add them to a `StackLayout`. The XAML consists of three `Button` elements with their `Command` properties set to a `Binding` and the `CommandParameter` properties set to types of three .NET MAUI views:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MarkupExtensions.TypeDemoPage"
              Title="x:Type Demo">
    <StackLayout x:Name="stackLayout"
                Padding="10, 0">
        <Button Text="Create a Slider"
                HorizontalOptions="Center"
                VerticalOptions="Center"
                Command="{Binding CreateCommand}"
                CommandParameter="{x:Type Slider}" />
        <Button Text="Create a Stepper"
                HorizontalOptions="Center"
                VerticalOptions="Center"
                Command="{Binding CreateCommand}"
                CommandParameter="{x:Type Stepper}" />
    </StackLayout>
</ContentPage>
```

```

        <Button Text="Create a Switch"
              HorizontalOptions="Center"
              VerticalOptions="Center"
              Command="{Binding CreateCommand}"
              CommandParameter="{x:Type Switch}" />
    </StackLayout>
</ContentPage>

```

The code-behind file defines and initializes the `CreateCommand` property:

```

C#

public partial class TypeDemoPage : ContentPage
{
    public ICommand CreateCommand { get; private set; }

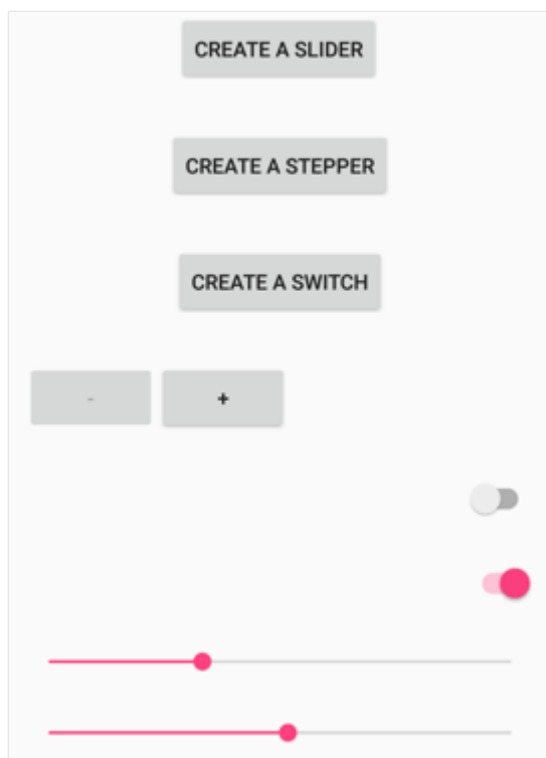
    public TypeDemoPage()
    {
        InitializeComponent();

        CreateCommand = new Command<Type>((Type viewType) =>
        {
            View view = (View)Activator.CreateInstance(viewType);
            view.VerticalOptions = LayoutOptions.Center;
            stackLayout.Add(view);
        });

        BindingContext = this;
    }
}

```

When a `Button` is pressed a new instance of the `CommandParameter` argument is created and added to the `StackLayout`. The three `Button` objects then share the page with dynamically created views:



x:Array markup extension

The `x:Array` markup extension enables you to define an array in markup. It is supported by the [ArrayExtension](#) class, which defines two properties:

- `Type` of type `Type`, which indicates the type of the elements in the array. This property should be set to an `x:Type` markup extension.
- `Items` of type `IList`, which is a collection of the items themselves. This is the content property of [ArrayExtension](#).

The `x:Array` markup extension itself never appears in curly braces. Instead, `x:Array` start and end tags delimit the list of items.

The following XAML example shows how to use `x:Array` to add items to a [ListView](#) by setting the `ItemsSource` property to an array:

XAML

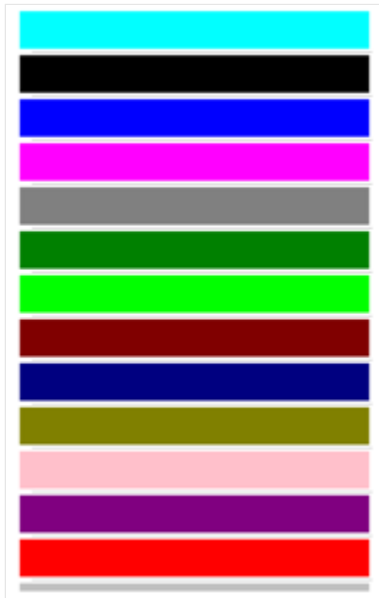
```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MarkupExtensions.ArrayDemoPage"
              Title="x:Array Demo Page">
  <ListView Margin="10">
    <ListView.ItemsSource>
      <x:Array Type="{x:Type Color}">
        <Color>Aqua</Color>
        <Color>Black</Color>
        <Color>Blue</Color>
      </x:Array>
    </ListView.ItemsSource>
  </ListView>
</ContentPage>
```

```

        <Color>Fuchsia</Color>
        <Color>Gray</Color>
        <Color>Green</Color>
        <Color>Lime</Color>
        <Color>Maroon</Color>
        <Color>Navy</Color>
        <Color>Olive</Color>
        <Color>Pink</Color>
        <Color>Purple</Color>
        <Color>Red</Color>
        <Color>Silver</Color>
        <Color>Teal</Color>
        <Color>White</Color>
        <Color>Yellow</Color>
    </x:Array>
</ListView.ItemsSource>
<ListView.ItemTemplate>
    <DataTemplate>
        <ViewCell>
            <BoxView Color="{Binding}"
                Margin="3" />
        </ViewCell>
    </DataTemplate>
</ListView.ItemTemplate>
</ListView>
</ContentPage>

```

In this example, the [ViewCell](#) creates a simple [BoxView](#) for each color entry:



ⓘ Note

When defining arrays of common types like strings or numbers, use the XAML language primitives tags listed in [Pass arguments](#).

x:Null markup extension

The `x:Null` markup extension is supported by the `NullExtension` class. It has no properties and is simply the XAML equivalent of the C# `null` keyword.

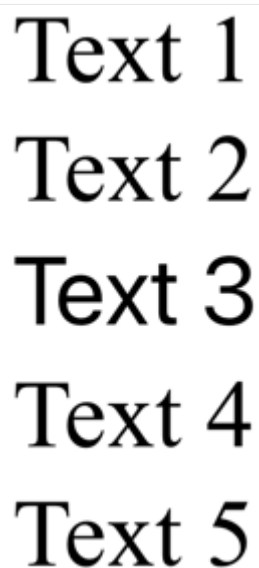
The following XAML example shows how to use the `x:Null` markup extension:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MarkupExtensions.NullDemoPage"
             Title="x:Null Demo">
    <ContentPage.Resources>
        <Style TargetType="Label">
            <Setter Property="FontSize" Value="48" />
            <Setter Property="FontFamily" Value="OpenSansRegular" />
        </Style>
    </ContentPage.Resources>

    <StackLayout Padding="10, 0">
        <Label Text="Text 1" />
        <Label Text="Text 2" />
        <Label Text="Text 3"
              FontFamily="{x:Null}" />
        <Label Text="Text 4" />
        <Label Text="Text 5" />
    </StackLayout>
</ContentPage>
```

In this example, an implicit `Style` is defined for `Label` that includes a `Setter` that sets the `FontFamily` property to a specific font. However, the third `Label` avoids using the font defined in the implicit style by setting its `FontFamily` to `x:Null`:



Text 1
Text 2
Text 3
Text 4
Text 5

DataTemplate markup extension

The [DataTemplate](#) markup extension enables you to convert a type into a [DataTemplate](#). It's supported by the [DataTemplateExtension](#) class, which defines a `TypeName` property, of type `string`, that is set to the name of the type to be converted into a [DataTemplate](#). The `TypeName` property is the content property of [DataTemplateExtension](#). Therefore, for XAML markup expressions expressed with curly braces, you can eliminate the `TypeName=` part of the expression.

❗ Note

The XAML parser allows the [DataTemplateExtension](#) class to be abbreviated as [DataTemplate](#).

A typical usage of this markup extension is in a Shell application, as shown in the following example:

XAML

```
<ShellContent Title="Monkeys"
              Icon="monkey.png"
              ContentTemplate="{DataTemplate views:MonkeysPage}" />
```

In this example, `MonkeysPage` is converted from a [ContentPage](#) to a [DataTemplate](#), which is set as the value of the `ShellContent.ContentTemplate` property. This ensures that `MonkeysPage` is only created when navigation to the page occurs, rather than at application startup.

For more information about Shell apps, see [Shell](#).

Create XAML markup extensions

Article • 09/25/2024

 [Browse the sample](#)

At the developer level, a .NET Multi-platform App UI (.NET MAUI) XAML markup extension is a class that implements the [IMarkupExtension](#) or [IMarkupExtension<T>](#) interface. It's also possible to define your own custom XAML markup extensions by deriving from [IMarkupExtension](#) or [IMarkupExtension<T>](#). Use the generic form if the markup extension obtains a value of a particular type. This is the case with several of the .NET MAUI markup extensions:

- `TypeExtension` derives from `IMarkupExtension<Type>`
- `ArrayExtension` derives from `IMarkupExtension<Array>`
- `DynamicResourceExtension` derives from `IMarkupExtension<DynamicResource>`
- `BindingExtension` derives from `IMarkupExtension<BindingBase>`

All classes that implement [IMarkupExtension](#) or [IMarkupExtension<T>](#) need to be annotated with either the [RequireServiceAttribute](#) or [AcceptEmptyServiceProviderAttribute](#). For more information, see [Service providers](#).

The two [IMarkupExtension](#) interfaces define only one method each, named `ProvideValue`:

C#

```
public interface IMarkupExtension
{
    object ProvideValue(IServiceProvider serviceProvider);
}

public interface IMarkupExtension<out T> : IMarkupExtension
{
    new T ProvideValue(IServiceProvider serviceProvider);
}
```

Since [IMarkupExtension<T>](#) derives from [IMarkupExtension](#) and includes the `new` keyword on `ProvideValue`, it contains both `ProvideValue` methods.

Often, XAML markup extensions define properties that contribute to the return value, and the `ProvideValue` method has a single argument of type `IServiceProvider`. For more information about service providers, see [Service providers](#).

Create a markup extension

The following XAML markup extension demonstrates how to create your own markup extension. It allows you to construct a [Color](#) value using hue, saturation, and luminosity components. It defines four properties for the four components of the color, including an alpha component that is initialized to 1. The class derives from

`IMarkupExtension<Color>` to indicate a [Color](#) return value:

C#

```
[AcceptEmptyServiceProvider]
public class HslColorExtension : IMarkupExtension<Color>
{
    public float H { get; set; }
    public float S { get; set; }
    public float L { get; set; }
    public float A { get; set; } = 1.0f;

    public Color ProvideValue(IServiceProvider serviceProvider)
    {
        return Color.FromHsla(H, S, L, A);
    }

    object IMarkupExtension.ProvideValue(IServiceProvider serviceProvider)
    {
        return (this as
IMarkupExtension<Color>).ProvideValue(serviceProvider);
    }
}
```

This markup extension is annotated with [AcceptEmptyServiceProviderAttribute](#) because it doesn't use a service from the service provider. For more information, see [Service providers](#).

Because `IMarkupExtension<T>` derives from `IMarkupExtension`, the class must contain two `ProvideValue` methods, one that returns a [Color](#) and another that returns an `object`, but the second method can call the first method.

Consume a markup extension

The following XAML demonstrates a variety of approaches that can be used to invoke the `HslColorExtension` to specify the color for a [BoxView](#):

XAML

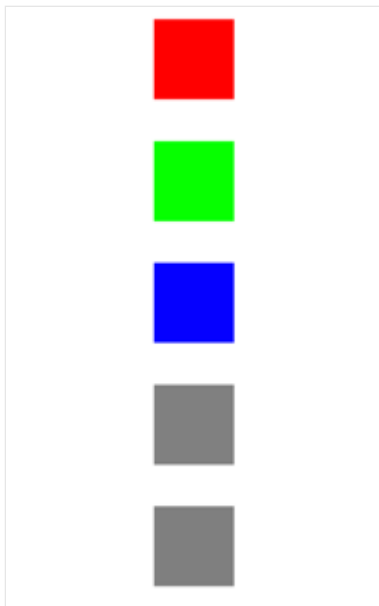
```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:MarkupExtensions"
  x:Class="MarkupExtensions.HslColorDemoPage"
  Title="HSL Color Demo">
  <ContentPage.Resources>
    <Style TargetType="BoxView">
      <Setter Property="WidthRequest" Value="80" />
      <Setter Property="HeightRequest" Value="80" />
      <Setter Property="HorizontalOptions" Value="Center" />
      <Setter Property="VerticalOptions" Value="Center" />
    </Style>
  </ContentPage.Resources>

  <StackLayout>
    <BoxView>
      <BoxView.Color>
        <local:HslColorExtension H="0" S="1" L="0.5" A="1" />
      </BoxView.Color>
    </BoxView>
    <BoxView>
      <BoxView.Color>
        <local:HslColor H="0.33" S="1" L="0.5" />
      </BoxView.Color>
    </BoxView>
    <BoxView Color="{local:HslColorExtension H=0.67, S=1, L=0.5}" />
    <BoxView Color="{local:HslColor H=0, S=0, L=0.5}" />
    <BoxView Color="{local:HslColor A=0.5}" />
  </StackLayout>
</ContentPage>

```

In this example, when `HslColorExtension` is an XML tag the four properties are set as attributes, but when it appears between curly braces, the four properties are separated by commas without quotation marks. The default values for `H`, `S`, and `L` are 0, and the default value of `A` is 1, so those properties can be omitted if you want them set to default values. The last example shows an example where the luminosity is 0, which normally results in black, but the alpha channel is 0.5, so it is half transparent and appears gray against the white background of the page:



Service providers

By using the `IServiceProvider` argument to `ProvideValue`, XAML markup extensions can get access to data about the XAML file in which they're being used. For example, the `IProvideValueTarget` service enables you to retrieve data about the object the markup extension is applied to:

C#

```
IProvideValueTarget provideValueTarget =  
serviceProvider.GetService(typeof(IProvideValueTarget)) as  
IProvideValueTarget;
```

The `IProvideValueTarget` interface defines two properties, `TargetObject` and `TargetProperty`. When this information is obtained in the `HslColorExtension` class, `TargetObject` is the `BoxView` and `TargetProperty` is the `Color` property of `BoxView`. This is the property on which the XAML markup extension has been set.

All classes that implement `IMarkupExtension` or `IMarkupExtension<T>` need to be annotated with either the `RequireServiceAttribute` or `AcceptEmptyServiceProviderAttribute`:

- For each use of `serviceProvider.GetService(typeof(T))` in the `ProvideValue` method, the class should be annotated with `[RequireService(typeof(T))]`:

C#

```
[RequireService([typeof(IReferenceProvider),  
typeof(IProvideValueTarget))]  
public class MyMarkupExtension : IMarkupExtension
```

```

{
    public object ProvideValue(IServiceProvider serviceProvider)
    {
        ...
        var referenceProvider =
            serviceProvider.GetService<IReferenceProvider>();
        var valueProvider =
            serviceProvider.GetService<IProvideValueTarget>() as
            IProvideParentValues
            ?? throw new
            ArgumentException("serviceProvider does not provide an
            IProvideValueTarget");
        ...
    }
}

```

- If the markup extension doesn't use any service from the service provider, the class should be annotated with `[AcceptEmptyServiceProvider]`.

These annotations are required due to a XAML compiler optimization that enables the generation of more efficient code, which helps reduce the app size and improve runtime performance.

XAML namespaces

Article • 05/14/2024

XAML uses the `xmlns` XML attribute for namespace declarations. There are two XAML namespace declarations that are always within the root element of a XAML file. The first defines the default namespace:

XAML

```
xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
```

The default namespace specifies that elements defined within the XAML file with no prefix refer to .NET Multi-platform App UI (.NET MAUI) classes, such as [ContentPage](#), [Label](#), and [Button](#).

The second namespace declaration uses the `x` prefix:

XAML

```
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
```

XAML uses prefixes to declare non-default namespaces, with the prefix being used when referencing types within the namespace. The `x` namespace declaration specifies that elements defined within XAML with a prefix of `x` are used for elements and attributes that are intrinsic to XAML (specifically the 2009 XAML specification).

The following table outlines the `x` constructs supported by .NET MAUI:

 Expand table

Construct	Description
<code>x:Arguments</code>	Specifies constructor arguments for a non-default constructor, or for a factory method object declaration.
<code>x:Class</code>	Specifies the namespace and class name for a class defined in XAML. The class name must match the class name of the code-behind file. Note that this construct can only appear in the root element of a XAML file.
<code>x:ClassModifier</code>	Specifies the access level for the generated class in the assembly.
<code>x:DataType</code>	Specifies the type of the object that the XAML element, and it's children, will bind to.

Construct	Description
<code>x:FactoryMethod</code>	Specifies a factory method that can be used to initialize an object.
<code>x:FieldModifier</code>	Specifies the access level for generated fields for named XAML elements.
<code>x:Key</code>	Specifies a unique user-defined key for each resource in a ResourceDictionary . The key's value is used to retrieve the XAML resource, and is typically used as the argument for the StaticResource markup extension.
<code>x:Name</code>	Specifies a runtime object name for the XAML element. Setting <code>x:Name</code> is similar to declaring a variable in code.
<code>x:TypeArguments</code>	Specifies the generic type arguments to the constructor of a generic type.

For more information about the `x:ClassModifier` attribute, see [Class modifiers](#). For more information about the `x:DataType` attribute, see [Compiled bindings](#). For more information about the `x:FieldModifier` attribute, see [Field modifiers](#). For more information about the `x:Arguments` and `x:FactoryMethod` attributes, see [Pass arguments](#). For more information about the `x:TypeArguments` attribute, see [Generics](#).

ⓘ Note

In addition to the constructs listed above, .NET MAUI also includes markup extensions that can be consumed through the `x` namespace prefix. For more information, see [Consume XAML Markup Extensions](#).

In XAML, namespace declarations inherit from parent element to child element. Therefore, when defining a namespace in the root element of a XAML file, all elements within that file inherit the namespace declaration.

Declare namespaces for types

Types can be referenced in XAML by declaring a XAML namespace with a prefix, with the namespace declaration specifying the Common Language Runtime (CLR) namespace name, and optionally an assembly name. This is achieved by defining values for the following keywords within the namespace declaration:

- `clr-namespace:` or `using:` – the CLR namespace declared within the assembly that contains the types to expose as XAML elements. This keyword is required.
- `assembly=` – the assembly that contains the referenced CLR namespace. This value is the name of the assembly, without the file extension. The path to the assembly should be established as a reference in the project that contains the XAML file that

will reference the assembly. This keyword can be omitted if the **clr-namespace** value is within the same assembly as the app code that's referencing the types.

❗ Note

The character separating the `clr-namespace` or `using` token from its value is a colon, whereas the character separating the `assembly` token from its value is an equal sign. The character to use between the two tokens is a semicolon.

The following code example shows a XAML namespace declaration:

XAML

```
<ContentPage ... xmlns:local="clr-namespace:MyMauiApp">
    ...
</ContentPage>
```

Alternatively, this can be written as:

XAML

```
<ContentPage ... xmlns:local="using:MyMauiApp">
    ...
</ContentPage>
```

The `local` prefix is a convention used to indicate that the types within the namespace are local to the app. Alternatively, if the types are in a different assembly, the assembly name should also be defined in the namespace declaration:

XAML

```
<ContentPage ... xmlns:controls="clr-
namespace:Controls;assembly=MyControlLibrary" ...>
    ...
</ContentPage>
```

The namespace prefix is then specified when declaring an instance of a type from an imported namespace:

XAML

```
<controls:Expander IsExpanded="True">
    ...
</controls:Expander>
```


For information about defining a custom namespace schema, see [Custom namespace schemas](#).

Custom namespace schemas

Article • 12/20/2023

Types in a .NET Multi-platform App UI (.NET MAUI) library can be referenced in XAML by declaring an XML namespace for the library, with the namespace declaration specifying the Common Language Runtime (CLR) namespace name and an assembly name:

XAML

```
<ContentPage ...  
    xmlns:controls="clr-  
namespace:MyCompany.Controls;assembly=MyCompany.Controls">  
    ...  
</ContentPage>
```

However, specifying a CLR namespace and assembly name in a `xmlns` definition can be awkward and error prone. In addition, multiple XML namespace declarations may be required if the library contains types in multiple namespaces.

An alternative approach is to define a custom namespace schema, such as `http://mycompany.com/schemas/controls`, that maps to one or more CLR namespaces. This enables a single XML namespace declaration to reference all the types in an assembly, even if they are in different namespaces. It also enables a single XML namespace declaration to reference types in multiple assemblies.

For more information about XAML namespaces, see [XAML namespaces](#).

Define a custom namespace schema

Imagine a library that exposes some simple controls, such as `CircleButton`:

C#

```
namespace MyCompany.Controls  
{  
    public class CircleButton : Button  
    {  
        ...  
    }  
}
```

All the controls in the library reside in the `MyCompany.Controls` namespace. These controls can be exposed to a calling assembly through a custom namespace schema.

A custom namespace schema is defined with the `XmlnsDefinitionAttribute` class, which specifies the mapping between a XAML namespace and one or more CLR namespaces. The `XmlnsDefinitionAttribute` takes two arguments: the XAML namespace name, and the CLR namespace name. The XAML namespace name is stored in the `XmlnsDefinitionAttribute.XmlNamespace` property, and the CLR namespace name is stored in the `XmlnsDefinitionAttribute.ClrNamespace` property.

ⓘ Note

The `XmlnsDefinitionAttribute` class also has a property named `AssemblyName`, which can be optionally set to the name of the assembly. This is only required when a CLR namespace referenced from a `XmlnsDefinitionAttribute` is in an external assembly.

The `XmlnsDefinitionAttribute` should be defined at the assembly level in the project that contains the CLR namespaces that will be mapped in the custom namespace schema:

C#

```
using MyCompany.Controls;

[assembly: XmlnsDefinition("http://mycompany.com/schemas/controls",
    "MyCompany.Controls")]
```

This code creates a custom namespace schema that maps the `http://mycompany.com/schemas/controls` URL to the `MyCompany.Controls` CLR namespace.

ⓘ Important

You should instruct the linker to preserve types in the assembly that are mapped through the custom namespace schema, or preserve the entire assembly. For more information, see [Preserve code](#).

The custom namespace schema can then be used for type resolution in XAML files.

Consume a custom namespace schema

To consume types from the custom namespace schema, the XAML compiler requires that there's a code reference from the assembly that consumes the types, to the assembly that defines the types. This can be accomplished by adding a class containing an `Init` method to the assembly that defines the types that will be consumed through XAML:

C#

```
namespace MyCompany.Controls
{
    public static class Controls
    {
        public static void Init()
        {
        }
    }
}
```

The `Init` method can then be called from the assembly that consumes types from the custom namespace schema:

C#

```
using MyCompany.Controls;

namespace CustomNamespaceSchemaDemo
{
    public partial class MainPage : ContentPage
    {
        public MainPage()
        {
            Controls.Init();
            InitializeComponent();
        }
    }
}
```

Warning

Failure to include such a code reference will result in the XAML compiler being unable to locate the assembly containing the custom namespace schema types.

To consume the `CircleButton` control, a XAML namespace is declared, with the namespace declaration specifying the custom namespace schema URL:

XAML

```

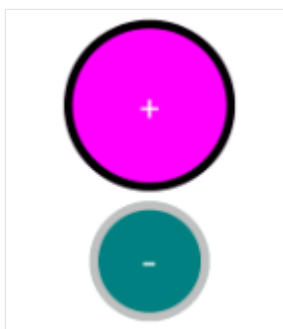
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="http://mycompany.com/schemas/controls"
             x:Class="CustomNamespaceSchemaDemo.MainPage">
    <StackLayout>
        ...
        <controls:CircleButton Text="+"
                               BackgroundColor="Fuchsia"
                               BorderColor="Black"
                               CircleDiameter="100" />
        <controls:CircleButton Text="-"
                               BackgroundColor="Teal"
                               BorderColor="Silver"
                               CircleDiameter="70" />
        ...
    </StackLayout>
</ContentPage>

```

`CircleButton` instances can then be added to the `ContentPage` by declaring them with the `controls` namespace prefix.

To find the custom namespace schema types, .NET MAUI will search referenced assemblies for `XmlnsDefinitionAttribute` instances. If the `xmlns` attribute for an element in a XAML file matches the `XmlNamespace` property value in a `XmlnsDefinitionAttribute`, .NET MAUI will attempt to use the `XmlnsDefinitionAttribute.ClrNamespace` property value for resolution of the type. If type resolution fails, .NET MAUI will continue to attempt type resolution based on any additional matching `XmlnsDefinitionAttribute` instances.

The result is that two `CircleButton` instances are displayed:



Custom namespace prefixes

Article • 08/22/2022

The NET Multi-platform App UI (.NET MAUI) `XmlnsPrefixAttribute` class can be used by control authors to specify a recommended prefix to associate with a XAML namespace, for XAML usage. The prefix is useful when supporting object tree serialization to XAML, or when interacting with a design environment that has XAML editing features. For example:

- XAML text editors could use the `XmlnsPrefixAttribute` as a hint for an initial XAML namespace `xmlns` mapping.
- XAML design environments could use the `XmlnsPrefixAttribute` to add mappings to the XAML when dragging objects out of a toolbox and onto a visual design surface.

Recommended namespace prefixes should be defined at the assembly level with the `XmlnsPrefixAttribute` constructor, which takes two arguments: a string that specifies the identifier of a XAML namespace, and a string that specifies a recommended prefix:

C#

```
[assembly: XmlnsPrefix("http://schemas.microsoft.com/dotnet/2021/maui",  
"maui")]
```

Prefixes should use short strings, because the prefix is typically applied to all serialized elements that come from the XAML namespace. Therefore, the prefix string length can have a noticeable effect on the size of the serialized XAML output.

ⓘ Note

More than one `XmlnsPrefixAttribute` can be applied to an assembly. For example, if you have an assembly that defines types for more than one XAML namespace, you could define different prefix values for each XAML namespace.

Pass arguments

Article • 04/03/2023

It's often necessary to instantiate objects with constructors that require arguments, or by calling a static creation method. This can be achieved in .NET Multi-platform App UI (.NET MAUI) XAML by using the `x:Arguments` and `x:FactoryMethod` attributes:

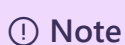
- The `x:Arguments` attribute is used to specify constructor arguments for a non-default constructor, or for a factory method object declaration. For more information, see [Pass constructor arguments](#).
- The `x:FactoryMethod` attribute is used to specify a factory method that can be used to initialize an object. For more information, see [Call factory methods](#).

In addition, the `x:TypeArguments` attribute can be used to specify the generic type arguments to the constructor of a generic type. For more information, see [Specify a generic type argument](#).

Arguments can be passed to constructors and factory methods using the following .NET MAUI XAML language primitives:

- `x:Array`, which corresponds to [Array](#).
- `x:Boolean`, which corresponds to [Boolean](#).
- `x:Byte`, which corresponds to [Byte](#).
- `x:Char`, which corresponds to [Char](#).
- `x:DateTime`, which corresponds to [DateTime](#).
- `x:Decimal`, which corresponds to [Decimal](#).
- `x:Double`, which corresponds to [Double](#).
- `x:Int16`, which corresponds to [Int16](#).
- `x:Int32`, which corresponds to [Int32](#).
- `x:Int64`, which corresponds to [Int64](#).
- `x:Object`, which corresponds to the [Object](#).
- `x:Single`, which corresponds to [Single](#).
- `x:String`, which corresponds to [String](#).
- `x:TimeSpan`, which corresponds to [TimeSpan](#).

With the exception of `x:DateTime`, the other language primitives are in the XAML 2009 specification.



The `x:Single` language primitive can be used to pass `float` arguments.

Pass constructor arguments

Arguments can be passed to a non-default constructor using the `x:Arguments` attribute. Each constructor argument must be delimited within an XML element that represents the type of the argument.

The following example demonstrates using the `x:Arguments` attribute with three different `Color` constructors:

XAML

```
<BoxView HeightRequest="150"
          WidthRequest="150"
          HorizontalOptions="Center">
  <BoxView.Color>
    <Color>
      <x:Arguments>
        <x:Single>0.9</x:Single>
      </x:Arguments>
    </Color>
  </BoxView.Color>
</BoxView>
<BoxView HeightRequest="150"
          WidthRequest="150"
          HorizontalOptions="Center">
  <BoxView.Color>
    <Color>
      <x:Arguments>
        <x:Single>0.25</x:Single>
        <x:Single>0.5</x:Single>
        <x:Single>0.75</x:Single>
      </x:Arguments>
    </Color>
  </BoxView.Color>
</BoxView>
<BoxView HeightRequest="150"
          WidthRequest="150"
          HorizontalOptions="Center">
  <BoxView.Color>
    <Color>
      <x:Arguments>
        <x:Single>0.8</x:Single>
        <x:Single>0.5</x:Single>
        <x:Single>0.2</x:Single>
        <x:Single>0.5</x:Single>
      </x:Arguments>
    </Color>
  </BoxView.Color>
</BoxView>
```



```
</BoxView.Color>
</BoxView>
```

The number of elements within the `x:Arguments` tag, and the types of these elements, must match one of the `Color` constructors. The `Color` constructor with a single parameter requires a grayscale `float` value from 0 (black) to 1 (white). The `Color` constructor with three parameters requires `float` red, green, and blue values ranging from 0 to 1. The `Color` constructor with four parameters adds a `float` alpha channel as the fourth parameter.

Call factory methods

Factory methods can be called in .NET MAUI XAML by specifying the method's name using the `x:FactoryMethod` attribute, and its arguments using the `x:Arguments` attribute. A factory method is a `public static` method that returns objects or values of the same type as the class or structure that defines the methods.

The `Color` class defines a number of factory methods, and the following example demonstrates calling three of them:

XAML

```
<BoxView HeightRequest="150"
          WidthRequest="150"
          HorizontalOptions="Center">
  <BoxView.Color>
    <Color x:FactoryMethod="FromRgba">
      <x:Arguments>
        <x:Byte>192</x:Byte>
        <x:Byte>75</x:Byte>
        <x:Byte>150</x:Byte>
        <x:Byte>128</x:Byte>
      </x:Arguments>
    </Color>
  </BoxView.Color>
</BoxView>
<BoxView HeightRequest="150"
          WidthRequest="150"
          HorizontalOptions="Center">
  <BoxView.Color>
    <Color x:FactoryMethod="FromHsla">
      <x:Arguments>
        <x:Double>0.23</x:Double>
        <x:Double>0.42</x:Double>
        <x:Double>0.69</x:Double>
        <x:Double>0.7</x:Double>
      </x:Arguments>
    </Color>
  </BoxView.Color>
</BoxView>
```

```

        </Color>
    </BoxView.Color>
</BoxView>
<BoxView HeightRequest="150"
    WidthRequest="150"
    HorizontalOptions="Center">
    <BoxView.Color>
        <Color x:FactoryMethod="FromHex">
            <x:Arguments>
                <x:String>#FF048B9A</x:String>
            </x:Arguments>
        </Color>
    </BoxView.Color>
</BoxView>

```

The number of elements within the `x:Arguments` tag, and the types of these elements, must match the arguments of the factory method being called. The `FromRgba` factory method requires four `byte` arguments, which represent the red, green, blue, and alpha values, ranging from 0 to 255 respectively. The `FromHsla` factory method requires four `float` arguments, which represent the hue, saturation, luminosity, and alpha values, ranging from 0 to 1 respectively. The `FromHex` factory method requires a `string` argument that represents the hexadecimal (A)RGB color.

Specify a generic type argument

Generic type arguments for the constructor of a generic type can be specified using the `x:TypeArguments` attribute, as demonstrated in the following example:

XAML

```

<StackLayout>
    <StackLayout.Margin>
        <OnPlatform x:TypeArguments="Thickness">
            <On Platform="iOS" Value="0,20,0,0" />
            <On Platform="Android" Value="5, 10" />
        </OnPlatform>
    </StackLayout.Margin>
</StackLayout>

```

The `OnPlatform` class is a generic class and must be instantiated with an `x:TypeArguments` attribute that matches the target type. In the `On` class, the `Platform` attribute can accept a single `string` value, or multiple comma-delimited `string` values. In this example, the `StackLayout.Margin` property is set to a platform-specific `Thickness`.

For more information about generic type arguments, see [Generics in XAML](#).

Load XAML at runtime

Article • 10/08/2024

When a .NET Multi-platform App UI (.NET MAUI) XAML class is constructed, a [LoadFromXaml](#) method is indirectly called. This occurs because the code-behind file for a XAML class calls the `InitializeComponent` method from its constructor:

C#

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
    }
}
```

When a project containing a XAML file is built, a source generator generates new C# source that contains the definition of the `InitializeComponent` method and adds it to the compilation object. The following example shows the generated `InitializeComponent` method for the `MainPage` class:

C#

```
private void InitializeComponent()
{
    global::Microsoft.Maui.Controls.Xaml.Extensions.LoadFromXaml(this,
    typeof(MainPage));
    ...
}
```

The `InitializeComponent` method calls the [LoadFromXaml](#) method to extract the XAML compiled binary (or its file) from the app package. After extraction, it initializes all of the objects defined in the XAML, connects them all together in parent-child relationships, attaches event handlers defined in code to events set in the XAML file, and sets the resultant tree of objects as the content of the page.

Load XAML at runtime

The `Extensions` class, in the `Microsoft.Maui.Controls.Xaml` namespace, includes [LoadFromXaml](#) extension methods that can be used to load and parse XAML at runtime. The [LoadFromXaml](#) methods are `public`, and therefore can be called from .NET MAUI

applications to load, and parse XAML at runtime. This enables scenarios such as an app downloading XAML from a web service, creating the required view from the XAML, and displaying it in the app.

⚠ Warning

Loading XAML at runtime isn't trim safe and shouldn't be used with full trimming or NativeAOT. It can be made trim safe by annotating all types that could be loaded at runtime with the [DynamicallyAccessedMembers](#) attribute or the [DynamicDependency](#) attribute. However, this is very error prone and isn't recommended. In addition, loading XAML at runtime has a significant performance cost. For more information, see [Trim a .NET MAUI app](#) and [Native AOT deployment](#).

The following code example shows a simple usage:

C#

```
string navigationButtonXAML = "<Button Text=\"Navigate\" />";
Button navigationButton = new Button().LoadFromXaml(navigationButtonXAML);
...
stackLayout.Add(navigationButton);
```

In this example, a [Button](#) instance is created, with its `Text` property value being set from the XAML defined in the `string`. The [Button](#) is then added to a [StackLayout](#) that has been defined in the XAML for the page.

📌 Note

The [LoadFromXaml](#) extension methods allow a generic type argument to be specified. However, it's rarely necessary to specify the type argument, as it will be inferred from the type of the instance it's operating on.

The [LoadFromXaml](#) method can be used to inflate any XAML, with the following example inflating a [ContentPage](#) and then navigating to it:

C#

```
// See the sample for the full XAML string
string pageXAML = "<?xml version=\"1.0\" encoding=\"utf-8\"?>\r\n<ContentPage xmlns=\"http://schemas.microsoft.com/dotnet/2021/maui\" xmlns:x=\"http://schemas.microsoft.com/winfx/2009/xaml\" nx:Class=\"LoadRuntimeXAML.CatalogItem
```

```
sPage\"\\nTitle=\"Catalog Items\">\\n</ContentPage>";
```

```
ContentPage page = new ContentPage().LoadFromXaml(pageXAML);  
await Navigation.PushAsync(page);
```

Access elements

Loading XAML at runtime with the [LoadFromXaml](#) method does not allow strongly-typed access to the XAML elements that have specified runtime object names (using `x:Name`). However, these XAML elements can be retrieved using the `FindByName` method, and then accessed as required:

C#

```
// See the sample for the full XAML string  
string pageXAML = "<?xml version=\"1.0\" encoding=\"utf-8\"?  
>\\r\\n<ContentPage  
xmlns=\"http://schemas.microsoft.com/dotnet/2021/maui\"\\nxmlns:x=\"http://sc  
hemas.microsoft.com/winfx/2009/xaml\"\\nx:Class=\"LoadRuntimeXAML.CatalogItem  
sPage\"\\nTitle=\"Catalog Items\">\\n<StackLayout>\\n<Label  
x:Name=\"monkeyName\"\\n />\\n</StackLayout>\\n</ContentPage>";  
ContentPage page = new ContentPage().LoadFromXaml(pageXAML);  
  
Label monkeyLabel = page.FindByName<Label>("monkeyName");  
monkeyLabel.Text = "Seated Monkey";
```

In this example, the XAML for a [ContentPage](#) is inflated. This XAML includes a [Label](#) named `monkeyName`, which is retrieved using the `FindByName` method, before its `Text` property is set.

XAML Hot Reload for .NET MAUI

Article • 08/30/2024

.NET Multi-platform App UI (.NET MAUI) XAML Hot Reload is a Visual Studio feature that enables you to view the result of XAML changes in your running app, without having to rebuild your project. Without XAML Hot Reload, you have to build and deploy your app every time you want to view the result of a XAML change.

When your .NET MAUI app is running in debug configuration, with the debugger attached, XAML Hot Reload parses your XAML edits and sends those changes to the running app. It preserves your UI state, since it doesn't recreate the UI for the full page, and updates changed properties on controls affected by edits. In addition, your navigate state and data will be maintained, enabling you to quickly iterate on your UI without losing your location in the app. Therefore, you'll spend less time rebuilding and deploying your apps to validate UI changes.

By default, you don't need to save your XAML file to see the results of your edits. Instead, updates are applied immediately as you type. However, on Windows you can change this behavior to update only on file save. This can be accomplished by checking the **Apply XAML Hot Reload on document save** checkbox in the Hot Reload IDE settings available by selecting **Debug > Options > XAML Hot Reload** from the Visual Studio menu bar. Only updating on file save can sometimes be useful if you make bigger XAML updates and don't wish them to be displayed until they are complete.

ⓘ Note

If you're writing a native UWP or WPF app, without using .NET MAUI, see [What is XAML Hot Reload for WPF and UWP apps?](#).

XAML Hot Reload is available in both Visual Studio 2022 and Visual Studio 2022 for Mac. On Windows, XAML Hot Reload is available on Android, iOS, and WinUI on emulators, simulators, and physical devices. On Mac, XAML Hot Reload is available on Android, iOS, and Mac Catalyst on emulators, simulators, and physical devices. In all cases, XAML Hot Reload requires your debug configuration to be named `Debug`.

ⓘ Important

XAML Hot Reload doesn't reload C# code. If you add a new event handler in code, it can only be used in XAML after the code is reloaded. In addition, if you add a new

XAML element with an `x:Name`, the name cannot be used in code until the code is reloaded.

Full page reload

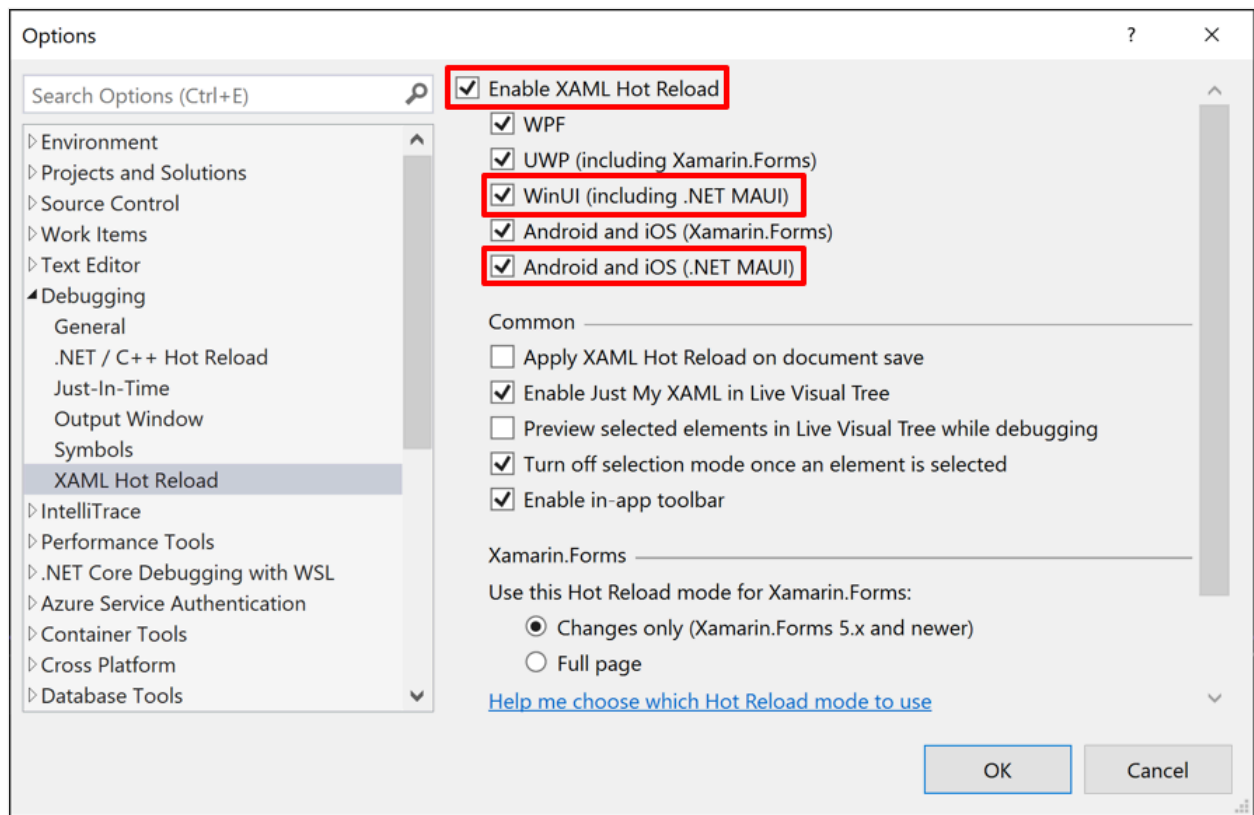
There may be cases where XAML Hot Reload doesn't update the running app as expected. For example, you might change a color property on a .NET MAUI control but not see the color change in the running app. To work around such issues you can fully reload the XAML file by clicking the Hot Reload toolbar button, or pressing `Alt+F10`. Only XAML files that have been edited will be reloaded. The exception to this is if you change a resource dictionary file, in which case all XAML files will be reloaded in order to apply new resources.

Important

When XAML is reloaded some UI state may be lost, such as current focus and selection.

Enable XAML Hot Reload

XAML Hot Reload is enabled by default in Visual Studio 2022. If it's been previously disabled, it can be enabled by selecting **Debug > Options > XAML Hot Reload** from the Visual Studio menu bar. Next, in the **Options** dialog box, ensure that the **Enable XAML Hot Reload, WinUI (including .NET MAUI)**, and **Android and iOS (.NET MAUI)** options are checked:



Then, on iOS in your build settings, check that the Linker is set to "Don't Link".

Reload on multiple platforms

XAML Hot Reload supports simultaneous debugging of multiple platforms in Visual Studio, provided that you have separate head projects per platform rather than a single project app. For example, you can deploy an Android and an iOS target at the same time to see your changes reflected on both platforms at once. To debug on multiple platforms on Windows, see [How To: Set multiple startup projects](#).

Troubleshooting

The XAML Hot Reload output displays status messages that can help with troubleshooting. In Visual Studio, these can be displayed by selecting **View > Output** from the menu bar, and then selecting **Hot Reload** in the **Show output from:** drop-down.

If XAML Hot Reload fails to initialize you should ensure that you're using the latest version of .NET MAUI, the latest version of the IDE, and that your iOS linker settings are set to **Don't Link** in the project's build settings.

XAML Hot Reload requires your debug configuration to be named `Debug`. If a custom build configuration is used, XAML Hot Reload will stop working if the build configuration isn't named `Debug`.

If nothing happens when saving your XAML file, ensure that XAML Hot Reload is enabled in the IDE. For more information, see [Enable XAML Hot Reload](#).

If you make a change that the XAML Hot Reload parser sees as invalid, it will show the error underlined in the editor and include it in the **Error List** window. Hot Reload errors have an error code starting with "XHR" (for XAML Hot Reload). If there are any such errors on the page, XAML Hot Reload won't apply changes to your running app until the errors have been fixed.

You can't add, remove, or rename files or NuGet packages during a XAML Hot Reload session. If you add or remove a file or NuGet package, rebuild and redeploy your app to continue using XAML Hot Reload.

Disabling XAML compilation with `[XamlCompilation(XamlCompilationOptions.Skip)]` isn't supported and can cause issues with the Live Visual Tree. For more information about Live Visual Tree, see [Inspect the visual tree of a .NET MAUI app](#).

Build accessible apps with semantic properties

Article • 11/20/2023

Semantics for accessibility is concerned with building experiences that make your apps inclusive for people who use technology in a wide range of environments and approach your UI with a range of needs and experiences. In many situations, legal requirements for accessibility may provide an impetus for developers to address accessibility issues. Regardless, it's advisable to build inclusive and accessible apps so that your apps reach the largest possible audience.

The [Web Content Accessibility Guidelines \(WCAG\)](#) are the global accessibility standard and legal benchmark for web and mobile. These guidelines describe the various ways in which apps can be made more perceivable, operable, understandable, and robust, for all.

Many user accessibility needs are met by assistive technology products installed by the user or by tools and settings provided by the operating system. This includes functionality such as screen readers, screen magnification, and high-contrast settings.

Screen readers typically provide auditory descriptions of controls that are displayed on the screen. These descriptions help users navigate through the app and provide references to controls, such as images, that have no input or text. Screen readers are often controlled through gestures on the touchscreen, trackpad, or keyboard. For information about enabling screen readers, see [Enable screen readers](#).

Operating systems have their own screen readers with their own unique behavior and configuration. For example, most screen readers read the text associated with a control when it receives focus, enabling users to orient themselves as they navigate through the app. However, some screen readers can also read the entire app user interface when a page appears, which enables the user to receive all of the page's available informational content before attempting to navigate it.

Most screen readers will automatically read any text associated with a control that receives accessibility focus. This means that controls, such as [Label](#) or [Button](#), that have a `Text` property set will be accessible for the user. However, [Image](#), [ImageButton](#), [ActivityIndicator](#), and others might not be in the accessibility tree because no text is associated with them.

.NET Multi-platform App UI (.NET MAUI) supports two approaches to providing access to the accessibility experience of the underlying platform. *Semantic properties* are the

.NET MAUI approach to providing accessibility values in apps, and are the recommended approach. *Automation properties* are the Xamarin.Forms approach to providing accessibility values in apps, and have been superseded by semantic properties. In both cases, the default accessibility order of controls is the same order in which they're listed in XAML or added to the layout. However, different layouts might have additional factors that influence accessibility order. For example, the accessibility order of [StackLayout](#) is also based on its orientation, and the accessibility order of [Grid](#) is based on its row and column arrangement. For more information about content ordering, see [Meaningful Content Ordering](#) [↗] on the Xamarin blog.

ⓘ Note

When a [WebView](#) displays a website that's accessible, it will also be accessible in a .NET MAUI app. Conversely, when a [WebView](#) displays a website that's not accessible, it won't be accessible in a .NET MAUI app.

Semantic properties

Semantic properties are used to define information about which controls should receive accessibility focus and which text should be read aloud to the user. Semantic properties are attached properties that can be added to any element to set the underlying platform accessibility APIs.

ⓘ Important

Semantic properties don't try to force equivalent behavior on each platform. Instead, they rely on the accessibility experience provided by each platform.

The [SemanticProperties](#) class defines the following attached properties:

- [Description](#), of type `string`, which represents a description that will be read aloud by the screen reader. For more information, see [Description](#).
- [Hint](#), of type `string`, which is similar to `Description`, but provides additional context such as the purpose of a control. For more information, see [Hint](#).
- [HeadingLevel](#), of type [SemanticHeadingLevel](#), which enables an element to be marked as a heading to organize the UI and make it easier to navigate. For more information, see [Heading levels](#).

These attached properties set platform accessibility values so that a screen reader can speak about the element. For more information about attached properties, see [Attached](#)

properties.

Description

The [Description](#) attached property represents a short, descriptive `string` that a screen reader uses to announce an element. This property should be set for elements that have a meaning that's important for understanding the content or interacting with the user interface. Setting this property can be accomplished in XAML:

XAML

```
<Image Source="dotnet_bot.png"
        SemanticProperties.Description="Cute dot net bot waving hi to you!"
/>
```

Alternatively, it can be set in C#:

C#

```
Image image = new Image { Source = "dotnet_bot.png" };
SemanticProperties.SetDescription(image, "Cute dot net bot waving hi to you!");
```

In addition, the [SetValue](#) method can also be used to set the [Description](#) attached property:

C#

```
image.SetValue(SemanticProperties.DescriptionProperty, "Cute dot net bot waving hi to you!");
```

The accessibility information for an element can also be defined on another element. For example, a [Label](#) next to an [Switch](#) can be used to describe what the [Switch](#) represents. This can be accomplished in XAML as follows:

XAML

```
<Label x:Name="label"
        Text="Enable dark mode: " />
<Switch SemanticProperties.Description="{Binding Source={x:Reference label}
        Path=Text}" />
```

Alternatively, it can be set in C# as follows:

C#

```
Label label = new Label
{
    Text = "Enable dark mode: "
};
Switch mySwitch = new Switch();
SemanticProperties.SetDescription(mySwitch, label.Text);
```

⚠ Warning

- Avoid setting the **Description** attached property on a **Label**. This will stop the **Text** property being spoken by the screen reader. This is because the visual text should ideally match the text read aloud by the screen reader.
- Avoid setting the **Description** attached property on an **Entry** or **Editor** on Android. Doing so will stop Talkback actions functioning. Instead, use the **Placeholder** property or the **Hint** attached property.
- On iOS, if you set the **Description** property on any control that has children the screen reader will be unable to reach the children. This is because iOS doesn't provide accessibility features that allow the navigation from a parent element into a child element.

Hint

The **Hint** attached property represents a **string** that provides additional context to the **Description** attached property, such as the purpose of a control. Setting this property can be accomplished in XAML:

XAML

```
<Image Source="like.png"
        SemanticProperties.Description="Like"
        SemanticProperties.Hint="Like this post." />
```

Alternatively, it can be set in C#:

C#

```
Image image = new Image { Source = "like.png" };
SemanticProperties.SetDescription(image, "Like");
SemanticProperties.SetHint(image, "Like this post.");
```

In addition, the [SetValue](#) method can also be used to set the [Hint](#) attached property:

C#

```
image.SetValue(SemanticProperties.HintProperty, "Like this post.");
```

On Android, this property behaves slightly differently depending on the control it's attached to. For example, for controls without text values, such as [Switch](#) and [CheckBox](#), the controls will display the hint with the control. However, for controls with text values, the hint is not displayed and is read after the text value.

Warning

The [Hint](#) property conflicts with the `Entry.Placeholder` property on Android, which both map to the same platform property. Therefore, setting a different [Hint](#) value to the `Entry.Placeholder` value isn't recommended.

Heading levels

The [HeadingLevel](#) attached property enables an element to be marked as a heading to organize the UI and make it easier to navigate. Some screen readers enable users to quickly jump between headings.

Headings have a level from 1 to 9, and are represented by the [SemanticHeadingLevel](#) enumeration, which defines `None`, and `Level1` through `Level9` members.

Important

While Windows offers 9 levels of headings, Android and iOS only offer a single heading. Therefore, when [HeadingLevel](#) is set on Windows it maps to the correct heading level. However, when set on Android and iOS it maps to a single heading level.

The following example demonstrates setting this attached property:

XAML

```
<Label Text="Get started with .NET MAUI"
        SemanticProperties.HeadingLevel="Level1" />
<Label Text="Paragraphs of text go here." />
<Label Text="Installation"
        SemanticProperties.HeadingLevel="Level2" />
```

```
<Label Text="Paragraphs of text go here." />
<Label Text="Build your first app"
        SemanticProperties.HeadingLevel="Level3" />
<Label Text="Paragraphs of text go here." />
<Label Text="Publish your app"
        SemanticProperties.HeadingLevel="Level4" />
<Label Text="Paragraphs of text go here." />
```

Alternatively, it can be set in C#:

C#

```
Label label1 = new Label { Text = "Get started with .NET MAUI" };
Label label2 = new Label { Text = "Paragraphs of text go here." };
Label label3 = new Label { Text = "Installation" };
Label label4 = new Label { Text = "Paragraphs of text go here." };
Label label5 = new Label { Text = "Build your first app" };
Label label6 = new Label { Text = "Paragraphs of text go here." };
Label label7 = new Label { Text = "Publish your app" };
Label label8 = new Label { Text = "Paragraphs of text go here." };
SemanticProperties.SetHeadingLevel(label1, SemanticHeadingLevel.Level1);
SemanticProperties.SetHeadingLevel(label3, SemanticHeadingLevel.Level1);
SemanticProperties.SetHeadingLevel(label5, SemanticHeadingLevel.Level1);
SemanticProperties.SetHeadingLevel(label7, SemanticHeadingLevel.Level1);
```

In addition, the [SetValue](#) method can also be used to set the [HeadingLevel](#) attached property:

C#

```
label1.SetValue(SemanticProperties.HeadingLevelProperty,
    SemanticHeadingLevel.Level1);
```

Semantic focus

Controls have a [SetSemanticFocus](#) extension method which forces screen reader focus to a specified element. For example, given a [Label](#) named `label1`, screen reader focus can be forced to the element with the following code:

C#

```
label1.SetSemanticFocus();
```

Semantic screen reader

.NET MAUI provides the [ISemanticScreenReader](#) interface, with which you can instruct a screen reader to announce text to the user. The interface is exposed through the [Default](#) property, and is available in the [Microsoft.Maui.Accessibility](#) namespace.

To instruct a screen reader to announce text, use the [Announce](#) method, passing a `string` argument that represents the text. The following example demonstrates using this method:

C#

```
SemanticScreenReader.Default.Announce("This is the announcement text.");
```

Limitations

The default platform screen reader must be enabled for text to be read aloud.

Automation properties

Automation properties are attached properties that can be added to any element to indicate how the element is reported to the underlying platform's accessibility framework.

The [AutomationProperties](#) class defines the following attached properties:

- [ExcludedWithChildren](#), of type `bool?`, determines if an element and its children should be excluded from the accessibility tree. For more information, see [ExcludedWithChildren](#).
- [IsInAccessibleTree](#), of type `bool?`, indicates whether the element is available in the accessibility tree. For more information, see [IsInAccessibleTree](#).
- [Name](#), of type `string`, represents a short description of the element that serves as a speakable identifier for that element. For more information, see [Name](#).
- [HelpText](#), of type `string`, represents a longer description of the element, which can be thought of as tooltip text that's associated with the element. For more information, see [HelpText](#).
- [LabeledBy](#), of type [VisualElement](#), which enables another element to define accessibility information for the current element. For more information, see [LabeledBy](#).

These attached properties set platform accessibility values so that a screen reader can speak about the element. For more information about attached properties, see [Attached properties](#).

Different screen readers read different accessibility values. Therefore, when using automation properties it's recommended that thorough accessibility testing is carried out on each platform to ensure an optimal experience.

Important

Automation properties are the Xamarin.Forms approach to providing accessibility values in apps, and have been superseded by semantic properties. For more information about semantic properties, see [Semantic properties](#).

ExcludedWithChildren

The [ExcludedWithChildren](#) attached property, of type `bool?`, determines if an element and its children should be excluded from the accessibility tree. This enables scenarios such as displaying an [AbsoluteLayout](#) over another layout such as a [StackLayout](#), with the [StackLayout](#) being excluded from the accessibility tree when it's not visible. It can be used from XAML as follows:

XAML

```
<StackLayout AutomationProperties.ExcludedWithChildren="true">
...
</StackLayout>
```

Alternatively, it can be set in C# as follows:

C#

```
StackLayout stackLayout = new StackLayout();
...
AutomationProperties.SetExcludedWithChildren(stackLayout, true);
```

When this attached property is set, .NET MAUI sets the [IsInAccessibleTree](#) attached property to `false` on the specified element and its children.

IsInAccessibleTree

Warning

This attached property should typically remain unset. The majority of controls should be present in the accessibility tree, and the

`AutomationProperties.ExcludedWithChildren` attached property can be set in scenarios where an element and its children need removing from the accessibility tree.

The `IsInAccessibleTree` attached property, of type `bool?`, determines if the element is visible to screen readers. It must be set to `true` to use the other automation properties. This can be accomplished in XAML as follows:

XAML

```
<Entry AutomationProperties.IsInAccessibleTree="true" />
```

Alternatively, it can be set in C# as follows:

C#

```
Entry entry = new Entry();  
AutomationProperties.SetIsInAccessibleTree(entry, true);
```

Warning

On iOS, if the `IsInAccessibleTree` property is `true` on any control that has children the screen reader will be unable to reach the children. This is because iOS doesn't provide accessibility features that allow the navigation from a parent element into a child element.

Name

Important

The `Name` attached property has been deprecated in .NET 8. Instead, use the `Description` attached property.

The `Name` attached property value should be a short, descriptive text string that a screen reader uses to announce an element. This property should be set for elements that have a meaning that is important for understanding the content or interacting with the user interface. This can be accomplished in XAML as follows:

XAML

```
<ActivityIndicator AutomationProperties.IsInAccessibleTree="true"
    AutomationProperties.Name="Progress indicator" />
```

Alternatively, it can be set in C# as follows:

C#

```
ActivityIndicator activityIndicator = new ActivityIndicator();
AutomationProperties.SetIsInAccessibleTree(activityIndicator, true);
AutomationProperties.SetName(activityIndicator, "Progress indicator");
```

HelpText

Important

The [HelpText](#) attached property has been deprecated in .NET 8. Instead, use the [Hint](#) attached property.

The [HelpText](#) attached property should be set to text that describes the user interface element, and can be thought of as tooltip text associated with the element. This can be accomplished in XAML as follows:

XAML

```
<Button Text="Toggle ActivityIndicator"
    AutomationProperties.IsInAccessibleTree="true"
    AutomationProperties.HelpText="Tap to toggle the activity indicator"
/>
```

Alternatively, it can be set in C# as follows:

C#

```
Button button = new Button { Text = "Toggle ActivityIndicator" };
AutomationProperties.SetIsInAccessibleTree(button, true);
AutomationProperties.SetHelpText(button, "Tap to toggle the activity indicator");
```

On some platforms, for edit controls such as an [Entry](#), the `HelpText` property can sometimes be omitted and replaced with placeholder text. For example, "Enter your name here" is a good candidate for the `Entry.Placeholder` property that places the text in the control prior to the user's actual input.

LabeledBy

Important

The [LabeledBy](#) attached property has been deprecated in .NET 8. Instead, use a `SemanticProperties.Description` binding. For more information, see [SemanticProperties: Description](#).

The [LabeledBy](#) attached property allows another element to define accessibility information for the current element. For example, a [Label](#) next to an [Entry](#) can be used to describe what the [Entry](#) represents. This can be accomplished in XAML as follows:

XAML

```
<Label x:Name="label" Text="Enter your name: " />
<Entry AutomationProperties.IsInAccessibleTree="true"
        AutomationProperties.LabeledBy="{x:Reference label}" />
```

Alternatively, it can be set in C# as follows:

C#

```
Label label = new Label { Text = "Enter your name: " };
Entry entry = new Entry();
AutomationProperties.SetIsInAccessibleTree(entry, true);
AutomationProperties.SetLabeledBy(entry, label);
```

Important

The `AutomationProperties.LabeledByProperty` is not supported on iOS.

Testing accessibility

.NET MAUI apps typically target multiple platforms, which means testing the accessibility features according to the platform. Follow these links to learn how to test accessibility on each platform:

- [Test your app's accessibility](#)  on Android.
- [Verifying app accessibility on iOS](#) .
- [Testing for accessibility on OS X](#) .
- [Accessibility testing](#) on Windows.

The following tools can assist with your accessibility testing:

- [Accessibility Insights](#) for Android and Windows apps.
- [Accessibility Scanner](#) for Android apps.
- [Accessibility Inspector](#) for iOS and macOS apps.
- [Android Studio Layout Inspector](#) for Android apps.
- [Xcode View Debugger](#) for iOS and macOS apps.

However, none of these tools can perfectly emulate the screen reader user experience, and the best way to test and troubleshoot your apps for accessibility will always be manually on physical devices with screen readers.

Enabling screen readers

Each platform has a different default screen reader to narrate accessibility values:

- Android has TalkBack. For information on enabling TalkBack, see [Enable TalkBack](#).
- iOS and macOS have VoiceOver. For information on enabling VoiceOver, see [Enable VoiceOver](#).
- Windows has Narrator. For information on enabling Narrator, see [Enable Narrator](#).

Enable TalkBack

TalkBack is the primary screen reader used on Android. How it's enabled depends on the device manufacturer, Android version, and TalkBack version. However, TalkBack can typically be enabled on your Android device via the device settings:

1. Open the **Settings** app.
2. Select **Accessibility** > **TalkBack**.
3. Turn **Use TalkBack** on.
4. Select **OK**.

ⓘ Note

While these steps apply to most devices, you might experience some differences.

A TalkBack tutorial opens automatically the first time you enable TalkBack.

For alternative methods of enabling TalkBack, see [Turn Talkback on or off](#).

Enable VoiceOver

VoiceOver is the primary screen reader used on iOS and macOS. On iOS, VoiceOver can be enabled as follows:

1. Open the **Settings** app.
2. Select **Accessibility > VoiceOver**.
3. Turn **VoiceOver** on.

A VoiceOver tutorial can be opened by selecting **VoiceOver Practice**, once VoiceOver is enabled.

For alternative methods of enabling VoiceOver, see [Turn on and practice VoiceOver on iPhone](#) and [Turn on and practice VoiceOver on iPad](#).

On macOS, VoiceOver can be enabled as follows:

1. Open the **System Preferences**.
2. Select **Accessibility > VoiceOver**.
3. Select **Enable VoiceOver**.
4. Select **Use VoiceOver**.

A VoiceOver tutorial can be opened by selecting **Open VoiceOver Training....**

For alternative methods of enabling VoiceOver, see [Turn VoiceOver on or off on Mac](#).

Enable Narrator

Narrator is the primary screen reader used on Windows. Narrator can be enabled by pressing the **Windows logo key + Ctrl + Enter** together. These keys can be pressed again to stop Narrator.

For more information about Narrator, see [Complete guide to Narrator](#).

Accessibility checklist

Follow these tips to ensure that your .NET MAUI apps are accessible to the widest audience possible:

- ✓ Ensure your app is perceivable, operable, understandable, and robust for all by following the Web Content Accessibility Guidelines (WCAG). WCAG is the global accessibility standard and legal benchmark for web and mobile. For more information, see [Web Content Accessibility Guidelines \(WCAG\) Overview](#).
- ✓ Make sure the user interface is self-describing. Test that all the elements of your user interface are screen reader accessible. Add descriptive text and hints when

necessary.

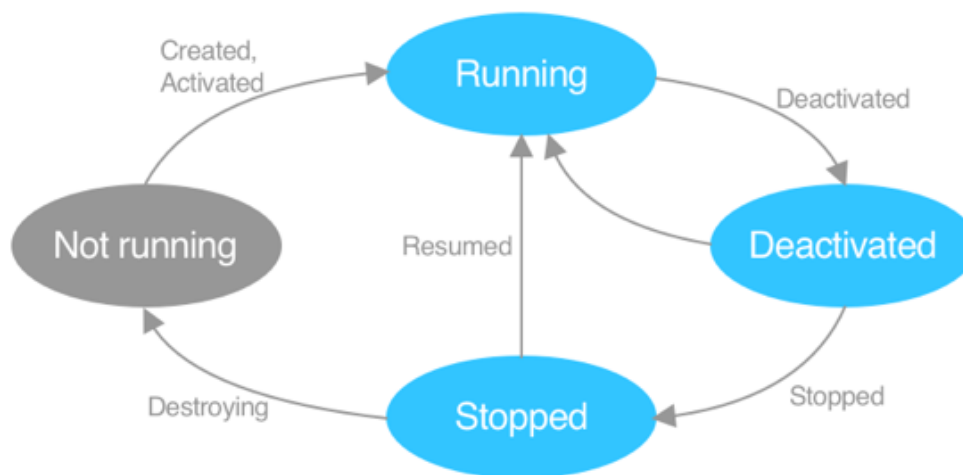
- ✓ Ensure that images and icons have alternate text descriptions.
- ✓ Support large fonts and high contrast. Avoid hardcoding control dimensions, and instead prefer layouts that resize to accommodate larger font sizes. Test color schemes in high-contrast mode to ensure they are readable.
- ✓ Design the visual tree with navigation in mind. Use appropriate layout controls so that navigating between controls using alternate input methods follows the same logical flow as using touch. In addition, exclude unnecessary elements from screen readers (for example, decorative images or labels for fields that are already accessible).
- ✓ Don't rely on audio or color cues alone. Avoid situations where the sole indication of progress, completion, or some other state is a sound or color change. Either design the user interface to include clear visual cues, with sound and color for reinforcement only, or add specific accessibility indicators. When choosing colors, try to avoid a palette that is hard to distinguish for users with color blindness.
- ✓ Provide captions for video content and a readable script for audio content. It's also helpful to provide controls that adjust the speed of audio or video content, and ensure that volume and transport controls are easy to find and use.
- ✓ Localize your accessibility descriptions when the app supports multiple languages.
- ✓ Test the accessibility features of your app on each platform it targets. For more information, see [Testing accessibility](#).

App lifecycle

Article • 10/08/2024

.NET Multi-platform App UI (.NET MAUI) apps generally have four execution states: *not running*, *running*, *deactivated*, and *stopped*. .NET MAUI raises cross-platform lifecycle events on the `Window` class when an app transitions from the not running state to the running state, the running state to the deactivated state, the deactivated state to the stopped state, the stopped state to the running state, and the stopped state to the not running state.

The following diagram shows an overview of the .NET MAUI app lifecycle:



In the diagram, the gray oval indicates that the app isn't loaded into memory. The light blue ovals indicate that the app is in memory. Text on arcs indicates events that are raised by .NET MAUI, that provide notifications to the running app.

The execution state of an app depends on the app's history. For example, when an app is installed for the first time, or a device is started, the app can be considered to be *not running*. When the app is started, the `Created` and `Activated` events are raised and the app is *running*. If a different app window gains focus, the `Deactivated` event is raised and the app is *deactivated*. If the user switches to a different app or returns to the device's Home screen, so that the app window is no longer visible, the `Deactivated` and `Stopped` events are raised and the app is *stopped*. If the user returns to the app, the `Resuming` event is raised and app is *running*. Alternatively, an app might be terminated by a user while it's running. In this situation the app is *deactivated* then *stopped*, the `Destroying` event is raised, and the app is *not running*. Similarly, a device might terminate an app while it's stopped, due to resource restrictions, and the `Destroying` event is raised and the app is *not running*.

In addition, .NET MAUI enables apps to be notified when platform lifecycle events are raised. For more information, see [Platform lifecycle events](#).

Cross-platform lifecycle events

The `Window` class defines the following cross-platform lifecycle events:

Event	Description	Action to take
Created	This event is raised after the native window has been created. At this point the cross-platform window will have a native window handler, but the window might not be visible yet.	
Activated	This event is raised when the window has been activated, and is, or will become, the focused window.	
Deactivated	This event is raised when the window is no longer the focused window. However, the window might still be visible.	
Stopped	This event is raised when the window is no longer visible. There's no guarantee that an app will resume from this state, because it may be terminated by the operating system.	Disconnect from any long running processes, or cancel any pending requests that might consume device resources.
Resumed	This event is raised when an app resumes after being stopped. This event won't be raised the first time your app launches, and can only be raised if the <code>Stopped</code> event has previously been raised.	Subscribe to any required events, and refresh any content that's on the visible page.
Destroying	This event is raised when the native window is being destroyed and deallocated. The same cross-platform window might be used against a new native window when the app is reopened.	Remove any event subscriptions that you've attached to the native window.

These cross-platform events map to different platform events, and the following table shows this mapping:

Event	Android	iOS	Windows
Created	<code>OnPostCreate</code>	<code>FinishedLaunching</code>	<code>Created</code>
Activated	<code>OnResume</code>	<code>OnActivated</code>	<code>Activated</code> (<code>CodeActivated</code> and <code>PointerActivated</code>)
Deactivated	<code>OnPause</code>	<code>OnResignActivation</code>	<code>Activated</code> (<code>Deactivated</code>)
Stopped	<code>OnStop</code>	<code>DidEnterBackground</code>	<code>VisibilityChanged</code>
Resumed	<code>OnRestart</code>	<code>WillEnterForeground</code>	<code>Resumed</code>
Destroying	<code>OnDestroy</code>	<code>WillTerminate</code>	<code>Closed</code>

In addition, the `Window` class also defines a `Backgrounding` event that's raised on iOS and Mac Catalyst when the Window is closed or enters a background state. A `BackgroundingEventArgs` object accompanies this event, and any `string` state should be persisted to the `State` property of the `BackgroundingEventArgs` object, which the OS will preserve until it's time to resume the window.

When the window is resumed the state is provided by the `IActivationState` argument to the `CreateWindow` override.

In addition to these events, the `Window` class also has the following overridable lifecycle methods:

- `OnCreated`, which is invoked when the `Created` event is raised.
- `OnActivated`, which is invoked when the `Activated` event is raised.
- `OnDeactivated`, which is invoked when the `Deactivated` event is raised.
- `OnStopped`, which is invoked when the `Stopped` event is raised.
- `OnResumed`, which is invoked when the `Resumed` event is raised.
- `OnDestroying`, which is invoked when the `Destroying` event is raised.
- `OnBackgrounding`, which is invoked when the `Backgrounding` event is raised.

To subscribe to the `Window` lifecycle events, override the `CreateWindow` method in your `App` class to create a `Window` instance on which you can subscribe to events:

```
C#

namespace MyMauiApp
{
    public partial class App : Application
    {
        public App()
        {
            InitializeComponent();

            protected override Window CreateWindow(IActivationState activationState)
            {
                Window window = base.CreateWindow(activationState);

                window.Created += (s, e) =>
                {
                    // Custom logic
                };

                return window;
            }
        }
    }
}
```

Alternatively, to consume the lifecycle overrides, create a class that derives from the `Window` class

```
C#

namespace MyMauiApp
{
    public class MyWindow : Window
    {
        public MyWindow() : base()
        {
        }

        public MyWindow(Page page) : base(page)
        {
        }
    }
}
```

```

    {
    }

    protected override void OnCreated()
    {
        // Register services
    }
}

```

The `Window`-derived class can then be consumed by overriding the `CreateWindow` method in your `App` class to return a `MyWindow` instance.

Platform lifecycle events

.NET MAUI defines delegates that are invoked in response to platform lifecycle events being raised. Handlers can be specified for these delegates, using named methods or anonymous functions, which are executed when the delegate is invoked. This mechanism enables apps to be notified when common platform lifecycle events are raised.

Important

The `ConfigureLifecycleEvents` method is in the `Microsoft.Maui.LifecycleEvents` namespace.

Android

The following table lists the .NET MAUI delegates that are invoked in response to Android lifecycle events being raised:

 Expand table

Delegate	Arguments	Description	Comments
<code>OnActivityResult</code>	<code>Android.App.Activity</code> , <code>int</code> , <code>Android.App.Result</code> , <code>Android.Content.Intent?</code>	Invoked when an activity you launched exits.	
<code>OnApplicationConfigurationChanged</code>	<code>Android.App.Application</code> , <code>Android.Content.Res.Configuration</code>	Invoked when the device configuration changes while your component is running.	
<code>OnApplicationCreate</code>	<code>Android.App.Application</code>	Invoked when the app has started, before an activity, service, or receiver objects (excluding content providers) have been created.	

Delegate	Arguments	Description	Comments
<code>OnApplicationCreating</code>	<code>Android.App.Application</code>	Invoked when the app is starting, before an activity, service, or receiver objects (excluding content providers) have been created.	
<code>OnApplicationLowMemory</code>	<code>Android.App.Application</code>	Invoked when the system is running low on memory, and actively running processes should trim their memory usage.	
<code>OnApplicationTrimMemory</code>	<code>Android.App.Application</code> , <code>Android.Content.TrimMemory</code>	Invoked when the operating system has determined that it's a good time for a process to trim unneeded memory from its process.	
<code>OnBackPressed</code>	<code>Android.App.Activity</code>	Invoked when the activity has detected a press of the back key.	
<code>OnConfigurationChanged</code>	<code>Android.App.Activity</code> , <code>Android.Content.Res.Configuration</code>	Invoked when the device configuration changes while your activity is running.	
<code>OnCreate</code>	<code>Android.App.Activity</code> , <code>Android.OS.Bundle?</code>	Raised when the activity is created.	
<code>OnDestroy</code>	<code>Android.App.Activity</code>	Invoked when the activity is finishing, or because the system is temporarily destroying the activity instance to save space.	Always call the super class's implementation.
<code>OnNewIntent</code>	<code>Android.App.Activity</code> , <code>Android.Content.Intent?</code>	Invoked when the activity is relaunched while at the top of the activity stack instead of a new instance of the activity being started.	
<code>OnPause</code>	<code>Android.App.Activity</code>	Invoked when an activity is going into the background, but has not yet been killed.	Always call the super class's implementation.

Delegate	Arguments	Description	Comments
<code>OnPostCreate</code>	<code>Android.App.Activity</code> , <code>Android.OS.Bundle?</code>	Invoked when activity startup is complete, after <code>OnStart</code> and <code>OnRestoreInstanceState</code> have been called.	Always call the super class's implementation. This is a system-only event that generally shouldn't be used by apps.
<code>OnPostResume</code>	<code>Android.App.Activity</code>	Invoked when activity resume is complete, after <code>OnResume</code> has been called.	Always call the super class's implementation. This is a system-only event that generally shouldn't be used by apps.
<code>OnRequestPermissionsResult</code>	<code>Android.App.Activity</code> , <code>int</code> , <code>string[]</code> , <code>Android.Content.PM.Permission[]</code>	Invoked as a callback for the result from requesting permissions.	
<code>OnRestart</code>	<code>Android.App.Activity</code>	Invoked after <code>OnStop</code> when the current activity is being redisplayed to the user (the user has navigated back to it).	Always call the super class's implementation.
<code>OnRestoreInstanceState</code>	<code>Android.App.Activity</code> , <code>Android.OS.Bundle</code>	Invoked after <code>OnStart</code> when the activity is being reinitialized from a previously saved state.	
<code>OnResume</code>	<code>Android.App.Activity</code>	Invoked after <code>OnRestoreInstanceState</code> , <code>OnRestart</code> , or <code>OnPause</code> , to indicate that the activity is active and is ready to receive input.	
<code>OnSaveInstanceState</code>	<code>Android.App.Activity</code> , <code>Android.OS.Bundle</code>	Invoked to retrieve per-instance state from an activity being killed so that the state can be restored in <code>OnCreate</code> or <code>OnRestoreInstanceState</code> .	
<code>OnStart</code>	<code>Android.App.Activity</code>	Invoked after <code>OnCreate</code> or <code>OnRestart</code> when the activity has been stopped, but is now	Always call the super class's implementation.

Delegate	Arguments	Description	Comments
		being displayed to the user.	
OnStop	Android.App.Activity	Invoked when the activity is no longer visible to the user.	Always call the super class's implementation.

❗ Important

Each delegate has a corresponding identically named extension method, that can be called to register a handler for the delegate.

To respond to an Android lifecycle delegate being invoked, call the `ConfigureLifecycleEvents` method on the `MauiAppBuilder` object in the `CreateMauiapp` method of your `MauiProgram` class. Then, on the `ILifecycleBuilder` object, call the `AddAndroid` method and specify the `Action` that registers handlers for the required delegates:

C#

```
using Microsoft.Maui.LifecycleEvents;

namespace PlatformLifecycleDemo
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureLifecycleEvents(events =>
                {
#if ANDROID
                    events.AddAndroid(android => android
                        .OnActivityResult((activity, requestCode, resultCode, data) =>
                            LogEvent(nameof(AndroidLifecycle.OnActivityResult), requestCode.ToString()))
                        .OnStart((activity) =>
                            LogEvent(nameof(AndroidLifecycle.OnStart)))
                        .OnCreate((activity, bundle) =>
                            LogEvent(nameof(AndroidLifecycle.OnCreate)))
                        .OnBackPressed((activity) =>
                            LogEvent(nameof(AndroidLifecycle.OnBackPressed)) && false)
                        .OnStop((activity) =>
                            LogEvent(nameof(AndroidLifecycle.OnStop))));
#endif
                    static bool LogEvent(string eventName, string type = null)
                    {
                        System.Diagnostics.Debug.WriteLine($"Lifecycle event:
{eventName}{{(type == null ? string.Empty : $" ({type})"}}");
                        return true;
                    }
                });

            return builder.Build();
        }
    }
}
```

```
}  
}  
}
```

For more information about the Android app lifecycle, see [Understand the Activity Lifecycle](#) on developer.android.com.

iOS and Mac Catalyst

The following table lists the .NET MAUI delegates that are invoked in response to iOS and Mac Catalyst lifecycle events being raised:

 Expand table

Delegate	Arguments	Description
<code>ApplicationSignificantTimeChange</code>	<code>UIKit.UIApplication</code>	Invoked when a significant time change occurs, such as midnight, carrier-changed time, or the start or stop of daylight savings.
<code>ContinueUserActivity</code>	<code>UIKit.UIApplication</code> , <code>Foundation.NSUserActivity</code> , <code>UIKit.UIApplicationRestorationHandler</code>	Invoked when the app receives data associated with a user activity, such as transferring an activity from a different device using Handoff.
<code>DidEnterBackground</code>	<code>UIKit.UIApplication</code>	Invoked when the app has entered the background.
<code>FinishedLaunching</code>	<code>UIKit.UIApplication</code> , <code>Foundation.NSDictionary</code>	Invoked when the app has launched.
<code>OnActivated</code>	<code>UIKit.UIApplication</code>	Invoked when the app is launched and every time the app returns to the foreground.
<code>OnResignActivation</code>	<code>UIKit.UIApplication</code>	Invoked when the app is about to enter the background, be suspended, or when the user receives an interruption such as a phone call or text.
<code>OpenUrl</code>	<code>UIKit.UIApplication</code> , <code>Foundation.NSDictionary</code>	Invoked when the app should open a

Delegate	Arguments	Description
		specified URL.
<code>PerformActionForShortcutItem</code>	<code>UIKit.UIApplication</code> , <code>UIKit.UIApplicationShortcutItem</code> , <code>UIKit.UIOperationHandler</code>	Invoked when a Home screen quick action is initiated.
<code>PerformFetch</code>	<code>UIKit.UIApplication</code> , <code>Action<UIBackgroundFetchResult></code>	Invoked to tell the app that it can begin a fetch operation if it has data to download.
<code>SceneContinueUserActivity</code>	<code>UIKit.UIScene</code> , <code>Foundation.NSUserActivity</code>	Invoked to handle the specified Handoff-related activity.
<code>SceneDidDisconnect</code>	<code>UIKit.UIScene</code>	Invoked when a scene is removed from the app.
<code>SceneDidEnterBackground</code>	<code>UIKit.UIScene</code>	Invoked when a scene is running in the background and isn't onscreen.
<code>SceneDidFailToContinueUserActivity</code>	<code>UIKit.UIScene</code> , <code>string</code> , <code>Foundation.NSError</code>	Invoked to inform the user that the activity couldn't be completed.
<code>SceneDidUpdateUserActivity</code>	<code>UIKit.UIScene</code> , <code>Foundation.NSUserActivity</code>	Invoked when the specified activity is updated.
<code>SceneOnActivated</code>	<code>UIKit.UIScene</code>	Invoked when the scene becomes active and able to respond to user events.
<code>SceneOnResignActivation</code>	<code>UIKit.UIScene</code>	Invoked when the scene is about to resign the active state and stop responding to user events.
<code>SceneOpenUrl</code>	<code>UIKit.UIScene</code> , <code>Foundation.NSSet<UIKit.UIOpenUrlContext></code>	Invoked when a scene asks to open one or more URLs.
<code>SceneRestoreInteractionState</code>	<code>UIKit.UIScene</code> , <code>Foundation.NSUserActivity</code>	Invoked to restore the activity state.
<code>SceneWillConnect</code>	<code>UIKit.UIScene</code> , <code>UIKit.UISceneSession</code> , <code>UIKit.UISceneConnectionOptions</code>	Invoked when a scene is added to the app.
<code>SceneWillContinueUserActivity</code>	<code>UIKit.UIScene</code> , <code>string</code>	Invoked to prepare to receive Handoff-

Delegate	Arguments	Description
		related data.
<code>SceneWillEnterForeground</code>	<code>UIKit.UIScene</code>	Invoked when a scene is about to run in the foreground and become visible to the user.
<code>WillEnterForeground</code>	<code>UIKit.UIApplication</code>	Invoked if the app will be returning from a backgrounded state.
<code>WillFinishLaunching</code>	<code>UIKit.UIApplication</code> , <code>Foundation.NSDictionary</code>	Invoked when app launching has begun, but state restoration has not yet occurred.
<code>WillTerminate</code>	<code>UIKit.UIApplication</code>	Invoked if the app is being terminated due to memory constraints, or directly by the user.
<code>WindowSceneDidUpdateCoordinateSpace</code>	<code>UIKit.UIWindowScene</code> , <code>UIKit.UICoordinateSpace</code> , <code>UIKit.UIInterfaceOrientation</code> , <code>UIKit.UITraitCollection</code>	Invoked when the size, orientation, or traits of a scene change.

Important

Each delegate, with the exception of `PerformFetch`, has a corresponding identically named extension method that can be called to register a handler for the delegate.

To respond to an iOS and Mac Catalyst lifecycle delegate being invoked, call the `ConfigureLifecycleEvents` method on the `MauiAppBuilder` object in the `CreateMauiapp` method of your `MauiProgram` class. Then, on the `ILifecycleBuilder` object, call the `AddiOS` method and specify the `Action` that registers handlers for the required delegates:

C#

```
using Microsoft.Maui.LifecycleEvents;

namespace PlatformLifecycleDemo
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureLifecycleEvents(events =>
```

```

        {
#if IOS || MACCATALYST
            events.AddiOS(ios => ios
                .OnActivated((app) =>
                    LogEvent(nameof(iOSLifecycle.OnActivated)))
                .OnResignActivation((app) =>
                    LogEvent(nameof(iOSLifecycle.OnResignActivation)))
                .DidEnterBackground((app) =>
                    LogEvent(nameof(iOSLifecycle.DidEnterBackground)))
                .WillTerminate((app) =>
                    LogEvent(nameof(iOSLifecycle.WillTerminate))));
#endif

            static bool LogEvent(string eventName, string type = null)
            {
                System.Diagnostics.Debug.WriteLine($"Lifecycle event:
{eventName}{{(type == null ? string.Empty : $" ({type})")}}");
                return true;
            }
        });

        return builder.Build();
    }
}

```

For more information about the iOS app lifecycle, see [Managing Your App's Life Cycle](#) on developer.apple.com.

Windows

The following table lists the .NET MAUI delegates that are invoked in response to Windows lifecycle events being raised:

 Expand table

Delegate	Arguments	Description
OnActivated	Microsoft.UI.Xaml.Window, Microsoft.UI.Xaml.WindowActivatedEventArgs	Invoked when the platform Activated event is raised, if the app isn't resuming.
OnClosed	Microsoft.UI.Xaml.Window, Microsoft.UI.Xaml.WindowEventArgs	Invoked when the platform Closed event is raised.
OnLaunched	Microsoft.UI.Xaml.Window, Microsoft.UI.Xaml.LaunchActivatedEventArgs	Invoked by .NET MAUI's Application.OnLaunched override once the native window has been created and activated.
OnLaunching	Microsoft.UI.Xaml.Window, Microsoft.UI.Xaml.LaunchActivatedEventArgs	Invoked by .NET MAUI's Application.OnLaunched override before the

Delegate	Arguments	Description
		native window has been created and activated.
<code>OnPlatformMessage</code>	<code>Microsoft.UI.Xaml.Window</code> , <code>WindowsPlatformMessageEventArgs</code>	Invoked when .NET MAUI receives specific native Windows messages.
<code>OnPlatformWindowSubclassed</code>	<code>Microsoft.UI.Xaml.Window</code> , <code>WindowsPlatformWindowSubclassedEventArgs</code>	Invoked by .NET MAUI when the Win32 window is subclassed.
<code>OnResumed</code>	<code>Microsoft.UI.Xaml.Window</code>	Invoked when the platform Activated event is raised, if the app is resuming.
<code>OnVisibilityChanged</code>	<code>Microsoft.UI.Xaml.Window</code> , <code>Microsoft.UI.Xaml.WindowVisibilityChangedEventArgs</code>	Invoked when the platform VisibilityChanged event is raised.
<code>OnWindowCreated</code>	<code>Microsoft.UI.Xaml.Window</code>	Invoked when the native window is created for the cross-platform <code>Window</code> .

.NET MAUI exposes specific native Windows messages as a lifecycle event with the `OnPlatformMessage` delegate. The `WindowsPlatformMessageEventArgs` object that accompanies this delegate includes a `MessageId` property, of type `uint`. The value of this property can be examined to determine which message has been passed to your app window. For more information about windows messages, see [Windows Messages \(Get Started with Win32 and C++\)](#). For a list of window message constants, see [Window notifications](#).

Important

Each delegate has a corresponding identically named extension method, that can be called to register a handler for the delegate.

To respond to a Windows lifecycle delegate being invoked, call the `ConfigureLifecycleEvents` method on the `MauiAppBuilder` object in the `CreateMauiApp` method of your `MauiProgram` class. Then, on the `ILifecycleBuilder` object, call the `AddWindows` method and specify the `Action` that registers handlers for the required delegates:

```
C#

using Microsoft.Maui.LifecycleEvents;

namespace PlatformLifecycleDemo
{
    public static class MauiProgram
```

```

{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureLifecycleEvents(events =>
            {
#if WINDOWS
                events.AddWindows(windows => windows
                    .OnActivated((window, args) =>
                        LogEvent(nameof(WindowsLifecycle.OnActivated)))
                    .OnClosed((window, args) =>
                        LogEvent(nameof(WindowsLifecycle.OnClosed)))
                    .OnLaunched((window, args) =>
                        LogEvent(nameof(WindowsLifecycle.OnLaunched)))
                    .OnLaunching((window, args) =>
                        LogEvent(nameof(WindowsLifecycle.OnLaunching)))
                    .OnVisibilityChanged((window, args) =>
                        LogEvent(nameof(WindowsLifecycle.OnVisibilityChanged)))
                    .OnPlatformMessage((window, args) =>
                        {
                            if (args.MessageId == Convert.ToInt32("031A", 16))
                            {
                                // System theme has changed
                            }
                        }
                    ));
#endif

                static bool LogEvent(string eventName, string type = null)
                {
                    System.Diagnostics.Debug.WriteLine($"Lifecycle event:
{eventName} {(type == null ? string.Empty : $" ({type})")");
                    return true;
                }
            });

        return builder.Build();
    }
}

```

Retrieve the Window object

Platform code can retrieve the app's `Window` object from platform lifecycle events, with the `GetWindow` extension method:

```

C#

using Microsoft.Maui.LifecycleEvents;

namespace PlatformLifecycleDemo
{
    public static class MauiProgram
    {
        {
            public static MauiApp CreateMauiApp()
            {
                var builder = MauiApp.CreateBuilder();
                builder

```

```

        .UseMauiApp<App>()
        .ConfigureLifecycleEvents(events =>
        {
            #if WINDOWS
                events.AddWindows(windows => windows
                    .OnClosed((window, args) =>
                    {
                        IWindow appWindow = window.GetWindow();
                    }));
            #endif
        });

        return builder.Build();
    }
}

```

Custom lifecycle events

While .NET MAUI defines delegates that are invoked in response to platform lifecycle events being raised, it only exposes a common set of platform lifecycle events. However, it also includes a mechanism, typically for library authors, that enables apps to be notified when additional platform lifecycle events are raised. The process for accomplishing this is as follows:

- Register an event handler for a platform lifecycle event that isn't exposed by .NET MAUI.
- In the event handler for the platform lifecycle event, retrieve the `ILifecycleEventService` instance and call its `InvokeEvents` method, specifying the platform event name as its argument.

Then, apps that want to receive notification of the platform lifecycle event should modify the `CreateMauiApp` method of their `MauiProgram` class to call the `ConfigureLifecycleEvents` method on the `MauiAppBuilder` object. Then, on the `ILifecycleBuilder` object, call the `AddEvent` method and specify the platform event name and the `Action` that will be invoked when the platform event is raised.

Example

The WinUI 3 `Window.SizeChanged` event occurs when the native app window has first rendered, or has changed its rendering size. .NET MAUI doesn't expose this platform event as a lifecycle event. However, apps can receive notification when this platform event is raised by using the following approach:

- Register an event handler for the `Window.SizeChanged` platform lifecycle event:

```

C#

using Microsoft.Maui.LifecycleEvents;
...

public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();

```

```

        builder
            .UseMauiApp<App>()
            .ConfigureLifecycleEvents(events =>
            {
                #if WINDOWS
                    events.AddWindows(windows => windows
                        .OnWindowCreated(window =>
                        {
                            window.SizeChanged += OnSizeChanged;
                        }));
                #endif
            });

        return builder.Build();
    }

```

- In the event handler for the platform lifecycle event, retrieve the `ILifecycleEventService` instance and call its `InvokeEvents` method, specifying the platform event name as its argument:

```

C#

using Microsoft.Maui.LifecycleEvents;
...

#if WINDOWS
    static void OnSizeChanged(object sender,
        Microsoft.UI.Xaml.WindowSizeChangedEventArgs args)
    {
        ILifecycleEventService service =
            MauiWinUIApplication.Current.Services.GetRequiredService<ILifecycleEventService>
            ();
        service.InvokeEvents(nameof(Microsoft.UI.Xaml.Window.SizeChanged));
    }
#endif

```

The `MauiWinUIApplication` type on Windows can be used to access the native app instance via its `Current` property. The `MauiApplication` type on Android can be used to access the native app instance. Similarly, the `MauiUIApplicationDelegate` type on iOS can be used to access the native app instance.

⚠ Warning

Invoking an unregistered event, with the `InvokeEvents` method, doesn't throw an exception.

- In the `CreateMauiApp` method of your `MauiProgram` class, call the `ConfigureLifecycleEvents` method on the `MauiAppBuilder` object. Then, on the `ILifecycleBuilder` object, call the `AddEvent` method and specify the platform event name and the `Action` that will be invoked when the platform event is raised:

```

C#

```

```

using Microsoft.Maui.LifecycleEvents;

namespace PlatformLifecycleDemo
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureLifecycleEvents(events =>
                {
#if WINDOWS
                    events.AddWindows(windows => windows
                        .OnWindowCreated(window =>
                        {
                            window.SizeChanged += OnSizeChanged;
                        }));

                    events.AddEvent(nameof(Microsoft.UI.Xaml.Window.SizeChanged),
                        () => LogEvent("Window SizeChanged"));
#endif

                    static bool LogEvent(string eventName, string type = null)
                    {
                        System.Diagnostics.Debug.WriteLine($"Lifecycle event:
{eventName} {(type == null ? string.Empty : $" ({type})")}");
                        return true;
                    }
                });

            return builder.Build();
        }
    }
}

```

The overall effect is that when a user changes the app window size on Windows, the action specified in the `AddEvent` method is executed.

ⓘ Note

The `AddEvent` method also has an overload that enables a delegate to be specified.

Behaviors

Article • 11/29/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) behaviors let you add extra functionality to user interface controls, such as [Button](#) or [Entry](#), without having to subclass the controls. Instead, the extra functionality is implemented in a [Behavior](#) class and attached to the control.

Behaviors can be used to add different functionality to UI controls, such as:

- Input validations. Behaviors can be used with input controls to ensure the data is valid. For example, you might write a behavior for an [Entry](#) to ensure that a phone number or email address is in the correct format.
- Enhancing the UI. Behaviors can be used to enhance UI controls in your app. For example, you might write a behavior for a [Button](#) to make it shake when it's clicked, or change color.
- Runtime responses. Behaviors can be used to make controls react to different situations. For example, you might automatically save user input in an [Entry](#) when a user finishes typing.

.NET MAUI supports three different types of behaviors:

- Attached behaviors are `static` classes with one or more attached properties. For more information about attached behaviors, see [Attached behaviors](#).
- .NET MAUI behaviors are classes that derive from the [Behavior](#) or [Behavior<T>](#) class, where `T` is the type of the control to which the behavior should apply. For more information, see [.NET MAUI Behaviors](#).
- Platform behaviors are classes that derive from the [PlatformBehavior<TView>](#) or [PlatformBehavior<TView,TPlatformView>](#) class. These behaviors can respond to arbitrary conditions and events on a native control. For more information, see [Platform behaviors](#).

Attached behaviors

Attached behaviors are static classes with one or more attached properties. An attached property is a special type of bindable property. They are defined in one class but attached to other objects, and they are recognizable in XAML as attributes that contain a class and a property name separated by a period. For more information about attached properties, see [Attached properties](#).

An attached property can define a `propertyChanged` delegate that will be executed when the value of the property changes, such as when the property is set on a control. When the `propertyChanged` delegate executes, it's passed a reference to the control on which it is being attached, and parameters that contain the old and new values for the property. This delegate can be used to add new functionality to the control that the property is attached to by manipulating the reference that is passed in, as follows:

1. The `propertyChanged` delegate casts the control reference, which is received as a `BindableObject`, to the control type that the behavior is designed to enhance.
2. The `propertyChanged` delegate modifies properties of the control, calls methods of the control, or registers event handlers for events exposed by the control, to implement the core behavior functionality.

Warning

Attached behaviors are defined in a `static` class, with `static` properties and methods. This makes it difficult to create attached behaviors that have state.

Create an attached behavior

An attached behavior can be implemented by creating a static class that contains an attached property that specifies a `propertyChanged` delegate.

The following example shows the `AttachedNumericValidationBehavior` class, which highlights the value entered by the user into an `Entry` control in red if it's not a `double`:

C#

```
public static class AttachedNumericValidationBehavior
{
    public static readonly BindableProperty AttachBehaviorProperty =
        BindableProperty.CreateAttached("AttachBehavior", typeof(bool),
        typeof(AttachedNumericValidationBehavior), false, propertyChanged:
        OnAttachBehaviorChanged);

    public static bool GetAttachBehavior(BindableObject view)
    {
        return (bool)view.GetValue(AttachBehaviorProperty);
    }

    public static void SetAttachBehavior(BindableObject view, bool value)
    {
        view.SetValue(AttachBehaviorProperty, value);
    }
}
```

```

static void OnAttachBehaviorChanged(BindableObject view, object
oldValue, object newValue)
{
    Entry entry = view as Entry;
    if (entry == null)
    {
        return;
    }

    bool attachBehavior = (bool)newValue;
    if (attachBehavior)
    {
        entry.TextChanged += OnEntryTextChanged;
    }
    else
    {
        entry.TextChanged -= OnEntryTextChanged;
    }
}

static void OnEntryTextChanged(object sender, TextChangedEventArgs args)
{
    double result;
    bool isValid = double.TryParse(args.NewTextValue, out result);
    ((Entry)sender).TextColor = isValid ? Colors.Black : Colors.Red;
}
}

```

In this example, the `AttachedNumericValidationBehavior` class contains an attached property named `AttachBehavior` with a `static` getter and setter, which controls the addition or removal of the behavior to the control to which it will be attached. This attached property registers the `OnAttachBehaviorChanged` method that will be executed when the value of the property changes. This method registers or de-registers an event handler for the `TextChanged` event, based on the value of the `AttachBehavior` attached property. The core functionality of the behavior is provided by the `OnEntryTextChanged` method, which parses the value entered in the `Entry` and sets the `TextColor` property to red if the value isn't a `double`.

Consume an attached behavior

An attached behavior can be consumed by setting its attached property on the target control.

The following example shows consuming the `AttachedNumericValidationBehavior` class on an `Entry` by adding the `AttachBehavior` attached property to the `Entry`:

```
<ContentPage ...
    xmlns:local="clr-namespace:BehaviorsDemos">
    <Entry Placeholder="Enter a System.Double"
local:AttachedNumericValidationBehavior.AttachBehavior="true" />
</ContentPage>
```

The equivalent `Entry` in C# is shown in the following code example:

C#

```
Entry entry = new Entry { Placeholder = "Enter a System.Double" };
AttachedNumericValidationBehavior.SetAttachBehavior(entry, true);
```

The following screenshot shows the attached behavior responding to invalid input:



ⓘ Note

Attached behaviors are written for a specific control type (or a superclass that can apply to many controls), and they should only be added to a compatible control.

Remove an attached behavior

The `AttachedNumericValidationBehavior` class can be removed from a control by setting the `AttachBehavior` attached property to `false`:

XAML

```
<Entry Placeholder="Enter a System.Double"
local:AttachedNumericValidationBehavior.AttachBehavior="false" />
```

At runtime, the `OnAttachBehaviorChanged` method will be executed when the value of the `AttachBehavior` attached property is set to `false`. The `OnAttachBehaviorChanged` method will then de-register the event handler for the `TextChanged` event, ensuring that the behavior isn't executed as you interact with the control.

.NET MAUI behaviors

.NET MAUI behaviors are created by deriving from the `Behavior` or `Behavior<T>` class.

The process for creating a .NET MAUI behavior is as follows:

1. Create a class that inherits from the [Behavior](#) or [Behavior<T>](#) class, where **T** is the type of the control to which the behavior should apply.
2. Override the [OnAttachedTo](#) method to perform any required setup.
3. Override the [OnDetachingFrom](#) method to perform any required cleanup.
4. Implement the core functionality of the behavior.

This results in the structure shown in the following example:

C#

```
public class MyBehavior : Behavior<View>
{
    protected override void OnAttachedTo(View bindable)
    {
        base.OnAttachedTo(bindable);
        // Perform setup
    }

    protected override void OnDetachingFrom(View bindable)
    {
        base.OnDetachingFrom(bindable);
        // Perform clean up
    }

    // Behavior implementation
}
```

The [OnAttachedTo](#) method is called immediately after the behavior is attached to a control. This method receives a reference to the control to which it is attached, and can be used to register event handlers or perform other setup that's required to support the behavior functionality. For example, you could subscribe to an event on a control. The behavior functionality would then be implemented in the event handler for the event.

The [OnDetachingFrom](#) method is called when the behavior is removed from the control. This method receives a reference to the control to which it is attached, and is used to perform any required cleanup. For example, you could unsubscribe from an event on a control to prevent memory leaks.

The behavior can then be consumed by attaching it to the [Behaviors](#) collection of the control.

Create a .NET MAUI Behavior

A .NET MAUI behavior can be implemented by creating a class that derives from the [Behavior](#) or [Behavior<T>](#) class, and overriding the [OnAttachedTo](#) and [OnDetachingFrom](#) methods.

The following example shows the `NumericValidationBehavior` class, which highlights the value entered by the user into an [Entry](#) control in red if it's not a `double`:

C#

```
public class NumericValidationBehavior : Behavior<Entry>
{
    protected override void OnAttachedTo(Entry entry)
    {
        entry.TextChanged += OnEntryTextChanged;
        base.OnAttachedTo(entry);
    }

    protected override void OnDetachingFrom(Entry entry)
    {
        entry.TextChanged -= OnEntryTextChanged;
        base.OnDetachingFrom(entry);
    }

    void OnEntryTextChanged(object sender, TextChangedEventArgs args)
    {
        double result;
        bool isValid = double.TryParse(args.NewTextValue, out result);
        ((Entry)sender).TextColor = isValid ? Colors.Black : Colors.Red;
    }
}
```

In this example, the `NumericValidationBehavior` class derives from the [Behavior<T>](#) class, where `T` is an [Entry](#). The [OnAttachedTo](#) method registers an event handler for the `TextChanged` event, with the [OnDetachingFrom](#) method de-registering the `TextChanged` event to prevent memory leaks. The core functionality of the behavior is provided by the `OnEntryTextChanged` method, which parses the value entered in the [Entry](#) and sets the `TextColor` property to red if the value isn't a `double`.

Important

.NET MAUI does not set the `BindingContext` of a behavior, because behaviors can be shared and applied to multiple controls through styles.

Consume a .NET MAUI behavior

Every .NET MAUI control has a [Behaviors](#) collection, to which one or more behaviors can be added:

XAML

```
<Entry Placeholder="Enter a System.Double">
    <Entry.Behaviors>
        <local:NumericValidationBehavior />
    </Entry.Behaviors>
</Entry>
```

The equivalent [Entry](#) in C# is shown in the following code example:

C#

```
Entry entry = new Entry { Placeholder = "Enter a System.Double" };
entry.Behaviors.Add(new NumericValidationBehavior());
```

The following screenshot shows the .NET MAUI behavior responding to invalid input:



Warning

.NET MAUI behaviors are written for a specific control type (or a superclass that can apply to many controls), and they should only be added to a compatible control. Attempting to attach a .NET MAUI behavior to an incompatible control will result in an exception being thrown.

Consume a .NET MAUI behavior with a style

.NET MAUI behaviors can be consumed by an explicit or implicit style. However, creating a style that sets the [Behaviors](#) property of a control is not possible because the property is read-only. The solution is to add an attached property to the behavior class that controls adding and removing the behavior. The process is as follows:

1. Add an attached property to the behavior class that will be used to control the addition or removal of the behavior to the control to which the behavior will be attached. Ensure that the attached property registers a `propertyChanged` delegate that will be executed when the value of the property changes.
2. Create a `static` getter and setter for the attached property.
3. Implement logic in the `propertyChanged` delegate to add and remove the behavior.

The following example shows the `NumericValidationStyleBehavior` class, which has an attached property that controls adding and removing the behavior:

C#

```
public class NumericValidationStyleBehavior : Behavior<Entry>
{
    public static readonly BindableProperty AttachBehaviorProperty =
        BindableProperty.CreateAttached("AttachBehavior", typeof(bool),
        typeof(NumericValidationStyleBehavior), false, propertyChanged:
        OnAttachBehaviorChanged);

    public static bool GetAttachBehavior(BindableObject view)
    {
        return (bool)view.GetValue(AttachBehaviorProperty);
    }

    public static void SetAttachBehavior(BindableObject view, bool value)
    {
        view.SetValue(AttachBehaviorProperty, value);
    }

    static void OnAttachBehaviorChanged(BindableObject view, object
oldValue, object newValue)
    {
        Entry entry = view as Entry;
        if (entry == null)
        {
            return;
        }

        bool attachBehavior = (bool)newValue;
        if (attachBehavior)
        {
            entry.Behaviors.Add(new NumericValidationStyleBehavior());
        }
        else
        {
            Behavior toRemove = entry.Behaviors.FirstOrDefault(b => b is
NumericValidationStyleBehavior);
            if (toRemove != null)
            {
                entry.Behaviors.Remove(toRemove);
            }
        }
    }

    protected override void OnAttachedTo(Entry entry)
    {
        entry.TextChanged += OnEntryTextChanged;
        base.OnAttachedTo(entry);
    }

    protected override void OnDetachingFrom(Entry entry)
```

```

{
    entry.TextChanged -= OnEntryTextChanged;
    base.OnDetachingFrom(entry);
}

void OnEntryTextChanged(object sender, TextChangedEventArgs args)
{
    double result;
    bool isValid = double.TryParse(args.NewTextValue, out result);
    ((Entry)sender).TextColor = isValid ? Colors.Black : Colors.Red;
}
}

```

In this example, the `NumericValidationStyleBehavior` class contains an attached property named `AttachBehavior` with a `static` getter and setter, which controls the addition or removal of the behavior to the control to which it will be attached. This attached property registers the `OnAttachBehaviorChanged` method that will be executed when the value of the property changes. This method adds or removes the behavior to the control, based on the value of the `AttachBehavior` attached property.

The following code example shows an *explicit* style for the `NumericValidationStyleBehavior` that uses the `AttachBehavior` attached property, and which can be applied to [Entry](#) controls:

XAML

```

<Style x:Key="NumericValidationStyle" TargetType="Entry">
    <Style.Setters>
        <Setter
            Property="local:NumericValidationStyleBehavior.AttachBehavior" Value="true"
        />
    </Style.Setters>
</Style>

```

The [Style](#) can be applied to an [Entry](#) by setting its [Style](#) property to the style using the [StaticResource](#) markup extension:

XAML

```

<Entry Placeholder="Enter a System.Double" Style="{StaticResource
    NumericValidationStyle}">

```

For more information about styles, see [Styles](#).

📌 Note

While you can add bindable properties to a behavior that is set or queried in XAML, if you do create behaviors that have state they should not be shared between controls in a [Style](#) in a [ResourceDictionary](#).

Remove a .NET MAUI behavior

The [OnDetachingFrom](#) method is called when a behavior is removed from a control, and is used to perform any required cleanup such as unsubscribing from an event to prevent a memory leak. However, behaviors are not implicitly removed from controls unless the control's [Behaviors](#) collection is modified by the `Remove` or `Clear` method:

C#

```
Behavior toRemove = entry.Behaviors.FirstOrDefault(b => b is
NumericValidationStyleBehavior);
if (toRemove != null)
{
    entry.Behaviors.Remove(toRemove);
}
```

Alternatively, the control's [Behaviors](#) collection can be cleared:

C#

```
entry.Behaviors.Clear();
```

ⓘ Note

.NET MAUI behaviors aren't implicitly removed from controls when pages are popped from the navigation stack. Instead, they must be explicitly removed prior to pages going out of scope.

Platform behaviors

Platform behaviors are created by deriving from the [PlatformBehavior<TView>](#) or [PlatformBehavior<TView,TPlatformView>](#) class. They respond to arbitrary conditions and events on a native control.

A platform behavior can be implemented through conditional compilation, or partial classes. The approach adopted here is to use partial classes, where a platform behavior typically consists of a cross-platform partial class that defines the behaviors API, and a

native partial class that implements the behavior on each platform. At build time, multi-targeting combines the partial classes to build the platform behavior on each platform.

The process for creating a platform behavior is as follows:

1. Create a cross-platform partial class that defines the API for the platform behavior.
2. Create a native partial class on each platform your app is built for, that has the same name as the cross-platform partial class. This native partial class should inherit from the `PlatformBehavior<TView>` or `PlatformBehavior<TView,TPlatformView>` class, where `TView` is the cross-platform control to which the behavior should apply, and `TPlatformView` is the native view that implements the cross-platform control on a particular platform.

⚠ Note

While it's required to create a native partial class on each platform your app is built for, it's not required to implement the platform behavior functionality on every platform. For example, you might create a platform behavior that modifies the border thickness of a native control on some, but not all, platforms.

3. In each native partial class that you require to implement the platform behavior you should:
 - a. Override the `OnAttachedTo` method to perform any setup.
 - b. Override the `OnDetachedFrom` method to perform any cleanup.
 - c. Implement the core functionality of the platform behavior.

The behavior can then be consumed by attaching it to the `Behaviors` collection of the control.

Create a platform behavior

To create a platform behavior you must first create a cross-platform partial class that defines the API for the platform behavior:

C#

```
namespace BehaviorsDemos
{
    public partial class TintColorBehavior
    {
        public static readonly BindableProperty TintColorProperty =
            BindableProperty.Create(nameof(TintColor), typeof(Color),
```

```

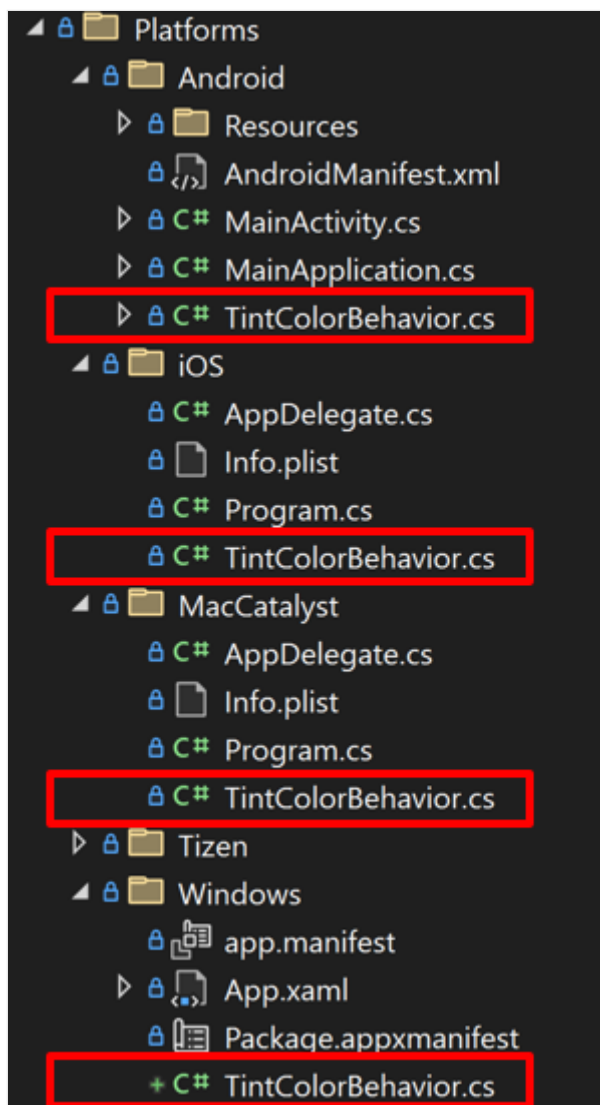
typeof(TintColorBehavior));

    public Color TintColor
    {
        get => (Color)GetValue(TintColorProperty);
        set => SetValue(TintColorProperty, value);
    }
}
}

```

The platform behavior is a partial class whose implementation will be completed on each required platform with an additional partial class that uses the same name. In this example, the `TintColorBehavior` class defines a single bindable property, `TintColor`, that will tint an image with the specified color.

After creating the cross-platform partial class you should create a native partial class on each platform you build the app for. This can be accomplished by adding partial classes to the required child folders of the *Platforms* folder:



Alternatively you could configure your project to support filename-based multi-targeting, or folder-based multi-targeting, or both. For more information about multi-

targeting, see [Configure multi-targeting](#).

The native partial classes should inherit from the [PlatformBehavior<TView>](#) class or the [PlatformBehavior<TView,TPlatformView>](#) class, where `TView` is the cross-platform control to which the behavior should apply, and `TPlatformView` is the native view that implements the cross-platform control on a particular platform. In each native partial class that you require to implement the platform behavior, you should override the [OnAttachedTo](#) method and the [OnDetachedFrom](#) method, and implement the core functionality of the platform behavior.

The [OnAttachedTo](#) method is called immediately after the platform behavior is attached to a cross-platform control. The method receives a reference to the cross-platform control to which it is attached, and optionally a reference to the native control that implements the cross-platform control. The method can be used to register event handlers or perform other setup that's required to support the platform behavior functionality. For example, you could subscribe to an event on a control. The behavior functionality would then be implemented in the event handler for the event.

The [OnDetachedFrom](#) method is called when the behavior is removed from the cross-platform control. The method receives a reference to the control to which it is attached, and optionally a reference to the native control that implements the cross-platform control. The method should be used to perform any required cleanup. For example, you could unsubscribe from an event on a control to prevent memory leaks.

Important

The partial classes must reside in the same namespace and use identical names.

The following example shows the `TintColorBehavior` partial class for Android, which tints an image with a specified color:

C#

```
using Android.Graphics;
using Android.Widget;
using Microsoft.Maui.Platform;
using Color = Microsoft.Maui.Graphics.Color;

namespace BehaviorsDemos
{
    public partial class TintColorBehavior : PlatformBehavior<Image,
    ImageView>
    {
        protected override void OnAttachedTo(Image bindable, ImageView
platformView)
```

```

    {
        base.OnAttachedTo(bindable, platformView);

        if (bindable is null)
            return;
        if (TintColor is null)
            ClearColor(platformView);
        else
            ApplyColor(platformView, TintColor);
    }

    protected override void OnDetachedFrom(Image bindable, ImageView
platformView)
    {
        base.OnDetachedFrom(bindable, platformView);

        if (bindable is null)
            return;
        ClearColor(platformView);
    }

    void ApplyColor(ImageView imageView, Color color)
    {
        imageView.SetColorFilter(new
PorterDuffColorFilter(color.ToPlatform(), PorterDuff.Mode.SrcIn ?? throw new
NullReferenceException()));
    }

    void ClearColor(ImageView imageView)
    {
        imageView.ClearColorFilter();
    }
}
}

```

In this example, the `TintColorBehavior` class derives from the `PlatformBehavior<TView,TPlatformView>` class, where `TView` is an `Image` and `TPlatformView` is an `ImageView`. The `OnAttachedTo` applies the tint color to the image, provided that the `TintColor` property has a value. The `OnDetachedFrom` method removes the tint color from the image.

A native partial class must be added on each platform you build your app for. However, you can make the native partial class NO-OP, if the platform behavior isn't required on a specific platform. This can be achieved by providing an empty class:

```

C#

using Microsoft.UI.Xaml;

namespace BehaviorsDemos
{

```

```

public partial class TintColorBehavior : PlatformBehavior<Image,
FrameworkElement>
{
    // NO-OP on Windows
}

```

❗ Important

.NET MAUI doesn't set the `BindingContext` of a platform behavior.

Consume a platform behavior

Every .NET MAUI control has a `Behaviors` collection, to which one or more platform behaviors can be added:

XAML

```

<Image Source="dotnet_bot.png"
        HeightRequest="200"
        HorizontalOptions="Center">
    <Image.Behaviors>
        <local:TintColorBehavior TintColor="Red" />
    </Image.Behaviors>
</Image>

```

The equivalent `Image` in C# is shown in the following example:

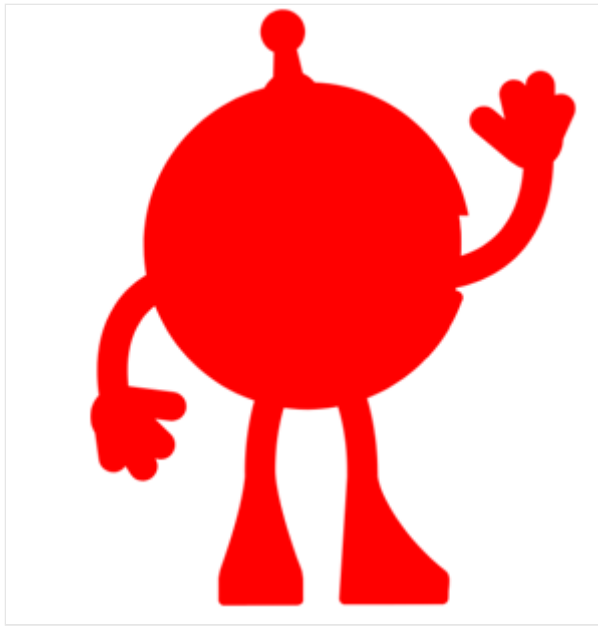
C#

```

Image image = new Image { Source = "dotnet_bot.png", HeightRequest = 200,
HorizontalOptions = LayoutOptions.Center };
image.Behaviors.Add(new TintColorBehavior());

```

The following screenshot shows the platform behavior tinting an image:



Warning

Platform behaviors are written for a specific control type (or a superclass that can apply to many controls), and they should only be added to a compatible control. Attempting to attach a platform behavior to an incompatible control will result in an exception being thrown.

Data binding

Article • 04/21/2023

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) app consists of one or more pages, each of which typically contains multiple user-interface objects called *views*. One of the primary tasks of the app is to keep these views synchronized and to keep track of the various values or selections that they represent. Often the views represent values from an underlying data source, and users manipulate these views to change that data. When the view changes, the underlying data must reflect that change, and similarly, when the underlying data changes, that change must be reflected in the view.

To handle these changes successfully, the app must be notified of changes in these views or the underlying data. The common solution is to define events that signal when a change occurs. An event handler can then be installed that is notified of these changes. It responds by transferring data from one object to another. However, when there are many views, there must also be many event handlers, which results in a lot of boilerplate code.

Data binding automates this task and renders the event handlers unnecessary. Data bindings can be implemented either in XAML or code, but they're much more common in XAML where they help to reduce the size of the code-behind file. By replacing procedural code in event handlers with declarative code or markup, the app is simplified and clarified.

Data binding is therefore the technique of linking properties of two objects so that changes in one property are automatically reflected in the other property. One of the two objects involved in a data binding is almost always an element that derives from [View](#) and forms part of the visual interface of a page. The other object is either:

- Another [View](#) derivative, usually on the same page.
- An object in a code file.

Data bindings between two [View](#) derivatives are often shown in these articles, for purposes of clarity and simplicity. However, the same principles can be applied to data bindings between a [View](#) and other objects. When an application is built using the Model-View-ViewModel (MVVM) architecture, the class with underlying data is often called a viewmodel.

 **Important**

.NET MAUI marshals binding updates to the UI thread. When using MVVM this enables you to update data-bound viewmodel properties from any thread, with .NET MAUI's binding engine bringing the updates to the UI thread.

Basic bindings

Article • 09/26/2024

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) data binding links a pair of properties between two objects, at least one of which is usually a user-interface object. These two objects are called the *target* and the *source*:

- The *target* is the object (and property) on which the data binding is set.
- The *source* is the object (and property) referenced by the data binding.

In the simplest case, data flows from the source to the target, which means that the value of the target property is set from the value of the source property. However, in some cases, data can alternatively flow from the target to the source, or in both directions.

Important

The target is always the object on which the data binding is set even if it's providing data rather than receiving data.

Bindings with a binding context

Consider the following XAML example, whose intent is to rotate a [Label](#) by manipulating a [Slider](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="DataBindingDemos.BasicCodeBindingPage"
              Title="Basic Code Binding">
    <StackLayout Padding="10, 0">
        <Label x:Name="label"
               Text="TEXT"
               FontSize="48"
               HorizontalOptions="Center"
               VerticalOptions="Center" />

        <Slider x:Name="slider"
                Maximum="360"
                VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

```
</StackLayout>
</ContentPage>
```

Without data bindings, you would set the `ValueChanged` event of the `Slider` to an event handler that accesses the `Value` property of the `Slider` and sets that value to the `Rotation` property of the `Label`. The data binding automates this task, and so the event handler and the code within it are no longer necessary.

You can set a binding on an instance of any class that derives from `BindableObject`, which includes `Element`, `VisualElement`, `View`, and `View` derivatives. The binding is always set on the target object. The binding references the source object. To set the data binding, use the following two members of the target class:

- The `BindingContext` property specifies the source object.
- The `SetBinding` method specifies the target property and source property.

In this example, the `Label` is the binding target, and the `Slider` is the binding source. Changes in the `Slider` source affect the rotation of the `Label` target. Data flows from the source to the target.

The `SetBinding` method defined by `BindableObject` has an argument of type `BindingBase` from which the `Binding` class derives, but there are other `SetBinding` methods defined by the `BindableObjectExtensions` class. The code-behind for the XAML uses a simpler `SetBinding` extension method from the `BindableObjectExtensions` class:

C#

```
public partial class BasicCodeBindingPage : ContentPage
{
    public BasicCodeBindingPage()
    {
        InitializeComponent();

        label.BindingContext = slider;
        label.SetBinding(Label.RotationProperty, "Value");
    }
}
```

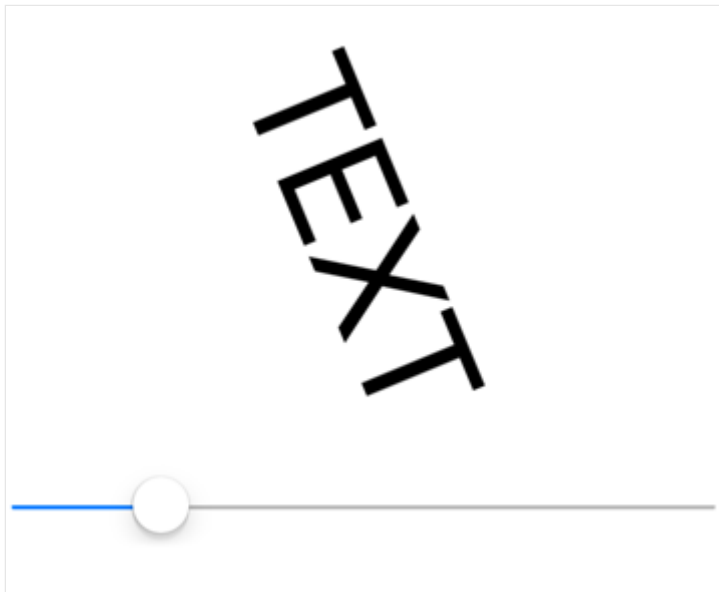
The `Label` object is the binding target so that's the object on which this property is set and on which the method is called. The `BindingContext` property indicates the binding source, which is the `Slider`. The `SetBinding` method is called on the binding target but specifies both the target property and the source property. The target property is specified as a `BindableProperty` object: `Label.RotationProperty`. The source property is specified as a string and indicates the `Value` property of `Slider`.

❗ Important

The target property must be backed by a bindable property. Therefore, the target object must be an instance of a class that derives from [BindableObject](#). For more information, see [Bindable properties](#).

The source property is specified as a string. Internally, reflection is used to access the actual property. In this particular case, however, the `Value` property is also backed by a bindable property.

As you manipulate the [Slider](#), the [Label](#) rotates accordingly:



Alternatively, the data binding can be specified in XAML:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="DataBindingDemos.BasicXamlBindingPage"
    Title="Basic XAML Binding">
    <StackLayout Padding="10, 0">
        <Label Text="TEXT"
            FontSize="80"
            HorizontalOptions="Center"
            VerticalOptions="Center"
            BindingContext="{x:Reference Name=slider}"
            Rotation="{Binding Path=Value}" />

        <Slider x:Name="slider"
            Maximum="360"
            VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

```
</StackLayout>
</ContentPage>
```

Just as in code, the data binding is set on the target object, which is the [Label](#). Two XAML markup extensions are used to define the data binding:

- The `x:Reference` markup extension is required to reference the source object, which is the [Slider](#) named `slider`.
- The `Binding` markup extension links the `Rotation` property of the [Label](#) to the `Value` property of the [Slider](#).

For more information about XAML markup extensions, see [Consume XAML markup extensions](#).

⚠ Note

The source property is specified with the `Path` property of the `Binding` markup extension, which corresponds with the `Path` property of the `Binding` class.

XAML markup extensions such as `x:Reference` and `Binding` can have *content property* attributes defined, which for XAML markup extensions means that the property name doesn't need to appear. The `Name` property is the content property of `x:Reference`, and the `Path` property is the content property of `Binding`, which means that they can be eliminated from the expressions:

XAML

```
<Label Text="TEXT"
      FontSize="80"
      HorizontalOptions="Center"
      VerticalOptions="Center"
      BindingContext="{x:Reference slider}"
      Rotation="{Binding Value}" />
```

📌 Important

Binding performance can be improved by using compiled bindings. For more information, see [Compiled bindings](#).

Bindings without a binding context

The `BindingContext` property is an important component of data bindings, but it is not always necessary. The source object can instead be specified in the `SetBinding` call or the `Binding` markup extension:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="DataBindingDemos.AlternativeCodeBindingPage"
              Title="Alternative Code Binding">
    <StackLayout Padding="10, 0">
        <Label x:Name="label"
               Text="TEXT"
               FontSize="40"
               HorizontalOptions="Center"
               VerticalOptions="CenterAndExpand" />

        <Slider x:Name="slider"
                Minimum="-2"
                Maximum="2"
                VerticalOptions="CenterAndExpand" />
    </StackLayout>
</ContentPage>
```

In this example, the `Slider` is defined to control the `Scale` property of the `Label`. For that reason, the `Slider` is set for a range of -2 to 2.

The code-behind file sets the binding with the `SetBinding` method, with the second argument being a constructor for the `Binding` class:

C#

```
public partial class AlternativeCodeBindingPage : ContentPage
{
    public AlternativeCodeBindingPage()
    {
        InitializeComponent();

        label.SetBinding(Label.ScaleProperty, new Binding("Value", source:
slider));
    }
}
```

The `Binding` constructor has 6 parameters, so the `source` parameter is specified with a named argument. The argument is the `slider` object.

📌 Note

The `VisualElement` class also defines `ScaleX` and `ScaleY` properties, which can scale the `VisualElement` differently in the horizontal and vertical directions.

Alternatively, the data binding can be specified in XAML:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="DataBindingDemos.AlternativeXamlBindingPage"
              Title="Alternative XAML Binding">
    <StackLayout Padding="10, 0">
        <Label Text="TEXT"
               FontSize="40"
               HorizontalOptions="Center"
               VerticalOptions="Center"
               Scale="{Binding Source={x:Reference slider},
                             Path=Value}" />

        <Slider x:Name="slider"
                Minimum="-2"
                Maximum="2"
                VerticalOptions="Center" />
    </StackLayout>
</ContentPage>
```

In this example, the `Binding` markup extension has two properties set, `Source` and `Path`, separated by a comma. The `Source` property is set to an embedded `x:Reference` markup extension that otherwise has the same syntax as setting the `BindingContext`.

The content property of the `Binding` markup extension is `Path`, but the `Path=` part of the markup extension can only be eliminated if it is the first property in the expression. To eliminate the `Path=` part, you need to swap the two properties:

XAML

```
Scale="{Binding Value, Source={x:Reference slider}}" />
```

Although XAML markup extensions are usually delimited by curly braces, they can also be expressed as object elements:

XAML

```
<Label Text="TEXT"
       FontSize="40"
       HorizontalOptions="Center"
       VerticalOptions="Center">
```

```

<Label.Scale>
  <Binding Source="{x:Reference slider}"
    Path="Value" />
</Label.Scale>
</Label>

```

In this example, the `Source` and `Path` properties are regular XAML attributes. The values appear within quotation marks and the attributes are not separated by a comma. The `x:Reference` markup extension can also become an object element:

XAML

```

<Label Text="TEXT"
  FontSize="40"
  HorizontalOptions="Center"
  VerticalOptions="Center">
  <Label.Scale>
    <Binding Path="Value">
      <Binding.Source>
        <x:Reference Name="slider" />
      </Binding.Source>
    </Binding>
  </Label.Scale>
</Label>

```

This syntax isn't common, but sometimes it's necessary when complex objects are involved.

The examples shown so far set the `BindingContext` property and the `Source` property of `Binding` to an `x:Reference` markup extension to reference another view on the page. These two properties are of type `Object`, and they can be set to any object that includes properties that are suitable for binding sources. You can also set the `BindingContext` or `Source` property to an `x:Static` markup extension to reference the value of a static property or field, or a `StaticResource` markup extension to reference an object stored in a resource dictionary, or directly to an object, which is often an instance of a viewmodel.

ⓘ Note

The `BindingContext` property can also be set to a `Binding` object so that the `Source` and `Path` properties of `Binding` define the binding context.

Binding context inheritance

You can specify the source object using the `BindingContext` property or the `Source` property of the `Binding` object. If both are set, the `Source` property of the `Binding` takes precedence over the `BindingContext`.

❗ Important

The `BindingContext` property value is inherited through the visual tree.

The following XAML example demonstrates binding context inheritance:

XAML

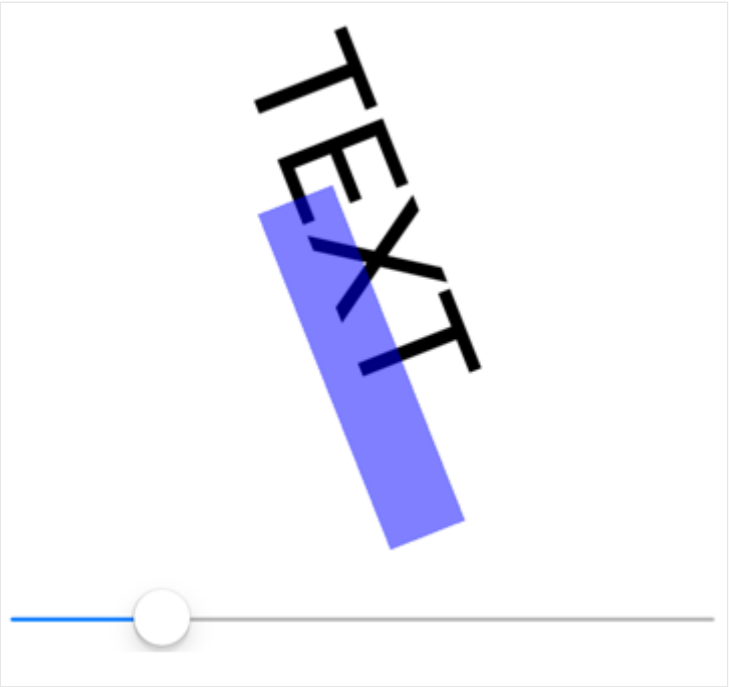
```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="DataBindingDemos.BindingContextInheritancePage"
             Title="BindingContext Inheritance">
    <StackLayout Padding="10">
        <StackLayout VerticalOptions="Fill"
                     BindingContext="{x:Reference slider}">

            <Label Text="TEXT"
                   FontSize="80"
                   HorizontalOptions="Center"
                   VerticalOptions="End"
                   Rotation="{Binding Value}" />

            <BoxView Color="#800000FF"
                     WidthRequest="180"
                     HeightRequest="40"
                     HorizontalOptions="Center"
                     VerticalOptions="Start"
                     Rotation="{Binding Value}" />
        </StackLayout>

        <Slider x:Name="slider"
                 Maximum="360" />
    </StackLayout>
</ContentPage>
```

In this example, the `BindingContext` property of the `StackLayout` is set to the `slider` object. This binding context is inherited by both the `Label` and the `BoxView`, both of which have their `Rotation` properties set to the `Value` property of the `Slider`:



Binding mode

Article • 07/25/2023

 [Browse the sample](#)

Every .NET Multi-platform App UI (.NET MAUI) bindable property has a default binding mode that is set when the bindable property is created, and which is available from the `DefaultBindingMode` property of the `BindableProperty` object. This default binding mode indicates the mode in effect when that property is a data-binding target. The default binding mode for most properties such as `Rotation`, `Scale`, and `Opacity` is `OneWay`. When these properties are data-binding targets, then the target property is set from the source.

The following example shows a data binding defined on a [Slider](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="DataBindingDemos.ReverseBindingPage"
             Title="Reverse Binding">
    <StackLayout Padding="10, 0">
        <Label x:Name="label"
               Text="TEXT"
               FontSize="80"
               HorizontalOptions="Center"
               VerticalOptions="Center" />
        <Slider x:Name="slider"
                VerticalOptions="Center"
                Value="{Binding Source={x:Reference label},
                             Path=Opacity}" />
    </StackLayout>
</ContentPage>
```

In this example, the [Label](#) is the data-binding source, and the [Slider](#) is the target. The binding references the `opacity` property of the [Label](#), which has a default value of 1. Therefore, the [Slider](#) is initialized to the value 1 from the initial `opacity` value of [Label](#). This is shown in the following screenshot:



In addition, the [Slider](#) continues to work. This is because the default binding mode for the `Value` property of [Slider](#) is `TwoWay`. This means that when the `Value` property is a data-binding target, then the target is set from the source but the source is also set from the target. This allows the [Slider](#) to be set from the initial `Opacity` value.

ⓘ Note

Bindable properties don't signal a property change unless the property actually changes. This prevents an infinite loop.

If the default binding mode on the target property is not suitable for a particular data binding, it's possible to override it by setting the `Mode` property of `Binding` (or the `Mode` property of the `Binding` markup extension) to one of the members of the `BindingMode` enumeration:

- `Default`
- `TwoWay` — data goes both ways between source and target
- `OneWay` — data goes from source to target
- `OneWayToSource` — data goes from target to source
- `OneTime` — data goes from source to target, but only when the `BindingContext` changes

Two-way bindings

Most bindable properties have a default binding mode of `OneWay` but some properties have a default binding mode of `TwoWay`, including the following:

- `Date` property of [DatePicker](#)
- `Text` property of [Editor](#), [Entry](#), [SearchBar](#), and [EntryCell](#)
- `IsRefreshing` property of [ListView](#)
- `SelectedItem` property of `MultiPage`
- `SelectedIndex` and `SelectedItem` properties of [Picker](#)
- `Value` property of [Slider](#) and [Stepper](#)
- `IsToggled` property of [Switch](#)
- `On` property of [SwitchCell](#)
- `Time` property of [TimePicker](#)

These properties are defined as `TwoWay` because when data bindings are used with the Model-View-ViewModel (MVVM) pattern, the viewmodel class is the data-binding

source, and the view, which consists of views such as [Slider](#), are data-binding targets. MVVM bindings resemble the example above, because it's likely that you want each view on the page to be initialized with the value of the corresponding property in the viewmodel, but changes in the view should also affect the viewmodel property.

One-way-to-source bindings

Read-only bindable properties have a default binding mode of `OneWayToSource`. For example, the `SelectedItem` property of [ListView](#) has a default binding mode of `OneWayToSource`. This is because a binding on the `SelectedItem` property should result in setting the binding source.

One-time bindings

Target properties with a binding mode of `OneTime` are updated only when the binding context changes. For bindings on these target properties, this simplifies the binding infrastructure because it is not necessary to monitor changes in the source properties.

Several properties have a default binding mode of `OneTime`, including the `IsTextPredictionEnabled` property of [Entry](#).

Viewmodels and property-change notifications

When using a viewmodel in a data-binding, the viewmodel is the data-binding source. The viewmodel doesn't define bindable properties, but it does implement a notification mechanism that allows the binding infrastructure to be notified when the value of a property changes. This notification mechanism is the [INotifyPropertyChanged](#) interface, which defines a single event named [PropertyChanged](#). A class that implements this interface typically fires the event when one of its public properties changes value. The event does not need to be raised if the property never changes. The `INotifyPropertyChanged` interface is also implemented by [BindableObject](#) and a `PropertyChanged` event is raised whenever a bindable property changes value.

In the following example, data bindings allow you to select a color using three [Slider](#) elements for the hue, saturation, and luminosity:

C#

```
public class HslColorViewModel : INotifyPropertyChanged
{
    Color color;
```

```
string name;
float hue;
float saturation;
float luminosity;

public event PropertyChangedEventHandler PropertyChanged;

public float Hue
{
    get
    {
        return hue;
    }
    set
    {
        if (hue != value)
        {
            Color = Color.FromHsla(value, saturation, luminosity);
        }
    }
}

public float Saturation
{
    get
    {
        return saturation;
    }
    set
    {
        if (saturation != value)
        {
            Color = Color.FromHsla(hue, value, luminosity);
        }
    }
}

public float Luminosity
{
    get
    {
        return luminosity;
    }
    set
    {
        if (luminosity != value)
        {
            Color = Color.FromHsla(hue, saturation, value);
        }
    }
}

public Color Color
{
    get
```

```

    {
        return color;
    }
    set
    {
        if (color != value)
        {
            color = value;
            hue = color.GetHue();
            saturation = color.GetSaturation();
            luminosity = color.GetLuminosity();
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Hue"));
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Saturation"));
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Luminosity"));
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Color"));

            Name = NamedColor.GetNearestColorName(color);
        }
    }
}

public string Name
{
    get
    {
        return name;
    }
    private set
    {
        if (name != value)
        {
            name = value;
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Name"));
        }
    }
}
}

```

In this example, the `HslColorViewModel` class defines `Hue`, `Saturation`, `Luminosity`, `Color`, and `Name` properties. When any one of the three color components changes value, the `Color` property is recalculated, and `PropertyChanged` events are raised for all four properties. When the `Color` property changes, the static `GetNearestColorName` method in the `NamedColor` class obtains the closest named color and sets the `Name` property.

When a viewmodel is set as a binding source, the binding infrastructure attaches a handler to the `PropertyChanged` event. In this way, the binding can be notified of changes to properties, and can then set the target properties from the changed values. However, when a target property (or the `Binding` definition on a target property) has a `BindingMode` of `OneTime`, it is not necessary for the binding infrastructure to attach a handler on the `PropertyChanged` event. The target property is updated only when the `BindingContext` changes and not when the source property itself changes.

The following XAML consumes the `HslColorViewModel`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:DataBindingDemos"
             x:Class="DataBindingDemos.SimpleColorSelectorPage">
    <ContentPage.BindingContext>
        <local:HslColorViewModel Color="MediumTurquoise" />
    </ContentPage.BindingContext>

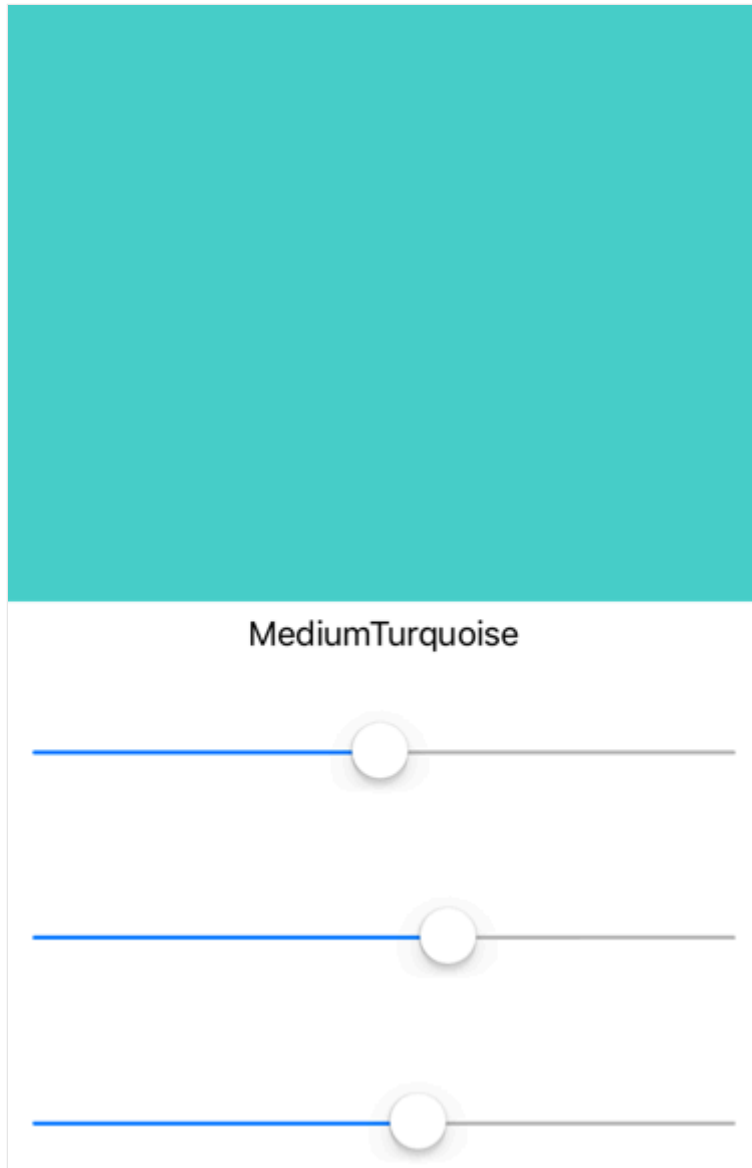
    <ContentPage.Resources>
        <Style TargetType="Slider">
            <Setter Property="VerticalOptions" Value="CenterAndExpand" />
        </Style>
    </ContentPage.Resources>

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="*" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <BoxView Color="{Binding Color}"
                Grid.Row="0" />
        <StackLayout Grid.Row="1"
                    Margin="10, 0">
            <Label Text="{Binding Name}"
                   HorizontalTextAlignment="Center" />
            <Slider Value="{Binding Hue}" />
            <Slider Value="{Binding Saturation}" />
            <Slider Value="{Binding Luminosity}" />
        </StackLayout>
    </Grid>
</ContentPage>
```

In this example, the `HslColorViewModel` is instantiated, and `Color` property set, and set as the page's `BindingContext`. The `BoxView`, `Label`, and three `Slider` views inherit the binding context from the `ContentPage`. These views are all binding targets that reference source properties in the viewmodel. For the `Color` property of the `BoxView`,

and the `Text` property of the `Label`, the data bindings are `OneWay` - the properties in the view are set from the properties in the viewmodel. The `Value` property of the `Slider`, however, uses a `TwoWay` binding mode. This enables each `Slider` to be set from the viewmodel, and also for the viewmodel to be set from each `Slider`.

When the example is first run, the `BoxView`, `Label`, and three `Slider` elements are all set from the viewmodel based on the initial `Color` property set when the viewmodel was instantiated:



As you manipulate the sliders, the `BoxView` and `Label` are updated accordingly.

Overriding the binding mode

The binding mode for a target property can be overridden by setting the `Mode` property of `Binding` (or the `Mode` property of the `Binding` markup extension) to one of the members of the `BindingMode` enumeration.

However, setting the `Mode` property doesn't always produce the expected result. For example, in the following example setting the `Mode` property to `TwoWay` doesn't work as you might expect:

XAML

```
<Label Text="TEXT"
        FontSize="40"
        HorizontalOptions="Center"
        VerticalOptions="CenterAndExpand"
        Scale="{Binding Source={x:Reference slider},
                    Path=Value,
                    Mode=TwoWay}" />
```

In this example, it might be expected that the `Slider` would be initialized to the initial value of the `Scale` property, which is 1, but that doesn't happen. When a `TwoWay` binding is initialized, the target is set from the source first, which means that the `Scale` property is set to the `Slider` default value of 0. When the `TwoWay` binding is set on the `Slider`, then the `Slider` is initially set from the source.

Alternatively, you can set the binding mode to `OneWayToSource`:

XAML

```
<Label Text="TEXT"
        FontSize="40"
        HorizontalOptions="Center"
        VerticalOptions="CenterAndExpand"
        Scale="{Binding Source={x:Reference slider},
                    Path=Value,
                    Mode=OneWayToSource}" />
```

Now the `Slider` is initialized to 1 (the default value of `Scale`) but manipulating the `Slider` doesn't affect the `Scale` property.

❗ Note

The `VisualElement` class also defines `ScaleX` and `ScaleY` properties, which can scale the `VisualElement` differently in the horizontal and vertical directions.

A very useful application of overriding the default binding mode with a `TwoWay` binding mode involves the `SelectedItem` property of `ListView`. The default binding mode is `OneWayToSource`. When a data binding is set on the `SelectedItem` property to reference a source property in a viewmodel, then that source property is set from the `ListView`

selection. However, in some circumstances, you might also want the [ListView](#) to be initialized from the viewmodel.

❗ Important

The default binding mode can vary from control to control, and is set when the bindable property is created. It's available from the `DefaultBindingMode` property of the [BindableProperty](#) object.

String formatting

Article • 12/23/2022

 [Browse the sample](#)

In a .NET Multi-platform App UI (.NET MAUI) app, it's sometimes convenient to use data bindings to display the string representation of an object or value. For example, you might want to use a [Label](#) to display the current value of a [Slider](#). In this data binding, the [Slider](#) is the source, and the target is the `Text` property of the [Label](#).

String formatting in code is typically accomplished with the static [String.Format](#) method. The formatting string includes formatting codes specific to various types of objects, and you can include other text along with the values being formatted. For more information, see [Formatting Types in .NET](#) for more information on string formatting.

String formatting can also be accomplished with data bindings by setting the `StringFormat` property of `Binding` (or the `StringFormat` property of the `Binding` markup extension) to a standard .NET formatting string with a placeholder:

XAML

```
<Slider x:Name="slider" />
<Label Text="{Binding Source={x:Reference slider},
                    Path=Value,
                    StringFormat='The slider value is {0:F2}}'" />
```

In XAML the formatting string is delimited by single-quote characters to help the XAML parser avoid treating the curly braces as another XAML markup extension. In this example, the formatting specification of `F2` causes the value to be displayed with two decimal places.

Note

Using the `StringFormat` property only makes sense when the target property is of type `string`, and the binding mode is `OneWay` or `TwoWay`. For two-way bindings, the `StringFormat` is only applicable for values passing from the source to the target.

The following example demonstrates several uses of the `StringFormat` property:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:sys="clr-namespace:System;assembly=netstandard"
             x:Class="DataBindingDemos.StringFormattingPage"
             Title="String Formatting">

    <ContentPage.Resources>
        <Style TargetType="Label">
            <Setter Property="HorizontalTextAlignment" Value="Center" />
        </Style>
        <Style TargetType="BoxView">
            <Setter Property="Color" Value="Blue" />
            <Setter Property="HeightRequest" Value="2" />
            <Setter Property="Margin" Value="0, 5" />
        </Style>
    </ContentPage.Resources>

    <StackLayout Margin="10">
        <Slider x:Name="slider" />
        <Label Text="{Binding Source={x:Reference slider},
                        Path=Value,
                        StringFormat='The slider value is {0:F2}}'" />

        <BoxView />
        <TimePicker x:Name="timePicker" />
        <Label Text="{Binding Source={x:Reference timePicker},
                        Path=Time,
                        StringFormat='The TimeSpan is {0:c}}'" />

        <BoxView />
        <Entry x:Name="entry" />
        <Label Text="{Binding Source={x:Reference entry},
                        Path=Text,
                        StringFormat='The Entry text is &quot;
{0}&quot;}'" />
        <BoxView />
        <StackLayout BindingContext="{x:Static sys:DateTime.Now}">
            <Label Text="{Binding}" />
            <Label Text="{Binding Path=Ticks,
                        StringFormat='{0:N0} ticks since 1/1/1}'"
            />

            <Label Text="{Binding StringFormat='The {{0:MMMM}} specifier
produces {0:MMMM}}'" />
            <Label Text="{Binding StringFormat='The long date is {0:D}}'" />
        </StackLayout>
        <BoxView />
        <StackLayout BindingContext="{x:Static sys:Math.PI}">
            <Label Text="{Binding}" />
            <Label Text="{Binding StringFormat='PI to 4 decimal points =
{0:F4}}'" />
            <Label Text="{Binding StringFormat='PI in scientific notation =
{0:E7}}'" />
        </StackLayout>
    </StackLayout>
</ContentPage>

```

In this example, the bindings on the [Slider](#) and [TimePicker](#) show the use of format specifications particular to `double` and `TimeSpan` data types. The `StringFormat` that displays the text from the [Entry](#) view demonstrates how to specify double quotation marks in the formatting string with the use of the `"` HTML entity.

The next section in the XAML file is a [StackLayout](#) with a `BindingContext` set to an `x:Static` markup extension that references the static `DateTime.Now` property. The first binding has no properties:

XAML

```
<Label Text="{Binding}" />
```

This simply displays the `DateTime` value of the `BindingContext` with default formatting. The second binding displays the `Ticks` property of `DateTime`, while the other two bindings display the `DateTime` itself with specific formatting.

ⓘ Note

If you need to display left or right curly braces in your formatting string, use a pair of them. For example, `StringFormat='{ {0:MMMM} }'`.

The last section sets the `BindingContext` to the value of `Math.PI` and displays it with default formatting and two different types of numeric formatting:

The slider value is 0.24

9:41 PM

The TimeSpan is 21:41:00

Some text

The Entry text is "Some text"

12/15/2017 11:55:17

636,489,357,176,684,200 ticks since 1/1/1

The {0:MMMM} specifier produces December

The long date is Friday, December 15, 2017

3.14159265358979

PI to 4 decimal points = 3.1416

PI in scientific notation = 3.1415927E+000

ViewModels and string formatting

When you're using [Label](#) and `StringFormat` to display the value of a view that is also the target of a viewmodel, you can either define the binding from the view to the [Label](#) or from the viewmodel to the [Label](#). In general, the second approach is best because it verifies that the bindings between the view and viewmodel are working.

This approach is shown in the following example:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:DataBindingDemos"
  x:Class="DataBindingDemos.BetterColorSelectorPage"
  Title="Better Color Selector">
  <ContentPage.BindingContext>
    <local:HslColorViewModel Color="Sienna" />
  </ContentPage.BindingContext>

  <ContentPage.Resources>
    <ResourceDictionary>
      <Style TargetType="Slider">
        <Setter Property="VerticalOptions" Value="Center" />
      </Style>
    </ResourceDictionary>
  </ContentPage.Resources>
</ContentPage>
```

```

        <Style TargetType="Label">
            <Setter Property="HorizontalTextAlignment" Value="Center" />
        </Style>
    </ResourceDictionary>
</ContentPage.Resources>

<StackLayout Margin="20">
    <BoxView Color="{Binding Color}"
        HeightRequest="100"
        WidthRequest="100"
        HorizontalOptions="Center" />
    <StackLayout Margin="10, 0">
        <Label Text="{Binding Name}" />
        <Slider Value="{Binding Hue}" />
        <Label Text="{Binding Hue, StringFormat='Hue = {0:F2}}'" />
        <Slider Value="{Binding Saturation}" />
        <Label Text="{Binding Saturation, StringFormat='Saturation =
{0:F2}}'" />
        <Slider Value="{Binding Luminosity}" />
        <Label Text="{Binding Luminosity, StringFormat='Luminosity =
{0:F2}}'" />
    </StackLayout>
</StackLayout>
</ContentPage>

```

In this example, there are three pairs of **Slider** and **Label** elements that are bound to the same source property in the `HslColorViewModel` object. Each **Label** that accompanies a **Slider** has a `StringFormat` property to display each **Slider** value:



Sienna



Hue = 0.05



Saturation = 0.56



Luminosity = 0.40

Binding path

Article • 12/23/2022

 [Browse the sample](#)

In .NET Multi-platform App UI (.NET MAUI), the `Path` property of the `Binding` class (or the `Path` property of the `Binding` markup extension) can be set to a single property, to a *sub-property* (a property of a property), or to a member of a collection.

For example, suppose a page contains a [TimePicker](#):

XAML

```
<TimePicker x:Name="timePicker">
```

The `Time` property of [TimePicker](#) is of type `TimeSpan`, and it has a `TotalSeconds` property. A data binding can be created that references the `TotalSeconds` property of that `TimeSpan` value:

XAML

```
{Binding Source={x:Reference timePicker},  
    Path=Time.TotalSeconds}
```

The `Time` and `TotalSeconds` properties are simply connected with a period.

ⓘ Note

The items in the `Path` string always refer to properties and not to the types of these properties.

The following XAML shows multiple examples of binding to sub-properties:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
    xmlns:globe="clr-  
namespace:System.Globalization;assembly=netstandard"  
    x:Class="DataBindingDemos.PathVariationsPage"  
    Title="Path Variations"  
    x:Name="page">  
    <ContentPage.Resources>
```

```

<Style TargetType="Label">
    <Setter Property="FontSize" Value="18" />
    <Setter Property="HorizontalTextAlignment" Value="Center" />
    <Setter Property="VerticalOptions" Value="Center" />
</Style>
</ContentPage.Resources>

<StackLayout Margin="10, 0">
    <TimePicker x:Name="timePicker" />
    <Label Text="{Binding Source={x:Reference timePicker},
        Path=Time.TotalSeconds,
        StringFormat='{0} total seconds'}" />
    <Label Text="{Binding Source={x:Reference page},
        Path=Content.Children.Count,
        StringFormat='There are {0} children in this
StackLayout'}" />
    <Label Text="{Binding Source={x:Static
globe:CultureInfo.CurrentCulture},
        Path=DateTimeFormat.DayNames[3],
        StringFormat='The middle day of the week is
{0}'}" />
    <Label>
        <Label.Text>
            <Binding Path="DateTimeFormat.DayNames[3]"
                StringFormat="The middle day of the week in France
is {0}">
                <Binding.Source>
                    <globe:CultureInfo>
                        <x:Arguments>
                            <x:String>fr-FR</x:String>
                        </x:Arguments>
                    </globe:CultureInfo>
                </Binding.Source>
            </Binding>
        </Label.Text>
    </Label>
    <Label Text="{Binding Source={x:Reference page},
        Path=Content.Children[1].Text.Length,
        StringFormat='The second Label has {0}
characters'}" />
</StackLayout>
</ContentPage>

```

In the second `Label`, the binding source is the page itself. The `Content` property is of type `StackLayout`, which has a `Children` property of type `IList<View>`, which has a `Count` property indicating the number of children.

Paths with indexers

In the example above, the binding in the third [Label](#) references the [CultureInfo](#) class in the `System.Globalization` namespace:

XAML

```
<Label Text="{Binding Source={x:Static globe:CultureInfo.CurrentCulture},
    Path=DateTimeFormat.DayNames[3],
    StringFormat='The middle day of the week is {0}'}" />
```

The source is set to the static `CultureInfo.CurrentCulture` property, which is an object of type `CultureInfo`. That class defines a property named `DateTimeFormat` of type [DateTimeFormatInfo](#) that contains a `DayNames` collection. The index selects the fourth item.

The fourth [Label](#) does something similar but for the culture associated with France. The `Source` property of the binding is set to `CultureInfo` object with a constructor:

XAML

```
<Label>
    <Label.Text>
        <Binding Path="DateTimeFormat.DayNames[3]"
            StringFormat="The middle day of the week in France is {0}">
            <Binding.Source>
                <globe:CultureInfo>
                    <x:Arguments>
                        <x:String>fr-FR</x:String>
                    </x:Arguments>
                </globe:CultureInfo>
            </Binding.Source>
        </Binding>
    </Label.Text>
</Label>
```

For more information about specifying constructor arguments in XAML, see [Pass constructor arguments](#).

The last [Label](#) is similar to the second, except that it references one of the children of the [StackLayout](#):

XAML

```
<Label Text="{Binding Source={x:Reference page},
    Path=Content.Children[1].Text.Length,
    StringFormat='The first Label has {0} characters'}" />
```

That child is a [Label](#), which has a `Text` property of type `String`, which has a `Length` property. The first [Label](#) reports the `TimeSpan` set in the [TimePicker](#), so when that text changes, the final [Label](#) changes as well:

Select a time:

12:16 AM

960 total seconds

There are 7 children in this
StackLayout

The middle day of the week
is Wednesday

The middle day of the week in
French is mercredi

The second Label has
17 characters

Debug complex paths

Complex path definitions can be difficult to construct. You need to know the type of each sub-property or the type of items in the collection to correctly add the next sub-property, but the types themselves do not appear in the path. One technique is to build up the path incrementally and look at the intermediate results. For that last example, you could start with no `Path` definition at all:

XAML

```
<Label Text="{Binding Source={x:Reference page},  
                StringFormat='{0}'}" />
```

That displays the type of the binding source, or `DataBindingDemos.PathVariationsPage`. You know `PathVariationsPage` derives from `ContentPage`, so it has a `Content` property:

XAML

```
<Label Text="{Binding Source={x:Reference page},  
                Path=Content,  
                StringFormat='{0}'}" />
```

The type of the `Content` property is now revealed to be `Microsoft.Maui.Controls.StackLayout`. Add the `Children` property to the `Path` and the type is also `Microsoft.Maui.Controls.StackLayout`. Add an index to that and the type is `Microsoft.Maui.Controls.Label`. Continue in this way.

As .NET MAUI processes the binding path, it installs a `PropertyChanged` handler on any object in the path that implements the `INotifyPropertyChanged` interface. For example, the final binding reacts to a change in the first `Label` because the `Text` property changes. If a property in the binding path does not implement `INotifyPropertyChanged`, any changes to that property will be ignored. Some changes could entirely invalidate the binding path, so you should use this technique only when the string of properties and sub-properties never become invalid.

Binding value converters

Article • 04/03/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) data bindings usually transfer data from a source property to a target property, and in some cases from the target property to the source property. This transfer is straightforward when the source and target properties are of the same type, or when one type can be converted to the other type through an implicit conversion. When that is not the case, a type conversion must take place.

In the [String formatting](#) article, you saw how you can use the `StringFormat` property of a data binding to convert any type into a string. For other types of conversions, you need to write some specialized code in a class that implements the [IValueConverter](#) interface. Classes that implement [IValueConverter](#) are called *value converters*, but they are also often referred to as *binding converters* or *binding value converters*.

Binding value converters

Suppose you want to define a data binding where the source property is of type `int` but the target property is a `bool`. You want this data binding to produce a `false` value when the integer source is equal to 0, and `true` otherwise. This can be achieved with a class that implements the [IValueConverter](#) interface:

C#

```
public class IntToBoolConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
        CultureInfo culture)
    {
        return (int)value != 0;
    }

    public object ConvertBack(object value, Type targetType, object
        parameter, CultureInfo culture)
    {
        return (bool)value ? 1 : 0;
    }
}
```

You then set an instance of this class to the `Converter` property of the `Binding` class or to the `Converter` property of the `Binding` markup extension. This class becomes part of

the data binding.

The `Convert` method is called when data moves from the source to the target in `OneWay` or `TwoWay` bindings. The `value` parameter is the object or value from the data-binding source. The method must return a value of the type of the data-binding target. The method shown here casts the `value` parameter to an `int` and then compares it with 0 for a `bool` return value.

The `ConvertBack` method is called when data moves from the target to the source in `TwoWay` or `OneWayToSource` bindings. `ConvertBack` performs the opposite conversion: It assumes the `value` parameter is a `bool` from the target, and converts it to an `int` return value for the source.

⚠ Note

If a data binding also includes a `StringFormat` setting, the value converter is invoked before the result is formatted as a string.

The following example demonstrates how to use this value converter in a data binding:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:DataBindingDemos"
             x:Class="DataBindingDemos.EnableButtonsPage"
             Title="Enable Buttons">
    <ContentPage.Resources>
        <local:IntToBoolConverter x:Key="intToBool" />
    </ContentPage.Resources>

    <StackLayout Padding="10, 0">
        <Entry x:Name="entry1"
              Text=""
              Placeholder="enter search term"
              VerticalOptions="Center" />
        <Button Text="Search"
              HorizontalOptions="Center"
              VerticalOptions="Center"
              IsEnabled="{Binding Source={x:Reference entry1},
                                Path=Text.Length,
                                Converter={StaticResource intToBool}}"/>
    </StackLayout>

    <Entry x:Name="entry2"
          Text=""
          Placeholder="enter destination"
          VerticalOptions="Center" />
    <Button Text="Submit" />
</ContentPage>
```



```

HorizontalOptions="Center"
VerticalOptions="Center"
IsEnabled="{Binding Source={x:Reference entry2},
                  Path=Text.Length,
                  Converter={StaticResource intToBool}}"
/>
</StackLayout>
</ContentPage>

```

In this example, the `IntToBoolConverter` is instantiated in the page's resource dictionary. It's then referenced with a `StaticResource` markup extension to set the `Converter` property in two data bindings. It is very common to share data converters among multiple data bindings on the page. If a value converter is used in multiple pages of your application, you can instantiate it in the application-level resource dictionary.

This example demonstrates a common need when a `Button` performs an operation based on text that the user types into an `Entry` view. The `Text` property of each `Entry` is initialized to an empty string, because the `Text` property is `null` by default, and the data binding will not work in that case. If nothing has been typed into the `Entry`, the `Button` should be disabled. Each `Button` contains a data binding on its `IsEnabled` property. The data-binding source is the `Length` property of the `Text` property of the corresponding `Entry`. If that `Length` property is not 0, the value converter returns `true` and the `Button` is enabled:

The image shows a mobile application interface with two search forms. The top form has a text input field with the placeholder text "enter search term" and a disabled "SEARCH" button. The bottom form has a text input field containing the text "microsoft.com" and an enabled "SUBMIT" button.

ⓘ Note

If you know that a value converter will only be used in `OneWay` bindings, then the `ConvertBack` method can simply return `null`.

The `Convert` method shown above assumes that the `value` argument is of type `int` and the return value must be of type `bool`. Similarly, the `ConvertBack` method assumes that the `value` argument is of type `bool` and the return value is `int`. If that is not the case, a runtime exception will occur.

You can write value converters to be more generalized and to accept several different types of data. The `Convert` and `ConvertBack` methods can use the `as` or `is` operators with the `value` parameter, or can call `GetType` on that parameter to determine its type, and then do something appropriate. The expected type of each method's return value is given by the `targetType` parameter. Sometimes, value converters are used with data bindings of different target types. In this case the value converter can use the `targetType` argument to perform a conversion for the correct type.

If the conversion being performed is different for different cultures, use the `culture` parameter for this purpose.

Binding converter properties

Value converter classes can have properties and generic parameters. The following value converter converts a `bool` from the source to an object of type `T` for the target:

C#

```
public class BoolToObjectConverter<T> : IValueConverter
{
    public T TrueObject { get; set; }
    public T FalseObject { get; set; }

    public object Convert(object value, Type targetType, object parameter,
        CultureInfo culture)
    {
        return (bool)value ? TrueObject : FalseObject;
    }

    public object ConvertBack(object value, Type targetType, object
        parameter, CultureInfo culture)
    {
        return ((T)value).Equals(TrueObject);
    }
}
```

The following example demonstrates how this converter can be used to display the value of a `Switch` view. Although it's common to instantiate value converters as resources in a resource dictionary, this example demonstrates an alternative. Here, each value converter is instantiated between `Binding.Converter` property-element tags. The

`x:TypeArguments` indicates the generic argument, and `TrueObject` and `FalseObject` are both set to objects of that type:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:DataBindingDemos"
    x:Class="DataBindingDemos.SwitchIndicatorsPage"
    Title="Switch Indicators">
    <ContentPage.Resources>
        <Style TargetType="Label">
            <Setter Property="FontSize" Value="18" />
            <Setter Property="VerticalOptions" Value="Center" />
        </Style>

        <Style TargetType="Switch">
            <Setter Property="VerticalOptions" Value="Center" />
        </Style>
    </ContentPage.Resources>

    <StackLayout Padding="10, 0">
        <StackLayout Orientation="Horizontal"
            VerticalOptions="Center">
            <Label Text="Subscribe?" />
            <Switch x:Name="switch1" />
            <Label>
                <Label.Text>
                    <Binding Source="{x:Reference switch1}"
                        Path="IsToggled">
                        <Binding.Converter>
                            <local:BoolToObjectConverter
                                x:TypeArguments="x:String"
                                TrueObject="Of
course!"
                                FalseObject="No
way!" />
                        </Binding.Converter>
                    </Binding>
                </Label.Text>
            </Label>
        </StackLayout>

        <StackLayout Orientation="Horizontal"
            VerticalOptions="Center">
            <Label Text="Allow popups?" />
            <Switch x:Name="switch2" />
            <Label>
                <Label.Text>
                    <Binding Source="{x:Reference switch2}"
                        Path="IsToggled">
                        <Binding.Converter>
                            <local:BoolToObjectConverter
                                x:TypeArguments="x:String"
                                TrueObject="Of
course!"
                                FalseObject="No
way!" />
                        </Binding.Converter>
                    </Binding>
                </Label.Text>
            </Label>
        </StackLayout>
    </StackLayout>
</ContentPage>
```

```

TrueObject="Yes"
FalseObject="No" />

        </Binding.Converter>
    </Binding>
</Label.Text>
<Label.TextColor>
    <Binding Source="{x:Reference switch2}"
        Path="IsToggled">
        <Binding.Converter>
            <local:BoolToObjectConverter
x:TypeArguments="Color"
TrueObject="Green"
FalseObject="Red"
/>
        </Binding.Converter>
    </Binding>
</Label.TextColor>
</Label>
</StackLayout>

<StackLayout Orientation="Horizontal"
    VerticalOptions="Center">
    <Label Text="Learn more?" />
    <Switch x:Name="switch3" />
    <Label FontSize="18"
        VerticalOptions="Center">
        <Label.Style>
            <Binding Source="{x:Reference switch3}"
                Path="IsToggled">
            <Binding.Converter>
                <local:BoolToObjectConverter
x:TypeArguments="Style">
                    <local:BoolToObjectConverter.TrueObject>
                        <Style TargetType="Label">
                            <Setter Property="Text"
Value="Indubitably!" />
                            <Setter Property="FontAttributes"
Value="Italic, Bold" />
                            <Setter Property="TextColor"
Value="Green" />
                        </Style>
                    </local:BoolToObjectConverter.TrueObject>
                    <local:BoolToObjectConverter.FalseObject>
                        <Style TargetType="Label">
                            <Setter Property="Text" Value="Maybe
later" />
                            <Setter Property="FontAttributes"
Value="None" />
                            <Setter Property="TextColor"
Value="Red" />
                        </Style>
                    </local:BoolToObjectConverter.FalseObject>
                </local:BoolToObjectConverter>
            </Binding.Converter>
        </local:BoolToObjectConverter>
    </Label.Style>
</Label>
</StackLayout>

```

```

        </Binding>
    </Label.Style>
</Label>
</StackLayout>
</StackLayout>
</ContentPage>

```

In this example, in the last of the three [Switch](#) and [Label](#) pairs, the generic argument is set to a [Style](#), and entire [Style](#) objects are provided for the values of `TrueObject` and `FalseObject`. These override the implicit style for [Label](#) set in the resource dictionary, so the properties in that style are explicitly assigned to the [Label](#). Toggling the [Switch](#) causes the corresponding [Label](#) to reflect the change:

Subscribe? ☒ Of course!

Allow popups? ☐ No

Learn more? ☒ *Indubitably!*

ⓘ Note

It's also possible to use triggers to implement changes in the user-interface based on other views. For more information, see [Triggers](#).

Binding converter parameters

The `Binding` class defines a `ConverterParameter` property, and the `Binding` markup extension also defines a `ConverterParameter` property. If this property is set, then the value is passed to the `Convert` and `ConvertBack` methods as the `parameter` argument. Even if the instance of the value converter is shared among several data bindings, the `ConverterParameter` can be different to perform different conversions.

The use of the `ConverterParameter` property can be demonstrated with a color-selection program. The following example shows the `RgbColorViewModel`, which has three properties of type `float` named `Red`, `Green`, and `Blue` that it uses to construct a [Color](#) value:

```
C#
```

```

public class RgbColorViewModel : INotifyPropertyChanged
{
    Color color;
    string name;

    public event PropertyChangedEventHandler PropertyChanged;

    public float Red
    {
        get { return color.Red; }
        set
        {
            if (color.Red != value)
            {
                Color = new Color(value, color.Green, color.Blue);
            }
        }
    }

    public float Green
    {
        get { return color.Green; }
        set
        {
            if (color.Green != value)
            {
                Color = new Color(color.Red, value, color.Blue);
            }
        }
    }

    public float Blue
    {
        get { return color.Blue; }
        set
        {
            if (color.Blue != value)
            {
                Color = new Color(color.Red, color.Green, value);
            }
        }
    }

    public Color Color
    {
        get { return color; }
        set
        {
            if (color != value)
            {
                color = value;
                PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Red"));
                PropertyChanged?.Invoke(this, new

```

```

PropertyChangedEventArgs("Green"));
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Blue"));
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Color"));

        Name = NamedColor.GetNearestColorName(color);
    }
}

public string Name
{
    get { return name; }
    private set
    {
        if (name != value)
        {
            name = value;
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs("Name"));
        }
    }
}
}

```

The `Red`, `Green`, and `Blue` property values can range between 0 and 1. However, you might prefer that the components be displayed as two-digit hexadecimal values. To display these as hexadecimal values in XAML, they must be multiplied by 255, converted to an integer, and then formatted with a specification of "X2" in the `StringFormat` property. Multiplying by 255 and converting to an integer can be performed by the value converter. To make the value converter as generalized as possible, the multiplication factor can be specified with the `ConverterParameter` property, which means that it enters the `Convert` and `ConvertBack` methods as the `parameter` argument:

C#

```

public class FloatToIntConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter,
CultureInfo culture)
    {
        return (int)Math.Round((float)value * GetParameter(parameter));
    }

    public object ConvertBack(object value, Type targetType, object
parameter, CultureInfo culture)
    {
        return (int)value / GetParameter(parameter);
    }
}

```

```

double GetParameter(object parameter)
{
    if (parameter is float)
        return (float)parameter;
    else if (parameter is int)
        return (int)parameter;
    else if (parameter is string)
        return float.Parse((string)parameter);

    return 1;
}

```

In this example, the `Convert` method converts from a `float` to `int` while multiplying by the `parameter` value. The `ConvertBack` method divides the integer `value` argument by `parameter` and returns a `float` result.

The type of the `parameter` argument is likely to be different depending on whether the data binding is defined in XAML or code. If the `ConverterParameter` property of `Binding` is set in code, it's likely to be set to a numeric value:

C#

```
binding.ConverterParameter = 255;
```

The `ConverterParameter` property is of type `Object`, so the C# compiler interprets the literal 255 as an integer, and sets the property to that value.

However, in XAML the `ConverterParameter` is likely to be set like this:

XAML

```

<Label Text="{Binding Red,
                Converter={StaticResource doubleToInt},
                ConverterParameter=255,
                StringFormat='Red = {0:X2}'}" />

```

While 255 looks like a number, because `ConverterParameter` is of type `Object`, the XAML parser treats 255 as a string. For this reason the value converter includes a separate `GetParameter` method that handles cases for `parameter` being of type `float`, `int`, or `string`.

The following XAML example instantiates `FloatToIntConverter` in its resource dictionary:

XAML


```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:DataBindingDemos"
              x:Class="DataBindingDemos.RgbColorSelectorPage"
              Title="RGB Color Selector">
    <ContentPage.BindingContext>
        <local:RgbColorViewModel Color="Gray" />
    </ContentPage.BindingContext>
    <ContentPage.Resources>
        <Style TargetType="Slider">
            <Setter Property="VerticalOptions" Value="Center" />
        </Style>

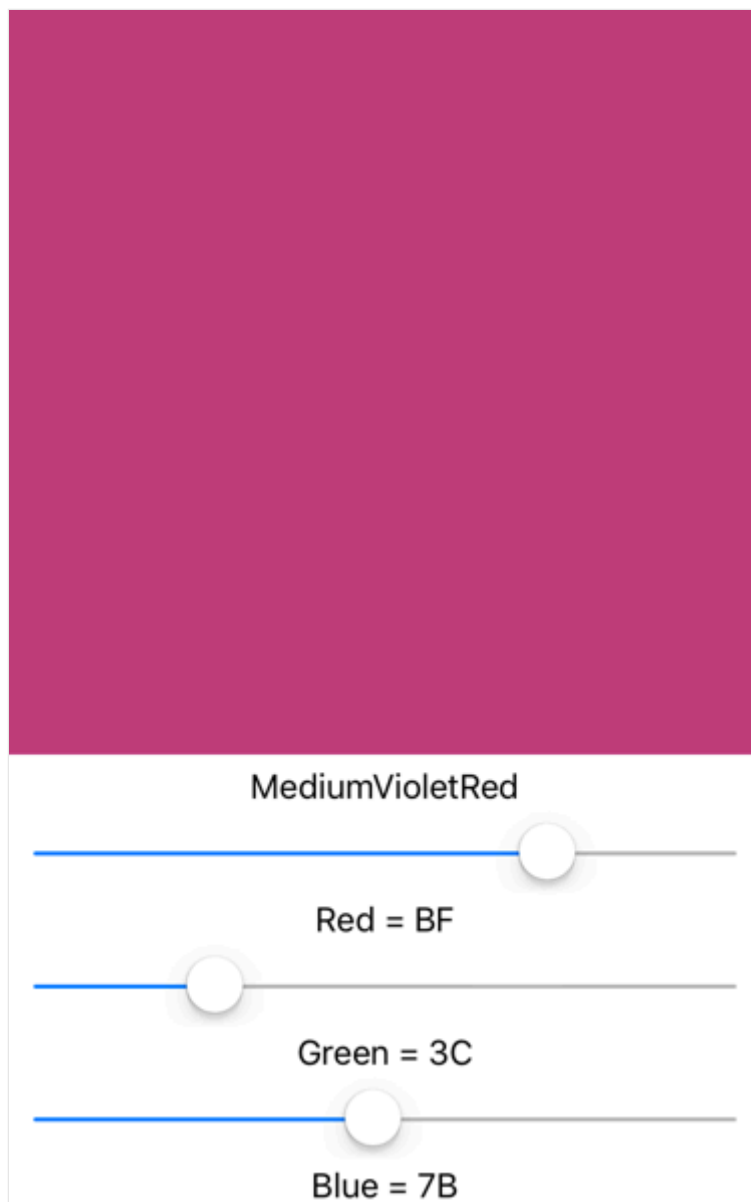
        <Style TargetType="Label">
            <Setter Property="HorizontalTextAlignment" Value="Center" />
        </Style>

        <local:FloatToIntConverter x:Key="floatToInt" />
    </ContentPage.Resources>

    <StackLayout Margin="20">
        <BoxView Color="{Binding Color}"
                HeightRequest="100"
                WidthRequest="100"
                HorizontalOptions="Center" />
        <StackLayout Margin="10, 0">
            <Label Text="{Binding Name}" />
            <Slider Value="{Binding Red}" />
            <Label Text="{Binding Red,
                        Converter={StaticResource floatToInt},
                        ConverterParameter=255,
                        StringFormat='Red = {0:X2}}'" />
            <Slider Value="{Binding Green}" />
            <Label Text="{Binding Green,
                        Converter={StaticResource floatToInt},
                        ConverterParameter=255,
                        StringFormat='Green = {0:X2}}'" />
            <Slider Value="{Binding Blue}" />
            <Label>
                <Label.Text>
                    <Binding Path="Blue"
                            StringFormat="Blue = {0:X2}"
                            Converter="{StaticResource floatToInt}">
                        <Binding.ConverterParameter>
                            <x:Single>255</x:Single>
                        </Binding.ConverterParameter>
                    </Binding>
                </Label.Text>
            </Label>
        </StackLayout>
    </StackLayout>
</ContentPage>

```

The values of the `Red` and `Green` properties are displayed with a `Binding` markup extension. The `Blue` property, however, instantiates the `Binding` class to demonstrate how an explicit `float` value can be set to `ConverterParameter` property:



Relative bindings

Article • 04/03/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) relative bindings provide the ability to set the binding source relative to the position of the binding target. They are created with the [RelativeSource](#) markup extension, and set as the `Source` property of a binding expression.

The [RelativeSource](#) markup extension is supported by the `RelativeSourceExtension` class, which defines the following properties:

- `Mode`, of type `RelativeBindingSourceMode`, describes the location of the binding source relative to the position of the binding target.
- `AncestorLevel`, of type `int`, an optional ancestor level to look for, when the `Mode` property is `FindAncestor`. An `AncestorLevel` of `n` skips `n-1` instances of the `AncestorType`.
- `AncestorType`, of type `Type`, the type of ancestor to look for, when the `Mode` property is `FindAncestor`.

ⓘ Note

The XAML parser allows the `RelativeSourceExtension` class to be abbreviated as [RelativeSource](#).

The `Mode` property should be set to one of the `RelativeBindingSourceMode` enumeration members:

- `TemplatedParent` indicates the element to which the template, in which the bound element exists, is applied. For more information, see [Bind to a templated parent](#).
- `Self` indicates the element on which the binding is being set, allowing you to bind one property of that element to another property on the same element. For more information, see [Bind to self](#).
- `FindAncestor` indicates the ancestor in the visual tree of the bound element. This mode should be used to bind to an ancestor control represented by the `AncestorType` property. For more information, see [Bind to an ancestor](#).
- `FindAncestorBindingContext` indicates the `BindingContext` of the ancestor in the visual tree of the bound element. This mode should be used to bind to the

`BindingContext` of an ancestor represented by the `AncestorType` property. For more information, see [Bind to an ancestor](#).

The `Mode` property is the content property of the `RelativeSourceExtension` class. Therefore, for XAML markup expressions expressed with curly braces, you can eliminate the `Mode=` part of the expression.

For more information about .NET MAUI markup extensions, see [Consume XAML markup extensions](#).

Bind to self

The `Self` relative binding mode is used to bind a property of an element to another property on the same element:

XAML

```
<BoxView Color="Red"
          WidthRequest="200"
          HeightRequest="{Binding Source={RelativeSource Self},
          Path=WidthRequest}"
          HorizontalOptions="Center" />
```

In this example, the `BoxView` sets its `WidthRequest` property to a fixed size, and the `HeightRequest` property binds to the `WidthRequest` property. Therefore, both properties are equal and so a square is drawn:



Important

When binding a property of an element to another property on the same element, the properties must be the same type. Alternatively, you can specify a converter on the binding to convert the value.

A common use of this binding mode is set an object's `BindingContext` to a property on itself. The following code shows an example of this:

XAML

```
<ContentPage ...
    BindingContext="{Binding Source={RelativeSource Self},
Path=DefaultViewModel}">
    <StackLayout>
        <ListView ItemsSource="{Binding Employees}">
            ...
        </ListView>
    </StackLayout>
</ContentPage>
```

In this example, the `BindingContext` of the page is set to the `DefaultViewModel` property of itself. This property is defined in the code-behind file for the page, and provides a viewmodel instance. The `ListView` binds to the `Employees` property of the viewmodel.

Bind to an ancestor

The `FindAncestor` and `FindAncestorBindingContext` relative binding modes are used to bind to parent elements, of a certain type, in the visual tree. The `FindAncestor` mode is used to bind to a parent element, which derives from the `Element` type. The `FindAncestorBindingContext` mode is used to bind to the `BindingContext` of a parent element.

⚠ Warning

The `AncestorType` property must be set to a `Type` when using the `FindAncestor` and `FindAncestorBindingContext` relative binding modes, otherwise a `XamlParseException` is thrown.

If the `Mode` property isn't explicitly set, setting the `AncestorType` property to a type that derives from `Element` will implicitly set the `Mode` property to `FindAncestor`. Similarly, setting the `AncestorType` property to a type that does not derive from `Element` will implicitly set the `Mode` property to `FindAncestorBindingContext`.

📌 Note

Relative bindings that use the `FindAncestorBindingContext` mode will be reapplied when the `BindingContext` of any ancestors change.

The following XAML shows an example where the `Mode` property will be implicitly set to `FindAncestorBindingContext`:

XAML

```
<ContentPage ...
    BindingContext="{Binding Source={RelativeSource Self},
Path=DefaultViewModel}">
    <StackLayout>
        <ListView ItemsSource="{Binding Employees}">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <ViewCell>
                        <StackLayout Orientation="Horizontal">
                            <Label Text="{Binding Fullname}"
                                VerticalOptions="Center" />
                            <Button Text="Delete"
                                Command="{Binding Source={RelativeSource
AncestorType={x:Type local:PeopleViewModel}}, Path=DeleteEmployeeCommand}"
                                CommandParameter="{Binding}"
                                HorizontalOptions="End" />
                        </StackLayout>
                    </ViewCell>
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>
    </StackLayout>
</ContentPage>
```

In this example, the `BindingContext` of the page is set to the `DefaultViewModel` property of itself. This property is defined in the code-behind file for the page, and provides a viewmodel instance. The `ListView` binds to the `Employees` property of the viewmodel. The `DataTemplate`, which defines the appearance of each item in the `ListView`, contains a `Button`. The button's `Command` property is bound to the `DeleteEmployeeCommand` in its parent's viewmodel. Tapping a `Button` deletes an employee:

John Doe	DELETE
Jane Doe	DELETE
Xamarin Monkey	DELETE

In addition, the optional `AncestorLevel` property can help disambiguate ancestor lookup in scenarios where there is possibly more than one ancestor of that type in the

visual tree:

XAML

```
<Label Text="{Binding Source={RelativeSource AncestorType={x:Type Entry},
AncestorLevel=2}, Path=Text}" />
```

In this example, the `Label.Text` property binds to the `Text` property of the second `Entry` that's encountered on the upward path, starting at the target element of the binding.

ⓘ Note

The `AncestorLevel` property should be set to 1 to find the ancestor nearest to the binding target element.

Bind to a templated parent

The `TemplatedParent` relative binding mode is used to bind from within a control template to the runtime object instance to which the template is applied (known as the templated parent). This mode is only applicable if the relative binding is within a control template, and is similar to setting a `TemplateBinding`.

The following XAML shows an example of the `TemplatedParent` relative binding mode:

XAML

```
<ContentPage ...>
    <ContentPage.Resources>
        <ControlTemplate x:Key="CardViewControlTemplate">
            <Border BindingContext="{Binding Source={RelativeSource
TemplatedParent}}">
                BackgroundColor="{Binding CardColor}"
                Stroke="{Binding BorderColor}"
                ...>
            <Grid>
                ...
                <Label Text="{Binding CardTitle}"
                    ... />
                <BoxView BackgroundColor="{Binding BorderColor}"
                    ... />
                <Label Text="{Binding CardDescription}"
                    ... />
            </Grid>
        </Border>
    </ControlTemplate>
</ContentPage.Resources>
```

```

<StackLayout>
  <controls:CardView BorderColor="DarkGray"
    CardTitle="John Doe"
    CardDescription="Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Nulla elit dolor, convallis non interdum."
    IconBackgroundColor="SlateGray"
    IconImageSource="user.png"
    ControlTemplate="{StaticResource
CardViewControlTemplate}" />
  <controls:CardView BorderColor="DarkGray"
    CardTitle="Jane Doe"
    CardDescription="Phasellus eu convallis mi. In
tempus augue eu dignissim fermentum. Morbi ut lacus vitae eros lacinia."
    IconBackgroundColor="SlateGray"
    IconImageSource="user.png"
    ControlTemplate="{StaticResource
CardViewControlTemplate}" />
  <controls:CardView BorderColor="DarkGray"
    CardTitle="Xamarin Monkey"
    CardDescription="Aliquam sagittis, odio lacinia
fermentum dictum, mi erat scelerisque erat, quis aliquet arcu."
    IconBackgroundColor="SlateGray"
    IconImageSource="user.png"
    ControlTemplate="{StaticResource
CardViewControlTemplate}" />
</StackLayout>
</ContentPage>

```

In this example, the `Border`, which is the root element of the `ControlTemplate`, has its `BindingContext` set to the runtime object instance to which the template is applied. Therefore, the `Border` and its children resolve their binding expressions against the properties of each `CardView` object:



For more information about control templates, see [Control templates](#).

Binding fallbacks

Article • 10/04/2024

 [Browse the sample](#)

Sometimes data bindings fail, because the binding source can't be resolved, or because the binding succeeds but returns a `null` value. While these scenarios can be handled with value converters, or other additional code, data bindings can be made more robust by defining fallback values to use if the binding process fails. In a .NET Multi-platform App UI (.NET MAUI) app this can be accomplished by defining the [FallbackValue](#) and [TargetNullValue](#) properties in a binding expression. Because these properties reside in the [BindingBase](#) class, they can be used with bindings, multi-bindings, compiled bindings, and with the `Binding` markup extension.

ⓘ Note

Use of the [FallbackValue](#) and [TargetNullValue](#) properties in a binding expression is optional.

Define a fallback value

The [FallbackValue](#) property allows a fallback value to be defined that will be used when the binding *source* can't be resolved. A common scenario for setting this property is when binding to source properties that might not exist on all objects in a bound collection of heterogeneous types.

The following example demonstrates setting the [FallbackValue](#) property:

XAML

```
<Label Text="{Binding Population, FallbackValue='Population size unknown'}"  
... />
```

The binding on the [Label](#) defines a [FallbackValue](#) value (delimited by single-quote characters) that will be set on the target if the binding source can't be resolved. Therefore, the value defined by the [FallbackValue](#) property will be displayed if the `Population` property doesn't exist on the bound object.

Rather than defining [FallbackValue](#) property values inline, it's recommended to define them as resources in a [ResourceDictionary](#). The advantage of this approach is that such

values are defined once in a single location, and are more easily localizable. The resources can then be retrieved using the [StaticResource](#) markup extension:

XAML

```
<Label Text="{Binding Population, FallbackValue={StaticResource  
populationUnknown}}"  
... />
```

ⓘ Note

It's not possible to set the [FallbackValue](#) property with a binding expression.

When the [FallbackValue](#) property isn't set in a binding expression and the binding path or part of the path isn't resolved, `BindableProperty.DefaultValue` is set on the target. However, when the [FallbackValue](#) property is set and the binding path or part of the path isn't resolved, the value of the [FallbackValue](#) value property is set on the target:

Face-Palm Monkey
Location unknown
Population size unknown

Therefore, in this example the [Label](#) displays "Population size unknown" because the bound object lacks a `Population` property.

ⓘ Important

A defined value converter is not executed in a binding expression when the [FallbackValue](#) property is set.

Define a null replacement value

The [TargetNullValue](#) property allows a replacement value to be defined that will be used when the binding *source* is resolved, but the value is `null`. A common scenario for setting this property is when binding to source properties that might be `null` in a bound collection.

The following example demonstrates setting the [TargetNullValue](#) property:

XAML

```

<ListView ItemsSource="{Binding Monkeys}"
    ...>
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                <Grid>
                    ...
                    <Image Source="{Binding ImageUrl,
TargetNullValue='https://upload.wikimedia.org/wikipedia/commons/2/20/Point_d
_interrogation.jpg'}"
                        ... />
                    ...
                    <Label Text="{Binding Location,
TargetNullValue='Location unknown'}"
                        ... />
                </Grid>
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>

```

The bindings on the [Image](#) and [Label](#) both define [TargetNullValue](#) values (delimited by single-quote characters) that will be applied if the binding path returns `null`. Therefore, the values defined by the [TargetNullValue](#) properties will be displayed for any objects in the collection where the `ImageUrl` and `Location` properties are not defined.

Rather than defining [TargetNullValue](#) property values inline, it's recommended to define them as resources in a [ResourceDictionary](#). The advantage of this approach is that such values are defined once in a single location, and are more easily localizable. The resources can then be retrieved using the [StaticResource](#) markup extension:

XAML

```

<Image Source="{Binding ImageUrl, TargetNullValue={StaticResource
fallbackImageUrl}}"
    ... />
<Label Text="{Binding Location, TargetNullValue={StaticResource
locationUnknown}}"
    ... />

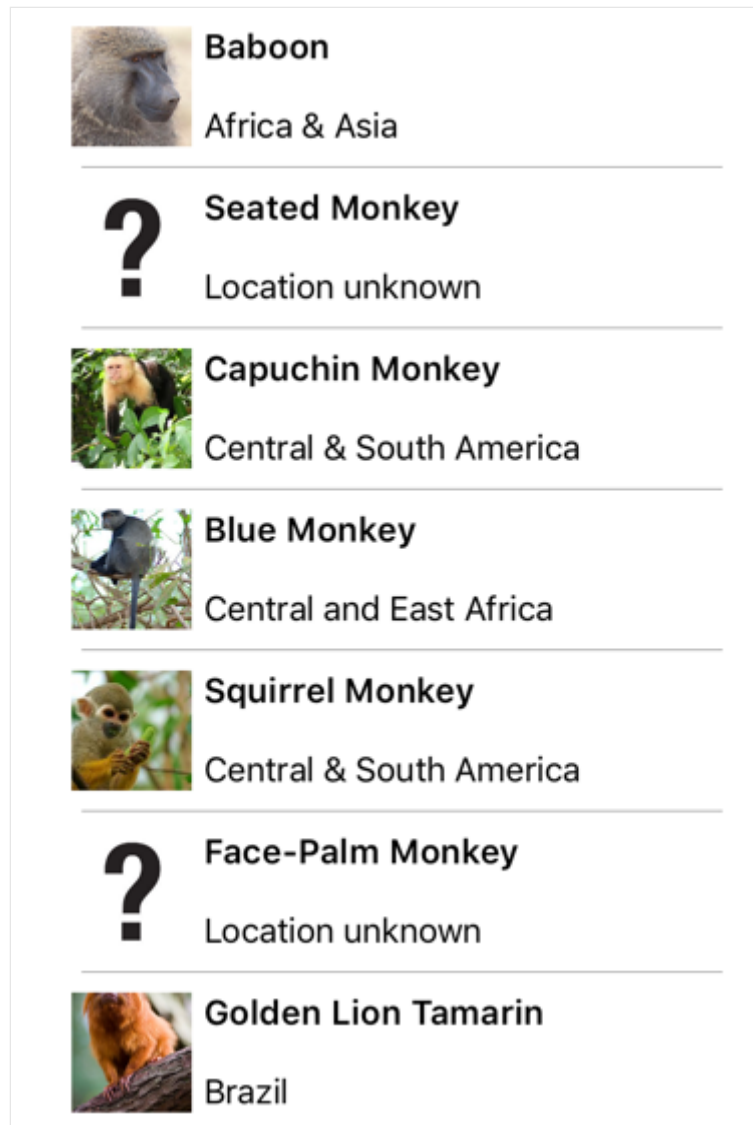
```

ⓘ Note

It's not possible to set the [TargetNullValue](#) property with a binding expression.

When the [TargetNullValue](#) property isn't set in a binding expression, a source value of `null` will be converted if a value converter is defined, formatted if a `StringFormat` is

defined, and the result is then set on the target. However, when the [TargetNullValue](#) property is set, a source value of `null` will be converted if a value converter is defined, and if it's still `null` after the conversion, the value of the [TargetNullValue](#) property is set on the target:



Therefore, in this example the [Image](#) and [Label](#) objects display their [TargetNullValue](#) when their source objects are `null`.

[i](#) Important

String formatting is not applied in a binding expression when the [TargetNullValue](#) property is set.

TargetNullValue and FallbackValue comparison

Both [TargetNullValue](#) and [FallbackValue](#) are used in data bindings to handle scenarios where the bound value is `null` or when the binding fails. Knowing when to use each

helps ensure a smooth user experience.

TargetNullValue

Use the [TargetNullValue](#) property to specify a value to display when the binding succeeds but the bound property is `null`:

XAML

```
<Label Text="{Binding FirstName, TargetNullValue='No name available'}" />
```

In this example, if `FirstName` is `null` the [Label](#) will display "No name available".

FallbackValue

Use the [FallbackValue](#) property to provide a value when the binding process fails entirely, such as an incorrect path, or a missing binding context:

XAML

```
<Label Text="{Binding FirstName, FallbackValue='Binding failed'}" />
```

In this example, if the binding fails the [Label](#) will display "Binding failed".

Combined use

The [TargetNullValue](#) and [FallbackValue](#) properties can be combined in a single binding expression:

XAML

```
<Label Text="{Binding FirstName, TargetNullValue='No name available',  
FallbackValue='Binding failed'}" />
```

Multi-bindings

Article • 04/03/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) multi-bindings provide the ability to attach a collection of `Binding` objects to a single binding target property. They're created with the `MultiBinding` class, which evaluates all of its `Binding` objects and returns a single value through a `IMultiValueConverter` instance provided by your app. In addition, `MultiBinding` reevaluates all of its `Binding` objects when any of the bound data changes.

The `MultiBinding` class defines the following properties:

- `Bindings`, of type `IList<BindingBase>`, which represents the collection of `Binding` objects within the `MultiBinding` instance.
- `Converter`, of type `IMultiValueConverter`, which represents the converter to use to convert the source values to or from the target value.
- `ConverterParameter`, of type `object`, which represents an optional parameter to pass to the `Converter`.

The `Bindings` property is the content property of the `MultiBinding` class, and therefore doesn't need to be explicitly set from XAML.

In addition, the `MultiBinding` class inherits the following properties from the `BindingBase` class:

- `FallbackValue`, of type `object`, which represents the value to use when the multi-binding is unable to return a value.
- `Mode`, of type `BindingMode`, which indicates the direction of the data flow of the multi-binding.
- `StringFormat`, of type `string`, which specifies how to format the multi-binding result if it's displayed as a string.
- `TargetNullValue`, of type `object`, which represents the value that's used in the target when the value of the source is `null`.

A `MultiBinding` must use a `IMultiValueConverter` to produce a value for the binding target, based on the value of the bindings in the `Bindings` collection. For example, a `Color` might be computed from red, blue, and green values, which can be values from the same or different binding source objects. When a value moves from the target to

the sources, the target property value is translated to a set of values that are fed back into the bindings.

Important

Individual bindings in the `Bindings` collection can have their own value converters.

The value of the `Mode` property determines the functionality of the `MultiBinding`, and is used as the binding mode for all the bindings in the collection unless an individual binding overrides the property. For example, if the `Mode` property on a `MultiBinding` object is set to `TwoWay`, then all the bindings in the collection are considered `TwoWay` unless you explicitly set a different `Mode` value on one of the bindings.

Define a `IMultiValueConverter`

The `IMultiValueConverter` interface enables custom logic to be applied to a `MultiBinding`. To associate a converter with a `MultiBinding`, create a class that implements the `IMultiValueConverter` interface, and then implement the `Convert` and `ConvertBack` methods:

C#

```
public class AllTrueMultiConverter : IMultiValueConverter
{
    public object Convert(object[] values, Type targetType, object
parameter, CultureInfo culture)
    {
        if (values == null || !targetType.IsAssignableFrom(typeof(bool)))
        {
            return false;
            // Alternatively, return BindableProperty.UnsetValue to use the
binding FallbackValue
        }

        foreach (var value in values)
        {
            if (!(value is bool b))
            {
                return false;
                // Alternatively, return BindableProperty.UnsetValue to use
the binding FallbackValue
            }
            else if (!b)
            {
                return false;
            }
        }
    }
}
```



```

    }
    return true;
}

public object[] ConvertBack(object value, Type[] targetTypes, object
parameter, CultureInfo culture)
{
    if (!(value is bool b) || targetTypes.Any(t =>
!t.IsAssignableFrom(typeof(bool))))
    {
        // Return null to indicate conversion back is not possible
        return null;
    }

    if (b)
    {
        return targetTypes.Select(t => (object>true).ToArray();
    }
    else
    {
        // Can't convert back from false because of ambiguity
        return null;
    }
}
}

```

The `Convert` method converts source values to a value for the binding target. .NET MAUI calls this method when it propagates values from source bindings to the binding target. This method accepts four arguments:

- `values`, of type `object[]`, is an array of values that the source bindings in the `MultiBinding` produces.
- `targetType`, of type `Type`, is the type of the binding target property.
- `parameter`, of type `object`, is the converter parameter to use.
- `culture`, of type `CultureInfo`, is the culture to use in the converter.

The `Convert` method returns an `object` that represents a converted value. This method should return:

- `BindableProperty.UnsetValue` to indicate that the converter did not produce a value, and that the binding will use the `FallbackValue`.
- `Binding.DoNothing` to instruct .NET MAUI not to perform any action. For example, to instruct .NET MAUI not to transfer a value to the binding target, or not to use the `FallbackValue`.
- `null` to indicate that the converter can't perform the conversion, and that the binding will use the `TargetNullValue`.

Important

A `MultiBinding` that receives `BindableProperty.UnsetValue` from a `Convert` method must define its `FallbackValue` property. Similarly, a `MultiBinding` that receives `null` from a `Convert` method must define its `TargetNullValue` property.

The `ConvertBack` method converts a binding target to the source binding values. This method accepts four arguments:

- `value`, of type `object`, is the value that the binding target produces.
- `targetTypes`, of type `Type[]`, is the array of types to convert to. The array length indicates the number and types of values that are suggested for the method to return.
- `parameter`, of type `object`, is the converter parameter to use.
- `culture`, of type `CultureInfo`, is the culture to use in the converter.

The `ConvertBack` method returns an array of values of type `object[]` that have been converted from the target values back to the source values. This method should return:

- `BindableProperty.UnsetValue` at position `i` to indicate that the converter is unable to provide a value for the source binding at index `i`, and that no value is to be set on it.
- `Binding.DoNothing` at position `i` to indicate that no value is to be set on the source binding at index `i`.
- `null` to indicate that the converter can't perform the conversion or that it doesn't support conversion in this direction.

Consume a `IMultiValueConverter`

A `IMultiValueConverter` is typically consumed by instantiating it in a resource dictionary, and then referencing it using the `StaticResource` markup extension to set the `MultiBinding.Converter` property:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:local="clr-namespace:DataBindingDemos"
  x:Class="DataBindingDemos.MultiBindingConverterPage"
  Title="MultiBinding Converter demo">
```

```
<ContentPage.Resources>
```

```

        <local:AllTrueMultiConverter x:Key="AllTrueConverter" />
        <local:InverterConverter x:Key="InverterConverter" />
    </ContentPage.Resources>

    <CheckBox>
        <CheckBox.IsChecked>
            <MultiBinding Converter="{StaticResource AllTrueConverter}">
                <Binding Path="Employee.IsOver16" />
                <Binding Path="Employee.HasPassedTest" />
                <Binding Path="Employee.IsSuspended"
                    Converter="{StaticResource InverterConverter}" />
            </MultiBinding>
        </CheckBox.IsChecked>
    </CheckBox>
</ContentPage>

```

In this example, the `MultiBinding` object uses the `AllTrueMultiConverter` instance to set the `CheckBox.IsChecked` property to `true`, provided that the three `Binding` objects evaluate to `true`. Otherwise, the `CheckBox.IsChecked` property is set to `false`.

By default, the `CheckBox.IsChecked` property uses a `TwoWay` binding. Therefore, the `ConvertBack` method of the `AllTrueMultiConverter` instance is executed when the `CheckBox` is unchecked by the user, which sets the source binding values to the value of the `CheckBox.IsChecked` property.

The equivalent C# code is shown here:

C#

```

public class MultiBindingConverterCodePage : ContentPage
{
    public MultiBindingConverterCodePage()
    {
        BindingContext = new GroupViewModel();

        CheckBox checkBox = new CheckBox();
        checkBox.SetBinding(CheckBox.IsCheckedProperty, new MultiBinding
        {
            Bindings = new Collection<BindingBase>
            {
                new Binding("Employee1.IsOver16"),
                new Binding("Employee1.HasPassedTest"),
                new Binding("Employee1.IsSuspended", converter: new
InverterConverter())
            },
            Converter = new AllTrueMultiConverter()
        });

        Title = "MultiBinding converter demo";
        Content = checkBox;
    }
}

```

```
}  
}
```

Format strings

A `MultiBinding` can format any multi-binding result that's displayed as a string, with the `StringFormat` property. This property can be set to a standard .NET formatting string, with placeholders, that specifies how to format the multi-binding result:

XAML

```
<Label>  
    <Label.Text>  
        <MultiBinding StringFormat="{0} {1} {2}">  
            <Binding Path="Employee1.Forename" />  
            <Binding Path="Employee1.MiddleName" />  
            <Binding Path="Employee1.Surname" />  
        </MultiBinding>  
    </Label.Text>  
</Label>
```

ⓘ Note

If the format string starts with the `{` character, the XAML parser will confuse it for a markup extension. To avoid this ambiguity, prefix the format string with an empty set of curly braces.

In this example, the `StringFormat` property combines the three bound values into a single string that's displayed by the [Label](#).

The equivalent C# code is shown here:

C#

```
Label label = new Label();  
label.SetBinding(Label.TextProperty, new MultiBinding  
{  
    Bindings = new Collection<BindingBase>  
    {  
        new Binding("Employee1.Forename"),  
        new Binding("Employee1.MiddleName"),  
        new Binding("Employee1.Surname")  
    },  
    StringFormat = "{0} {1} {2}"  
});
```

Important

The number of parameters in a composite string format can't exceed the number of child `Binding` objects in the `MultiBinding`.

When setting the `Converter` and `StringFormat` properties, the converter is applied to the data value first, and then the `StringFormat` is applied.

For more information about string formatting in .NET MAUI, see [String formatting](#).

Provide fallback values

Data bindings can be made more robust by defining fallback values to use if the binding process fails. Set fallback values by defining the `FallbackValue` and `TargetNullValue` properties on a `MultiBinding` object.

A `MultiBinding` will use its `FallbackValue` when the `Convert` method of an `IMultiValueConverter` instance returns `BindableProperty.UnsetValue`, which indicates that the converter did not produce a value. A `MultiBinding` will use its `TargetNullValue` when the `Convert` method of an `IMultiValueConverter` instance returns `null`, which indicates that the converter can't perform the conversion.

For more information about binding fallbacks, see [Binding fallbacks](#).

Nest MultiBinding objects

`MultiBinding` objects can be nested so that multiple `MultiBinding` objects are evaluated to return a value through an `IMultiValueConverter` instance:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:DataBindingDemos"
    x:Class="DataBindingDemos.NestedMultiBindingPage"
    Title="Nested MultiBinding demo">

    <ContentPage.Resources>
        <local:AllTrueMultiConverter x:Key="AllTrueConverter" />
        <local:AnyTrueMultiConverter x:Key="AnyTrueConverter" />
        <local:InverterConverter x:Key="InverterConverter" />
    </ContentPage.Resources>
```

```

<CheckBox>
    <CheckBox.IsChecked>
        <MultiBinding Converter="{StaticResource AnyTrueConverter}">
            <MultiBinding Converter="{StaticResource AllTrueConverter}">
                <Binding Path="Employee.IsOver16" />
                <Binding Path="Employee.HasPassedTest" />
                <Binding Path="Employee.IsSuspended" Converter="
{StaticResource InverterConverter}" />
            </MultiBinding>
            <Binding Path="Employee.IsMonarch" />
        </MultiBinding>
    </CheckBox.IsChecked>
</CheckBox>
</ContentPage>

```

In this example, the `MultiBinding` object uses its `AnyTrueMultiConverter` instance to set the `CheckBox.IsChecked` property to `true`, provided that all of the `Binding` objects in the inner `MultiBinding` object evaluate to `true`, or if the `Binding` object in the outer `MultiBinding` object evaluates to `true`. Otherwise, the `CheckBox.IsChecked` property is set to `false`.

Use a RelativeSource binding in a MultiBinding

`MultiBinding` objects support relative bindings, which let you set the binding source relative to the position of the binding target:

XAML

```

<ContentPage ...
    xmlns:local="clr-namespace:DataBindingDemos">
    <ContentPage.Resources>
        <local:AllTrueMultiConverter x:Key="AllTrueConverter" />

        <ControlTemplate x:Key="CardViewExpanderControlTemplate">
            <local:Expander BindingContext="{Binding Source={RelativeSource
TemplatedParent}}">
                IsExpanded="{Binding IsExpanded, Source=
{RelativeSource TemplatedParent}}">
                BackgroundColor="{Binding CardColor}"
                RowDefinitions="Auto,Auto"
                Padding="8">
                <local:Expander.IsVisible>
                    <MultiBinding Converter="{StaticResource
AllTrueConverter}">
                        <Binding Path="IsExpanded" />
                        <Binding Path="IsEnabled" />
                    </MultiBinding>
                </local:Expander.IsVisible>
            </Grid>

```

```

        <!-- XAML that defines Expander header goes here -->
    </Grid>
    <Grid>
        <!-- XAML that defines Expander content goes here -->
    </Grid>
</local:Expander>
</ControlTemplate>
</ContentPage.Resources>

<StackLayout>
    <controls:CardViewExpander BorderColor="DarkGray"
        CardTitle="John Doe"
        CardDescription="Lorem ipsum dolor sit
amet, consectetur adipiscing elit. Nulla elit dolor, convallis non
interdum."

        IconBackgroundColor="SlateGray"
        IconImageSource="user.png"
        ControlTemplate="{StaticResource
CardViewExpanderControlTemplate}"

        IsEnabled="True"
        IsExpanded="True" />

</StackLayout>
</ContentPage>

```

In this example, the `TemplatedParent` relative binding mode is used to bind from within a control template to the runtime object instance to which the template is applied. The `Expander`, which is the root element of the [ControlTemplate](#), has its `BindingContext` set to the runtime object instance to which the template is applied. Therefore, the `Expander` and its children resolve their binding expressions, and `Binding` objects, against the properties of the `CardViewExpander` object. The `MultiBinding` uses the `AllTrueMultiConverter` instance to set the `Expander.IsVisible` property to `true` provided that the two `Binding` objects evaluate to `true`. Otherwise, the `Expander.IsVisible` property is set to `false`.

For more information about relative bindings, see [Relative bindings](#). For more information about control templates, see [Control templates](#).

Commanding

Article • 11/15/2023

 [Browse the sample](#)

In a .NET Multi-platform App UI (.NET MAUI) app that uses the Model-View-ViewModel (MVVM) pattern, data bindings are defined between properties in the viewmodel, which is typically a class that derives from `INotifyPropertyChanged`, and properties in the view, which is typically the XAML file. Sometimes an app has needs that go beyond these property bindings by requiring the user to initiate commands that affect something in the viewmodel. These commands are generally signaled by button clicks or finger taps, and traditionally they are processed in the code-behind file in a handler for the `Clicked` event of the `Button` or the `Tapped` event of a `TapGestureRecognizer`.

The commanding interface provides an alternative approach to implementing commands that is much better suited to the MVVM architecture. The viewmodel can contain commands, which are methods that are executed in reaction to a specific activity in the view such as a `Button` click. Data bindings are defined between these commands and the `Button`.

To allow a data binding between a `Button` and a viewmodel, the `Button` defines two properties:

- `Command` of type `System.Windows.Input.ICommand`
- `CommandParameter` of type `Object`

To use the command interface, you define a data binding that targets the `Command` property of the `Button` where the source is a property in the viewmodel of type `ICommand`. The viewmodel contains code associated with that `ICommand` property that is executed when the button is clicked. You can set the `CommandParameter` property to arbitrary data to distinguish between multiple buttons if they are all bound to the same `ICommand` property in the viewmodel.

Many other views also define `Command` and `CommandParameter` properties. All these commands can be handled within a viewmodel using an approach that doesn't depend on the user-interface object in the view.

ICommands

The `ICommand` interface is defined in the `System.Windows.Input` namespace, and consists of two methods and one event:

C#

```
public interface ICommand
{
    public void Execute (Object parameter);
    public bool CanExecute (Object parameter);
    public event EventHandler CanExecuteChanged;
}
```

To use the command interface, your viewmodel should contain properties of type `ICommand`:

C#

```
public ICommand MyCommand { private set; get; }
```

The viewmodel must also reference a class that implements the `ICommand` interface. In the view, the `Command` property of a `Button` is bound to that property:

XAML

```
<Button Text="Execute command"
        Command="{Binding MyCommand}" />
```

When the user presses the `Button`, the `Button` calls the `Execute` method in the `ICommand` object bound to its `Command` property.

When the binding is first defined on the `Command` property of the `Button`, and when the data binding changes in some way, the `Button` calls the `CanExecute` method in the `ICommand` object. If `CanExecute` returns `false`, then the `Button` disables itself. This indicates that the particular command is currently unavailable or invalid.

The `Button` also attaches a handler on the `CanExecuteChanged` event of `ICommand`. The event is raised from within the viewmodel. When that event is raised, the `Button` calls `CanExecute` again. The `Button` enables itself if `CanExecute` returns `true` and disables itself if `CanExecute` returns `false`.

Warning

Do not use the `IsEnabled` property of `Button` if you're using the command interface.

When your viewmodel defines a property of type `ICommand`, the viewmodel must also contain or reference a class that implements the `ICommand` interface. This class must contain or reference the `Execute` and `CanExecute` methods, and fire the `CanExecuteChanged` event whenever the `CanExecute` method might return a different value. You can use the `Command` or `Command<T>` class included in .NET MAUI to implement the `ICommand` interface. These classes allow you to specify the bodies of the `Execute` and `CanExecute` methods in class constructors.

Tip

Use `Command<T>` when you use the `CommandParameter` property to distinguish between multiple views bound to the same `ICommand` property, and the `Command` class when that isn't a requirement.

Basic commanding

The following examples demonstrate basic commands implemented in a viewmodel.

The `PersonViewModel` class defines three properties named `Name`, `Age`, and `Skills` that define a person:

C#

```
public class PersonViewModel : INotifyPropertyChanged
{
    string name;
    double age;
    string skills;

    public event PropertyChangedEventHandler PropertyChanged;

    public string Name
    {
        set { SetProperty(ref name, value); }
        get { return name; }
    }

    public double Age
    {
        set { SetProperty(ref age, value); }
        get { return age; }
    }

    public string Skills
    {
        set { SetProperty(ref skills, value); }
```

```

        get { return skills; }
    }

    public override string ToString()
    {
        return Name + ", age " + Age;
    }

    bool SetProperty<T>(ref T storage, T value, [CallerMemberName] string
propertyName = null)
    {
        if (Object.Equals(storage, value))
            return false;

        storage = value;
        OnPropertyChanged(propertyName);
        return true;
    }

    protected void OnPropertyChanged([CallerMemberName] string propertyName
= null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}

```

The `PersonCollectionViewModel` class shown below creates new objects of type `PersonViewModel` and allows the user to fill in the data. For that purpose, the class defines `IsEditing`, of type `bool`, and `PersonEdit`, of type `PersonViewModel`, properties. In addition, the class defines three properties of type `ICommand` and a property named `Persons` of type `IList<PersonViewModel>`:

C#

```

public class PersonCollectionViewModel : INotifyPropertyChanged
{
    PersonViewModel personEdit;
    bool isEditing;

    public event PropertyChangedEventHandler PropertyChanged;
    ...

    public bool IsEditing
    {
        private set { SetProperty(ref isEditing, value); }
        get { return isEditing; }
    }

    public PersonViewModel PersonEdit
    {

```

```

        set { SetProperty(ref personEdit, value); }
        get { return personEdit; }
    }

    public ICommand NewCommand { private set; get; }
    public ICommand SubmitCommand { private set; get; }
    public ICommand CancelCommand { private set; get; }

    public IList<PersonViewModel> Persons { get; } = new
ObservableCollection<PersonViewModel>();

    bool SetProperty<T>(ref T storage, T value, [CallerMemberName] string
propertyName = null)
    {
        if (Object.Equals(storage, value))
            return false;

        storage = value;
        OnPropertyChanged(propertyName);
        return true;
    }

    protected void OnPropertyChanged([CallerMemberName] string propertyName
= null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}

```

In this example, changes to the three `ICommand` properties and the `Persons` property do not result in `PropertyChanged` events being raised. These properties are all set when the class is first created and do not change.

The following example shows the XAML that consumes the `PersonCollectionViewModel`:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:DataBindingDemos"
    x:Class="DataBindingDemos.PersonEntryPage"
    Title="Person Entry">
    <ContentPage.BindingContext>
        <local:PersonCollectionViewModel />
    </ContentPage.BindingContext>
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
    </Grid>
</ContentPage>

```

```

</Grid.RowDefinitions>

<!-- New Button -->
<Button Text="New"
        Grid.Row="0"
        Command="{Binding NewCommand}"
        HorizontalOptions="Start" />

<!-- Entry Form -->
<Grid Grid.Row="1"
      IsEnabled="{Binding IsEditing}">
  <Grid BindingContext="{Binding PersonEdit}">
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto" />
      <RowDefinition Height="Auto" />
      <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="Auto" />
      <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <Label Text="Name: " Grid.Row="0" Grid.Column="0" />
    <Entry Text="{Binding Name}"
          Grid.Row="0" Grid.Column="1" />
    <Label Text="Age: " Grid.Row="1" Grid.Column="0" />
    <StackLayout Orientation="Horizontal"
                Grid.Row="1" Grid.Column="1">
      <Stepper Value="{Binding Age}"
              Maximum="100" />
      <Label Text="{Binding Age, StringFormat='{0} years
old'}"

              VerticalOptions="Center" />
    </StackLayout>
    <Label Text="Skills: " Grid.Row="2" Grid.Column="0" />
    <Entry Text="{Binding Skills}"
          Grid.Row="2" Grid.Column="1" />
  </Grid>
</Grid>

<!-- Submit and Cancel Buttons -->
<Grid Grid.Row="2">
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="*" />
    <ColumnDefinition Width="*" />
  </Grid.ColumnDefinitions>

  <Button Text="Submit"
          Grid.Column="0"
          Command="{Binding SubmitCommand}"
          VerticalOptions="Center" />
  <Button Text="Cancel"
          Grid.Column="1"
          Command="{Binding CancelCommand}"
          VerticalOptions="Center" />
</Grid>

```

```

</Grid>

<!-- List of Persons -->
<ListView Grid.Row="3"
          ItemsSource="{Binding Persons}" />
</Grid>
</ContentPage>

```

In this example, the page's `BindingContext` property is set to the `PersonCollectionViewModel`. The `Grid` contains a `Button` with the text **New** with its `Command` property bound to the `NewCommand` property in the viewmodel, an entry form with properties bound to the `IsEditing` property, as well as properties of `PersonViewModel`, and two more buttons bound to the `SubmitCommand` and `CancelCommand` properties of the viewmodel. The `ListView` displays the collection of persons already entered:

The following screenshot shows the **Submit** button enabled after an age has been set:

NEW

Name:

Age: 33 years old

Skills:

SUBMIT CANCEL

Amy Strande, age 30

Andreas Hanefeld Dziegiel, age 28

Arvind B. Rao, age 24

When the user first presses the **New** button, this enables the entry form but disables the **New** button. The user then enters a name, age, and skills. At any time during the editing, the user can press the **Cancel** button to start over. Only when a name and a valid age have been entered is the **Submit** button enabled. Pressing this **Submit** button transfers the person to the collection displayed by the [ListView](#). After either the **Cancel** or **Submit** button is pressed, the entry form is cleared and the **New** button is enabled again.

All the logic for the **New**, **Submit**, and **Cancel** buttons is handled in `PersonCollectionViewModel` through definitions of the `NewCommand`, `SubmitCommand`, and `CancelCommand` properties. The constructor of the `PersonCollectionViewModel` sets these three properties to objects of type `Command`.

A constructor of the `Command` class allows you to pass arguments of type `Action` and `Func<bool>` corresponding to the `Execute` and `CanExecute` methods. This action and function can be defined as lambda functions in the `Command` constructor:

C#

```
public class PersonCollectionViewModel : INotifyPropertyChanged
{
    ...
    public PersonCollectionViewModel()
    {
        NewCommand = new Command(
            execute: () =>
            {
                PersonEdit = new PersonViewModel();
                PersonEdit.PropertyChanged += OnPersonEditPropertyChanged;
                IsEditing = true;
                RefreshCanExecutes();
            },
            canExecute: () =>
            {
                return !IsEditing;
            }
        );
        ...
    }

    void OnPersonEditPropertyChanged(object sender, PropertyChangedEventArgs args)
    {
        (SubmitCommand as Command).ChangeCanExecute();
    }

    void RefreshCanExecutes()
    {
        (NewCommand as Command).ChangeCanExecute();
        (SubmitCommand as Command).ChangeCanExecute();
        (CancelCommand as Command).ChangeCanExecute();
    }
}
```

```
...  
}
```

When the user clicks the **New** button, the `execute` function passed to the `Command` constructor is executed. This creates a new `PersonViewModel` object, sets a handler on that object's `PropertyChanged` event, sets `IsEditing` to `true`, and calls the `RefreshCanExecutes` method defined after the constructor.

Besides implementing the `ICommand` interface, the `Command` class also defines a method named `ChangeCanExecute`. A viewmodel should call `ChangeCanExecute` for an `ICommand` property whenever anything happens that might change the return value of the `CanExecute` method. A call to `ChangeCanExecute` causes the `Command` class to fire the `CanExecuteChanged` method. The **Button** has attached a handler for that event and responds by calling `CanExecute` again, and then enabling itself based on the return value of that method.

When the `execute` method of `NewCommand` calls `RefreshCanExecutes`, the `NewCommand` property gets a call to `ChangeCanExecute`, and the **Button** calls the `canExecute` method, which now returns `false` because the `IsEditing` property is now `true`.

The `PropertyChanged` handler for the new `PersonViewModel` object calls the `ChangeCanExecute` method of `SubmitCommand`:

C#

```
public class PersonCollectionViewModel : INotifyPropertyChanged  
{  
    ...  
    public PersonCollectionViewModel()  
    {  
        ...  
        SubmitCommand = new Command(  
            execute: () =>  
            {  
                Persons.Add(PersonEdit);  
                PersonEdit.PropertyChanged -= OnPersonEditPropertyChanged;  
                PersonEdit = null;  
                IsEditing = false;  
                RefreshCanExecutes();  
            },  
            canExecute: () =>  
            {  
                return PersonEdit != null &&  
                    PersonEdit.Name != null &&  
                    PersonEdit.Name.Length > 1 &&  
                    PersonEdit.Age > 0;  
            }  
        ));  
    }  
}
```



```

    ...
}
    ...
}

```

The `canExecute` function for `SubmitCommand` is called every time there's a property changed in the `PersonViewModel` object being edited. It returns `true` only when the `Name` property is at least one character long, and `Age` is greater than 0. At that time, the **Submit** button becomes enabled.

The `execute` function for **Submit** removes the property-changed handler from the `PersonViewModel`, adds the object to the `Persons` collection, and returns everything to its initial state.

The `execute` function for the **Cancel** button does everything that the **Submit** button does except add the object to the collection:

C#

```

public class PersonCollectionViewModel : INotifyPropertyChanged
{
    ...
    public PersonCollectionViewModel()
    {
        ...
        CancelCommand = new Command(
            execute: () =>
            {
                PersonEdit.PropertyChanged -= OnPersonEditPropertyChanged;
                PersonEdit = null;
                IsEditing = false;
                RefreshCanExecutes();
            },
            canExecute: () =>
            {
                return IsEditing;
            });
    }
    ...
}

```

The `canExecute` method returns `true` at any time a `PersonViewModel` is being edited.

❗ Note

It isn't necessary to define the `execute` and `canExecute` methods as lambda functions. You can write them as private methods in the viewmodel and reference

them in the `Command` constructors. However, this approach can result in a lot of methods that are referenced only once in the viewmodel.

Using Command parameters

It's sometimes convenient for one or more buttons, or other user-interface objects, to share the same `ICommand` property in the viewmodel. In this case, you can use the `CommandParameter` property to distinguish between the buttons.

You can continue to use the `Command` class for these shared `ICommand` properties. The class defines an alternative constructor that accepts `execute` and `canExecute` methods with parameters of type `Object`. This is how the `CommandParameter` is passed to these methods. However, when specifying a `CommandParameter`, it's easiest to use the generic `Command<T>` class to specify the type of the object set to `CommandParameter`. The `execute` and `canExecute` methods that you specify have parameters of that type.

The following example demonstrates a keyboard for entering decimal numbers:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:DataBindingDemos"
    x:Class="DataBindingDemos.DecimalKeypadPage"
    Title="Decimal Keyboard">
    <ContentPage.BindingContext>
        <local:DecimalKeypadViewModel />
    </ContentPage.BindingContext>
    <ContentPage.Resources>
        <Style TargetType="Button">
            <Setter Property="FontSize" Value="32" />
            <Setter Property="BorderWidth" Value="1" />
            <Setter Property="BorderColor" Value="Black" />
        </Style>
    </ContentPage.Resources>

    <Grid WidthRequest="240"
        HeightRequest="480"
        ColumnDefinitions="80, 80, 80"
        RowDefinitions="Auto, Auto, Auto, Auto, Auto, Auto"
        ColumnSpacing="2"
        RowSpacing="2"
        HorizontalOptions="Center"
        VerticalOptions="Center">
        <Label Text="{Binding Entry}"
            Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="3"
            Margin="0,0,10,0"/>
```

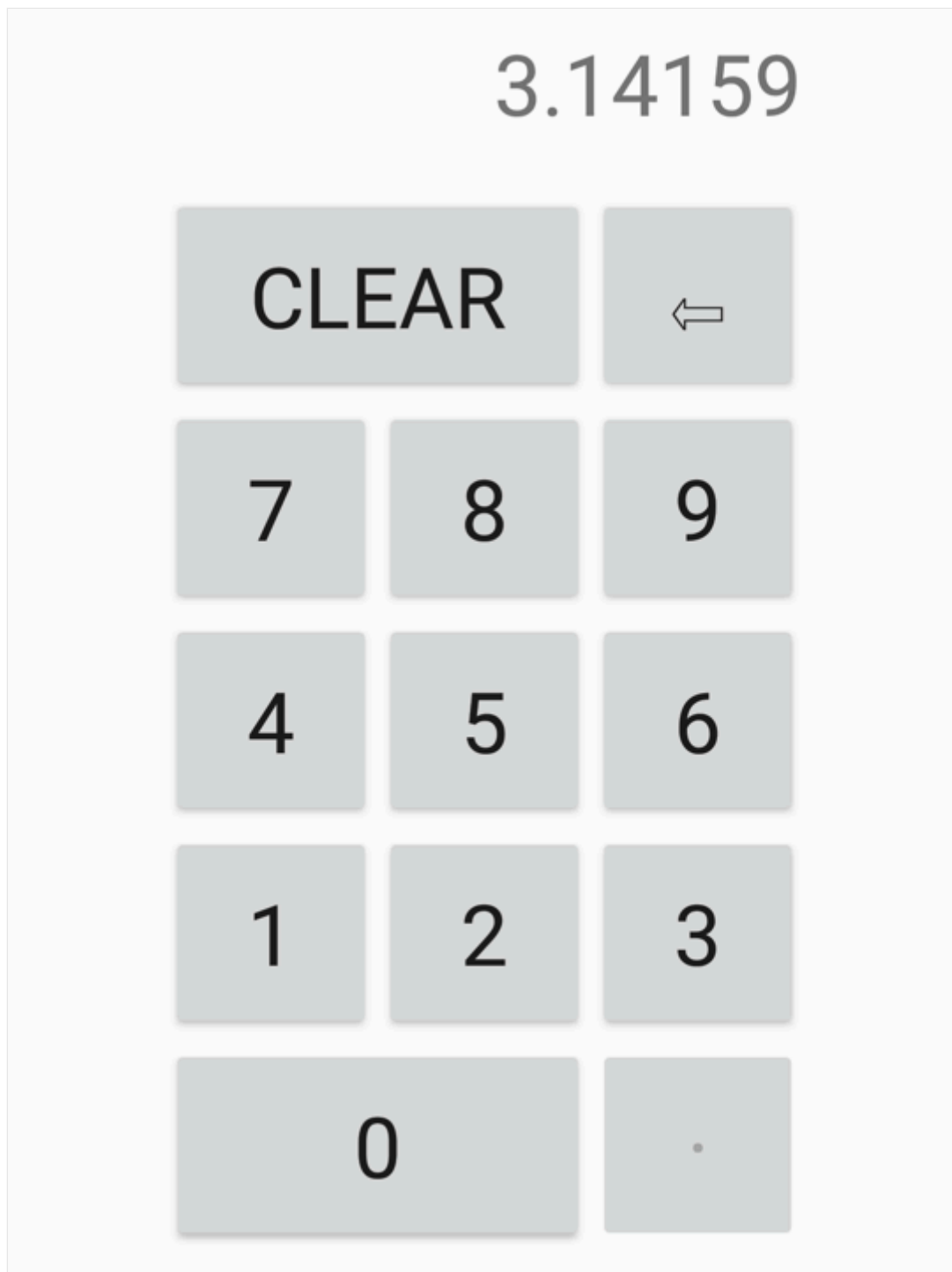
```

        FontSize="32"
        LineBreakMode="HeadTruncation"
        VerticalTextAlignment="Center"
        HorizontalTextAlignment="End" />
<Button Text="CLEAR"
        Grid.Row="1" Grid.Column="0" Grid.ColumnSpan="2"
        Command="{Binding ClearCommand}" />
<Button Text="⌫;"
        Grid.Row="1" Grid.Column="2"
        Command="{Binding BackspaceCommand}" />
<Button Text="7"
        Grid.Row="2" Grid.Column="0"
        Command="{Binding DigitCommand}"
        CommandParameter="7" />
<Button Text="8"
        Grid.Row="2" Grid.Column="1"
        Command="{Binding DigitCommand}"
        CommandParameter="8" />
<Button Text="9"
        Grid.Row="2" Grid.Column="2"
        Command="{Binding DigitCommand}"
        CommandParameter="9" />
<Button Text="4"
        Grid.Row="3" Grid.Column="0"
        Command="{Binding DigitCommand}"
        CommandParameter="4" />
<Button Text="5"
        Grid.Row="3" Grid.Column="1"
        Command="{Binding DigitCommand}"
        CommandParameter="5" />
<Button Text="6"
        Grid.Row="3" Grid.Column="2"
        Command="{Binding DigitCommand}"
        CommandParameter="6" />
<Button Text="1"
        Grid.Row="4" Grid.Column="0"
        Command="{Binding DigitCommand}"
        CommandParameter="1" />
<Button Text="2"
        Grid.Row="4" Grid.Column="1"
        Command="{Binding DigitCommand}"
        CommandParameter="2" />
<Button Text="3"
        Grid.Row="4" Grid.Column="2"
        Command="{Binding DigitCommand}"
        CommandParameter="3" />
<Button Text="0"
        Grid.Row="5" Grid.Column="0" Grid.ColumnSpan="2"
        Command="{Binding DigitCommand}"
        CommandParameter="0" />
<Button Text="⌵;"
        Grid.Row="5" Grid.Column="2"
        Command="{Binding DigitCommand}"
        CommandParameter="." />

```

```
</Grid>
</ContentPage>
```

In this example, the page's `BindingContext` is a `DecimalKeypadViewModel`. The `Entry` property of this viewmodel is bound to the `Text` property of a `Label`. All the `Button` objects are bound to commands in the viewmodel: `ClearCommand`, `BackspaceCommand`, and `DigitCommand`. The 11 buttons for the 10 digits and the decimal point share a binding to `DigitCommand`. The `CommandParameter` distinguishes between these buttons. The value set to `CommandParameter` is generally the same as the text displayed by the button except for the decimal point, which for purposes of clarity is displayed with a middle dot character:



The `DecimalKeypadViewModel` defines an `Entry` property of type `string` and three properties of type `ICommand`:

C#

```
public class DecimalKeypadViewModel : INotifyPropertyChanged
{
    string entry = "0";

    public event PropertyChangedEventHandler PropertyChanged;
    ...

    public string Entry
    {
        private set
        {
            if (entry != value)
            {
                entry = value;
                PropertyChanged?.Invoke(this, new
PropertyChangeEventArgs("Entry"));
            }
        }
        get
        {
            return entry;
        }
    }

    public ICommand ClearCommand { private set; get; }
    public ICommand BackspaceCommand { private set; get; }
    public ICommand DigitCommand { private set; get; }
}
```

The button corresponding to the `ClearCommand` is always enabled and sets the entry back to "0":

C#

```
public class DecimalKeypadViewModel : INotifyPropertyChanged
{
    ...
    public DecimalKeypadViewModel()
    {
        ClearCommand = new Command(
            execute: () =>
            {
                Entry = "0";
                RefreshCanExecutes();
            });
        ...
    }
}
```

```

void RefreshCanExecutes()
{
    ((Command)BackspaceCommand).ChangeCanExecute();
    ((Command)DigitCommand).ChangeCanExecute();
}
...
}

```

Because the button is always enabled, it is not necessary to specify a `canExecute` argument in the `Command` constructor.

The **Backspace** button is enabled only when the length of the entry is greater than 1, or if `Entry` is not equal to the string "0":

C#

```

public class DecimalKeypadViewModel : INotifyPropertyChanged
{
    ...
    public DecimalKeypadViewModel()
    {
        ...
        BackspaceCommand = new Command(
            execute: () =>
            {
                Entry = Entry.Substring(0, Entry.Length - 1);
                if (Entry == "")
                {
                    Entry = "0";
                }
                RefreshCanExecutes();
            },
            canExecute: () =>
            {
                return Entry.Length > 1 || Entry != "0";
            }
        );
        ...
    }
    ...
}

```

The logic for the `execute` function for the **Backspace** button ensures that the `Entry` is at least a string of "0".

The `DigitCommand` property is bound to 11 buttons, each of which identifies itself with the `CommandParameter` property. The `DigitCommand` is set to an instance of the `Command<T>` class. When using the commanding interface with XAML, the

`CommandParameter` properties are usually strings, which is type of the generic argument. The `execute` and `canExecute` functions then have arguments of type `string`:

C#

```
public class DecimalKeypadViewModel : INotifyPropertyChanged
{
    ...
    public DecimalKeypadViewModel()
    {
        ...
        DigitCommand = new Command<string>(
            execute: (string arg) =>
            {
                Entry += arg;
                if (Entry.StartsWith("0") && !Entry.StartsWith("0."))
                {
                    Entry = Entry.Substring(1);
                }
                RefreshCanExecutes();
            },
            canExecute: (string arg) =>
            {
                return !(arg == "." && Entry.Contains("."));
            });
    }
    ...
}
```

The `execute` method appends the string argument to the `Entry` property. However, if the result begins with a zero (but not a zero and a decimal point) then that initial zero must be removed using the `Substring` function. The `canExecute` method returns `false` only if the argument is the decimal point (indicating that the decimal point is being pressed) and `Entry` already contains a decimal point. All the `execute` methods call `RefreshCanExecutes`, which then calls `ChangeCanExecute` for both `DigitCommand` and `ClearCommand`. This ensures that the decimal point and backspace buttons are enabled or disabled based on the current sequence of entered digits.

Compiled bindings

Article • 11/14/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) data bindings have two main issues:

1. There's no compile-time validation of binding expressions. Instead, bindings are resolved at runtime. Therefore, any invalid bindings aren't detected until runtime when the application doesn't behave as expected or error messages appear.
2. They aren't cost efficient. Bindings are resolved at runtime using general-purpose object inspection (reflection), and the overhead of doing this varies from platform to platform.

Compiled bindings improve data binding performance in .NET MAUI applications by resolving binding expressions at compile-time rather than runtime. In addition, this compile-time validation of binding expressions enables a better developer troubleshooting experience because invalid bindings are reported as build errors.

Important

Compiled bindings are required instead of string-based bindings in NativeAOT apps, and in apps with full trimming enabled. For more information, see [Trim a .NET MAUI app](#) and [Native AOT deployment](#).

Compiled bindings in XAML

To use compiled bindings in XAML, set an `x:DataType` attribute on a [VisualElement](#) to the type of the object that the [VisualElement](#) and its children will bind to. It's recommended to set the `x:DataType` attribute at the same level in the view hierarchy as the [BindingContext](#) is set. However, this attribute can be re-defined at any location in a view hierarchy.

Important

Compiled bindings require the use of XAML compilation, which is enabled by default in .NET MAUI. If you've disabled XAML compilation, you'll need to enable it. For more information, see [XAML Compilation](#).

To use compiled bindings in XAML, the `x:DataType` attribute must be set to a string literal, or a type using the `x:Type` markup extension. At XAML compile time, any invalid binding expressions will be reported as build errors. However, the XAML compiler will only report a

build error for the first invalid binding expression that it encounters. Any valid binding expressions that are defined on the [VisualElement](#) or its children will be compiled, regardless of whether the [BindingContext](#) is set in XAML or code. Compiling a binding expression generates compiled code that will get a value from a property on the *source*, and set it on the property on the *target* that's specified in the markup. In addition, depending on the binding expression, the generated code may observe changes in the value of the *source* property and refresh the *target* property, and may push changes from the *target* back to the *source*.

By default, .NET MAUI produces build warnings for XAML bindings that don't use compiled bindings.

For more information about XAML compiled bindings warnings, see [XAML compiled bindings warnings](#).

Use compiled bindings in XAML

The following example demonstrates using compiled bindings between .NET MAUI views and viewmodel properties:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:DataBindingDemos"
             x:Class="DataBindingDemos.CompiledColorSelectorPage"
             x:DataType="local:HslColorViewModel"
             Title="Compiled Color Selector">
    <ContentPage.BindingContext>
        <local:HslColorViewModel Color="Sienna" />
    </ContentPage.BindingContext>
    ...
    <StackLayout>
        <BoxView Color="{Binding Color}"
                ... />
        <StackLayout Margin="10, 0">
            <Label Text="{Binding Name}" />
            <Slider Value="{Binding Hue}" />
            <Label Text="{Binding Hue, StringFormat='Hue = {0:F2}}'" />
            <Slider Value="{Binding Saturation}" />
            <Label Text="{Binding Saturation, StringFormat='Saturation = {0:F2}}'" />
            <Slider Value="{Binding Luminosity}" />
            <Label Text="{Binding Luminosity, StringFormat='Luminosity = {0:F2}}'" />
        </StackLayout>
    </StackLayout>
</ContentPage>
```

The [ContentPage](#) instantiates the `HslColorViewModel` and initializes the `Color` property within property element tags for the [BindingContext](#) property. The [ContentPage](#) also defines the `x:DataType` attribute as the viewmodel type, indicating that any binding expressions in the [ContentPage](#) view hierarchy will be compiled. This can be verified by changing any of the binding expressions to bind to a non-existent viewmodel property, which will result in a build error. While this example sets the `x:DataType` attribute to a string literal, it can also be set to a type with the `x:type` markup extension. For more information about the `x:type` markup extension, see [x:type Markup Extension](#).

Important

The `x:DataType` attribute can be re-defined at any point in a view hierarchy.

The [BoxView](#), [Label](#) elements, and [Slider](#) views inherit the binding context from the [ContentPage](#). These views are all binding targets that reference source properties in the viewmodel. For the `BoxView.Color` property, and the `Label.Text` property, the data bindings are `OneWay` – the properties in the view are set from the properties in the viewmodel. However, the `Slider.Value` property uses a `TwoWay` binding. This allows each [Slider](#) to be set from the viewmodel, and also for the viewmodel to be set from each [Slider](#).

When the example is first run, the [BoxView](#), [Label](#) elements, and [Slider](#) elements are all set from the viewmodel based on the initial `Color` property set when the viewmodel was instantiated. As the sliders are manipulated, the [BoxView](#) and [Label](#) elements are updated accordingly:



For more information about this color selector, see [ViewModels and property-change notifications](#).

Use compiled bindings in XAML in a DataTemplate

Bindings in a [DataTemplate](#) are interpreted in the context of the object being templated. Therefore, when using compiled bindings in a [DataTemplate](#), the [DataTemplate](#) needs to declare the type of its data object using the `x:DataType` attribute. Failure to do this could result in the [DataTemplate](#) inheriting an incorrect `x:DataType` from its parent scope:

XAML

```
<ContentPage ...
    x:DataType="local:AnimalsPageViewModel">
    <!-- Binding to AnimalsPageViewModel.Animals -->
    <CollectionView ItemsSource="{Binding Animals}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <!-- incorrect: compiler thinks you want to bind to
AnimalsPageViewModel.Name -->
```

```

        <Label Text="{Binding Name}" />
    </DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</ContentPage>

```

The following example demonstrates correctly setting the `x:DataType` on a [DataTemplate](#):

XAML

```

<ContentPage ...
    x:DataType="local:AnimalsPageViewModel">
    <!-- Binding to AnimalsPageViewModel.Animals -->
    <CollectionView ItemsSource="{Binding Animals}">
        <CollectionView.ItemTemplate>
            <DataTemplate x:DataType="local:Animal">
                <!-- correct: compiler knows you want to bind to Animal.Name -->
                <Label Text="{Binding Name}" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</ContentPage>

```

While this example sets the `x:DataType` attribute to a string literal, it can also be set to a type with the `x:type` markup extension. For more information about the `x:type` markup extension, see [x:type Markup Extension](#).

Compile bindings that define the `Source` property

Prior to .NET MAUI 9, the XAML compiler would skip compilation of bindings that define the `Source` property instead of the [BindingContext](#). From .NET MAUI 9, these bindings can be compiled to take advantage of better runtime performance. However, this optimization isn't enabled by default to avoid breaking existing app code. To enable this optimization, set the `$(MauiEnableXamlCBindingWithSourceCompilation)` build property to `true` in your app's project file:

XML

```

<MauiEnableXamlCBindingWithSourceCompilation>true</MauiEnableXamlCBindingWithSourceCompilation>

```

Then, ensure that all your bindings are annotated with the correct `x:DataType` and that they don't inherit incorrect data types from their parent scope:

XAML

```

<HorizontalStackLayout BindingContext="{x:Reference slider}"
    x:DataType="Slider">

```

```
<Label Text="{Binding Value}" />
<Label Text="{Binding Text, Source={x:Reference entry}, x:DataType=Entry}"
/>
</HorizontalStackLayout>
```

⚠ Note

In cases where there's a binding with a `Source`, but it inherits the `x:DataType` from the parent, there can be a mismatch between the `x:DataType` and the type of the `Source`. In this scenario, a warning will be generated and a fallback to a reflection-based binding that resolves the binding path at runtime will occur.

Combine compiled bindings with classic bindings in XAML

Binding expressions are only compiled for the view hierarchy that the `x:DataType` attribute is defined on. Conversely, any views in a hierarchy on which the `x:DataType` attribute is not defined will use classic bindings. It's therefore possible to combine compiled bindings and classic bindings on a page. For example, in the previous section the views within the [DataTemplate](#) use compiled bindings, while the [BoxView](#) that's set to the color selected in the [ListView](#) does not.

Careful structuring of `x:DataType` attributes can therefore lead to a page using compiled and classic bindings. Alternatively, the `x:DataType` attribute can be re-defined at any point in a view hierarchy to `null` using the `x:Null` markup extension. Doing this indicates that any binding expressions within the view hierarchy will use classic bindings. The following example demonstrates this approach:

XAML

```
<StackLayout x:DataType="local:HslColorViewModel">
  <StackLayout.BindingContext>
    <local:HslColorViewModel Color="Sienna" />
  </StackLayout.BindingContext>
  <BoxView Color="{Binding Color}"
    VerticalOptions="FillAndExpand" />
  <StackLayout x:DataType="{x:Null}"
    Margin="10, 0">
    <Label Text="{Binding Name}" />
    <Slider Value="{Binding Hue}" />
    <Label Text="{Binding Hue, StringFormat='Hue = {0:F2}}'" />
    <Slider Value="{Binding Saturation}" />
    <Label Text="{Binding Saturation, StringFormat='Saturation = {0:F2}}'"
  />
  <Slider Value="{Binding Luminosity}" />
  <Label Text="{Binding Luminosity, StringFormat='Luminosity = {0:F2}}'"
/>
```

```
</StackLayout>
</StackLayout>
```

The root `StackLayout` sets the `x:DataType` attribute to be the `HslColorViewModel` type, indicating that any binding expression in the root `StackLayout` view hierarchy will be compiled. However, the inner `StackLayout` redefines the `x:DataType` attribute to `null` with the `x:Null` markup expression. Therefore, the binding expressions within the inner `StackLayout` use classic bindings. Only the `BoxView`, within the root `StackLayout` view hierarchy, uses compiled bindings.

For more information about the `x:Null` markup expression, see [x:Null Markup Extension](#).

XAML compiled bindings warnings

The following table lists the compiler warnings for compiled bindings, and how to resolve them:

 Expand table

Code	Message
XC0022	Binding could be compiled to improve runtime performance if <code>x:DataType</code> is specified. To fix this warning, add <code>x:DataType</code> to your XAML to specify the type of the current BindingContext. It's best practice to add <code>x:DataType</code> to all elements where the binding context changes.
XC0023	Binding could be compiled to improve runtime performance if <code>x:DataType</code> is not explicitly <code>null</code>. To fix this warning, replace <code>x:DataType="{x:Null}"</code> with the right type.
XC0024	Binding might be compiled incorrectly since the <code>x:DataType</code> annotation comes from an outer scope. Make sure you annotate all <code>DataTemplate</code> XAML elements with the correct <code>x:DataType</code>. To fix this warning, ensure that all <code>DataTemplate</code> elements are annotated with the correct <code>x:DataType</code>.
XC0025	Binding was not compiled because it has an explicitly set <code>Source</code> property and compilation of bindings with <code>Source</code> is not enabled. Consider enabling this optimization by setting the <code><MauiEnableXamlCBindingWithSourceCompilation>true</MauiEnableXamlCBindingWithSourceCompilation></code> in your project file and make sure the correct <code>x:DataType</code> is specified for this binding. To fix this warning, enable the <code>\$(MauiEnableXamlCBindingWithSourceCompilation)</code> build property in your project file, and annotate all your bindings with the appropriate <code>x:DataType</code>.

To ensure these warnings aren't ignored, consider changing specific warnings to build errors with the `$(WarningsAsErrors)` build property:

```
<WarningsAsErrors>$(WarningsAsErrors);XC0022;XC0023</WarningsAsErrors>
```

To ignore these warnings, use the `$(NoWarn)` build property with specific warning codes:

XML

```
<NoWarn>$(NoWarn);XC0022;XC0023</NoWarn>
```

If you set the `$(TreatWarningsAsErrors)` build property to `true` in your app's project file, but you want to ignore certain XAML compiler warnings, use either the `$(NoWarn)` build property to silence these warnings or the `$(WarningsNotAsErrors)` build property to reduce the severity of some specific codes.

By default, .NET MAUI produces build warnings for XAML bindings that don't use compiled bindings. You can opt into compiled bindings warnings being treated as errors by setting the `$(MauiStrictXamlCompilation)` and `$(TreatWarningsAsErrors)` build properties to `true` in your app's project file (*.csproj):

XML

```
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
<MauiStrictXamlCompilation>true</MauiStrictXamlCompilation>
```

⚠ Note

By default, the `$(MauiStrictXamlCompilation)` build property is `false` unless you are publishing your app using full trimming or NativeAOT.

Compiled bindings in code

Bindings written in code typically use string paths that are resolved at runtime with reflection. However, the `SetBinding` extension method also has an overload that defines bindings using a `Func` argument instead of a string path:

C#

```
MyLabel.SetBinding(Label.TextProperty, static (Entry entry) => entry.Text);
```

Not all methods can be used to define a compiled binding. The expression must be a simple property access expression. The following examples show valid and invalid binding expressions:

C#

```
// Valid: Property access
static (PersonViewModel vm) => vm.Name;
static (PersonViewModel vm) => vm.Address?.Street;

// Valid: Array and indexer access
static (PersonViewModel vm) => vm.PhoneNumbers[0];
static (PersonViewModel vm) => vm.Config["Font"];

// Valid: Casts
static (Label label) => (label.BindingContext as PersonViewModel).Name;
static (Label label) => ((PersonViewModel)label.BindingContext).Name;

// Invalid: Method calls
static (PersonViewModel vm) => vm.GetAddress();
static (PersonViewModel vm) => vm.Address?.ToString();

// Invalid: Complex expressions
static (PersonViewModel vm) => vm.Address?.Street + " " + vm.Address?.City;
static (PersonViewModel vm) => $"Name: {vm.Name}";
```

Warning

A CS0272 compiler error will occur if the set accessor for a property or indexer is inaccessible. If this occurs, increase the accessibility of the accessor.

In addition, the [BindingBase.Create](#) method sets the binding directly on the object with a `Func`, and returns the binding object instance:

C#

```
myEntry.SetBinding(Entry.TextProperty, new MultiBinding
{
    Bindings = new Collection<BindingBase>
    {
        Binding.Create(static (Entry entry) => entry.FontFamily, source:
RelativeBindingSource.Self),
        Binding.Create(static (Entry entry) => entry.FontSize, source:
RelativeBindingSource.Self),
        Binding.Create(static (Entry entry) => entry.FontAttributes, source:
RelativeBindingSource.Self),
    },
    Converter = new StringConcatenationConverter()
});
```

These compiled binding approaches provide the following benefits:

- Improved data binding performance by resolving binding expressions at compile-time rather than runtime.

- A better developer troubleshooting experience because invalid bindings are reported as build errors.
- Intellisense while editing.

Performance

Compiled bindings improve data binding performance, with the performance benefit varying:

- A compiled binding that uses property-change notification (i.e. a `OneWay`, `OneWayToSource`, or `TwoWay` binding) is resolved approximately 8 times quicker than a classic binding.
- A compiled binding that doesn't use property-change notification (i.e. a `OneTime` binding) is resolved approximately 20 times quicker than a classic binding.
- Setting the `BindingContext` on a compiled binding that uses property change notification (i.e. a `OneWay`, `OneWayToSource`, or `TwoWay` binding) is approximately 5 times quicker than setting the `BindingContext` on a classic binding.
- Setting the `BindingContext` on a compiled binding that doesn't use property change notification (i.e. a `OneTime` binding) is approximately 7 times quicker than setting the `BindingContext` on a classic binding.

These performance differences can be magnified on mobile devices, dependent upon the platform being used, the version of the operating system being used, and the device on which the application is running.

Dependency injection

Article • 05/03/2024

.NET Multi-platform App UI (.NET MAUI) provides in-built support for using dependency injection. Dependency injection is a specialized version of the [Inversion of Control](#) (IoC) pattern, where the concern being inverted is the process of obtaining the required dependency. With dependency injection, another class is responsible for injecting dependencies into an object at runtime.

Typically, a class constructor is invoked when instantiating an object, and any values that the object needs are passed as arguments to the constructor. This is an example of dependency injection known as *constructor injection*. The dependencies the object needs are injected into the constructor.

ⓘ Note

There are also other types of dependency injection, such as *property setter injection* and *method call injection*, but they are less commonly used.

By specifying dependencies as interface types, dependency injection enables decoupling the concrete types from the code that depends on these types. It generally uses a container that holds a list of registrations and mappings between interfaces and abstract types, and the concrete types that implement or extend these types.

Dependency injection containers

If a class doesn't directly instantiate the objects it needs, another class must take on this responsibility. Consider the following example, which shows a view-model class that requires constructor arguments:

C#

```
public class MainPageViewModel
{
    readonly ILoggingService _loggingService;
    readonly ISettingsService _settingsService;

    public MainPageViewModel(ILoggingService loggingService,
        ISettingsService settingsService)
    {
        _loggingService = loggingService;
        _settingsService = settingsService;
    }
}
```

```
}  
}
```

In this example, the `MainPageViewModel` constructor requires two interface object instances as arguments injected by another class. The only dependency in the `MainPageViewModel` class is on the interface types. Therefore, the `MainPageViewModel` class doesn't have any knowledge of the class that's responsible for instantiating the interface objects.

Similarly, consider the following example that shows a page class that requires a constructor argument:

C#

```
public MainPage(MainPageViewModel viewModel)  
{  
    InitializeComponent();  
  
    BindingContext = viewModel;  
}
```

In this example, the `MainPage` constructor requires a concrete type as an argument that's injected by another class. The only dependency in the `MainPage` class is on the `MainPageViewModel` type. Therefore, the `MainPage` class doesn't have any knowledge of the class that's responsible for instantiating the concrete type.

In both cases, the class that's responsible for instantiating the dependencies, and inserting them into the dependent class, is known as the *dependency injection container*.

Dependency injection containers reduce the coupling between objects by providing a facility to instantiate class instances and manage their lifetime based on the configuration of the container. During object creation, the container injects any dependencies that the object requires. If those dependencies haven't been created, the container creates and resolves their dependencies first.

There are several advantages to using a dependency injection container:

- A container removes the need for a class to locate its dependencies and manage its lifetimes.
- A container allows the mapping of implemented dependencies without affecting the class.
- A container facilitates testability by allowing dependencies to be mocked.
- A container increases maintainability by allowing new classes to be easily added to the app.

In the context of a .NET MAUI app that uses the Model-View-ViewModel (MVVM) pattern, a dependency injection container will typically be used for registering and resolving views, registering and resolving view models, and for registering services and injecting them into view-models. For more information about the MVVM pattern, see [Model-View-ViewModel \(MVVM\)](#).

There are many dependency injection containers available for .NET. .NET MAUI has in-built support for using [Microsoft.Extensions.DependencyInjection](#) to manage the instantiation of views, view models, and service classes in an app.

[Microsoft.Extensions.DependencyInjection](#) facilitates building loosely coupled apps, and provides all of the features commonly found in dependency injection containers, including methods to register type mappings and object instances, resolve objects, manage object lifetimes, and inject dependent objects into constructors of objects that it resolves. For more information about [Microsoft.Extensions.DependencyInjection](#), see [Dependency injection in .NET](#).

At runtime, the container must know which implementation of the dependencies are being requested in order to instantiate them for the requested objects. In the example above, the `ILoggingService` and `ISettingsService` interfaces need to be resolved before the `MainPageViewModel` object can be instantiated. This involves the container performing the following actions:

- Deciding how to instantiate an object that implements the interface. This is known as *registration*. For more information, see [Registration](#).
- Instantiating the object that implements the required interface and the `MainPageViewModel` object. This is known as *resolution*. For more information, see [Resolution](#).

Eventually, an app will finish using the `MainPageViewModel` object, and it will become available for garbage collection. At this point, the garbage collector should dispose of any short-lived interface implementations if other classes do not share the same instances.

Registration

Before dependencies can be injected into an object, the types for the dependencies must first be registered with the container. Registering a type typically involves passing the container a concrete type, or an interface and a concrete type that implements the interface.

There are two main approaches to registering types and objects with the container:

- Register a type or mapping with the container. This is known as transient registration. When required, the container will build an instance of the specified type.
- Register an existing object in the container as a singleton. When required, the container will return a reference to the existing object.

⊗ Caution

Dependency injection containers are not always suitable for a .NET MAUI app. Dependency injection introduces additional complexity and requirements that might not be appropriate or useful to smaller apps. If a class doesn't have any dependencies, or isn't a dependency for other types, it might not make sense to put it in the container. In addition, if a class has a single set of dependencies that are integral to the type and will never change, it might not make sense to put them in the container.

The registration of types requiring dependency injection should be performed in a single method in your app. This method should be invoked early in the app's lifecycle to ensure it's aware of the dependencies between its classes. Apps should typically perform this in the `CreateMauiApp` method in the `MauiProgram` class. The `MauiProgram` class calls into the `CreateMauiApp` method to create a `MauiAppBuilder` object. The `MauiAppBuilder` object has a `Services` property of type `IServiceCollection`, which provides a place to register your types, such as views, view-models, and services for dependency injection:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            });

        builder.Services.AddTransient<ILoggingService, LoggingService>();
        builder.Services.AddTransient<ISettingsService, SettingsService>();
        builder.Services.AddSingleton<MainPageViewModel>();
        builder.Services.AddSingleton<MainPage>();

#if DEBUG
        builder.Logging.AddDebug();
#endif
    }
}
```

```
#endif

    return builder.Build();
}
}
```

Types that are registered with the [Services](#) property are provided to the dependency injection container when [MauiAppBuilder.Build\(\)](#) is called.

When registering dependencies, you need to register all dependencies including including any types that require the dependencies. Therefore, if you have a view-model that takes a dependency as a constructor parameter, you need to register the view-model along with all of its dependencies. Similarly, if you have a view that takes a view-model dependency as a constructor parameter, you need to register the view, and the view-model along with all its dependencies.

Tip

A dependency injection container is ideal for creating view-model instances. If a view-model has dependencies, it will manage the creation and injection of any required services. Just ensure that you register your view-models and any dependencies that they may have in the `CreateMauiApp` method in the `MauiProgram` class.

In a Shell app, you don't need to register your pages with the dependency injection container unless you want to influence the lifetime of the page relative to the container with the [AddSingleton](#), [AddTransient](#), or [AddScoped](#) methods. For more information, see [Dependency lifetime](#).

Dependency lifetime

Depending on the needs of your app, you may need to register dependencies with different lifetimes. The following table lists the main methods you can use to register dependencies, and their registration lifetimes:

 [Expand table](#)

Method	Description
AddSingleton<T>	Creates a single instance of the object which will remain for the lifetime of the app.

Method	Description
AddTransient<T>	Creates a new instance of the object when requested during resolution. Transient objects do not have a pre-defined lifetime, but will typically follow the lifetime of their host.
AddScoped<T>	Creates an instance of the object that shares the lifetime of its host. When the host goes out of scope, so does its dependency. Therefore, resolving the same dependency multiple times within the same scope yields the same instance, while resolving the same dependency in different scopes will yield different instances.

ⓘ Note

If an object doesn't inherit from an interface, such as a view or view-model, only its concrete type needs to be provided to the [AddSingleton<T>](#), [AddTransient<T>](#), or [AddScoped<T>](#) method.

The `MainPageViewModel1` class is used near the app's root and should always be available, so registering it with [AddSingleton<T>](#) is beneficial. Other view-models may be situationally navigated to or used later in an app. If you have a type that might not always be used, or if it's memory or computationally intensive or requires just-in-time data, it may be a better candidate for [AddTransient<T>](#) registration.

Another common way to register dependencies is using the [AddSingleton<TService, TImplementation>](#), [AddTransient<TService, TImplementation>](#), or [AddScoped<TService, TImplementation>](#) methods. These methods take two types - the interface definition and the concrete implementation. This type of registration is best for cases where you are implementing services based on interfaces.

Once all types have been registered, [MauiAppBuilder.Build\(\)](#) should be called to create the [MauiApp](#) object and populate the dependency injection container with all the registered types.

ⓘ Important

Once [MauiAppBuilder.Build\(\)](#) has been called, the types registered with the dependency injection container will be immutable and no longer can be updated or modified.

Register dependencies with an extension method

The [MauiApp.CreateBuilder](#) method creates a [MauiAppBuilder](#) object that can be used to register dependencies. If your app needs to register many dependencies, you can create extension methods to help provide an organized and maintainable registration workflow:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    => MauiApp.CreateBuilder()
        .UseMauiApp<App>()
        .RegisterServices()
        .RegisterViewModels()
        .RegisterViews()
        .Build();

    public static MauiAppBuilder RegisterServices(this MauiAppBuilder
mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddTransient<ILoggingService,
LoggingService>();
        mauiAppBuilder.Services.AddTransient<ISettingsService,
SettingsService>();

        // More services registered here.

        return mauiAppBuilder;
    }

    public static MauiAppBuilder RegisterViewModels(this MauiAppBuilder
mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddSingleton<MainPageViewModel>();

        // More view-models registered here.

        return mauiAppBuilder;
    }

    public static MauiAppBuilder RegisterViews(this MauiAppBuilder
mauiAppBuilder)
    {
        mauiAppBuilder.Services.AddSingleton<MainPage>();

        // More views registered here.

        return mauiAppBuilder;
    }
}
```


In this example, the three registration extension methods use the [MauiAppBuilder](#) instance to access the [Services](#) property to register dependencies.

Resolution

After a type is registered, it can be resolved or injected as a dependency. When a type is being resolved, and the container needs to create a new instance, it injects any dependencies into the instance.

Generally, when a type is resolved, one of three scenarios happen:

1. If the type hasn't been registered, the container throws an exception.
2. If the type has been registered as a singleton, the container returns the singleton instance. If this is the first time the type is called for, the container creates it if required and maintains a reference to it.
3. If the type has been registered as transient, the container returns a new instance and doesn't maintain a reference to it.

.NET MAUI supports *automatic* and *explicit* dependency resolution. Automatic dependency resolution uses constructor injection without explicitly requesting the dependency from the container. Explicit dependency resolution occurs on demand by explicitly requesting a dependency from the container.

Automatic dependency resolution

Automatic dependency resolution occurs in apps that use [.NET MAUI Shell](#), provided that you've registered the dependency's type and the type that uses the dependency with the dependency injection container.

During Shell-based navigation, .NET MAUI will look for page registrations, and if any are found, it will create that page and inject any dependencies into its constructor:

```
C#  
  
public MainPage(MainPageViewModel viewModel)  
{  
    InitializeComponent();  
  
    BindingContext = viewModel;  
}
```

In this example, the `MainPage` constructor receives a `MainPageViewModel` instance that's injected. In turn, the `MainPageViewModel` instance has `ILoggingService` and

`ISettingsService` instances injected:

C#

```
public class MainPageViewModel
{
    readonly ILoggerService _loggingService;
    readonly ISettingsService _settingsService;

    public MainPageViewModel(ILoggerService loggingService,
        ISettingsService settingsService)
    {
        _loggingService = loggingService;
        _settingsService = settingsService;
    }
}
```

In addition, in a Shell-based app, .NET MAUI will inject dependencies into detail pages that are registered with the [Routing.RegisterRoute](#) method.

Explicit dependency resolution

A Shell-based app can't use constructor injection when a type only exposes a parameterless constructor. Alternatively, if your app doesn't use Shell then you'll need to use explicit dependency resolution.

The dependency injection container can be explicitly accessed from an [Element](#) through its [Handler.MauiContext.Services](#) property, which is of type [IServiceProvider](#):

C#

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();

        Handler.Changed += OnHandlerChanged;
    }

    void OnHandlerChanged(object sender, EventArgs e)
    {
        BindingContext =
        Handler.MauiContext.Services.GetService<MainPageViewModel>();
    }
}
```

This approach can be useful if you need to resolve a dependency from an [Element](#), or from outside the constructor of an [Element](#). In this example, accessing the dependency injection container in the `HandlerChanged` event handler ensures that a handler has been set for the page, and therefore that the `Handler` property won't be `null`.

⚠ Warning

The `Handler` property of your `Element` could be `null`, so be aware that you may need to account for this situation. For more information, see [Handler lifecycle](#).

In a view-model, the dependency injection container can be explicitly accessed through the `Handler.MauiContext.Services` property of the `Window.Page`:

C#

```
public class MainPageViewModel
{
    readonly ILoggerService _loggingService;
    readonly ISettingsService _settingsService;

    public MainPageViewModel()
    {
        _loggingService =
            Application.Current.Windows[0].Page.Handler.MauiContext.Services.GetService<
            ILoggerService>();
        _settingsService =
            Application.Current.Windows[0].Page.Handler.MauiContext.Services.GetService<
            ISettingsService>();
    }
}
```

A drawback of this approach is that the view-model now has a dependency on the [Application](#) type. However, this drawback can be eliminated by passing an [IServiceProvider](#) argument to the view-model constructor. The [IServiceProvider](#) is resolved through automatic dependency resolution without having to register it with the dependency injection container. With this approach a type and its [IServiceProvider](#) dependency can be automatically resolved provided that the type is registered with the dependency injection container. The [IServiceProvider](#) can then be used for explicit dependency resolution:

C#

```
public class MainPageViewModel
{
    readonly ILoggerService _loggingService;
    readonly ISettingsService _settingsService;
```

```

public MainPageViewModel(IServiceProvider serviceProvider)
{
    _loggingService = serviceProvider.GetService<ILoggingService>();
    _settingsService = serviceProvider.GetService<ISettingsService>();
}
}

```

In addition, an [IServiceProvider](#) instance can be accessed on each platform through the `IPlatformApplication.Current.Services` property.

Limitations with XAML resources

A common scenario is to register a page with the dependency injection container, and use automatic dependency resolution to inject it into the `App` constructor and set it as the first page to be displayed in the app:

C#

```

MyFirstAppPage _firstPage;

public App(MyFirstAppPage page)
{
    InitializeComponent();
    _firstPage = page;
}

protected override Window CreateWindow(IActivationState? activationState)
{
    return new Window(_firstPage);
}

```

However, in this scenario if `MyFirstAppPage` attempts to access a `StaticResource` that's been declared in XAML in the `App` resource dictionary, a [XamlParseException](#) will be thrown with a message similar to `Position {row}:{column}. StaticResource not found for key {key}`. This occurs because the page resolved through constructor injection has been created before the application-level XAML resources have been initialized.

A workaround for this issue is to inject an [IServiceProvider](#) into your `App` class and then use it to resolve the page inside the `App` class:

C#

```

MyFirstAppPage _firstPage;

public App(IServiceProvider serviceProvider)

```

```
{  
    InitializeComponent();  
    _firstPage = serviceProvider.GetService<MyFirstAppPage>();  
}  
  
protected override Window CreateWindow(IActivationState? activationState)  
{  
    return new Window(_firstPage);  
}
```

This approach forces the XAML object tree to be created and initialized before the page is resolved.

Recognize a drag and drop gesture

Article • 11/06/2024

A .NET Multi-platform App UI (.NET MAUI) drag and drop gesture recognizer enables items, and their associated data packages, to be dragged from one onscreen location to another location using a continuous gesture. Drag and drop can take place in a single application, or it can start in one application and end in another.

The *drag source*, which is the element on which the drag gesture is initiated, can provide data to be transferred by populating a data package object. When the drag source is released, drop occurs. The *drop target*, which is the element under the drag source, then processes the data package.

The process for enabling drag and drop in an app is as follows:

1. Enable drag on an element by adding a [DragGestureRecognizer](#) object to its [GestureRecognizers](#) collection. For more information, see [Enable drag](#).
2. [optional] Build a data package. .NET MAUI automatically populates the data package for image and text controls, but for other content you'll need to construct your own data package. For more information, see [Build a data package](#).
3. Enable drop on an element by adding a [DropGestureRecognizer](#) object to its [GestureRecognizers](#) collection. For more information, see [Enable drop](#).
4. [optional] Handle the `DropGestureRecognizer.DragOver` event to indicate the type of operation allowed by the drop target. For more information, see [Handle the DragOver event](#).
5. [optional] Process the data package to receive the dropped content. .NET MAUI will automatically retrieve image and text data from the data package, but for other content you'll need to process the data package. For more information, see [Process the data package](#).

Enable drag

In .NET MAUI, drag gesture recognition is provided by the [DragGestureRecognizer](#) class. This class defines the following properties:

- [CanDrag](#), of type `bool`, which indicates whether the element the gesture recognizer is attached to can be a drag source. The default value of this property is `true`.
- [DragStartingCommand](#), of type [ICommand](#), which is executed when a drag gesture is first recognized.

- [DragStartingCommandParameter](#), of type `object`, which is the parameter that's passed to the [DragStartingCommand](#).
- [DropCompletedCommand](#), of type `ICommand`, which is executed when the drag source is dropped.
- [DropCompletedCommandParameter](#), of type `object`, which is the parameter that's passed to the [DropCompletedCommand](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [DragGestureRecognizer](#) class also defines [DragStarting](#) and [DropCompleted](#) events that fire if the [CanDrag](#) property is `true`. When a [DragGestureRecognizer](#) object detects a drag gesture, it executes the [DragStartingCommand](#) and invokes the [DragStarting](#) event. Then, when the [DragGestureRecognizer](#) object detects the completion of a drop gesture, it executes the [DropCompletedCommand](#) and invokes the [DropCompleted](#) event.

The [DragStartingEventArgs](#) object that accompanies the [DragStarting](#) event defines the following properties:

- [Cancel](#), of type `bool`, indicates whether the event should be canceled.
- [Data](#), of type [DataPackage](#), indicates the data package that accompanies the drag source. This is a read-only property.
- [PlatformArgs](#), of type `PlatformDragStartingEventArgs?`, represents the platform-specific arguments associated with the event.

Windows

On Windows, the [PlatformDragStartingEventArgs](#) class defines the following properties:

- [Sender](#), of type [FrameworkElement](#), represents the native view attached to the event.
- [DragStartingEventArgs](#), of type [DragStartingEventArgs](#), provides event data for the native event.
- [Handled](#), of type `bool`, determines if the event arguments have changed. This property should be set to `true` when changing the [DragStartingEventArgs](#) so that the changes aren't overridden.

The [DropCompletedEventArgs](#) object that accompanies the [DropCompleted](#) event defines a [PlatformArgs](#) property, of type `PlatformDropCompletedEventArgs?`, which

represents the platform-specific arguments associated with the event.

Windows

On Windows, the [PlatformDropCompletedEventArgs](#) class defines the following properties:

- `Sender`, of type [FrameworkElement](#), represents the native view attached to the event.
- `DropCompletedEventArgs`, of type [DropCompletedEventArgs](#), provides event data for the native event.

The following XAML example shows a [DragGestureRecognizer](#) attached to an [Image](#):

XAML

```
<Image Source="monkeyface.png">
  <Image.GestureRecognizers>
    <DragGestureRecognizer />
  </Image.GestureRecognizers>
</Image>
```

In this example, a drag gesture can be initiated on the [Image](#).

💡 Tip

A drag gesture is initiated with a long-press followed by a drag.

Build a data package

.NET MAUI will automatically build a data package for you, when a drag is initiated, for the following controls:

- Text controls. Text values can be dragged from [CheckBox](#), [DatePicker](#), [Editor](#), [Entry](#), [Label](#), [RadioButton](#), [Switch](#), and [TimePicker](#) objects.
- Image controls. Images can be dragged from [Button](#), [Image](#), and [ImageButton](#) controls.

The following table shows the properties that are read, and any conversion that's attempted, when a drag is initiated on a text control:

Control	Property	Conversion
CheckBox	<code>IsChecked</code>	<code>bool</code> converted to a <code>string</code> .
DatePicker	<code>Date</code>	<code>DateTime</code> converted to a <code>string</code> .
Editor	<code>Text</code>	
Entry	<code>Text</code>	
Label	<code>Text</code>	
RadioButton	<code>IsChecked</code>	<code>bool</code> converted to a <code>string</code> .
Switch	<code>IsToggled</code>	<code>bool</code> converted to a <code>string</code> .
TimePicker	<code>Time</code>	<code>TimeSpan</code> converted to a <code>string</code> .

For content other than text and images, you'll need to build a data package yourself.

Data packages are represented by the [DataPackage](#) class, which defines the following properties:

- [Properties](#), of type [DataPackagePropertySet](#), which is a collection of properties that comprise the data contained in the [DataPackage](#). This property is a read-only property.
- [Image](#), of type [ImageSource](#), which is the image contained in the [DataPackage](#).
- [Text](#), of type `string`, which is the text contained in the [DataPackage](#).
- [View](#), of type [DataPackageView](#), which is a read-only version of the [DataPackage](#).

The [DataPackagePropertySet](#) class represents a property bag stored as a `Dictionary<string,object>`. For information about the [DataPackageView](#) class, see [Process the data package](#).

Store image or text data

Image or text data can be associated with a drag source by storing the data in the `DataPackage.Image` or `DataPackage.Text` property. You can add the data in the handler for the [DragStarting](#) event.

The following XAML example shows a [DragGestureRecognizer](#) that registers a handler for the [DragStarting](#) event:

XAML

```

<Path Stroke="Black"
      StrokeThickness="4">
  <Path.GestureRecognizers>
    <DragGestureRecognizer DragStarting="OnDragStarting" />
  </Path.GestureRecognizers>
  <Path.Data>
    <!-- PathGeometry goes here -->
  </Path.Data>
</Path>

```

In this example, the [DragGestureRecognizer](#) is attached to a [Path](#) object. The [DragStarting](#) event is raised when a drag gesture is detected on the [Path](#), which executes the `OnDragStarting` event handler:

```

C#

void OnDragStarting(object sender, DragStartingEventArgs e)
{
    e.Data.Text = "My text data goes here";
}

```

The [DragStartingEventArgs](#) object that accompanies the [DragStarting](#) event has a `Data` property, of type [DataPackage](#). In this example, the `Text` property of the [DataPackage](#) object is set to a `string`. The [DataPackage](#) can then be accessed on drop, to retrieve the `string`.

Store data in the property bag

Any data, including images and text, can be associated with a drag source by storing the data in the `DataPackage.Properties` collection. You can add the data in the handler for the [DragStarting](#) event.

The following XAML example shows a [DragGestureRecognizer](#) that registers a handler for the [DragStarting](#) event:

```

XAML

<Rectangle Stroke="Red"
          Fill="DarkBlue"
          StrokeThickness="4"
          HeightRequest="200"
          WidthRequest="200">
  <Rectangle.GestureRecognizers>
    <DragGestureRecognizer DragStarting="OnDragStarting" />
  </Rectangle.GestureRecognizers>
</Rectangle>

```

```
</Rectangle.GestureRecognizers>
</Rectangle>
```

In this example, the [DragGestureRecognizer](#) is attached to a [Rectangle](#) object. The [DragStarting](#) event is raised when a drag gesture is detected on the [Rectangle](#), which executes the `OnDragStarting` event handler:

C#

```
void OnDragStarting(object sender, DragStartingEventArgs e)
{
    Shape shape = (sender as Element).Parent as Shape;
    e.Data.Properties.Add("Square", new Square(shape.Width, shape.Height));
}
```

The [DragStartingEventArgs](#) object that accompanies the [DragStarting](#) event has a `Data` property, of type [DataPackage](#). The `Properties` collection of the [DataPackage](#) object, which is a `Dictionary<string, object>` collection, can be modified to store any required data. In this example, the `Properties` dictionary is modified to store a [Square](#) object that represents the size of the [Rectangle](#) against a "Square" key.

Enable drop

In .NET MAUI, drop gesture recognition is provided by the [DropGestureRecognizer](#) class. This class defines the following properties:

- [AllowDrop](#), of type `bool`, which indicates whether the element the gesture recognizer is attached to can be a drop target. The default value of this property is `true`.
- [DragOverCommand](#), of type [ICommand](#), which is executed when the drag source is dragged over the drop target.
- [DragOverCommandParameter](#), of type `object`, which is the parameter that's passed to the `DragOverCommand`.
- [DragLeaveCommand](#), of type [ICommand](#), which is executed when the drag source is dragged off the drop target.
- [DragLeaveCommandParameter](#), of type `object`, which is the parameter that's passed to the `DragLeaveCommand`.
- [DropCommand](#), of type [ICommand](#), which is executed when the drag source is dropped over the drop target.
- [DropCommandParameter](#), of type `object`, which is the parameter that's passed to the `DropCommand`.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [DropGestureRecognizer](#) class also defines [DragOver](#), [DragLeave](#), and [Drop](#) events that fire if the [AllowDrop](#) property is `true`. When a [DropGestureRecognizer](#) recognizes a drag source over the drop target, it executes the [DragOverCommand](#) and invokes the [DragOver](#) event. Then, if the drag source is dragged off the drop target, the [DropGestureRecognizer](#) executes the [DragLeaveCommand](#) and invokes the [DragLeave](#) event. Finally, when the [DropGestureRecognizer](#) recognizes a drop gesture over the drop target, it executes the [DropCommand](#) and invokes the [Drop](#) event.

The [DragEventArgs](#) class, which accompanies the [DragOver](#) and [DragLeave](#) events, defines the following properties:

- [Data](#), of type [DataPackage](#), which contains the data associated with the drag source. This property is read-only.
- [AcceptedOperation](#), of type [DataPackageOperation](#), which specifies which operations are allowed by the drop target.
- [PlatformArgs](#), of type `PlatformDragEventArgs?`, represents the platform-specific arguments associated with the event.

Windows

On Windows, the [PlatformDragEventArgs](#) class defines the following properties:

- `Sender`, of type [FrameworkElement](#), represents the native view attached to the event.
- `DragEventArgs`, of type [DragEventArgs](#), provides event data for the native event.
- `Handled`, of type `bool`, determines if the event arguments have changed. This property should be set to `true` when changing the `DragEventArgs` so that the changes aren't overridden.

For example, the [DragEventArgs](#) property can be used to access native properties:

C#

```
void OnDragOver(object sender, DragEventArgs e)
{
    #if WINDOWS
        var dragUI = e.PlatformArgs.DragEventArgs.DragUIOverride;
        dragUI.IsCaptionVisible = false;
        dragUI.IsGlyphVisible = false;
    #endif
}
```

```
#endif  
}
```

In this example, the drag glyph is disabled and caption text that overlays the drag visual is also disabled.

For information about the [DataPackageOperation](#) enumeration, see [Handle the DragOver event](#).

The [DropEventArgs](#) class that accompanies the [Drop](#) event defines the following properties:

- [Data](#), of type [DataPackageView](#), which is a read-only version of the data package.
- [Handled](#), of type `bool`, indicates whether the event handler has handled the event or whether .NET MAUI should continue its own processing.
- [PlatformArgs](#), of type `PlatformDropEventArgs?`, represents the platform-specific arguments associated with the event.

Windows

On Windows, the [PlatformDropEventArgs](#) class defines the following properties:

- [Sender](#), of type [FrameworkElement](#), represents the native view attached to the event.
- [DragEventArgs](#), of type [DragEventArgs](#), provides event data for the native event. -->

The following XAML example shows a [DropGestureRecognizer](#) attached to an [Image](#):

XAML

```
<Image BackgroundColor="Silver"  
        HeightRequest="300"  
        WidthRequest="250">  
    <Image.GestureRecognizers>  
        <DropGestureRecognizer />  
    </Image.GestureRecognizers>  
</Image>
```

In this example, when a drag source is dropped on the [Image](#) drop target, the drag source will be copied to the drop target if the drag source is an [ImageSource](#). .NET MAUI automatically copies dragged images, and text, to compatible drop targets.

Handle the DragOver event

The `DropGestureRecognizer.DragOver` event can be optionally handled to indicate which type of operations are allowed by the drop target. You can indicate the allowable operations by setting the `AcceptedOperation` property, of type `DataPackageOperation`, on the `DragEventArgs` object that accompanies the `DragOver` event.

The `DataPackageOperation` enumeration defines the following members:

- `None`, indicates that no action will be performed.
- `Copy`, indicates that the drag source content will be copied to the drop target.

Important

When a `DragEventArgs` object is created, the `AcceptedOperation` property defaults to `DataPackageOperation.Copy`.

The following XAML example shows a `DropGestureRecognizer` that registers a handler for the `DragOver` event:

XAML

```
<Image BackgroundColor="Silver"
        HeightRequest="300"
        WidthRequest="250">
    <Image.GestureRecognizers>
        <DropGestureRecognizer DragOver="OnDragOver" />
    </Image.GestureRecognizers>
</Image>
```

In this example, the `DropGestureRecognizer` is attached to an `Image` object. The `DragOver` event is raised when a drag source is dragged over the drop target, but hasn't been dropped, which executes the `OnDragOver` event handler:

C#

```
void OnDragOver(object sender, DragEventArgs e)
{
    e.AcceptedOperation = DataPackageOperation.None;
}
```

In this example, the `AcceptedOperation` property of the `DragEventArgs` object is set to `DataPackageOperation.None`. This value ensures that no action is taken when a drag

source is dropped over the drop target.

Process the data package

The [Drop](#) event is raised when a drag source is released over a drop target. .NET MAUI automatically attempts to retrieve data from the data package when a drag source is dropped onto the following controls:

- Text controls. Text values can be dropped onto [CheckBox](#), [DatePicker](#), [Editor](#), [Entry](#), [Label](#), [RadioButton](#), [Switch](#), and [TimePicker](#) objects.
- Image controls. Images can be dropped onto [Button](#), [Image](#), and [ImageButton](#) controls.

The following table shows the properties that are set and any conversion that's attempted when a text-based drag source is dropped on a text control:

 Expand table

Control	Property	Conversion
CheckBox	IsChecked	string is converted to a bool.
DatePicker	Date	string is converted to a DateTime.
Editor	Text	
Entry	Text	
Label	Text	
RadioButton	IsChecked	string is converted to a bool.
Switch	IsToggled	string is converted to a bool.
TimePicker	Time	string is converted to a TimeSpan.

For content other than text and images, you'll need to process the data package yourself.

The [DropEventArgs](#) class that accompanies the [Drop](#) event defines a `Data` property, of type [DataPackageView](#). This property represents a read-only version of the data package.

Retrieve image or text data

Image or text data can be retrieved from a data package in the handler for the [Drop](#) event, using methods defined in the [DataPackageView](#) class.

The [DataPackageView](#) class includes `GetImageAsync` and `GetTextAsync` methods. The `GetImageAsync` method retrieves an image from the data package that was stored in the `DataPackage.Image` property and returns `Task<ImageSource>`. Similarly, the `GetTextAsync` method retrieves text from the data package that was stored in the `DataPackage.Text` property and returns `Task<string>`.

The following example shows a `Drop` event handler that retrieves text from the data package for a [Path](#):

C#

```
async void OnDrop(object sender, DropEventArgs e)
{
    string text = await e.Data.GetTextAsync();

    // Perform logic to take action based on the text value.
}
```

In this example, text data is retrieved from the data package using the `GetTextAsync` method. An action based on the text value can then be taken.

Retrieve data from the property bag

Any data can be retrieved from a data package in the handler for the [Drop](#) event, by accessing the `Properties` collection of the data package.

The [DataPackageView](#) class defines a `Properties` property, of type `DataPackagePropertySetView`. The `DataPackagePropertySetView` class represents a read-only property bag stored as a `Dictionary<string, object>`.

The following example shows a [Drop](#) event handler that retrieves data from the property bag of a data package for a [Rectangle](#):

C#

```
void OnDrop(object sender, DropEventArgs e)
{
    Square square = (Square)e.Data.Properties["Square"];

    // Perform logic to take action based on retrieved value.
}
```


In this example, the `Square` object is retrieved from the property bag of the data package, by specifying the "Square" dictionary key. An action based on the retrieved value can then be taken.

Drag and drop between applications

On iOS, Mac Catalyst, and Windows, drag can start in one application with the corresponding drop operation ending in a .NET MAUI application. The app from which an item is dragged is the *source* application, and the .NET MAUI app on which an item is dropped is the *destination* application.

Windows

The following example shows an event handler for the [DropGestureRecognizer.Drop](#) event that processes an item that's been dragged from a *source* application to your *destination* application:

C#

```
#if WINDOWS
using Windows.ApplicationModel.DataTransfer;
using Windows.Storage;
#endif

async void OnDropGestureRecognizerDrop(object? sender, DropEventArgs e)
{
    var filePaths = new List<string>();

    #if WINDOWS
        if (e.PlatformArgs is not null &&
            e.PlatformArgs.DragEventArgs.DataView.Contains(StandardDataFormats.StorageItems))
        {
            var items = await
                e.PlatformArgs.DragEventArgs.DataView.GetStorageItemsAsync();
            if (items.Any())
            {
                foreach (var item in items)
                {
                    if (item is StorageFile file)
                        filePaths.Add(item.Path);
                }
            }
        }
    #endif

    string filePath = filePaths.FirstOrDefault();
```

```
// Process the dropped file  
}
```

Get the gesture position

The position at which a drag or drop gesture occurred can be obtained by calling the [GetPosition](#) method on a [DragEventArgs](#), [DragStartingEventArgs](#), or [DropEventArgs](#) object. The [GetPosition](#) method accepts an `Element?` argument, and returns a position as a `Point?` object:

C#

```
void OnDragStarting(object sender, DragStartingEventArgs e)  
{  
    // Position relative to screen  
    Point? screenPosition = e.GetPosition(null);  
  
    // Position relative to specified element  
    Point? relativeToImagePosition = e.GetPosition(image);  
}
```

The `Element?` argument defines the element the position should be obtained relative to. Supplying a `null` value as this argument means that the [GetPosition](#) method returns a `Point?` object that defines the position of the drag or drop gesture relative to the screen.

Recognize a pan gesture

Article • 06/19/2023

A .NET Multi-platform App UI (.NET MAUI) pan gesture recognizer detects the movement of fingers around the screen and can be used to apply that movement to content. A typical scenario for the pan gesture is to horizontally and vertically pan an image, so that all of the image content can be viewed when it's being displayed in a viewport smaller than the image dimensions. This is accomplished by moving the image within the viewport.

In .NET MAUI, pan gesture recognition is provided by the [PanGestureRecognizer](#) class. This class defines the [TouchPoints](#) property, of type `int`, which represents the number of touch points in the gesture. The default value of this property is 1. This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

The [PanGestureRecognizer](#) class also defines a [PanUpdated](#) event that's raised when the detected pan gesture changes. The [PanUpdatedEventArgs](#) object that accompanies this event defines the following properties:

- [GestureId](#), of type `int`, which represents the id of the gesture that raised the event.
- [StatusType](#), of type `GestureStatus`, which indicates if the event has been raised for a newly started gesture, a running gesture, a completed gesture, or a canceled gesture.
- [TotalX](#), of type `double`, which indicates the total change in the X direction since the beginning of the gesture.
- [TotalY](#), of type `double`, which indicates the total change in the Y direction since the beginning of the gesture.

Create a PanGestureRecognizer

To make a [View](#) recognize a pan gesture, create a [PanGestureRecognizer](#) object, handle the [PanUpdated](#) event, and add the new gesture recognizer to the `GestureRecognizers` collection on the view. The following code example shows a [PanGestureRecognizer](#) attached to an [Image](#):

XAML

```
<Image Source="monkey.jpg">  
    <Image.GestureRecognizers>
```

```
<PanGestureRecognizer PanUpdated="OnPanUpdated" />
</Image.GestureRecognizers>
</Image>
```

The code for the `OnPanUpdated` event handler should be added to the code-behind file:

C#

```
void OnPanUpdated(object sender, PanUpdatedEventArgs e)
{
    // Handle the pan
}
```

The equivalent C# code is:

C#

```
PanGestureRecognizer panGesture = new PanGestureRecognizer();
panGesture.PanUpdated += (s, e) =>
{
    // Handle the pan
};
image.GestureRecognizers.Add(panGesture);
```

Create a pan container

Freeform panning is typically suited to navigating within images and maps. The `PanContainer` class, which is shown in the following example, is a generalized helper class that performs freeform panning:

C#

```
public class PanContainer : ContentView
{
    double panX, panY;

    public PanContainer()
    {
        // Set PanGestureRecognizer.TouchPoints to control the
        // number of touch points needed to pan
        PanGestureRecognizer panGesture = new PanGestureRecognizer();
        panGesture.PanUpdated += OnPanUpdated;
        GestureRecognizers.Add(panGesture);
    }

    void OnPanUpdated(object sender, PanUpdatedEventArgs e)
    {
        switch (e.StatusType)
        {
```

```

    {
        case GestureStatus.Running:
            // Translate and pan.
            double boundsX = Content.Width;
            double boundsY = Content.Height;
            Content.TranslationX = Math.Clamp(panX + e.TotalX, -boundsX,
boundsX);
            Content.TranslationY = Math.Clamp(panY + e.TotalY, -boundsY,
boundsY);
            break;

        case GestureStatus.Completed:
            // Store the translation applied during the pan
            panX = Content.TranslationX;
            panY = Content.TranslationY;
            break;
    }
}
}

```

In this example, the `OnPanUpdated` method updates the viewable content of the wrapped view, based on the user's pan gesture. This is achieved by using the values of the `TotalX` and `TotalY` properties of the `PanUpdatedEventArgs` instance to calculate the direction and distance of the pan. The wrapped user element is then panned by setting its `TranslationX` and `TranslationY` properties to the calculated values. When panning content in an element that does not occupy the full screen, the height and width of the viewport can be obtained from the element's `Height` and `Width` properties.

The `PanContainer` class can be wrapped around a `View` so that a recognized pan gesture will pan the wrapped view. The following XAML example shows the `PanContainer` wrapping an `Image`:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:PanGesture"
    x:Class="PanGesture.MainPage">
    <AbsoluteLayout>
        <local:PanContainer>
            <Image Source="monkey.jpg" WidthRequest="1024"
HeightRequest="768" />
        </local:PanContainer>
    </AbsoluteLayout>
</ContentPage>

```

In this example, when the `Image` receives a pan gesture, the displayed image will be panned.

Recognize a pinch gesture

Article • 02/09/2023

A .NET Multi-platform App UI (.NET MAUI) pinch gesture recognizer is used for performing interactive zoom. A common scenario for the pinch gesture is to perform interactive zoom of an image at the pinch location. This is accomplished by scaling the content of the viewport.

In .NET MAUI, pinch gesture recognition is provided by the [PinchGestureRecognizer](#) class, which defines a [PinchUpdated](#) event that's raised when the detected pinch gesture changes. The [PinchGestureUpdatedEventArgs](#) object that accompanies the [PinchUpdated](#) event defines the following properties:

- [Scale](#), of type `double`, which indicates the relative size of the pinch gesture since the last update was received.
- [ScaleOrigin](#), of type `Point`, which indicates the updated origin of the pinch's gesture.
- [Status](#), of type [GestureStatus](#), which indicates if the event has been raised for a newly started gesture, a running gesture, a completed gesture, or a canceled gesture.

Create a PinchGestureRecognizer

To make a [View](#) recognize a pinch gesture, create a [PinchGestureRecognizer](#) object, handle the [PinchUpdated](#) event, and add the new gesture recognizer to the `GestureRecognizers` collection on the view. The following code example shows a [PinchGestureRecognizer](#) attached to an [Image](#):

XAML

```
<Image Source="waterfront.jpg">
  <Image.GestureRecognizers>
    <PinchGestureRecognizer PinchUpdated="OnPinchUpdated" />
  </Image.GestureRecognizers>
</Image>
```

The code for the `OnPinchUpdated` event handler should be added to the code-behind file:

C#

```

void OnPinchUpdated(object sender, PinchGestureUpdatedEventArgs e)
{
    // Handle the pinch
}

```

The equivalent C# code is:

C#

```

PinchGestureRecognizer pinchGesture = new PinchGestureRecognizer();
pinchGesture.PinchUpdated += (s, e) =>
{
    // Handle the pinch
};
image.GestureRecognizers.Add(pinchGesture);

```

Create a pinch container

The `PinchToZoomContainer` class, which is shown in the following example, is a generalized helper class that can be used to interactively zoom a [View](#):

C#

```

public class PinchToZoomContainer : ContentView
{
    double currentScale = 1;
    double startScale = 1;
    double xOffset = 0;
    double yOffset = 0;

    public PinchToZoomContainer()
    {
        PinchGestureRecognizer pinchGesture = new PinchGestureRecognizer();
        pinchGesture.PinchUpdated += OnPinchUpdated;
        GestureRecognizers.Add(pinchGesture);
    }

    void OnPinchUpdated(object sender, PinchGestureUpdatedEventArgs e)
    {
        if (e.Status == GestureStatus.Started)
        {
            // Store the current scale factor applied to the wrapped user
            // interface element,
            // and zero the components for the center point of the translate
            // transform.
            startScale = Content.Scale;
            Content.AnchorX = 0;
            Content.AnchorY = 0;
        }
    }
}

```



```

if (e.Status == GestureStatus.Running)
{
    // Calculate the scale factor to be applied.
    currentScale += (e.Scale - 1) * startScale;
    currentScale = Math.Max(1, currentScale);

    // The ScaleOrigin is in relative coordinates to the wrapped
user interface element,
    // so get the X pixel coordinate.
    double renderedX = Content.X + xOffset;
    double deltaX = renderedX / Width;
    double deltaWidth = Width / (Content.Width * startScale);
    double originX = (e.ScaleOrigin.X - deltaX) * deltaWidth;

    // The ScaleOrigin is in relative coordinates to the wrapped
user interface element,
    // so get the Y pixel coordinate.
    double renderedY = Content.Y + yOffset;
    double deltaY = renderedY / Height;
    double deltaHeight = Height / (Content.Height * startScale);
    double originY = (e.ScaleOrigin.Y - deltaY) * deltaHeight;

    // Calculate the transformed element pixel coordinates.
    double targetX = xOffset - (originX * Content.Width) *
(currentScale - startScale);
    double targetY = yOffset - (originY * Content.Height) *
(currentScale - startScale);

    // Apply translation based on the change in origin.
    Content.TranslationX = Math.Clamp(targetX, -Content.Width *
(currentScale - 1), 0);
    Content.TranslationY = Math.Clamp(targetY, -Content.Height *
(currentScale - 1), 0);

    // Apply scale factor
    Content.Scale = currentScale;
}
if (e.Status == GestureStatus.Completed)
{
    // Store the translation delta's of the wrapped user interface
element.
    xOffset = Content.TranslationX;
    yOffset = Content.TranslationY;
}
}
}

```

In this example, the `OnPinchUpdated` method updates the zoom level of the wrapped view, based on the user's pinch gesture. This is achieved by using the values of the `Scale`, `ScaleOrigin` and `Status` properties of the `PinchGestureUpdatedEventArgs` object to calculate the scale factor to be applied at the origin of the pinch gesture. The wrapped

view is then zoomed at the origin of the pinch gesture by setting its `TranslationX`, `TranslationY`, and `Scale` properties to the calculated values.

The `PinchToZoomContainer` class can be wrapped around a `View` so that a recognized pinch gesture will zoom the wrapped view. The following XAML example shows the `PinchToZoomContainer` wrapping an `Image`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:PinchGesture;assembly=PinchGesture"
             x:Class="PinchGesture.HomePage">
    <Grid>
        <local:PinchToZoomContainer>
            <Image Source="waterfront.jpg" />
        </local:PinchToZoomContainer>
    </Grid>
</ContentPage>
```

In this example, when the `Image` receives a pinch gesture, the displayed image will be zoomed-in or out.

Recognize a pointer gesture

Article • 05/14/2024

A .NET Multi-platform App UI (.NET MAUI) pointer gesture recognizer detects when the pointer enters, exits, and moves within a view and is implemented with the [PointerGestureRecognizer](#) class. This class defines the following properties:

- [PointerEnteredCommand](#), of type [ICommand](#), which is the command to invoke when the pointer enters the bounding area of the view.
- [PointerEnteredCommandParameter](#), of type `object`, which is the parameter that's passed to [PointerEnteredCommand](#).
- [PointerExitedCommand](#), of type [ICommand](#), which is the command to invoke when the pointer that's in the view's bounding area leaves that bounding area.
- [PointerExitedCommandParameter](#), of type `object`, which is the parameter that's passed to [PointerExitedCommand](#).
- [PointerMovedCommand](#), of type [ICommand](#), which is the command to invoke when the pointer moves while remaining within the bounding area of the view.
- [PointerMovedCommandParameter](#), of type `object`, which is the parameter that's passed to [PointerMovedCommand](#).
- [PointerPressedCommand](#), of type [ICommand](#), which is the command to invoke when the pointer initiates a press within the view.
- [PointerPressedCommandParameter](#), of type `object`, which is the parameter that's passed to the [PointerPressedCommand](#).
- [PointerReleasedCommand](#), of type [ICommand](#), which is the command to invoke when the pointer that has previously initiated a press is released, while within the view.
- [PointerReleasedCommandParameter](#), of type `object`, which is the parameter that's passed to the [PointerReleasedCommand](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [PointerGestureRecognizer](#) class also defines the following events:

- [PointerEntered](#), that's raised when the pointer enters the bounding area of the view.
- [PointerExited](#), that's raised when the pointer that's in the view's bounding area leaves that bounding area.
- [PointerMoved](#), that's raised when the pointer moves while remaining within the bounding area of the view.
- [PointerPressed](#), that's raised when the pointer initiates a press within the view.

- [PointerReleased](#), that's raised when the pointer that has previously initiated a press is released, while within the view.

A [PointerEventArgs](#) object accompanies the events, and defines a [PlatformArgs](#) property of type [PlatformPointerEventArgs](#) that provides access to platform-specific arguments for the event.

Windows

On Windows, the [PlatformPointerEventArgs](#) class defines the following properties:

- `Sender`, of type [FrameworkElement](#), represents the native view attached to the event.
- `PointerRoutedEventArgs`, of type [PointerRoutedEventArgs](#), indicates the native event or handler attached to the view.

In addition, the [PointerEventArgs](#) object defines a [GetPosition](#) method that returns a `Point?` object that represents the position of the pointer when the gesture was detected. For more information about the [GetPosition](#) method, see [Get the gesture position](#).

❗ Important

Pointer gesture recognition is supported on Android, iPadOS, Mac Catalyst, and Windows.

.NET MAUI also defines a `PointerOver` visual state. This state can change the visual appearance of a view when it has a mouse cursor hovering over it, but isn't pressed. For more information, see [Visual states](#).

Create a PointerGestureRecognizer

To make a [View](#) recognize pointer gestures, create a [PointerGestureRecognizer](#) object, handle the required events, and add the gesture recognizer to the [GestureRecognizers](#) collection on the view. Alternatively, create a [PointerGestureRecognizer](#) object, and bind the required commands to [ICommand](#) implementations, and add the gesture recognizer to the [GestureRecognizers](#) collection on the view.

The following code example shows a [PointerGestureRecognizer](#) attached to an [Image](#). The [PointerGestureRecognizer](#) uses events to respond to the detection of pointer

gestures:

XAML

```
<Image Source="dotnet_bot.png">
    <Image.GestureRecognizers>
        <PointerGestureRecognizer PointerEntered="OnPointerEntered"
                                PointerExited="OnPointerExited"
                                PointerMoved="OnPointerMoved" />
    </Image.GestureRecognizers>
</Image>
```

The code for the event handlers should be added to the code-behind file:

C#

```
void OnPointerEntered(object sender, PointerEventArgs e)
{
    // Handle the pointer entered event
}

void OnPointerExited(object sender, PointerEventArgs e)
{
    // Handle the pointer exited event
}

void OnPointerMoved(object sender, PointerEventArgs e)
{
    // Handle the pointer moved event
}
```

The equivalent C# code is:

C#

```
PointerGestureRecognizer pointerGestureRecognizer = new
PointerGestureRecognizer();
pointerGestureRecognizer.PointerEntered += (s, e) =>
{
    // Handle the pointer entered event
};
pointerGestureRecognizer.PointerExited += (s, e) =>
{
    // Handle the pointer exited event
};
pointerGestureRecognizer.PointerMoved += (s, e) =>
{
    // Handle the pointer moved event
};
```

```
Image image = new Image();  
image.GestureRecognizers.Add(pointerGestureRecognizer);
```

Get the gesture position

The position at which a pointer gesture occurred can be obtained by calling the [GetPosition](#) method on a [PointerEventArgs](#) object. The [GetPosition](#) method accepts an `Element?` argument, and returns a position as a `Point?` object:

C#

```
void OnPointerExited(object sender, PointerEventArgs e)  
{  
    // Position inside window  
    Point? windowPosition = e.GetPosition(null);  
  
    // Position relative to an Image  
    Point? relativeToImagePosition = e.GetPosition(image);  
  
    // Position relative to the container view  
    Point? relativeToContainerPosition = e.GetPosition((View)sender);  
}
```

The `Element?` argument defines the element the position should be obtained relative to. Supplying a `null` value as this argument means that the [GetPosition](#) method returns a `Point?` object that defines the position of the pointer gesture inside the window.

Recognize a swipe gesture

Article • 11/15/2023

A .NET Multi-platform App UI (.NET MAUI) swipe gesture recognizer detects when a finger is moved across the screen in a horizontal or vertical direction, and is often used to initiate navigation through content.

In .NET MAUI, drag gesture recognition is provided by the [SwipeGestureRecognizer](#) class. This class defines the following properties:

- [Command](#), of type [ICommand](#), which is executed when a swipe gesture is recognized.
- [CommandParameter](#), of type `object`, which is the parameter that's passed to the `Command`.
- [Direction](#), of type [SwipeDirection](#), which defines the direction
- [Threshold](#), of type `uint`, which represents the minimum swipe distance that must be achieved for a swipe to be recognized, in device-independent units. The default value of this property is 100, which means that any swipes that are less than 100 device-independent units will be ignored.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [SwipeGestureRecognizer](#) also defines a [Swiped](#) event that's raised when a swipe is recognized. The [SwipedEventArgs](#) object that accompanies the [Swiped](#) event defines the following properties:

- [Direction](#), of type [SwipeDirection](#), indicates the direction of the swipe gesture.
- [Parameter](#), of type `object`, indicates the value passed by the `CommandParameter` property, if defined.

Create a SwipeGestureRecognizer

To make a [View](#) recognize a swipe gesture, create a [SwipeGestureRecognizer](#) object, set the [Direction](#) property to a [SwipeDirection](#) enumeration value (`Left`, `Right`, `Up`, or `Down`), optionally set the [Threshold](#) property, handle the [Swiped](#) event, and add the new gesture recognizer to the `GestureRecognizers` collection on the view. The following example shows a [SwipeGestureRecognizer](#) attached to a [BoxView](#):

XAML

```
<BoxView Color="Teal" ...>
  <BoxView.GestureRecognizers>
    <SwipeGestureRecognizer Direction="Left" Swiped="OnSwiped"/>
  </BoxView.GestureRecognizers>
</BoxView>
```

The equivalent C# code is:

C#

```
BoxView boxView = new BoxView { Color = Colors.Teal, ... };
SwipeGestureRecognizer leftSwipeGesture = new SwipeGestureRecognizer {
    Direction = SwipeDirection.Left };
leftSwipeGesture.Swiped += OnSwiped;

boxView.GestureRecognizers.Add(leftSwipeGesture);
```

Recognize the swipe direction

The `SwipeGestureRecognizer.Direction` property can be set to a single value from the `SwipeDirection` enumeration, or multiple values. This enables the `Swiped` event to be raised in response to a swipe in more than one direction. However, the constraint is that a single `SwipeGestureRecognizer` can only recognize swipes that occur on the same axis. Therefore, swipes that occur on the horizontal axis can be recognized by setting the `Direction` property to `Left` and `Right`:

XAML

```
<SwipeGestureRecognizer Direction="Left,Right" Swiped="OnSwiped"/>
```

Similarly, swipes that occur on the vertical axis can be recognized by setting the `Direction` property to `Up` and `Down`:

C#

```
SwipeGestureRecognizer swipeGesture = new SwipeGestureRecognizer { Direction
    = SwipeDirection.Up | SwipeDirection.Down };
```

Alternatively, a `SwipeGestureRecognizer` for each swipe direction can be created to recognize swipes in every direction:

XAML


```

<BoxView Color="Teal" ...>
  <BoxView.GestureRecognizers>
    <SwipeGestureRecognizer Direction="Left" Swiped="OnSwiped"/>
    <SwipeGestureRecognizer Direction="Right" Swiped="OnSwiped"/>
    <SwipeGestureRecognizer Direction="Up" Swiped="OnSwiped"/>
    <SwipeGestureRecognizer Direction="Down" Swiped="OnSwiped"/>
  </BoxView.GestureRecognizers>
</BoxView>

```

The equivalent C# code is:

C#

```

BoxView boxView = new BoxView { Color = Colors.Teal, ... };
SwipeGestureRecognizer leftSwipeGesture = new SwipeGestureRecognizer {
    Direction = SwipeDirection.Left };
leftSwipeGesture.Swiped += OnSwiped;
SwipeGestureRecognizer rightSwipeGesture = new SwipeGestureRecognizer {
    Direction = SwipeDirection.Right };
rightSwipeGesture.Swiped += OnSwiped;
SwipeGestureRecognizer upSwipeGesture = new SwipeGestureRecognizer {
    Direction = SwipeDirection.Up };
upSwipeGesture.Swiped += OnSwiped;
SwipeGestureRecognizer downSwipeGesture = new SwipeGestureRecognizer {
    Direction = SwipeDirection.Down };
downSwipeGesture.Swiped += OnSwiped;

boxView.GestureRecognizers.Add(leftSwipeGesture);
boxView.GestureRecognizers.Add(rightSwipeGesture);
boxView.GestureRecognizers.Add(upSwipeGesture);
boxView.GestureRecognizers.Add(downSwipeGesture);

```

Respond to a swipe

A recognized swipe can be responded to by a handler for the [Swiped](#) event:

C#

```

void OnSwiped(object sender, SwipedEventArgs e)
{
    switch (e.Direction)
    {
        case SwipeDirection.Left:
            // Handle the swipe
            break;
        case SwipeDirection.Right:
            // Handle the swipe
            break;
        case SwipeDirection.Up:

```

```

        // Handle the swipe
        break;
    case SwipeDirection.Down:
        // Handle the swipe
        break;
    }
}

```

The [SwipedEventArgs](#) can be examined to determine the direction of the swipe, with custom logic responding to the swipe as required. The direction of the swipe can be obtained from the [Direction](#) property of the event arguments, which will be set to one of the values of the [SwipeDirection](#) enumeration. In addition, the event arguments also have a [Parameter](#) property that will be set to the value of the `CommandParameter` property, if defined.

Create a swipe container

The `SwipeContainer` class, which is shown in the following example, is a generalized swipe recognition class that be wrapped around a [View](#) to perform swipe gesture recognition:

C#

```

public class SwipeContainer : ContentView
{
    public event EventHandler<SwipedEventArgs> Swipe;

    public SwipeContainer()
    {
        GestureRecognizers.Add(GetSwipeGestureRecognizer(SwipeDirection.Left));
        GestureRecognizers.Add(GetSwipeGestureRecognizer(SwipeDirection.Right));
        GestureRecognizers.Add(GetSwipeGestureRecognizer(SwipeDirection.Up));
        GestureRecognizers.Add(GetSwipeGestureRecognizer(SwipeDirection.Down));
    }

    SwipeGestureRecognizer GetSwipeGestureRecognizer(SwipeDirection
direction)
    {
        SwipeGestureRecognizer swipe = new SwipeGestureRecognizer {
Direction = direction };
        swipe.Swiped += (sender, e) => Swipe?.Invoke(this, e);
        return swipe;
    }
}

```

The `SwipeContainer` class creates `SwipeGestureRecognizer` objects for all four swipe directions, and attaches `Swipe` event handlers. These event handlers invoke the `Swipe` event defined by the `SwipeContainer`.

The following XAML code example shows the `SwipeContainer` class wrapping a `BoxView`:

XAML

```
<StackLayout>
    <local:SwipeContainer Swipe="OnSwiped" ...>
        <BoxView Color="Teal" ... />
    </local:SwipeContainer>
</StackLayout>
```

In this example, when the `BoxView` receives a swipe gesture, the `Swiped` event in the `SwipeGestureRecognizer` is raised. This is handled by the `SwipeContainer` class, which raises its own `Swipe` event. This `Swipe` event is handled on the page. The `SwipedEventArgs` can then be examined to determine the direction of the swipe, with custom logic responding to the swipe as required.

The equivalent C# code is:

C#

```
BoxView boxView = new BoxView { Color = Colors.Teal, ... };
SwipeContainer swipeContainer = new SwipeContainer { Content = boxView, ...
};
swipeContainer.Swipe += (sender, e) =>
{
    // Handle the swipe
};

StackLayout stackLayout = new StackLayout();
stackLayout.Add(swipeContainer);
```

Recognize a tap gesture

Article • 11/15/2023

A .NET Multi-platform App UI (.NET MAUI) tap gesture recognizer is used for tap detection and is implemented with the [TapGestureRecognizer](#) class. This class defines the following properties:

- [Buttons](#), of type [ButtonsMask](#), which defines whether the primary or secondary mouse button, or both, triggers the gesture on Android, Mac Catalyst, and Windows. For more information, see [Define the button masks](#).
- [Command](#), of type [ICommand](#), which is executed when a tap is recognized.
- [CommandParameter](#), of type `object`, which is the parameter that's passed to the `Command`.
- [NumberOfTapsRequired](#), of type `int`, which represents the number of taps required to recognize a tap gesture. The default value of this property is 1.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [TapGestureRecognizer](#) class also defines a [Tapped](#) event that's raised when a tap is recognized. The [TappedEventArgs](#) object that accompanies the [Tapped](#) event defines a [Parameter](#) property of type `object` that indicates the value passed by the `CommandParameter` property, if defined. The [TappedEventArgs](#) object also defines a [Buttons](#) property, and a `GetPosition` method. The [Buttons](#) property is of type [ButtonsMask](#), and can be used to determine whether the primary or secondary mouse button triggered the gesture recognizer on Android, Mac Catalyst, and Windows. The `GetPosition` method returns a `Point?` object that represents the position at which the tap gesture was detected. For more information about button masks, see [Define the button mask](#). For more information about the `GetPosition` method, see [Get the gesture position](#).

Warning

A [TapGestureRecognizer](#) can't recognize more than a double tap on Windows.

Create a TapGestureRecognizer

To make a [View](#) recognize a tap gesture, create a [TapGestureRecognizer](#) object, handle the [Tapped](#) event, and add the new gesture recognizer to the `GestureRecognizers`

collection on the view. The following code example shows a [TapGestureRecognizer](#) attached to an [Image](#):

XAML

```
<Image Source="dotnet_bot.png">
  <Image.GestureRecognizers>
    <TapGestureRecognizer Tapped="OnTapGestureRecognizerTapped"
                          NumberOfTapsRequired="2" />
  </Image.GestureRecognizers>
</Image>
```

The code for the `OnTapGestureRecognizerTapped` event handler should be added to the code-behind file:

C#

```
void OnTapGestureRecognizerTapped(object sender, TappedEventArgs args)
{
    // Handle the tap
}
```

The equivalent C# code is:

C#

```
TapGestureRecognizer tapGestureRecognizer = new TapGestureRecognizer();
tapGestureRecognizer.Tapped += (s, e) =>
{
    // Handle the tap
};
Image image = new Image();
image.GestureRecognizers.Add(tapGestureRecognizer);
```

By default the [Image](#) will respond to single taps. When the [NumberOfTapsRequired](#) property is set to greater than one, the event handler will only be executed if the taps occur within a set period of time. If the second (or subsequent) taps don't occur within that period, they're effectively ignored.

Define the button mask

A [TapGestureRecognizer](#) object has a [Buttons](#) property, of type [ButtonsMask](#), that defines whether the primary or secondary mouse button, or both, triggers the gesture on Android, Mac Catalyst, and Windows. The [ButtonsMask](#) enumeration defines the following members:

- **Primary** represents the primary mouse button, which is typically the left mouse button.
- **Secondary** represents the secondary mouse button, which is typically the right mouse button.

The following example shows a **TapGestureRecognizer** that detects taps with the secondary mouse button:

XAML

```
<Image Source="dotnet_bot.png">
  <Image.GestureRecognizers>
    <TapGestureRecognizer Tapped="OnTapGestureRecognizerTapped"
                          Buttons="Secondary" />
  </Image.GestureRecognizers>
</Image>
```

The event handler for the **Tapped** event can determine which button triggered the gesture:

C#

```
void OnTapGestureRecognizerTapped(object sender, TappedEventArgs args)
{
    // Handle the tap
    if (args.Buttons == ButtonsMask.Secondary)
    {
        // Do something
    }
}
```

The equivalent C# code is:

C#

```
TapGestureRecognizer tapGestureRecognizer = new TapGestureRecognizer
{
    Buttons = ButtonsMask.Secondary
};
tapGestureRecognizer.Tapped += (s, e) =>
{
    // Handle the tap
    if (args.Buttons == ButtonsMask.Secondary)
    {
        // Do something
    }
};
```

```
Image image = new Image();
image.GestureRecognizers.Add(tapGestureRecognizer);
```

⚠ Warning

On Windows, a [TapGestureRecognizer](#) that sets the `Buttons` property to `Secondary` doesn't respect the [NumberOfTapsRequired](#) property when it's greater than one.

In addition, a [TapGestureRecognizer](#) can be defined so that either the primary or secondary mouse button triggers the gesture:

XAML

```
<TapGestureRecognizer Tapped="OnTapGestureRecognizerTapped"
                      Buttons="Primary,Secondary" />
```

The equivalent C# code is:

C#

```
TapGestureRecognizer tapGestureRecognizer = new TapGestureRecognizer
{
    Buttons = ButtonsMask.Primary | ButtonsMask.Secondary
};
```

Get the gesture position

The position at which a tap gesture occurred can be obtained by calling the `GetPosition` method on a [TappedEventArgs](#) object. The `GetPosition` method accepts an `Element?` argument, and returns a position as a `Point?` object:

C#

```
void OnTapGestureRecognizerTapped(object sender, TappedEventArgs e)
{
    // Position inside window
    Point? windowPosition = e.GetPosition(null);

    // Position relative to an Image
    Point? relativeToImagePosition = e.GetPosition(image);

    // Position relative to the container view
    Point? relativeToContainerPosition = e.GetPosition((View)sender);
}
```

The `Element?` argument defines the element the position should be obtained relative to. Supplying a `null` value as this argument means that the `GetPosition` method returns a `Point?` object that defines the position of the tap gesture inside the window.

Localization

Article • 11/11/2024

 [Browse the sample](#)

Localization is the process of adapting an app to meet the specific language or cultural requirements of a target market. To localize an app, its text and images might need to be translated into multiple languages. A localized app automatically displays translated text based on the culture settings of the device.

.NET includes a mechanism for localizing apps using [resource files](#). A resource file stores text and other content as name/value pairs that allow the app to retrieve content for a provided key. Resource files allow localized content to be separated from app code. In addition to storing text, resource files can also store images and binary data. However, devices have a range of screen sizes and densities and each platform has functionality for displaying density-dependent images. Therefore, platform functionality should be used for localizing images instead of storing them in resource files.

To localize a .NET Multi-platform App UI (.NET MAUI) app, you should:

1. Create resource files to store strings. For more information, see [Create resource files to store strings](#).
2. Specify the app's neutral language. For more information, see [Specify the app's neutral language](#).
3. Perform platform setup. For more information, see [Perform platform setup](#).
4. Localize text. For more information, see [Localize text](#).
5. Localize images. For more information, see [Localize images](#).
6. Localize the app name. For more information, see [Localize the app name](#).
7. Test localization. For more information, see [Test localization](#).

In addition, the layout direction of an app can be specified. For more information, see [Right to left localization](#).

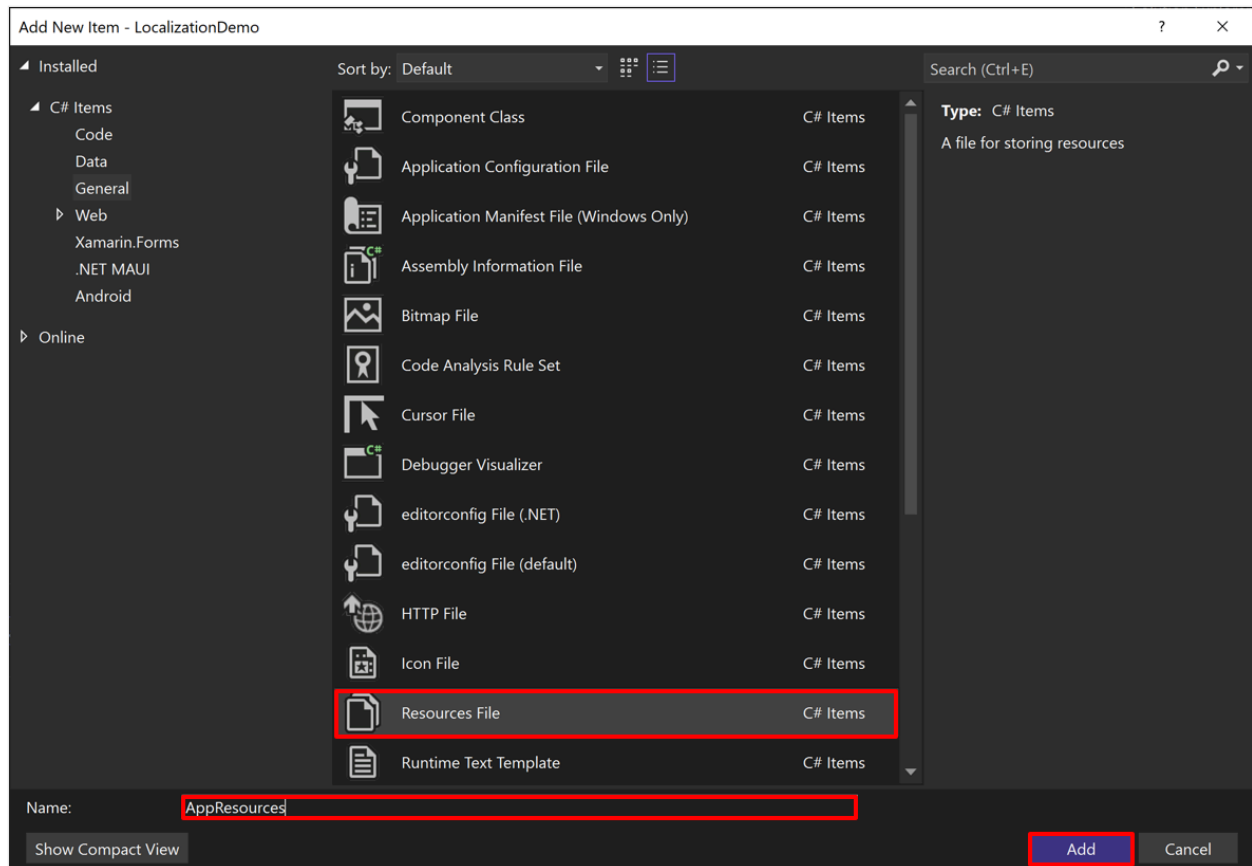
Create resource files to store strings

.NET resource files are XML files with a *.resx* extension that are compiled into binary resource (*.resources*) files during the build process. A localized app typically contains a default resource file with all strings used in the app, and resource files for each supported language.

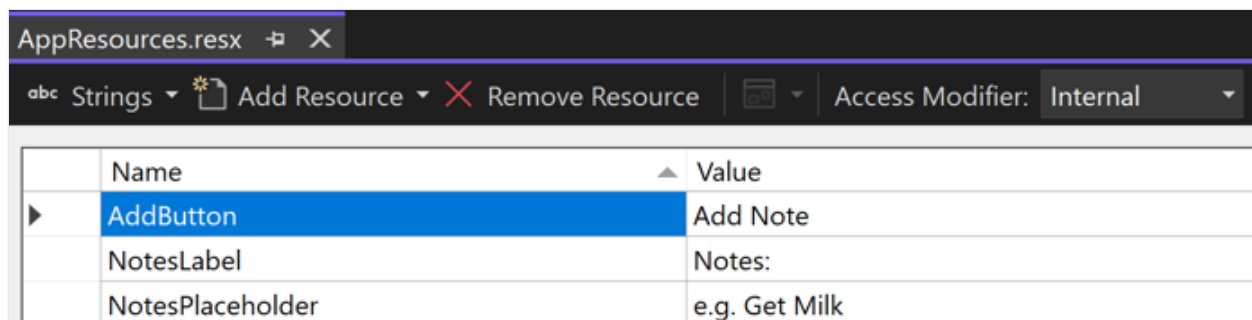
Resource files contain the following information for each item:

- **Name** specifies the key used to access the text in code.
- **Value** specifies the translated text.
- **Comment** is an optional field containing additional information.

A resource file can be added with the **Add New Item** dialog in Visual Studio:



Once the file is added, rows can be added for each text resource:



The **Access Modifier** drop-down determines how Visual Studio generates the class used to access resources. Setting the Access Modifier to **Public** or **Internal** results in a generated class with the specified accessibility level. Setting the Access Modifier to **No code generation** doesn't generate a class file. The default resource file should be configured to generate a class file, which results in a file with the *.Designer.cs* extension being added to the project.

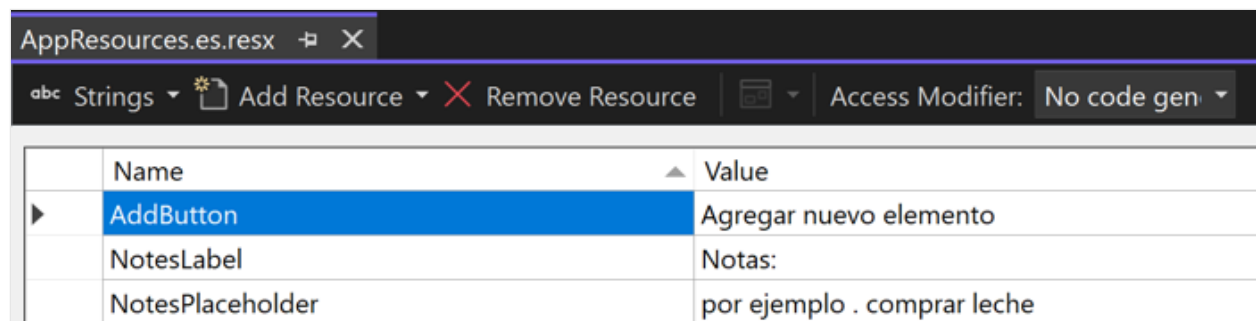
Once the default resource file is created, additional files can be created for each locale the app supports. Each additional resource file should have the same root filename as

the default resource file, but should also include the language and optional culture in the filename. For example, if you add a resource file named *AppResources.resx*, you might also create resource files named *AppResources.en-US.resx* and *AppResources.fr-FR.resx* to hold localized resources for the English (United States) and French (France) cultures, respectively. In addition, you should set the **Access Modifier** for each additional resource file to **No code generation**.

At runtime, your app attempts to resolve a resource request in order of specificity. For example, if the device culture is **en-US** the application looks for resource files in this order:

1. *AppResources.en-US.resx*
2. *AppResources.en.resx*
3. *AppResources.resx* (default)

The following screenshot shows a Spanish translation file named *AppResources.es.resx*:



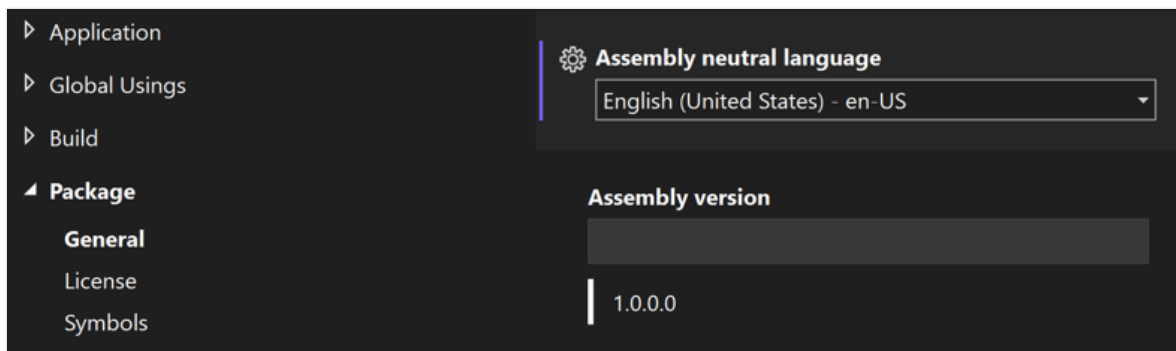
Name	Value
AddButton	Agregar nuevo elemento
NotesLabel	Notas:
NotesPlaceholder	por ejemplo . comprar leche

The localized resource file uses the same **Name** values specified in the default file but contains Spanish language strings in the **Value** column. Additionally, the **Access Modifier** is set to **No code generation**.

Specify the app's neutral language

For .NET resource files to work correctly, the app must have a neutral language specified. This is the language whose resources are used if resources for a locale can't be found. To specify the neutral language:

1. In Solution Explorer, right-click your .NET MAUI app project and select **Properties**.
2. Select the **Package > General** property page and select the appropriate language and culture from the **Assembly neutral language** drop-down:



3. Save your changes.

Alternatively, add the `<NeutralLanguage>` element to the first `<PropertyGroup>` in your project file, and specify your chosen locale as its value:

XML

```
<NeutralLanguage>en-US</NeutralLanguage>
```

⚠ Warning

If you don't specify a neutral language, the **ResourceManager** class returns `null` values for any languages without a resource file. When a neutral language is specified, the **ResourceManager** class returns results from the neutral language resource file for unsupported languages. Therefore, it's recommended that you always specify a neutral language so that text is displayed for unsupported languages.

Perform platform setup

Additional setup is required on iOS, Mac Catalyst, and Windows, so that all .NET MAUI controls are localized.

iOS and Mac Catalyst

On iOS and Mac Catalyst, you must declare all supported languages in the platform's *Info.plist* file in your .NET MAUI app project. To do this, open the *Info.plist* file for your chosen platform in an XML editor and create an array for the `CFBundleLocalizations` key. Then provide array values that correspond to the resource files. In addition, ensure you set an expected language via the `CFBundleDevelopmentRegion` key:

XML

```

<key>CFBundleLocalizations</key>
<array>
  <string>de</string>
  <string>es</string>
  <string>fr</string>
  <string>ja</string>
  <string>pt</string> <!-- Brazil -->
  <string>pt-PT</string> <!-- Portugal -->
  <string>ru</string>
  <string>zh-Hans</string>
  <string>zh-Hant</string>
</array>
<key>CFBundleDevelopmentRegion</key>
<string>en</string>

```

Alternatively, in Solution Explorer in Visual Studio, open the *Info.plist* file for your chosen platform in the **Generic PList Editor**. Then, create an array for the `CFBundleLocalizations` key and provide array values that correspond to the resource files. In addition, ensure you set an expected language via the `CFBundleDevelopmentRegion` key:

Property	Type	Value
	String	Landscape (left button)
	String	Landscape (right button)
+		
XSApplIconAssets	String	Assets.xcassets/appicon.appiconset
Localizations	Array	(9 items)
	String	de
	String	es
	String	fr
	String	ja
	String	pt
	String	pt-PT
	String	ru
	String	zh-Hans
	String	zh-Hant
+		
Localization native development region	String	en

For more information about the *Info.plist* file, see [Information property list](#).

Windows

To support multiple languages in a .NET MAUI app on Windows, you must declare each supported language in the *Platforms\Windows\Package.appxmanifest* file of your .NET MAUI app project:

1. Open the *Package.appxmanifest* file in a text editor and locate the following section:

XML

```
<Resources>
  <Resource Language="x-generate"/>
</Resources>
```

2. Replace `<Resource Language="x-generate">` with `<Resource />` elements for each of your supported languages:

XML

```
<Resources>
  <Resource Language="en-US"/>
  <Resource Language="de-DE"/>
  <Resource Language="es-ES"/>
  <Resource Language="fr-FR"/>
  <Resource Language="ja-JP"/>
  <Resource Language="pt-BR"/>
  <Resource Language="pt-PT"/>
  <Resource Language="ru-RU"/>
  <Resource Language="zh-CN"/>
  <Resource Language="zh-TW"/>
</Resources>
```

3. Save your changes.

Localize text

Text is localized using a class generated from your default resources file. The class is named based on the default resource filename. Given a default resource filename of *AppResources.resx*, Visual Studio generates a matching class named `AppResources` containing static properties for each entry in the resource file.

In XAML, localized text can be retrieved by using the `x:Static` markup extension to access the generated static properties:

XAML

```
<ContentPage ...
  xmlns:strings="clr-
namespace:LocalizationDemo.Resources.Strings">
  <VerticalStackLayout>
    <Label Text="{x:Static strings:AppResources.NotesLabel}" />
```

```
<Entry Placeholder="{x:Static
strings:AppResources.NotesPlaceholder}" />
<Button Text="{x:Static strings:AppResources.AddButton}" />
</VerticalStackLayout>
</ContentPage>
```

For more information about the `x:Static` markup extension, see [x:Static markup extension](#).

Localized text can also be retrieved in code:

C#

```
Label notesLabel = new Label();
notesLabel.Text = AppResources.NotesLabel,

Entry notesEntry = new Entry();
notesEntry.Placeholder = AppResources.NotesPlaceholder,

Button addButton = new Button();
addButton.Text = AppResources.AddButton,
```

The properties in the `AppResources` class use the `CurrentUICulture` property value to determine which resource file to retrieve values from.

Localize images

In addition to storing text, resource files can also store images and binary data. However, devices have a range of screen sizes and densities and each platform has functionality for displaying density-dependent images. Therefore, platform functionality should be used for localizing images instead of storing them in resource files.

Android

On Android, localized images, known as drawables, are stored using a folder-based naming convention in the `Platforms\Android\Resources` folder. Folders should be named *drawable* with a suffix for the language and culture. For example, the Spanish-language folder is named *drawable-es*. The folder name *drawable* should contain the images for your default language and culture. The build action of each image should be set to **AndroidResource**.

📌 Note

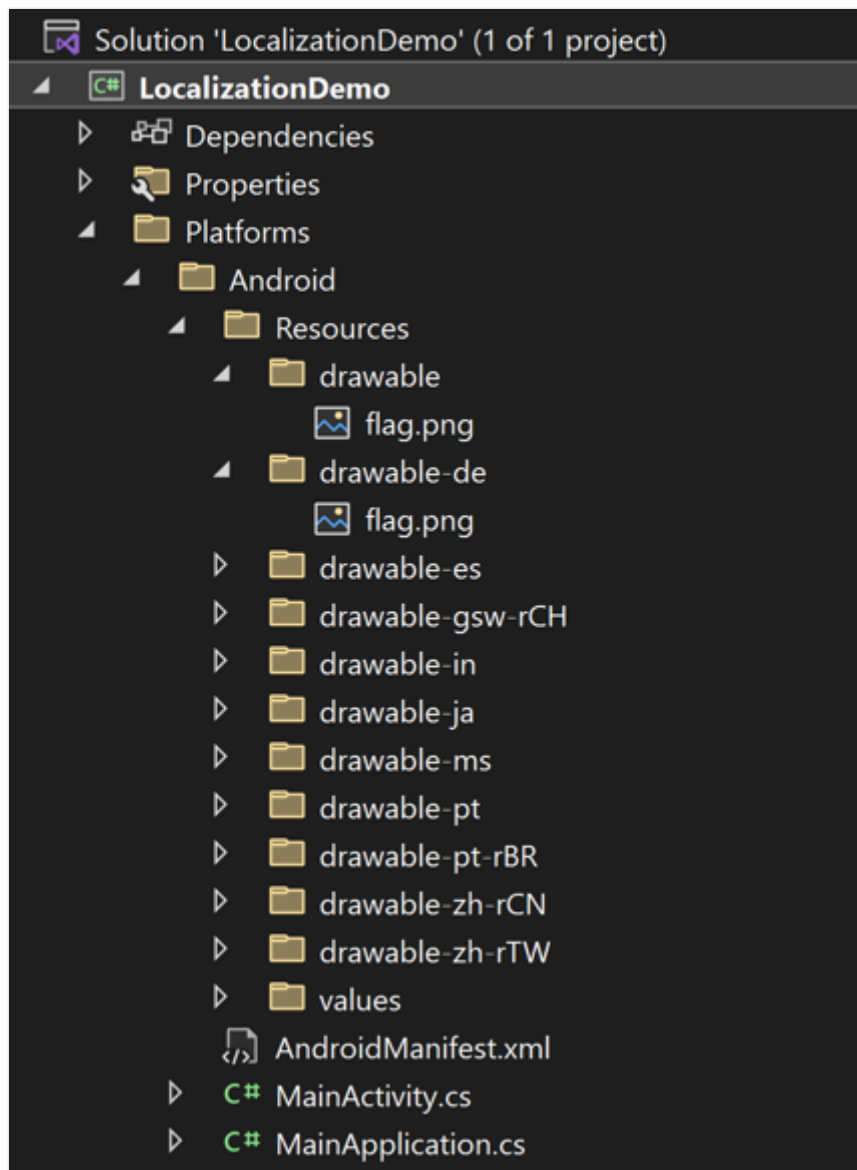
Rather than set individual files to the **AndroidResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-android'))">
  <AndroidResource Include="Platforms\Android\Resources\*"
    TargetPath="%RecursiveDir%\Filename%(Extension)" />
</ItemGroup>
```

This example sets any content in the *Platforms\Android\Resources* folder, including content in sub-folders, to the **AndroidResource** build action. It also sets the output path for each file with this build action.

Only two characters are required in the folder name when specifying a top-level language, such as *es*. However, when specifying a full locale, the folder name format requires a dash and lowercase *r* to separate the language from the culture. For example, the Mexico locale (*es-MX*) folder should be named *drawable-es-rMX*. The image file names in each locale folder should be identical:



iOS

On iOS, localized images are stored using a folder-based naming convention in the *Platforms\iOS\Resources* folder. Folders should be named with the language, and optional culture, followed by *.lproj*. For example, the Spanish-language folder is named *es.lproj*. The build action of each image should be set to **BundleResource**.

ⓘ Note

Rather than set individual files to the **BundleResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

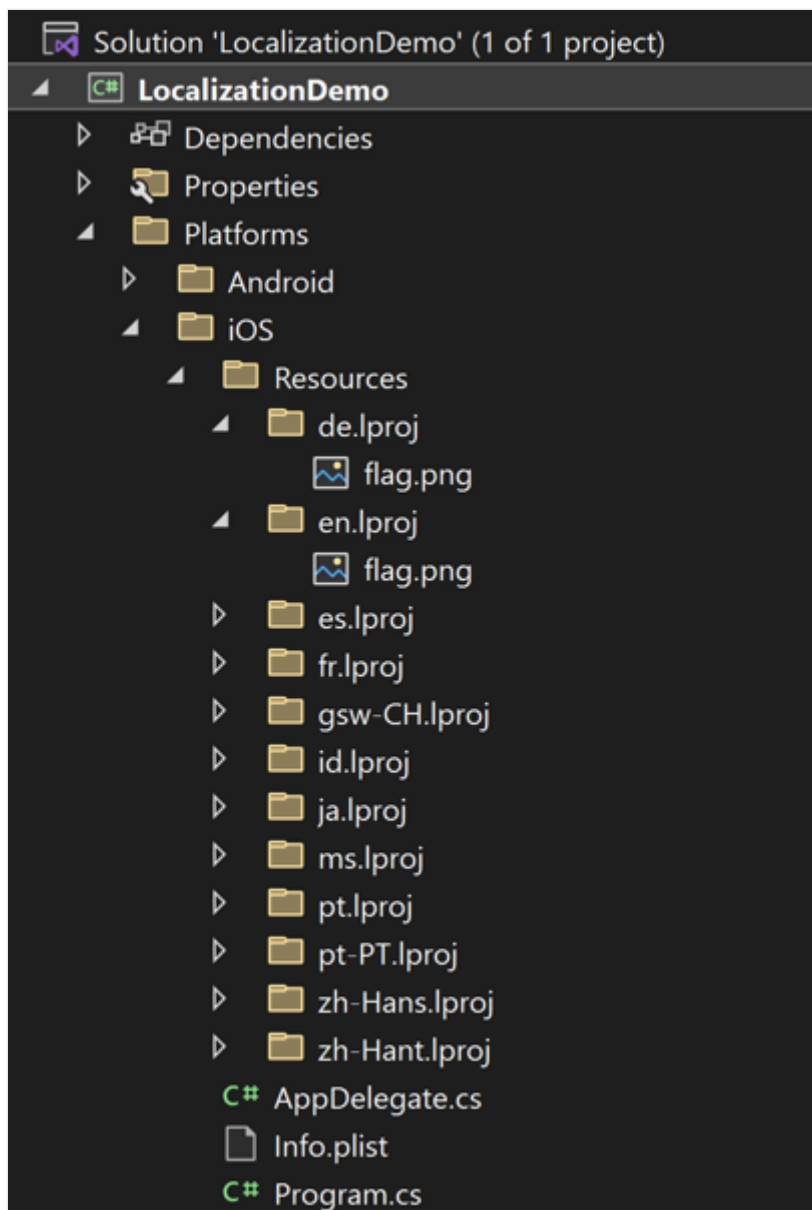
XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-ios'))">
  <BundleResource Include="Platforms\iOS\Resources\*" TargetPath="%
```

```
(RecursiveDir)%(Filename)%(Extension)" />  
</ItemGroup>
```

This example sets any content in the *Platforms\iOS\Resources* folder, including content in sub-folders, to the **BundleResource** build action. It also sets the output path for each file with this build action.

Only two characters are required in the folder name when specifying a top-level language, such as *es*. However, when specifying a full locale, the folder name format requires a dash to separate the language from the culture. For example, the Mexico locale (*es-MX*) folder should be named *es-MX.lproj*. The image file names in each locale folder should be identical:



If an image isn't present for a particular language, iOS falls back to the default native language folder and loads the image from there.

Mac Catalyst

On Mac Catalyst, localized images are stored using a folder-based naming convention in the *Platforms\MacCatalyst\Resources* folder. Folders should be named with the language, and optional culture, followed by *.lproj*. For example, the Spanish-language folder is named *es.lproj*. The build action of each image should be set to **BundleResource**.

ⓘ Note

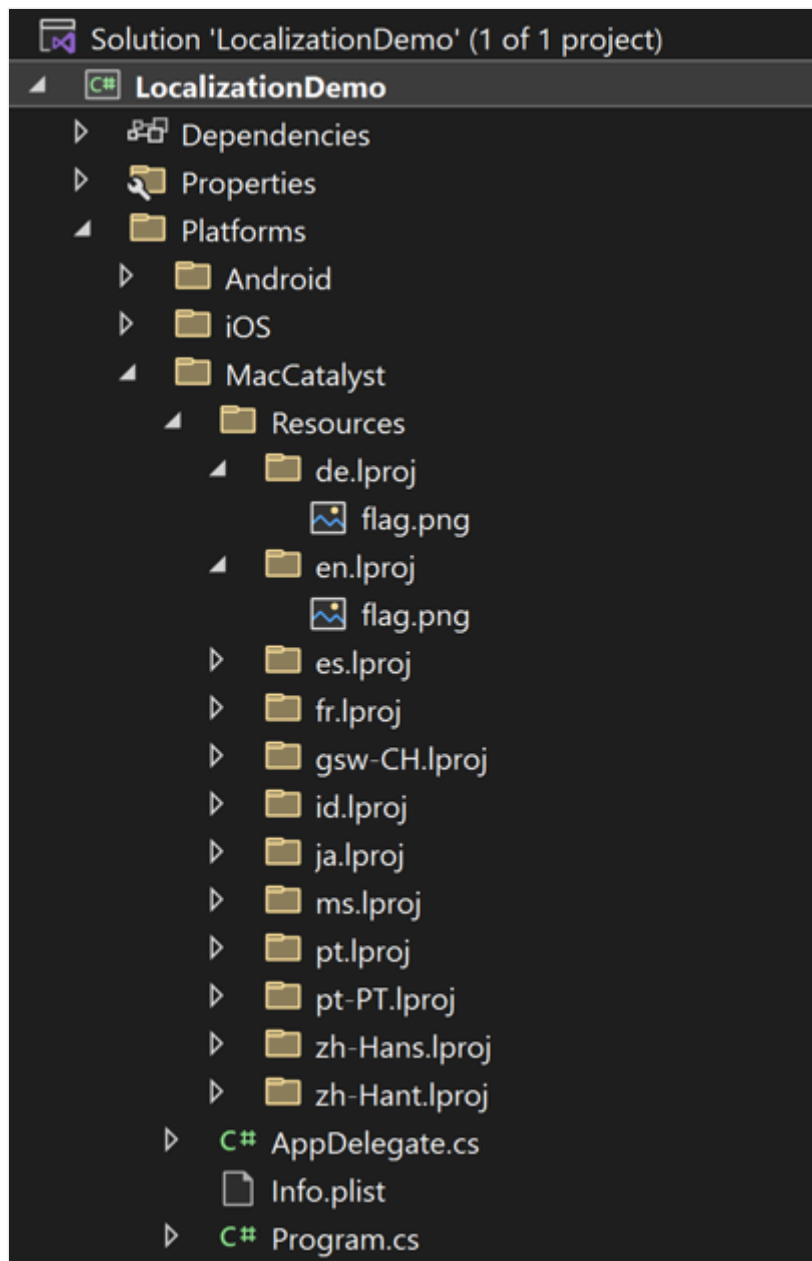
Rather than set individual files to the **BundleResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-maccatalyst'))">
  <BundleResource Include="Platforms\MacCatalyst\Resources\*"
    TargetPath="%\RecursiveDir%\Filename%\Extension" />
</ItemGroup>
```

This example sets any content in the *Platforms\MacCatalyst\Resources* folder, including content in sub-folders, to the **BundleResource** build action. It also sets the output path for each file with this build action.

Only two characters are required in the folder name when specifying a top-level language, such as *es*. However, when specifying a full locale, the folder name format requires a dash to separate the language from the culture. For example, the Mexico locale (*es-MX*) folder should be named *es-MX.lproj*. The image file names in each locale folder should be identical:



If an image isn't present for a particular language, Mac Catalyst falls back to the default native language folder and loads the image from there.

Windows

On Windows, localized images are stored using a folder-based naming convention in the *Platforms\Windows\Assets\Images* folder. Folders should be named with the language, and optional culture. For example, the Spanish-language folder is named *es* and the Mexico locale folder should be named *es-MX*. The build action of each image should be set to **Content**.

ⓘ Note

Rather than set individual files to the **Content** build action, the contents of a specific folder can be set to this build action by adding the following XML to your

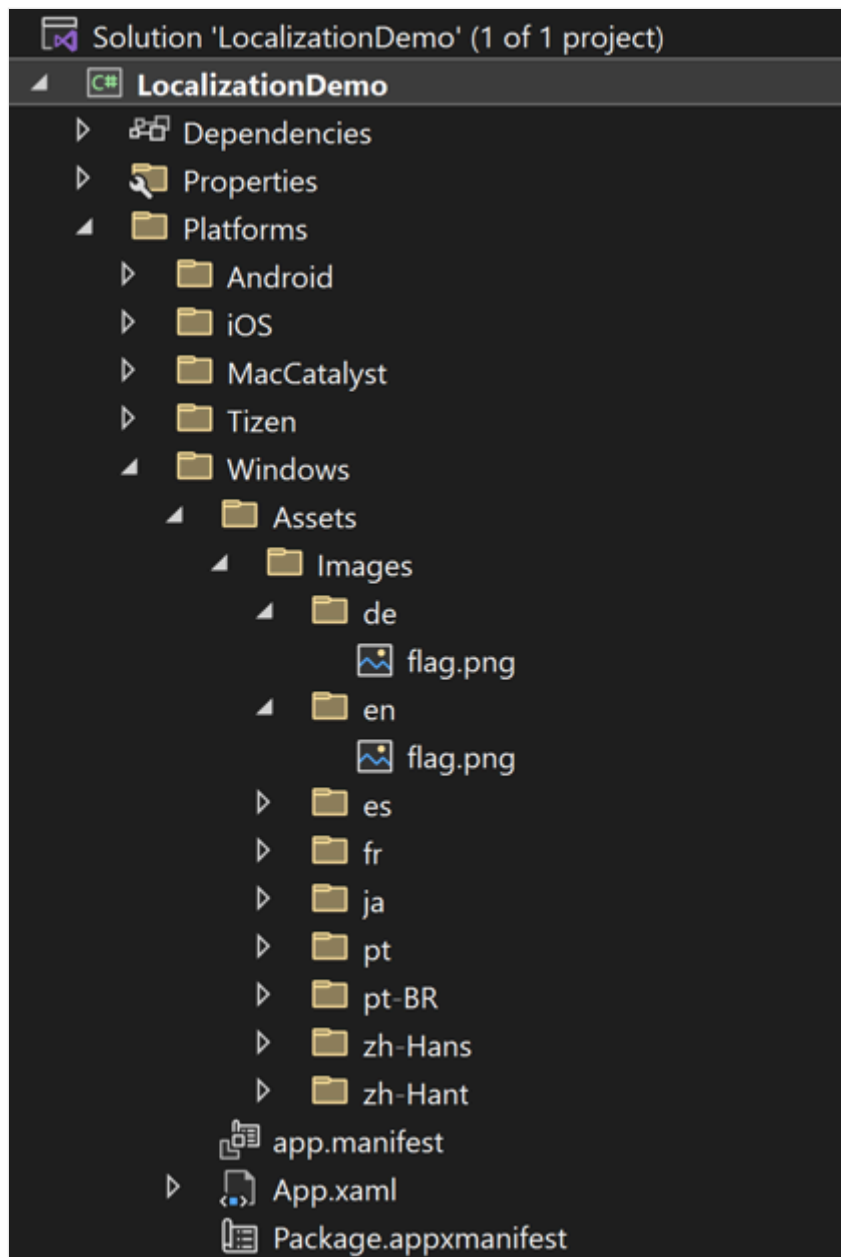
app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-windows'))">
  <Content Include="Platforms\Windows\Assets\Images\*" TargetPath="%
(RecursiveDir)%(Filename)%(Extension)" />
</ItemGroup>
```

This example sets any content in the *Platforms\Windows\Assets\Images* folder, including content in sub-folders, to the **Content** build action. It also sets the output path for each file with this build action.

Only two characters are required in the folder name when specifying a top-level language, such as *es*. However, when specifying a full locale, the folder name format requires a dash to separate the language from the culture. For example, the Mexico locale (*es-MX*) folder should be named *es-MX*. The image file names in each locale folder should be identical:



Consume localized images

On Android, iOS, Mac Catalyst, and Windows, localized images can be consumed by setting the [Source](#) property of an [Image](#) to the image filename:

XAML

```
<Image Source="flag.png" />
```

However, for this to work on Windows it's necessary to modify your app's project file if you've added a `<Content />` MSBuild item for each localized image. This can be accomplished by modifying your .csproj file to remove the `<Content />` MSBuild item for each image. Then, add the following MSBuild item:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-windows'))">
  <Content Include="Platforms\Windows\Assets\Images\*" TargetPath="%
(RecursiveDir)%(Filename)%(Extension)" />
</ItemGroup>
```

This ensures that all images in the subfolders of the *Platforms\Windows\Assets\Images* folder are copied to the root of your app package.

Localize the app name

Platform functionality is required for localizing the app's name.

Android

On Android, the localized app name can be stored using a folder-based naming convention in the *Platforms\Android\Resources* folder. Folders should be named *values* with a suffix for the language and culture. For example, the Spanish-language folder is named *values-es*. Add a *Strings.xml* file with a build action of **AndroidResource** to each folder that sets a string to the localized app name.

ⓘ Note

Rather than set individual files to the **AndroidResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-android'))">
  <AndroidResource Include="Platforms\Android\Resources\*"
TargetPath="% (RecursiveDir)%(Filename)%(Extension)" />
</ItemGroup>
```

This example sets any content in the *Platforms\Android\Resources* folder, including content in sub-folders, to the **AndroidResource** build action. It also sets the output path for each file with this build action.

Only two characters are required in the folder name when specifying a top-level language, such as *es*. However, when specifying a full locale, the folder name format requires a dash and lowercase *r* to separate the language from the culture. For example, the Mexico locale (*es-MX*) folder should be named *values-es-rMX*.

Each translatable string is an XML element with the resource ID specified as the `name` attribute and the translated string as the value. You need to escape your string according to normal XML rules, and the `name` must be a valid Android resource ID (no spaces or dashes).

Therefore, to localize the app name create a *Strings.xml* file and add a `<string>` element as the child of a `<resources>` element. Then, set its `name` attribute to a suitable ID with the translated string as the value:

XML

```
<resources>
  <!-- French -->
  <string name="app_name">Maison</string>
</resources>
```

Then, to use the localized app name in your app add the `Label` property to the `Activity` in your app's `MainActivity` class, and set its value to `@string/id`:

C#

```
[Activity(Label = "@string/app_name", Theme = "@style/Maui.SplashTheme",
MainLauncher = true, ConfigurationChanges = ConfigChanges.ScreenSize |
ConfigChanges.Orientation | ConfigChanges.UiMode |
ConfigChanges.ScreenLayout | ConfigChanges.SmallestScreenSize |
ConfigChanges.Density)]
public class MainActivity : MauiAppCompatActivity
{
    protected override void OnCreate(Bundle savedInstanceState)
    {
        base.OnCreate(savedInstanceState);
    }
}
```

iOS

On iOS, the localized app name is stored using a folder-based naming convention in the *Platforms\iOS\Resources* folder. Folders should be named with the language, and optional culture, followed by *.lproj*. For example, the Spanish-language folder is named *es.lproj*. Add a *InfoPlist.strings* file with a build action of **BundleResource** to each folder that sets the `CFBundleDisplayName` key and value.

❗ Note

Rather than set individual files to the **BundleResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-ios'))">
  <BundleResource Include="Platforms\iOS\Resources\*" TargetPath="%
    (RecursiveDir)%(Filename)%(Extension)" />
</ItemGroup>
```

This example sets any content in the *Platforms\iOS\Resources* folder, including content in sub-folders, to the **BundleResource** build action. It also sets the output path for each file with this build action.

The syntax for localized string values is:

text

```
/* comment */
"key"="localized-value";
```

You should escape the following characters in strings:

- `\` quote
- `\\` backslash
- `\n` newline

Therefore, to localize the app name create a *InfoPlist.strings* file and add a value for the `CFBundleDisplayName` key to the file:

text

```
/* French */
CFBundleDisplayName="Maisons";
```

Other keys that you can use to localize app-specific strings are:

- `CFBundleName` - specifies the short name of the app bundle, which might be displayed to users in situations such as the absence of a value for `CFBundleDisplayName`.
- `CFBundleShortVersionString` - specifies the release version number of the app bundle.

- `NSHumanReadableCopyright` - the copyright notice for the app bundle.

Mac Catalyst

On Mac Catalyst, the localized app name is stored using a folder-based naming convention in the *Platforms\MacCatalyst\Resources* folder. Folders should be named with the language, and optional culture, followed by *.lproj*. For example, the Spanish-language folder is named *es.lproj*. Add a *InfoPlist.strings* file with a build action of **BundleResource** to each folder that sets the `CFBundleDisplayName` key and value.

ⓘ Note

Rather than set individual files to the **BundleResource** build action, the contents of a specific folder can be set to this build action by adding the following XML to your app's project (.csproj) file:

XML

```
<ItemGroup Condition="$(TargetFramework.Contains('-maccatalyst'))">
  <BundleResource Include="Platforms\MacCatalyst\Resources\*"
    TargetPath="%(RecursiveDir)%(Filename)%(Extension)" />
</ItemGroup>
```

This example sets any content in the *Platforms\MacCatalyst\Resources* folder, including content in sub-folders, to the **BundleResource** build action. It also sets the output path for each file with this build action.

The syntax for localized string values is:

text

```
/* comment */
"key"="localized-value";
```

You should escape the following characters in strings:

- `\` quote
- `\\` backslash
- `\n` newline

Therefore, to localize the app name create a *InfoPlist.strings* file and add a value for the `CFBundleDisplayName` key to the file:

text

```
/* French */  
CFBundleDisplayName="Maisons";
```

Other keys that you can use to localize app-specific strings are:

- `CFBundleName` - specifies the short name of the app bundle, which might be displayed to users in situations such as the absence of a value for `CFBundleDisplayName`.
- `CFBundleShortVersionString` - specifies the release version number of the app bundle.
- `NSHumanReadableCopyright` - the copyright notice for the app bundle.

Windows

On Windows, the app name is defined in your app package manifest. Localizing the app name requires you to first specify the default language for the app, and then create a string resource file for each locale you intend to support. The string resource that represents the localized app name can then be consumed in your app package manifest by using the `ms-resource` URI scheme.

For more information about localizing strings in your app package manifest, see [Localize strings in your UI and app package manifest](#).

Specify the default language

To localize an app name, your Windows app must first have a default language specified. This is the language whose resources are used if no localized resources for a particular language can be found. To specify the default language:

1. In Solution Explorer, open the *Packageappxmanifest* file in the package manifest editor.
2. In the package manifest editor, on the **Application** tab, set the **Default language** field to your chosen default language:

Application	Visual Assets	Capabilities	Declarations
Use this page to set the properties that identify and describe your app.			
Display name:	\$placeholder\$		
Entry point:	\$targetentrypoint\$		
Default language:	en		More information

3. Save your changes.

At a minimum, you need to provide a string resource for the app name for the default language. This is the resource that is loaded if no better match can be found for the user's preferred language or display language settings.

Create Windows resource files

On Windows, the localized app name should be stored in a Windows resource file for each locale. A Windows resource file is an XML file with a *.resw* extension that's compiled into a binary format and stored in a *.pri* file. The *.resw* file for each locale should be named *Resources.resw* and stored using a folder-based naming convention in the *Platforms\Windows\Strings* folder. Folders should be named with the language, and optional culture. For example, the Spanish-language folder is named *es* and the Mexico locale folder should be named *es-MX*.

There's currently no Visual Studio item template for creating a Windows resource file in a .NET MAUI app. Therefore, to create a Windows resource file for each locale:

1. In the *Platforms\Windows* folder of your .NET MAUI app project, create a *Strings* folder.
2. In the *Strings* folder, create a folder for each locale.
3. In the folder for each locale, create a file named *Resources.resw* that contains the following XML:

XML

```
<?xml version="1.0" encoding="utf-8"?>
<root>
  <!--
    Microsoft ResX Schema

    Version 2.0
```

The primary goals of this format is to allow a simple XML format that is mostly human readable. The generation and parsing of the various data types are done through the TypeConverter classes associated with the data types.

Example:

```
... ado.net/XML headers & schema ...
<resheader name="resmimetype">text/microsoft-resx</resheader>
<resheader name="version">2.0</resheader>
<resheader name="reader">System.Resources.ResXResourceReader,
System.Windows.Forms, ...</resheader>
<resheader name="writer">System.Resources.ResXResourceWriter,
System.Windows.Forms, ...</resheader>
<data name="Name1"><value>this is my long string</value>
<comment>this is a comment</comment></data>
<data name="Color1" type="System.Drawing.Color,
System.Drawing">Blue</data>
<data name="Bitmap1" mimetype="application/x-
microsoft.net.object.binary.base64">
    <value>[base64 mime encoded serialized .NET Framework object]
</value>
</data>
<data name="Icon1" type="System.Drawing.Icon, System.Drawing"
mimetype="application/x-microsoft.net.object.bytearray.base64">
    <value>[base64 mime encoded string representing a byte array
form of the .NET Framework object]</value>
    <comment>This is a comment</comment>
</data>
```

There are any number of "resheader" rows that contain simple name/value pairs.

Each data row contains a name, and value. The row also contains a type or mimetype. Type corresponds to a .NET class that support text/value conversion through the TypeConverter architecture. Classes that don't support this are serialized and stored with the mimetype set.

The mimetype is used for serialized objects, and tells the ResXResourceReader how to depersist the object. This is currently not extensible. For a given mimetype the value must be set accordingly:

Note - application/x-microsoft.net.object.binary.base64 is the format that the ResXResourceWriter will generate, however the reader can read any of the formats listed below.

```
mimetype: application/x-microsoft.net.object.binary.base64
value    : The object must be serialized with
          :
          : System.Runtime.Serialization.Formatters.Binary.BinaryFormatter
          : and then encoded with base64 encoding.
```

```

mimetype: application/x-microsoft.net.object.soap.base64
value : The object must be serialized with
      :
System.Runtime.Serialization.Formatters.Soap.SoapFormatter
      : and then encoded with base64 encoding.

mimetype: application/x-microsoft.net.object.bytearray.base64
value : The object must be serialized into a byte array
      : using a System.ComponentModel.TypeConverter
      : and then encoded with base64 encoding.

-->
<xsd:schema id="root" xmlns=""
xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:msdata="urn:schemas-
microsoft-com:xml-msdata">
  <xsd:import namespace="http://www.w3.org/XML/1998/namespace" />
  <xsd:element name="root" msdata:IsDataSet="true">
    <xsd:complexType>
      <xsd:choice maxOccurs="unbounded">
        <xsd:element name="metadata">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="value" type="xsd:string"
minOccurs="0" />
            </xsd:sequence>
            <xsd:attribute name="name" use="required"
type="xsd:string" />
            <xsd:attribute name="type" type="xsd:string" />
            <xsd:attribute name="mimetype" type="xsd:string" />
            <xsd:attribute ref="xml:space" />
          </xsd:complexType>
        </xsd:element>
        <xsd:element name="assembly">
          <xsd:complexType>
            <xsd:attribute name="alias" type="xsd:string" />
            <xsd:attribute name="name" type="xsd:string" />
          </xsd:complexType>
        </xsd:element>
        <xsd:element name="data">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="value" type="xsd:string"
minOccurs="0" msdata:Ordinal="1" />
              <xsd:element name="comment" type="xsd:string"
minOccurs="0" msdata:Ordinal="2" />
            </xsd:sequence>
            <xsd:attribute name="name" type="xsd:string"
use="required" msdata:Ordinal="1" />
            <xsd:attribute name="type" type="xsd:string"
msdata:Ordinal="3" />
            <xsd:attribute name="mimetype" type="xsd:string"
msdata:Ordinal="4" />
            <xsd:attribute ref="xml:space" />
          </xsd:complexType>
        </xsd:element>
        <xsd:element name="resheader">

```

```

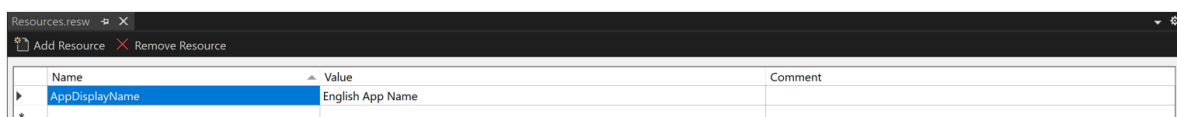
        <xsd:complexType>
            <xsd:sequence>
                <xsd:element name="value" type="xsd:string"
minOccurs="0" msdata:Ordinal="1" />
            </xsd:sequence>
            <xsd:attribute name="name" type="xsd:string"
use="required" />
        </xsd:complexType>
    </xsd:element>
</xsd:choice>
</xsd:complexType>
</xsd:element>
</xsd:schema>
<resheader name="resmimetype">
    <value>text/microsoft-resx</value>
</resheader>
<resheader name="version">
    <value>2.0</value>
</resheader>
<resheader name="reader">
    <value>System.Resources.ResXResourceReader, System.Windows.Forms,
Version=4.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</value>
</resheader>
<resheader name="writer">
    <value>System.Resources.ResXResourceWriter, System.Windows.Forms,
Version=4.0.0.0, Culture=neutral,
PublicKeyToken=b77a5c561934e089</value>
</resheader>
</root>

```

⚠ Note

Windows resource files use a build action of `PRResource`. This build action doesn't need setting on each `.resw` file in a .NET MAUI app, because it's implicitly applied.

4. Open each `Resources.resw` file and add a string resource that represents the app's name:



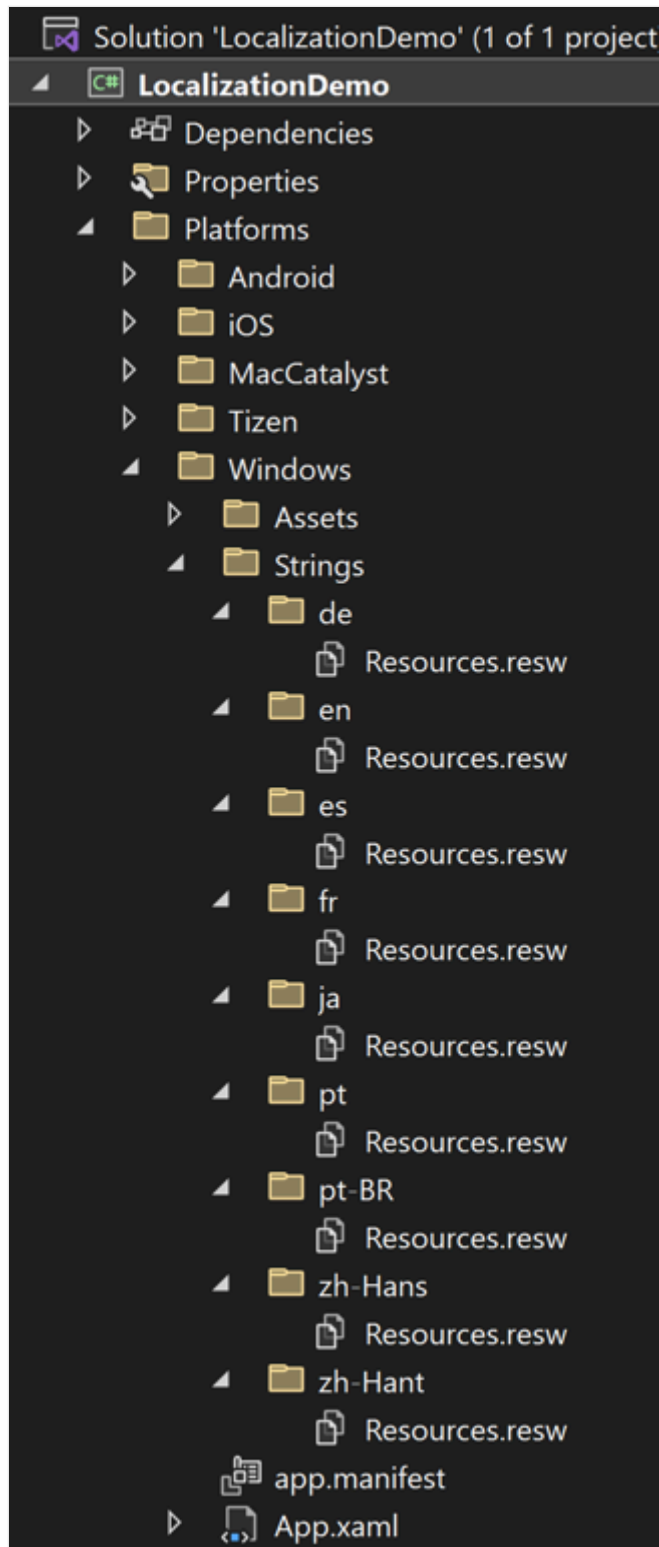
Name	Value	Comment
AppDisplayName	English App Name	

⚠ Note

Resource identifiers are case insensitive, and must be unique per resource file.

5. Save each Windows resource file.

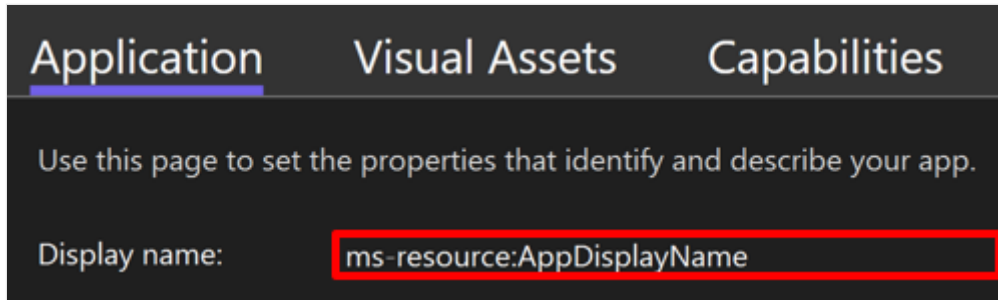
An example of the required folder and file structure is shown in the following screenshot:



Consume the localized app name

The string resource that represents the localized app name can be consumed by using the `ms-resource` URI scheme:

1. In Solution Explorer, open the *Packageappxmanifest* file in the package manifest editor.
2. In the package manifest editor, on the **Application** tab, set the **Display name** field to `ms-resource:` followed by the name of the string resource that identifies your app name:



3. Save your changes.

Important

If your *.resw* files are stored in a different assembly to your .NET MAUI app project, you'll have to specify a fully qualified path to your resource name. This uses the format `ms-resource:Assembly/ResourceFilename/Resource`.

Right-to-left localization

Flow direction, or layout direction, is the direction in which the UI elements on the page are scanned by the eye. Some languages, such as Arabic and Hebrew, require that UI elements are laid out in a right-to-left flow direction. .NET MAUI apps automatically respect the device's flow direction based on the selected language and region. For information about how to retrieve the flow direction of the device, based on its locale, see [Get the layout direction](#).

To override the flow direction of an app, set the [Window.FlowDirection](#) property. Alternatively, set the [VisualElement.FlowDirection](#) property on a per element basis. These properties get or set the direction in which UI elements flow within any parent element that controls their layout, and should be set to one of the [FlowDirection](#) enumeration values:

- `LeftToRight`
- `RightToLeft`
- `MatchParent`

Setting the [FlowDirection](#) property to `RightToLeft` on an element sets the alignment to the right, the reading order to right-to-left, and the layout of the control to flow from right-to-left.

⚠ Warning

Changing the [FlowDirection](#) property at runtime causes an expensive layout process that will affect performance.

The default [FlowDirection](#) property value for an element is `MatchParent`. Therefore, an element inherits the `FlowDirection` property value from its parent in the visual tree, and any element can override the value it gets from its parent.

💡 Tip

If you do need to change the flow direction, set the [FlowDirection](#) property on a window, page or root layout. This causes all of the elements contained within the app, page, or root layout, to respond appropriately to the flow direction.

Platform setup

Specific platform setup is required to enable right-to-left locales.

Android

Apps created using the .NET MAUI app project template automatically include support for right-to-left locales. This support is enabled by the `android:supportsRtl` attribute being set to `true` on the `<application>` node in the app's *AndroidManifest.xml* file:

XML

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <application ... android:supportsRtl="true" />
  ...
</manifest>
```

Right-to-left localization can then be tested by changing the device or emulator to use the right-to-left language. Alternatively, if you've activated developer options in the Settings app, you can enable **Force RTL layout direction** in **Settings > Developer**

Options. For information on configuring developer options, see [Configure on-device developer options](#) on developer.android.com.

iOS and Mac Catalyst

The required right-to-left locale should be added as a supported language to the array items for the `CFBundleLocalizations` key in *Info.plist*. The following example shows Arabic having been added to the array for the `CFBundleLocalizations` key:

XML

```
<key>CFBundleLocalizations</key>
<array>
  <string>en</string>
  <string>ar</string>
</array>
```

Right-to-left localization can then be tested by changing the language and region on the device or simulator to a right-to-left locale that was specified in *Info.plist*.

Windows

The required language resources should be specified in the `<Resources>` node of the *Package.appxmanifest* file. Replace `<Resource Language="x-generate">` with `<Resource />` elements for each of your supported languages. For example, the following markup specifies that "en" and "ar" localized resources are available:

XML

```
<Resources>
  <Resource Language="en" />
  <Resource Language="ar" />
</Resources>
```

Right-to-left localization can then be tested by changing the language and region on the device to the appropriate right-to-left locale.

Test localization

At runtime, your app loads the appropriate localized resources on a per-thread basis, based on the culture specified by the [CurrentUICulture](#) property.

Testing localization is best accomplished by changing your device language in the Settings app on each device.

 Warning

While it's possible to set the value of [CurrentUICulture](#) in code, the resulting behavior is inconsistent across platforms so this isn't recommended for testing.

Bindable properties

Article • 02/29/2024

.NET Multi-platform App UI (.NET MAUI) bindable properties extend Common Language Runtime (CLR) property functionality by backing a property with a [BindableProperty](#) type, instead of with a field. The purpose of bindable properties is to provide a property system that supports data binding, styles, templates, and values set through parent-child relationships. In addition, bindable properties can provide default values, validation of property values, and callbacks that monitor property changes.

In .NET MAUI apps, properties should be implemented as bindable properties to support one or more of the following features:

- Acting as a valid *target* property for data binding. For more information about target properties, see [Basic bindings](#).
- Setting the property through a style.
- Providing a default property value that's different from the default for the type of the property.
- Validating the value of the property.
- Monitoring property changes.

Examples of .NET MAUI bindable properties include `Label.Text`, `Button.BorderRadius`, and `StackLayout.Orientation`. Each bindable property has a corresponding `public static readonly` field of type [BindableProperty](#) that is exposed on the same class and that is the identifier of the bindable property. For example, the corresponding bindable property identifier for the `Label.Text` property is `Label.TextProperty`.

Create a bindable property

The process for creating a bindable property is as follows:

1. Create a [BindableProperty](#) instance with one of the `BindableProperty.Create` method overloads.
2. Define property accessors for the [BindableProperty](#) instance.

All [BindableProperty](#) instances must be created on the UI thread. This means that only code that runs on the UI thread can get or set the value of a bindable property. However, [BindableProperty](#) instances can be accessed from other threads by marshaling to the UI thread. For more information, see [Run code on the UI thread](#).

Create a property

To create a [BindableProperty](#) instance, the containing class must derive from the [BindableObject](#) class. However, the [BindableObject](#) class is high in the class hierarchy, so the majority of classes used for UI functionality support bindable properties.

A bindable property can be created by declaring a `public static readonly` property of type [BindableProperty](#). The bindable property should be set to the returned value of one of the `BindableProperty.Create` method overloads. The declaration should be within the body of [BindableObject](#) derived class, but outside of any member definitions.

At a minimum, an identifier must be specified when creating a [BindableProperty](#), along with the following parameters:

- The name of the [BindableProperty](#).
- The type of the property.
- The type of the owning object.
- The default value for the property. This ensures that the property always returns a particular default value when it is unset, and it can be different from the default value for the type of the property. The default value will be restored when the `ClearValue` method is called on the bindable property.

❗ Important

The naming convention for bindable properties is that the bindable property identifier must match the property name specified in the `Create` method, with "Property" appended to it.

The following code shows an example of a bindable property, with an identifier and values for the four required parameters:

C#

```
public static readonly BindableProperty IsExpandedProperty =  
    BindableProperty.Create ("IsExpanded", typeof(bool), typeof(Expander),  
    false);
```

This creates a [BindableProperty](#) instance named `IsExpandedProperty`, of type `bool`. The property is owned by the `Expander` class, and has a default value of `false`.

❗ Note

`Expander` is a control in .NET MAUI Community Toolkit. For more information, see [Expander](#).

Optionally, when creating a `BindableProperty` instance, the following parameters can be specified:

- The binding mode. This is used to specify the direction in which property value changes will propagate. In the default binding mode, changes will propagate from the *source* to the *target*. For more information, see [Basic bindings](#).
- A validation delegate that will be invoked when the property value is set. For more information, see [Validation callbacks](#).
- A property changed delegate that will be invoked when the property value has changed. For more information, see [Detect property changes](#).
- A property changing delegate that will be invoked when the property value will change. This delegate has the same signature as the property changed delegate.
- A coerce value delegate that will be invoked when the property value has changed. For more information, see [Coerce value callbacks](#).
- A `Func` that's used to initialize a default property value. For more information, see [Create a default value with a Func](#).

Create accessors

Property accessors are required to use property syntax to access a bindable property. The `Get` accessor should return the value that's contained in the corresponding bindable property. This can be achieved by calling the `GetValue` method, passing in the bindable property identifier on which to get the value, and then casting the result to the required type. The `Set` accessor should set the value of the corresponding bindable property. This can be achieved by calling the `SetValue` method, passing in the bindable property identifier on which to set the value, and the value to set.

The following code example shows accessors for the `IsExpanded` bindable property:

```
C#

public bool IsExpanded
{
    get => (bool)GetValue(IsExpandedProperty);
    set => SetValue(IsExpandedProperty, value);
}
```

Consume a bindable property

Once a bindable property has been created, it can be consumed from XAML or code. In XAML, this is achieved by declaring a namespace with a prefix, with the namespace declaration indicating the CLR namespace name, and optionally, an assembly name. For more information, see [XAML Namespaces](#).

The following code example demonstrates a XAML namespace for a custom type that contains a bindable property, which is defined within the same assembly as the application code that's referencing the custom type:

XAML

```
<ContentPage ... xmlns:local="clr-namespace:DataBindingDemos" ...>
    ...
</ContentPage>
```

The namespace declaration is used when setting the `IsExpanded` bindable property, as demonstrated in the following XAML code example:

XAML

```
<Expander IsExpanded="true">
    ...
</Expander>
```

The equivalent C# code is shown in the following code example:

C#

```
Expander expander = new Expander
{
    IsExpanded = true
};
```

Advanced scenarios

When creating a `BindableProperty` instance, there are a number of optional parameters that can be set to enable advanced bindable property scenarios. This section explores these scenarios.

Detect property changes

A `static` property-changed callback method can be registered with a bindable property by specifying the `propertyChanged` parameter for the `BindableProperty.Create` method.

The specified callback method will be invoked when the value of the bindable property has changed.

The following code example shows how the `IsExpanded` bindable property registers the `OnIsExpandedChanged` method as a property-changed callback method:

C#

```
public static readonly BindableProperty IsExpandedProperty =
    BindableProperty.Create(nameof(IsExpanded), typeof(bool),
    typeof(Expander), false, propertyChanged: OnIsExpandedChanged);
...

static void OnIsExpandedChanged (BindableObject bindable, object oldValue,
object newValue)
{
    // Property changed implementation goes here
}
```

In the property-changed callback method, the `BindableObject` parameter is used to denote which instance of the owning class has reported a change, and the values of the two `object` parameters represent the old and new values of the bindable property.

Validation callbacks

A `static` validation callback method can be registered with a bindable property by specifying the `validateValue` parameter for the `BindableProperty.Create` method. The specified callback method will be invoked when the value of the bindable property is set.

The following code example shows how the `Angle` bindable property registers the `IsValidValue` method as a validation callback method:

C#

```
public static readonly BindableProperty AngleProperty =
    BindableProperty.Create("Angle", typeof(double), typeof(MainPage), 0.0,
    validateValue: IsValidValue);
...

static bool IsValidValue(BindableObject view, object value)
{
    double result;
    double.TryParse(value.ToString(), out result);
    return (result >= 0 && result <= 360);
}
```

Validation callbacks are provided with a value, and should return `true` if the value is valid for the property, otherwise `false`. An exception will be raised if a validation callback returns `false`, which you should handle. A typical use of a validation callback method is constraining the values of integers or doubles when the bindable property is set. For example, the `IsValidValue` method checks that the property value is a `double` within the range 0 to 360.

Coerce value callbacks

A `static` coerce value callback method can be registered with a bindable property by specifying the `coerceValue` parameter for the `BindableProperty.Create` method. The specified callback method will be invoked when the value of the bindable property is about to change, so that you can adjust the new value before it's applied.

Important

In addition to being triggered by the bindable property engine, you can invoke coerce-value callbacks from code. The [BindableObject](#) type has a `CoerceValue` method that can be called to force a reevaluation of the value of its [BindableProperty](#) argument, by invoking its coerce value callback.

Coerce value callbacks are used to force a reevaluation of a bindable property when the value of the property is about to change. For example, a coerce value callback can be used to ensure that the value of one bindable property is not greater than the value of another bindable property.

The following code example shows how the `Angle` bindable property registers the `CoerceAngle` method as a coerce value callback method:

C#

```
public static readonly BindableProperty AngleProperty =
    BindableProperty.Create("Angle", typeof(double), typeof(MainPage), 0.0,
        coerceValue: CoerceAngle);
public static readonly BindableProperty MaximumAngleProperty =
    BindableProperty.Create("MaximumAngle", typeof(double),
        typeof(MainPage), 360.0, propertyChanged: ForceCoerceValue);
...

static object CoerceAngle(BindableObject bindable, object value)
{
    MainPage page = bindable as MainPage;
    double input = (double)value;
```

```

        if (input > page.MaximumAngle)
        {
            input = page.MaximumAngle;
        }

        return input;
    }

    static void ForceCoerceValue(BindableObject bindable, object oldValue,
    object newValue)
    {
        bindable.CoerceValue(AngleProperty);
    }

```

The `CoerceAngle` method checks the value of the `MaximumAngle` property, and if the `Angle` property value is greater than it, it coerces the value to the `MaximumAngle` property value. In addition, when the `MaximumAngle` property changes the coerce value callback is invoked on the `Angle` property by calling the `CoerceValue` method.

Create a default value with a Func

A `Func` can be used to initialize the default value of a bindable property, as demonstrated in the following example:

C#

```

public static readonly BindableProperty DateProperty =
    BindableProperty.Create("Date", typeof(DateTime), typeof(MyPage),
    default(DateTime), BindingMode.TwoWay, defaultValueCreator: bindable =>
    DateTime.Today);

```

The `defaultValueCreator` parameter is set to a `Func` that returns a `DateTime` that represents today's date.

Attached properties

Article • 02/29/2024

.NET Multi-platform App UI (.NET MAUI) attached properties enable an object to assign a value for a property that its own class doesn't define. For example, child elements can use attached properties to inform their parent element of how they are to be presented in the user interface. The [Grid](#) layout enables the row and column of a child to be specified by setting the `Grid.Row` and `Grid.Column` attached properties. `Grid.Row` and `Grid.Column` are attached properties because they are set on elements that are children of a [Grid](#), rather than on the [Grid](#) itself.

Bindable properties should be implemented as attached properties in the following scenarios:

- When there's a need to have a property setting mechanism available for classes other than the defining class.
- When the class represents a service that needs to be easily integrated with other classes.

For more information about bindable properties, see [Bindable properties](#).

Create an attached property

The process for creating an attached property is as follows:

1. Create a [BindableProperty](#) instance with one of the `CreateAttached` method overloads.
2. Provide `static Get<PropertyName>` and `Set<PropertyName>` methods as accessors for the attached property.

Create a property

When creating an attached property for use on other types, the class where the property is created does not have to derive from [BindableObject](#). However, the *target* property for accessors should be of, or derive from, [BindableObject](#). For more information about target properties, see [Basic bindings](#).

An attached property can be created by declaring a `public static readonly` property of type [BindableProperty](#). The bindable property should be set to the returned value of one of the `BindableProperty.CreateAttached` method overloads. The declaration should be within the body of the owning class, but outside of any member definitions.

Important

The naming convention for attached properties is that the attached property identifier must match the property name specified in the `CreateAttached` method, with "Property" appended to it.

The following code shows an example of an attached property:

C#

```
public class MyControl
{
    public static readonly BindableProperty HasShadowProperty =
        BindableProperty.CreateAttached("HasShadow", typeof(bool),
        typeof(MyControl), false);
}
```

This creates an attached property named `HasShadowProperty`, of type `bool`. The property is owned by the `MyControl` class, and has a default value of `false`. Ownership, in this case, means that the property will be accessed in XAML using the format `Type.Property`, for example `MyControl.HasShadow`.

For more information about creating bindable properties, including parameters that can be specified during creation, see [Create a bindable property](#).

Create accessors

Static `Get<PropertyName>` and `Set<PropertyName>` methods are required as accessors for the attached property, otherwise the property system will be unable to use the attached property. The `Get<PropertyName>` accessor should conform to the following signature:

C#

```
public static valueType GetPropertyName(BindableObject target)
```

The `Get<PropertyName>` accessor should return the value that's contained in the corresponding [BindableProperty](#) field for the attached property. This can be achieved by calling the `GetValue` method, passing in the bindable property identifier on which to get the value, and then casting the resulting value to the required type.

The `Set<PropertyName>` accessor should conform to the following signature:

C#

```
public static void SetPropertyNames(BindableObject target, valueType value)
```

The `SetPropertyName` accessor should set the value of the corresponding `BindableProperty` field for the attached property. This can be achieved by calling the `SetValue` method, passing in the bindable property identifier on which to set the value, and the value to set.

For both accessors, the *target* object should be of, or derive from, `BindableObject`. For more information about target objects, see [Basic bindings](#).

The following code example shows accessors for the `HasShadow` attached property:

C#

```
public class MyControl
{
    public static bool GetHasShadow(BindableObject view)
    {
        return (bool)view.GetValue (HasShadowProperty);
    }

    public static void SetHasShadow(BindableObject view, bool value)
    {
        view.SetValue(HasShadowProperty, value);
    }
}
```

Consume an attached property

Once an attached property has been created, it can be consumed from XAML or code. In XAML, this is achieved by declaring a namespace with a prefix, with the namespace declaration indicating the Common Language Runtime (CLR) namespace name, and optionally an assembly name. For more information, see [XAML Namespaces](#).

The following example demonstrates a XAML namespace for a custom type that contains an attached property, which is defined within the same assembly as the app code that's referencing the custom type:

XAML

```
<ContentPage ... xmlns:local="clr-namespace:ShadowDemo" ...>
...
</ContentPage>
```

The namespace declaration is then used when setting the attached property on a specific control, as demonstrated in the following XAML:

XAML

```
<Label Text="Label with shadow" local:MyControl.HasShadow="true" />
```

The equivalent C# code is shown in the following code example:

C#

```
Label label = new Label { Text = "Label with shadow" };  
MyControl.SetHasShadow (label, true);
```

Consume an attached property with a style

Attached properties can also be added to a control by a style. The following XAML code example shows an *explicit* style for [Label](#) controls that uses the `HasShadow` attached property:

XAML

```
<Style x:Key="ShadowStyle" TargetType="Label">  
    <Style.Setters>  
        <Setter Property="local:MyControl.HasShadow" Value="true" />  
    </Style.Setters>  
</Style>
```

The [Style](#) can be applied to a [Label](#) by setting its [Style](#) property to the [Style](#) instance using the [StaticResource](#) markup extension, as demonstrated in the following code example:

XAML

```
<Label Text="Label with shadow" Style="{StaticResource ShadowStyle}" />
```

For more information about styles, see [Styles](#).

Advanced scenarios

When creating an attached property, there are some optional parameters that can be set to enable advanced attached property scenarios. This includes detecting property

changes, validating property values, and coercing property values. For more information, see [Advanced scenarios](#).

Publish and subscribe to messages

Article • 05/05/2023

The publish-subscribe pattern is a messaging pattern in which publishers send messages without having knowledge of any receivers, known as subscribers. Similarly, subscribers listen for specific messages, without having knowledge of any publishers.

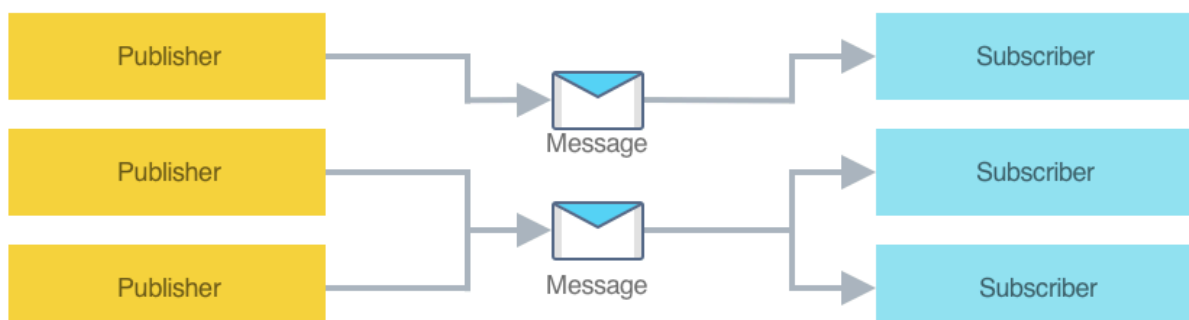
Events in .NET implement the publish-subscribe pattern, and are the most simple and straightforward approach for a communication layer between components if loose coupling is not required, such as a control and the page that contains it. However, the publisher and subscriber lifetimes are coupled by object references to each other, and the subscriber type must have a reference to the publisher type. This can create memory management issues, especially when there are short lived objects that subscribe to an event of a static or long-lived object. If the event handler isn't removed, the subscriber will be kept alive by the reference to it in the publisher, and this will prevent or delay the garbage collection of the subscriber.

The .NET Multi-platform App UI (.NET MAUI) `MessagingCenter` class implements the publish-subscribe pattern, allowing message-based communication between components that are inconvenient to link by object and type references. This mechanism allows publishers and subscribers to communicate without having a reference to each other, helping to reduce dependencies between them.

Important

`MessagingCenter` has been deprecated and we recommend that you replace it with `WeakReferenceMessenger` in the [CommunityToolkit.Mvvm](#) NuGet package. For more information, see [Messenger](#).

The `MessagingCenter` class provides multicast publish-subscribe functionality. This means that there can be multiple publishers that publish a single message, and there can be multiple subscribers listening for the same message:



Publishers send messages using the `MessagingCenter.Send` method, while subscribers listen for messages using the `MessagingCenter.Subscribe` method. In addition, subscribers can also unsubscribe from message subscriptions, if required, with the `MessagingCenter.Unsubscribe` method.

Important

Internally, the `MessagingCenter` class uses weak references. This means that it will not keep objects alive, and will allow them to be garbage collected. Therefore, it should only be necessary to unsubscribe from a message when a class no longer wishes to receive the message.

Publish a message

`MessagingCenter` messages are strings. Publishers notify subscribers of a message with one of the `MessagingCenter.Send` overloads. The following code example publishes a `Hi` message:

C#

```
MessagingCenter.Send<MainPage>(this, "Hi");
```

In this example the `Send` method specifies a generic argument that represents the sender. To receive the message, a subscriber must also specify the same generic argument, indicating that they are listening for a message from that sender. In addition, this example specifies two method arguments:

- The first argument specifies the sender instance.
- The second argument specifies the message.

Payload data can also be sent with a message:

C#

```
MessagingCenter.Send<MainPage, string>(this, "Hi", "John");
```

In this example, the `Send` method specifies two generic arguments. The first is the type that's sending the message, and the second is the type of the payload data being sent. To receive the message, a subscriber must also specify the same generic arguments. This enables multiple messages that share a message identity but send different payload data types to be received by different subscribers. In addition, this example specifies a

third method argument that represents the payload data to be sent to the subscriber. In this case the payload data is a `string`.

The `Send` method will publish the message, and any payload data, using a fire-and-forget approach. Therefore, the message is sent even if there are no subscribers registered to receive the message. In this situation, the sent message is ignored.

Subscribe to a message

Subscribers can register to receive a message using one of the `MessagingCenter.Subscribe` overloads. The following code example shows an example of this:

C#

```
MessagingCenter.Subscribe<MainPage> (this, "Hi", (sender) =>
{
    // Do something whenever the "Hi" message is received
});
```

In this example, the `Subscribe` method subscribes the `this` object to `Hi` messages that are sent by the `MainPage` type, and executes a callback delegate in response to receiving the message. The callback delegate, specified as a lambda expression, could be code that updates the UI, saves some data, or triggers some other operation.

ⓘ Note

A subscriber might not need to handle every instance of a published message, and this can be controlled by the generic type arguments that are specified on the `Subscribe` method.

The following example shows how to subscribe to a message that contains payload data:

C#

```
MessagingCenter.Subscribe<MainPage, string>(this, "Hi", async (sender, arg)
=>
{
    await DisplayAlert("Message received", "arg=" + arg, "OK");
});
```

In this example, the `Subscribe` method subscribes to `Hi` messages that are sent by the `MainPage` type, whose payload data is a `string`. A callback delegate is executed in response to receiving such a message, that displays the payload data in an alert.

Important

The delegate that's executed by the `Subscribe` method will be executed on the same thread that publishes the message using the `Send` method.

Unsubscribe from a message

Subscribers can unsubscribe from messages they no longer want to receive. This is achieved with one of the `MessagingCenter.Unsubscribe` overloads:

C#

```
MessagingCenter.Unsubscribe<MainPage>(this, "Hi");
```

In this example, the `Unsubscribe` method unsubscribes the `this` object from the `Hi` message sent by the `MainPage` type.

Messages containing payload data should be unsubscribed from using the `Unsubscribe` overload that specifies two generic arguments:

C#

```
MessagingCenter.Unsubscribe<MainPage, string>(this, "Hi");
```

In this example, the `Unsubscribe` method unsubscribes the `this` object from the `Hi` message sent by the `MainPage` type, whose payload data is a `string`.

Resource dictionaries

Article • 09/25/2024

A .NET Multi-platform App UI (.NET MAUI) [ResourceDictionary](#) is a repository for resources that are used by a .NET MAUI app. Typical resources that are stored in a [ResourceDictionary](#) include styles, control templates, data templates, converters, and colors.

XAML resources that are stored in a [ResourceDictionary](#) can be referenced and applied to elements by using the [StaticResource](#) or [DynamicResource](#) markup extension. In C#, resources can also be defined in a [ResourceDictionary](#) and then referenced and applied to elements by using a string-based indexer.

💡 Tip

In Visual Studio, a XAML-based [ResourceDictionary](#) file that's backed by a code-behind file can be added to your project by the **.NET MAUI ResourceDictionary (XAML)** item template.

Create resources

Every [VisualElement](#) derived object has a `Resources` property, which is a [ResourceDictionary](#) that can contain resources. Similarly, an `Application` derived object has a `Resources` property, which is a [ResourceDictionary](#) that can contain resources.

A .NET MAUI app can contain only a single class that derives from `Application`, but often makes use of many classes that derive from [VisualElement](#), including pages, layouts, and views. Any of these objects can have its `Resources` property set to a [ResourceDictionary](#) containing resources. Choosing where to put a particular [ResourceDictionary](#) impacts where the resources can be used:

- Resources in a [ResourceDictionary](#) that is attached to a view, such as [Button](#) or [Label](#), can only be applied to that particular object.
- Resources in a [ResourceDictionary](#) attached to a layout, such as [StackLayout](#) or [Grid](#), can be applied to the layout and all the children of that layout.
- Resources in a [ResourceDictionary](#) defined at the page level can be applied to the page and to all its children.
- Resources in a [ResourceDictionary](#) defined at the application level can be applied throughout the app.

With the exception of implicit styles, each resource in resource dictionary must have a unique string key that's defined with the `x:Key` attribute.

The following XAML shows resources defined in an application level [ResourceDictionary](#) in the **App.xaml** file:

XAML

```
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="ResourceDictionaryDemo.App">
    <Application.Resources>

        <Thickness x:Key="PageMargin">20</Thickness>

        <!-- Colors -->
        <Color x:Key="AppBackgroundColor">AliceBlue</Color>
        <Color x:Key="NavigationBarColor">#1976D2</Color>
        <Color x:Key="NavigationBarTextColor">White</Color>
        <Color x:Key="NormalTextColor">Black</Color>

        <!-- Images -->
        <x:String x:Key="BackgroundImage">background</x:String>
        <x:String x:Key="MenuIcon">menu.png</x:String>
        <x:String x:Key="SearchIcon">search.png</x:String>

        <!-- Implicit styles -->
        <Style TargetType="NavigationPage">
            <Setter Property="BarBackgroundColor"
                Value="{StaticResource NavigationBarColor}" />
            <Setter Property="BarTextColor"
                Value="{StaticResource NavigationBarTextColor}" />
        </Style>

        <Style TargetType="ContentPage"
            ApplyToDerivedTypes="True">
            <Setter Property="BackgroundColor"
                Value="{StaticResource AppBackgroundColor}" />
        </Style>

    </Application.Resources>
</Application>
```

In this example, the resource dictionary defines a `Thickness` resource, multiple `Color` resources, and two implicit `Style` resources.

❗ Important

Inserting resources directly between the `Resources` property-element tags automatically creates a [ResourceDictionary](#) object. However, it's also valid to place

all resources between optional [ResourceDictionary](#) tags.

Consume resources

Each resource has a key that is specified using the `x:Key` attribute, which becomes its dictionary key in the [ResourceDictionary](#). The key is used to reference a resource from the [ResourceDictionary](#) with the [StaticResource](#) or [DynamicResource](#) XAML markup extension.

The [StaticResource](#) markup extension is similar to the [DynamicResource](#) markup extension in that both use a dictionary key to reference a value from a resource dictionary. However, while the [StaticResource](#) markup extension performs a single dictionary lookup, the [DynamicResource](#) markup extension maintains a link to the dictionary key. Therefore, if the dictionary entry associated with the key is replaced, the change is applied to the visual element. This enables runtime resource changes to be made in an app. For more information about markup extensions, see [XAML markup extensions](#).

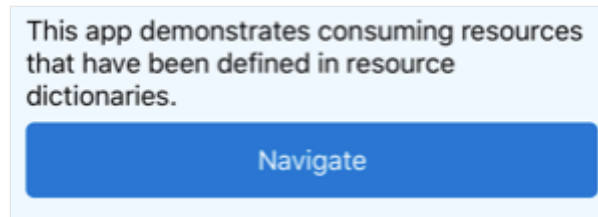
The following XAML example shows how to consume resources, and also define an additional resource in a [StackLayout](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="ResourceDictionaryDemo.MainPage"
             Title="Main page">
    <StackLayout Margin="{StaticResource PageMargin}"
                 Spacing="6">
        <StackLayout.Resources>
            <!-- Implicit style -->
            <Style TargetType="Button">
                <Setter Property="FontSize" Value="14" />
                <Setter Property="BackgroundColor" Value="#1976D2" />
                <Setter Property="TextColor" Value="White" />
                <Setter Property="CornerRadius" Value="5" />
            </Style>
        </StackLayout.Resources>

        <Label Text="This app demonstrates consuming resources that have
        been defined in resource dictionaries." />
        <Button Text="Navigate"
                Clicked="OnNavigateButtonClicked" />
    </StackLayout>
</ContentPage>
```

In this example, the [ContentPage](#) object consumes the implicit style defined in the application level resource dictionary. The [StackLayout](#) object consumes the `PageMargin` resource defined in the application level resource dictionary, while the [Button](#) object consumes the implicit style defined in the [StackLayout](#) resource dictionary. This results in the appearance shown in the following screenshot:



Important

Resources that are specific to a single page shouldn't be included in an application level resource dictionary, as such resources will then be parsed at app startup instead of when required by a page. For more information, see [Reduce the application resource dictionary size](#).

Resource lookup behavior

The following lookup process occurs when a resource is referenced with the [StaticResource](#) or [DynamicResource](#) markup extension:

- The requested key is checked for in the resource dictionary, if it exists, for the element that sets the property. If the requested key is found, its value is returned and the lookup process terminates.
- If a match isn't found, the lookup process searches the visual tree upwards, checking the resource dictionary of each parent element. If the requested key is found, its value is returned and the lookup process terminates. Otherwise the process continues upwards until the root element is reached.
- If a match isn't found at the root element, the application level resource dictionary is examined.
- If a match still isn't found, a `XamlParseException` is thrown.

Therefore, when the XAML parser encounters a [StaticResource](#) or [DynamicResource](#) markup extension, it searches for a matching key by traveling up through the visual tree, using the first match it finds. If this search ends at the page and the key still hasn't been found, the XAML parser searches the [ResourceDictionary](#) attached to the `App` object. If the key still isn't found, an exception is thrown.

Override resources

When resources share keys, resources defined lower in the visual tree will take precedence over those defined higher up. For example, setting an `AppBackgroundColor` resource to `AliceBlue` at the application level will be overridden by a page level `AppBackgroundColor` resource set to `Teal`. Similarly, a page level `AppBackgroundColor` resource will be overridden by a layout or view level `AppBackgroundColor` resource.

Stand-alone resource dictionaries

A [ResourceDictionary](#) can also be created as a stand-alone XAML file that isn't backed by a code-behind file. To create a stand-alone [ResourceDictionary](#), add a new [ResourceDictionary](#) file to the project with the **.NET MAUI ResourceDictionary (XAML)** item template and delete its code-behind file. Then, in the XAML file remove the `x:Class` attribute from the [ResourceDictionary](#) tag near the start of the file. By default, a stand-alone [ResourceDictionary](#) has its XAML compiled, unless `<?xml-comp compile="false" ?>` is specified after the XML header.

❗ Note

A stand-alone [ResourceDictionary](#) must have a build action of **MauiXaml**.

The following XAML example shows a stand-alone [ResourceDictionary](#) named **MyResourceDictionary.xaml**:

XAML

```
<?xml version="1.0" encoding="UTF-8" ?>
<ResourceDictionary xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml">
    <DataTemplate x:Key="PersonDataTemplate">
        <ViewCell>
            <Grid RowSpacing="6"
                ColumnSpacing="6">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="0.5*" />
                    <ColumnDefinition Width="0.2*" />
                    <ColumnDefinition Width="0.3*" />
                </Grid.ColumnDefinitions>
                <Label Text="{Binding Name}"
                    TextColor="{StaticResource NormalTextColor}"
                    FontAttributes="Bold" />
                <Label Grid.Column="1"
                    Text="{Binding Age}"
```

```

        TextColor="{StaticResource NormalTextColor}" />
    <Label Grid.Column="2"
        Text="{Binding Location}"
        TextColor="{StaticResource NormalTextColor}"
        HorizontalTextAlignment="End" />
    </Grid>
</ViewCell>
</DataTemplate>
</ResourceDictionary>

```

In this example, the [ResourceDictionary](#) contains a single resource, which is an object of type [DataTemplate](#). [MyResourceDictionary.xaml](#) can be consumed by merging it into another resource dictionary.

Merge resource dictionaries

Resource dictionaries can be combined by merging one or more [ResourceDictionary](#) objects into another [ResourceDictionary](#).

Merge local resource dictionaries

A local [ResourceDictionary](#) file can be merged into another [ResourceDictionary](#) by creating a [ResourceDictionary](#) object whose `Source` property is set to the filename of the XAML file with the resources:

XAML

```

<ContentPage ...>
    <ContentPage.Resources>
        <!-- Add more resources here -->
        <ResourceDictionary Source="MyResourceDictionary.xaml" />
        <!-- Add more resources here -->
    </ContentPage.Resources>
    ...
</ContentPage>

```

This syntax does not instantiate the `MyResourceDictionary` class. Instead, it references the XAML file. For that reason, when setting the `Source` property, a code-behind file isn't required, and the `x:Class` attribute can be removed from the root tag of the [MyResourceDictionary.xaml](#) file.

 **Important**

The `ResourceDictionary.Source` property can only be set from XAML.

Merge resource dictionaries from other assemblies

A `ResourceDictionary` can also be merged into another `ResourceDictionary` by adding it into the `MergedDictionaries` property of the `ResourceDictionary`. This technique allows resource dictionaries to be merged, regardless of the assembly in which they reside. Merging resource dictionaries from external assemblies requires the `ResourceDictionary` to have a build action set to **MauiXaml**, to have a code-behind file, and to define the `x:Class` attribute in the root tag of the file.

⚠ Warning

The `ResourceDictionary` class also defines a `MergedWith` property. However, this property has been deprecated and should no longer be used.

The following code example shows two resource dictionaries being added to the `MergedDictionaries` collection of a page level `ResourceDictionary`:

XAML

```
<ContentPage ...
    xmlns:local="clr-namespace:ResourceDictionaryDemo"
    xmlns:theme="clr-namespace:MyThemes;assembly=MyThemes">
    <ContentPage.Resources>
        <ResourceDictionary>
            <!-- Add more resources here -->
            <ResourceDictionary.MergedDictionaries>
                <!-- Add more resource dictionaries here -->
                <local:MyResourceDictionary />
                <theme:DefaultTheme />
                <!-- Add more resource dictionaries here -->
            </ResourceDictionary.MergedDictionaries>
            <!-- Add more resources here -->
        </ResourceDictionary>
    </ContentPage.Resources>
    ...
</ContentPage>
```

In this example, a resource dictionary from the same assembly, and a resource dictionary from an external assembly, are merged into the page level resource dictionary. In addition, you can also add other `ResourceDictionary` objects within the `MergedDictionaries` property-element tags, and other resources outside of those tags.

❗ Important

There can be only one `MergedDictionaries` property-element tag in a [ResourceDictionary](#), but you can put as many [ResourceDictionary](#) objects in there as required.

When merged [ResourceDictionary](#) resources share identical `x:Key` attribute values, .NET MAUI uses the following resource precedence:

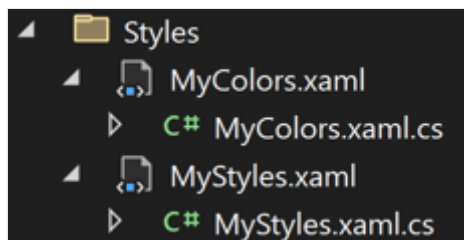
1. The resources local to the resource dictionary.
2. The resources contained in the resource dictionaries that were merged via the `MergedDictionaries` collection, in the reverse order they are listed in the `MergedDictionaries` property.

💡 Tip

Searching resource dictionaries can be a computationally intensive task if an app contains multiple, large resource dictionaries. Therefore, to avoid unnecessary searching, you should ensure that each page in an application only uses resource dictionaries that are appropriate to the page.

Consume a XAML-based resource dictionary from code

Resource dictionaries that are defined in XAML can be consumed in code, provided that the [ResourceDictionary](#) is backed by a code-behind file. In Visual Studio, XAML-based [ResourceDictionary](#) files that are backed by code-behind files can be added to your project by the **.NET MAUI ResourceDictionary (XAML)** item template:



XAML-based resource dictionaries that are backed by code-behind files can then be consumed from C# by adding them to the [MergedDictionaries](#) collection of the resource dictionary:

C#

```
Resources.MergedDictionaries.Add(new MyMauiApp.Resources.Styles.MyColors());  
Resources.MergedDictionaries.Add(new MyMauiApp.Resources.Styles.MyStyles());
```

Access resources by key from code

You can access resources in a resource dictionary from code like any other dictionary.

The following example shows how to retrieve and apply a resource from a page's resource dictionary:

C#

```
// Retrieve the Primary color value which is in the page's resource  
dictionary  
var hasValue = Resources.TryGetValue("Primary", out object primaryColor);  
  
if (hasValue)  
{  
    myLabel.TextColor = (Color)primaryColor;  
}
```

This is the recommended approach that ensures that .NET MAUI won't throw a `KeyNotFoundException` if it's unable to retrieve a resource from code. This can occur when a merged resource dictionary is made up of resources defined in a XAML file, and inline resources. For more information, see [GitHub issue #11214](#).

ⓘ Note

To retrieve app-wide resources from code, access the `App.Current.Resources` resource dictionary.

.NET MAUI Shell overview

Article • 08/30/2024



[Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) Shell reduces the complexity of app development by providing the fundamental features that most apps require, including:

- A single place to describe the visual hierarchy of an app.
- A common navigation user experience.
- A URI-based navigation scheme that permits navigation to any page in the app.
- An integrated search handler.

App visual hierarchy

In a .NET MAUI Shell app, the visual hierarchy of the app is described in a class that subclasses the [Shell](#) class. This class can consist of three main hierarchical objects:

1. [FlyoutItem](#) or [TabBar](#). A [FlyoutItem](#) represents one or more items in the flyout, and should be used when the navigation pattern for the app requires a flyout. A [TabBar](#) represents the bottom tab bar, and should be used when the navigation pattern for the app begins with bottom tabs and doesn't require a flyout. For more information about flyout items, see [.NET MAUI Shell flyout](#). For more information about tab bars, see [.NET MAUI Shell tabs](#).
2. [Tab](#), which represents grouped content, navigable by bottom tabs. For more information, see [.NET MAUI Shell tabs](#).
3. [ShellContent](#), which represents the [ContentPage](#) objects for each tab. For more information, see [.NET MAUI Shell pages](#).

These objects don't represent any user interface, but rather the organization of the app's visual hierarchy. Shell will take these objects and produce the navigation user interface for the content.

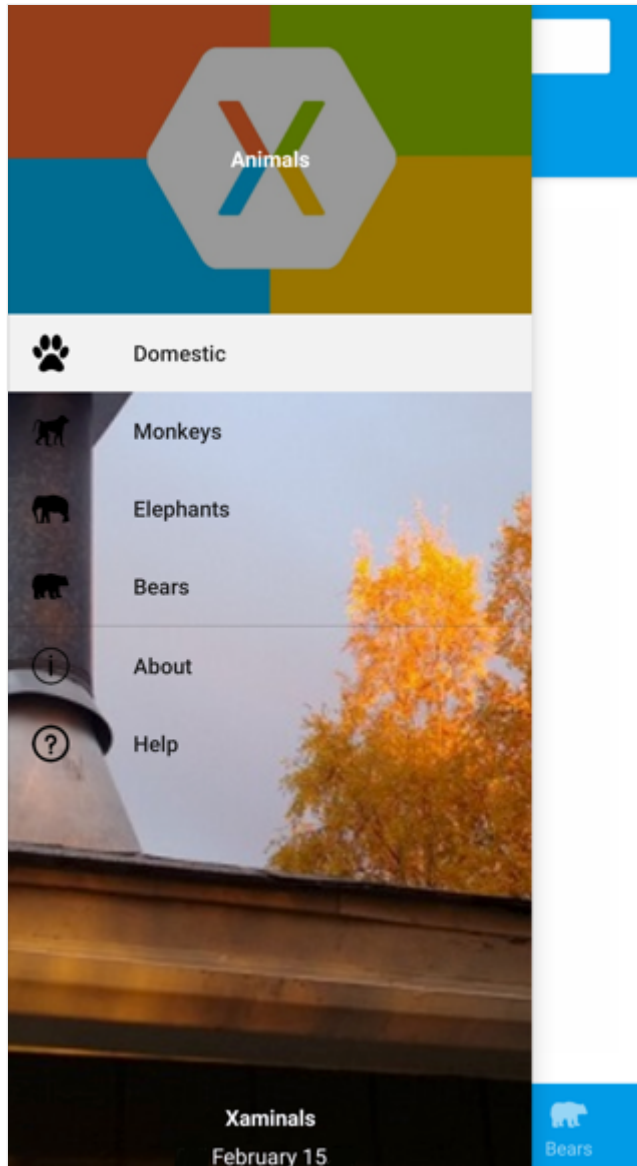
ⓘ Note

Pages are created on demand in Shell apps, in response to navigation.

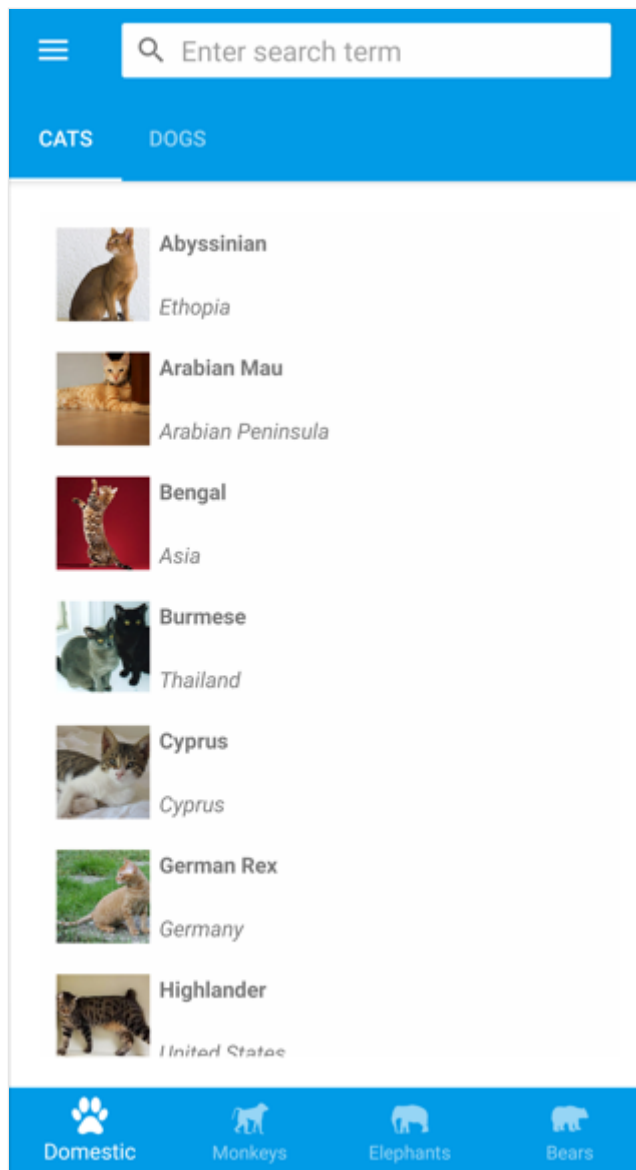
For more information, see [Create a .NET MAUI Shell app](#).

Navigation user experience

The navigation experience provided by .NET MAUI Shell is based on flyouts and tabs. The top level of navigation in a Shell app is either a flyout or a bottom tab bar, depending on the navigation requirements of the app. The following example shows an app where the top level of navigation is a flyout:



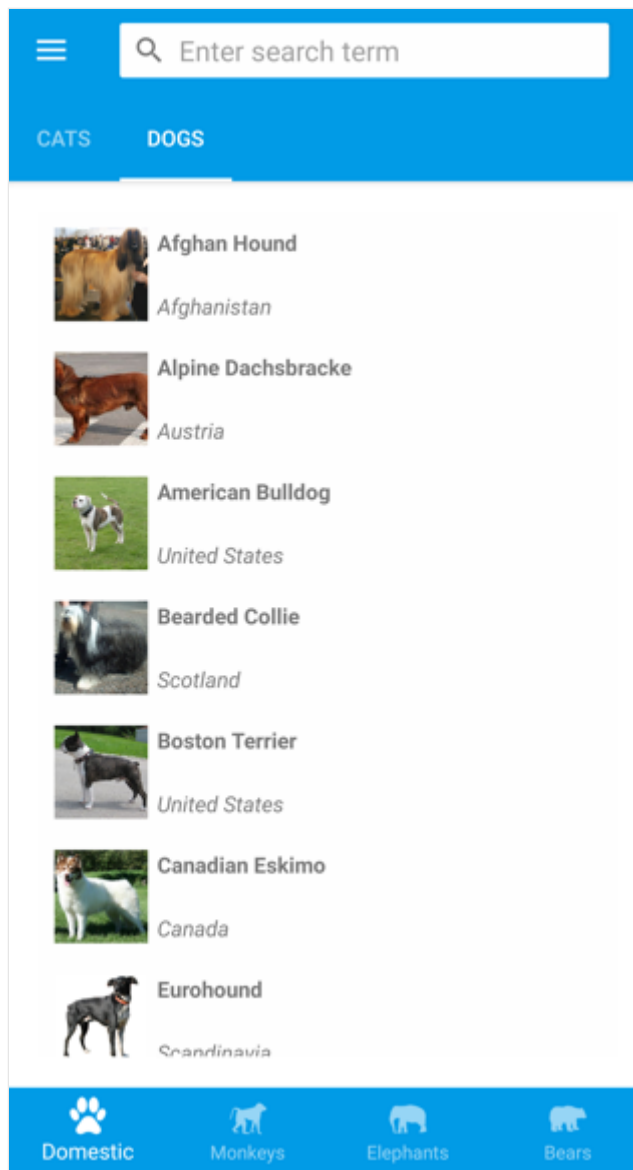
In this example, some flyout items are duplicated as tab bar items. However, there are also items that can only be accessed from the flyout. Selecting a flyout item results in the bottom tab that represents the item being selected and displayed:



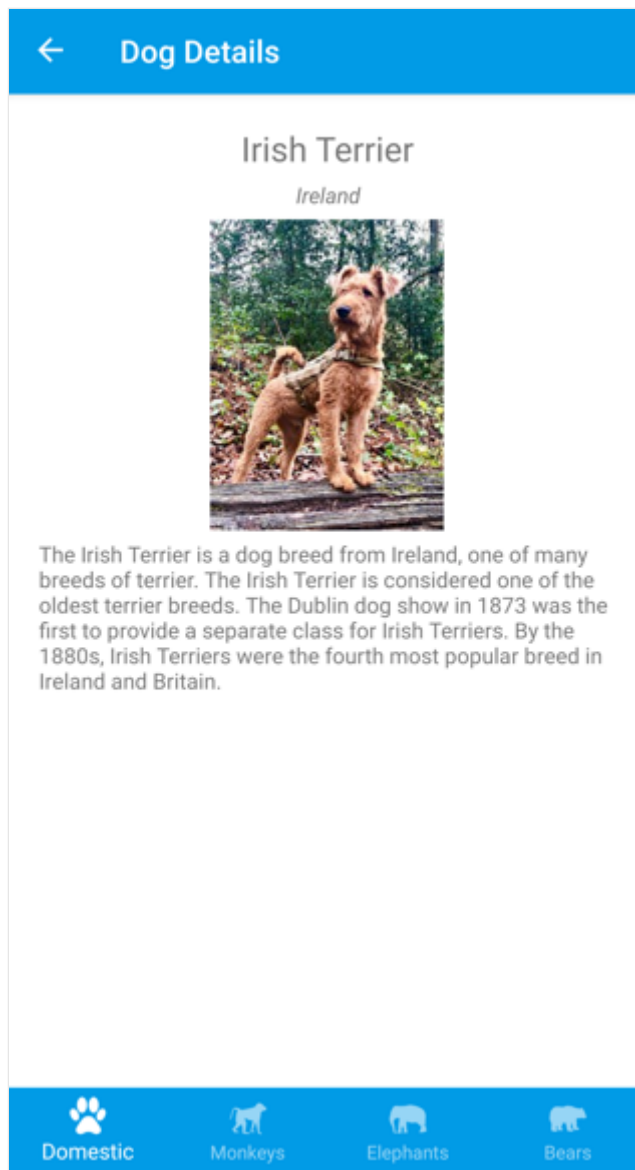
ⓘ Note

When the flyout isn't open the bottom tab bar can be considered to be the top level of navigation in the app.

Each tab on the tab bar displays a [ContentPage](#). However, if a bottom tab contains more than one page, the pages are navigable by the top tab bar:



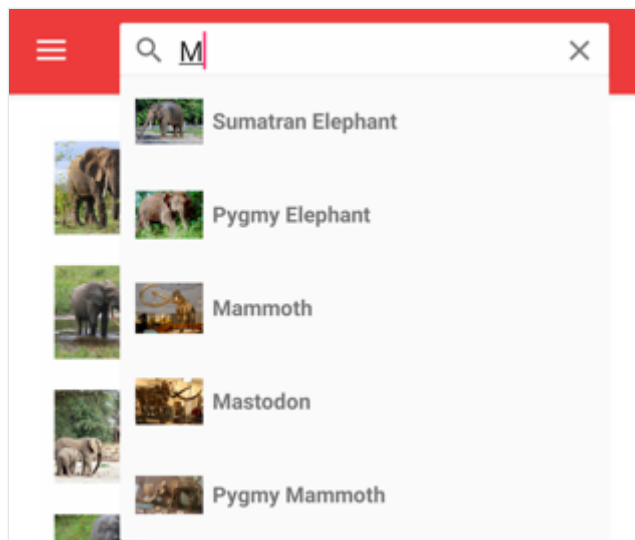
Within each tab, additional [ContentPage](#) objects that are known as detail pages, can be navigated to:



Shell uses a URI-based navigation experience that uses routes to navigate to any page in the app, without having to follow a set navigation hierarchy. In addition, it also provides the ability to navigate backwards without having to visit all of the pages on the navigation stack. For more information, see [.NET MAUI Shell navigation](#).

Search

.NET MAUI Shell includes integrated search functionality that's provided by the [SearchHandler](#) class. Search capability can be added to a page by adding a subclassed [SearchHandler](#) object to it. This results in a search box being added at the top of the page. When data is entered into the search box, the search suggestions area is populated with data:



Then, when a result is selected from the search suggestions area, custom logic can be executed such as navigating to a detail page.

For more information, see [.NET MAUI Shell search](#).

Create a .NET MAUI Shell app

Article • 08/30/2024

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) Shell app can be created with the **.NET MAUI App** project template, and then by describing the visual hierarchy of the app in the `AppShell` class.

For a step-by-step walkthrough of how to create a Shell app, see [Create a .NET MAUI app](#).

Describe the visual hierarchy of the app

The visual hierarchy of a .NET MAUI Shell app is described in the subclassed `Shell` class, which the project template names `AppShell`. A subclassed `Shell` class consists of three main hierarchical objects:

1. `FlyoutItem` or `TabBar`. A `FlyoutItem` represents one or more items in the flyout, and should be used when the navigation pattern for the app requires a flyout. A `TabBar` represents the bottom tab bar, and should be used when the navigation pattern for the app begins with bottom tabs and doesn't require a flyout. Every `FlyoutItem` object or `TabBar` object is a child of the `Shell` object.
2. `Tab`, which represents grouped content, navigable by bottom tabs. Every `Tab` object is a child of a `FlyoutItem` object or `TabBar` object.
3. `ShellContent`, which represents the `ContentPage` objects for each tab. Every `ShellContent` object is a child of a `Tab` object. When more than one `ShellContent` object is present in a `Tab`, the objects will be navigable by top tabs.

These objects don't represent any user interface, but rather the organization of the app's visual hierarchy. Shell will take these objects and produce the navigation user interface for the content.

The following XAML shows an example of a subclassed `Shell` class:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      xmlns:views="clr-namespace:Xaminals.Views"
      x:Class="Xaminals.AppShell">
    ...
    <FlyoutItem FlyoutDisplayOptions="AsMultipleItems">
```

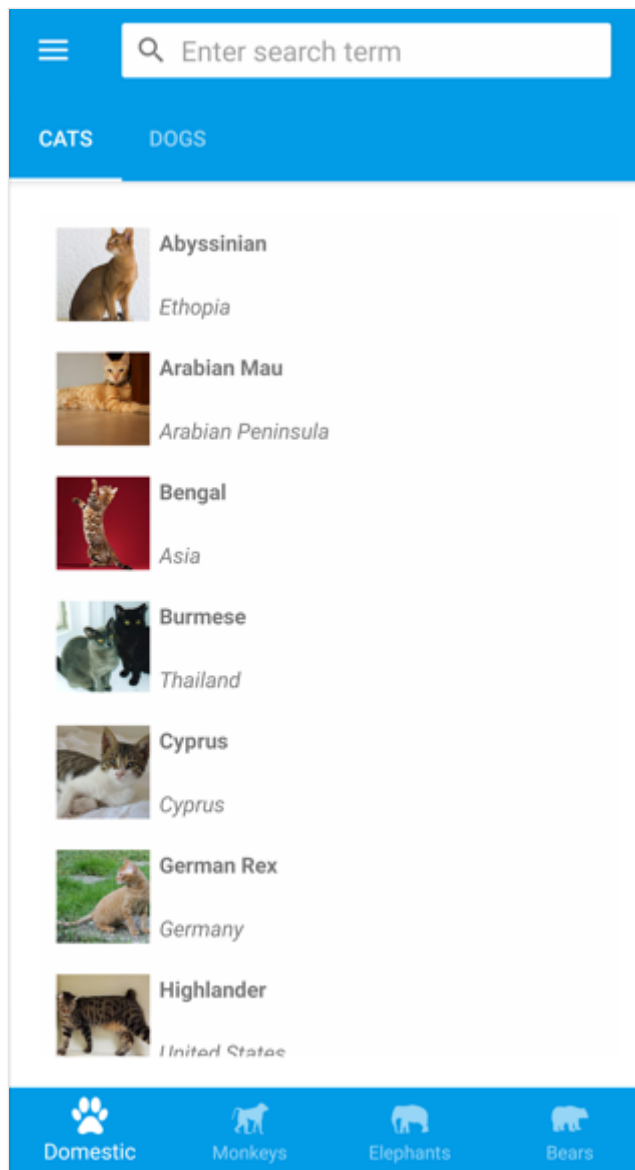
```

<Tab Title="Domestic"
      Icon="paw.png">
  <ShellContent Title="Cats"
                Icon="cat.png"
                ContentTemplate="{DataTemplate views:CatsPage}" />
  <ShellContent Title="Dogs"
                Icon="dog.png"
                ContentTemplate="{DataTemplate views:DogsPage}" />
</Tab>
<!--
  Shell has implicit conversion operators that enable the Shell visual
  hierarchy to be simplified.
  This is possible because a subclassed Shell object can only ever
  contain a FlyoutItem object or a TabBar object,
  which can only ever contain Tab objects, which can only ever contain
  ShellContent objects.

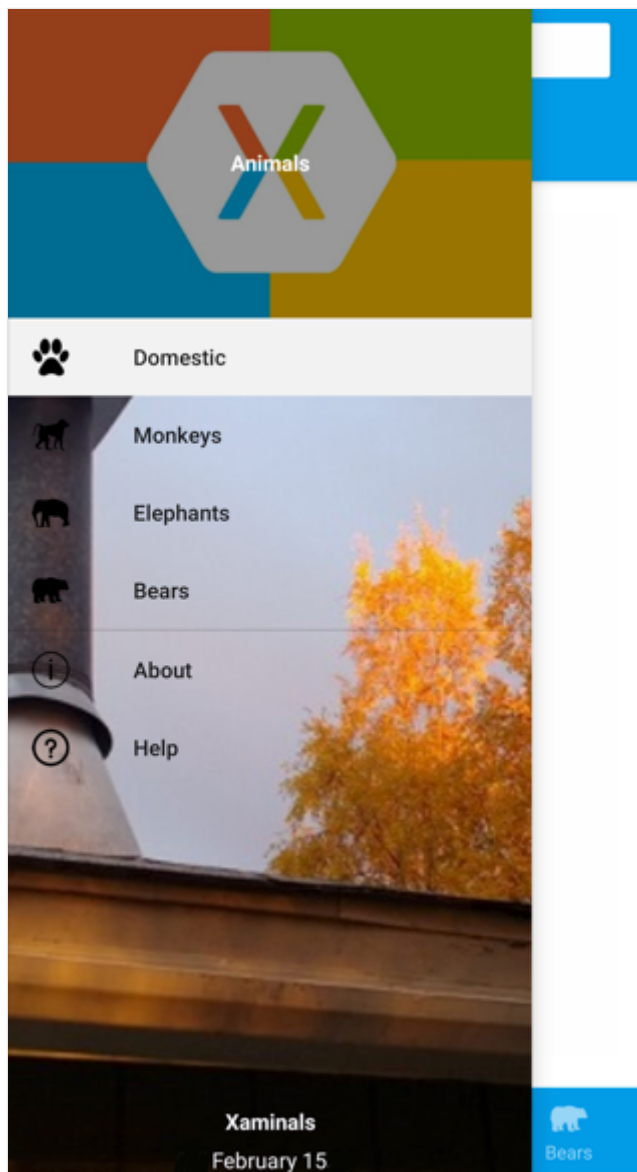
  The implicit conversion automatically wraps the ShellContent objects
  below in Tab objects.
-->
<ShellContent Title="Monkeys"
              Icon="monkey.png"
              ContentTemplate="{DataTemplate views:MonkeysPage}" />
<ShellContent Title="Elephants"
              Icon="elephant.png"
              ContentTemplate="{DataTemplate views:ElephantsPage}"
/>
<ShellContent Title="Bears"
              Icon="bear.png"
              ContentTemplate="{DataTemplate views:BearsPage}" />
</FlyoutItem>
...
</Shell>

```

When run, this XAML displays the `CatsPage`, because it's the first item of content declared in the subclassed `Shell` class:



Pressing the hamburger icon, or swiping from the left, displays the flyout:



Multiple items are displayed on the flyout because the [FlyoutDisplayOptions](#) property is set to `AsMultipleItems`. For more information, see [Flyout display options](#).

Important

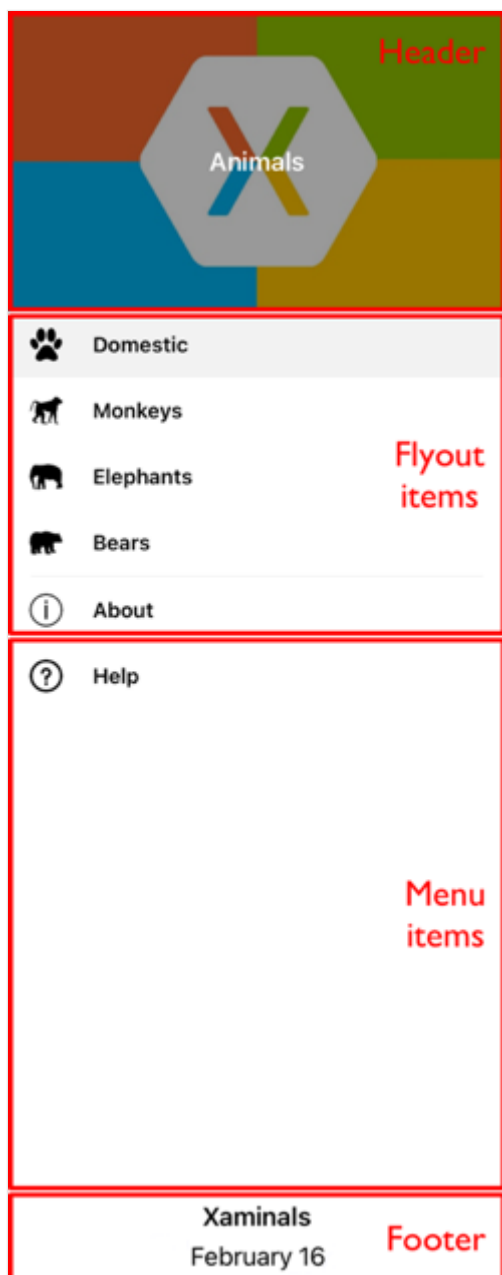
In a Shell app, pages are created on demand in response to navigation. This is accomplished by using the [DataTemplate](#) markup extension to set the `ContentTemplate` property of each [ShellContent](#) object to a [ContentPage](#) object.

.NET MAUI Shell flyout

Article • 08/30/2024

 [Browse the sample](#)

The navigation experience provided by .NET Multi-platform App UI (.NET MAUI) Shell is based on flyouts and tabs. A flyout is the optional root menu for a Shell app, and is fully customizable. It's accessible through an icon or by swiping from the side of the screen. The flyout consists of an optional header, flyout items, optional menu items, and an optional footer:



Flyout items

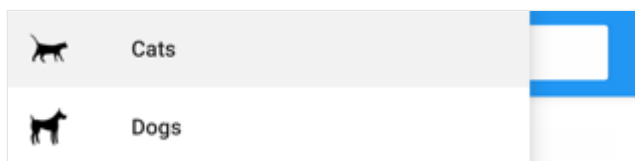
One or more flyout items can be added to the flyout, and each flyout item is represented by a [FlyoutItem](#) object. Each [FlyoutItem](#) object should be a child of the subclassed [Shell](#) object. Flyout items appear at the top of the flyout when a flyout header isn't present.

The following example creates a flyout containing two flyout items:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:controls="clr-namespace:Xaminals.Controls"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
  <FlyoutItem Title="Cats"
              Icon="cat.png">
    <Tab>
      <ShellContent ContentTemplate="{DataTemplate views:CatsPage}" />
    </Tab>
  </FlyoutItem>
  <FlyoutItem Title="Dogs"
              Icon="dog.png">
    <Tab>
      <ShellContent ContentTemplate="{DataTemplate views:DogsPage}" />
    </Tab>
  </FlyoutItem>
</Shell>
```

The `FlyoutItem.Title` property, of type `string`, defines the title of the flyout item. The `FlyoutItem.Icon` property, of type [ImageSource](#), defines the icon of the flyout item:



In this example, each [ShellContent](#) object can only be accessed through flyout items, and not through tabs. This is because by default, tabs will only be displayed if the flyout item contains more than one tab.

Important

In a Shell app, pages are created on demand in response to navigation. This is accomplished by using the [DataTemplate](#) markup extension to set the `ContentTemplate` property of each [ShellContent](#) object to a [ContentPage](#) object.

Shell has implicit conversion operators that enable the Shell visual hierarchy to be simplified, without introducing additional views into the visual tree. This is possible because a subclassed [Shell](#) object can only ever contain [FlyoutItem](#) objects or a [TabBar](#) object, which can only ever contain [Tab](#) objects, which can only ever contain [ShellContent](#) objects. These implicit conversion operators can be used to remove the [FlyoutItem](#) and [Tab](#) objects from the previous example:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      xmlns:controls="clr-namespace:Xaminals.Controls"
      xmlns:views="clr-namespace:Xaminals.Views"
      x:Class="Xaminals.AppShell">
  <ShellContent Title="Cats"
                Icon="cat.png"
                ContentTemplate="{DataTemplate views:CatsPage}" />
  <ShellContent Title="Dogs"
                Icon="dog.png"
                ContentTemplate="{DataTemplate views:DogsPage}" />
</Shell>
```

This implicit conversion automatically wraps each [ShellContent](#) object in [Tab](#) objects, which are wrapped in [FlyoutItem](#) objects.

ⓘ Note

All [FlyoutItem](#) objects in a subclassed [Shell](#) object are automatically added to the `Shell.FlyoutItems` collection, which defines the list of items that will be shown in the flyout.

Flyout display options

The `FlyoutItem.FlyoutDisplayOptions` property configures how a flyout item and its children are displayed in the flyout. This property should be set to a [FlyoutDisplayOptions](#) enumeration member:

- `AsSingleItem`, indicates that the item will be visible as a single item. This is the default value of the [FlyoutDisplayOptions](#) property.
- `AsMultipleItems`, indicates that the item and its direct children will be visible in the flyout as a group of items.

A flyout item for each [Tab](#) object within a [FlyoutItem](#) can be displayed by setting the `FlyoutItem.FlyoutDisplayOptions` property to `AsMultipleItems`:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:controls="clr-namespace:Xaminals.Controls"
        xmlns:views="clr-namespace:Xaminals.Views"
        FlyoutHeaderBehavior="CollapseOnScroll"
        x:Class="Xaminals.AppShell">

    <FlyoutItem FlyoutDisplayOptions="AsMultipleItems">
        <Tab Title="Domestic"
            Icon="paw.png">
            <ShellContent Title="Cats"
                Icon="cat.png"
                ContentTemplate="{DataTemplate views:CatsPage}" />
            <ShellContent Title="Dogs"
                Icon="dog.png"
                ContentTemplate="{DataTemplate views:DogsPage}" />
        </Tab>
        <ShellContent Title="Monkeys"
            Icon="monkey.png"
            ContentTemplate="{DataTemplate views:MonkeysPage}" />
        <ShellContent Title="Elephants"
            Icon="elephant.png"
            ContentTemplate="{DataTemplate views:ElephantsPage}"
        />

        <ShellContent Title="Bears"
            Icon="bear.png"
            ContentTemplate="{DataTemplate views:BearsPage}" />
    </FlyoutItem>

    <ShellContent Title="About"
        Icon="info.png"
        ContentTemplate="{DataTemplate views:AboutPage}" />

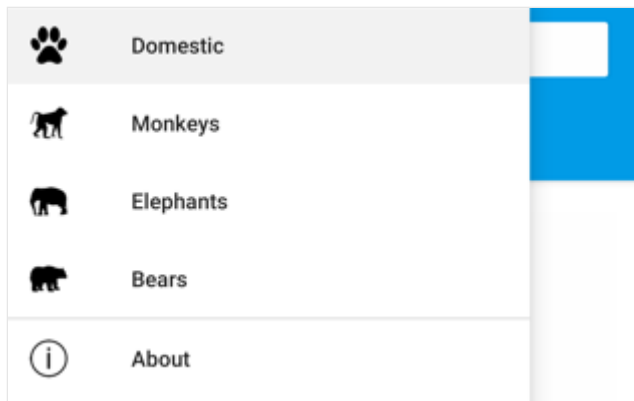
</Shell>
```

In this example, flyout items are created for the [Tab](#) object that's a child of the [FlyoutItem](#) object, and the [ShellContent](#) objects that are children of the [FlyoutItem](#) object. This occurs because each [ShellContent](#) object that's a child of the [FlyoutItem](#) object is automatically wrapped in a [Tab](#) object. In addition, a flyout item is created for the final [ShellContent](#) object, which is automatically wrapped in a [Tab](#) object, and then in a [FlyoutItem](#) object.

ⓘ Note

Tabs are displayed when a [FlyoutItem](#) contains more than one [ShellContent](#) object.

This results in the following flyout items:



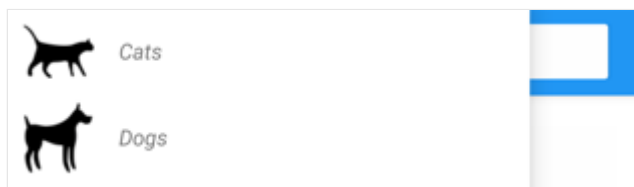
Define FlyoutItem appearance

The appearance of each [FlyoutItem](#) can be customized by setting the `Shell.ItemTemplate` attached property to a [DataTemplate](#):

XAML

```
<Shell ...>
  ...
  <Shell.ItemTemplate>
    <DataTemplate>
      <Grid ColumnDefinitions="0.2*,0.8*">
        <Image Source="{Binding FlyoutIcon}"
          Margin="5"
          HeightRequest="45" />
        <Label Grid.Column="1"
          Text="{Binding Title}"
          FontAttributes="Italic"
          VerticalTextAlignment="Center" />
      </Grid>
    </DataTemplate>
  </Shell.ItemTemplate>
</Shell>
```

This example displays the title of each [FlyoutItem](#) object in italics:



Because `Shell.ItemTemplate` is an attached property, different templates can be attached to specific [FlyoutItem](#) objects.

ⓘ Note

Shell provides the `Title` and `FlyoutIcon` properties to the `BindingContext` of the `ItemTemplate`.

In addition, Shell includes three style classes, which are automatically applied to `FlyoutItem` objects. For more information, see [Style FlyoutItem and MenuItem objects](#).

Default template for FlyoutItems

The default `DataTemplate` used for each `FlyoutItem` is shown below:

XAML

```
<DataTemplate x:Key="FlyoutTemplate">
  <Grid x:Name="FlyoutItemLayout"
    HeightRequest="{OnPlatform 44, Android=50}"
    ColumnSpacing="{OnPlatform WinUI=0}"
    RowSpacing="{OnPlatform WinUI=0}">
    <VisualStateManager.VisualStateGroups>
      <VisualStateGroupList>
        <VisualStateGroup x:Name="CommonStates">
          <VisualState x:Name="Normal">
            <VisualState.Setters>
              <Setter Property="BackgroundColor"
                Value="Transparent" />
            </VisualState.Setters>
          </VisualState>
          <VisualState x:Name="Selected">
            <VisualState.Setters>
              <Setter Property="BackgroundColor"
                Value="{AppThemeBinding Light=Black,
Dark=White}" />
            </VisualState.Setters>
          </VisualState>
        </VisualStateGroup>
      </VisualStateGroupList>
    </VisualStateManager.VisualStateGroups>
    <Grid.ColumnDefinitions>
      <ColumnDefinition Width="{OnPlatform Android=54, iOS=50,
WinUI=Auto}" />
      <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <Image x:Name="FlyoutItemImage"
      Source="{Binding FlyoutIcon}"
      VerticalOptions="Center"
      HorizontalOptions="{OnPlatform Default=Center, WinUI=Start}"
      HeightRequest="{OnPlatform Android=24, iOS=22, WinUI=16}"
      WidthRequest="{OnPlatform Android=24, iOS=22, WinUI=16}">
      <Image.Margin>
        <OnPlatform x:TypeArguments="Thickness">
          <OnPlatform.Platforms>
            <On Platform="WinUI">
```

```

        Value="12,0,12,0" />
    </OnPlatform.Platforms>
</OnPlatform>
</Image.Margin>
</Image>
<Label x:Name="FlyoutItemLabel"
    Grid.Column="1"
    Text="{Binding Title}"
    FontSize="{OnPlatform Android=14, iOS=14}"
    FontAttributes="{OnPlatform iOS=Bold}"
    HorizontalOptions="{OnPlatform WinUI=Start}"
    HorizontalTextAlignment="{OnPlatform WinUI=Start}"
    VerticalTextAlignment="Center">
    <Label.TextColor>
        <OnPlatform x:TypeArguments="Color">
            <OnPlatform.Platforms>
                <On Platform="Android"
                    Value="{AppThemeBinding Light=Black,
Dark=White}" />
            </OnPlatform.Platforms>
        </OnPlatform>
    </Label.TextColor>
    <Label.Margin>
        <OnPlatform x:TypeArguments="Thickness">
            <OnPlatform.Platforms>
                <On Platform="Android"
                    Value="20, 0, 0, 0" />
            </OnPlatform.Platforms>
        </OnPlatform>
    </Label.Margin>
    <Label.FontFamily>
        <OnPlatform x:TypeArguments="x:String">
            <OnPlatform.Platforms>
                <On Platform="Android"
                    Value="sans-serif-medium" />
            </OnPlatform.Platforms>
        </OnPlatform>
    </Label.FontFamily>
</Label>
</Grid>
</DataTemplate>

```

This template can be used for as a basis for making alterations to the existing flyout layout, and also shows the visual states that are implemented for flyout items.

In addition, the [Grid](#), [Image](#), and [Label](#) elements all have `x:Name` values and so can be targeted with the Visual State Manager. For more information, see [Set state on multiple elements](#).

❗ Note

The same template can also be used for [MenuItem](#) objects.

Replace flyout content

Flyout items, which represent the flyout content, can optionally be replaced with your own content by setting the `Shell.FlyoutContent` bindable property to an `object`:

XAML

```
<Shell ...
  x:Name="shell">
  ...
  <Shell.FlyoutContent>
    <CollectionView BindingContext="{x:Reference shell}"
      IsGrouped="True"
      ItemsSource="{Binding FlyoutItems}">
      <CollectionView.ItemTemplate>
        <DataTemplate>
          <Label Text="{Binding Title}"
            TextColor="White"
            FontSize="18" />
        </DataTemplate>
      </CollectionView.ItemTemplate>
    </CollectionView>
  </Shell.FlyoutContent>
</Shell>
```

In this example, the flyout content is replaced with a [CollectionView](#) that displays the title of each item in the `FlyoutItems` collection.

ⓘ Note

The `FlyoutItems` property, in the [Shell](#) class, is a read-only collection of flyout items.

Alternatively, flyout content can be defined by setting the `Shell.FlyoutContentTemplate` bindable property to a [DataTemplate](#):

XAML

```
<Shell ...
  x:Name="shell">
  ...
  <Shell.FlyoutContentTemplate>
    <DataTemplate>
      <CollectionView BindingContext="{x:Reference shell}"
```

```

        IsGrouped="True"
        ItemsSource="{Binding FlyoutItems}">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Label Text="{Binding Title}"
                TextColor="White"
                FontSize="18" />
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
</DataTemplate>
</Shell.FlyoutContentTemplate>
</Shell>

```

Important

A flyout header can optionally be displayed above your flyout content, and a flyout footer can optionally be displayed below your flyout content. If your flyout content is scrollable, Shell will attempt to honor the scroll behavior of your flyout header.

Menu items

Menu items can be optionally added to the flyout, and each menu item is represented by a `MenuItem` object. The position of `MenuItem` objects on the flyout is dependent upon their declaration order in the Shell visual hierarchy. Therefore, any `MenuItem` objects declared before `FlyoutItem` objects will appear before the `FlyoutItem` objects in the flyout, and any `MenuItem` objects declared after `FlyoutItem` objects will appear after the `FlyoutItem` objects in the flyout.

The `MenuItem` class has a `Clicked` event, and a `Command` property. Therefore, `MenuItem` objects enable scenarios that execute an action in response to the `MenuItem` being tapped.

`MenuItem` objects can be added to the flyout as shown in the following example:

XAML

```

<Shell ...>
    ...
    <MenuItem Text="Help"
        IconImageSource="help.png"
        Command="{Binding HelpCommand}"

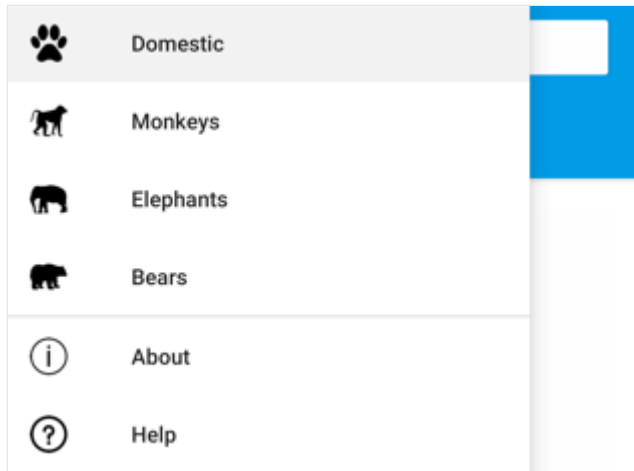
        CommandParameter="https://learn.microsoft.com/dotnet/maui/fundamentals/shell

```



```
" />  
</Shell>
```

This example adds a [MenuItem](#) object to the flyout, beneath all the flyout items:



The [MenuItem](#) object executes an [ICommand](#) named `HelpCommand`, which opens the URL specified by the `CommandParameter` property in the system web browser.

ⓘ Note

The `BindingContext` of each [MenuItem](#) is inherited from the subclassed [Shell](#) object.

Define MenuItem appearance

The appearance of each [MenuItem](#) can be customized by setting the `Shell.MenuItemTemplate` attached property to a [DataTemplate](#):

XAML

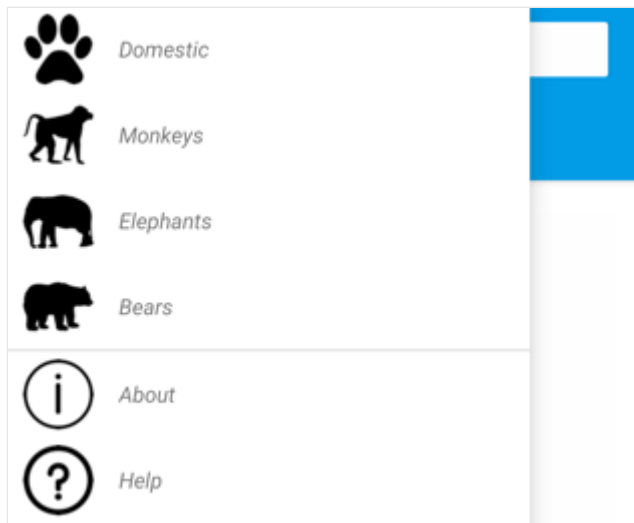
```
<Shell ...>  
  <Shell.MenuItemTemplate>  
    <DataTemplate>  
      <Grid ColumnDefinitions="0.2*,0.8*">  
        <Image Source="{Binding Icon}"  
          Margin="5"  
          HeightRequest="45" />  
        <Label Grid.Column="1"  
          Text="{Binding Text}"  
          FontAttributes="Italic"  
          VerticalTextAlignment="Center" />  
      </Grid>  
    </DataTemplate>  
  </Shell.MenuItemTemplate>
```

```

...
<MenuItem Text="Help"
          IconImageSource="help.png"
          Command="{Binding HelpCommand}"
          CommandParameter="https://learn.microsoft.com/xamarin/xamarin-
forms/app-fundamentals/shell" />
</Shell>

```

This example attaches the [DataTemplate](#) to each [MenuItem](#) object, displaying the title of the [MenuItem](#) object in italics:



Because `Shell.MenuItemTemplate` is an attached property, different templates can be attached to specific [MenuItem](#) objects.

ⓘ Note

Shell provides the `Text` and `IconImageSource` properties to the `BindingContext` of the `MenuItemTemplate`. You can also use `Title` in place of `Text` and `Icon` in place of `IconImageSource` which will let you reuse the same template for menu items and flyout items.

The default template for [FlyoutItem](#) objects can also be used for [MenuItem](#) objects. For more information, see [Default template for FlyoutItems](#).

Style FlyoutItem and MenuItem objects

Shell includes three style classes, which are automatically applied to [FlyoutItem](#) and [MenuItem](#) objects. The style class names are `FlyoutItemLabelStyle`, `FlyoutItemImageStyle`, and `FlyoutItemLayoutStyle`.

The following XAML shows an example of defining styles for these style classes:

XAML

```
<Style TargetType="Label"
      Class="FlyoutItemLabelStyle">
  <Setter Property="TextColor"
    Value="Black" />
  <Setter Property="HeightRequest"
    Value="100" />
</Style>

<Style TargetType="Image"
      Class="FlyoutItemImageStyle">
  <Setter Property="Aspect"
    Value="Fill" />
</Style>

<Style TargetType="Layout"
      Class="FlyoutItemLayoutStyle"
      ApplyToDerivedTypes="True">
  <Setter Property="BackgroundColor"
    Value="Teal" />
</Style>
```

These styles will automatically be applied to [FlyoutItem](#) and [MenuItem](#) objects, without having to set their `StyleClass` properties to the style class names.

In addition, custom style classes can be defined and applied to [FlyoutItem](#) and [MenuItem](#) objects. For more information about style classes, see [Style classes](#).

Flyout header

The flyout header is the content that optionally appears at the top of the flyout, with its appearance being defined by an `object` that can be set with the `Shell.FlyoutHeader` bindable property:

XAML

```
<Shell ...>
  <Shell.FlyoutHeader>
    <controls:FlyoutHeader />
  </Shell.FlyoutHeader>
</Shell>
```

The `FlyoutHeader` type is shown in the following example:

XAML

```

<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Xaminals.Controls.FlyoutHeader"
              HeightRequest="200">
    <Grid BackgroundColor="Black">
        <Image Aspect="AspectFill"
              Source="store.jpg"
              Opacity="0.6" />
        <Label Text="Animals"
              TextColor="White"
              FontAttributes="Bold"
              HorizontalTextAlignment="Center"
              VerticalTextAlignment="Center" />
    </Grid>
</ContentView>

```

This results in the following flyout header:



Alternatively, the flyout header appearance can be defined by setting the `Shell.FlyoutHeaderTemplate` bindable property to a [DataTemplate](#):

XAML

```

<Shell ...>
    <Shell.FlyoutHeaderTemplate>
        <DataTemplate>
            <Grid BackgroundColor="Black"
                  HeightRequest="200">
                <Image Aspect="AspectFill"
                      Source="store.jpg"
                      Opacity="0.6" />
                <Label Text="Animals"
                      TextColor="White"
                      FontAttributes="Bold"
                      HorizontalTextAlignment="Center"
                      VerticalTextAlignment="Center" />
            </Grid>
        </DataTemplate>
    </Shell.FlyoutHeaderTemplate>
</Shell>

```

By default, the flyout header will be fixed in the flyout while the content below will scroll if there are enough items. However, this behavior can be changed by setting the `Shell.FlyoutHeaderBehavior` bindable property to one of the `FlyoutHeaderBehavior` enumeration members:

- `Default` – indicates that the default behavior for the platform will be used. This is the default value of the `FlyoutHeaderBehavior` property.
- `Fixed` – indicates that the flyout header remains visible and unchanged at all times.
- `Scroll` – indicates that the flyout header scrolls out of view as the user scrolls the items.
- `CollapseOnScroll` – indicates that the flyout header collapses to a title only, as the user scrolls the items.

The following example shows how to collapse the flyout header as the user scrolls:

XAML

```
<Shell ...  
    FlyoutHeaderBehavior="CollapseOnScroll">  
    ...  
</Shell>
```

Flyout footer

The flyout footer is the content that optionally appears at the bottom of the flyout, with its appearance being defined by an `object` that can be set with the `Shell.FlyoutFooter` bindable property:

XAML

```
<Shell ...>  
    <Shell.FlyoutFooter>  
        <controls:FlyoutFooter />  
    </Shell.FlyoutFooter>  
</Shell>
```

The `FlyoutFooter` type is shown in the following example:

XAML

```
<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
    xmlns:sys="clr-namespace:System;assembly=netstandard"
```

```
x:Class="Xaminals.Controls.FlyoutFooter">
<StackLayout>
    <Label Text="Xaminals"
        TextColor="GhostWhite"
        FontAttributes="Bold"
        HorizontalOptions="Center" />
    <Label Text="{Binding Source={x:Static sys:DateTime.Now},
StringFormat='{0:MMMM dd, yyyy}}'"
        TextColor="GhostWhite"
        HorizontalOptions="Center" />
</StackLayout>
</ContentView>
```

Important

The previous XAML example defined a new XAML namespace named `sys`:
`xmlns:sys="clr-namespace:System;assembly=netstandard"`. This XAML namespace maps `sys` to the .NET `System` namespace. The mapping allows you to use the .NET types defined in that namespace, such as `DateTime`, in the XAML. For more information, see [XAML Namespaces](#).

This results in the following flyout footer:

Xaminals
April 08, 2022

```
<Shell ...>
  <Shell.FlyoutFooterTemplate>
    <DataTemplate>
      <StackLayout>
        <Label Text="Xaminals"
              TextColor="GhostWhite"
              FontAttributes="Bold"
              HorizontalOptions="Center" />
        <Label Text="{Binding Source={x:Static sys:DateTime.Now},
              StringFormat='{0:MMMM dd, yyyy}}'" />
      </StackLayout>
    </DataTemplate>
  </Shell.FlyoutFooterTemplate>
</Shell>
```

```

                TextColor="GhostWhite"
                HorizontalOptions="Center" />
            </StackLayout>
        </DataTemplate>
    </Shell.FlyoutFooterTemplate>
</Shell>

```

The flyout footer is fixed to the bottom of the flyout, and can be any height. In addition, the footer never obscures any menu items.

Flyout width and height

The width and height of the flyout can be customized by setting the `Shell.FlyoutWidth` and `Shell.FlyoutHeight` attached properties to `double` values:

XAML

```

<Shell ...
    FlyoutWidth="400"
    FlyoutHeight="200">
    ...
</Shell>

```

This enables scenarios such as expanding the flyout across the entire screen, or reducing the height of the flyout so that it doesn't obscure the tab bar.

Flyout icon

By default, Shell apps have a hamburger icon which, when pressed, opens the flyout. This icon can be changed by setting the `Shell.FlyoutIcon` bindable property, of type [ImageSource](#), to an appropriate icon:

XAML

```

<Shell ...
    FlyoutIcon="flyouticon.png">
    ...
</Shell>

```

Flyout background

The background color of the flyout can be set with the `Shell.FlyoutBackgroundColor` bindable property:

XAML

```
<Shell ...  
    FlyoutBackgroundColor="AliceBlue">  
    ...  
</Shell>
```

❗ Note

The `Shell.FlyoutBackgroundColor` can also be set from a Cascading Style Sheet (CSS). For more information, see [.NET MAUI Shell specific properties](#).

Alternatively, the background of the flyout can be specified by setting the `Shell.FlyoutBackground` bindable property to a [Brush](#):

XAML

```
<Shell ...  
    FlyoutBackground="LightGray">  
    ...  
</Shell>
```

In this example, the flyout background is painted with a light gray [SolidColorBrush](#).

The following example shows setting the flyout background to a [LinearGradientBrush](#):

XAML

```
<Shell ...>  
    <Shell.FlyoutBackground>  
        <LinearGradientBrush StartPoint="0,0"  
            EndPoint="1,1">  
            <GradientStop Color="#8A2387"  
                Offset="0.1" />  
            <GradientStop Color="#E94057"  
                Offset="0.6" />  
            <GradientStop Color="#F27121"  
                Offset="1.0" />  
        </LinearGradientBrush>  
    </Shell.FlyoutBackground>  
    ...  
</Shell>
```

For more information about brushes, see [.NET MAUI Brushes](#).

Flyout background image

The flyout can have an optional background image, which appears beneath the flyout header and behind any flyout items, menu items, and the flyout footer. The background image can be specified by setting the `FlyoutBackgroundImage` bindable property, of type [ImageSource](#), to a file, embedded resource, URI, or stream.

The aspect ratio of the background image can be configured by setting the `FlyoutBackgroundImageAspect` bindable property, of type `Aspect`, to one of the `Aspect` enumeration members:

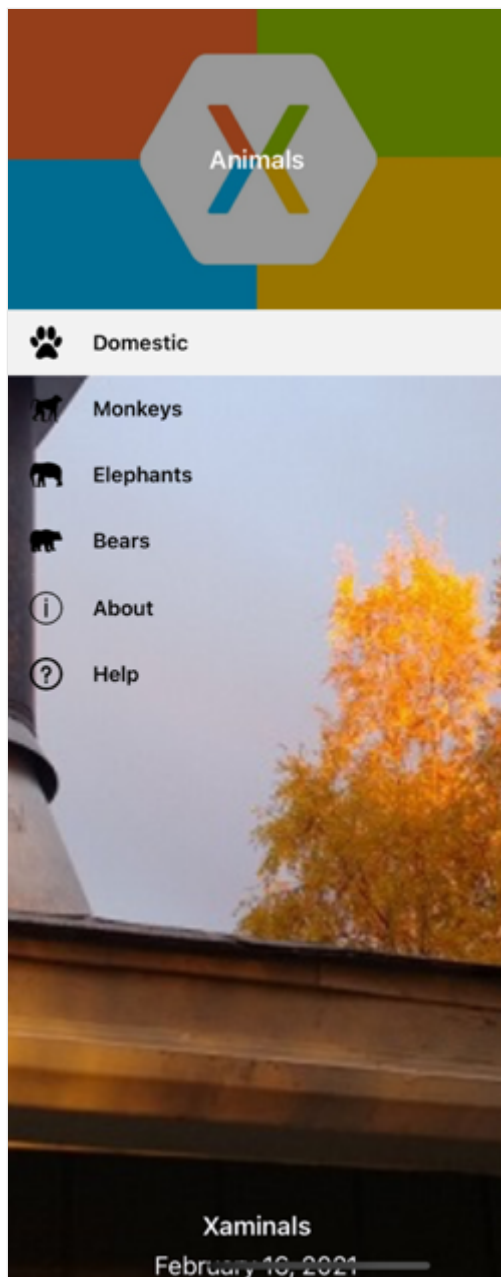
- `AspectFill` - clips the image so that it fills the display area while preserving the aspect ratio.
- `AspectFit` - letterboxes the image, if required, so that the image fits into the display area, with blank space added to the top/bottom or sides depending on whether the image is wide or tall. This is the default value of the `FlyoutBackgroundImageAspect` property.
- `Fill` - stretches the image to completely and exactly fill the display area. This may result in image distortion.

The following example shows setting these properties:

XAML

```
<Shell ...  
    FlyoutBackgroundImage="photo.jpg"  
    FlyoutBackgroundImageAspect="AspectFill">  
    ...  
</Shell>
```

This results in a background image appearing in the flyout, below the flyout header:



Flyout backdrop

The backdrop of the flyout, which is the appearance of the flyout overlay, can be specified by setting the `Shell.FlyoutBackdrop` attached property to a [Brush](#):

XAML

```
<Shell ...  
    FlyoutBackdrop="Silver">  
    ...  
</Shell>
```

In this example, the flyout backdrop is painted with a silver [SolidColorBrush](#).

Important

The `FlyoutBackdrop` attached property can be set on any Shell element, but will only be applied when it's set on [Shell](#), [FlyoutItem](#), or [TabBar](#) objects.

The following example shows setting the flyout backdrop to a [LinearGradientBrush](#):

XAML

```
<Shell ...>
  <Shell.FlyoutBackdrop>
    <LinearGradientBrush StartPoint="0,0"
                        EndPoint="1,1">
      <GradientStop Color="#8A2387"
                    Offset="0.1" />
      <GradientStop Color="#E94057"
                    Offset="0.6" />
      <GradientStop Color="#F27121"
                    Offset="1.0" />
    </LinearGradientBrush>
  </Shell.FlyoutBackdrop>
  ...
</Shell>
```

For more information about brushes, see [.NET MAUI Brushes](#).

Flyout behavior

The flyout can be accessed through the hamburger icon or by swiping from the side of the screen. However, this behavior can be changed by setting the `Shell.FlyoutBehavior` attached property to one of the `FlyoutBehavior` enumeration members:

- `Disabled` – indicates that the flyout can't be opened by the user.
- `Flyout` – indicates that the flyout can be opened and closed by the user. This is the default value for the `FlyoutBehavior` property.
- `Locked` – indicates that the flyout can't be closed by the user, and that it doesn't overlap content.

The following example shows how to disable the flyout:

XAML

```
<Shell ...
  FlyoutBehavior="Disabled">
  ...
</Shell>
```

ⓘ Note

The `FlyoutBehavior` attached property can be set on [Shell](#), [FlyoutItem](#), [ShellContent](#), and page objects, to override the default flyout behavior.

Flyout vertical scroll

By default, a flyout can be scrolled vertically when the flyout items don't fit in the flyout. This behavior can be changed by setting the `Shell.FlyoutVerticalScrollMode` bindable property to one of the `ScrollMode` enumeration members:

- `Disabled` – indicates that vertical scrolling will be disabled.
- `Enabled` – indicates that vertical scrolling will be enabled.
- `Auto` – indicates that vertical scrolling will be enabled if the flyout items don't fit in the flyout. This is the default value of the `FlyoutVerticalScrollMode` property.

The following example shows how to disable vertical scrolling:

XAML

```
<Shell ...  
    FlyoutVerticalScrollMode="Disabled">  
    ...  
</Shell>
```

FlyoutItem selection

When a Shell app that uses a flyout is first run, the `Shell.CurrentItem` property will be set to the first `FlyoutItem` object in the subclassed `Shell` object. However, the property can be set to another `FlyoutItem`, as shown in the following example:

XAML

```
<Shell ...  
    CurrentItem="{x:Reference aboutItem}">  
    <FlyoutItem FlyoutDisplayOptions="AsMultipleItems">  
        ...  
    </FlyoutItem>  
    <ShellContent x:Name="aboutItem"  
        Title="About"  
        Icon="info.png"/>
```

```
ContentTemplate="{DataTemplate views:AboutPage}" />
</Shell>
```

This example sets the `CurrentItem` property to the `ShellContent` object named `aboutItem`, which results in it being selected and displayed. In this example, an implicit conversion is used to wrap the `ShellContent` object in a `Tab` object, which is wrapped in a `FlyoutItem` object.

The equivalent C# code, given a `ShellContent` object named `aboutItem`, is:

```
C#

CurrentItem = aboutItem;
```

In this example, the `CurrentItem` property is set in the subclassed `Shell` class. Alternatively, the `CurrentItem` property can be set in any class through the `Shell.Current` static property:

```
C#

Shell.Current.CurrentItem = aboutItem;
```

⚠ Note

An app may enter a state where selecting a flyout item is not a valid operation. In such cases, the `FlyoutItem` can be disabled by setting its `IsEnabled` property to `false`. This will prevent users from being able to select the flyout item.

FlyoutItem visibility

Flyout items are visible in the flyout by default. However, an item can be hidden in the flyout with the `FlyoutItemIsVisible` property, and removed from the flyout with the `IsVisible` property:

- `FlyoutItemIsVisible`, of type `bool`, indicates if the item is hidden in the flyout, but is still reachable with the `GoToAsync` navigation method. The default value of this property is `true`.
- `IsVisible`, of type `bool`, indicates if the item should be removed from the visual tree and therefore not appear in the flyout. Its default value is `true`.

The following example shows hiding an item in the flyout:

XAML

```
<Shell ...>
  <FlyoutItem ...
    FlyoutItemIsVisible="False">
    ...
  </FlyoutItem>
</Shell>
```

ⓘ Note

There's also a `Shell.FlyoutItemIsVisible` attached property, which can be set on [FlyoutItem](#), [MenuItem](#), [Tab](#), and [ShellContent](#) objects.

Open and close the flyout programmatically

The flyout can be programmatically opened and closed by setting the `Shell.FlyoutIsPresented` bindable property to a `boolean` value that indicates whether the flyout is currently open:

XAML

```
<Shell ...
  FlyoutIsPresented="{Binding IsFlyoutOpen}">
</Shell>
```

Alternatively, this can be performed in code:

C#

```
Shell.Current.FlyoutIsPresented = false;
```

.NET MAUI Shell tabs

Article • 08/30/2024

 [Browse the sample](#)

The navigation experience provided by .NET Multi-platform App UI (.NET MAUI) Shell is based on flyouts and tabs. The top level of navigation in a Shell app is either a flyout or a bottom tab bar, depending on the navigation requirements of the app. When the navigation experience for an app begins with bottom tabs, the child of the subclassed [Shell](#) object should be a [TabBar](#) object, which represents the bottom tab bar.

A [TabBar](#) object can contain one or more [Tab](#) objects, with each [Tab](#) object representing a tab on the bottom tab bar. Each [Tab](#) object can contain one or more [ShellContent](#) objects, with each [ShellContent](#) object displaying a single [ContentPage](#). When more than one [ShellContent](#) object is present in a [Tab](#) object, the [ContentPage](#) objects are navigable by top tabs. Within a tab, you can navigate to other [ContentPage](#) objects that are known as detail pages.

Important

The [TabBar](#) type disables the flyout.

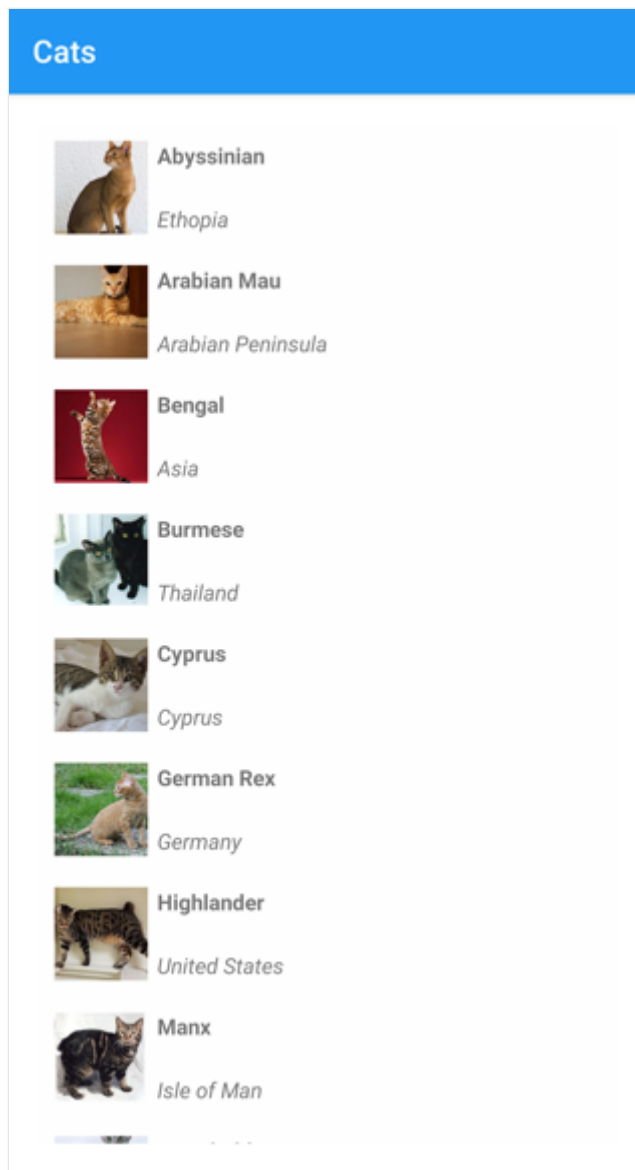
Single page

A single page Shell app can be created by adding a [Tab](#) object to a [TabBar](#) object. Within the [Tab](#) object, a [ShellContent](#) object should be set to a [ContentPage](#) object:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
  <TabBar>
    <Tab>
      <ShellContent ContentTemplate="{DataTemplate views:CatsPage}" />
    </Tab>
  </TabBar>
</Shell>
```

This example results in the following single page app:



Shell has implicit-conversion operators that enable the Shell visual hierarchy to be simplified without introducing more views into the visual tree. This simplification is possible because a subclassed [Shell](#) object can only ever contain [FlyoutItem](#) objects or a [TabBar](#) object, which can only ever contain [Tab](#) objects, which can only ever contain [ShellContent](#) objects. These implicit-conversion operators can be used to remove the [Tab](#) objects from the previous example:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
    <Tab>
        <ShellContent ContentTemplate="{DataTemplate views:CatsPage}" />
    </Tab>
</Shell>
```


This implicit conversion automatically wraps the [ShellContent](#) object in a [Tab](#) object, which is wrapped in a [TabBar](#) object.

Important

In a Shell app, pages are created on demand in response to navigation. This is accomplished by using the [DataTemplate](#) markup extension to set the `ContentTemplate` property of each [ShellContent](#) object to a [ContentPage](#) object.

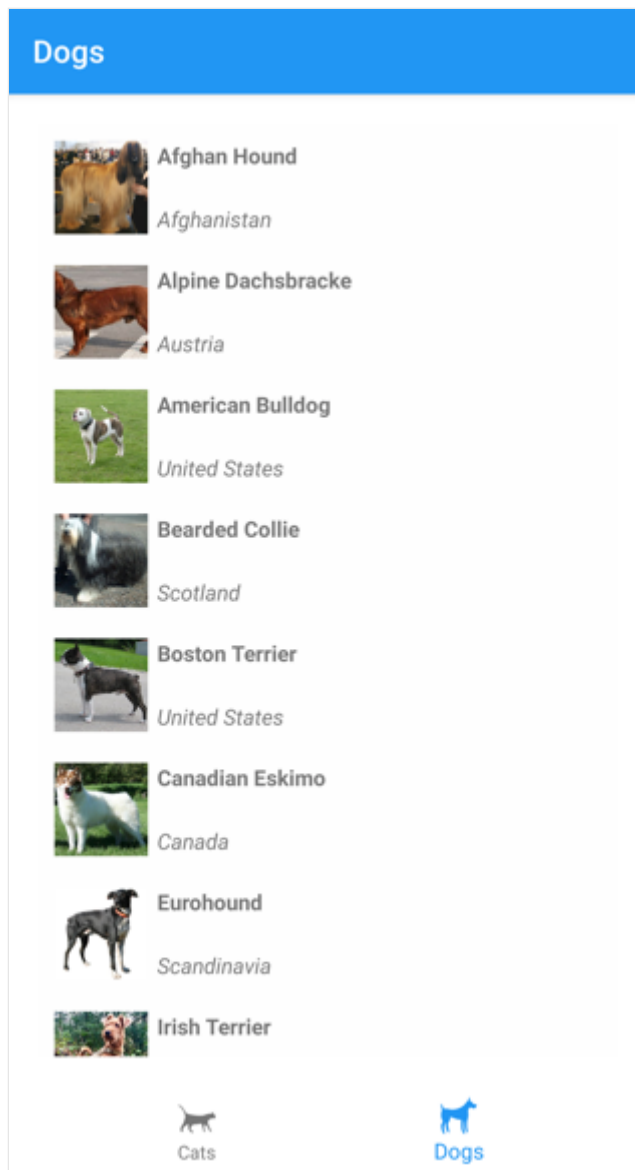
Bottom tabs

If there are multiple [Tab](#) objects in a single [TabBar](#) object, [Tab](#) objects are rendered as bottom tabs:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
    <TabBar>
        <Tab Title="Cats"
            Icon="cat.png">
            <ShellContent ContentTemplate="{DataTemplate views:CatsPage}" />
        </Tab>
        <Tab Title="Dogs"
            Icon="dog.png">
            <ShellContent ContentTemplate="{DataTemplate views:DogsPage}" />
        </Tab>
    </TabBar>
</Shell>
```

The `Title` property, of type `string`, defines the tab title. The `Icon` property, of type [ImageSource](#), defines the tab icon:



When there are more than five tabs on a [TabBar](#), a **More** tab appears, which can be used to access the other tabs:



In addition, Shell's implicit conversion operators can be used to remove the [ShellContent](#) and [Tab](#) objects from the previous example:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
  <TabBar>
    <ShellContent Title="Cats"
                  Icon="cat.png"
                  ContentTemplate="{DataTemplate views:CatsPage}" />
    <ShellContent Title="Dogs"
                  Icon="dog.png"
```

```
ContentTemplate="{DataTemplate views:DogsPage}" />
</TabBar>
</Shell>
```

This implicit conversion automatically wraps each [ShellContent](#) object in a [Tab](#) object.

❗ Important

In a Shell app, pages are created on demand in response to navigation. This is accomplished by using the [DataTemplate](#) markup extension to set the `ContentTemplate` property of each [ShellContent](#) object to a [ContentPage](#) object.

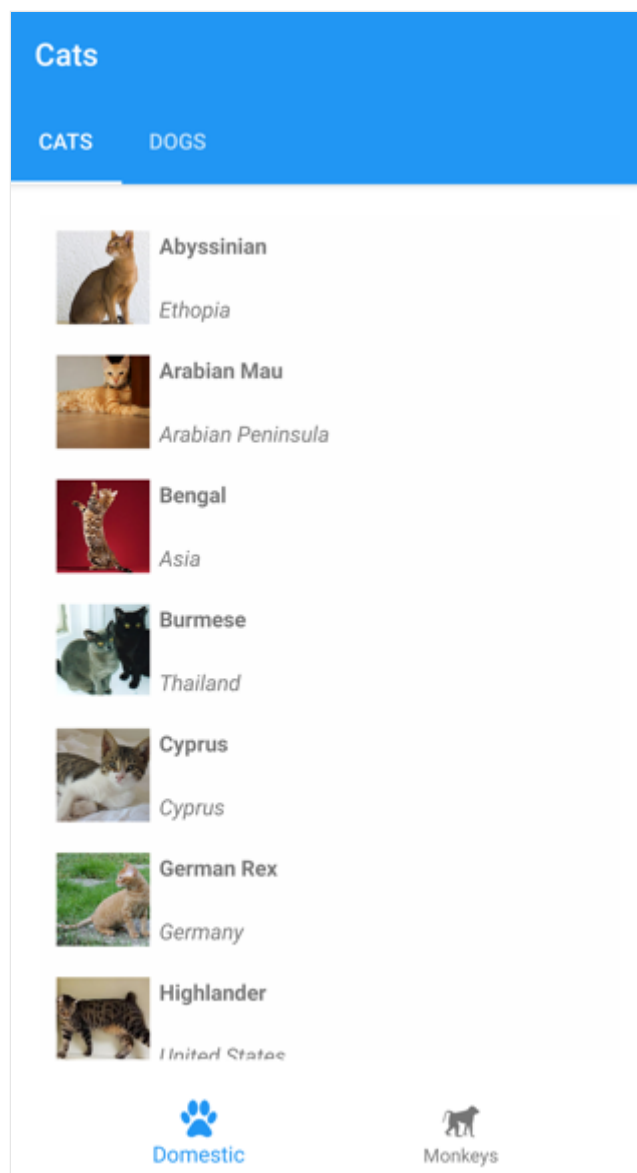
Bottom and top tabs

When more than one [ShellContent](#) object is present in a [Tab](#) object, a top tab bar is added to the bottom tab, through which the [ContentPage](#) objects are navigable:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:views="clr-namespace:Xaminals.Views"
  x:Class="Xaminals.AppShell">
  <TabBar>
    <Tab Title="Domestic"
      Icon="paw.png">
      <ShellContent Title="Cats"
        ContentTemplate="{DataTemplate views:CatsPage}" />
      <ShellContent Title="Dogs"
        ContentTemplate="{DataTemplate views:DogsPage}" />
    </Tab>
    <Tab Title="Monkeys"
      Icon="monkey.png">
      <ShellContent ContentTemplate="{DataTemplate views:MonkeysPage}"
    />
  </Tab>
</TabBar>
</Shell>
```

This code results in the layout shown in the following screenshot:



In addition, Shell's implicit conversion operators can be used to remove the second [Tab](#) object from the previous example:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:views="clr-namespace:Xaminals.Views"
  x:Class="Xaminals.AppShell">
  <TabBar>
    <Tab Title="Domestic"
      Icon="paw.png">
      <ShellContent Title="Cats"
        Icon="cat.png"
        ContentTemplate="{DataTemplate views:CatsPage}" />
      <ShellContent Title="Dogs"
        Icon="dog.png"
        ContentTemplate="{DataTemplate views:DogsPage}" />
    </Tab>
    <ShellContent Title="Monkeys"
      Icon="monkey.png"
      ContentTemplate="{DataTemplate views:MonkeysPage}" />
  </TabBar>
</Shell>
```

```
</TabBar>  
</Shell>
```

This implicit conversion automatically wraps the third [ShellContent](#) object in a [Tab](#) object.

Tab appearance

The [Shell](#) class defines the following attached properties that control the appearance of tabs:

- `TabBarBackgroundColor`, of type [Color](#), that defines the background color for the tab bar. If the property is unset, the `BackgroundColor` property value is used.
- `TabBarDisabledColor`, of type [Color](#), that defines the disabled color for the tab bar. If the property is unset, the `DisabledColor` property value is used.
- `TabBarForegroundColor`, of type [Color](#), that defines the foreground color for the tab bar. If the property is unset, the `ForegroundColor` property value is used.
- `TabBarTitleColor`, of type [Color](#), that defines the title color for the tab bar. If the property is unset, the `TitleColor` property value is used.
- `TabBarUnselectedColor`, of type [Color](#), that defines the unselected color for the tab bar. If the property is unset, the `UnselectedColor` property value is used.

All of these properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings, and styled.

The three properties that most influence the color of a tab are `TabBarForegroundColor`, `TabBarTitleColor`, and `TabBarUnselectedColor`:

- If only the `TabBarTitleColor` property is set then its value will be used to color the title and icon of the selected tab. If `TabBarTitleColor` isn't set then the title color will match the value of the `TabBarForegroundColor` property.
- If the `TabBarForegroundColor` property is set and the `TabBarUnselectedColor` property isn't set then the value of the `TabBarForegroundColor` property will be used to color the title and icon of the selected tab.
- If only the `TabBarUnselectedColor` property is set then its value will be used to color the title and icon of the unselected tab.

For example:

- When the `TabBarTitleColor` property is set to `Green` the title and icon for the selected tab is green, and unselected tabs match system colors.

- When the `TabBarForegroundColor` property is set to `Blue` the title and icon for the selected tab is blue, and unselected tabs match system colors.
- When the `TabBarTitleColor` property is set to `Green` and the `TabBarForegroundColor` property is set to `Blue` the title is green and the icon is blue for the selected tab, and unselected tabs match system colors.
- When the `TabBarTitleColor` property is set to `Green` and the `Shell.ForegroundColor` property is set to `Blue` the title is green and the icon is blue for the selected tab, and unselected tabs match system colors. This occurs because the `Shell.ForegroundColor` property value propagates to the `TabBarForegroundColor` property.
- When the `TabBarTitleColor` property is set to `Green`, the `TabBarForegroundColor` property is set to `Blue`, and the `TabBarUnselectedColor` property is set to `Red`, the title is green and the icon is blue for the selected tab, and unselected tab titles and icons are red.

The following example shows a XAML style that sets different tab bar color properties:

XAML

```
<Style TargetType="TabBar">
    <Setter Property="Shell.TabBarBackgroundColor"
        Value="CornflowerBlue" />
    <Setter Property="Shell.TabBarTitleColor"
        Value="Black" />
    <Setter Property="Shell.TabBarUnselectedColor"
        Value="AntiqueWhite" />
</Style>
```

In addition, tabs can also be styled using Cascading Style Sheets (CSS). For more information, see [.NET MAUI Shell specific properties](#).

Tab selection

When a Shell app that uses a tab bar is first run, the `Shell.CurrentItem` property is set to the first `Tab` object in the subclassed `Shell` object. However, the property can be set to another `Tab`, as shown in the following example:

XAML

```
<Shell ...
    CurrentItem="{x:Reference dogsItem}">
    <TabBar>
        <ShellContent Title="Cats"
            Icon="cat.png"
```

```

        ContentTemplate="{DataTemplate views:CatsPage}" />
    <ShellContent x:Name="dogsItem"
        Title="Dogs"
        Icon="dog.png"
        ContentTemplate="{DataTemplate views:DogsPage}" />
</TabBar>
</Shell>

```

This example sets the `CurrentItem` property to the `ShellContent` object named `dogsItem`, which results in it being selected and displayed. In this example, an implicit conversion is used to wrap each `ShellContent` object in a `Tab` object.

The equivalent C# code, given a `ShellContent` object named `dogsItem`, is:

```

C#

CurrentItem = dogsItem;

```

In this example, the `CurrentItem` property is set in the subclassed `Shell` class. Alternatively, the `CurrentItem` property can be set in any class through the `Shell.Current` static property:

```

C#

Shell.Current.CurrentItem = dogsItem;

```

TabBar and Tab visibility

The tab bar and tabs are visible in Shell apps by default. However, the tab bar can be hidden by setting the `Shell.TabBarIsVisible` attached property to `false`.

While this property can be set on a subclassed `Shell` object, it's typically set on any `ShellContent` or `ContentPage` objects that want to make the tab bar invisible:

```

XAML

<TabBar>
    <Tab Title="Domestic"
        Icon="paw.png">
        <ShellContent Title="Cats"
            ContentTemplate="{DataTemplate views:CatsPage}" />
        <ShellContent Shell.TabBarIsVisible="false"
            Title="Dogs"
            ContentTemplate="{DataTemplate views:DogsPage}" />
    </Tab>
</TabBar>

```

```
<Tab Title="Monkeys"
      Icon="monkey.png">
  <ShellContent ContentTemplate="{DataTemplate views:MonkeysPage}" />
</Tab>
</TabBar>
```

In this example, the tab bar is hidden when the upper **Dogs** tab is selected.

In addition, **Tab** objects can be hidden by setting the `IsVisible` bindable property to `false`:

XAML

```
<TabBar>
  <ShellContent Title="Cats"
    Icon="cat.png"
    ContentTemplate="{DataTemplate views:CatsPage}" />
  <ShellContent Title="Dogs"
    Icon="dog.png"
    ContentTemplate="{DataTemplate views:DogsPage}"
    IsVisible="False" />
  <ShellContent Title="Monkeys"
    Icon="monkey.png"
    ContentTemplate="{DataTemplate views:MonkeysPage}" />
</TabBar>
```

In this example, the second tab is hidden.

.NET MAUI Shell pages

Article • 08/30/2024

 [Browse the sample](#)

A [ShellContent](#) object represents the [ContentPage](#) object for each [FlyoutItem](#) or [Tab](#). When more than one [ShellContent](#) object is present in a [Tab](#) object, the [ContentPage](#) objects are navigable by top tabs. Within a page, you can navigate to additional [ContentPage](#) objects known as detail pages.

In addition, the [Shell](#) class defines attached properties that can be used to configure the appearance of pages in .NET Multi-platform App UI (.NET MAUI) Shell apps. This configuration includes setting page colors, setting the page presentation mode, disabling the navigation bar, disabling the tab bar, and displaying views in the navigation bar.

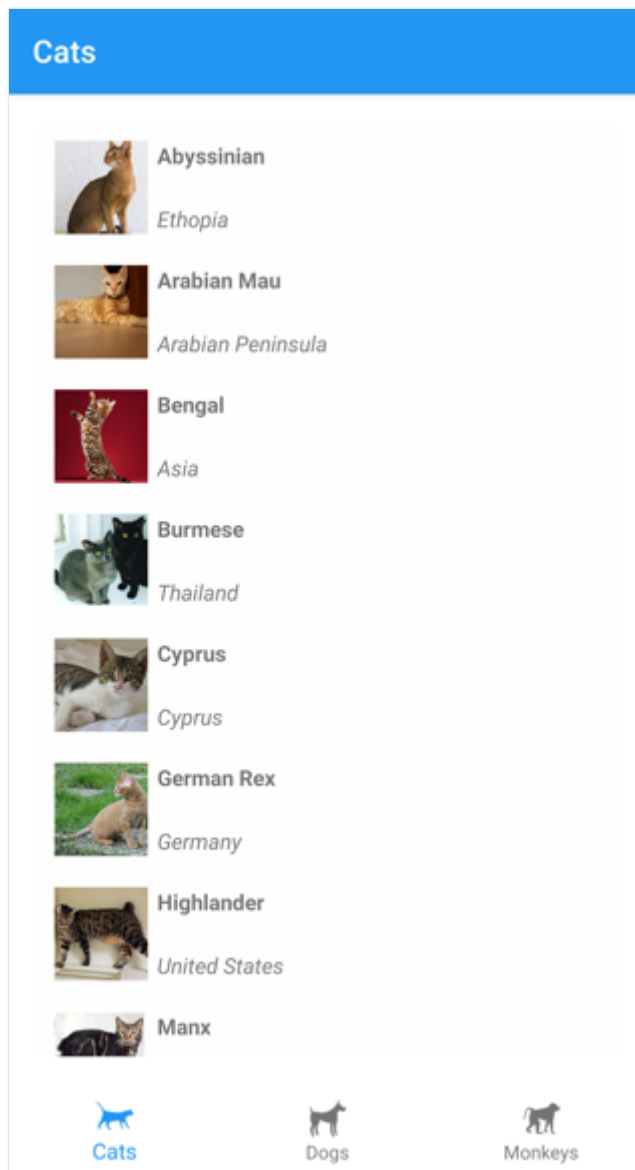
Display pages

In .NET MAUI Shell apps, pages are typically created on demand in response to navigation. This creation is accomplished by using the [DataTemplate](#) markup extension to set the `ContentTemplate` property of each [ShellContent](#) object to a [ContentPage](#) object:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
    <TabBar>
        <ShellContent Title="Cats"
                      Icon="cat.png"
                      ContentTemplate="{DataTemplate views:CatsPage}" />
        <ShellContent Title="Dogs"
                      Icon="dog.png"
                      ContentTemplate="{DataTemplate views:DogsPage}" />
        <ShellContent Title="Monkeys"
                      Icon="monkey.png"
                      ContentTemplate="{DataTemplate views:MonkeysPage}" />
    </TabBar>
</Shell>
```

In this example, Shell's implicit conversion operators are used to remove the [Tab](#) objects from the visual hierarchy. However, each [ShellContent](#) object is rendered in a tab:



⚠ Note

The `BindingContext` of each [ShellContent](#) object is inherited from the parent [Tab](#) object.

Within each [ContentPage](#) object, additional [ContentPage](#) objects can be navigated to. For more information about navigation, see [.NET MAUI Shell navigation](#).

Load pages at app startup

In a Shell app, each [ContentPage](#) object is typically created on demand, in response to navigation. However, it's also possible to create [ContentPage](#) objects at app startup.

⚠ Warning

ContentPage objects that are created at app startup can lead to a poor startup experience.

ContentPage objects can be created at app startup by setting the `ShellContent.Content` properties to **ContentPage** objects:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:views="clr-namespace:Xaminals.Views"
        x:Class="Xaminals.AppShell">
  <TabBar>
    <ShellContent Title="Cats"
                  Icon="cat.png">
      <views:CatsPage />
    </ShellContent>
    <ShellContent Title="Dogs"
                  Icon="dog.png">
      <views:DogsPage />
    </ShellContent>
    <ShellContent Title="Monkeys"
                  Icon="monkey.png">
      <views:MonkeysPage />
    </ShellContent>
  </TabBar>
</Shell>
```

In this example, `CatsPage`, `DogsPage`, and `MonkeysPage` are all created at app startup, rather than on demand in response to navigation.

ⓘ Note

The `Content` property is the content property of the **ShellContent** class, and therefore does not need to be explicitly set.

Set page colors

The **Shell** class defines the following attached properties that can be used to set page colors in a Shell app:

- `BackgroundColor`, of type **Color**, that defines the background color in the Shell chrome. The color won't fill in behind the Shell content.

- `DisabledColor`, of type `Color`, that defines the color to shade text and icons that are disabled.
- `ForegroundColor`, of type `Color`, that defines the color to shade text and icons.
- `TitleColor`, of type `Color`, that defines the color used for the title of the current page.
- `UnselectedColor`, of type `Color`, that defines the color used for unselected text and icons in the Shell chrome.

All of these properties are backed by `BindableProperty` objects, which mean that the properties can be targets of data bindings, and styled using XAML styles. In addition, the properties can be set using Cascading Style Sheets (CSS). For more information, see [.NET MAUI Shell specific properties](#).

ⓘ Note

There are also properties that enable tab colors to be defined. For more information, see [Tab appearance](#).

The following XAML shows setting the color properties in a subclassed `Shell` class:

XAML

```
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      x:Class="Xaminals.AppShell"
      BackgroundColor="#455A64"
      ForegroundColor="White"
      TitleColor="White"
      DisabledColor="#B4FFFFFF"
      UnselectedColor="#95FFFFFF">

</Shell>
```

In this example, the color values are applied to all pages in the Shell app, unless they're overridden at the page level.

Because the color properties are attached properties, they can also be set on individual pages, to set the colors on that page:

XAML

```
<ContentPage ...
    Shell.BackgroundColor="Gray"
    Shell.ForegroundColor="White"
    Shell.TitleColor="Blue"
```

```
Shell.DisabledColor="#95FFFFFF"
Shell.UnselectedColor="#B4FFFFFF">

</ContentPage>
```

Alternatively, the color properties can be set with a XAML style:

XAML

```
<Style x:Key="DomesticShell"
    TargetType="Element" >
    <Setter Property="Shell.BackgroundColor"
        Value="#039BE6" />
    <Setter Property="Shell.ForegroundColor"
        Value="White" />
    <Setter Property="Shell.TitleColor"
        Value="White" />
    <Setter Property="Shell.DisabledColor"
        Value="#B4FFFFFF" />
    <Setter Property="Shell.UnselectedColor"
        Value="#95FFFFFF" />
</Style>
```

For more information about XAML styles, see [Style apps using XAML](#).

Set page presentation mode

By default, a small navigation animation occurs when a page is navigated to with the [GoToAsync](#) method. However, this behavior can be changed by setting the `Shell.PresentationMode` attached property on a [ContentPage](#) to one of the `PresentationMode` enumeration members:

- `NotAnimated` indicates that the page will be displayed without a navigation animation.
- `Animated` indicates that the page will be displayed with a navigation animation. This is the default value of the `Shell.PresentationMode` attached property.
- `Modal` indicates that the page will be displayed as a modal page.
- `ModalAnimated` indicates that the page will be displayed as a modal page, with a navigation animation.
- `ModalNotAnimated` indicates that the page will be displayed as a modal page, without a navigation animation.

 **Important**

The `PresentationMode` type is a flags enumeration. This means that a combination of enumeration members can be applied in code. However, for ease of use in XAML, the `ModalAnimated` member is a combination of the `Animated` and `Modal` members, and the `ModalNotAnimated` member is a combination of the `NotAnimated` and `Modal` members. For more information about flag enumerations, see [Enumeration types as bit flags](#).

The following XAML example sets the `Shell.PresentationMode` attached property on a `ContentPage`:

XAML

```
<ContentPage ...  
    Shell.PresentationMode="Modal">  
    ...  
</ContentPage>
```

In this example, the `ContentPage` is set to be displayed as a modal page, when the page is navigated to with the `GoToAsync` method.

Enable navigation bar shadow

The `Shell.NavBarHasShadow` attached property, of type `bool`, controls whether the navigation bar has a shadow. By default the value of the property is `true` on Android, and `false` on other platforms.

While this property can be set on a subclassed `Shell` object, it can also be set on any pages that want to enable the navigation bar shadow. For example, the following XAML shows enabling the navigation bar shadow from a `ContentPage`:

XAML

```
<ContentPage ...  
    Shell.NavBarHasShadow="true">  
    ...  
</ContentPage>
```

This results in the navigation bar shadow being enabled.

Disable the navigation bar

The `Shell.NavBarIsVisible` attached property, of type `bool`, controls if the navigation bar is visible when a page is presented. By default the value of the property is `true`.

While this property can be set on a subclassed `Shell` object, it's typically set on any pages that want to make the navigation bar invisible. For example, the following XAML shows disabling the navigation bar from a `ContentPage`:

XAML

```
<ContentPage ...  
    Shell.NavBarIsVisible="false">  
    ...  
</ContentPage>
```

Display views in the navigation bar

The `Shell.TitleView` attached property, of type `View`, enables any `View` to be displayed in the navigation bar.

While this property can be set on a subclassed `Shell` object, it can also be set on any pages that want to display a view in the navigation bar. For example, the following XAML shows displaying an `Image` in the navigation bar of a `ContentPage`:

XAML

```
<ContentPage ...>  
    <Shell.TitleView>  
        <Image Source="logo.png"  
            HorizontalOptions="Center"  
            VerticalOptions="Center" />  
    </Shell.TitleView>  
    ...  
</ContentPage>
```

Important

If the navigation bar has been made invisible, with the `NavBarIsVisible` attached property, the title view will not be displayed.

Many views won't appear in the navigation bar unless the size of the view is specified with the `WidthRequest` and `HeightRequest` properties or the location of the view is specified with the `HorizontalOptions` and `VerticalOptions` properties.

The `TitleView` attached property can be set to display a layout class that contains multiple views. Similarly, because the `ContentView` class ultimately derives from the `View` class, the `TitleView` attached property can be set to display a `ContentView` that contains a single view.

Page visibility

Shell respects page visibility, which is set with the `IsVisible` property. When a page's `IsVisible` property is set to `false`, it's not visible in the Shell app and it's not possible to navigate to it.

.NET MAUI Shell navigation

Article • 10/08/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) Shell includes a URI-based navigation experience that uses routes to navigate to any page in the app, without having to follow a set navigation hierarchy. In addition, it also provides the ability to navigate backwards without having to visit all of the pages on the navigation stack.

The [Shell](#) class defines the following navigation-related properties:

- [BackButtonBehavior](#), of type [BackButtonBehavior](#), an attached property that defines the behavior of the back button.
- `CurrentItem`, of type `ShellItem`, the currently selected item.
- `CurrentPage`, of type [Page](#), the currently presented page.
- `CurrentState`, of type `ShellNavigationState`, the current navigation state of the [Shell](#).
- `Current`, of type [Shell](#), which provides access to the current Shell.

The [BackButtonBehavior](#), `CurrentItem`, and `CurrentState` properties are backed by [BindableProperty](#) objects, which means that these properties can be targets of data bindings.

Navigation is performed by invoking the [GoToAsync](#) method, from the [Shell](#) class. When navigation is about to be performed, the `Navigating` event is fired, and the `Navigated` event is fired when navigation completes.

Note

Navigation can still be performed between pages in a Shell app by using the `Navigation` property. For more information, see [Perform modeless navigation](#).

Routes

Navigation is performed in a Shell app by specifying a URI to navigate to. Navigation URIs can have three components:

- A *route*, which defines the path to content that exists as part of the Shell visual hierarchy.

- A *page*. Pages that don't exist in the Shell visual hierarchy can be pushed onto the navigation stack from anywhere within a Shell app. For example, a details page won't be defined in the Shell visual hierarchy, but can be pushed onto the navigation stack as required.
- One or more *query parameters*. Query parameters are parameters that can be passed to the destination page while navigating.

When a navigation URI includes all three components, the structure is: `//route/page?queryParameters`

Register routes

Routes can be defined on [FlyoutItem](#), [TabBar](#), [Tab](#), and [ShellContent](#) objects, through their `Route` properties:

XAML

```
<Shell ...>
  <FlyoutItem ...
    Route="animals">
      <Tab ...
        Route="domestic">
          <ShellContent ...
            Route="cats" />
          <ShellContent ...
            Route="dogs" />
        </Tab>
      <ShellContent ...
        Route="monkeys" />
      <ShellContent ...
        Route="elephants" />
      <ShellContent ...
        Route="bears" />
    </FlyoutItem>
  <ShellContent ...
    Route="about" />
  ...
</Shell>
```

ⓘ Note

All items in the Shell hierarchy have a route associated with them. If you don't set a route, one is generated at runtime. However, generated routes are not guaranteed to be consistent across different app sessions.

The above example creates the following route hierarchy, which can be used in programmatic navigation:

```
animals
  domestic
    cats
    dogs
  monkeys
  elephants
  bears
  about
```

To navigate to the [ShellContent](#) object for the `dogs` route, the absolute route URI is `//animals/domestic/dogs`. Similarly, to navigate to the [ShellContent](#) object for the `about` route, the absolute route URI is `//about`.

Warning

An `ArgumentException` will be thrown on app startup if a duplicate route is detected. This exception will also be thrown if two or more routes at the same level in the hierarchy share a route name.

Register detail page routes

In the [Shell](#) subclass constructor, or any other location that runs before a route is invoked, additional routes can be explicitly registered for any detail pages that aren't represented in the Shell visual hierarchy. This is accomplished with the `Routing.RegisterRoute` method:

C#

```
Routing.RegisterRoute("monkeydetails", typeof(MonkeyDetailPage));
Routing.RegisterRoute("beardetails", typeof(BearDetailPage));
Routing.RegisterRoute("catdetails", typeof(CatDetailPage));
Routing.RegisterRoute("dogdetails", typeof(DogDetailPage));
Routing.RegisterRoute("elephantdetails", typeof(ElephantDetailPage));
```

This example registers detail pages, that aren't defined in the [Shell](#) subclass, as routes. These detail pages can then be navigated to using URI-based navigation, from anywhere within the app. The routes for such pages are known as *global routes*.

Warning

An `ArgumentException` will be thrown if the `Routing.RegisterRoute` method attempts to register the same route to two or more different types.

Alternatively, pages can be registered at different route hierarchies if required:

C#

```
Routing.RegisterRoute("monkeys/details", typeof(MonkeyDetailPage));
Routing.RegisterRoute("bears/details", typeof(BearDetailPage));
Routing.RegisterRoute("cats/details", typeof(CatDetailPage));
Routing.RegisterRoute("dogs/details", typeof(DogDetailPage));
Routing.RegisterRoute("elephants/details", typeof(ElephantDetailPage));
```

This example enables contextual page navigation, where navigating to the `details` route from the page for the `monkeys` route displays the `MonkeyDetailPage`. Similarly, navigating to the `details` route from the page for the `elephants` route displays the `ElephantDetailPage`. For more information, see [Contextual navigation](#).

Note

Pages whose routes have been registered with the `Routing.RegisterRoute` method can be deregistered with the `Routing.UnRegisterRoute` method, if required.

Perform navigation

To perform navigation, a reference to the `Shell` subclass must first be obtained. This reference can be obtained through the `Shell.Current` property. Navigation can then be performed by calling the `GoToAsync` method on the `Shell` object. This method navigates to a `ShellNavigationState` and returns a `Task` that will complete once the navigation animation has completed. The `ShellNavigationState` object is constructed by the `GoToAsync` method, from a `string`, or a `Uri`, and it has its `Location` property set to the `string` or `Uri` argument.

Important

When a route from the Shell visual hierarchy is navigated to, a navigation stack isn't created. However, when a page that's not in the Shell visual hierarchy is navigated to, a navigation stack is created.

The current navigation state of the [Shell](#) object can be retrieved through the `Shell.Current.CurrentState` property, which includes the URI of the displayed route in the `Location` property.

Absolute routes

Navigation can be performed by specifying a valid absolute URI as an argument to the [GoToAsync](#) method:

C#

```
await Shell.Current.GoToAsync("//animals/monkeys");
```

This example navigates to the page for the `monkeys` route, with the route being defined on a [ShellContent](#) object. The [ShellContent](#) object that represents the `monkeys` route is a child of a [FlyoutItem](#) object, whose route is `animals`.

Relative routes

Navigation can also be performed by specifying a valid relative URI as an argument to the [GoToAsync](#) method. The routing system will attempt to match the URI to a [ShellContent](#) object. Therefore, if all the routes in an app are unique, navigation can be performed by only specifying the unique route name as a relative URI.

The following relative route formats are supported:

[Expand table](#)

Format	Description
<i>route</i>	The route hierarchy will be searched for the specified route, upwards from the current position. The matching page will be pushed to the navigation stack.
<i>/route</i>	The route hierarchy will be searched from the specified route, downwards from the current position. The matching page will be pushed to the navigation stack.
<i>//route</i>	The route hierarchy will be searched for the specified route, upwards from the current position. The matching page will replace the navigation stack.

Format	Description
<code>///route</code>	The route hierarchy will be searched for the specified route, downwards from the current position. The matching page will replace the navigation stack.

The following example navigates to the page for the `monkeydetails` route:

```
C#
await Shell.Current.GoToAsync("monkeydetails");
```

In this example, the `monkeyDetails` route is searched for up the hierarchy until the matching page is found. When the page is found, it's pushed to the navigation stack.

Contextual navigation

Relative routes enable contextual navigation. For example, consider the following route hierarchy:

```
monkeys
  details
bears
  details
```

When the registered page for the `monkeys` route is displayed, navigating to the `details` route will display the registered page for the `monkeys/details` route. Similarly, when the registered page for the `bears` route is displayed, navigating to the `details` route will display the registered page for the `bears/details` route. For information on how to register the routes in this example, see [Register page routes](#).

Backwards navigation

Backwards navigation can be performed by specifying `".."` as the argument to the `GoToAsync` method:

```
C#
await Shell.Current.GoToAsync("..");
```

Backwards navigation with `".."` can also be combined with a route:

C#

```
await Shell.Current.GoToAsync("../route");
```

In this example, backwards navigation is performed, and then navigation to the specified route.

❗ Important

Navigating backwards and into a specified route is only possible if the backwards navigation places you at the current location in the route hierarchy to navigate to the specified route.

Similarly, it's possible to navigate backwards multiple times, and then navigate to a specified route:

C#

```
await Shell.Current.GoToAsync("../../route");
```

In this example, backwards navigation is performed twice, and then navigation to the specified route.

In addition, data can be passed through query properties when navigating backwards:

C#

```
await Shell.Current.GoToAsync($"../?parameterToPassBack={parameterValueToPassBack}");
```

In this example, backwards navigation is performed, and the query parameter value is passed to the query parameter on the previous page.

❗ Note

Query parameters can be appended to any backwards navigation request.

For more information about passing data when navigating, see [Pass data](#).

Invalid routes

The following route formats are invalid:

Format	Explanation
<code>//page</code> or <code>///page</code>	Global routes currently can't be the only page on the navigation stack. Therefore, absolute routing to global routes is unsupported.

Use of these route formats results in an `Exception` being thrown.

⚠ Warning

Attempting to navigate to a non-existent route results in an `ArgumentException` exception being thrown.

Debugging navigation

Some of the Shell classes are decorated with the `DebuggerDisplayAttribute`, which specifies how a class or field is displayed by the debugger. This can help to debug navigation requests by displaying data related to the navigation request. For example, the following screenshot shows the `CurrentItem` and `CurrentState` properties of the `Shell.Current` object:

▶ 🔧 <code>CurrentItem</code>	<code>Title = null, Route = "animals"</code>
▶ 🔧 <code>CurrentState</code>	<code>Location = {/animals/domestic/cats}</code>

In this example, the `CurrentItem` property, of type `FlyoutItem`, displays the title and route of the `FlyoutItem` object. Similarly, the `CurrentState` property, of type `ShellNavigationState`, displays the URI of the displayed route within the Shell app.

Navigation stack

The `Tab` class defines a `Stack` property, of type `IReadOnlyList<Page>`, which represents the current navigation stack within the `Tab`. The class also provides the following overridable navigation methods:

- `GetNavigationStack`, returns `IReadOnlyList<Page>`, the current navigation stack.
- `OnInsertPageBefore`, that's called when `INavigation.InsertPageBefore` is called.
- `OnPopAsync`, returns `Task<Page>`, and is called when `INavigation.PopAsync` is called.
- `OnPopToRootAsync`, returns `Task`, and is called when `INavigation.OnPopToRootAsync` is called.
- `OnPushAsync`, returns `Task`, and is called when `INavigation.PushAsync` is called.

- `OnRemovePage`, that's called when `INavigation.RemovePage` is called.

The following example shows how to override the `OnRemovePage` method:

```
C#

public class MyTab : Tab
{
    protected override void OnRemovePage(Page page)
    {
        base.OnRemovePage(page);

        // Custom logic
    }
}
```

In this example, `MyTab` objects should be consumed in your Shell visual hierarchy instead of `Tab` objects.

Navigation events

The `Shell` class defines the `Navigating` event, which is fired when navigation is about to be performed, either due to programmatic navigation or user interaction. The `ShellNavigatingEventArgs` object that accompanies the `Navigating` event provides the following properties:

 Expand table

Property	Type	Description
<code>Current</code>	<code>ShellNavigationState</code>	The URI of the current page.
<code>Source</code>	<code>ShellNavigationSource</code>	The type of navigation that occurred.
<code>Target</code>	<code>ShellNavigationState</code>	The URI representing where the navigation is destined.
<code>CanCancel</code>	<code>bool</code>	A value indicating if it's possible to cancel the navigation.
<code>Cancelled</code>	<code>bool</code>	A value indicating if the navigation was canceled.

In addition, the `ShellNavigatingEventArgs` class provides a `Cancel` method that can be used to cancel navigation, and a `GetDeferral` method that returns a `ShellNavigatingDeferral` token that can be used to complete navigation. For more information about navigation deferral, see [Navigation deferral](#).

The [Shell](#) class also defines the `Navigated` event, which is fired when navigation has completed. The `ShellNavigatedEventArgs` object that accompanies the `Navigated` event provides the following properties:

[Expand table](#)

Property	Type	Description
<code>Current</code>	<code>ShellNavigationState</code>	The URI of the current page.
<code>Previous</code>	<code>ShellNavigationState</code>	The URI of the previous page.
<code>Source</code>	<code>ShellNavigationSource</code>	The type of navigation that occurred.

Important

The `OnNavigating` method is called when the `Navigating` event fires. Similarly, the `OnNavigated` method is called when the `Navigated` event fires. Both methods can be overridden in your [Shell](#) subclass to intercept navigation requests.

The `ShellNavigatedEventArgs` and `ShellNavigatingEventArgs` classes both have `Source` properties, of type `ShellNavigationSource`. This enumeration provides the following values:

- `Unknown`
- `Push`
- `Pop`
- `PopToRoot`
- `Insert`
- `Remove`
- `ShellItemChanged`
- `ShellSectionChanged`
- `ShellContentChanged`

Therefore, navigation can be intercepted in an `OnNavigating` override and actions can be performed based on the navigation source. For example, the following code shows how to cancel backwards navigation if the data on the page is unsaved:

C#

```
protected override void OnNavigating(ShellNavigatingEventArgs args)
{
```

```

base.OnNavigating(args);

// Cancel any back navigation.
if (args.Source == ShellNavigationSource.Pop)
{
    args.Cancel();
}
}

```

Navigation deferral

Shell navigation can be intercepted and completed or canceled based on user choice. This can be achieved by overriding the `OnNavigating` method in your `Shell` subclass, and by calling the `GetDeferral` method on the `ShellNavigatingEventArgs` object. This method returns a `ShellNavigatingDeferral` token that has a `Complete` method, which can be used to complete the navigation request:

C#

```

public MyShell : Shell
{
    // ...
    protected override async void OnNavigating(ShellNavigatingEventArgs
args)
    {
        base.OnNavigating(args);

        ShellNavigatingDeferral token = args.GetDeferral();

        var result = await DisplayActionSheet("Navigate?", "Cancel", "Yes",
"No");
        if (result != "Yes")
        {
            args.Cancel();
        }
        token.Complete();
    }
}

```

In this example, an action sheet is displayed that invites the user to complete the navigation request, or cancel it. Navigation is canceled by invoking the `Cancel` method on the `ShellNavigatingEventArgs` object. Navigation is completed by invoking the `Complete` method on the `ShellNavigatingDeferral` token that was retrieved by the `GetDeferral` method on the `ShellNavigatingEventArgs` object.

 **Warning**

The [GoToAsync](#) method will throw a `InvalidOperationException` if a user tries to navigate while there is a pending navigation deferral.

Pass data

Primitive data can be passed as string-based query parameters when performing URI-based programmatic navigation. This is achieved by appending `?` after a route, followed by a query parameter id, `=`, and a value:

```
C#  
  
async void OnCollectionViewSelectionChanged(object sender,  
SelectionChangedEventArgs e)  
{  
    string elephantName = (e.CurrentSelection.FirstOrDefault() as  
Animal).Name;  
    await Shell.Current.GoToAsync($"elephantdetails?name={elephantName}");  
}
```

This example retrieves the currently selected elephant in the [CollectionView](#), and navigates to the `elephantdetails` route, passing `elephantName` as a query parameter.

Pass multiple use object-based navigation data

Multiple use object-based navigation data can be passed with a [GoToAsync](#) overload that specifies an `IDictionary<string, object>` argument:

```
C#  
  
async void OnCollectionViewSelectionChanged(object sender,  
SelectionChangedEventArgs e)  
{  
    Animal animal = e.CurrentSelection.FirstOrDefault() as Animal;  
    var navigationParameter = new Dictionary<string, object>  
    {  
        { "Bear", animal }  
    };  
    await Shell.Current.GoToAsync($"beardetails", navigationParameter);  
}
```

This example retrieves the currently selected bear in the [CollectionView](#), as an `Animal`. The `Animal` object is added to a `Dictionary` with the key `Bear`. Then, navigation to the

`beardetails` route is performed, with the `Dictionary` being passed as a navigation parameter.

Any data that's passed as an `IDictionary<string, object>` argument is retained in memory for the lifetime of the page, and isn't released until the page is removed from the navigation stack. This can be problematic, as shown in the following scenario:

1. `Page1` navigates to `Page2` using the `GoToAsync` method, passing in an object called `MyData`. `Page2` then receives `MyData` as a query parameter.
2. `Page2` navigates to `Page3` using the `GoToAsync` method, without passing any data.
3. `Page3` navigates backwards with the `GoToAsync` method. `Page2` then receives `MyData` again as a query parameter.

While this is desirable in many scenarios, if it isn't desired you should clear the `IDictionary<string, object>` argument with the `Clear` method after it's first been received by a page.

Pass single use object-based navigation data

Single use object-based navigation data can be passed with a `GoToAsync` overload that specifies a `ShellNavigationQueryParameters` argument. A

`ShellNavigationQueryParameters` object is intended for single use navigation data that's cleared after navigation has occurred. The following example shows navigating while passing single use data:

C#

```
async void OnCollectionViewSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    Animal animal = e.CurrentSelection.FirstOrDefault() as Animal;
    var navigationParameter = new ShellNavigationQueryParameters
    {
        { "Bear", animal }
    };
    await Shell.Current.GoToAsync($"beardetails", navigationParameter);
}
```

This example retrieves the currently selected bear in the `CollectionView`, as an `Animal` that's added to the `ShellNavigationQueryParameters` object. Then, navigation to the `beardetails` route is performed, with the `ShellNavigationQueryParameters` object being passed as a navigation parameter. After navigation has occurred the data in the `ShellNavigationQueryParameters` object is cleared.

Receive navigation data

There are two approaches to receiving navigation data:

1. The class that represents the page being navigated to, or the class for the page's `BindingContext`, can be decorated with a `QueryPropertyAttribute` for each query parameter. For more information, see [Process navigation data using query property attributes](#).
2. The class that represents the page being navigated to, or the class for the page's `BindingContext`, can implement the `IQueryAttributable` interface. For more information, see [Process navigation data using a single method](#).

Process navigation data using query property attributes

Navigation data can be received by decorating the receiving class with a `QueryPropertyAttribute` for each string-based query parameter, object-based navigation parameter, or `ShellNavigationQueryParameters` object:

C#

```
[QueryProperty(nameof(Bear), "Bear")]
public partial class BearDetailPage : ContentPage
{
    Animal bear;
    public Animal Bear
    {
        get => bear;
        set
        {
            bear = value;
            OnPropertyChanged();
        }
    }

    public BearDetailPage()
    {
        InitializeComponent();
        BindingContext = this;
    }
}
```

In this example the first argument for the `QueryPropertyAttribute` specifies the name of the property that will receive the data, with the second argument specifying the parameter id. Therefore, the `QueryPropertyAttribute` in the above example specifies that the `Bear` property will receive the data passed in the `Bear` navigation parameter in the `GoToAsync` method call.

Important

String-based query parameter values that are received via the [QueryPropertyAttribute](#) are automatically URL decoded.

Warning

Receiving navigation data using the [QueryPropertyAttribute](#) isn't trim safe and shouldn't be used with full trimming or NativeAOT. Instead, you should implement the [IQueryAttributable](#) interface on types that need to accept query parameters. For more information, see [Process navigation data using a single method](#), [Trim a .NET MAUI app](#), and [Native AOT deployment](#).

Process navigation data using a single method

Navigation data can be received by implementing the [IQueryAttributable](#) interface on the receiving class. The [IQueryAttributable](#) interface specifies that the implementing class must implement the `ApplyQueryAttributes` method. This method has a `query` argument, of type `IDictionary<string, object>`, that contains any data passed during navigation. Each key in the dictionary is a query parameter id, with its value corresponding to the object that represents the data. The advantage of using this approach is that navigation data can be processed using a single method, which can be useful when you have multiple items of navigation data that require processing as a whole.

The following example shows a view model class that implements the [IQueryAttributable](#) interface:

C#

```
public class MonkeyDetailViewModel : IQueryAttributable,
INotifyPropertyChanged
{
    public Animal Monkey { get; private set; }

    public void ApplyQueryAttributes(IDictionary<string, object> query)
    {
        Monkey = query["Monkey"] as Animal;
        OnPropertyChanged("Monkey");
    }
    ...
}
```

In this example, the `ApplyQueryAttributes` method retrieves the object that corresponds to the `Monkey` key in the `query` dictionary, which was passed as an argument to the `GoToAsync` method call.

Important

String-based query parameter values that are received via the [IQueryAttributable](#) interface aren't automatically URL decoded.

Pass and process multiple items of data

Multiple string-based query parameters can be passed by connecting them with `&`. For example, the following code passes two data items:

C#

```
async void OnCollectionViewSelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    string elephantName = (e.CurrentSelection.FirstOrDefault() as
Animal).Name;
    string elephantLocation = (e.CurrentSelection.FirstOrDefault() as
Animal).Location;
    await Shell.Current.GoToAsync($"elephantdetails?name=
{elephantName}&location={elephantLocation}");
}
```

This code example retrieves the currently selected elephant in the `CollectionView`, and navigates to the `elephantdetails` route, passing `elephantName` and `elephantLocation` as query parameters.

To receive multiple items of data, the class that represents the page being navigated to, or the class for the page's `BindingContext`, can be decorated with a [QueryPropertyAttribute](#) for each string-based query parameter:

C#

```
[QueryProperty(nameof(Name), "name")]
[QueryProperty(nameof(Location), "location")]
public partial class ElephantDetailPage : ContentPage
{
    public string Name
    {
        set
        {
```



```

        // Custom logic
    }
}

public string Location
{
    set
    {
        // Custom logic
    }
}
...
}

```

In this example, the class is decorated with a [QueryPropertyAttribute](#) for each query parameter. The first [QueryPropertyAttribute](#) specifies that the `Name` property will receive the data passed in the `name` query parameter, while the second [QueryPropertyAttribute](#) specifies that the `Location` property will receive the data passed in the `location` query parameter. In both cases, the query parameter values are specified in the URI in the [GoToAsync](#) method call.

⚠ Warning

Receiving navigation data using the [QueryPropertyAttribute](#) isn't trim safe and shouldn't be used with full trimming or NativeAOT. Instead, you should implement the [IQueryAttributable](#) interface on types that need to accept query parameters. For more information, see [Trim a .NET MAUI app](#) and [Native AOT deployment](#).

Alternatively, navigation data can be processed by a single method by implementing the [IQueryAttributable](#) interface on the class that represents the page being navigated to, or the class for the page's `BindingContext`:

C#

```

public class ElephantDetailViewModel : IQueryAttributable,
INotifyPropertyChanged
{
    public Animal Elephant { get; private set; }

    public void ApplyQueryAttributes(IDictionary<string, object> query)
    {
        string name = HttpUtility.UrlDecode(query["name"].ToString());
        string location =
HttpUtility.UrlDecode(query["location"].ToString());
        ...
    }
}

```

```
...  
}
```

In this example, the `ApplyQueryAttributes` method retrieves the value of the `name` and `location` query parameters from the URI in the `GoToAsync` method call.

ⓘ Note

String-based query parameters and object-based navigation parameters can be simultaneously passed when performing route-based navigation.

Back button behavior

Back button appearance and behavior can be redefined by setting the `BackButtonBehavior` attached property to a `BackButtonBehavior` object. The `BackButtonBehavior` class defines the following properties:

- `Command`, of type `ICommand`, which is executed when the back button is pressed.
- `CommandParameter`, of type `object`, which is the parameter that's passed to the `Command`.
- `IconOverride`, of type `ImageSource`, the icon used for the back button.
- `IsEnabled`, of type `boolean`, indicates whether the back button is enabled. The default value is `true`.
- `IsVisible`, of type `boolean`, indicates whether the back button is visible. The default value is `true`.
- `TextOverride`, of type `string`, the text used for the back button.

All of these properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings. The `Command`, `CommandParameter`, `IconOverride`, and `TextOverride` `BindableProperty` objects have `OneTime` binding modes, which means that data goes from the source to the target but only when the `BindingContext` changes. The `IsEnabled` and `IsVisible` `BindableProperty` objects have `OneWay` binding modes, which means that data goes from the source to the target.

The following code shows an example of redefining back button appearance and behavior:

XAML

```
<ContentPage ...>  
    <Shell.BackButtonBehavior>
```

```
<BackButtonBehavior Command="{Binding BackCommand}"  
                    IconOverride="back.png" />  
</Shell.BackButtonBehavior>  
...  
</ContentPage>
```

The `Command` property is set to an `ICommand` to be executed when the back button is pressed, and the `IconOverride` property is set to the icon that's used for the back button:



.NET MAUI Shell search

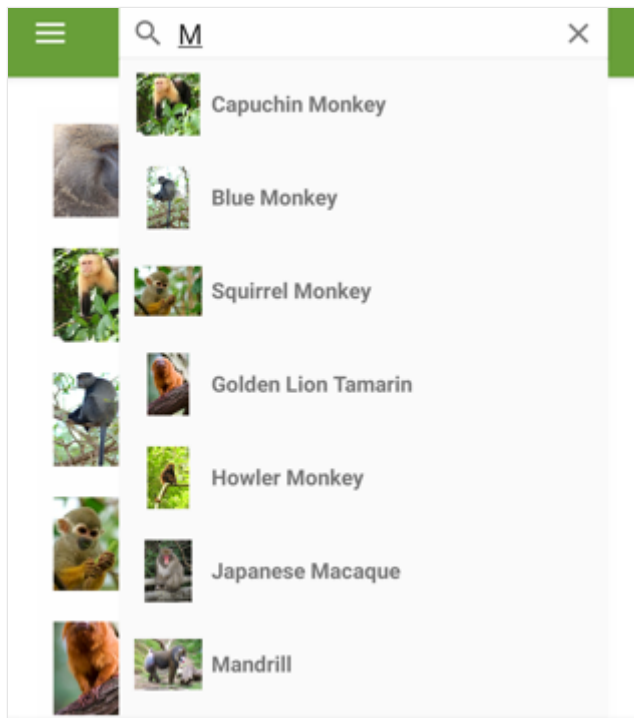
Article • 10/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) Shell includes integrated search functionality that's provided by the [SearchHandler](#) class. Search capability can be added to a page by setting the `Shell.SearchHandler` attached property to a subclassed [SearchHandler](#) object. This results in a search box being added at the top of the page:



When a query is entered into the search box, the `Query` property is updated, and on each update the `OnQueryChanged` method is executed. This method can be overridden to populate the search suggestions area with data:



Then, when a result is selected from the search suggestions area, the `OnItemSelected` method is executed. This method can be overridden to respond appropriately, such as by navigating to a detail page.

Create a SearchHandler

Search functionality can be added to a Shell app by subclassing the [SearchHandler](#) class, and overriding the `OnQueryChanged` and `OnItemSelected` methods:

C#

```
public class AnimalSearchHandler : SearchHandler
{
    public IList<Animal> Animals { get; set; }
    public Type SelectedItemNavigationTarget { get; set; }

    protected override void OnQueryChanged(string oldValue, string newValue)
    {
        base.OnQueryChanged(oldValue, newValue);

        if (string.IsNullOrEmpty(newValue))
        {
            ItemsSource = null;
        }
        else
        {
            ItemsSource = Animals
                .Where(animal =>
                    animal.Name.ToLower().Contains(newValue.ToLower()))
                .ToList<Animal>();
        }
    }

    protected override async void OnItemSelected(object item)
    {
        base.OnItemSelected(item);

        Animal animal = item as Animal;
        string navigationTarget = GetNavigationTarget();

        if (navigationTarget.Equals("catdetails") ||
            navigationTarget.Equals("dogdetails"))
        {
            // Navigate, passing a string
            await Shell.Current.GoToAsync($"{navigationTarget}?name=
{((Animal)item).Name}");
        }
        else
        {
            string lowerCasePropertyName =
                navigationTarget.Replace("details", string.Empty);
            // Capitalise the property name
            string propertyName = char.ToUpper(lowerCasePropertyName[0]) +
                lowerCasePropertyName.Substring(1);

            var navigationParameters = new Dictionary<string, object>
            {
                { propertyName, animal }
            };

            // Navigate, passing an object
            await Shell.Current.GoToAsync($"{navigationTarget}",
                navigationParameters);
        }
    }
}
```

```

    }
}

string GetNavigationTarget()
{
    return (Shell.Current as AppShell).Routes.FirstOrDefault(route =>
route.Value.Equals(SelectedItemNavigationTarget)).Key;
}
}

```

The `OnQueryChanged` override has two arguments: `oldValue`, which contains the previous search query, and `newValue`, which contains the current search query. The search suggestions area can be updated by setting the `SearchHandler.ItemsSource` property to an `IEnumerable` collection that contains items that match the current search query.

When a search result is selected by the user, the `OnItemSelected` override is executed and the `SelectedItem` property is set. In this example, the method navigates to another page that displays data about the selected `Animal`. For more information about navigation, see [Shell navigation](#).

ⓘ Note

Additional [SearchHandler](#) properties can be set to control the search box appearance.

Consume a SearchHandler

The subclassed [SearchHandler](#) can be consumed by setting the `Shell.SearchHandler` attached property to an object of the subclassed type, on the consuming page:

XAML

```

<ContentPage ...
    xmlns:controls="clr-namespace:Xaminals.Controls">
    <Shell.SearchHandler>
        <controls:AnimalSearchHandler Placeholder="Enter search term"
            ShowsResults="true"
            DisplayMemberName="Name" />
    </Shell.SearchHandler>
    ...
</ContentPage>

```

The equivalent C# code is:

C#

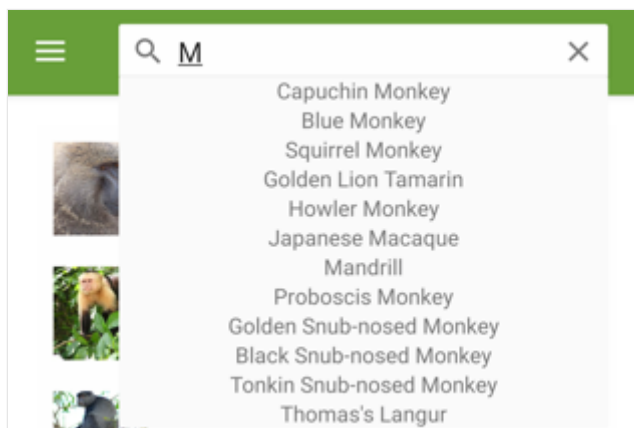
```
Shell.SetSearchHandler(this, new AnimalSearchHandler
{
    Placeholder = "Enter search term",
    ShowsResults = true,
    DisplayMemberName = "Name"
});
```

The `AnimalSearchHandler.OnQueryChanged` method returns a `List` of `Animal` objects. The `DisplayMemberName` property is set to the `Name` property of each `Animal` object, and so the data displayed in the suggestions area will be each animal name.

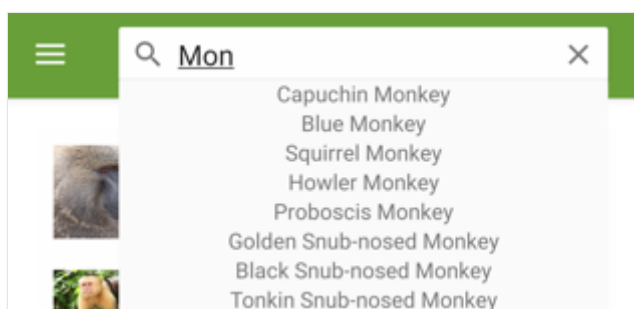
⚠ Warning

`SearchHandler.DisplayMemberName` isn't trim safe and shouldn't be used with full trimming or NativeAOT. Instead, you should provide an `ItemTemplate` to define the appearance of `SearchHandler` results. For more information, see [Define search results item appearance](#), [Trim a .NET MAUI app](#) and [Native AOT deployment](#).

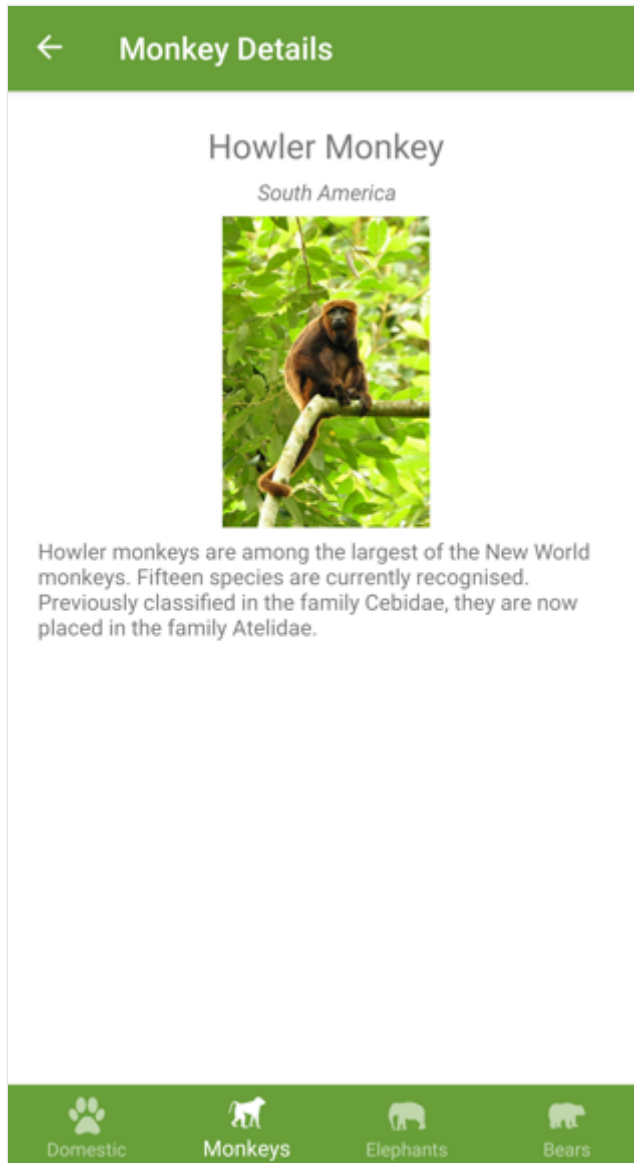
The `ShowsResults` property is set to `true`, so that search suggestions are displayed as the user enters a search query:



As the search query changes, the search suggestions area is updated:



When a search result is selected, the `MonkeyDetailPage` is navigated to, and a detail page about the selected monkey is displayed:



Define search results item appearance

In addition to displaying `string` data in the search results, the appearance of each search result item can be defined by setting the `SearchHandler.ItemTemplate` property to a [DataTemplate](#):

XAML

```
<ContentPage ...
    xmlns:controls="clr-namespace:Xaminals.Controls">
    <Shell.SearchHandler>
        <controls:AnimalSearchHandler Placeholder="Enter search term"
            ShowsResults="true">
            <controls:AnimalSearchHandler.ItemTemplate>
                <DataTemplate>
                    <Grid Padding="10"
```



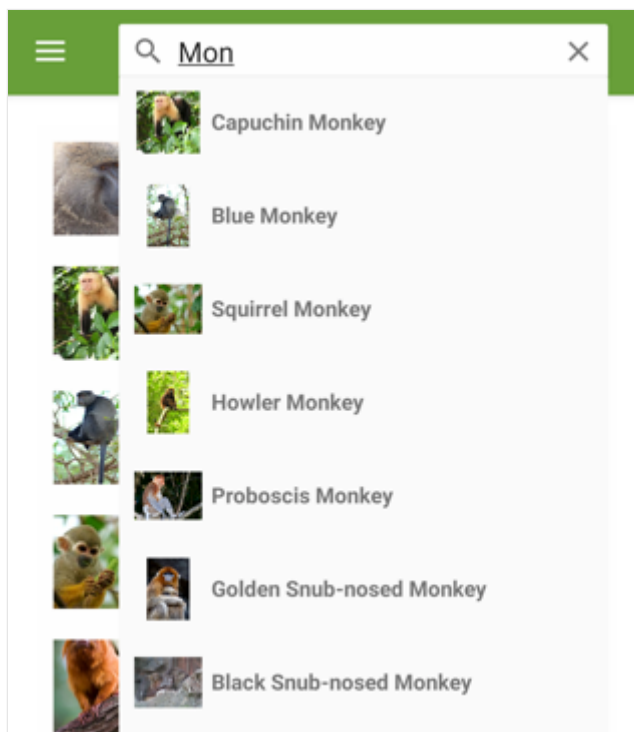
```

        ColumnDefinitions="0.15*,0.85*">
        <Image Source="{Binding ImageUrl}"
            HeightRequest="40"
            WidthRequest="40" />
        <Label Grid.Column="1"
            Text="{Binding Name}"
            FontAttributes="Bold"
            VerticalOptions="Center" />
    </Grid>
</DataTemplate>
</controls:AnimalSearchHandler.ItemTemplate>
</controls:AnimalSearchHandler>
</Shell.SearchHandler>
...
</ContentPage>

```

The elements specified in the [DataTemplate](#) define the appearance of each item in the suggestions area. In this example, layout within the [DataTemplate](#) is managed by a [Grid](#). The [Grid](#) contains an [Image](#) object, and a [Label](#) object, that both bind to properties of each `Monkey` object.

The following screenshot shows the result of templating each item in the suggestions area:



For more information about data templates, see [Data templates](#).

Search box visibility

By default, when a `SearchHandler` is added at the top of a page, the search box is visible and fully expanded. However, this behavior can be changed by setting the `SearchHandler.SearchBoxVisibility` property to one of the `SearchBoxVisibility` enumeration members:

- `Hidden` – the search box is not visible or accessible.
- `Collapsible` – the search box is hidden until the user performs an action to reveal it. On iOS the search box is revealed by vertically bouncing the page content, and on Android the search box is revealed by tapping the question mark icon.
- `Expanded` – the search box is visible and fully expanded. This is the default value of the `SearchBoxVisibility` property.

Important

On iOS, a collapsible search box requires iOS 11 or greater.

The following example shows to how to hide the search box:

XAML

```
<ContentPage ...
    xmlns:controls="clr-namespace:Xaminals.Controls">
    <Shell.SearchHandler>
        <controls:AnimalSearchHandler SearchBoxVisibility="Hidden"
            ... />
    </Shell.SearchHandler>
    ...
</ContentPage>
```

Search box focus

Tapping in a search box invokes the onscreen keyboard, with the search box gaining input focus. This can also be achieved programmatically by calling the `Focus` method, which attempts to set input focus on the search box, and returns `true` if successful.

When a search box gains focus, the `Focused` event is fired and the overridable `OnFocused` method is called.

When a search box has input focus, tapping elsewhere on the screen dismisses the onscreen keyboard, and the search box loses input focus. This can also be achieved programmatically by calling the `Unfocus` method. When a search box loses focus, the `Unfocused` event is fired and the overridable `OnUnfocus` method is called.

The focus state of a search box can be retrieved through the `IsFocused` property, which returns `true` if a `SearchHandler` currently has input focus.

SearchHandler keyboard

The keyboard that's presented when users interact with a `SearchHandler` can be set programmatically via the `Keyboard` property, to one of the following properties from the `Keyboard` class:

- `Chat` – used for texting and places where emoji are useful.
- `Default` – the default keyboard.
- `Email` – used when entering email addresses.
- `Numeric` – used when entering numbers.
- `Plain` – used when entering text, without any `KeyboardFlags` specified.
- `Telephone` – used when entering telephone numbers.
- `Text` – used when entering text.
- `Url` – used for entering file paths & web addresses.

This can be accomplished in XAML as follows:

XAML

```
<SearchHandler Keyboard="Email" />
```

The `Keyboard` class also has a `Create` factory method that can be used to customize a keyboard by specifying capitalization, spellcheck, and suggestion behavior.

`KeyboardFlags` enumeration values are specified as arguments to the method, with a customized `Keyboard` being returned. The `KeyboardFlags` enumeration contains the following values:

- `None` – no features are added to the keyboard.
- `CapitalizeSentence` – indicates that the first letter of the first word of each entered sentence will be automatically capitalized.
- `Spellcheck` – indicates that spellcheck will be performed on entered text.
- `Suggestions` – indicates that word completions will be offered on entered text.
- `CapitalizeWord` – indicates that the first letter of each word will be automatically capitalized.
- `CapitalizeCharacter` – indicates that every character will be automatically capitalized.
- `CapitalizeNone` – indicates that no automatic capitalization will occur.

- **A11** – indicates that spellcheck, word completions, and sentence capitalization will occur on entered text.

The following XAML code example shows how to customize the default **Keyboard** to offer word completions and capitalize every entered character:

XAML

```
<SearchHandler Placeholder="Enter search terms">
  <SearchHandler.Keyboard>
    <Keyboard x:FactoryMethod="Create">
      <x:Arguments>

<KeyboardFlags>Suggestions,CapitalizeCharacter</KeyboardFlags>
      </x:Arguments>
    </Keyboard>
  </SearchHandler.Keyboard>
</SearchHandler>
```

.NET MAUI Shell lifecycle

Article • 08/30/2024

 [Browse the sample](#)

Shell apps respect the .NET Multi-platform App UI (.NET MAUI) lifecycle, and additionally fire an [Appearing](#) event when a page is about to appear on the screen, and a [Disappearing](#) event when a page is about to disappear from the screen. These events are propagated to pages, and can be handled by overriding the [OnAppearing\(\)](#) or [OnDisappearing\(\)](#) methods on the page.

Note

In a Shell app, the [Appearing](#) and [Disappearing](#) events are raised from cross-platform code, prior to platform code making a page visible, or removing a page from the screen.

Modeless navigation

In a Shell app, pushing a page onto the navigation stack will result in the currently visible [ShellContent](#) object, and its page content, raising the [Disappearing](#) event. Similarly, popping the last page from the navigation stack will result in the newly visible [ShellContent](#) object, and its page content, raising the [Appearing](#) event.

For more information about modeless navigation, see [Perform modeless navigation](#).

Modal navigation

In a Shell app, pushing a modal page onto the modal navigation stack will result in all visible Shell objects raising the [Disappearing](#) event. Similarly, popping the last modal page from the modal navigation stack will result in all visible Shell objects raising the [Appearing](#) event.

For more information about modal navigation, see [Perform modal navigation](#).

Target multiple platforms from .NET MAUI single project

Article • 11/14/2023

.NET Multi-platform App UI (.NET MAUI) single project takes the platform-specific development experiences you typically encounter while developing apps and abstracts them into a single shared project that can target Android, iOS, macOS, and Windows.

.NET MAUI single project provides a simplified and consistent cross-platform development experience, regardless of the platforms being targeted. .NET MAUI single project provides the following features:

- A single shared project that can target Android, iOS, macOS, Tizen, and Windows.
- A simplified debug target selection for running your .NET MAUI apps.
- Shared resource files within the single project.
- A single app manifest that specifies the app title, ID, and version.
- Access to platform-specific APIs and tools when required.
- A single cross-platform app entry point.

.NET MAUI single project is enabled using multi-targeting and the use of SDK-style projects.

Resource files

Resource management for cross-platform app development has traditionally been problematic, because each platform has its own approach to managing resources. For example, each platform has differing image requirements that typically involves creating multiple versions of each image at different resolutions. Therefore, a single image typically has to be duplicated multiple times at different resolutions, with the resulting images having to use different filename and folder conventions on each platform.

.NET MAUI single project enables resource files to be stored in a single location while being consumed on each platform. This includes fonts, images, the app icon, the splash screen, raw assets, and CSS files for styling .NET MAUI apps. Each image resource file is used as a source image, from which images of the required resolutions are generated for each platform at build time.

 **Note**

iOS Asset Catalogs are currently unsupported in .NET MAUI single projects.

Resource files should typically be placed in the *Resources* folder of your .NET MAUI app project, or child folders of the *Resources* folder, and must have their build action set correctly. The following table shows the build actions for each resource file type:

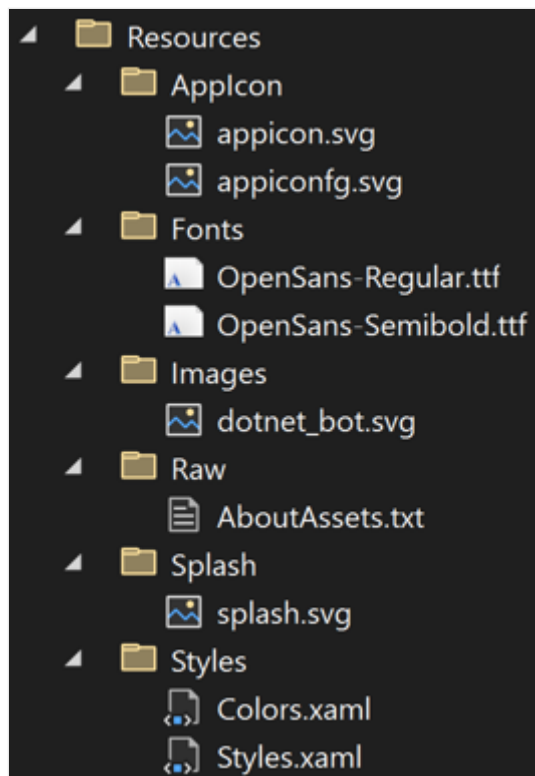
 Expand table

Resource	Build action
App icon	Mauilcon
Fonts	MauiFont
Images	MauilImage
Splash screen	MauiSplashScreen
Raw assets	MauiAsset
CSS files	MauiCss

Note

XAML files are also stored in your .NET MAUI app project, and are automatically assigned the **MauiXaml** build action when created by project and item templates. However, only XAML resource dictionaries will typically be placed in the *Resources* folder of the app project.

When a resource file is added to a .NET MAUI app project, a corresponding entry for the resource is created in the project file, with the exception of CSS files. The following screenshot shows a typical *Resources* folder containing child-folders for each resource type:



The build action for a resource file will be correctly set, if the resource has been added to the correct *Resources* child folder.

Child folders of the *Resources* folder can be designated for each resource type by editing the project file for your app:

XML

```
<ItemGroup>
  <!-- Images -->
  <MauiImage Include="Resources\Images\*" />

  <!-- Fonts -->
  <MauiFont Include="Resources\Fonts\*" />

  <!-- Raw assets -->
  <MauiAsset Include="Resources\Raw\*" />
</ItemGroup>
```

The wildcard character (*) indicates that all the files within the folder will be treated as being of the specified resource type. In addition, it's possible to include all files from child folders:

XML

```
<ItemGroup>
  <!-- Images -->
```



```
<MauiImage Include="Resources\Images\**\*" />
</ItemGroup>
```

In this example, the double wildcard character ('**') specifies that the *Images* folder can contain child folders. Therefore, `<MauiImage Include="Resources\Images\**\*" />` specifies that any files in the *Resources\Images* folder, or any child folders of the *Images* folder, will be used as source images from which images of the required resolution are generated for each platform.

Platform-specific resources will override their shared resource counterparts. For example, if you have an Android-specific image located at *Platforms\Android\Resources\drawable-xhdpi\logo.png*, and you also provide a shared *Resources\Images\logo.svg* image, the Scalable Vector Graphics (SVG) file will be used to generate the required Android images, except for the XHDPI image that already exists as a platform-specific image.

App icons

An app icon can be added to your app project by dragging an image into the *Resources\AppIcon* folder of the project, where its build action will automatically be set to **MauiIcon**. This creates a corresponding entry in your project file:

XML

```
<MauiIcon Include="Resources\AppIcon\appicon.svg" />
```

At build time, the app icon will be resized to the correct sizes for the target platform and device. The resized app icons are then added to your app package. App icons are resized to multiple resolutions because they have multiple uses, including being used to represent the app on the device, and in the app store.

For more information, see [Add an app icon to a .NET MAUI app project](#).

Images

An image can be added to your app project by dragging it into the *Resources\Images* folder of the project, where its build action will automatically be set to **MauiImage**. This creates a corresponding entry in your project file:

XML

```
<MauiImage Include="Resources\Images\logo.svg" />
```

At build time, images can be resized to the correct resolutions for the target platform and device. The resulting images are then added to your app package.

For more information, see [Add images to a .NET MAUI app project](#).

Fonts

A true type format (TTF) or open type font (OTF) font can be added to your app project by dragging it into the *Resources\Fonts* folder of your project, where its build action will automatically be set to **MauiFont**. This creates a corresponding entry per font in your project file:

XML

```
<MauiFont Include="Resources\Fonts\OpenSans-Regular.ttf" />
```

At build time, the fonts are copied to your app package.

For more information, see [Fonts](#).

Splash screen

A splash screen can be added to your app project by dragging an image into the *Resources\Splash* folder of the project, where its build action will automatically be set to **MauiSplashScreen**. This creates a corresponding entry in your project file:

XML

```
<ItemGroup>
  <MauiSplashScreen Include="Resources\Splash\splashscreen.svg" />
</ItemGroup>
```

At build time, the splash screen image is resized to the correct size for the target platform and device. The resized splash screen is then added to your app package.

For more information, see [Add a splash screen to a .NET MAUI app project](#).

Raw assets

A raw asset file, such as HTML, JSON, and video, can be added to your app project by dragging it into the *Resources\Raw* folder of your project, where its build action will automatically be set to **MauiAsset**. This creates a corresponding entry per asset in your project file:

XML

```
<MauiAsset Include="Resources\Raw\index.html" />
```

Raw assets can then be consumed by controls, as required:

XAML

```
<WebView Source="index.html" />
```

At build time, raw assets are copied to your app package. For information about disabling asset packaging, see [Disable asset file packaging](#).

CSS files

.NET MAUI apps can be partially styled with Cascading Style Sheet (CSS) files. CSS files can be added to your app project by dragging them into any folder of your project, and setting their build action to `MauiCss` in the **Properties** window.

CSS files must be loaded by the `StyleSheet` class before being added to a [ResourceDictionary](#):

XAML

```
<Application ...>
  <Application.Resources>
    <StyleSheet Source="/Resources/styles.css" />
  </Application.Resources>
</Application>
```

For more information, see [Style apps with CSS](#).

App manifest

Each platform uses its own native app manifest file to specify information such as the app title, ID, version, and more. .NET MAUI single project enables you to specify this common app data in a single location in the project file.

To specify the shared app manifest data for a project, open the shortcut menu for the project in **Solution Explorer**, and then choose **Properties**. The app title, ID, and version can then be specified in **MAUI Shared > General**:

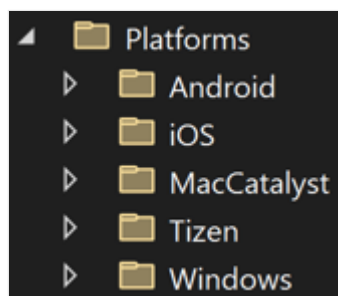
The screenshot shows the 'MAUI Shared' project settings in Visual Studio. The left sidebar lists project categories: Application, Build, Package, Code Analysis, Debug, Resources, MAUI Shared (selected), and then sub-categories for MAUI Shared: General (selected), Android, and iOS. The main area is titled 'MAUI Shared' and contains the 'General' tab with the following fields:

- Application Title**: The display name of the application. Value: MauiApp1
- Application ID**: The identifier of the application in reverse domain name format e.g. com.microsoft.maui. Value: com.companyname.mauiapp1
- Application ID (GUID)**: The identifier of the application in GUID format e.g. DA583B0B-9202-42A7-B68C-87225195B0D2. Value: 442A62EB-F23C-4286-AF14-D7551EF08428
- Application Display Version**: The display version of the application. This should be at most a three part version number e.g. 1.0.0. Value: 1.0
- Application Version**: The version of the application. This should be a single digit integer e.g. 1. Value: 1

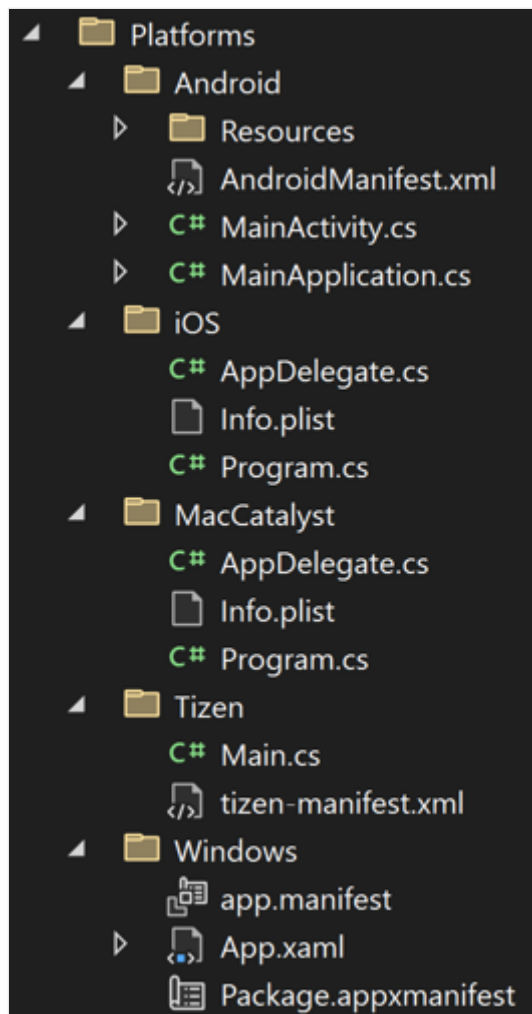
At build time the shared app manifest data is merged with platform-specific data in the native app manifest file, to produce the manifest file for the app package. For more information, see [Project configuration in .NET MAUI - MAUI Shared](#).

Platform-specific code

A .NET MAUI app project contains a *Platforms* folder, with each child folder representing a platform that .NET MAUI can target:



The folders for each platform contain platform-specific resources, and code that starts the app on each platform:



At build time, the build system only includes the code from each folder when building for that specific platform. For example, when you build for Android the files in the *Platforms\Android* folder will be built into the app package, but the files in the other *Platforms* folders won't be. This approach uses multi-targeting to target multiple platforms from a single project. Multi-targeting can be combined with partial classes and partial methods to invoke native platform functionality from cross-platform code. For more information, see [Invoke platform code](#).

In addition to this default multi-targeting approach, .NET MAUI apps can also be multi-targeted based on your own filename and folder criteria. This enables you to structure your .NET MAUI app project so that you don't have to place your platform code into child-folders of the *Platforms* folder. For more information, see [Configure multi-targeting](#).

Multi-targeting can also be combined with conditional compilation so that code is targeted to specific platforms:

```
C#
```

```
#if ANDROID
```

```
handler.NativeView.SetBackgroundColor(Colors.Red.ToNative());
```

```
#elif IOS
        handler.NativeView.BackgroundColor =
Colors.Red.ToNative();
        handler.NativeView.BorderStyle =
UIKit.UITextBorderStyle.Line;
#elif WINDOWS
        handler.NativeView.Background = Colors.Red.ToNative();

#endif
```

For more information about conditional compilation, see [Conditional compilation](#).

App entry point

While the *Platforms* folders contain platform-specific code that starts the app on each platform, .NET MAUI apps have a single cross-platform app entry point. Each platform entry point calls a `CreateMauiApp` method on the static `MauiProgram` class in your app project, and returns a `MauiApp`, which is the entry point for your app.

The `MauiProgram` class must at a minimum provide an app to run:

```
C#

namespace MyMauiApp;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>();

        return builder.Build();
    }
}
```

The `App` class derives from the `Application` class:

```
C#

namespace MyMauiApp;

public class App : Application
{
    public App()
    {
        InitializeComponent();
    }
}
```

```
protected override Window CreateWindow(IActivationState?
activationState)
{
    return new Window(new AppShell());
}
```

In the preceding example, a new [Window](#) is created whose initial content is set to the `AppShell` object. `AppShell` is a subclassed [Shell](#) class that describes the visual hierarchy of the app. For more information, see [Create a .NET MAUI Shell app](#).

Control templates

Article • 02/29/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) control templates enable you to define the visual structure of [ContentView](#) derived custom controls, and [ContentPage](#) derived pages. Control templates separate the user interface (UI) for a custom control, or page, from the logic that implements the control or page. Additional content can also be inserted into the templated custom control, or templated page, at a pre-defined location.

For example, a control template can be created that redefines the UI provided by a custom control. The control template can then be consumed by the required custom control instance. Alternatively, a control template can be created that defines any common UI that will be used by multiple pages in an app. The control template can then be consumed by multiple pages, with each page still displaying its unique content.

Create a ControlTemplate

The following example shows the code for a `CardView` custom control:

C#

```
public class CardView : ContentView
{
    public static readonly BindableProperty CardTitleProperty =
        BindableProperty.Create(nameof(CardTitle), typeof(string),
        typeof(CardView), string.Empty);
    public static readonly BindableProperty CardDescriptionProperty =
        BindableProperty.Create(nameof(CardDescription), typeof(string),
        typeof(CardView), string.Empty);

    public string CardTitle
    {
        get => (string)GetValue(CardTitleProperty);
        set => SetValue(CardTitleProperty, value);
    }

    public string CardDescription
    {
        get => (string)GetValue(CardDescriptionProperty);
        set => SetValue(CardDescriptionProperty, value);
    }
}
```



```
...  
}
```

The `CardView` class, which derives from the `ContentView` class, represents a custom control that displays data in a card-like layout. The class contains properties, which are backed by bindable properties, for the data it displays. However, the `CardView` class does not define any UI. Instead, the UI will be defined with a control template. For more information about creating `ContentView` derived custom controls, see [ContentView](#).

A control template is created with the `ControlTemplate` type. When you create a `ControlTemplate`, you combine `View` objects to build the UI for a custom control, or page. A `ControlTemplate` must have only one `View` as its root element. However, the root element usually contains other `View` objects. The combination of objects makes up the control's visual structure.

While a `ControlTemplate` can be defined inline, the typical approach to declaring a `ControlTemplate` is as a resource in a resource dictionary. Because control templates are resources, they obey the same scoping rules that apply to all resources. For example, if you declare a control template in your app-level resource dictionary, the template can be used anywhere in your app. If you define the template in a page, only that page can use the control template. For more information about resources, see [Resource dictionaries](#).

The following XAML example shows a `ControlTemplate` for `CardView` objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             ...>  
    <ContentPage.Resources>  
        <ControlTemplate x:Key="CardViewControlTemplate">  
            <Frame BindingContext="{Binding Source={RelativeSource  
TemplatedParent}}"  
                BackgroundColor="{Binding CardColor}"  
                BorderColor="{Binding BorderColor}"  
                ...>  
                <!-- UI objects that define the CardView visual structure -->  
            </Frame>  
        </ControlTemplate>  
    </ContentPage.Resources>  
    ...  
</ContentPage>
```

When a `ControlTemplate` is declared as a resource, it must have a key specified with the `x:Key` attribute so that it can be identified in the resource dictionary. In this example,

the root element of the `CardViewControlTemplate` is a `Frame` object. The `Frame` object uses the `RelativeSource` markup extension to set its `BindingContext` to the runtime object instance to which the template will be applied, which is known as the *templated parent*. The `Frame` object uses a combination of controls to define the visual structure of a `CardView` object. The binding expressions of these objects resolve against `CardView` properties, due to inheriting the `BindingContext` from the root `Frame` element. For more information about the `RelativeSource` markup extension, see [Relative bindings](#).

Consume a ControlTemplate

A `ControlTemplate` can be applied to a `ContentView` derived custom control by setting its `ControlTemplate` property to the control template object. Similarly, a `ControlTemplate` can be applied to a `ContentPage` derived page by setting its `ControlTemplate` property to the control template object. At runtime, when a `ControlTemplate` is applied, all of the controls that are defined in the `ControlTemplate` are added to the visual tree of the templated custom control, or templated page.

The following example shows the `CardViewControlTemplate` being assigned to the `ControlTemplate` property of two `CardView` objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:controls="clr-namespace:ControlTemplateDemos.Controls"
  ...>
  <StackLayout Margin="30">
    <controls:CardView BorderColor="DarkGray"
      CardTitle="John Doe"
      CardDescription="Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Nulla elit dolor, convallis non interdum."
      IconBackgroundColor="SlateGray"
      IconImageSource="user.png"
      ControlTemplate="{StaticResource
CardViewControlTemplate}" />
    <controls:CardView BorderColor="DarkGray"
      CardTitle="Jane Doe"
      CardDescription="Phasellus eu convallis mi. In
tempus augue eu dignissim fermentum. Morbi ut lacus vitae eros lacinia."
      IconBackgroundColor="SlateGray"
      IconImageSource="user.png"
      ControlTemplate="{StaticResource
CardViewControlTemplate}" />
  </StackLayout>
</ContentPage>
```

In this example, the controls in the `CardViewControlTemplate` become part of the visual tree for each `CardView` object. Because the root `Frame` object for the control template sets its `BindingContext` to the templated parent, the `Frame` and its children resolve their binding expressions against the properties of each `CardView` object.

The following screenshot shows the `CardViewControlTemplate` applied to the the `CardView` objects:



Important

The point in time that a [ControlTemplate](#) is applied to a control instance can be detected by overriding the [OnApplyTemplate](#) method in the templated custom control, or templated page. For more information, see [Get a named element from a template](#).

Pass parameters with TemplateBinding

The [TemplateBinding](#) markup extension binds a property of an element that is in a [ControlTemplate](#) to a public property that is defined by the templated custom control or templated page. When you use a [TemplateBinding](#), you enable properties on the control to act as parameters to the template. Therefore, when a property on a templated custom control or templated page is set, that value is passed onto the element that has the [TemplateBinding](#) on it.

Important

The [TemplateBinding](#) markup expression enables the [RelativeSource](#) binding from the previous control template to be removed, and replaces the `Binding` expressions.

The [TemplateBinding](#) markup extension defines the following properties:

- [Path](#), of type `string`, the path to the property.
- [Mode](#), of type [BindingMode](#), the direction in which changes propagate between the *source* and *target*.
- [Converter](#), of type [IValueConverter](#), the binding value converter.
- [ConverterParameter](#), of type `object`, the parameter to the binding value converter.
- [StringFormat](#), of type `string`, the string format for the binding.

The [ContentProperty](#) for the [TemplateBinding](#) markup extension is [Path](#). Therefore, the "Path=" part of the markup extension can be omitted if the path is the first item in the [TemplateBinding](#) expression. For more information about using these properties in a binding expression, see [Data binding](#).

Warning

The [TemplateBinding](#) markup extension should only be used within a [ControlTemplate](#). However, attempting to use a [TemplateBinding](#) expression outside of a [ControlTemplate](#) will not result in a build error or an exception being thrown.

The following XAML example shows a [ControlTemplate](#) for `CardView` objects, that uses the [TemplateBinding](#) markup extension:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             ...>
    <ContentPage.Resources>
        <ControlTemplate x:Key="CardViewControlTemplate">
            <Frame BackgroundColor="{TemplateBinding CardColor}"
                  BorderColor="{TemplateBinding BorderColor}"
                  ...>
                <!-- UI objects that define the CardView visual structure -->
            </Frame>
        </ControlTemplate>
    </ContentPage.Resources>
```

...

`</ContentPage>`

In this example, the [TemplateBinding](#) markup extension resolves binding expressions against the properties of each `CardView` object. The following screenshot shows the `CardViewControlTemplate` applied to the `CardView` objects:



Important

Using the [TemplateBinding](#) markup extension is equivalent to setting the [BindingContext](#) of the root element in the template to its templated parent with the [RelativeSource](#) markup extension, and then resolving bindings of child objects with the `Binding` markup extension. In fact, the [TemplateBinding](#) markup extension creates a `Binding` whose `Source` is `RelativeBindingSource.TemplatedParent`.

Apply a ControlTemplate with a style

Control templates can also be applied with styles. This is achieved by creating an *implicit* or *explicit* style that consumes the [ControlTemplate](#).

The following XAML example shows an *implicit* style that consumes the `CardViewControlTemplate`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:controls="clr-namespace:ControlTemplateDemos.Controls"
```

```

        ...>
    <ContentPage.Resources>
        <ControlTemplate x:Key="CardViewControlTemplate">
            ...
        </ControlTemplate>

        <Style TargetType="controls:CardView">
            <Setter Property="ControlTemplate"
                Value="{StaticResource CardViewControlTemplate}" />
        </Style>
    </ContentPage.Resources>
    <StackLayout Margin="30">
        <controls:CardView BorderColor="DarkGray"
            CardTitle="John Doe"
            CardDescription="Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Nulla elit dolor, convallis non interdum."
            IconBackgroundColor="SlateGray"
            IconImageSource="user.png" />
        ...
    </StackLayout>
</ContentPage>

```

In this example, the *implicit* [Style](#) is automatically applied to each `CardView` object, and sets the [ControlTemplate](#) property of each `CardView` to `CardViewControlTemplate`.

For more information about styles, see [Styles](#).

Redefine a control's UI

When a [ControlTemplate](#) is instantiated and assigned to the [ControlTemplate](#) property of a [ContentView](#) derived custom control, or a [ContentPage](#) derived page, the visual structure defined for the custom control or page is replaced with the visual structure defined in the [ControlTemplate](#).

For example, the `CardViewUI` custom control defines its user interface using the following XAML:

XAML

```

<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="ControlTemplateDemos.Controls.CardViewUI"
    x:Name="this">
    <Frame BindingContext="{x:Reference this}"
        BackgroundColor="{Binding CardColor}"
        BorderColor="{Binding BorderColor}"
        ...>
        <!-- UI objects that define the CardView visual structure -->

```

```
</Frame>
</ContentView>
```

However, the controls that comprise this UI can be replaced by defining a new visual structure in a [ControlTemplate](#), and assigning it to the [ControlTemplate](#) property of a `CardViewUI` object:

XAML

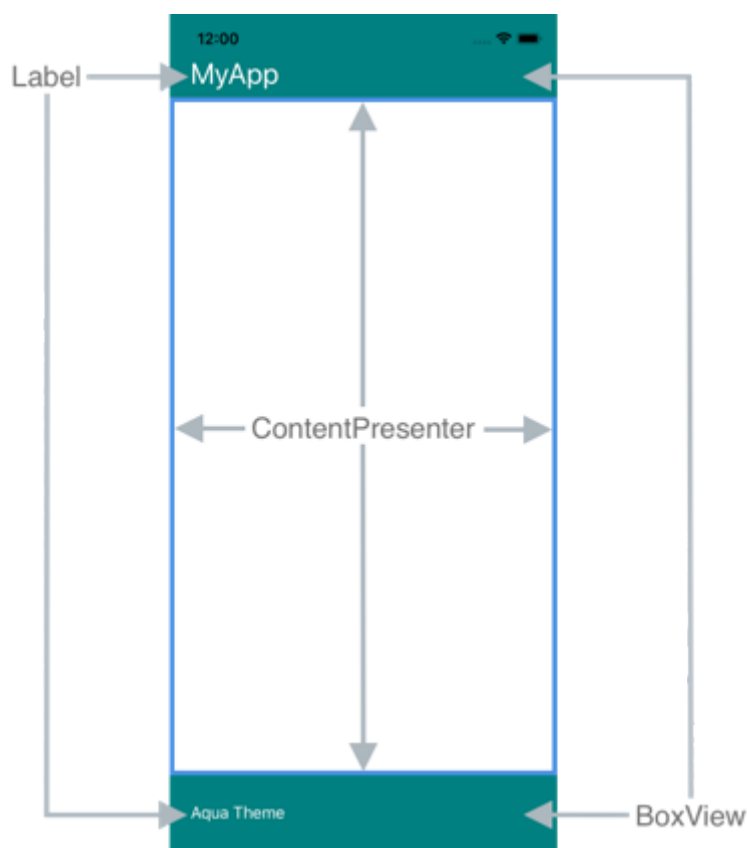
```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             ...>
    <ContentPage.Resources>
        <ControlTemplate x:Key="CardViewCompressed">
            <Grid RowDefinitions="100"
                  ColumnDefinitions="100, *"
                  >
                <Image Source="{TemplateBinding IconImageSource}"
                       BackgroundColor="{TemplateBinding
IconBackgroundColor}"
                       ...>
                <!-- Other UI objects that define the CardView visual
structure -->
            </Grid>
        </ControlTemplate>
    </ContentPage.Resources>
    <StackLayout Margin="30">
        <controls:CardViewUI BorderColor="DarkGray"
                             CardTitle="John Doe"
                             CardDescription="Lorem ipsum dolor sit amet,
consectetur adipiscing elit. Nulla elit dolor, convallis non interdum."
                             IconBackgroundColor="SlateGray"
                             IconImageSource="user.png"
                             ControlTemplate="{StaticResource
CardViewCompressed}" />
        ...
    </StackLayout>
</ContentPage>
```

In this example, the visual structure of the `CardViewUI` object is redefined in a [ControlTemplate](#) that provides a more compact visual structure that's suitable for a condensed list:



Substitute content into a ContentPresenter

A [ContentPresenter](#) can be placed in a control template to mark where content to be displayed by the templated custom control or templated page will appear. The custom control or page that consumes the control template will then define content to be displayed by the [ContentPresenter](#). The following diagram illustrates a [ControlTemplate](#) for a page that contains a number of controls, including a [ContentPresenter](#) marked by a blue rectangle:



The following XAML shows a control template named `TealTemplate` that contains a [ContentPresenter](#) in its visual structure:

XAML


```

<ControlTemplate x:Key="TealTemplate">
    <Grid RowDefinitions="0.1*, 0.8*, 0.1*">
        <BoxView Color="Teal" />
        <Label Margin="20,0,0,0"
            Text="{TemplateBinding HeaderText}"
            ... />
        <ContentPresenter Grid.Row="1" />
        <BoxView Grid.Row="2"
            Color="Teal" />
        <Label x:Name="changeThemeLabel"
            Grid.Row="2"
            Margin="20,0,0,0"
            Text="Change Theme"
            ...>
            <Label.GestureRecognizers>
                <TapGestureRecognizer Tapped="OnChangeThemeLabelTapped" />
            </Label.GestureRecognizers>
        </Label>
        <controls:HyperlinkLabel Grid.Row="2"
            Margin="0,0,20,0"
            Text="Help"

Url="https://learn.microsoft.com/dotnet/maui/"
            ... />

    </Grid>
</ControlTemplate>

```

The following example shows `TealTemplate` assigned to the `ControlTemplate` property of a `ContentPage` derived page:

XAML

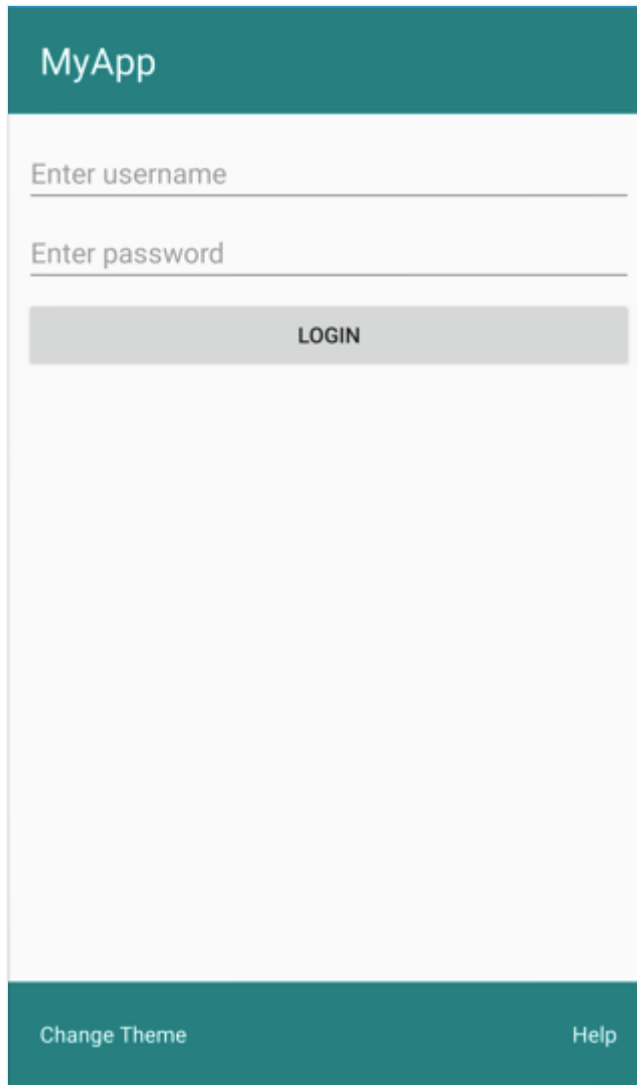
```

<controls:HeaderFooterPage
xmlns="http://schemas.microsoft.com/dotnet/2021/maui"

xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        xmlns:controls="clr-
namespace:ControlTemplateDemos.Controls"
        ControlTemplate="{StaticResource TealTemplate}"
        HeaderText="MyApp"
        ...>
    <StackLayout Margin="10">
        <Entry Placeholder="Enter username" />
        <Entry Placeholder="Enter password"
            IsPassword="True" />
        <Button Text="Login" />
    </StackLayout>
</controls:HeaderFooterPage>

```

At runtime, when `TealTemplate` is applied to the page, the page content is substituted into the [ContentPresenter](#) defined in the control template:



Get a named element from a template

Named elements within a control template can be retrieved from the templated custom control or templated page. This can be achieved with the [GetTemplateChild](#) method, which returns the named element in the instantiated [ControlTemplate](#) visual tree, if found. Otherwise, it returns `null`.

After a control template has been instantiated, the template's [OnApplyTemplate](#) method is called. The [GetTemplateChild](#) method should therefore be called from a [OnApplyTemplate](#) override in the templated control or templated page.

Important

The [GetTemplateChild](#) method should only be called after the [OnApplyTemplate](#) method has been called.

The following XAML shows a control template named `TealTemplate` that can be applied to `ContentPage` derived pages:

XAML

```
<ControlTemplate x:Key="TealTemplate">
    <Grid>
        ...
        <Label x:Name="changeThemeLabel"
            Text="Change Theme"
            ...>
            <Label.GestureRecognizers>
                <TapGestureRecognizer Tapped="OnChangeThemeLabelTapped" />
            </Label.GestureRecognizers>
        </Label>
        ...
    </Grid>
</ControlTemplate>
```

In this example, the `Label` element is named, and can be retrieved in the code for the templated page. This is achieved by calling the `GetTemplateChild` method from the `OnApplyTemplate` override for the templated page:

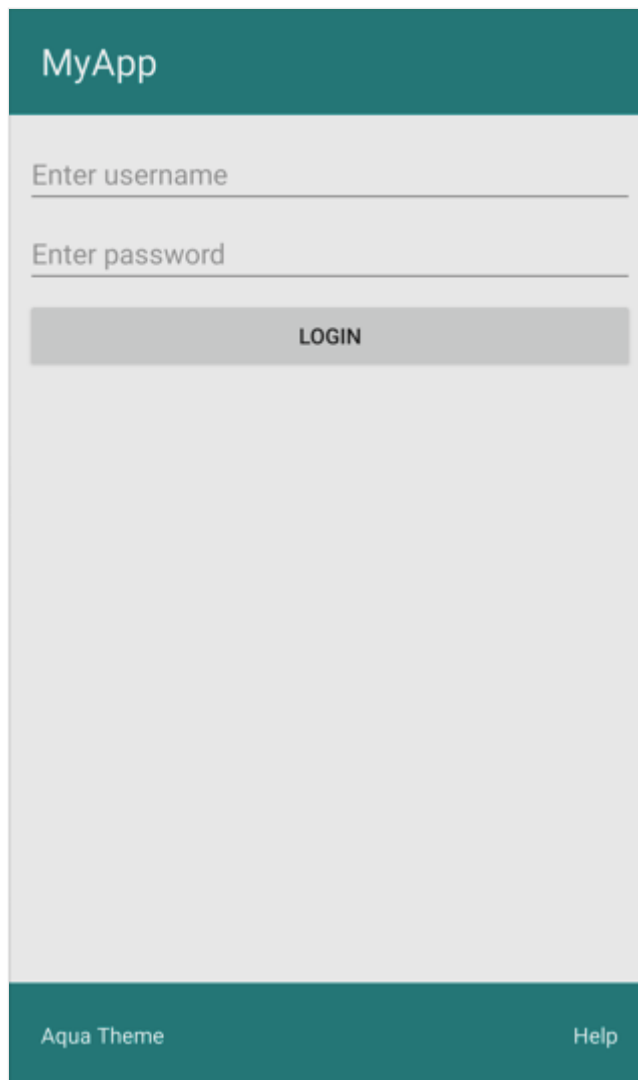
C#

```
public partial class AccessTemplateElementPage : HeaderFooterPage
{
    Label themeLabel;

    public AccessTemplateElementPage()
    {
        InitializeComponent();
    }

    protected override void OnApplyTemplate()
    {
        base.OnApplyTemplate();
        themeLabel = (Label)GetTemplateChild("changeThemeLabel");
        themeLabel.Text = OriginalTemplate ? "Aqua Theme" : "Teal Theme";
    }
}
```

In this example, the `Label` object named `changeThemeLabel` is retrieved once the `ControlTemplate` has been instantiated. `changeThemeLabel` can then be accessed and manipulated by the `AccessTemplateElementPage` class. The following screenshot shows that the text displayed by the `Label` has been changed:



Bind to a viewmodel

A [ControlTemplate](#) can data bind to a viewmodel, even when the [ControlTemplate](#) binds to the templated parent (the runtime object instance to which the template is applied).

The following XAML example shows a page that consumes a viewmodel named

`PeopleViewModel`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:ControlTemplateDemos"
              xmlns:controls="clr-namespace:ControlTemplateDemos.Controls"
              ...>
    <ContentPage.BindingContext>
        <local:PeopleViewModel />
    </ContentPage.BindingContext>

    <ContentPage.Resources>
        <DataTemplate x:Key="PersonTemplate">
            <controls:CardView BorderColor="DarkGray">
```

```

        CardTitle="{Binding Name}"
        CardDescription="{Binding Description}"
        ControlTemplate="{StaticResource
CardViewControlTemplate}" />
    </DataTemplate>
</ContentPage.Resources>

    <StackLayout Margin="10"
        BindableLayout.ItemsSource="{Binding People}"
        BindableLayout.ItemTemplate="{StaticResource
PersonTemplate}" />
</ContentPage>

```

In this example, the `BindingContext` of the page is set to a `PeopleViewModel` instance. This viewmodel exposes a `People` collection and an `ICommand` named `DeletePersonCommand`. The `StackLayout` on the page uses a bindable layout to data bind to the `People` collection, and the `ItemTemplate` of the bindable layout is set to the `PersonTemplate` resource. This `DataTemplate` specifies that each item in the `People` collection will be displayed using a `CardView` object. The visual structure of the `CardView` object is defined using a `ControlTemplate` named `CardViewControlTemplate`:

XAML

```

<ControlTemplate x:Key="CardViewControlTemplate">
    <Frame BindingContext="{Binding Source={RelativeSource
TemplatedParent}}">
        BackgroundColor="{Binding CardColor}"
        BorderColor="{Binding BorderColor}"
        ...>
        <!-- UI objects that define the CardView visual structure -->
    </Frame>
</ControlTemplate>

```

In this example, the root element of the `ControlTemplate` is a `Frame` object. The `Frame` object uses the `RelativeSource` markup extension to set its `BindingContext` to the templated parent. The binding expressions of the `Frame` object and its children resolve against `CardView` properties, due to inheriting the `BindingContext` from the root `Frame` element. The following screenshot shows the page displaying the `People` collection:



While the objects in the [ControlTemplate](#) bind to properties on its templated parent, the [Button](#) within the control template binds to both its templated parent, and to the `DeletePersonCommand` in the viewmodel. This is because the `Button.Command` property redefines its binding source to be the binding context of the ancestor whose binding context type is `PeopleViewModel`, which is the [StackLayout](#). The `Path` part of the binding expressions can then resolve the `DeletePersonCommand` property. However, the `Button.CommandParameter` property doesn't alter its binding source, instead inheriting it from its parent in the [ControlTemplate](#). Therefore, the `CommandParameter` property binds to the `CardTitle` property of the `CardView`.

The overall effect of the [Button](#) bindings is that when the [Button](#) is tapped, the `DeletePersonCommand` in the `PeopleViewModel` class is executed, with the value of the `CardName` property being passed to the `DeletePersonCommand`. This results in the specified `CardView` being removed from the bindable layout.

For more information about relative bindings, see [Relative bindings](#).

Data templates

Article • 04/03/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) data templates provide the ability to define the presentation of data on supported controls.

Consider a [CollectionView](#) that displays a collection of `Person` objects. The following example shows the definition of the `Person` class:

C#

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Location { get; set; }
}
```

The `Person` class defines `Name`, `Age`, and `Location` properties, which can be set when a `Person` object is created. A control that displays collections, such as [CollectionView](#), can be used to display `Person` objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:DataTemplates"
    x:Class="DataTemplates.WithoutDataTemplatePage">
    <StackLayout>
        <CollectionView>
            <CollectionView.ItemsSource>
                <x:Array Type="{x:Type local:Person}">
                    <local:Person Name="Steve" Age="21" Location="USA" />
                    <local:Person Name="John" Age="37" Location="USA" />
                    <local:Person Name="Tom" Age="42" Location="UK" />
                    <local:Person Name="Lucas" Age="29" Location="Germany" />
                </x:Array>
            </CollectionView.ItemsSource>
        </CollectionView>
    </StackLayout>
</ContentPage>
```

In this example, items are added to the `CollectionView` by initializing its `ItemsSource` property from an array of `Person` objects. `CollectionView` calls `ToString` when displaying the objects in the collection. However, because there is no `Person.ToString` override, `ToString` returns the type name of each object:

DataTemplates.Person
DataTemplates.Person
DataTemplates.Person
DataTemplates.Person
DataTemplates.Person
DataTemplates.Person

The `Person` object can override the `ToString` method to display meaningful data:

```
C#  
  
public class Person  
{  
    ...  
    public override string ToString ()  
    {  
        return Name;  
    }  
}
```

This results in the `CollectionView` displaying the `Person.Name` property value for each object in the collection:

Steve
John
Tom
Lucas
Tariq
Jane

The `Person.ToString` override could return a formatted string consisting of the `Name`, `Age`, and `Location` properties. However, this approach only offers limited control over

the appearance of each item of data. For more flexibility, a [DataTemplate](#) can be created that defines the appearance of the data.

Create a DataTemplate

A [DataTemplate](#) is used to specify the appearance of data, and typically uses data binding to display data. A common usage scenario for data templates is when displaying data from a collection of objects in a control such as a [CollectionView](#) or [CarouselView](#). For example, when a [CollectionView](#) is bound to a collection of `Person` objects, the [ItemTemplate](#) property can be set to a [DataTemplate](#) that defines the appearance of each `Person` object in the [CollectionView](#). The [DataTemplate](#) will contain objects that bind to property values of each `Person` object. For more information about data binding, see [Data binding](#).

A [DataTemplate](#) that's defined inline in a control is known as an *inline template*. Alternatively, data templates can be defined as a control-level, page-level, or app-level resource. Choosing where to define a [DataTemplate](#) impacts where it can be used:

- A [DataTemplate](#) defined at the control-level can only be applied to the control.
- A [DataTemplate](#) defined at the page-level can be applied to multiple controls on the page.
- A [DataTemplate](#) defined at the app-level can be applied to valid controls throughout the app.

ⓘ Note

Data templates lower in the view hierarchy take precedence over those defined higher up when they share `x:Key` attributes. For example, an app-level data template will be overridden by a page-level data template, and a page-level data template will be overridden by a control-level data template, or an inline data template.

A [DataTemplate](#) can be created inline, with a type, or as a resource, regardless of where it's defined.

Create an inline DataTemplate

An inline data template, which is one that's defined inline in a control, should be used if there's no need to reuse the data template elsewhere. The objects specified in the

[DataTemplate](#) define the appearance of each item of data. A control such as [CollectionView](#) can then set its [ItemTemplate](#) property to the inline [DataTemplate](#):

XAML

```
<CollectionView>
  <CollectionView.ItemsSource>
    <x:Array Type="{x:Type local:Person}">
      <local:Person Name="Steve" Age="21" Location="USA" />
      <local:Person Name="John" Age="37" Location="USA" />
      <local:Person Name="Tom" Age="42" Location="UK" />
      <local:Person Name="Lucas" Age="29" Location="Germany" />
      <local:Person Name="Tariq" Age="39" Location="UK" />
      <local:Person Name="Jane" Age="30" Location="USA" />
    </x:Array>
  </CollectionView.ItemsSource>
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Grid>
        ...
        <Label Text="{Binding Name}" FontAttributes="Bold" />
        <Label Grid.Column="1" Text="{Binding Age}" />
        <Label Grid.Column="2" Text="{Binding Location}"
HorizontalTextAlignment="End" />
      </Grid>
    </DataTemplate>
  </CollectionView.ItemTemplate>
</CollectionView>
```

In a [CollectionView](#), the child of an inline [DataTemplate](#) must derive from [BindableObject](#). In this example, a [Grid](#), which derives from [Layout](#) is used. The [Grid](#) contains three [Label](#) objects that bind their `Text` properties to properties of each [Person](#) object in the collection. The following screenshot shows the resulting appearance:

Steve	21	USA
John	37	USA
Tom	42	UK
Lucas	29	Germany
Tariq	39	UK
Jane	30	USA

Create a DataTemplate with a type

A [DataTemplate](#) can be created with a custom view type. The advantage of this approach is that the appearance defined by the view can be reused by multiple data templates throughout an app. A control such as [CollectionView](#) can then set its [ItemTemplate](#) property to the [DataTemplate](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:DataTemplates"
             x:Class="DataTemplates.WithDataTemplatePageFromType">
    <StackLayout>
        <CollectionView>
            <CollectionView.ItemsSource>
                <x:Array Type="{x:Type local:Person}">
                    <local:Person Name="Steve" Age="21" Location="USA" />
                    ...
                </x:Array>
            </CollectionView.ItemsSource>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <local:PersonView />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </StackLayout>
</ContentPage>
```

In this example, the [ItemTemplate](#) property is set to a [DataTemplate](#) that's created from a custom type that defines the view appearance. The custom type must derive from [ContentView](#):

XAML

```
<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="DataTemplates.PersonView">
    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="0.5*" />
            <ColumnDefinition Width="0.2*" />
            <ColumnDefinition Width="0.3*" />
        </Grid.ColumnDefinitions>
        <Label Text="{Binding Name}" FontAttributes="Bold" />
        <Label Grid.Column="1" Text="{Binding Age}" />
        <Label Grid.Column="2" Text="{Binding Location}"
            HorizontalTextAlignment="End" />
    </Grid>
</ContentView>
```

```
</Grid>
</ContentView>
```

In this example, layout within the [ContentView](#) is managed by a [Grid](#). The [Grid](#) contains three [Label](#) objects that bind their `Text` properties to properties of each `Person` object in the collection.

For more information about creating custom views, see [ContentView](#).

Create a DataTemplate as a resource

Data templates can be created as reusable objects in a [ResourceDictionary](#). This is achieved by giving each [DataTemplate](#) a unique `x:Key` value, which provides it with a descriptive key in the [ResourceDictionary](#). A control such as [CollectionView](#) can then set its [ItemTemplate](#) property to the [DataTemplate](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:DataTemplates"
             x:Class="DataTemplates.WithDataTemplateResource">
    <ContentPage.Resources>
        <ResourceDictionary>
            <DataTemplate x:Key="personTemplate">
                <Grid>
                    ...
                </Grid>
            </DataTemplate>
        </ResourceDictionary>
    </ContentPage.Resources>

    <StackLayout>
        <CollectionView ItemTemplate="{StaticResource personTemplate}">
            <CollectionView.ItemsSource>
                <x:Array Type="{x:Type local:Person}">
                    <local:Person Name="Steve" Age="21" Location="USA" />
                    ...
                </x:Array>
            </CollectionView.ItemsSource>
        </CollectionView>
    </StackLayout>
</ContentPage>
```

In this example, the [DataTemplate](#) is assigned to the [ItemTemplate](#) property by using the [StaticResource](#) markup extension. While the [DataTemplate](#) is defined in the page's [ResourceDictionary](#), it could also be defined at the control-level or app-level.

Create a DataTemplateSelector

A [DataTemplateSelector](#) can be used to choose a [DataTemplate](#) at runtime based on the value of a data-bound property. This enables multiple data templates to be applied to the same type of object, to choose their appearance at runtime. A data template selector enables scenarios such as a [CollectionView](#) or [CarouselView](#) binding to a collection of objects where the appearance of each object can be chosen at runtime by the data template selector returning a specific [DataTemplate](#).

A data template selector is implemented by creating a class that inherits from [DataTemplateSelector](#). The [OnSelectTemplate](#) method should then be overridden to return a specific [DataTemplate](#):

C#

```
public class PersonDataTemplateSelector : DataTemplateSelector
{
    public DataTemplate ValidTemplate { get; set; }
    public DataTemplate InvalidTemplate { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
        BindableObject container)
    {
        return ((Person)item).DateOfBirth.Year >= 1980 ? ValidTemplate :
            InvalidTemplate;
    }
}
```

In this example, the [OnSelectTemplate](#) method returns a specific data template based on the value of the `DateOfBirth` property. The returned data template is defined by the `ValidTemplate` or `InvalidTemplate` property, which are set when consuming the data template selector.

Limitations

[DataTemplateSelector](#) objects have the following limitations:

- The [DataTemplateSelector](#) subclass must always return the same template for the same data if queried multiple times.
- The [DataTemplateSelector](#) subclass must not return another [DataTemplateSelector](#) subclass.
- The [DataTemplateSelector](#) subclass must not return new instances of a [DataTemplate](#) on each call. Instead, the same instance must be returned. Failure to do so will create a memory leak and will disable control virtualization.

Consume a DataTemplateSelector

A data template selector can be consumed by creating it as a resource and assigning its instance to .NET MAUI control properties of type [DataTemplate](#), such as [ItemTemplate](#).

The following example shows declaring `PersonDataTemplateSelector` as a page-level resource:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:local="clr-namespace:Selector"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Selector.MainPage">
    <ContentPage.Resources>
        <DataTemplate x:Key="validPersonTemplate">
            <Grid>
                ...
            </Grid>
        </DataTemplate>
        <DataTemplate x:Key="invalidPersonTemplate">
            <Grid>
                ...
            </Grid>
        </DataTemplate>
        <local:PersonDataTemplateSelector x:Key="personDataTemplateSelector"
                                         ValidTemplate="{StaticResource
validPersonTemplate}"
                                         InvalidTemplate="{StaticResource
invalidPersonTemplate}" />
    </ContentPage.Resources>
    ...
</ContentPage>
```

In this example, the page-level [ResourceDictionary](#) defines two [DataTemplate](#) objects and a `PersonDataTemplateSelector` object. The `PersonDataTemplateSelector` object sets its `ValidTemplate` and `InvalidTemplate` properties to the [DataTemplate](#) objects using the [StaticResource](#) markup extension. While the resources are defined in the page's [ResourceDictionary](#), they could also be defined at the control-level or app-level.

The `PersonDataTemplateSelector` object can be consumed by assigning it to the [ItemTemplate](#) property:

XAML

```
<CollectionView x:Name="collectionView"
                ItemTemplate="{StaticResource personDataTemplateSelector}"
/>
```

At runtime, the [CollectionView](#) calls the `PersonDataTemplateSelector.OnSelectTemplate` method for each of the items in the underlying collection, with the call passing the data object as the `item` parameter. The returned [DataTemplate](#) is then applied to that object.

The following screenshot shows the result of the [CollectionView](#) applying the `PersonDataTemplateSelector` to each object in the underlying collection:

Kath	11/20/1985	France
Steve	1/15/1975	USA
Lucas	2/5/1988	Germany
John	2/20/1976	USA
Tariq	1/10/1987	UK
Jane	8/30/1982	USA
Tom	3/10/1977	UK

In this example, any `Person` object that has a `DateOfBirth` property value greater than or equal to 1980 is displayed in green, with the remaining objects being displayed in red.

Triggers

Article • 03/03/2023

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) triggers allow you to express actions declaratively in XAML that change the appearance of controls based on events or data changes. In addition, state triggers, which are a specialized group of triggers, define when a [VisualState](#) should be applied.

You can assign a trigger directly to a control's [Triggers](#) collection, or add it to a page-level or app-level resource dictionary to be applied to multiple controls.

Property triggers

A [Trigger](#) represents a trigger that applies property values, or performs actions, when the specified property meets a specified condition.

The following example shows a [Trigger](#) that changes an [Entry](#) background color when it receives focus:

XAML

```
<Entry Placeholder="Enter name">
  <Entry.Triggers>
    <Trigger TargetType="Entry"
      Property="IsFocused"
      Value="True">
      <Setter Property="BackgroundColor"
        Value="Yellow" />
      <!-- Multiple Setter elements are allowed -->
    </Trigger>
  </Entry.Triggers>
</Entry>
```

The trigger's declaration specifies the following:

- [TargetType](#) - the control type that the trigger applies to.
- [Property](#) - the property on the control that is monitored.
- [Value](#) - the value, when it occurs for the monitored property, that causes the trigger to activate.
- [Setter](#) - a collection of [Setter](#) elements that are applied when the trigger condition is met.

In addition, optional [EnterActions](#) and [ExitActions](#) collections can be specified. For more information, see [EnterActions and ExitActions](#).

Apply a trigger using a style

Triggers can also be added to a [Style](#) declaration on a control, in a page, or an application [ResourceDictionary](#). The following example declares an *implicit* style for all [Entry](#) controls on the page:

XAML

```
<ContentPage.Resources>
    <Style TargetType="Entry">
        <Style.Triggers>
            <Trigger TargetType="Entry"
                Property="IsFocused"
                Value="True">
                <Setter Property="BackgroundColor"
                    Value="Yellow" />
            <!-- Multiple Setter elements are allowed -->
            </Trigger>
        </Style.Triggers>
    </Style>
</ContentPage.Resources>
```

Data triggers

A [DataTrigger](#) represents a trigger that applies property values, or performs actions, when the bound data meets a specified condition. The `Binding` markup extension is used to monitor for the specified condition.

The following example shows a [DataTrigger](#) that disables a [Button](#) when the [Entry](#) is empty:

XAML

```
<Entry x:Name="entry"
    Text=""
    Placeholder="Enter text" />
<Button Text="Save">
    <Button.Triggers>
        <DataTrigger TargetType="Button"
            Binding="{Binding Source={x:Reference entry},
                Path=Text.Length}"
            Value="0">
            <Setter Property="IsEnabled"
                Value="False" />
        </DataTrigger>
    </Button.Triggers>
</Button>
```

```
        <!-- Multiple Setter elements are allowed -->
    </DataTrigger>
</Button.Triggers>
</Button>
```

In this example, when the length of the [Entry](#) is zero, the trigger is activated.

💡 Tip

When evaluating `Path=Text.Length` always provide a default value for the target property (eg. `Text=""`) because otherwise it will be `null` and the trigger won't work like you expect.

In addition, optional [EnterActions](#) and [ExitActions](#) collections can be specified. For more information, see [EnterActions](#) and [ExitActions](#).

Event triggers

An [EventTrigger](#) represents a trigger that applies a set of actions in response to an event. Unlike [Trigger](#), [EventTrigger](#) has no concept of termination of state, so the actions will not be undone once the condition that raised the event is no longer true.

An [EventTrigger](#) only requires an `Event` property to be set:

XAML

```
<EventTrigger Event="TextChanged">
    <local:NumericValidationTriggerAction />
</EventTrigger>
```

In this example, there are no [Setter](#) elements. Instead, there's a `NumericValidationTriggerAction` object.

⚠ Note

Event triggers don't support [EnterActions](#) and [ExitActions](#).

A trigger action implementation must:

- Implement the generic [TriggerAction<T>](#) class, with the generic parameter corresponding with the type of control the trigger will be applied to. You can use

classes such as [VisualElement](#) to write trigger actions that work with a variety of controls, or specify a control type like [Entry](#).

- Override the [Invoke](#) method. This method is called whenever the trigger event occurs.
- Optionally expose properties that can be set in XAML when the trigger is declared.

The following example shows the `NumericValidationTriggerAction` class:

C#

```
public class NumericValidationTriggerAction : TriggerAction<Entry>
{
    protected override void Invoke(Entry entry)
    {
        double result;
        bool isValid = Double.TryParse(entry.Text, out result);
        entry.TextColor = isValid ? Colors.Black : Colors.Red;
    }
}
```

⚠ Warning

Be careful when sharing triggers in a [ResourceDictionary](#). One instance will be shared among controls so any state that is configured once will apply to them all.

Multi-triggers

A [MultiTrigger](#) represents a trigger that applies property values, or performs actions, when a set of conditions are satisfied. All the conditions must be true before the [Setter](#) objects are applied.

The following example shows a [MultiTrigger](#) that binds to two [Entry](#) objects:

XAML

```
<Entry x:Name="email"
      Text="" />
<Entry x:Name="phone"
      Text="" />
<Button Text="Save">
    <Button.Triggers>
        <MultiTrigger TargetType="Button">
            <MultiTrigger.Conditions>
                <BindingCondition Binding="{Binding Source={x:Reference
email}},
                                Path=Text.Length}"
```

```

                                Value="0" />
        <BindingCondition Binding="{Binding Source={x:Reference
phone},
                                Path=Text.Length}"
                                Value="0" />
    </MultiTrigger.Conditions>
    <Setter Property="IsEnabled" Value="False" />
    <!-- multiple Setter elements are allowed -->
</MultiTrigger>
</Button.Triggers>
</Button>

```

In addition, the `MultiTrigger.Conditions` collection can also contain [PropertyCondition](#) objects:

XAML

```

<PropertyCondition Property="Text"
    Value="OK" />

```

EnterActions and ExitActions

An alternative approach to implementing changes when a trigger occurs is by specifying [EnterActions](#) and [ExitActions](#) collections, and creating [TriggerAction<T>](#) implementations.

The [EnterActions](#) collection, of type `IList<TriggerAction>`, defines a collection that will be invoked when the trigger condition is met. The [ExitActions](#) collection, of type `IList<TriggerAction>`, defines a collection that will be invoked after the trigger condition is no longer met.

⚠ Note

The [TriggerAction](#) objects defined in the [EnterActions](#) and [ExitActions](#) collections are ignored by the [EventTrigger](#) class.

The following example shows a property trigger that specifies an `EnterAction` and an `ExitAction`:

XAML

```

<Entry Placeholder="Enter job title">
    <Entry.Triggers>
        <Trigger TargetType="Entry"

```

```

        Property="Entry.IsFocused"
        Value="True">
        <Trigger.EnterActions>
            <local:FadeTriggerAction StartsFrom="0" />
        </Trigger.EnterActions>

        <Trigger.ExitActions>
            <local:FadeTriggerAction StartsFrom="1" />
        </Trigger.ExitActions>
    </Trigger>
</Entry.Triggers>
</Entry>

```

A trigger action implementation must:

- Implement the generic `TriggerAction<T>` class, with the generic parameter corresponding with the type of control the trigger will be applied to. You can use classes such as `VisualElement` to write trigger actions that work with a variety of controls, or specify a control type like `Entry`.
- Override the `Invoke` method. This method is called whenever the trigger event occurs.
- Optionally expose properties that can be set in XAML when the trigger is declared.

The following example shows the `FadeTriggerAction` class:

```

C#

public class FadeTriggerAction : TriggerAction<VisualElement>
{
    public int StartsFrom { get; set; }

    protected override void Invoke(VisualElement sender)
    {
        sender.Animate("FadeTriggerAction", new Animation((d) =>
        {
            var val = StartsFrom == 1 ? d : 1 - d;
            sender.BackgroundColor = Color.FromRgb(1, val, 1);
        }),
            length: 1000, // milliseconds
            easing: Easing.Linear);
    }
}

```

ⓘ Note

You can provide [EnterActions](#) and [ExitActions](#) as well as [Setter](#) objects in a trigger, but be aware that the [Setter](#) objects are called immediately (they do not wait for the [EnterAction](#) or [ExitAction](#) to complete).

State triggers

State triggers are a specialized group of triggers that define the conditions under which a [VisualState](#) should be applied.

State triggers are added to the [StateTriggers](#) collection of a [VisualState](#). This collection can contain a single state trigger, or multiple state triggers. A [VisualState](#) will be applied when any state triggers in the collection are active.

When using state triggers to control visual states, .NET MAUI uses the following precedence rules to determine which trigger (and corresponding [VisualState](#)) will be active:

1. Any trigger that derives from [StateTriggerBase](#).
2. An [AdaptiveTrigger](#) activated due to the [MinWindowWidth](#) condition being met.
3. An [AdaptiveTrigger](#) activated due to the [MinWindowHeight](#) condition being met.

If multiple triggers are simultaneously active (for example, two custom triggers) then the first trigger declared in the markup takes precedence.

ⓘ Note

State triggers can be set in a [Style](#), or directly on elements.

For more information about visual states, see [Visual states](#).

State trigger

The [StateTrigger](#) class, which derives from the [StateTriggerBase](#) class, has an [IsActive](#) bindable property. A [StateTrigger](#) triggers a [VisualState](#) change when the [IsActive](#) property changes value.

The [StateTriggerBase](#) class, which is the base class for all state triggers, has an [IsActive](#) property and an [IsActiveChanged](#) event. This event fires whenever a [VisualState](#) change occurs. In addition, the [StateTriggerBase](#) class has overridable [OnAttached](#) and [OnDetached](#) methods.

❗ Important

The `StateTrigger.IsActive` bindable property hides the inherited `StateTriggerBase.IsActive` property.

The following XAML example shows a [Style](#) that includes [StateTrigger](#) objects:

XAML

```
<Style TargetType="Grid">
    <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
            <VisualStateGroup>
                <VisualState x:Name="Checked">
                    <VisualState.StateTriggers>
                        <StateTrigger IsActive="{Binding IsToggled}"
IsActiveChanged="OnCheckedStateIsActiveChanged" />
                    </VisualState.StateTriggers>
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
Value="Black" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState x:Name="Unchecked">
                    <VisualState.StateTriggers>
                        <StateTrigger IsActive="{Binding IsToggled,
Converter={StaticResource inverseBooleanConverter}}"
IsActiveChanged="OnUncheckedStateIsActiveChanged" />
                    </VisualState.StateTriggers>
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
Value="White" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </Setter>
</Style>
```

In this example, the implicit [Style](#) targets [Grid](#) objects. When the `IsToggled` property of the bound object is `true`, the background color of the [Grid](#) is set to black. When the `IsToggled` property of the bound object becomes `false`, a [VisualState](#) change is triggered, and the background color of the [Grid](#) becomes white.

In addition, every time a [VisualState](#) change occurs, the `IsActiveChanged` event for the [VisualState](#) is raised. Each [VisualState](#) registers an event handler for this event:

C#

```
void OnCheckedStateIsActiveChanged(object sender, EventArgs e)
{
    StateTriggerBase stateTrigger = sender as StateTriggerBase;
    Console.WriteLine($"Checked state active: {stateTrigger.IsActive}");
}

void OnUncheckedStateIsActiveChanged(object sender, EventArgs e)
{
    StateTriggerBase stateTrigger = sender as StateTriggerBase;
    Console.WriteLine($"Unchecked state active: {stateTrigger.IsActive}");
}
```

In this example, when a handler for the `IsActiveChanged` event is raised, the handler outputs whether the `VisualState` is active or not. For example, the following messages are output to the console window when changing from the `Checked` visual state to the `Unchecked` visual state:

```
Checked state active: False
Unchecked state active: True
```

ⓘ Note

Custom state triggers can be created by deriving from the `StateTriggerBase` class, and overriding the `OnAttached` and `OnDetached` methods to perform any required registrations and cleanup.

Adaptive trigger

An `AdaptiveTrigger` triggers a `VisualState` change when the window is a specified height or width. This trigger has two bindable properties:

- `MinWindowHeight`, of type `double`, which indicates the minimum window height at which the `VisualState` should be applied.
- `MinWindowWidth`, of type `double`, which indicates the minimum window width at which the `VisualState` should be applied.

ⓘ Note

The [AdaptiveTrigger](#) derives from the [StateTriggerBase](#) class and can therefore attach an event handler to the `IsActiveChanged` event.

The following XAML example shows a [Style](#) that includes [AdaptiveTrigger](#) objects:

XAML

```
<Style TargetType="StackLayout">
  <Setter Property="VisualStateManager.VisualStateGroups">
    <VisualStateGroupList>
      <VisualStateGroup>
        <VisualState x:Name="Vertical">
          <VisualState.StateTriggers>
            <AdaptiveTrigger MinWindowWidth="0" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="Orientation"
              Value="Vertical" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
      <VisualStateGroup>
        <VisualState x:Name="Horizontal">
          <VisualState.StateTriggers>
            <AdaptiveTrigger MinWindowWidth="800" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="Orientation"
              Value="Horizontal" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
    </VisualStateGroupList>
  </Setter>
</Style>
```

In this example, the implicit [Style](#) targets [StackLayout](#) objects. When the window width is between 0 and 800 device-independent units, [StackLayout](#) objects to which the [Style](#) is applied will have a vertical orientation. When the window width is ≥ 800 device-independent units, the [VisualState](#) change is triggered, and the [StackLayout](#) orientation changes to horizontal.

The [MinWindowHeight](#) and [MinWindowWidth](#) properties can be used independently or in conjunction with each other. The following XAML shows an example of setting both properties:

XAML

```
<AdaptiveTrigger MinWindowWidth="800"
  MinWindowHeight="1200"/>
```

In this example, the [AdaptiveTrigger](#) indicates that the corresponding [VisualState](#) will be applied when the current window width is ≥ 800 device-independent units and the current window height is ≥ 1200 device-independent units.

Compare state trigger

The [CompareStateTrigger](#) triggers a [VisualState](#) change when a property is equal to a specific value. This trigger has two bindable properties:

- [Property](#), of type `object`, which indicates the property being compared by the trigger.
- [Value](#), of type `object`, which indicates the value at which the [VisualState](#) should be applied.

ⓘ Note

The [CompareStateTrigger](#) derives from the [StateTriggerBase](#) class and can therefore attach an event handler to the `IsActiveChanged` event.

The following XAML example shows a [Style](#) that includes [CompareStateTrigger](#) objects:

XAML

```
<Style TargetType="Grid">
    <Setter Property="VisualStateManager.VisualStateGroups">
        <VisualStateGroupList>
            <VisualStateGroup>
                <VisualState x:Name="Checked">
                    <VisualState.StateTriggers>
                        <CompareStateTrigger Property="{Binding Source=
{x:Reference checkBox}, Path=IsChecked}"
                                           Value="True" />
                    </VisualState.StateTriggers>
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
                            Value="Black" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState x:Name="Unchecked">
                    <VisualState.StateTriggers>
                        <CompareStateTrigger Property="{Binding Source=
{x:Reference checkBox}, Path=IsChecked}"
                                           Value="False" />
                    </VisualState.StateTriggers>
                    <VisualState.Setters>
                        <Setter Property="BackgroundColor"
                            Value="White" />
                    </VisualState.Setters>
                </VisualState>
            </VisualStateGroup>
        </VisualStateGroupList>
    </Setter>
</Style>
```

```

        </VisualState.Setters>
    </VisualState>
</VisualStateGroup>
</VisualStateGroupList>
</Setter>
</Style>
...
<Grid>
    <Border BackgroundColor="White"
        StrokeShape="RoundRectangle 12"
        Margin="24"
        Padding="24"
        HorizontalOptions="Center"
        VerticalOptions="Center">
        <StackLayout Orientation="Horizontal">
            <CheckBox x:Name="checkBox"
                VerticalOptions="Center" />
            <Label Text="Check the CheckBox to modify the Grid background
color."
                VerticalOptions="Center" />
        </StackLayout>
    </Border>
</Grid>

```

In this example, the implicit [Style](#) targets [Grid](#) objects. When the `IsChecked` property of the [CheckBox](#) is `false`, the background color of the [Grid](#) is set to white. When the `CheckBox.IsChecked` property becomes `true`, a [VisualState](#) change is triggered, and the background color of the [Grid](#) becomes black.

Device state trigger

The [DeviceStateTrigger](#) triggers a [VisualState](#) change based on the device platform the app is running on. This trigger has a single bindable property:

- [Device](#), of type `string`, which indicates the device platform on which the [VisualState](#) should be applied.

ⓘ Note

The [DeviceStateTrigger](#) derives from the [StateTriggerBase](#) class and can therefore attach an event handler to the `IsActiveChanged` event.

The following XAML example shows a [Style](#) that includes [DeviceStateTrigger](#) objects:

XAML

```

<Style x:Key="DeviceStateTriggerPageStyle"
      TargetType="ContentPage">
  <Setter Property="VisualStateManager.VisualStateGroups">
    <VisualStateGroupList>
      <VisualStateGroup>
        <VisualState x:Name="iOS">
          <VisualState.StateTriggers>
            <DeviceStateTrigger Device="iOS" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="BackgroundColor"
                  Value="Silver" />
          </VisualState.Setters>
        </VisualState>
        <VisualState x:Name="Android">
          <VisualState.StateTriggers>
            <DeviceStateTrigger Device="Android" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="BackgroundColor"
                  Value="#2196F3" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
    </VisualStateGroupList>
  </Setter>
</Style>

```

In this example, the explicit [Style](#) targets [ContentPage](#) objects. [ContentPage](#) objects that consume the style set their background color to silver on iOS, and to pale blue on Android.

Orientation state trigger

The [OrientationStateTrigger](#) triggers a [VisualState](#) change when the orientation of the device changes. This trigger has a single bindable property:

- [Orientation](#), of type [DisplayOrientation](#), which indicates the orientation to which the [VisualState](#) should be applied.

ⓘ Note

The `OrientationStateTrigger` derives from the [StateTriggerBase](#) class and can therefore attach an event handler to the `IsActiveChanged` event.

The following XAML example shows a [Style](#) that includes [OrientationStateTrigger](#) objects:

XAML

```
<Style x:Key="OrientationStateTriggerPageStyle"
      TargetType="ContentPage">
  <Setter Property="VisualStateManager.VisualStateGroups">
    <VisualStateGroupList>
      <VisualStateGroup>
        <VisualState x:Name="Portrait">
          <VisualState.StateTriggers>
            <OrientationStateTrigger Orientation="Portrait" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="BackgroundColor"
                    Value="Silver" />
          </VisualState.Setters>
        </VisualState>
        <VisualState x:Name="Landscape">
          <VisualState.StateTriggers>
            <OrientationStateTrigger Orientation="Landscape" />
          </VisualState.StateTriggers>
          <VisualState.Setters>
            <Setter Property="BackgroundColor"
                    Value="White" />
          </VisualState.Setters>
        </VisualState>
      </VisualStateGroup>
    </VisualStateGroupList>
  </Setter>
</Style>
```

In this example, the explicit [Style](#) targets [ContentPage](#) objects. [ContentPage](#) objects that consume the style set their background color to silver when the orientation is portrait, and set their background color to white when the orientation is landscape.

.NET MAUI windows

Article • 08/09/2024

The .NET Multi-platform App UI (.NET MAUI) [Window](#) class provides the ability to create, configure, show, and manage multiple windows.

[Window](#) defines the following properties:

- [FlowDirection](#), of type `FlowDirection`, defines the direction in which the UI element of the window are laid out.
- [Height](#), of type `double`, specifies the height of the window on Windows.
- [MaximumHeight](#), of type `double`, represents the maximum height of the window on desktop platforms. Valid values are between 0 and `double.PositiveInfinity`.
- [MaximumWidth](#), of type `double`, represents the maximum width of the window on desktop platforms. Valid values are between 0 and `double.PositiveInfinity`.
- [MinimumHeight](#), of type `double`, represents the minimum height of the window on desktop platforms. Valid values are between 0 and `double.PositiveInfinity`.
- [MinimumWidth](#), of type `double`, represents the minimum width of the window on desktop platforms. Valid values are between 0 and `double.PositiveInfinity`.
- [Overlays](#), of type `IReadOnlyCollection<IWindowOverlay>`, represents the collection of window overlays.
- [Page](#), of type [Page](#), indicates the page being displayed by the window. This property is the content property of the [Window](#) class, and therefore does not need to be explicitly set.
- [Title](#), of type `string`, represents the title of the window.
- [Width](#), of type `double`, specifies the width of the window on Windows.
- [X](#), of type `double`, specifies the X coordinate of the window on Windows.
- [Y](#), of type `double`, specifies the Y coordinate of the window on Windows.

These properties, with the exception of the `overlays` property, are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [Window](#) class defines the following events:

- [Created](#), which is raised when the window is created.
- [Resumed](#), which is raised when the window is resumed from a sleeping state.
- [Activated](#), which is raised when the window is activated.
- [Deactivated](#), which is raised when the window is deactivated.
- [Stopped](#), which is raised when the window is stopped.

- [Destroying](#), which is raised when the window is destroyed.
- [SizeChanged](#), which is raised on desktop platforms when the window changes size.
- [Backgrounding](#), with an accompanying `BackgroundingEventArgs` object, which is raised on iOS and Mac Catalyst when the window is closed or enters a background state. This event can be used to persist any `string` state to the `State` property of the `BackgroundingEventArgs` object, which the OS will preserve until it's time to resume the window. When the window is resumed the state is provided via the `IActivationState` argument to the `CreateWindow` method.
- [DisplayDensityChanged](#), with an accompanying `DisplayDensityChangedEventArgs` object, which is raised on Android and Windows when the effective dots per inch (DPI) for the window has changed.

For more information about the lifecycle events, and their associated overrides, see [App lifecycle](#).

The [Window](#) class also defines the following modal navigation events:

- [ModalPopped](#), with `ModalPoppedEventArgs`, which is raised when a view has been popped modally.
- [ModalPopping](#), with `ModalPoppingEventArgs`, which is raised when a view is modally popped.
- [ModalPushed](#), with `ModalPushedEventArgs`, which is raised after a view has been pushed modally.
- [ModalPushing](#), with `ModalPushingEventArgs`, which is raised when a view is modally pushed.
- [PopCanceled](#), which is raised when a modal pop is cancelled.

The [VisualElement](#) class has a `Window` property that exposes the parent [Window](#) object. This property can be accessed from any page, layout, or view, to manipulate [Window](#) objects.

Create a Window

By default, your .NET MAUI app overrides the `CreateWindow` method in your `App` class to create a [Window](#) object:

```
C#  
  
namespace MyMauiApp  
{  
    public partial class App : Application  
    {  
        public App()  
    }  
}
```

```

    {
        InitializeComponent();
    }

    protected override Window CreateWindow(IActivationState?
activationState)
    {
        return new Window(new AppShell());
    }
}

```

The [Window](#) class has a default constructor and a constructor that accepts a [Page](#) argument, which represents the root page of the app.

In addition, you can also create your own [Window](#)-derived object:

```

C#

namespace MyMauiApp
{
    public class MyWindow : Window
    {
        public MyWindow() : base()
        {
        }

        public MyWindow(Page page) : base(page)
        {
        }

        // Override Window methods
    }
}

```

The [Window](#)-derived class can then be consumed by creating a `MyWindow` object in the `CreateWindow` override in your `App` class.

Regardless of how your [Window](#) object is created, it will be the parent of the root page in your app.

Multi-window support

Multiple windows can be simultaneously opened on Android, iOS on iPad (iPadOS), Mac Catalyst, and Windows. This can be achieved by creating a [Window](#) object and opening it using the `OpenWindow` method on the `Application` object:

C#

```
Window secondWindow = new Window(new MyPage());  
Application.Current?.OpenWindow(secondWindow);
```

The `Application.Current.Windows` collection, of type `ReadOnlyList<Window>` maintains references to all `Window` objects that are registered with the `Application` object.

A specific window can be brought to the front on Mac Catalyst and Windows with the `Application.Current.ActivateWindow` method:

C#

```
Application.Current?.ActivateWindow(secondWindow);
```

Windows can be closed with the `Application.Current.CloseWindow` method:

C#

```
// Close a specific window  
Application.Current?.CloseWindow(secondWindow);  
  
// Close the active window  
Application.Current?.CloseWindow(GetParentWindow());
```

Important

Multi-window support works on Windows without additional configuration. However, additional configuration is required on Android, iPadOS and Mac Catalyst.

Android configuration

To use multi-window support on Android, you must change the `MainActivity` launch mode in *Platforms > Android > MainActivity.cs* from `LaunchMode.SingleTop` to `LaunchMode.Multiple`:

C#

```
using Android.App;  
using Android.Content.PM;  
using Android.OS;  
  
namespace MyMauiApp;
```

```
[Activity(Theme = "@style/Maui.SplashTheme", MainLauncher = true, LaunchMode = LaunchMode.Multiple, ...)]
public class MainActivity : MauiAppCompatActivity
{
}
```

iPadOS and macOS configuration

To use multi-window support on iPadOS and Mac Catalyst, add a class named `SceneDelegate` to the **Platforms > iOS** and **Platforms > MacCatalyst** folders:

C#

```
using Foundation;
using Microsoft.Maui;
using UIKit;

namespace MyMauiApp;

[Register("SceneDelegate")]
public class SceneDelegate : MauiUISceneDelegate
{
}
```

Then, in the XML editor, open the **Platforms > iOS > Info.plist** file and the **Platforms > MacCatalyst > Info.plist** file and add the following XML to the end of each file:

XML

```
<key>UIApplicationSceneManifest</key>
<dict>
  <key>UIApplicationSupportsMultipleScenes</key>
  <true/>
  <key>UISceneConfigurations</key>
  <dict>
    <key>UIWindowSceneSessionRoleApplication</key>
    <array>
      <dict>
        <key>UISceneConfigurationName</key>
        <string>__MAUI_DEFAULT_SCENE_CONFIGURATION__</string>
        <key>UISceneDelegateClassName</key>
        <string>SceneDelegate</string>
      </dict>
    </array>
  </dict>
</dict>
```

Multi-window support doesn't work on iOS for iPhone.

Position and size a Window

The position and size of a window can be programmatically defined for a .NET MAUI app on Windows by setting the `X`, `Y`, `Width`, and `Height` properties on a [Window](#) object.

⚠ Warning

Mac Catalyst doesn't support resizing or repositioning windows programmatically by setting the `X`, `Y`, `Width`, and `Height` properties.

For example, to set the window position and size on launch you should override the `CreateWindow` method in your `App` class and set the `X`, `Y`, `Width`, and `Height` properties on a [Window](#) object:

```
C#

public partial class App : Application
{
    public App()
    {
        InitializeComponent();

        protected override Window CreateWindow(IActivationState activationState)
=>
        new Window(new AppShell())
        {
            Width = 700,
            Height = 500,
            X = 100,
            Y = 100
        };
    }
}
```

Alternatively, a window can be positioned and sized by accessing the `Window` property from any page, layout, or view. For example, the following code shows how to position a window in the center of the screen:

```
C#

// Get display size
var displayInfo = DeviceDisplay.Current.MainDisplayInfo;
```

```
// Center the window
Window.X = (displayInfo.Width / displayInfo.Density - Window.Width) / 2;
Window.Y = (displayInfo.Height / displayInfo.Density - Window.Height) / 2;
```

For information about obtaining the device's screen metrics, see [Device display information](#).

Mac Catalyst

Mac Catalyst doesn't support resizing or repositioning windows programmatically. However, a workaround to enable resizing is to set the `MinimumWidth` and `MaximumWidth` properties to the desired width of the window, and the `MinimumHeight` and `MaximumHeight` properties to the desired height of the window. This will trigger a resize, and you can then revert the properties back to their original values:

C#

```
Window.MinimumWidth = 700;
Window.MaximumWidth = 700;
Window.MinimumHeight = 500;
Window.MaximumHeight = 500;

// Give the Window time to resize
Dispatcher.Dispatch(() =>
{
    Window.MinimumWidth = 0;
    Window.MinimumHeight = 0;
    Window.MaximumWidth = double.PositiveInfinity;
    Window.MaximumHeight = double.PositiveInfinity;
});
```

Decouple window management from the App class

Window management can be decoupled from the `App` class by creating a class that implements the `IWindowCreator` interface, and adding your window management code in the `CreateWindow` method:

C#

```
public class WindowCreator : IWindowCreator
{
    public Window CreateWindow(Application app, IActivationState
activationState)
```

```

{
    var window = new Window(new ContentPage
    {
        Content = new Grid
        {
            new Label
            {
                Text = "Hello from IWindowCreator",
                HorizontalOptions = LayoutOptions.Center,
                VerticalOptions = LayoutOptions.Center
            }
        }
    });

    return window;
}
}

```

Then, in the `MauiProgram` class you should register your window management type as a dependency in the app's service container:

C#

```
builder.Services.AddSingleton<IWindowCreator, WindowCreator>();
```

Important

Ensure that your registration code specifies the `IWindowCreator` interface as well as its concrete type.

Then, ensure that your `App` class doesn't set the `MainPage` property:

C#

```

public partial class App : Application
{
    public App()
    {
        InitializeComponent();
    }
}

```

Provided that the `IWindowCreator` interface and its concrete type have been registered with the app's service container, and that the `MainPage` property of the `Application` class isn't set, your registered type will be used to create the `Window`.

Basic animation

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) animation classes target different properties of visual elements, with a typical basic animation progressively changing a property from one value to another over a period of time.

Basic animations can be created with extension methods provided by the [ViewExtensions](#) class, which operate on [VisualElement](#) objects:

- [CancelAnimations](#) cancels any animations.
- [FadeTo](#) animates the [Opacity](#) property of a [VisualElement](#).
- [RelScaleTo](#) applies an animated incremental increase or decrease to the [Scale](#) property of a [VisualElement](#).
- [RotateTo](#) animates the [Rotation](#) property of a [VisualElement](#).
- [RelRotateTo](#) applies an animated incremental increase or decrease to the [Rotation](#) property of a [VisualElement](#).
- [RotateXTo](#) animates the [RotationX](#) property of a [VisualElement](#).
- [RotateYTo](#) animates the [RotationY](#) property of a [VisualElement](#).
- [ScaleTo](#) animates the [Scale](#) property of a [VisualElement](#).
- [ScaleXTo](#) animates the [ScaleX](#) property of a [VisualElement](#).
- [ScaleYTo](#) animates the [ScaleY](#) property of a [VisualElement](#).
- [TranslateTo](#) animates the [TranslationX](#) and [TranslationY](#) properties of a [VisualElement](#).

By default, each animation will take 250 milliseconds. However, a duration for each animation can be specified when creating the animation.

ⓘ Note

The [ViewExtensions](#) class also provides a [LayoutTo](#) extension method. However, this method is intended to be used by layouts to animate transitions between layout states that contain size and position changes.

The animation extension methods in the [ViewExtensions](#) class are all asynchronous and return a `Task<bool>` object. The return value is `false` if the animation completes, and `true` if the animation is cancelled. Therefore, when animation operations are combined with the `await` operator it becomes possible to create sequential animations with subsequent animation methods executing after the previous method has completed. For more information, see [Compound animations](#).

If there's a requirement to let an animation complete in the background, then the `await` operator can be omitted. In this scenario, the animation extension methods will quickly return after initiating the animation, with the animation occurring in the background. This operation can be taken advantage of when creating composite animations. For more information, see [Composite animations](#).

On Android, animations respect the system animation settings:

- If the system's animations are disabled (either by accessibility features or developer features), new animations will jump immediately to their finished state.
- If the device's power saving mode is activated while animations are in progress, the animations will immediately jump to their finished state.
- If the device's animation durations is set to zero (disabled) while animations are in progress and the API version is 33 or greater, the animations will immediately jump to their finished state.

Single animations

Each extension method in the [ViewExtensions](#) class implements a single animation operation that progressively changes a property from one value to another value over a period of time.

Rotation

Rotation is performed with the [RotateTo](#) method, which progressively changes the [Rotation](#) property of an element:

C#

```
await image.RotateTo(360, 2000);  
image.Rotation = 0;
```

In this example, an [Image](#) instance is rotated up to 360 degrees over 2 seconds (2000 milliseconds). The [RotateTo](#) method obtains the current [Rotation](#) property value of the element for the start of the animation, and then rotates from that value to its first argument (360). Once the animation is complete, the image's [Rotation](#) property is reset to 0. This ensures that the [Rotation](#) property doesn't remain at 360 after the animation concludes, which would prevent additional rotations.

📌 **Note**

In addition to the [RotateTo](#) method, there are also [RotateXTo](#) and [RotateYTo](#) methods that animate the `RotationX` and `RotationY` properties, respectively.

Relative rotation

Relative rotation is performed with the [RelRotateTo](#) method, which progressively changes the `Rotation` property of an element:

C#

```
await image.RelRotateTo(360, 2000);
```

In this example, an [Image](#) instance is rotated 360 degrees from its starting position over 2 seconds (2000 milliseconds). The [RelRotateTo](#) method obtains the current `Rotation` property value of the element for the start of the animation, and then rotates from that value to the value plus its first argument (360). This ensures that each animation will always be a 360 degrees rotation from the starting position. Therefore, if a new animation is invoked while an animation is already in progress, it will start from the current position and may end at a position that is not an increment of 360 degrees.

Scaling

Scaling is performed with the [ScaleTo](#) method, which progressively changes the `Scale` property of an element:

C#

```
await image.ScaleTo(2, 2000);
```

In this example, an [Image](#) instance is scaled up to twice its size over 2 seconds (2000 milliseconds). The [ScaleTo](#) method obtains the current `Scale` property value of the element for the start of the animation, and then scales from that value to its first argument. This has the effect of expanding the size of the image to twice its size.

ⓘ Note

In addition to the [ScaleTo](#) method, there are also [ScaleXTo](#) and [ScaleYTo](#) methods that animate the `ScaleX` and `ScaleY` properties, respectively.

Relative scaling

Relative scaling is performed with the [RelScaleTo](#) method, which progressively changes the [Scale](#) property of an element:

C#

```
await image.RelScaleTo(2, 2000);
```

In this example, an [Image](#) instance is scaled up to twice its size over 2 seconds (2000 milliseconds). The [RelScaleTo](#) method obtains the current [Scale](#) property value of the element for the start of the animation, and then scales from that value to the value plus its first argument. This ensures that each animation will always be a scaling of 2 from the starting position.

Scaling and rotation with anchors

The [AnchorX](#) and [AnchorY](#) properties of a visual element set the center of scaling or rotation for the [Rotation](#) and [Scale](#) properties. Therefore, their values also affect the [RotateTo](#) and [ScaleTo](#) methods.

Given an [Image](#) that has been placed at the center of a layout, the following code example demonstrates rotating the image around the center of the layout by setting its [AnchorY](#) property:

C#

```
double radius = Math.Min(absoluteLayout.Width, absoluteLayout.Height) / 2;  
image.AnchorY = radius / image.Height;  
await image.RotateTo(360, 2000);
```

To rotate the [Image](#) instance around the center of the layout, the [AnchorX](#) and [AnchorY](#) properties must be set to values that are relative to the width and height of the [Image](#). In this example, the center of the [Image](#) is defined to be at the center of the layout, and so the default [AnchorX](#) value of 0.5 does not require changing. However, the [AnchorY](#) property is redefined to be a value from the top of the [Image](#) to the center point of the layout. This ensures that the [Image](#) makes a full rotation of 360 degrees around the center point of the layout.

Translation

Translation is performed with the [TranslateTo](#) method, which progressively changes the [TranslationX](#) and [TranslationY](#) properties of an element:

C#

```
await image.TranslateTo(-100, -100, 1000);
```

In this example, the [Image](#) instance is translated horizontally and vertically over 1 second (1000 milliseconds). The [TranslateTo](#) method simultaneously translates the image 100 device-independent units to the left, and 100 device-independent units upwards. This is because the first and second arguments are both negative numbers. Providing positive numbers would translate the image to the right, and down.

Important

If an element is initially laid out off screen and then translated onto the screen, after translation the element's input layout remains off screen and the user can't interact with it. Therefore, it's recommended that a view should be laid out in its final position, and then any required translations performed.

Fading

Fading is performed with the [FadeTo](#) method, which progressively changes the [Opacity](#) property of an element:

C#

```
image.Opacity = 0;  
await image.FadeTo(1, 4000);
```

In this example, the [Image](#) instance fades in over 4 seconds (4000 milliseconds). The [FadeTo](#) method obtains the current [Opacity](#) property value of the element for the start of the animation, and then fades in from that value to its first argument.

Compound animations

A compound animation is a sequential combination of animations, and can be created with the `await` operator:

C#

```
await image.TranslateTo(-100, 0, 1000);    // Move image left
await image.TranslateTo(-100, -100, 1000); // Move image diagonally up and
left
await image.TranslateTo(100, 100, 2000);    // Move image diagonally down and
right
await image.TranslateTo(0, 100, 1000);      // Move image left
await image.TranslateTo(0, 0, 1000);        // Move image up
```

In this example, the `Image` instance is translated over 6 seconds (6000 milliseconds). The translation of the `Image` uses five animations, with the `await` operator indicating that each animation executes sequentially. Therefore, subsequent animation methods execute after the previous method has completed.

Composite animations

A composite animation is a combination of animations where two or more animations run simultaneously. Composite animations can be created by combining awaited and non-awaited animations:

C#

```
image.RotateTo(360, 4000);
await image.ScaleTo(2, 2000);
await image.ScaleTo(1, 2000);
```

In this example, the `Image` instance is scaled and simultaneously rotated over 4 seconds (4000 milliseconds). The scaling of the `Image` uses two sequential animations that occur at the same time as the rotation. The `RotateTo` method executes without an `await` operator and returns immediately, with the first `ScaleTo` animation then beginning. The `await` operator on the first `ScaleTo` method delays the second `ScaleTo` method until the first `ScaleTo` method has completed. At this point the `RotateTo` animation is half completed and the `Image` will be rotated 180 degrees. During the final 2 seconds (2000 milliseconds), the second `ScaleTo` animation and the `RotateTo` animation both complete.

Run multiple animations concurrently

The `Task.WhenAny` and `Task.WhenAll` methods can be used to run multiple asynchronous methods concurrently, and therefore can create composite animations. Both methods return a `Task` object and accept a collection of methods that each return a `Task` object. The `Task.WhenAny` method completes when any method in its collection completes execution, as demonstrated in the following code example:

C#

```
await Task.WhenAny<bool>
(
    image.RotateTo(360, 4000),
    image.ScaleTo(2, 2000)
);
await image.ScaleTo(1, 2000);
```

In this example, the `Task.WhenAny` method contains two tasks. The first task rotates an [Image](#) instance over 4 seconds (4000 milliseconds), and the second task scales the image over 2 seconds (2000 milliseconds). When the second task completes, the `Task.WhenAny` method call completes. However, even though the [RotateTo](#) method is still running, the second [ScaleTo](#) method can begin.

The `Task.WhenAll` method completes when all the methods in its collection have completed, as demonstrated in the following code example:

C#

```
// 10 minute animation
uint duration = 10 * 60 * 1000;
await Task.WhenAll
(
    image.RotateTo(307 * 360, duration),
    image.RotateXTo(251 * 360, duration),
    image.RotateYTo(199 * 360, duration)
);
```

In this example, the `Task.WhenAll` method contains three tasks, each of which executes over 10 minutes. Each `Task` makes a different number of 360 degree rotations – 307 rotations for [RotateTo](#), 251 rotations for [RotateXTo](#), and 199 rotations for [RotateYTo](#). These values are prime numbers, therefore ensuring that the rotations aren't synchronized and hence won't result in repetitive patterns.

Canceling animations

An app can cancel one or more animations with a call to the [CancelAnimations](#) extension method:

C#

```
image.CancelAnimations();
```

In this example, all animations that are running on the `Image` instance are immediately canceled.

Easing functions

Article • 09/30/2024

.NET Multi-platform App UI (.NET MAUI) includes an [Easing](#) class that enables you to specify a transfer function that controls how animations speed up or slow down as they're running.

The [Easing](#) class defines a number of easing functions that can be consumed by animations:

- The [BounceIn](#) easing function bounces the animation at the beginning.
- The [BounceOut](#) easing function bounces the animation at the end.
- The [CubicIn](#) easing function slowly accelerates the animation.
- The [CubicInOut](#) easing function accelerates the animation at the beginning, and decelerates the animation at the end.
- The [CubicOut](#) easing function quickly decelerates the animation.
- The [Linear](#) easing function uses a constant velocity, and is the default easing function.
- The [SinIn](#) easing function smoothly accelerates the animation.
- The [SinInOut](#) easing function smoothly accelerates the animation at the beginning, and smoothly decelerates the animation at the end.
- The [SinOut](#) easing function smoothly decelerates the animation.
- The [SpringIn](#) easing function causes the animation to very quickly accelerate towards the end.
- The [SpringOut](#) easing function causes the animation to quickly decelerate towards the end.

The `In` and `Out` suffixes indicate if the effect provided by the easing function is noticeable at the beginning of the animation, at the end, or both.

In addition, custom easing functions can be created. For more information, see [Custom easing functions](#).

Consume an easing function

The animation extension methods in the [ViewExtensions](#) class allow an easing function to be specified as the final method argument:

```
C#
```

```
await image.TranslateTo(0, 200, 2000, Easing.BounceIn);  
await image.ScaleTo(2, 2000, Easing.CubicIn);
```

```
await image.RotateTo(360, 2000, Easing.SinInOut);
await image.ScaleTo(1, 2000, Easing.CubicOut);
await image.TranslateTo(0, -200, 2000, Easing.BounceOut);
```

By specifying an easing function for an animation, the animation velocity becomes non-linear and produces the effect provided by the easing function. Omitting an easing function when creating an animation causes the animation to use the default [Linear](#) easing function, which produces a linear velocity.

For more information about using the animation extension methods in the [ViewExtensions](#) class, see [Basic animation](#). Easing functions can also be consumed by the [Animation](#) class. For more information, see [Custom animation](#).

Custom easing functions

There are three main approaches to creating a custom easing function:

1. Create a method that takes a `double` argument, and returns a `double` result.
2. Create a `Func<double, double>`.
3. Specify the easing function as the argument to the [Easing](#) constructor.

In all three cases, the custom easing function should return a value between 0 and 1.

Custom easing method

A custom easing function can be defined as a method that takes a `double` argument, and returns a `double` result:

```
C#

double CustomEase (double t)
{
    return t == 0 || t == 1 ? t : (int)(5 * t) / 5.0;
}

await image.TranslateTo(0, 200, 2000, (Easing)CustomEase);
```

In this example, the `CustomEase` method truncates the incoming value to the values 0, 0.2, 0.4, 0.6, 0.8, and 1. Therefore, the [Image](#) instance is translated in discrete jumps, rather than smoothly.

Custom easing func

A custom easing function can also be defined as a `Func<double, double>`:

C#

```
Func<double, double> CustomEaseFunc = t => 9 * t * t * t - 13.5 * t * t +  
5.5 * t;  
await image.TranslateTo(0, 200, 2000, CustomEaseFunc);
```

In this example, the `CustomEaseFunc` represents an easing function that starts off fast, slows down and reverses course, and then reverses course again to accelerate quickly towards the end. Therefore, while the overall movement of the `Image` instance is downwards, it also temporarily reverses course halfway through the animation.

Custom easing constructor

A custom easing function can also be defined as the argument to the `Easing` constructor:

C#

```
await image.TranslateTo(0, 200, 2000, new Easing (t => 1 - Math.Cos (10 *  
Math.PI * t) * Math.Exp (-5 * t)));
```

In this example, the custom easing function is specified as a lambda function argument to the `Easing` constructor, and uses the `Math.Cos` method to create a slow drop effect that's dampened by the `Math.Exp` method. Therefore, the `Image` instance is translated so that it appears to drop to its final position.

Custom animation

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Animation](#) class is the building block of all .NET MAUI animations, with the extension methods in the [ViewExtensions](#) class creating one or more [Animation](#) objects.

A number of parameters must be specified when creating an [Animation](#) object, including start and end values of the property being animated, and a callback that changes the value of the property. An [Animation](#) object can also maintain a collection of child animations that can be run and synchronized. For more information, see [Child animations](#).

Running an animation created with the [Animation](#) class, which may or may not include child animations, is achieved by calling the [Commit](#) method. This method specifies the duration of the animation, and amongst other items, a callback that controls whether to repeat the animation.

ⓘ Note

The [Animation](#) class has an [IsEnabled](#) property that can be examined to determine if animations have been disabled by the operating system, such as when power saving mode is activated.

On Android, animations respect the system animation settings:

- If the system's animations are disabled (either by accessibility features or developer features), new animations will jump immediately to their finished state.
- If the device's power saving mode is activated while animations are in progress, the animations will immediately jump to their finished state.
- If the device's animation durations is set to zero (disabled) while animations are in progress and the API version is 33 or greater, the animations will immediately jump to their finished state.

Create an animation

When creating an [Animation](#) object, typically, a minimum of three parameters are required, as demonstrated in the following code example:

```
C#
```

```
var animation = new Animation(v => image.Scale = v, 1, 2);
```

In this example, an animation of the [Scale](#) property of an [Image](#) instance is defined from a value of 1 to a value of 2. The animated value is passed to the callback specified as the first argument, where it's used to change the value of the [Scale](#) property.

The animation is started with a call to the [Commit](#) method:

C#

```
animation.Commit(this, "SimpleAnimation", 16, 2000, Easing.Linear, (v, c) =>
image.Scale = 1, () => true);
```

ⓘ Note

The [Commit](#) method doesn't return a [Task](#) object. Instead, notifications are provided through callback methods.

The following arguments are specified in the [Commit](#) method:

- The first argument ([owner](#)) identifies the owner of the animation. This can be the visual element on which the animation is applied, or another visual element, such as the page.
- The second argument ([name](#)) identifies the animation with a name. The name is combined with the owner to uniquely identify the animation. This unique identification can then be used to determine whether the animation is running ([AnimationIsRunning](#)), or to cancel it ([AbortAnimation](#)).
- The third argument ([rate](#)) indicates the number of milliseconds between each call to the callback method defined in the [Animation](#) constructor.
- The fourth argument ([length](#)) indicates the duration of the animation, in milliseconds.
- The fifth argument ([Easing](#)) defines the easing function to be used in the animation. Alternatively, the easing function can be specified as an argument to the [Animation](#) constructor. For more information about easing functions, see [Easing functions](#).
- The sixth argument ([finished](#)) is a callback that will be executed when the animation has completed. This callback takes two arguments, with the first argument indicating a final value, and the second argument being a [bool](#) that's set to [true](#) if the animation was canceled. Alternatively, the [finished](#) callback can be specified as an argument to the [Animation](#) constructor. However, with a single

animation, if `finished` callbacks are specified in both the [Animation](#) constructor and the [Commit](#) method, only the callback specified in the [Commit](#) method will be executed.

- The seventh argument (`repeat`) is a callback that allows the animation to be repeated. It's called at the end of the animation, and returning `true` indicates that the animation should be repeated.

In the above example, the overall effect is to create an animation that increases the [Scale](#) property of an [Image](#) instance from 1 to 2, over 2 seconds (2000 milliseconds), using the [Linear](#) easing function. Each time the animation completes, its [Scale](#) property is reset to 1 and the animation repeats.

ⓘ Note

Concurrent animations, that run independently of each other can be constructed by creating an [Animation](#) object for each animation, and then calling the [Commit](#) method on each animation.

Child animations

The [Animation](#) class also supports child animations, which are [Animation](#) objects to which other [Animation](#) objects are added as children. This enables a series of animations to be run and synchronized. The following code example demonstrates creating and running child animations:

C#

```
var parentAnimation = new Animation();
var scaleUpAnimation = new Animation(v => image.Scale = v, 1, 2,
Easing.SpringIn);
var rotateAnimation = new Animation(v => image.Rotation = v, 0, 360);
var scaleDownAnimation = new Animation(v => image.Scale = v, 2, 1,
Easing.SpringOut);

parentAnimation.Add(0, 0.5, scaleUpAnimation);
parentAnimation.Add(0, 1, rotateAnimation);
parentAnimation.Add(0.5, 1, scaleDownAnimation);

parentAnimation.Commit(this, "ChildAnimations", 16, 4000, null, (v, c) =>
SetIsEnabledButtonState(true, false));
```

Alternatively, the code example can be written more concisely:

C#

```

new Animation
{
    { 0, 0.5, new Animation (v => image.Scale = v, 1, 2) },
    { 0, 1, new Animation (v => image.Rotation = v, 0, 360) },
    { 0.5, 1, new Animation (v => image.Scale = v, 2, 1) }
}.Commit (this, "ChildAnimations", 16, 4000, null, (v, c) =>
    SetIsEnabledButtonState (true, false));

```

In both examples, a parent [Animation](#) object is created, to which additional [Animation](#) objects are then added. The first two arguments to the [Add](#) method specify when to begin and finish the child animation. The argument values must be between 0 and 1, and represent the relative period within the parent animation that the specified child animation will be active. Therefore, in this example the `scaleUpAnimation` will be active for the first half of the animation, the `scaleDownAnimation` will be active for the second half of the animation, and the `rotateAnimation` will be active for the entire duration.

The overall effect of this example is that the animation occurs over 4 seconds (4000 milliseconds). The `scaleUpAnimation` animates the [Scale](#) property from 1 to 2, over 2 seconds. The `scaleDownAnimation` then animates the [Scale](#) property from 2 to 1, over 2 seconds. While both scale animations are occurring, the `rotateAnimation` animates the [Rotation](#) property from 0 to 360, over 4 seconds. Both scaling animations also use easing functions. The [SpringIn](#) easing function causes the [Image](#) instance to initially shrink before getting larger, and the [SpringOut](#) easing function causes the [Image](#) to become smaller than its actual size towards the end of the complete animation.

There are a number of differences between an [Animation](#) object that uses child animations, and one that doesn't:

- When using child animations, the `finished` callback on a child animation indicates when the child has completed, and the `finished` callback passed to the [Commit](#) method indicates when the entire animation has completed.
- When using child animations, returning `true` from the `repeat` callback on the [Commit](#) method will not cause the animation to repeat, but the animation will continue to run without new values.
- When including an easing function in the [Commit](#) method, and the easing function returns a value greater than 1, the animation will be terminated. If the easing function returns a value less than 0, the value is clamped to 0. To use an easing function that returns a value less than 0 or greater than 1, it must be specified in one of the child animations, rather than in the [Commit](#) method.

The [Animation](#) class also includes [WithConcurrent](#) methods that can be used to add child animations to a parent [Animation](#) object. However, their `begin` and `finish`

argument values aren't restricted to 0 to 1, but only that part of the child animation that corresponds to a range of 0 to 1 will be active. For example, if a [WithConcurrent](#) method call defines a child animation that targets a [Scale](#) property from 1 to 6, but with `begin` and `finish` values of -2 and 3, the `begin` value of -2 corresponds to a [Scale](#) value of 1, and the `finish` value of 3 corresponds to a [Scale](#) value of 6. Because values outside the range of 0 and 1 play no part in an animation, the [Scale](#) property will only be animated from 3 to 6.

Cancel an animation

An app can cancel a custom animation with a call to the [AbortAnimation](#) extension method:

C#

```
this.AbortAnimation ("SimpleAnimation");
```

Because animations are uniquely identified by a combination of the animation owner, and the animation name, the owner and name specified when running the animation must be specified to cancel it. Therefore, this example will immediately cancel the animation named `SimpleAnimation` that's owned by the page.

Create a custom animation

The examples shown here so far have demonstrated animations that could equally be achieved with the methods in the [ViewExtensions](#) class. However, the advantage of the [Animation](#) class is that it has access to the callback method, which is executed when the animated value changes. This allows the callback to implement any desired animation. For example, the following code example animates the [BackgroundColor](#) property of a page by setting it to [Color](#) values created by the `Color.FromHsla` method, with hue values ranging from 0 to 1:

C#

```
new Animation (callback: v => BackgroundColor = Color.FromHsla (v, 1, 0.5),
    start: 0,
    end: 1).Commit (this, "Animation", 16, 4000, Easing.Linear, (v, c) =>
    BackgroundColor = Colors.Black);
```

The resulting animation provides the appearance of advancing the page background through the colors of the rainbow.

Create a custom animation extension method

The extension methods in the [ViewExtensions](#) class animate a property from its current value to a specified value. This makes it difficult to create, for example, a `ColorTo` animation method that can be used to animate a color from one value to another. This is because different controls have different properties of type [Color](#). While the [VisualElement](#) class defines a [BackgroundColor](#) property, this isn't always the desired `Color` property to animate.

The solution to this problem is to not have the `ColorTo` method target a particular `Color` property. Instead, it can be written with a callback method that passes the interpolated [Color](#) value back to the caller. In addition, the method will take start and end [Color](#) arguments.

The `ColorTo` method can be implemented as an extension method that uses the [Animate](#) method in the [AnimationExtensions](#) class to provide its functionality. This is because the [Animate](#) method can be used to target properties that aren't of type `double`, as demonstrated in the following code example:

C#

```
public static class ViewExtensions
{
    public static Task<bool> ColorTo(this VisualElement self, Color
fromColor, Color toColor, Action<Color> callback, uint length = 250, Easing
easing = null)
    {
        Func<double, Color> transform = (t) =>
            Color.FromRgba(fromColor.Red + t * (toColor.Red -
fromColor.Red),
                           fromColor.Green + t * (toColor.Green -
fromColor.Green),
                           fromColor.Blue + t * (toColor.Blue -
fromColor.Blue),
                           fromColor.Alpha + t * (toColor.Alpha -
fromColor.Alpha));
        return ColorAnimation(self, "ColorTo", transform, callback, length,
easing);
    }

    public static void CancelAnimation(this VisualElement self)
    {
        self.AbortAnimation("ColorTo");
    }

    static Task<bool> ColorAnimation(VisualElement element, string name,
Func<double, Color> transform, Action<Color> callback, uint length, Easing
easing)
```

```

    {
        easing = easing ?? Easing.Linear;
        var taskCompletionSource = new TaskCompletionSource<bool>();

        element.Animate<Color>(name, transform, callback, 16, length,
            easing, (v, c) => taskCompletionSource.SetResult(c));
        return taskCompletionSource.Task;
    }
}

```

The `Animate` method requires a `transform` argument, which is a callback method. The input to this callback is always a `double` ranging from 0 to 1. Therefore, in this example the `ColorTo` method defines its own transform `Func` that accepts a `double` ranging from 0 to 1, and that returns a `Color` value corresponding to that value. The `Color` value is calculated by interpolating the `Red`, `Green`, `Blue`, and `Alpha` values of the two supplied `Color` arguments. The `Color` value is then passed to the callback method to be applied to a property. This approach allows the `ColorTo` method to animate any specified `Color` property:

C#

```

await Task.WhenAll(
    label.ColorTo(Colors.Red, Colors.Blue, c => label.TextColor = c, 5000),
    label.ColorTo(Colors.Blue, Colors.Red, c => label.BackgroundColor = c,
        5000));
await this.ColorTo(Color.FromRgb(0, 0, 0), Color.FromRgb(255, 255, 255), c
    => BackgroundColor = c, 5000);
await boxView.ColorTo(Colors.Blue, Colors.Red, c => boxView.Color = c,
    4000);

```

In this code example, the `ColorTo` method animates the `TextColor` and `BackgroundColor` properties of a `Label`, the `BackgroundColor` property of a page, and the `Color` property of a `BoxView`.

Brushes

Article • 09/30/2024

 [Browse the sample](#)

A .NET Multi-platform App UI (.NET MAUI) brush enables you to paint an area, such as the background of a control, using different approaches.

The [Brush](#) class is an abstract class that paints an area with its output. Classes that derive from [Brush](#) describe different ways of painting an area. The following list describes the different brush types available in .NET MAUI:

- [SolidColorBrush](#), which paints an area with a solid color. For more information, see [Solid color brushes](#).
- [LinearGradientBrush](#), which paints an area with a linear gradient. For more information, see [Linear gradient brushes](#).
- [RadialGradientBrush](#), which paints an area with a radial gradient. For more information, see [Radial gradient brushes](#).

Instances of these brush types can be assigned to the `Stroke` and `Fill` properties of a [Shape](#), the `Stroke` property of a [Border](#), the `Brush` property of a [Shadow](#), and the `Background` property of a [VisualElement](#).

Note

The `VisualElement.Background` property enables brushes to be used as the background in any control.

The [Brush](#) class also has an `IsNullOrEmpty` method that returns a `bool` that represents whether the brush is defined or not.

Solid color brushes

Article • 11/27/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [SolidColorBrush](#) class derives from the [Brush](#) class, and is used to paint an area with a solid color. There are a variety of approaches to specifying the color of a [SolidColorBrush](#). For example, you can specify its color with a [Color](#) value or by using one of the predefined [SolidColorBrush](#) objects provided by the [Brush](#) class.

The [SolidColorBrush](#) class defines the `Color` property, of type [Color](#), which represents the color of the brush. This property is the [ContentProperty](#) of the [SolidColorBrush](#) class, and therefore does not need to be explicitly set from XAML. In addition, this property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

The [SolidColorBrush](#) class also has an `IsEmpty` method that returns a `bool` that represents whether the brush has been assigned a color.

Create a SolidColorBrush

There are three main techniques for creating a [SolidColorBrush](#). You can create a [SolidColorBrush](#) from a [Color](#), use a predefined brush, or create a [SolidColorBrush](#) using hexadecimal notation.

Use a predefined Color

.NET MAUI includes a type converter that creates a [SolidColorBrush](#) from a [Color](#) value. In XAML, this enables a [SolidColorBrush](#) to be created from a predefined [Color](#) value:

XAML

```
<Border Background="DarkBlue"
        Stroke="LightGray"
        StrokeShape="RoundRectangle 12"
        HeightRequest="120"
        WidthRequest="120" />
```

In this example, the background of the [Border](#) is painted with a dark blue [SolidColorBrush](#):



Alternatively, the [Color](#) value can be specified using property tag syntax:

XAML

```
<Border Stroke="LightGray"
        StrokeShape="RoundRectangle 12"
        HeightRequest="120"
        WidthRequest="120">
    <Border.Background>
        <SolidColorBrush Color="DarkBlue" />
    </Border.Background>
</Border>
```

In this example, the background of the [Border](#) is painted with a [SolidColorBrush](#) whose color is specified by setting the `SolidColorBrush.Color` property.

Use a predefined Brush

The [Brush](#) class defines a set of commonly used [SolidColorBrush](#) objects. The following example uses one of these predefined [SolidColorBrush](#) objects:

XAML

```
<Border Background="{x:Static Brush.Indigo}"
        Stroke="LightGray"
        StrokeShape="RoundRectangle 12"
        HeightRequest="120"
        WidthRequest="120" />
```

The equivalent C# code is:

C#

```
Border border = new Border
{
    Background = Brush.Indigo,
    Stroke = Colors.LightGray,
```

```
// ...  
};
```

In this example, the background of the [Border](#) is painted with an indigo [SolidColorBrush](#):



For a list of predefined [SolidColorBrush](#) objects provided by the [Brush](#) class, see [Solid color brushes](#).

Use hexadecimal notation

[SolidColorBrush](#) objects can also be created using hexadecimal notation. With this approach, a color is specified in terms of the amount of red, green, and blue to combine into a single color. The main format for specifying a color using hexadecimal notation is `#rrggbb`, where:

- `rr` is a two-digit hexadecimal number specifying the relative amount of red.
- `gg` is a two-digit hexadecimal number specifying the relative amount of green.
- `bb` is a two-digit hexadecimal number specifying the relative amount of blue.

In addition, a color can be specified as `#aarrggbb` where `aa` specifies the alpha value, or transparency, of the color. This approach enables you to create colors that are partially transparent.

The following example sets the color value of a [SolidColorBrush](#) using hexadecimal notation:

XAML

```
<Border Background="#FF9988"  
        Stroke="LightGray"  
        StrokeShape="RoundRectangle 12"  
        HeightRequest="120"  
        WidthRequest="120" />
```

In this example, the background of the `Border` is painted with a salmon-colored `SolidColorBrush`:

For other ways of describing color, see [Colors](#).

Solid color brushes

For convenience, the `Brush` class provides a set of commonly used `SolidColorBrush` objects, such as `AliceBlue` and `YellowGreen`. The following image shows the color of each predefined brush, its name, and its hexadecimal value:

AliceBlue	#FF6F9F	DarkGreen	#F0F0A0	Gray	#F8B080	LightYellow	#FFFFE0	OrangeRed	#FF4500	SlateGray	#7F7B80
AntiqueWhite	#FFFAE0	DarkKhaki	#F0D0B0	Gray	#F0D0B0	Light	#FFD700	Orchid	#FFD700	Snow	#FFFAFA
Aqua	#F0FFFF	DarkMagenta	#F08080	GreenYellow	#FADF2F	LightGreen	#F3D23D	PaleGoldenrod	#FEEEA8	SpringGreen	#F0E67F
Aquamarine	#7FFFD4	DarkOliveGreen	#556B2F	Honeydew	#F0FF00	LightGreen	#FAF0E6	PaleTurquoise	#FAF0E6	SteelBlue	#4682B4
Azure	#F0F0FF	DarkOrange	#FF5500	HotPink	#FF69B4	Magenta	#FFD700	PaleTurquoise	#FAF0E6	Tan	#F0D08C
Beige	#FFF5DC	DarkOrchid	#FF5500	IndianRed	#FFC0CB	Maroon	#FF6900	PaleVioletRed	#F8BBD0	Teal	#F08080
Bisque	#FFFAE4	DarkRed	#8B0000	Indigo	#483D8B	MediumAquamarine	#F08000	PapayaWhip	#FFDAB9	Thistle	#D8BFD8
Black	#FF0000	DarkSalmon	#FF69B4	Ivory	#FFFFFF	MediumBlue	#0000CD	PeachPuff	#FFDAB9	Tomato	#FF6347
BlanchedAlmond	#FFEBD3	DarkSeaGreen	#8FBC8F	Khaki	#F0E68C	MediumOrchid	#F8BBD0	Peru	#CD853F	Transparent	#0000FF
Blue	#0000FF	DarkSlateBlue	#483D8B	Lavender	#E6E6FA	MediumPurple	#9370DB	Pink	#FFB6C1	Turquoise	#40E0D0
BlueViolet	#6A5ACD	DarkSlateGray	#2F4F4F	LavenderBlush	#FFF0F5	MediumSeaGreen	#3CB371	Plum	#DDA0DD	Violet	#EE82EE
Brown	#A52A2A	DarkTurquoise	#00CED1	LawnGreen	#7FFFD4	MediumSlateBlue	#7B68EE	PowderBlue	#B0E0E6	Wheat	#F5DEB3
BurlyWood	#DEB887	DarkViolet	#8A2BE2	LemonChiffon	#FFFACD	MediumSpringGreen	#00FA9B	Purple	#800080	White	#FFFFFF
CadetBlue	#F5F5EA	DeepPink	#FF1493	LightBlue	#ADD8E6	MediumTurquoise	#4682B4	Red	#FF0000	WhiteSmoke	#F5F5F5
Chartreuse	#7FFD00	DeepSkyBlue	#00BFFF	LightCoral	#F08080	MediumVioletRed	#D81B60	RosyBrown	#FBC02D	Yellow	#FFFF00
Chocolate	#D2691E	DimGray	#696969	LightCyan	#E0FFFF	MidnightBlue	#191970	RoyalBlue	#4169E1	YellowGreen	#9ACD32
Coral	#FF7F50	DodgerBlue	#1E90FF	LightGoldenrodYellow	#FFFAD2	MintCream	#90EE90	SaddleBrown	#8B4513		
CornflowerBlue	#6495ED	Firebrick	#B22222	LightGray	#D3D3D3	MistyRose	#F5DEB3	Salmon	#FA8072		
Cornsil	#FFFBD0	FloralWhite	#FFFACD	LightGreen	#90EE90	Occasion	#FFD700	SandyBrown	#F4A460		
Crimson	#DC143C	ForestGreen	#228B22	LightPink	#FFB6C1	NauvauWhite	#FFDAB9	SeaGreen	#2F8B57		
Cyan	#00FFFF	Fuchsia	#FF00FF	LightSalmon	#FFDAB9	Navy	#000080	SeaShell	#FFF5EE		
DarkBlue	#00008B	Gainsboro	#F0F0F0	LightSeaGreen	#20B2AA	Oldlace	#F0E68C	Sienna	#A0522D		
DarkCyan	#008B8B	GhostWhite	#F0F0F0	LightSkyBlue	#87CEFA	Olive	#808000	Silver	#C0C0C0		
DarkGoldenrod	#DAA520	Gold	#FFD700	LightSlateGray	#778899	OliveDrab	#6B8E23	SkyBlue	#87CEEB		
DarkGray	#696969	Goldenrod	#DAA520	LightSteelBlue	#B0C4DE	Orange	#FFA500	SlateBlue	#6A5ACD		

Gradients

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [GradientBrush](#) class derives from the [Brush](#) class, and is an abstract class that describes a gradient, which is composed of gradient stops. A gradient brush paints an area with multiple colors that blend into each other along an axis.

Classes that derive from [GradientBrush](#) describe different ways of interpreting gradient stops, and .NET MAUI provides the following gradient brushes:

- [LinearGradientBrush](#), which paints an area with a linear gradient. For more information, see [Linear gradient brushes](#).
- [RadialGradientBrush](#), which paints an area with a radial gradient. For more information, see [Radial gradient brushes](#).

The [GradientBrush](#) class defines the [GradientStops](#) property, of type [GradientStopCollection](#), which represents the brush's gradient stops, each of which specifies a color and an offset along the brush's gradient axis. A [GradientStopCollection](#) is an `ObservableCollection` of [GradientStop](#) objects. The [GradientStops](#) property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings, and styled.

Note

The [GradientStops](#) property is the [ContentProperty](#) of the [GradientBrush](#) class, and so does not need to be explicitly set from XAML.

Gradient stops

Gradient stops are the building blocks of a gradient brush, and specify the colors in the gradient and their location along the gradient axis. Gradient stops are specified using [GradientStop](#) objects.

The [GradientStop](#) class defines the following properties:

- `Color`, of type [Color](#), which represents the color of the gradient stop.

- `Offset`, of type `float`, which represents the location of the gradient stop within the gradient vector. The default value of this property is 0, and valid values are in the range 0.0-1.0. The closer this value is to 0, the closer the color is to the start of the gradient. Similarly, the closer this value is to 1, the closer the color is to the end of the gradient.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

Important

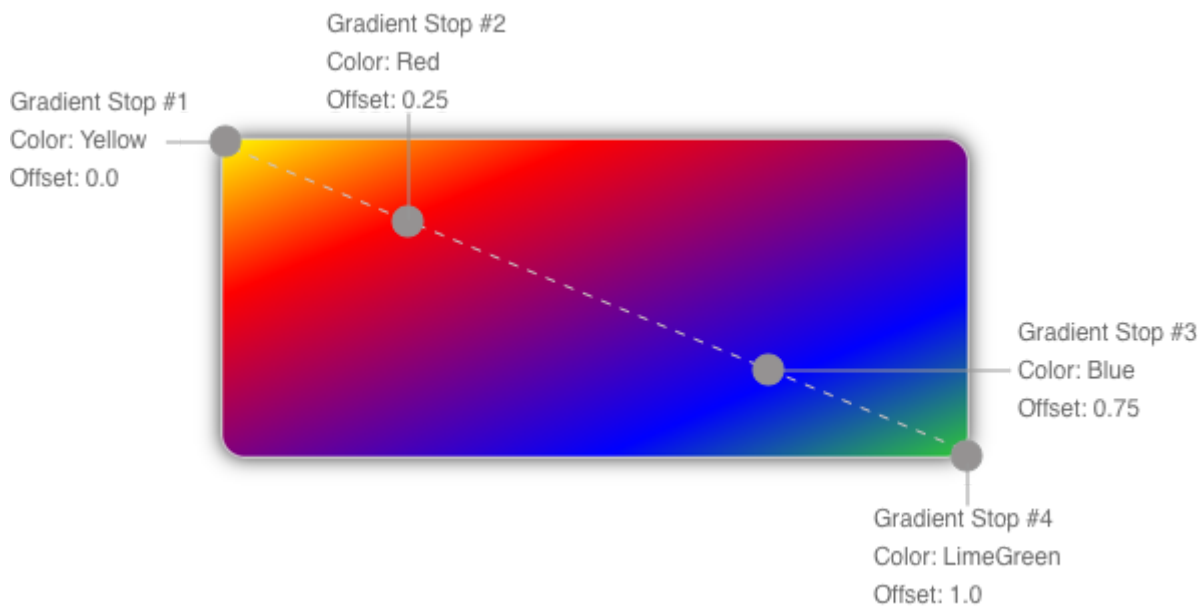
The coordinate system used by gradients is relative to a bounding box for the output area. 0 indicates 0 percent of the bounding box, and 1 indicates 100 percent of the bounding box. Therefore, (0.5,0.5) describes a point in the middle of the bounding box, and (1,1) describes a point at the bottom right of the bounding box.

The following XAML example creates a diagonal [LinearGradientBrush](#) with four colors:

XAML

```
<LinearGradientBrush StartPoint="0,0"
    EndPoint="1,1">
    <GradientStop Color="Yellow"
        Offset="0.0" />
    <GradientStop Color="Red"
        Offset="0.25" />
    <GradientStop Color="Blue"
        Offset="0.75" />
    <GradientStop Color="LimeGreen"
        Offset="1.0" />
</LinearGradientBrush>
```

The color of each point between gradient stops is interpolated as a combination of the color specified by the two bounding gradient stops. The following diagram shows the gradient stops from the previous example:



In this diagram, the circles mark the position of gradient stops, and the dashed line shows the gradient axis. The first gradient stop specifies the color yellow at an offset of 0.0. The second gradient stop specifies the color red at an offset of 0.25. The points between these two gradient stops gradually change from yellow to red as you move from left to right along the gradient axis. The third gradient stop specifies the color blue at an offset of 0.75. The points between the second and third gradient stops gradually change from red to blue. The fourth gradient stop specifies the color lime green at an offset of 1.0. The points between the third and fourth gradient stops gradually change from blue to lime green.

Linear gradient brushes

Article • 09/30/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [LinearGradientBrush](#) class derives from the [GradientBrush](#) class, and paints an area with a linear gradient, which blends two or more colors along a line known as the gradient axis. [GradientStop](#) objects are used to specify the colors in the gradient and their positions. For more information about [GradientStop](#) objects, see [Gradients](#).

The [LinearGradientBrush](#) class defines the following properties:

- `StartPoint`, of type `Point`, which represents the starting two-dimensional coordinates of the linear gradient. The default value of this property is (0,0).
- `EndPoint`, of type `Point`, which represents the ending two-dimensional coordinates of the linear gradient. The default value of this property is (1,1).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [LinearGradientBrush](#) class also has an `IsEmpty` method that returns a `bool` that represents whether the brush has been assigned any [GradientStop](#) objects.

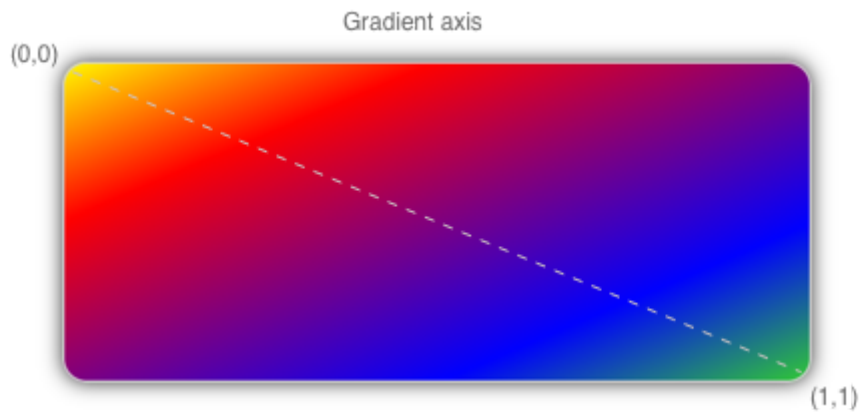
ⓘ Note

Linear gradients can also be created with the `linear-gradient()` CSS function.

Create a LinearGradientBrush

A linear gradient brush's gradient stops are positioned along the gradient axis. The orientation and size of the gradient axis can be changed using the brush's `StartPoint` and `EndPoint` properties. By manipulating these properties, you can create horizontal, vertical, and diagonal gradients, reverse the gradient direction, condense the gradient spread, and more.

The `StartPoint` and `EndPoint` properties are relative to the area being painted. (0,0) represents the top-left corner of the area being painted, and (1,1) represents the bottom-right corner of the area being painted. The following diagram shows the gradient axis for a diagonal linear gradient brush:



In this diagram, the dashed line shows the gradient axis, which highlights the interpolation path of the gradient from the start point to the end point.

Create a horizontal linear gradient

To create a horizontal linear gradient, create a [LinearGradientBrush](#) object and set its `StartPoint` to (0,0) and its `EndPoint` to (1,0). Then, add two or more [GradientStop](#) objects to the `LinearGradientBrush.GradientStops` collection, that specify the colors in the gradient and their positions.

The following XAML example shows a horizontal [LinearGradientBrush](#) that's set as the `Background` of a [Frame](#):

XAML

```
<Frame BorderColor="LightGray"
  HasShadow="True"
  CornerRadius="12"
  HeightRequest="120"
  WidthRequest="120">
  <Frame.Background>
    <!-- StartPoint defaults to (0,0) -->
    <LinearGradientBrush EndPoint="1,0">
      <GradientStop Color="Yellow"
        Offset="0.1" />
      <GradientStop Color="Green"
        Offset="1.0" />
    </LinearGradientBrush>
  </Frame.Background>
</Frame>
```

In this example, the background of the [Frame](#) is painted with a [LinearGradientBrush](#) that interpolates from yellow to green horizontally:



Create a vertical linear gradient

To create a vertical linear gradient, create a [LinearGradientBrush](#) object and set its `StartPoint` to (0,0) and its `EndPoint` to (0,1). Then, add two or more [GradientStop](#) objects to the `LinearGradientBrush.GradientStops` collection, that specify the colors in the gradient and their positions.

The following XAML example shows a vertical [LinearGradientBrush](#) that's set as the `Background` of a [Frame](#):

XAML

```
<Frame BorderColor="LightGray"
      HasShadow="True"
      CornerRadius="12"
      HeightRequest="120"
      WidthRequest="120">
  <Frame.Background>
    <!-- StartPoint defaults to (0,0) -->
    <LinearGradientBrush EndPoint="0,1">
      <GradientStop Color="Yellow"
                    Offset="0.1" />
      <GradientStop Color="Green"
                    Offset="1.0" />
    </LinearGradientBrush>
  </Frame.Background>
</Frame>
```

In this example, the background of the [Frame](#) is painted with a [LinearGradientBrush](#) that interpolates from yellow to green vertically:



Create a diagonal linear gradient

To create a diagonal linear gradient, create a [LinearGradientBrush](#) object and set its `StartPoint` to (0,0) and its `EndPoint` to (1,1). Then, add two or more [GradientStop](#) objects to the `LinearGradientBrush.GradientStops` collection, that specify the colors in the gradient and their positions.

The following XAML example shows a diagonal [LinearGradientBrush](#) that's set as the `Background` of a [Frame](#):

XAML

```
<Frame BorderColor="LightGray"
      HasShadow="True"
      CornerRadius="12"
      HeightRequest="120"
      WidthRequest="120">
  <Frame.Background>
    <!-- StartPoint defaults to (0,0)
         Endpoint defaults to (1,1) -->
    <LinearGradientBrush>
      <GradientStop Color="Yellow"
                   Offset="0.1" />
      <GradientStop Color="Green"
                   Offset="1.0" />
    </LinearGradientBrush>
  </Frame.Background>
</Frame>
```

In this example, the background of the [Frame](#) is painted with a [LinearGradientBrush](#) that interpolates from yellow to green diagonally:



Radial gradient brushes

Article • 11/27/2024

 [Browse the sample](#)

The .NET Multi-platform App UI (.NET MAUI) [RadialGradientBrush](#) class derives from the [GradientBrush](#) class, and paints an area with a radial gradient, which blends two or more colors across a circle. [GradientStop](#) objects are used to specify the colors in the gradient and their positions. For more information about [GradientStop](#) objects, see [Gradients](#).

The [RadialGradientBrush](#) class defines the following properties:

- `Center`, of type `Point`, which represents the center point of the circle for the radial gradient. The default value of this property is (0.5,0.5).
- `Radius`, of type `double`, which represents the radius of the circle for the radial gradient. The default value of this property is 0.5.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [RadialGradientBrush](#) class also has an `IsEmpty` method that returns a `bool` that represents whether the brush has been assigned any [GradientStop](#) objects.

Note

Radial gradients can also be created with the `radial-gradient()` CSS function.

Create a RadialGradientBrush

A radial gradient brush's gradient stops are positioned along a gradient axis defined by a circle. The gradient axis radiates from the center of the circle to its circumference. The position and size of the circle can be changed using the brush's `Center` and `Radius` properties. The circle defines the end point of the gradient. Therefore, a gradient stop at 1.0 defines the color at the circle's circumference. A gradient stop at 0.0 defines the color at the center of the circle.

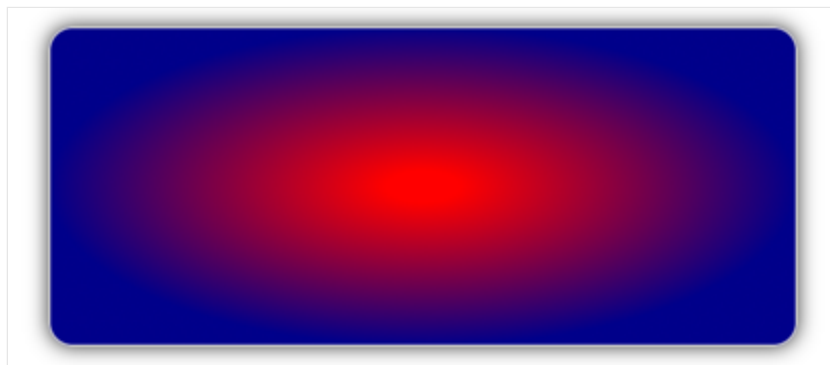
To create a radial gradient, create a [RadialGradientBrush](#) object and set its `Center` and `Radius` properties. Then, add two or more [GradientStop](#) objects to the `RadialGradientBrush.GradientStops` collection, that specify the colors in the gradient and their positions.

The following XAML example shows a [RadialGradientBrush](#) that's set as the `Background` of a [Border](#):

XAML

```
<Border Stroke="LightGray"
        StrokeShape="RoundRectangle 12"
        HeightRequest="120"
        WidthRequest="120">
    <Border.Background>
        <!-- Center defaults to (0.5,0.5)
             Radius defaults to (0.5) -->
        <RadialGradientBrush>
            <GradientStop Color="Red"
                           Offset="0.1" />
            <GradientStop Color="DarkBlue"
                           Offset="1.0" />
        </RadialGradientBrush>
    </Border.Background>
</Border>
```

In this example, the background of the [Border](#) is painted with a [RadialGradientBrush](#) that interpolates from red to dark blue. The center of the radial gradient is positioned in the center of the [Border](#):



The following XAML example moves the center of the radial gradient to the top-left corner of the [Border](#):

XAML

```
<!-- Radius defaults to (0.5) -->
<RadialGradientBrush Center="0.0,0.0">
    <GradientStop Color="Red"
                   Offset="0.1" />
    <GradientStop Color="DarkBlue"
                   Offset="1.0" />
</RadialGradientBrush>
```

In this example, the background of the [Border](#) is painted with a [RadialGradientBrush](#) that interpolates from red to dark blue. The center of the radial gradient is positioned in the top-left of the [Border](#):



The following XAML example moves the center of the radial gradient to the bottom-right corner of the [Border](#):

XAML

```
<!-- Radius defaults to (0.5) -->
<RadialGradientBrush Center="1.0,1.0">
  <GradientStop Color="Red"
    Offset="0.1" />
  <GradientStop Color="DarkBlue"
    Offset="1.0" />
</RadialGradientBrush>
```

In this example, the background of the [Border](#) is painted with a [RadialGradientBrush](#) that interpolates from red to dark blue. The center of the radial gradient is positioned in the bottom-right of the [Border](#):



Controls

Article • 08/30/2024

 [Browse the sample](#)

The user interface of a .NET Multi-platform App UI (.NET MAUI) app is constructed of objects that map to the native controls of each target platform.

The main control groups used to create the user interface of a .NET MAUI app are pages, layouts, and views. A .NET MAUI page generally occupies the full screen or window. The page usually contains a layout, which contains views and possibly other layouts. Pages, layouts, and views derive from the [VisualElement](#) class. This class provides a variety of properties, methods, and events that are useful in derived classes.

 Note

[ListView](#) and [TableView](#) also support the use of cells. Cells are specialized elements used for items in a table, that describe how each item should be rendered.

Pages

.NET MAUI apps consist of one or more pages. A page usually occupies all of the screen, or window, and each page typically contains at least one layout.

.NET MAUI contains the following pages:

 [Expand table](#)

Page	Description
ContentPage	ContentPage displays a single view, and is the most common page type. For more information, see ContentPage .
FlyoutPage	FlyoutPage is a page that manages two related pages of information – a flyout page that presents items, and a detail page that presents details about items on the flyout page. For more information, see FlyoutPage .
NavigationPage	NavigationPage provides a hierarchical navigation experience where you're able to navigate through pages, forwards and backwards, as desired. For more information, see NavigationPage .
TabbedPage	TabbedPage consists of a series of pages that are navigable by tabs across the top or bottom of the page, with each tab loading the page content. For more

Page	Description
	information, see TabbedPage .

Layouts

.NET MAUI layouts are used to compose user-interface controls into visual structures, and each layout typically contains multiple views. Layout classes typically contain logic to set the position and size of child elements.

.NET MAUI contains the following layouts:

 Expand table

Layout	Description
AbsoluteLayout	AbsoluteLayout positions child elements at specific locations relative to its parent. For more information, see AbsoluteLayout .
BindableLayout	BindableLayout enables any layout class to generate its content by binding to a collection of items, with the option to set the appearance of each item. For more information, see BindableLayout .
FlexLayout	FlexLayout enables its children to be stacked or wrapped with different alignment and orientation options. FlexLayout is based on the CSS Flexible Box Layout Module, known as <i>flex layout</i> or <i>flex-box</i> . For more information, see FlexLayout .
Grid	Grid positions its child elements in a grid of rows and columns. For more information, see Grid .
HorizontalStackLayout	HorizontalStackLayout positions child elements in a horizontal stack. For more information, see HorizontalStackLayout .
StackLayout	StackLayout positions child elements in either a vertical or horizontal stack. For more information, see StackLayout .
VerticalStackLayout	VerticalStackLayout positions child elements in a vertical stack. For more information, see VerticalStackLayout .

Views

.NET MAUI views are the UI objects such as labels, buttons, and sliders that are commonly known as *controls* or *widgets* in other environments.

.NET MAUI contains the following views:

View	Description
ActivityIndicator	ActivityIndicator uses an animation to show that the app is engaged in a lengthy activity, without giving any indication of progress. For more information, see ActivityIndicator .
BlazorWebView	BlazorWebView enables you to host a Blazor web app in your .NET MAUI app. For more information, see BlazorWebView .
Border	Border is a container control that draws a border, background, or both, around another control. For more information, see Border .
BoxView	BoxView draws a rectangle or square, of a specified width, height, and color. For more information, see BoxView .
Button	Button displays text and responds to a tap or click that directs an app to carry out a task. For more information, see Button .
CarouselView	CarouselView displays a scrollable list of data items, where users swipe to move through the collection. For more information, see CarouselView .
CheckBox	CheckBox enables you to select a boolean value using a type of button that can either be checked or empty. For more information, see CheckBox .
CollectionView	CollectionView displays a scrollable list of selectable data items, using different layout specifications. For more information, see CollectionView .
ContentView	ContentView is a control that enables the creation of custom, reusable controls. For more information, see ContentView .
DatePicker	DatePicker enables you to select a date with the platform date picker. For more information, see DatePicker .
Editor	Editor enables you to enter and edit multiple lines of text. For more information, see Editor .
Ellipse	Ellipse displays an ellipse or circle. For more information, see Ellipse .
Entry	Entry enables you to enter and edit a single line of text. For more information, see Entry .
Frame	Frame is used to wrap a view or layout with a border that can be configured with color, shadow, and other options. For more information, see Frame .
GraphicsView	GraphicsView is a graphics canvas on which 2D graphics can be drawn using types from the Microsoft.Maui.Graphics namespace. For more information, see GraphicsView .
HybridWebView	HybridWebView enables you to host arbitrary HTML/JS/CSS content in a web view, and enables communication between the code in the web view

View	Description
	(JavaScript) and the code that hosts the web view (C#/.NET). For more information, see HybridWebView .
Image	Image displays an image that can be loaded from a local file, a URI, an embedded resource, or a stream. For more information, see Image .
ImageButton	ImageButton displays an image and responds to a tap or click that direct an app to carry out a task. For more information, see ImageButton .
IndicatorView	IndicatorView displays indicators that represent the number of items in a CarouselView . For more information, see IndicatorView .
Label	Label displays single-line and multi-line text. For more information, see Label .
Line	Line displays a line from a start point to an end point. For more information, see Line .
ListView	ListView displays a scrollable list of selectable data items. For more information, see ListView .
Map	Map displays a map, and requires the Microsoft.Maui.Controls.Maps NuGet package to be installed in your app.
Path	Path display curves and complex shapes. For more information, see Path .
Picker	Picker displays a short list of items, from which an item can be selected. For more information, see Picker .
Polygon	Polygon displays a polygon. For more information, see Polygon .
Polyline	Polyline displays a series of connected straight lines. For more information, see Polyline .
ProgressBar	ProgressBar uses an animation to show that the app is progressing through a lengthy activity. For more information, see ProgressBar .
RadioButton	RadioButton is a type of button that allows the selection of one option from a set. For more information, see RadioButton .
Rectangle	Rectangle displays a rectangle or square. For more information, see Rectangle .
RefreshView	RefreshView is a container control that provides pull-to-refresh functionality for scrollable content. For more information, see RefreshView .
RoundRectangle	RoundRectangle displays a rectangle or square with rounded corners. For more information, see Rectangle .
ScrollView	ScrollView provides scrolling of its content, which is typically a layout. For more information, see ScrollView .

View	Description
SearchBar	SearchBar is a user input control used to initiate a search. For more information, see SearchBar .
Slider	Slider enables you to select a <code>double</code> value from a continuous range. For more information, see Slider .
Stepper	Stepper enables you to select a <code>double</code> value from a range of incremental values. For more information, see Stepper .
SwipeView	SwipeView is a container control that wraps around an item of content, and provides context menu items that are revealed by a swipe gesture. For more information, see SwipeView .
Switch	Switch enables you to select a boolean value using a type of button that can either be on or off. For more information, see Switch .
TableView	TableView displays a table of scrollable items that can be grouped into sections. For more information, see TableView .
TimePicker	TimePicker enables you to select a time with the platform time picker. For more information, see TimePicker .
TitleBar	TitleBar enables you to add a custom title bar to a Window to match the personality of your app. For more information, see TitleBar .
TwoPaneView	TwoPaneView represents a container with two views that size and position their content in the available space, either side-by-side or top-to-bottom. For more information, see TwoPaneView .
WebView	WebView displays web pages or local HTML content. For more information, see WebView .

Align and position .NET MAUI controls

Article • 09/30/2024

Every .NET Multi-platform App UI (.NET MAUI) control that derives from [View](#), which includes views and layouts, has `HorizontalOptions` and `VerticalOptions` properties, of type `LayoutOptions`. The `LayoutOptions` structure encapsulates a view's preferred alignment, which determines its position and size within its parent layout when the parent layout contains unused space (that is, the parent layout is larger than the combined size of all its children).

In addition, the `Margin` and `Padding` properties position controls relative to adjacent, or child controls. For more information, see [Position controls](#).

Align views in layouts

The alignment of a [View](#), relative to its parent, can be controlled by setting the `HorizontalOptions` or `VerticalOptions` property of the [View](#) to one of the public fields from the `LayoutOptions` structure. The public fields are `Start`, `Center`, `End`, and `Fill`.

The `Start`, `Center`, `End`, and `Fill` fields are used to define the view's alignment within the parent layout:

- For horizontal alignment, `Start` positions the [View](#) on the left hand side of the parent layout, and for vertical alignment, it positions the [View](#) at the top of the parent layout.
- For horizontal and vertical alignment, `Center` horizontally or vertically centers the [View](#).
- For horizontal alignment, `End` positions the [View](#) on the right hand side of the parent layout, and for vertical alignment, it positions the [View](#) at the bottom of the parent layout.
- For horizontal alignment, `Fill` ensures that the [View](#) fills the width of the parent layout, and for vertical alignment, it ensures that the [View](#) fills the height of the parent layout.

📌 Note

The default value of a view's `HorizontalOptions` and `VerticalOptions` properties is `LayoutOptions.Fill`.

A [StackLayout](#) only respects the `Start`, `Center`, `End`, and `Fill` `LayoutOptions` fields on child views that are in the opposite direction to the [StackLayout](#) orientation. Therefore, child views within a vertically oriented [StackLayout](#) can set their `HorizontalOptions` properties to one of the `Start`, `Center`, `End`, or `Fill` fields. Similarly, child views within a horizontally oriented [StackLayout](#) can set their `VerticalOptions` properties to one of the `Start`, `Center`, `End`, or `Fill` fields.

A [StackLayout](#) does not respect the `Start`, `Center`, `End`, and `Fill` `LayoutOptions` fields on child views that are in the same direction as the [StackLayout](#) orientation. Therefore, a vertically oriented [StackLayout](#) ignores the `Start`, `Center`, `End`, or `Fill` fields if they are set on the `VerticalOptions` properties of child views. Similarly, a horizontally oriented [StackLayout](#) ignores the `Start`, `Center`, `End`, or `Fill` fields if they are set on the `HorizontalOptions` properties of child views.

❗ Important

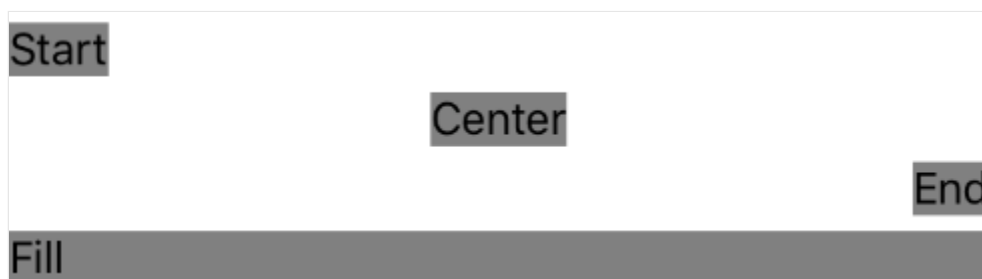
`LayoutOptions.Fill` generally overrides size requests specified using the [HeightRequest](#) and [WidthRequest](#) properties.

The following XAML example demonstrates a vertically oriented [StackLayout](#) where each child [Label](#) sets its `HorizontalOptions` property to one of the four alignment fields from the `LayoutOptions` structure:

XAML

```
<StackLayout>
  ...
  <Label Text="Start" BackgroundColor="Gray" HorizontalOptions="Start" />
  <Label Text="Center" BackgroundColor="Gray" HorizontalOptions="Center" />
  <Label Text="End" BackgroundColor="Gray" HorizontalOptions="End" />
  <Label Text="Fill" BackgroundColor="Gray" HorizontalOptions="Fill" />
</StackLayout>
```

The following screenshot shows the resulting alignment of each [Label](#):

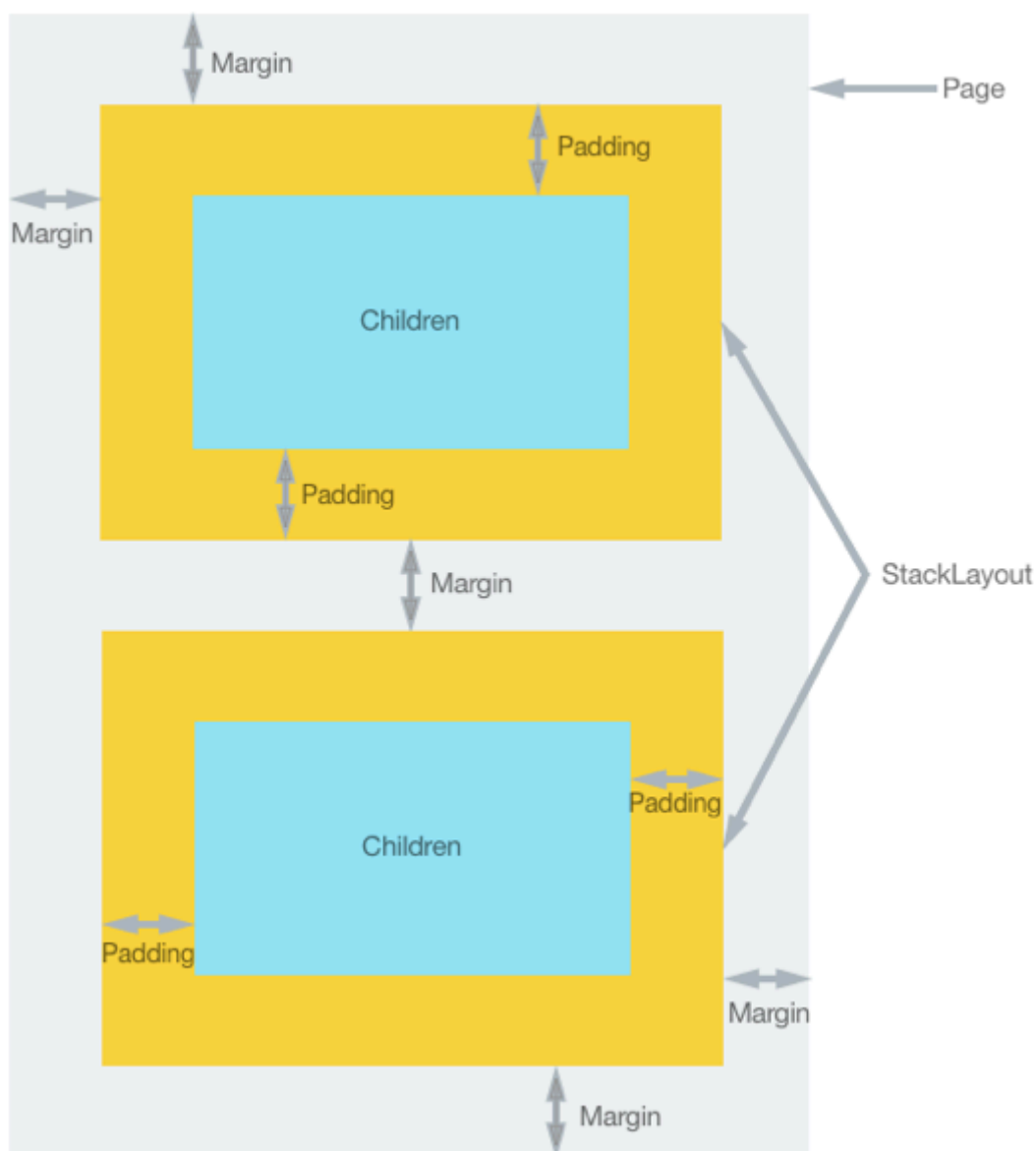


Position controls

The `Margin` and `Padding` properties position controls relative to adjacent, or child controls. Margin and padding are related layout concepts:

- The `Margin` property represents the distance between an element and its adjacent elements, and is used to control the element's rendering position, and the rendering position of its neighbors. `Margin` values can be specified on layouts and views.
- The `Padding` property represents the distance between an element and its child elements, and is used to separate the control from its own content. `Padding` values can be specified on pages, layouts, and views.

The following diagram illustrates the two concepts:



Margin values are additive. Therefore, if two adjacent elements specify a margin of 20 device-independent units, the distance between the elements will be 40 device-independent units. In addition, margin and padding values are additive when both are applied, in that the distance between an element and any content will be the margin plus padding.

The **Margin** and **Padding** properties are both of type **Thickness**. There are three possibilities when creating a **Thickness** structure:

- Create a **Thickness** structure defined by a single uniform value. The single value is applied to the left, top, right, and bottom sides of the element.
- Create a **Thickness** structure defined by horizontal and vertical values. The horizontal value is symmetrically applied to the left and right sides of the element, with the vertical value being symmetrically applied to the top and bottom sides of the element.
- Create a **Thickness** structure defined by four distinct values that are applied to the left, top, right, and bottom sides of the element.

The following XAML example shows all three possibilities:

XAML

```
<StackLayout Padding="0,20,0,0">
  <!-- Margin defined by a single uniform value. -->
  <Label Text=".NET MAUI" Margin="20" />
  <!-- Margin defined by horizontal and vertical values. -->
  <Label Text=".NET for iOS" Margin="10,15" />
  <!-- Margin defined by four distinct values that are applied to the left,
  top, right, and bottom. -->
  <Label Text=".NET for Android" Margin="0,20,15,5" />
</StackLayout>
```

The equivalent C# code is:

C#

```
StackLayout stackLayout = new StackLayout
{
    Padding = new Thickness(0,20,0,0)
};
// Margin defined by a single uniform value.
stackLayout.Add(new Label { Text = ".NET MAUI", Margin = new Thickness(20)
});
// Margin defined by horizontal and vertical values.
stackLayout.Add(new Label { Text = ".NET for iOS", Margin = new
Thickness(10,25) });
```



```
// Margin defined by four distinct values that are applied to the left, top, right, and bottom.  
stackLayout.Add(new Label { Text = ".NET for Android", Margin = new Thickness(0,20,15,5) });
```

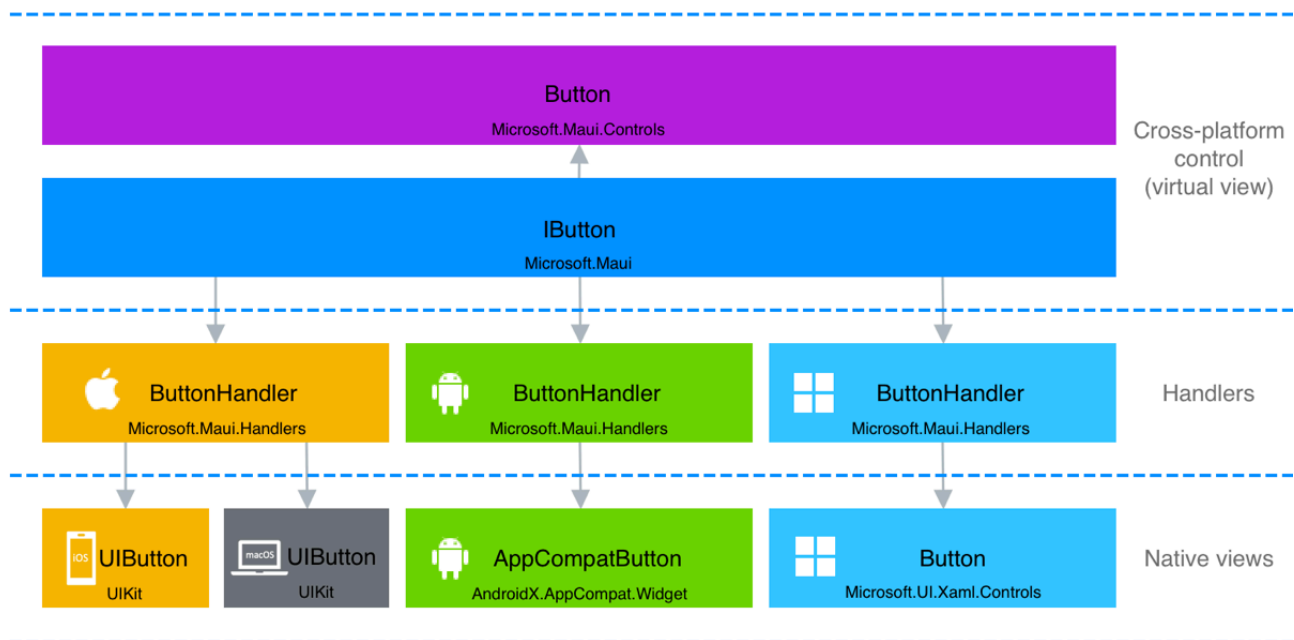
ⓘ Note

Thickness values can be negative, which typically clips or overdraws the content.

Handlers

Article • 01/17/2024

.NET Multi-platform App UI (.NET MAUI) provides a collection of cross-platform controls that can be used to display data, initiate actions, indicate activity, display collections, pick data, and more. Each control has an interface representation that abstracts the control. Cross-platform controls that implement these interfaces are known as *virtual views*. *Handlers* map these virtual views to controls on each platform, which are known as *native views*. Handlers are also responsible for instantiating the underlying native view, and mapping the cross-platform control API to the native view API. For example, on iOS a handler maps a .NET MAUI `Button` to an iOS `UIButton`. On Android, the `Button` is mapped to an `AppCompatButton`:



.NET MAUI handlers are accessed through their control-specific interface, such as `IButton` for a `Button`. This avoids the cross-platform control having to reference its handler, and the handler having to reference the cross-platform control.

Each handler class exposes the native view for the cross-platform control via its `PlatformView` property. This property can be accessed to set native view properties, invoke native view methods, and subscribe to native view events. In addition, the cross-platform control implemented by the handler is exposed via its `VirtualView` property.

When you create a cross-platform control whose implementation is provided on each platform by native views, you should implement a handler that maps the cross-platform control API to the native view APIs. For more information, see [Create custom controls with handlers](#).

You can also customize handlers to augment the appearance and behavior of existing cross-platform controls beyond the customization that's possible through the control's API. This handler customization modifies the native views for the cross-platform control. Handlers are global, and customizing a handler for a control will result in all controls of the same type being customized in your app. For more information, see [Customize .NET MAUI controls with handlers](#).

Mappers

A key concept of .NET MAUI handlers is mappers. Each handler typically provides a *property mapper*, and sometimes a *command mapper*, that maps the cross-platform control's API to the native view's API.

A *property mapper* defines what Actions to take when a property change occurs in the cross-platform control. It's a `Dictionary` that maps the cross-platform control's properties to their associated Actions. Each platform handler then provides implementations of the Actions, which manipulate the native view API. This ensures that when a property is set on a cross-platform control, the underlying native view is updated as required.

A *command mapper* defines what Actions to take when the cross-platform control sends commands to native views. They're similar to property mappers, but allow for additional data to be passed. A command in this context doesn't mean an `ICommand` implementation. Instead, a command is just an instruction, and optionally its data, that's sent to a native view. The command mapper is a `Dictionary` that maps the cross-platform control's command to their associated Actions. Each handler then provides implementations of the Actions, which manipulate the native view API. This ensures that when a cross-platform control sends a command to its native view, the native view is updated as required. For example, when a `ScrollView` is scrolled, the `ScrollViewHandler` uses a command mapper to invoke an Action that accepts a scroll position argument. The Action then instructs the underlying native view to scroll to that position.

The advantage of using *mappers* to update native views is that native views can be decoupled from cross-platform controls. This removes the need for native views to subscribe to and unsubscribe from cross-platform control events. It also allows for easy customization because mappers can be modified without subclassing.

Handler lifecycle

All handler-based .NET MAUI controls support two handler lifecycle events:

- `HandlerChanging` is raised when a new handler is about to be created for a cross-platform control, and when an existing handler is about to be removed from a cross-platform control. The `HandlerChangingEventArgs` object that accompanies this event has `NewHandler` and `OldHandler` properties, of type `IElementHandler`. When the `NewHandler` property isn't `null`, the event indicates that a new handler is about to be created for a cross-platform control. When the `OldHandler` property isn't `null`, the event indicates that the existing native control is about to be removed from the cross-platform control, and therefore any native events should be unwired and other cleanup performed.
- `HandlerChanged` is raised after the handler for a cross-platform control has been created. This event indicates that the native control that implements the cross-platform control is available, and all the property values set on the cross-platform control have been applied to the native control.

ⓘ Note

The `HandlerChanging` event is raised on a cross-platform control before the `HandlerChanged` event.

In addition to these events, each cross-platform control also has an overridable `OnHandlerChanging` method that's invoked when the `HandlerChanging` event is raised, and a `OnHandlerChanged` method that's invoked when the `HandlerChanged` event is raised.

View handlers

The following table lists the types that implement views in .NET MAUI:

 Expand table

View	Interface	Handler	Property Mapper	Command Mapper
ActivityIndicator	IActivityIndicator	ActivityIndicatorHandler	Mapper	CommandMapper
BlazorWebView	IBlazorWebView	BlazorWebViewHandler	BlazorWebViewMapper	
Border	IBorderView	BorderHandler	Mapper	CommandMapper
BoxView	IShapeView, IShape	ShapeViewHandler	Mapper	CommandMapper
Button	IButton	ButtonHandler	ImageButtonMapper, TextButtonMapper, Mapper	CommandMapper
CarouselView		CarouselViewHandler	Mapper	
Cell		CellRenderer	Mapper	CommandMapper
CheckBox	ICheckBox	CheckBoxHandler	Mapper	CommandMapper
CollectionView		CollectionViewHandler	<Mapper	
ContentView	IContentView	ContentViewHandler	Mapper	CommandMapper
DatePicker	IDatePicker	DatePickerHandler	Mapper	CommandMapper
Editor	IEditor	EditorHandler	Mapper	CommandMapper
Ellipse	IShape	ShapeViewHandler	Mapper	CommandMapper
Entry	IEntry	EntryHandler	Mapper	CommandMapper
EntryCell		EntryCellRenderer	Mapper	CommandMapper
Frame		FrameRenderer	Mapper	CommandMapper
GraphicsView	IGraphicsView	GraphicsViewHandler	Mapper	CommandMapper
Image	IImage	ImageHandler	Mapper	CommandMapper
ImageButton	IImageButton	ImageButtonHandler	ImageMapper, Mapper	
ImageCell		ImageCellRenderer	Mapper	CommandMapper
IndicatorView	IIndicatorView	IndicatorViewHandler	Mapper	CommandMapper
Label	ILabel	LabelHandler	Mapper	CommandMapper
Line	IShape	LineHandler	Mapper	CommandMapper
ListView		ListViewRenderer	Mapper	CommandMapper
Map	IMap	MapHandler	Mapper	CommandMapper
Path	IShape	PathHandler	Mapper	CommandMapper
Picker	IPicker	PickerHandler	Mapper	CommandMapper
Polygon	IShape	PolygonHandler	Mapper	CommandMapper

View	Interface	Handler	Property Mapper	Command Mapper
Polyline	IShape	PolylineHandler	Mapper	CommandMapper
ProgressBar	IProgress	ProgressBarHandler	Mapper	CommandMapper
RadioButton	IRadioButton	RadioButtonHandler	Mapper	CommandMapper
Rectangle	IShape	RectangleHandler	Mapper	CommandMapper
RefreshView	IRefreshView	RefreshViewHandler	Mapper	CommandMapper
RoundRectangle	IShape	RoundRectangleHandler	Mapper	CommandMapper
ScrollView	IScrollView	ScrollViewHandler	Mapper	CommandMapper
SearchBar	ISearchBar	SearchBarHandler	Mapper	CommandMapper
Slider	ISlider	SliderHandler	Mapper	CommandMapper
Stepper	IStepper	StepperHandler	Mapper	CommandMapper
SwipeView	ISwipeView	SwipeViewHandler	Mapper	CommandMapper
Switch	ISwitch	SwitchHandler	Mapper	CommandMapper
SwitchCell		SwitchCellRenderer	Mapper	CommandMapper
TableView		TableViewRenderer	Mapper	CommandMapper
TextCell		TextCellRenderer	Mapper	CommandMapper
TimePicker	ITimePicker	TimePickerHandler	Mapper	CommandMapper
ViewCell		ViewCellRenderer	Mapper	CommandMapper
WebView	IWebView	WebViewHandler	Mapper	CommandMapper

Page handlers

The following table lists the types that implement pages in .NET MAUI:

 Expand table

Page	Android Handler	iOS/Mac Catalyst Handler	Windows Handler	Property Mapper	Command Mapper
ContentPage	PageHandler	PageHandler	PageHandler	Mapper	CommandMapper
FlyoutPage	FlyoutViewHandler	PhoneFlyoutPageRenderer	FlyoutViewHandler	Mapper	CommandMapper
NavigationPage	NavigationViewHandler	NavigationRenderer	NavigationViewHandler	Mapper	CommandMapper
TabbedPage	TabbedViewHandler	TabbedRenderer	TabbedViewHandler	Mapper	CommandMapper
Shell	ShellHandler	ShellRenderer	ShellRenderer	Mapper	CommandMapper

Create a custom control using handlers

Article • 10/31/2024

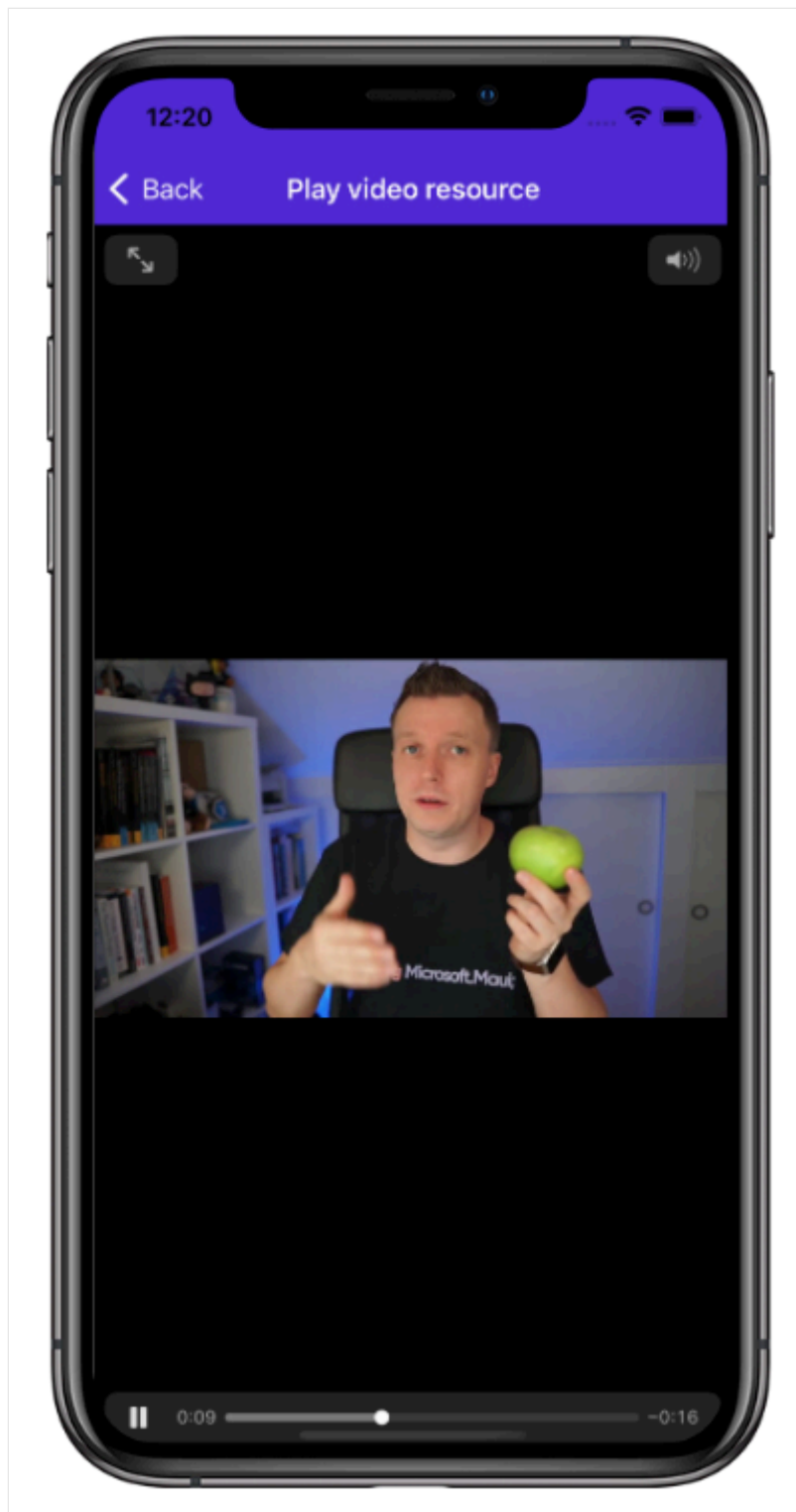


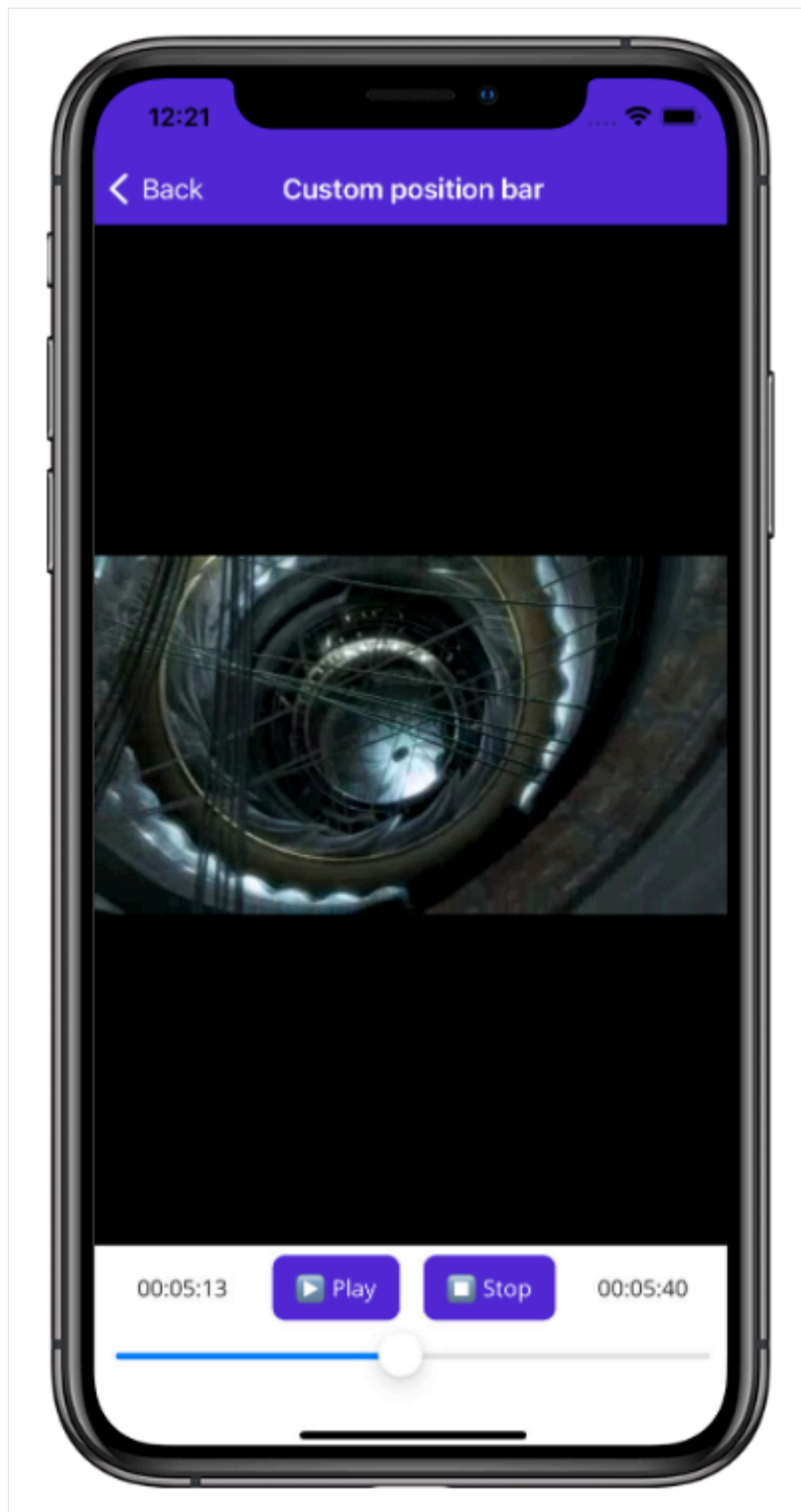
[Browse the sample](#)

A standard requirement for apps is the ability to play videos. This article examines how to create a .NET Multi-platform App UI (.NET MAUI) cross-platform `Video` control that uses a handler to map the cross-platform control API to the native views on Android, iOS, and Mac Catalyst that play videos. This control can play video from three sources:

- A URL, which represents a remote video.
- A resource, which is a file embedded in the app.
- A file, from the device's video library.

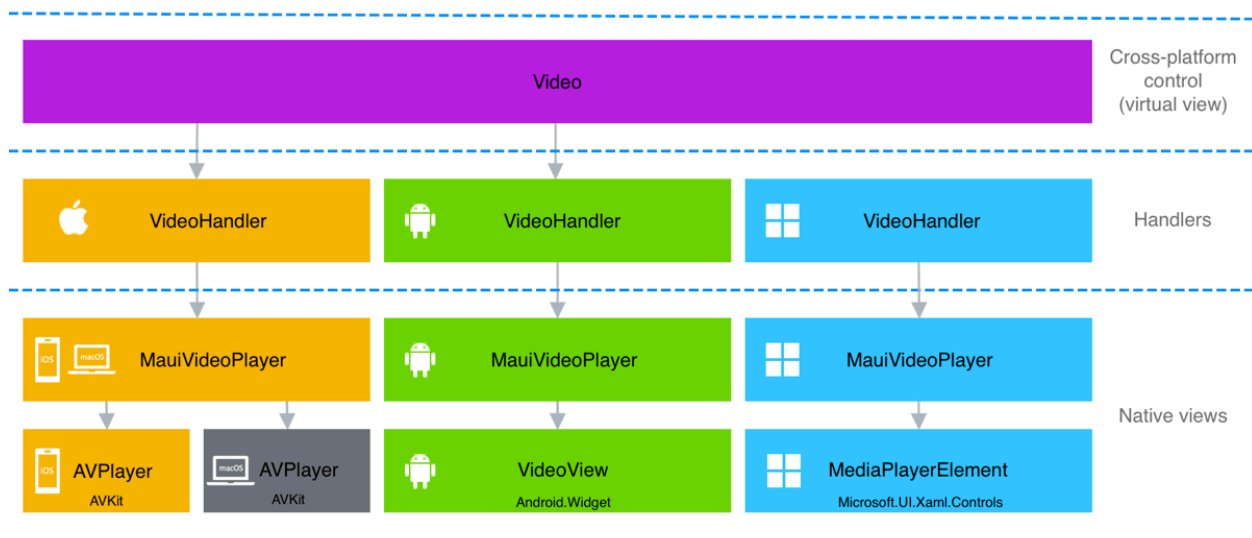
Video controls require *transport controls*, which are buttons for playing and pausing the video, and a positioning bar that shows the progress through the video and allows the user to move quickly to a different location. The `Video` control can either use the transport controls and positioning bar provided by the platform, or you can supply custom transport controls and a positioning bar. The following screenshots show the control on iOS, with and without custom transport controls:





A more sophisticated video control would have additional features, such as a volume control, a mechanism to interrupt video playback when a call is received, and a way of keeping the screen active during playback.

The architecture of the `video` control is shown in the following diagram:



The `Video` class provides the cross-platform API for the control. Mapping of the cross-platform API to the native view APIs is performed by the `VideoHandler` class on each platform, which maps the `Video` class to the `MauiVideoPlayer` class. On iOS and Mac Catalyst, the `MauiVideoPlayer` class uses the `AVPlayer` type to provide video playback. On Android, the `MauiVideoPlayer` class uses the `VideoView` type to provide video playback. On Windows, the `MauiVideoPlayer` class uses the `MediaPlayerElement` type to provide video playback.

Important

.NET MAUI decouples its handlers from its cross-platform controls through interfaces. This enables experimental frameworks such as Comet and Fabulous to provide their own cross-platform controls, that implement the interfaces, while still using .NET MAUI's handlers. Creating an interface for your cross-platform control is only necessary if you need to decouple your handler from its cross-platform control for a similar purpose, or for testing purposes.

The process for creating a cross-platform .NET MAUI custom control, whose platform implementations are provided by handlers, is as follows:

1. Create a class for the cross-platform control, which provides the control's public API. For more information, see [Create the cross-platform control](#).
2. Create any required additional cross-platform types.
3. Create a `partial` handler class. For more information, see [Create the handler](#).
4. In the handler class, create a `PropertyMapper` dictionary, which defines the Actions to take when cross-platform property changes occur. For more information, see [Create the property mapper](#).

5. Optionally, in your handler class, create a [CommandMapper](#) dictionary, which defines the Actions to take when the cross-platform control sends instructions to the native views that implement the cross-platform control. For more information, see [Create the command mapper](#).
6. Create `partial` handler classes for each platform that create the native views that implement the cross-platform control. For more information, see [Create the platform controls](#).
7. Register the handler using the [ConfigureMauiHandlers](#) and [AddHandler](#) methods in your app's `MauiProgram` class. For more information, see [Register the handler](#).

Then, the cross-platform control can be consumed. For more information, see [Consume the cross-platform control](#).

Create the cross-platform control

To create a cross-platform control, you should create a class that derives from [View](#):

C#

```
using System.ComponentModel;

namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        public static readonly BindableProperty
AreTransportControlsEnabledProperty =
        BindableProperty.Create(nameof(AreTransportControlsEnabled),
typeof(bool), typeof(Video), true);

        public static readonly BindableProperty SourceProperty =
        BindableProperty.Create(nameof(Source), typeof(VideoSource),
typeof(Video), null);

        public static readonly BindableProperty AutoPlayProperty =
        BindableProperty.Create(nameof(AutoPlay), typeof(bool),
typeof(Video), true);

        public static readonly BindableProperty IsLoopingProperty =
        BindableProperty.Create(nameof(IsLooping), typeof(bool),
typeof(Video), false);

        public bool AreTransportControlsEnabled
        {
            get { return
(bool)GetValue(AreTransportControlsEnabledProperty); }
            set { SetValue(AreTransportControlsEnabledProperty, value); }
        }
    }
}
```

```

[TypeConverter(typeof(VideoSourceConverter))]
public VideoSource Source
{
    get { return (VideoSource)GetValue(SourceProperty); }
    set { SetValue(SourceProperty, value); }
}

public bool AutoPlay
{
    get { return (bool)GetValue(AutoPlayProperty); }
    set { SetValue(AutoPlayProperty, value); }
}

public bool IsLooping
{
    get { return (bool)GetValue(IsLoopingProperty); }
    set { SetValue(IsLoopingProperty, value); }
}
...
}
}

```

The control should provide a public API that will be accessed by its handler, and control consumers. Cross-platform controls should derive from [View](#), which represents a visual element that's used to place layouts and views on the screen.

Create the handler

After creating your cross-platform control, you should create a `partial` class for your handler:

C#

```

#if IOS || MACCATALYST
using PlatformView = VideoDemos.Platforms.MaciOS.MauiVideoPlayer;
#elif ANDROID
using PlatformView = VideoDemos.Platforms.Android.MauiVideoPlayer;
#elif WINDOWS
using PlatformView = VideoDemos.Platforms.Windows.MauiVideoPlayer;
#elif (NETSTANDARD || !PLATFORM) || (NET6_0_OR_GREATER && !IOS && !ANDROID)
using PlatformView = System.Object;
#endif
using VideoDemos.Controls;
using Microsoft.Maui.Handlers;

namespace VideoDemos.Handlers
{
    public partial class VideoHandler
    {

```

```
}  
}
```

The handler class is a partial class whose implementation will be completed on each platform with an additional partial class.

The conditional `using` statements define the `PlatformView` type on each platform. On Android, iOS, Mac Catalyst, and Windows, the native views are provided by the custom `MauiVideoPlayer` class. The final conditional `using` statement defines `PlatformView` to be equal to `System.Object`. This is necessary so that the `PlatformView` type can be used within the handler for usage across all platforms. The alternative would be to have to define the `PlatformView` property once per platform, using conditional compilation.

Create the property mapper

Each handler typically provides a *property mapper*, which defines what Actions to take when a property change occurs in the cross-platform control. The `PropertyMapper` type is a `Dictionary` that maps the cross-platform control's properties to their associated Actions.

`PropertyMapper` is defined in .NET MAUI's `ViewHandler<TVirtualView,TPlatformView>` class, and requires two generic arguments to be supplied:

- The class for the cross-platform control, which derives from `View`.
- The class for the handler.

The following code example shows the `VideoHandler` class extended with the `PropertyMapper` definition:

C#

```
public partial class VideoHandler  
{  
    public static IPropertyMapper<Video, VideoHandler> PropertyMapper = new  
    PropertyMapper<Video, VideoHandler>(ViewHandler.ViewMapper)  
    {  
        [nameof(Video.AreTransportControlsEnabled)] =  
        MapAreTransportControlsEnabled,  
        [nameof(Video.Source)] = MapSource,  
        [nameof(Video.IsLooping)] = MapIsLooping,  
        [nameof(Video.Position)] = MapPosition  
    };  
  
    public VideoHandler() : base(PropertyMapper)  
    {
```

```
}  
}
```

The [PropertyMapper](#) is a `Dictionary` whose key is a `string` and whose value is a generic `Action`. The `string` represents the cross-platform control's property name, and the `Action` represents a `static` method that requires the handler and cross-platform control as arguments. For example, the signature of the `MapSource` method is `public static void MapSource(VideoHandler handler, Video video)`.

Each platform handler must provide implementations of the Actions, which manipulate the native view APIs. This ensures that when a property is set on a cross-platform control, the underlying native view will be updated as required. The advantage of this approach is that it allows for easy cross-platform control customization, because the property mapper can be modified by cross-platform control consumers without subclassing.

Create the command mapper

Each handler can also provide a *command mapper*, which defines what Actions to take when the cross-platform control sends commands to native views. Command mappers are similar to property mappers, but allow for additional data to be passed. In this context, a command is an instruction, and optionally its data, that's sent to a native view. The [CommandMapper](#) type is a `Dictionary` that maps cross-platform control members to their associated Actions.

[CommandMapper](#) is defined in .NET MAUI's [ViewHandler<TVirtualView,TPlatformView>](#) class, and requires two generic arguments to be supplied:

- The class for the cross-platform control, which derives from [View](#).
- The class for the handler.

The following code example shows the `VideoHandler` class extended with the [CommandMapper](#) definition:

C#

```
public partial class VideoHandler  
{  
    public static IPropertyMapper<Video, VideoHandler> PropertyMapper = new  
    PropertyMapper<Video, VideoHandler>(ViewHandler.ViewMapper)  
    {  
        [nameof(Video.AreTransportControlsEnabled)] =  
        MapAreTransportControlsEnabled,  
        [nameof(Video.Source)] = MapSource,  
    }  
}
```

```

        [nameof(Video.IsLooping)] = MapIsLooping,
        [nameof(Video.Position)] = MapPosition
    };

    public static CommandMapper<Video, VideoHandler> CommandMapper =
    new(ViewCommandMapper)
    {
        [nameof(Video.UpdateStatus)] = MapUpdateStatus,
        [nameof(Video.PlayRequested)] = MapPlayRequested,
        [nameof(Video.PauseRequested)] = MapPauseRequested,
        [nameof(Video.StopRequested)] = MapStopRequested
    };

    public VideoHandler() : base(PropertyMapper, CommandMapper)
    {
    }
}

```

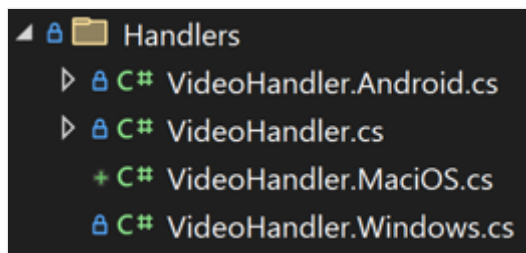
The `CommandMapper` is a `Dictionary` whose key is a `string` and whose value is a generic `Action`. The `string` represents the cross-platform control's command name, and the `Action` represents a `static` method that requires the handler, cross-platform control, and optional data as arguments. For example, the signature of the `MapPlayRequested` method is `public static void MapPlayRequested(VideoHandler handler, Video video, object? args)`.

Each platform handler must provide implementations of the Actions, which manipulate the native view APIs. This ensures that when a command is sent from the cross-platform control, the underlying native view will be manipulated as required. The advantage of this approach is that it removes the need for native views to subscribe to and unsubscribe from cross-platform control events. In addition, it allows for easy customization because the command mapper can be modified by cross-platform control consumers without subclassing.

Create the platform controls

After creating the mappers for your handler, you must provide handler implementations on all platforms. This can be accomplished by adding partial class handler implementations in the child folders of the *Platforms* folder. Alternatively you could configure your project to support filename-based multi-targeting, or folder-based multi-targeting, or both.

The sample app is configured to support filename-based multi-targeting, so that the handler classes all are located in a single folder:



The `VideoHandler` class containing the mappers is named `VideoHandler.cs`. Its platform implementations are in the `VideoHandler.Android.cs`, `VideoHandler.MacIOS.cs`, and `VideoHandler.Windows.cs` files. This filename-based multi-targeting is configured by adding the following XML to the project file, as children of the `<Project>` node:

XML

```
<!-- Android -->
<ItemGroup Condition="$(TargetFramework.StartsWith('net8.0-android')) !=
true">
  <Compile Remove="**\*.Android.cs" />
  <None Include="**\*.Android.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>

<!-- iOS and Mac Catalyst -->
<ItemGroup Condition="$(TargetFramework.StartsWith('net8.0-ios')) != true
AND $(TargetFramework.StartsWith('net8.0-maccatalyst')) != true">
  <Compile Remove="**\*.MacIOS.cs" />
  <None Include="**\*.MacIOS.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>

<!-- Windows -->
<ItemGroup Condition="$(TargetFramework.Contains('-windows')) != true ">
  <Compile Remove="**\*.Windows.cs" />
  <None Include="**\*.Windows.cs"
Exclude="$(DefaultItemExcludes);$(DefaultExcludesInProjectFolder)" />
</ItemGroup>
```

For more information about configuring multi-targeting, see [Configure multi-targeting](#).

Each platform handler class should be a partial class and derive from the `ViewHandler<TVirtualView,TPlatformView>` class, which requires two type arguments:

- The class for the cross-platform control, which derives from [View](#).
- The type of the native view that implements the cross-platform control on the platform. This should be identical to the type of the `PlatformView` property in the handler.

Important

The [ViewHandler<TVirtualView,TPlatformView>](#) class provides [VirtualView](#) and [PlatformView](#) properties. The [VirtualView](#) property is used to access the cross-platform control from its handler. The [PlatformView](#) property, is used to access the native view on each platform that implements the cross-platform control.

Each of the platform handler implementations should override the following methods:

- [CreatePlatformView](#), which should create and return the native view that implements the cross-platform control.
- [ConnectHandler](#), which should perform any native view setup, such as initializing the native view and performing event subscriptions.
- [DisconnectHandler](#), which should perform any native view cleanup, such as unsubscribing from events and disposing objects.

Important

The [DisconnectHandler](#) method is automatically invoked by .NET MAUI by default, although this behavior can be changed. For more information, see [Control handler disconnection](#).

Each platform handler should also implement the Actions that are defined in the mapper dictionaries.

In addition, each platform handler should also provide code, as required, to implement the functionality of the cross-platform control on the platform. Alternatively, this can be provided by an additional type, which is the approach adopted here.

Android

Video is played on Android with a `VideoView`. However, here, the `VideoView` has been encapsulated in a `MauiVideoPlayer` type to keep the native view separated from its handler. The following example shows the `VideoHandler` partial class for Android, with its three overrides:

C#

```
#nullable enable
using Microsoft.Maui.Handlers;
using VideoDemos.Controls;
using VideoDemos.Platforms.Android;

namespace VideoDemos.Handlers
{
```



```

public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    protected override MauiVideoPlayer CreatePlatformView() => new
    MauiVideoPlayer(Context, VirtualView);

    protected override void ConnectHandler(MauiVideoPlayer platformView)
    {
        base.ConnectHandler(platformView);

        // Perform any control setup here
    }

    protected override void DisconnectHandler(MauiVideoPlayer
platformView)
    {
        platformView.Dispose();
        base.DisconnectHandler(platformView);
    }
    ...
}
}

```

`VideoHandler` derives from the `ViewHandler<TVirtualView,TPlatformView>` class, with the generic `Video` argument specifying the cross-platform control type, and the `MauiVideoPlayer` argument specifying the type that encapsulates the `VideoView` native view.

The `CreatePlatformView` override creates and returns a `MauiVideoPlayer` object. The `ConnectHandler` override is the location to perform any required native view setup. The `DisconnectHandler` override is the location to perform any native view cleanup, and so calls the `Dispose` method on the `MauiVideoPlayer` instance.

The platform handler also has to implement the Actions defined in the property mapper dictionary:

```

C#

public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    ...
    public static void MapAreTransportControlsEnabled(VideoHandler handler,
Video video)
    {
        handler.PlatformView?.UpdateTransportControlsEnabled();
    }

    public static void MapSource(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdateSource();
    }
}

```

```

    public static void MapIsLooping(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdateIsLooping();
    }

    public static void MapPosition(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdatePosition();
    }
    ...
}

```

Each Action is executed in response to a property changing on the cross-platform control, and is a `static` method that requires handler and cross-platform control instances as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` type.

The platform handler also has to implement the Actions defined in the command mapper dictionary:

C#

```

public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    ...
    public static void MapUpdateStatus(VideoHandler handler, Video video,
    object? args)
    {
        handler.PlatformView?.UpdateStatus();
    }

    public static void MapPlayRequested(VideoHandler handler, Video video,
    object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.PlayRequested(position);
    }

    public static void MapPauseRequested(VideoHandler handler, Video video,
    object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.PauseRequested(position);
    }
}

```

```

    public static void MapStopRequested(VideoHandler handler, Video video,
object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.StopRequested(position);
    }
    ...
}

```

Each Action is executed in response to a command being sent from the cross-platform control, and is a `static` method that requires handler and cross-platform control instances, and optional data as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` class, after extracting the optional data.

On Android, the `MauiVideoPlayer` class encapsulates the `VideoView` to keep the native view separated from its handler:

C#

```

using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        MediaController _mediaController;
        bool _isPrepared;
        Context _context;
        Video _video;

        public MauiVideoPlayer(Context context, Video video) : base(context)
        {
            _context = context;
            _video = video;

            SetBackgroundColor(Color.Black);

            // Create a RelativeLayout for sizing the video
            RelativeLayout relativeLayout = new RelativeLayout(_context)
            {

```

```

        LayoutParameters = new
CoordinatorLayout.LayoutParams(LayoutParams.MatchParent,
LayoutParams.MatchParent)
    {
        Gravity = (int)GravityFlags.Center
    }
};

// Create a VideoView and position it in the RelativeLayout
_videoView = new VideoView(context)
{
    LayoutParameters = new
RelativeLayout.LayoutParams(LayoutParams.MatchParent,
LayoutParams.MatchParent)
};

// Add to the layouts
relativeLayout.AddView(_videoView);
AddView(relativeLayout);

// Handle events
_videoView.Prepared += OnVideoViewPrepared;
}
...
}
}

```

`MauiVideoPlayer` derives from `CoordinatorLayout`, because the root native view in a .NET MAUI app on Android is `CoordinatorLayout`. While the `MauiVideoPlayer` class could derive from other native Android types, it can be difficult to control native view positioning in some scenarios.

The `VideoView` could be added directly to the `CoordinatorLayout`, and positioned in the layout as required. However, here, an Android `RelativeLayout` is added to the `CoordinatorLayout`, and the `VideoView` is added to the `RelativeLayout`. Layout parameters are set on both the `RelativeLayout` and `VideoView` so that the `VideoView` is centered in the page, and expands to fill the available space while maintaining its aspect ratio.

The constructor also subscribes to the `VideoView.Prepared` event. This event is raised when the video is ready for playback, and is unsubscribed from in the `Dispose` override:

C#

```

public class MauiVideoPlayer : CoordinatorLayout
{
    VideoView _videoView;
    Video _video;
    ...
}

```

```

protected override void Dispose(bool disposing)
{
    if (disposing)
    {
        _videoView.Prepared -= OnVideoViewPrepared;
        _videoView.Dispose();
        _videoView = null;
        _video = null;
    }

    base.Dispose(disposing);
}
...
}

```

In addition to unsubscribing from the `Prepared` event, the `Dispose` override also performs native view cleanup.

ⓘ Note

The `Dispose` override is called by the handler's [DisconnectHandler](#) override.

The platform transport controls include buttons that play, pause, and stop the video, and are provided by Android's `MediaController` type. If the `Video.AreTransportControlsEnabled` property is set to `true`, a `MediaController` is set as the media player of the `VideoView`. This occurs because when the `AreTransportControlsEnabled` property is set, the handler's property mapper ensures that the `MapAreTransportControlsEnabled` method is invoked, which in turn calls the `UpdateTransportControlsEnabled` method in `MauiVideoPlayer`:

C#

```

public class MauiVideoPlayer : CoordinatorLayout
{
    VideoView _videoView;
    MediaController _mediaController;
    Video _video;
    ...

    public void UpdateTransportControlsEnabled()
    {
        if (_video.AreTransportControlsEnabled)
        {
            _mediaController = new MediaController(_context);
            _mediaController.SetMediaPlayer(_videoView);
            _videoView.SetMediaController(_mediaController);
        }
    }
}

```

```

else
{
    _videoView.SetMediaController(null);
    if (_mediaController != null)
    {
        _mediaController.SetMediaPlayer(null);
        _mediaController = null;
    }
}
...
}

```

The transport controls fade out if they're not used but can be restored by tapping on the video.

If the `Video.AreTransportControlsEnabled` property is set to `false`, the `MediaController` is removed as the media player of the `VideoView`. In this scenario, you can then control video playback programmatically or supply your own transport controls. For more information, see [Create custom transport controls](#).

iOS and Mac Catalyst

Video is played on iOS and Mac Catalyst with an `AVPlayer` and an `AVPlayerViewController`. However, here, these types are encapsulated in a `MauiVideoPlayer` type to keep the native views separated from their handler. The following example shows the `VideoHandler` partial class for iOS, with its three overrides:

C#

```

using Microsoft.Maui.Handlers;
using VideoDemos.Controls;
using VideoDemos.Platforms.MaciOS;

namespace VideoDemos.Handlers
{
    public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
    {
        protected override MauiVideoPlayer CreatePlatformView() => new
        MauiVideoPlayer(VirtualView);

        protected override void ConnectHandler(MauiVideoPlayer platformView)
        {
            base.ConnectHandler(platformView);

            // Perform any control setup here
        }
    }
}

```

```

        protected override void DisconnectHandler(MauiVideoPlayer
platformView)
        {
            platformView.Dispose();
            base.DisconnectHandler(platformView);
        }
        ...
    }
}

```

`VideoHandler` derives from the `ViewHandler<TVirtualView,TPlatformView>` class, with the generic `Video` argument specifying the cross-platform control type, and the `MauiVideoPlayer` argument specifying the type that encapsulates the `AVPlayer` and `AVPlayerViewController` native views.

The `CreatePlatformView` override creates and returns a `MauiVideoPlayer` object. The `ConnectHandler` override is the location to perform any required native view setup. The `DisconnectHandler` override is the location to perform any native view cleanup, and so calls the `Dispose` method on the `MauiVideoPlayer` instance.

The platform handler also has to implement the Actions defined in the property mapper dictionary:

C#

```

public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    ...
    public static void MapAreTransportControlsEnabled(VideoHandler handler,
Video video)
    {
        handler?.PlatformView.UpdateTransportControlsEnabled();
    }

    public static void MapSource(VideoHandler handler, Video video)
    {
        handler?.PlatformView.UpdateSource();
    }

    public static void MapIsLooping(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdateIsLooping();
    }

    public static void MapPosition(VideoHandler handler, Video video)
    {
        handler?.PlatformView.UpdatePosition();
    }
}

```

```
...  
}
```

Each Action is executed in response to a property changing on the cross-platform control, and is a `static` method that requires handler and cross-platform control instances as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` type.

The platform handler also has to implement the Actions defined in the command mapper dictionary:

C#

```
public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>  
{  
    ...  
    public static void MapUpdateStatus(VideoHandler handler, Video video,  
object? args)  
    {  
        handler.PlatformView?.UpdateStatus();  
    }  
  
    public static void MapPlayRequested(VideoHandler handler, Video video,  
object? args)  
    {  
        if (args is not VideoPositionEventArgs)  
            return;  
  
        TimeSpan position = ((VideoPositionEventArgs)args).Position;  
        handler.PlatformView?.PlayRequested(position);  
    }  
  
    public static void MapPauseRequested(VideoHandler handler, Video video,  
object? args)  
    {  
        if (args is not VideoPositionEventArgs)  
            return;  
  
        TimeSpan position = ((VideoPositionEventArgs)args).Position;  
        handler.PlatformView?.PauseRequested(position);  
    }  
  
    public static void MapStopRequested(VideoHandler handler, Video video,  
object? args)  
    {  
        if (args is not VideoPositionEventArgs)  
            return;  
  
        TimeSpan position = ((VideoPositionEventArgs)args).Position;  
        handler.PlatformView?.StopRequested(position);  
    }  
}
```



```
...  
}
```

Each Action is executed in response to a command being sent from the cross-platform control, and is a `static` method that requires handler and cross-platform control instances, and optional data as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` class, after extracting the optional data.

On iOS and Mac Catalyst, the `MauiVideoPlayer` class encapsulates the `AVPlayer` and `AVPlayerViewController` types to keep the native views separated from their handler:

C#

```
using AVFoundation;  
using AVKit;  
using CoreMedia;  
using Foundation;  
using System.Diagnostics;  
using UIKit;  
using VideoDemos.Controls;  
  
namespace VideoDemos.Platforms.MaciOS  
{  
    public class MauiVideoPlayer : UIView  
    {  
        AVPlayer _player;  
        AVPlayerViewController _playerViewController;  
        Video _video;  
        ...  
  
        public MauiVideoPlayer(Video video)  
        {  
            _video = video;  
  
            _playerViewController = new AVPlayerViewController();  
            _player = new AVPlayer();  
            _playerViewController.Player = _player;  
            _playerViewController.View.Frame = this.Bounds;  
  
#if IOS16_0_OR_GREATER || MACCATALYST16_1_OR_GREATER  
            // On iOS 16 and Mac Catalyst 16, for Shell-based apps, the  
            // AVPlayerViewController has to be added to the parent ViewController,  
            // otherwise the transport controls won't be displayed.  
            var viewController =  
                WindowStateManager.Default.GetCurrentUIViewController();  
  
            // If there's no view controller, assume it's not Shell and  
            // continue because the transport controls will still be displayed.  
            if (viewController?.View is not null)  
            {  
                // Zero out the safe area insets of the
```

```

AVPlayerViewController
    UIEdgeInsets insets = viewController.View.SafeAreaInsets;
    _playerViewController.AdditionalSafeAreaInsets = new
UIEdgeInsets(insets.Top * -1, insets.Left, insets.Bottom * -1,
insets.Right);

    // Add the View from the AVPlayerViewController to the
parent ViewController
    viewController.View.AddSubview(_playerViewController.View);
}
#endif
// Use the View from the AVPlayerViewController as the native
control
AddSubview(_playerViewController.View);
}
...
}
}

```

`MauiVideoPlayer` derives from `UIView`, which is the base class on iOS and Mac Catalyst for objects that display content and handle user interaction with that content. The constructor creates an `AVPlayer` object, which manages the playback and timing of a media file, and sets it as the `Player` property value of an `AVPlayerViewController`. The `AVPlayerViewController` displays content from the `AVPlayer` and presents transport controls and other features. The size and location of the control is then set, which ensures that the video is centered in the page and expands to fill the available space while maintaining its aspect ratio. On iOS 16 and Mac Catalyst 16, the `AVPlayerViewController` has to be added to the parent `ViewController` for Shell-based apps, otherwise the transport controls aren't displayed. The native view, which is the view from the `AVPlayerViewController`, is then added to the page.

The `Dispose` method is responsible for performing native view cleanup:

```

C#

public class MauiVideoPlayer : UIView
{
    AVPlayer _player;
    AVPlayerViewController _playerViewController;
    Video _video;
    ...

    protected override void Dispose(bool disposing)
    {
        if (disposing)
        {
            if (_player != null)
            {
                DestroyPlayedToEndObserver();
            }
        }
    }
}

```

```

        _player.ReplaceCurrentItemWithPlayerItem(null);
        _player.Dispose();
    }
    if (_playerViewController != null)
        _playerViewController.Dispose();

    _video = null;
}

base.Dispose(disposing);
}
...
}

```

In some scenarios, videos continue playing after a video playback page has been navigated away from. To stop the video, the `ReplaceCurrentItemWithPlayerItem` is set to `null` in the `Dispose` override, and other native view cleanup is performed.

ⓘ Note

The `Dispose` override is called by the handler's `DisconnectHandler` override.

The platform transport controls include buttons that play, pause, and stop the video, and are provided by the `AVPlayerViewController` type. If the `Video.AreTransportControlsEnabled` property is set to `true`, the `AVPlayerViewController` will display its playback controls. This occurs because when the `AreTransportControlsEnabled` property is set, the handler's property mapper ensures that the `MapAreTransportControlsEnabled` method is invoked, which in turn calls the `UpdateTransportControlsEnabled` method in `MauiVideoPlayer`:

C#

```

public class MauiVideoPlayer : UIView
{
    AVPlayerViewController _playerViewController;
    Video _video;
    ...

    public void UpdateTransportControlsEnabled()
    {
        _playerViewController.ShowsPlaybackControls =
        _video.AreTransportControlsEnabled;
    }
    ...
}

```

The transport controls fade out if they're not used but can be restored by tapping on the video.

If the `Video.AreTransportControlsEnabled` property is set to `false`, the `AVPlayerViewController` doesn't show its playback controls. In this scenario, you can then control video playback programmatically or supply your own transport controls. For more information, see [Create custom transport controls](#).

Windows

Video is played on Windows with the `MediaPlayerElement`. However, here, the `MediaPlayerElement` has been encapsulated in a `MauiVideoPlayer` type to keep the native view separated from its handler. The following example shows the `VideoHandler` partial class for Windows, with its three overrides:

```
C#

#nullable enable
using Microsoft.Maui.Handlers;
using VideoDemos.Controls;
using VideoDemos.Platforms.Windows;

namespace VideoDemos.Handlers
{
    public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
    {
        protected override MauiVideoPlayer CreatePlatformView() => new
        MauiVideoPlayer(VirtualView);

        protected override void ConnectHandler(MauiVideoPlayer platformView)
        {
            base.ConnectHandler(platformView);

            // Perform any control setup here
        }

        protected override void DisconnectHandler(MauiVideoPlayer
platformView)
        {
            platformView.Dispose();
            base.DisconnectHandler(platformView);
        }
        ...
    }
}
```

`VideoHandler` derives from the `ViewHandler<TVirtualView,TPlatformView>` class, with the generic `video` argument specifying the cross-platform control type, and the

`MauiVideoPlayer` argument specifying the type that encapsulates the `MediaPlayerElement` native view.

The `CreatePlatformView` override creates and returns a `MauiVideoPlayer` object. The `ConnectHandler` override is the location to perform any required native view setup. The `DisconnectHandler` override is the location to perform any native view cleanup, and so calls the `Dispose` method on the `MauiVideoPlayer` instance.

The platform handler also has to implement the Actions defined in the property mapper dictionary:

C#

```
public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    ...
    public static void MapAreTransportControlsEnabled(VideoHandler handler,
Video video)
    {
        handler.PlatformView?.UpdateTransportControlsEnabled();
    }

    public static void MapSource(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdateSource();
    }

    public static void MapIsLooping(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdateIsLooping();
    }

    public static void MapPosition(VideoHandler handler, Video video)
    {
        handler.PlatformView?.UpdatePosition();
    }
    ...
}
```

Each Action is executed in response to a property changing on the cross-platform control, and is a `static` method that requires handler and cross-platform control instances as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` type.

The platform handler also has to implement the Actions defined in the command mapper dictionary:

C#

```

public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
{
    ...
    public static void MapUpdateStatus(VideoHandler handler, Video video,
object? args)
    {
        handler.PlatformView?.UpdateStatus();
    }

    public static void MapPlayRequested(VideoHandler handler, Video video,
object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.PlayRequested(position);
    }

    public static void MapPauseRequested(VideoHandler handler, Video video,
object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.PauseRequested(position);
    }

    public static void MapStopRequested(VideoHandler handler, Video video,
object? args)
    {
        if (args is not VideoPositionEventArgs)
            return;

        TimeSpan position = ((VideoPositionEventArgs)args).Position;
        handler.PlatformView?.StopRequested(position);
    }
    ...
}

```

Each Action is executed in response to a command being sent from the cross-platform control, and is a `static` method that requires handler and cross-platform control instances, and optional data as arguments. In each case, the Action calls a method defined in the `MauiVideoPlayer` class, after extracting the optional data.

On Windows, the `MauiVideoPlayer` class encapsulates the `MediaPlayerElement` to keep the native view separated from its handler:

```

using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        ...

        public MauiVideoPlayer(Video video)
        {
            _video = video;
            _mediaPlayerElement = new MediaPlayerElement();
            this.Children.Add(_mediaPlayerElement);
        }
        ...
    }
}

```

`MauiVideoPlayer` derives from `Grid`, and the `MediaPlayerElement` is added as a child of the `Grid`. This enables the `MediaPlayerElement` to automatically size to fill all available space.

The `Dispose` method is responsible for performing native view cleanup:

C#

```

public class MauiVideoPlayer : Grid, IDisposable
{
    MediaPlayerElement _mediaPlayerElement;
    Video _video;
    bool _isMediaPlayerAttached;
    ...

    public void Dispose()
    {
        if (_isMediaPlayerAttached)
        {
            _mediaPlayerElement.MediaPlayer.MediaOpened -=
OnMediaPlayerMediaOpened;
            _mediaPlayerElement.MediaPlayer.Dispose();
        }
        _mediaPlayerElement = null;
    }
}

```

```
...  
}
```

In addition to unsubscribing from the `MediaOpened` event, the `Dispose` override also performs native view cleanup.

⚠ Note

The `Dispose` override is called by the handler's `DisconnectHandler` override.

The platform transport controls include buttons that play, pause, and stop the video, and are provided by the `MediaPlayerElement` type. If the `Video.AreTransportControlsEnabled` property is set to `true`, the `MediaPlayerElement` will display its playback controls. This occurs because when the `AreTransportControlsEnabled` property is set, the handler's property mapper ensures that the `MapAreTransportControlsEnabled` method is invoked, which in turn calls the `UpdateTransportControlsEnabled` method in `MauiVideoPlayer`:

C#

```
public class MauiVideoPlayer : Grid, IDisposable  
{  
    MediaPlayerElement _mediaPlayerElement;  
    Video _video;  
    bool _isMediaPlayerAttached;  
    ...  
  
    public void UpdateTransportControlsEnabled()  
    {  
        _mediaPlayerElement.AreTransportControlsEnabled =  
        _video.AreTransportControlsEnabled;  
    }  
    ...  
}
```

If the `Video.AreTransportControlsEnabled` property is set to `false`, the `MediaPlayerElement` doesn't show its playback controls. In this scenario, you can then control video playback programmatically or supply your own transport controls. For more information, see [Create custom transport controls](#).

Convert a cross-platform control into a platform control

Any .NET MAUI cross-platform control, that derives from [Element](#), can be converted to its underlying platform control with the [ToPlatform](#) extension method:

- On Android, [ToPlatform](#) converts a .NET MAUI control to an Android [View](#) object.
- On iOS and Mac Catalyst, [ToPlatform](#) converts a .NET MAUI control to a [UIView](#) object.
- On Windows, [ToPlatform](#) converts a .NET MAUI control to a `FrameworkElement` object.

⚠ Note

The [ToPlatform](#) method is in the `Microsoft.Maui.Platform` namespace.

On all platforms, the [ToPlatform](#) method requires a [MauiContext](#) argument.

The [ToPlatform](#) method can convert a cross-platform control to its underlying platform control from platform code, such as in a partial handler class for a platform:

C#

```
using Microsoft.Maui.Handlers;
using Microsoft.Maui.Platform;
using VideoDemos.Controls;
using VideoDemos.Platforms.Android;

namespace VideoDemos.Handlers
{
    public partial class VideoHandler : ViewHandler<Video, MauiVideoPlayer>
    {
        ...
        public static void MapSource(VideoHandler handler, Video video)
        {
            handler.PlatformView?.UpdateSource();

            // Convert cross-platform control to its underlying platform
            control
            MauiVideoPlayer mvp =
            (MauiVideoPlayer)video.ToPlatform(handler.MauiContext);
            ...
        }
        ...
    }
}
```

In this example, in the `VideoHandler` partial class for Android, the `MapSource` method converts the `video` instance to a `MauiVideoPlayer` object.

The [ToPlatform](#) method can also convert a cross-platform control to its underlying platform control from cross-platform code:

C#

```
using Microsoft.Maui.Platform;

namespace VideoDemos.Views;

public partial class MyPage : ContentPage
{
    ...
    protected override void OnHandlerChanged()
    {
        // Convert cross-platform control to its underlying platform control
        #if ANDROID
            Android.Views.View nativeView =
                video.ToPlatform(video.Handler.MauiContext);
        #elif IOS || MACCATALYST
            UIKit.UIView nativeView =
                video.ToPlatform(video.Handler.MauiContext);
        #elif WINDOWS
            Microsoft.UI.Xaml.FrameworkElement nativeView =
                video.ToPlatform(video.Handler.MauiContext);
        #endif
        ...
    }
    ...
}
```

In this example, a cross-platform `Video` control named `video` is converted to its underlying native view on each platform in the `OnHandlerChanged()` override. This override is called when the native view that implements the cross-platform control is available and initialized. The object returned by the [ToPlatform](#) method could be cast to its exact native type, which here is a `MauiVideoPlayer`.

Play a video

The `Video` class defines a `Source` property, which is used to specify the source of the video file, and an `AutoPlay` property. `AutoPlay` defaults to `true`, which means that the video should begin playing automatically after `Source` has been set. For the definition of these properties, see [Create the cross-platform control](#).

The `Source` property is of type `VideoSource`, which is an abstract class that consists of three static methods that instantiate the three classes that derive from `VideoSource`:

C#

```

using System.ComponentModel;

namespace VideoDemos.Controls
{
    [TypeConverter(typeof(VideoSourceConverter))]
    public abstract class VideoSource : Element
    {
        public static VideoSource FromUri(string uri)
        {
            return new UriVideoSource { Uri = uri };
        }

        public static VideoSource FromFile(string file)
        {
            return new FileVideoSource { File = file };
        }

        public static VideoSource FromResource(string path)
        {
            return new ResourceVideoSource { Path = path };
        }
    }
}

```

The `VideoSource` class includes a `TypeConverter` attribute that references `VideoSourceConverter`:

C#

```

using System.ComponentModel;

namespace VideoDemos.Controls
{
    public class VideoSourceConverter : TypeConverter,
    IExtendedTypeConverter
    {
        object IExtendedTypeConverter.ConvertFromInvariantString(string
value, IServiceProvider serviceProvider)
        {
            if (!string.IsNullOrEmpty(value))
            {
                Uri uri;
                return Uri.TryCreate(value, UriKind.Absolute, out uri) &&
uri.Scheme != "file" ?
                    VideoSource.FromUri(value) :
VideoSource.FromResource(value);
            }
            throw new InvalidOperationException("Cannot convert null or
whitespace to VideoSource.");
        }
    }
}

```

```
}  
}
```

The type converter is invoked when the `Source` property is set to a string in XAML. The `ConvertFromInvariantString` method attempts to convert the string to a `Uri` object. If it succeeds, and the scheme isn't `file`, then the method returns a `UriVideoSource`. Otherwise it returns a `ResourceVideoSource`.

Play a web video

The `UriVideoSource` class is used to specify a remote video with a URL. It defines a `Uri` property of type `string`:

C#

```
namespace VideoDemos.Controls  
{  
    public class UriVideoSource : VideoSource  
    {  
        public static readonly BindableProperty UriProperty =  
            BindableProperty.Create(nameof(Uri), typeof(string),  
            typeof(UriVideoSource));  
  
        public string Uri  
        {  
            get { return (string)GetValue(UriProperty); }  
            set { SetValue(UriProperty, value); }  
        }  
    }  
}
```

When the `Source` property is set to a `UriVideoSource`, the handler's property mapper ensures that the `MapSource` method is invoked:

C#

```
public static void MapSource(VideoHandler handler, Video video)  
{  
    handler?.PlatformView.UpdateSource();  
}
```

The `MapSource` method in turns calls the `UpdateSource` method on the handler's `PlatformView` property. The `PlatformView` property, which is of type `MauiVideoPlayer`, represents the native view that provides the video player implementation on each platform.

Android

Video is played on Android with a `VideoView`. The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `UriVideoSource`:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        bool _isPrepared;
        Video _video;
        ...

        public void UpdateSource()
        {
            _isPrepared = false;
            bool hasSetSource = false;

            if (_video.Source is UriVideoSource)
            {
                string uri = (_video.Source as UriVideoSource).Uri;
                if (!string.IsNullOrEmpty(uri))
                {
                    _videoView.SetVideoURI(Uri.Parse(uri));
                    hasSetSource = true;
                }
            }
            ...

            if (hasSetSource && _video.AutoPlay)
            {
                _videoView.Start();
            }
        }
        ...
    }
}
```

When processing objects of type `UriVideoSource`, the `SetVideoUri` method of `VideoView` is used to specify the video to be played, with an Android `Uri` object created from the string URI.

The `AutoPlay` property has no equivalent on `VideoView`, so the `Start` method is called if a new video has been set.

iOS and Mac Catalyst

To play a video on iOS and Mac Catalyst, an object of type `AVAsset` is created to encapsulate the video, and that is used to create an `AVPlayerItem`, which is then handed off to the `AVPlayer` object. The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `UriVideoSource`:

C#

```
using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayer _player;
        AVPlayerItem _playerItem;
        Video _video;
        ...

        public void UpdateSource()
        {
            AVAsset asset = null;

            if (_video.Source is UriVideoSource)
            {
                string uri = (_video.Source as UriVideoSource).Uri;
                if (!string.IsNullOrEmpty(uri))
                    asset = AVAsset.FromUrl(new NSURL(uri));
            }
            ...

            if (asset != null)
                _playerItem = new AVPlayerItem(asset);
            else
                _playerItem = null;
        }
    }
}
```

```

        _player.ReplaceCurrentItemWithPlayerItem(_playerItem);
        if (_playerItem != null && _video.AutoPlay)
        {
            _player.Play();
        }
        ...
    }
}

```

When processing objects of type `UriVideoSource`, the static `AVAsset.FromUrl` method is used to specify the video to be played, with an iOS `NSURL` object created from the string URI.

The `AutoPlay` property has no equivalent in the iOS video classes, so the property is examined at the end of the `UpdateSource` method to call the `Play` method on the `AVPlayer` object.

In some cases on iOS, videos continue playing after the video playback page has been navigated away from. To stop the video, the `ReplaceCurrentItemWithPlayerItem` is set to `null` in the `Dispose` override:

C#

```

protected override void Dispose(bool disposing)
{
    if (disposing)
    {
        if (_player != null)
        {
            _player.ReplaceCurrentItemWithPlayerItem(null);
            ...
        }
        ...
    }
    base.Dispose(disposing);
}

```

Windows

Video is played on Windows with a `MediaPlayerElement`. The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `UriVideoSource`:

C#

```

using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        bool _isMediaPlayerAttached;
        ...

        public async void UpdateSource()
        {
            bool hasSetSource = false;

            if (_video.Source is UriVideoSource)
            {
                string uri = (_video.Source as UriVideoSource).Uri;
                if (!string.IsNullOrEmpty(uri))
                {
                    _mediaPlayerElement.Source =
MediaSource.CreateFromUri(new Uri(uri));
                    hasSetSource = true;
                }
            }
            ...

            if (hasSetSource && !_isMediaPlayerAttached)
            {
                _isMediaPlayerAttached = true;
                _mediaPlayerElement.MediaPlayer.MediaOpened +=
OnMediaPlayerMediaOpened;
            }

            if (hasSetSource && _video.AutoPlay)
            {
                _mediaPlayerElement.AutoPlay = true;
            }
        }
        ...
    }
}

```

When processing objects of type `UriVideoSource`, the `MediaPlayerElement.Source` property is set to a `MediaSource` object that initializes a `Uri` with the URI of the video to be played. When the `MediaPlayerElement.Source` has been set, the

`OnMediaPlayerMediaOpened` event handler method is registered against the `MediaPlayerElement.MediaPlayer.MediaOpened` event. This event handler is used to set the `Duration` property of the `Video` control.

At the end of the `UpdateSource` method, the `Video.AutoPlay` property is examined and if it's true the `MediaPlayerElement.AutoPlay` property is set to `true` to start video playback.

Play a video resource

The `ResourceVideoSource` class is used to access video files that are embedded in the app. It defines a `Path` property of type `string`:

C#

```
namespace VideoDemos.Controls
{
    public class ResourceVideoSource : VideoSource
    {
        public static readonly BindableProperty PathProperty =
            BindableProperty.Create(nameof(Path), typeof(string),
            typeof(ResourceVideoSource));

        public string Path
        {
            get { return (string)GetValue(PathProperty); }
            set { SetValue(PathProperty, value); }
        }
    }
}
```

When the `Source` property is set to a `ResourceVideoSource`, the handler's property mapper ensures that the `MapSource` method is invoked:

C#

```
public static void MapSource(VideoHandler handler, Video video)
{
    handler?.PlatformView.UpdateSource();
}
```

The `MapSource` method in turns calls the `UpdateSource` method on the handler's `PlatformView` property. The `PlatformView` property, which is of type `MauiVideoPlayer`, represents the native view that provides the video player implementation on each platform.

Android

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `ResourceVideoSource`:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        bool _isPrepared;
        Context _context;
        Video _video;
        ...

        public void UpdateSource()
        {
            _isPrepared = false;
            bool hasSetSource = false;
            ...

            else if (_video.Source is ResourceVideoSource)
            {
                string package = Context.PackageName;
                string path = (_video.Source as ResourceVideoSource).Path;
                if (!string.IsNullOrEmpty(path))
                {
                    string assetFilePath = "content://" + package + "/" +
path;

                    _videoView.SetVideoPath(assetFilePath);
                    hasSetSource = true;
                }
            }
            ...
        }
    }
}
```

When processing objects of type `ResourceVideoSource`, the `SetVideoPath` method of `VideoView` is used to specify the video to be played, with a string argument combining

the app's package name with the video's filename.

A resource video file is stored in the package's *assets* folder, and requires a content provider to access it. The content provider is provided by the `VideoProvider` class, which creates an `AssetFileDescriptor` object that provides access to the video file:

C#

```
using Android.Content;
using Android.Content.Res;
using Android.Database;
using Debug = System.Diagnostics.Debug;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    [ContentProvider(new string[] { "com.companyname.videodemos" })]
    public class VideoProvider : ContentProvider
    {
        public override AssetFileDescriptor OpenAssetFile(Uri uri, string
mode)
        {
            var assets = Context.Assets;
            string fileName = uri.LastPathSegment;
            if (fileName == null)
                throw new FileNotFoundException();

            AssetFileDescriptor afd = null;
            try
            {
                afd = assets.OpenFd(fileName);
            }
            catch (IOException ex)
            {
                Debug.WriteLine(ex);
            }
            return afd;
        }

        public override bool OnCreate()
        {
            return false;
        }
        ...
    }
}
```

iOS and Mac Catalyst

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `ResourceVideoSource`:

C#

```
using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        Video _video;
        ...

        public void UpdateSource()
        {
            AVAsset asset = null;
            ...

            else if (_video.Source is ResourceVideoSource)
            {
                string path = (_video.Source as ResourceVideoSource).Path;
                if (!string.IsNullOrEmpty(path))
                {
                    string directory = Path.GetDirectoryName(path);
                    string filename =
                        Path.GetFileNameWithoutExtension(path);
                    string extension = Path.GetExtension(path).Substring(1);
                    NSURL url =
                        NSBundle.MainBundle.GetUrlForResource(filename, extension, directory);
                    asset = AVAsset.FromUrl(url);
                }
            }
            ...
        }
    }
}
```

When processing objects of type `ResourceVideoSource`, the `GetUrlForResource` method of `NSBundle` is used to retrieve the file from the app package. The complete path must be divided into a filename, extension, and directory.

In some cases on iOS, videos continue playing after the video playback page has been navigated away from. To stop the video, the `ReplaceCurrentItemWithPlayerItem` is set to

`null` in the `Dispose` override:

C#

```
protected override void Dispose(bool disposing)
{
    if (disposing)
    {
        if (_player != null)
        {
            _player.ReplaceCurrentItemWithPlayerItem(null);
            ...
        }
        ...
    }
    base.Dispose(disposing);
}
```

Windows

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `ResourceVideoSource`:

C#

```
using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        ...

        public async void UpdateSource()
        {
            bool hasSetSource = false;

            ...
            else if (_video.Source is ResourceVideoSource)
            {
                string path = "ms-appx:///\" + (_video.Source as
ResourceVideoSource).Path;
                if (!string.IsNullOrEmpty(path))
                {

```

```

        _mediaPlayerElement.Source =
MediaSource.CreateFromUri(new Uri(path));
        hasSetSource = true;
    }
}
...
}
...
}
}

```

When processing objects of type `ResourceVideoSource`, the `MediaPlayerElement.Source` property is set to a `MediaSource` object that initializes a `Uri` with the path of the video resource prefixed with `ms-appx:///`.

Play a video file from the device's library

The `FileVideoSource` class is used to access videos in the device's video library. It defines a `File` property of type `string`:

C#

```

namespace VideoDemos.Controls
{
    public class FileVideoSource : VideoSource
    {
        public static readonly BindableProperty FileProperty =
            BindableProperty.Create(nameof(File), typeof(string),
            typeof(FileVideoSource));

        public string File
        {
            get { return (string)GetValue(FileProperty); }
            set { SetValue(FileProperty, value); }
        }
    }
}

```

When the `Source` property is set to a `FileVideoSource`, the handler's property mapper ensures that the `MapSource` method is invoked:

C#

```

public static void MapSource(VideoHandler handler, Video video)
{
    handler?.PlatformView.UpdateSource();
}

```

The `MapSource` method in turns calls the `UpdateSource` method on the handler's `PlatformView` property. The `PlatformView` property, which is of type `MauiVideoPlayer`, represents the native view that provides the video player implementation on each platform.

Android

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `FileVideoSource`:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        bool _isPrepared;
        Video _video;
        ...

        public void UpdateSource()
        {
            _isPrepared = false;
            bool hasSetSource = false;
            ...

            else if (_video.Source is FileVideoSource)
            {
                string filename = (_video.Source as FileVideoSource).File;
                if (!string.IsNullOrEmpty(filename))
                {
                    _videoView.SetVideoPath(filename);
                    hasSetSource = true;
                }
            }
            ...
        }
    }
}
```

When processing objects of type `FileVideoSource`, the `SetVideoPath` method of `VideoView` is used to specify the video file to be played.

iOS and Mac Catalyst

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `FileVideoSource`:

C#

```
using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        Video _video;
        ...

        public void UpdateSource()
        {
            AVAsset asset = null;
            ...

            else if (_video.Source is FileVideoSource)
            {
                string uri = (_video.Source as FileVideoSource).File;
                if (!string.IsNullOrEmpty(uri))
                    asset = AVAsset.FromUrl(NSUrl.CreateFileUrl(new [] { uri
            }));
        }
        ...
    }
}
```

When processing objects of type `FileVideoSource`, the static `AVAsset.FromUrl` method is used to specify the video file to be played, with the `NSUrl.CreateFileUrl` method creating an iOS `NSUrl` object from the string URI.

Windows

The following code example shows how the `UpdateSource` method processes the `Source` property when it's of type `FileVideoSource`:

C#

```
using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        ...

        public async void UpdateSource()
        {
            bool hasSetSource = false;

            ...
            else if (_video.Source is FileVideoSource)
            {
                string filename = (_video.Source as FileVideoSource).File;
                if (!string.IsNullOrEmpty(filename))
                {
                    StorageFile storageFile = await
StorageFile.GetFileFromPathAsync(filename);
                    _mediaPlayerElement.Source =
MediaSource.CreateFromStorageFile(storageFile);
                    hasSetSource = true;
                }
            }
            ...
        }
    }
}
```

When processing objects of type `FileVideoSource`, the video filename is converted to a `StorageFile` object. Then, the `MediaSource.CreateFromStorageFile` method returns a `MediaSource` object that's set as the value of the `MediaPlayerElement.Source` property.

Loop a video

The `Video` class defines an `IsLooping` property, which enables the control to automatically set the video position to the start after reaching its end. It defaults to `false`, which indicates that videos don't automatically loop.

When the `IsLooping` property is set, the handler's property mapper ensures that the `MapIsLooping` method is invoked:

C#

```
public static void MapIsLooping(VideoHandler handler, Video video)
{
    handler.PlatformView?.UpdateIsLooping();
}
```

The `MapIsLooping` method in turn calls the `UpdateIsLooping` method on the handler's `PlatformView` property. The `PlatformView` property, which is of type `MauiVideoPlayer`, represents the native view that provides the video player implementation on each platform.

Android

The following code example shows how the `UpdateIsLooping` method on Android enables video looping:

C#

```
using Android.Content;
using Android.Media;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout,
        MediaPlayer.IOnPreparedListener
    {
        VideoView _videoView;
        Video _video;
        ...

        public void UpdateIsLooping()
        {
            if (_video.IsLooping)
            {

```

```

        _videoView.SetOnPreparedListener(this);
    }
    else
    {
        _videoView.SetOnPreparedListener(null);
    }
}

public void OnPrepared(MediaPlayer mp)
{
    mp.Looping = _video.IsLooping;
}
...
}
}

```

To enable video looping, the `MauiVideoPlayer` class implements the `MediaPlayer.IOnPreparedListener` interface. This interface defines an `OnPrepared` callback that's invoked when the media source is ready for playback. When the `Video.IsLooping` property is `true`, the `UpdateIsLooping` method sets `MauiVideoPlayer` as the object that provides the `OnPrepared` callback. The callback sets the `MediaPlayer.IsLooping` property to the value of the `Video.IsLooping` property.

iOS and Mac Catalyst

The following code example shows how the `UpdateIsLooping` method on iOS and Mac Catalyst enables video looping:

C#

```

using System.Diagnostics;
using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayer _player;
        AVPlayerViewController _playerViewController;
        Video _video;
        NSObject? _playedToEndObserver;
        ...

        public void UpdateIsLooping()
    }
}

```

```

    {
        DestroyPlayedToEndObserver();
        if (_video.IsLooping)
        {
            _player.ActionAtItemEnd = AVPlayerActionAtItemEnd.None;
            _playedToEndObserver =
                NSNotificationCenter.DefaultCenter.AddObserver(AVPlayerItem.DidPlayToEndTime
                    Notification, PlayedToEnd);
        }
        else
            _player.ActionAtItemEnd = AVPlayerActionAtItemEnd.Pause;
    }

    void PlayedToEnd(NSNotification notification)
    {
        if (_video == null || notification.Object !=
            _playerViewController.Player?.CurrentItem)
            return;

        _playerViewController.Player?.Seek(CMTime.Zero);
    }
    ...
}
}

```

On iOS and Mac Catalyst, a notification is used to execute a callback when the video has been played to the end. When the `Video.IsLooping` property is `true`, the `UpdateIsLooping` method adds an observer for the `AVPlayerItem.DidPlayToEndTimeNotification` notification, and executes the `PlayedToEnd` method when the notification is received. In turn, this method resumes playback from the beginning of the video. If the `Video.IsLooping` property is `false`, the video pauses at the end of playback.

Because `MauiVideoPlayer` adds an observer for a notification it must also remove the observer when performing native view cleanup. This is accomplished in the `Dispose` override:

C#

```

public class MauiVideoPlayer : UIView
{
    AVPlayer _player;
    AVPlayerViewController _playerViewController;
    Video _video;
    NSObject? _playedToEndObserver;
    ...

    protected override void Dispose(bool disposing)
    {
        if (disposing)

```

```

        {
            if (_player != null)
            {
                DestroyPlayedToEndObserver();
                ...
            }
            ...
        }

        base.Dispose(disposing);
    }

    void DestroyPlayedToEndObserver()
    {
        if (_playedToEndObserver != null)
        {
            NSNotificationCenter.DefaultCenter.RemoveObserver(_playedToEndObserver);
            DisposeObserver(ref _playedToEndObserver);
        }
    }

    void DisposeObserver(ref NSObject? disposable)
    {
        disposable?.Dispose();
        disposable = null;
    }
    ...
}

```

The `Dispose` override calls the `DestroyPlayedToEndObserver` method that removes the observer for the `AVPlayerItem.DidPlayToEndTimeNotification` notification, and which also invokes the `Dispose` method on the `NSObject`.

Windows

The following code example shows how the `UpdateIsLooping` method on `Windows` enables video looping:

```

C#

public void UpdateIsLooping()
{
    if (_isMediaPlayerAttached)
        _mediaPlayerElement.MediaPlayer.IsLoopingEnabled = _video.IsLooping;
}

```

To enable video looping, the `UpdateIsLooping` method sets the `MediaPlayerElement.MediaPlayer.IsLoopingEnabled` property to the value of the

`Video.IsLooping` property.

Create custom transport controls

The transport controls of a video player include buttons that play, pause, and stop the video. These buttons are often identified with familiar icons rather than text, and the play and pause buttons are often combined into one button.

By default, the `Video` control displays transport controls supported by each platform. However, when you set the `AreTransportControlsEnabled` property to `false`, these controls are suppressed. You can then control video playback programmatically or supply your own transport controls.

Implementing your own transport controls requires the `Video` class to be able to notify its native views to play, pause, or stop the video, and know the current status of video playback. The `Video` class defines methods named `Play`, `Pause`, and `Stop` that raise a corresponding event, and send a command to the `VideoHandler`:

C#

```
namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        public event EventHandler<VideoPositionEventArgs> PlayRequested;
        public event EventHandler<VideoPositionEventArgs> PauseRequested;
        public event EventHandler<VideoPositionEventArgs> StopRequested;

        public void Play()
        {
            VideoPositionEventArgs args = new
            VideoPositionEventArgs(Position);
            PlayRequested?.Invoke(this, args);
            Handler?.Invoke(nameof(Video.PlayRequested), args);
        }

        public void Pause()
        {
            VideoPositionEventArgs args = new
            VideoPositionEventArgs(Position);
            PauseRequested?.Invoke(this, args);
            Handler?.Invoke(nameof(Video.PauseRequested), args);
        }

        public void Stop()
        {
            VideoPositionEventArgs args = new
```

```

VideoPositionEventArgs(Position);
    StopRequested?.Invoke(this, args);
    Handler?.Invoke(nameof(Video.StopRequested), args);
    }
}
}

```

The `VideoPositionEventArgs` class defines a `Position` property that can be set through its constructor. This property represents the position at which video playback was started, paused, or stopped.

The final line in the `Play`, `Pause`, and `Stop` methods sends a command and associated data to `VideoHandler`. The [CommandMapper](#) for `VideoHandler` maps command names to Actions that are executed when a command is received. For example, when `VideoHandler` receives the `PlayRequested` command, it executes its `MapPlayRequested` method. The advantage of this approach is that it removes the need for native views to subscribe to and unsubscribe from cross-platform control events. In addition, it allows for easy customization because the command mapper can be modified by cross-platform control consumers without subclassing. For more information about [CommandMapper](#), see [Create the command mapper](#).

The `MauiVideoPlayer` implementation on Android, iOS and Mac Catalyst, has `PlayRequested`, `PauseRequested`, and `StopRequested` methods that are executed in response to the `Video` control sending `PlayRequested`, `PauseRequested`, and `StopRequested` commands. Each method invokes a method on its native view to play, pause, or stop the video. For example, the following code shows the `PlayRequested`, `PauseRequested`, and `StopRequested` methods on iOS and Mac Catalyst:

C#

```

using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayer _player;
        ...

        public void PlayRequested(TimeSpan position)
        {

```

```

        _player.Play();
        Debug.WriteLine($"Video playback from {position.Hours:X2}:
{position.Minutes:X2}:{position.Seconds:X2}.");
    }

    public void PauseRequested(TimeSpan position)
    {
        _player.Pause();
        Debug.WriteLine($"Video paused at {position.Hours:X2}:
{position.Minutes:X2}:{position.Seconds:X2}.");
    }

    public void StopRequested(TimeSpan position)
    {
        _player.Pause();
        _player.Seek(new CMTime(0, 1));
        Debug.WriteLine($"Video stopped at {position.Hours:X2}:
{position.Minutes:X2}:{position.Seconds:X2}.");
    }
}

```

Each of the three methods logs the position at which the video was played, paused, or stopped, using the data that's sent with the command.

This mechanism ensures that when the `Play`, `Pause`, or `Stop` method is invoked on the `Video` control, its native view is instructed to play, pause, or stop the video and log the position at which the video was played, paused, or stopped. This all happens using a decoupled approach, without native views having to subscribe to cross-platform events.

Video status

Implementing play, pause, and stop functionality isn't sufficient for supporting custom transport controls. Often the play and pause functionality should be implemented with the same button, which changes its appearance to indicate whether the video is currently playing or paused. In addition, the button shouldn't even be enabled if the video hasn't yet loaded.

These requirements imply that the video player needs to make available a current status indicating if it's playing or paused, or if it's not yet ready to play a video. This status can be represented by an enumeration:

C#

```

public enum VideoStatus
{
    NotReady,
    Playing,

```


Paused

```
}
```

The `Video` class defines a read-only bindable property named `Status` of type `VideoStatus`. This property is defined as read-only because it should only be set from the control's handler:

C#

```
namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        private static readonly BindablePropertyKey StatusPropertyKey =
            BindableProperty.CreateReadOnly(nameof(Status),
            typeof(VideoStatus), typeof(Video), VideoStatus.NotReady);

        public static readonly BindableProperty StatusProperty =
            StatusPropertyKey.BindableProperty;

        public VideoStatus Status
        {
            get { return (VideoStatus)GetValue(StatusProperty); }
        }

        VideoStatus IVideoController.Status
        {
            get { return Status; }
            set { SetValue(StatusPropertyKey, value); }
        }
        ...
    }
}
```

Usually, a read-only bindable property would have a private `set` accessor on the `Status` property to allow it to be set from within the class. However, for a `View` derivative supported by handlers, the property must be set from outside the class but only by the control's handler.

For this reason, another property is defined with the name `IVideoController.Status`. This is an explicit interface implementation, and is made possible by the `IVideoController` interface that the `Video` class implements:

C#

```
public interface IVideoController
{
    VideoStatus Status { get; set; }
```

```
        TimeSpan Duration { get; set; }
    }
```

This interface makes it possible for a class external to `Video` to set the `Status` property by referencing the `IVideoController` interface. The property can be set from other classes and the handler, but it's unlikely to be set inadvertently. Most importantly, the `Status` property can't be set through a data binding.

To assist the handler implementations in keeping the `Status` property updated, the `Video` class defines an `UpdateStatus` event and command:

```
C#

using System.ComponentModel;

namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        public event EventHandler UpdateStatus;

        IDispatcherTimer _timer;

        public Video()
        {
            _timer = Dispatcher.CreateTimer();
            _timer.Interval = TimeSpan.FromMilliseconds(100);
            _timer.Tick += OnTimerTick;
            _timer.Start();
        }

        ~Video() => _timer.Tick -= OnTimerTick;

        void OnTimerTick(object sender, EventArgs e)
        {
            UpdateStatus?.Invoke(this, EventArgs.Empty);
            Handler?.Invoke(nameof(Video.UpdateStatus));
        }
        ...
    }
}
```

The `OnTimerTick` event handler is executed every tenth of a second, which raises the `UpdateStatus` event and invokes the `UpdateStatus` command.

When the `UpdateStatus` command is sent from the `Video` control to its handler, the handler's command mapper ensures that the `MapUpdateStatus` method is invoked:

C#

```
public static void MapUpdateStatus(VideoHandler handler, Video video,
object? args)
{
    handler.PlatformView?.UpdateStatus();
}
```

The `MapUpdateStatus` method in turns calls the `UpdateStatus` method on the handler's `PlatformView` property. The `PlatformView` property, which is of type `MauiVideoPlayer`, encapsulates the native views that provide the video player implementation on each platform.

Android

The following code example shows the `UpdateStatus` method on Android sets the `Status` property:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        bool _isPrepared;
        Video _video;
        ...

        public MauiVideoPlayer(Context context, Video video) : base(context)
        {
            _video = video;
            ...
            _videoView.Prepared += OnVideoViewPrepared;
        }

        protected override void Dispose(bool disposing)
        {
            if (disposing)
            {
                _videoView.Prepared -= OnVideoViewPrepared;
                ...
            }
        }
    }
}
```

```

    }

    base.Dispose(disposing);
}

void OnVideoViewPrepared(object sender, EventArgs args)
{
    _isPrepared = true;
    ((IVideoController)_video).Duration =
    TimeSpan.FromMilliseconds(_videoView.Duration);
}

public void UpdateStatus()
{
    VideoStatus status = VideoStatus.NotReady;

    if (_isPrepared)
        status = _videoView.IsPlaying ? VideoStatus.Playing :
    VideoStatus.Paused;

    ((IVideoController)_video).Status = status;
    ...
}
...
}
}

```

The `VideoView.IsPlaying` property is a Boolean that indicates if the video is playing or paused. To determine if the `VideoView` can't play or pause the video, its `Prepared` event must be handled. This event is raised when the media source is ready for playback. The event is subscribed to in the `MauiVideoPlayer` constructor, and unsubscribed from in its `Dispose` override. The `UpdateStatus` method then uses the `isPrepared` field and the `VideoView.IsPlaying` property to set the `Status` property on the `Video` object by casting it to `IVideoController`.

iOS and Mac Catalyst

The following code example shows the `UpdateStatus` method on iOS and Mac Catalyst sets the `Status` property:

C#

```

using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;

```

```

using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayer _player;
        Video _video;
        ...

        public void UpdateStatus()
        {
            VideoStatus videoStatus = VideoStatus.NotReady;

            switch (_player.Status)
            {
                case AVPlayerStatus.ReadyToPlay:
                    switch (_player.TimeControlStatus)
                    {
                        case AVPlayerTimeControlStatus.Playing:
                            videoStatus = VideoStatus.Playing;
                            break;

                        case AVPlayerTimeControlStatus.Paused:
                            videoStatus = VideoStatus.Paused;
                            break;
                    }
                    break;
            }
            ((IVideoController)_video).Status = videoStatus;
            ...
        }
        ...
    }
}

```

Two properties of `AVPlayer` must be accessed to set the `Status` property - the `Status` property of type `AVPlayerStatus` and the `TimeControlStatus` property of type `AVPlayerTimeControlStatus`. The `Status` property can then be set on the `Video` object by casting it to `IVideoController`.

Windows

The following code example shows the `UpdateStatus` method on Windows sets the `Status` property:

```

C#

using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;

```

```

using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        bool _isMediaPlayerAttached;
        ...

        public void UpdateStatus()
        {
            if (_isMediaPlayerAttached)
            {
                VideoStatus status = VideoStatus.NotReady;

                switch (_mediaPlayerElement.MediaPlayer.CurrentState)
                {
                    case MediaPlayerState.Playing:
                        status = VideoStatus.Playing;
                        break;
                    case MediaPlayerState.Paused:
                    case MediaPlayerState.Stopped:
                        status = VideoStatus.Paused;
                        break;
                }

                ((IVideoController)_video).Status = status;
                _video.Position = _mediaPlayerElement.MediaPlayer.Position;
            }
            ...
        }
    }
}

```

The `UpdateStatus` method uses the value of the `MediaPlayerElement.MediaPlayer.CurrentState` property to determine the value of the `Status` property. The `Status` property can then be set on the `Video` object by casting it to `IVideoController`.

Positioning bar

The transport controls implemented by each platform include a positioning bar. This bar resembles a slider or scroll bar, and shows the current location of the video within its total duration. Users can manipulate the positioning bar to move forwards or backwards to a new position in the video.

Implementing your own positioning bar requires the `Video` class to know the duration of the video, and its current position within that duration.

Duration

One item of information that the `Video` control needs to support a custom positioning bar is the duration of the video. The `Video` class defines a read-only bindable property named `Duration`, of type `TimeSpan`. This property is defined as read-only because it should only be set from the control's handler:

```
C#

namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        private static readonly BindablePropertyKey DurationPropertyKey =
            BindableProperty.CreateReadOnly(nameof(Duration),
            typeof(TimeSpan), typeof(Video), new TimeSpan(),
            propertyChanged: (bindable, oldValue, newValue) =>
            ((Video)bindable).SetTimeToEnd());

        public static readonly BindableProperty DurationProperty =
            DurationPropertyKey.BindableProperty;

        public TimeSpan Duration
        {
            get { return (TimeSpan)GetValue(DurationProperty); }
        }

        TimeSpan IVideoController.Duration
        {
            get { return Duration; }
            set { SetValue(DurationPropertyKey, value); }
        }
        ...
    }
}
```

Usually, a read-only bindable property would have a private `set` accessor on the `Duration` property to allow it to be set from within the class. However, for a `View` derivative supported by handlers, the property must be set from outside the class but only by the control's handler.

🚨 Note

The property-changed event handler for the `Duration` bindable property calls a method named `SetTimeToEnd`, which is described in [Calculating time to end](#).

For this reason, another property is defined with the name `IVideoController.Duration`. This is an explicit interface implementation, and is made possible by the `IVideoController` interface that the `Video` class implements:

C#

```
public interface IVideoController
{
    VideoStatus Status { get; set; }
    TimeSpan Duration { get; set; }
}
```

This interface makes it possible for a class external to `Video` to set the `Duration` property by referencing the `IVideoController` interface. The property can be set from other classes and the handler, but it's unlikely to be set inadvertently. Most importantly, the `Duration` property can't be set through a data binding.

The duration of a video isn't available immediately after the `Source` property of the `Video` control is set. The video must be partially downloaded before the native view can determine its duration.

Android

On Android, the `VideoView.Duration` property reports a valid duration in milliseconds after the `VideoView.Prepared` event has been raised. The `MauiVideoPlayer` class uses the `Prepared` event handler to obtain the `Duration` property value:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
```



```

        Video _video;
        ...

        void OnVideoViewPrepared(object sender, EventArgs args)
        {
            ...
            ((IVideoController)_video).Duration =
            TimeSpan.FromMilliseconds(_videoView.Duration);
        }
        ...
    }
}

```

iOS and Mac Catalyst

On iOS and Mac Catalyst, the duration of a video is obtained from the `AVPlayerItem.Duration` property, but not immediately after the `AVPlayerItem` is created. It's possible to set an iOS observer for the `Duration` property, but the `MauiVideoPlayer` class obtains the duration in the `UpdateStatus` method that's called 10 times a second:

C#

```

using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayerItem _playerItem;
        ...

        TimeSpan ConvertTime(CMTime cmTime)
        {
            return TimeSpan.FromSeconds(Double.IsNaN(cmTime.Seconds) ? 0 :
            cmTime.Seconds);
        }

        public void UpdateStatus()
        {
            ...
            if (_playerItem != null)
            {
                ((IVideoController)_video).Duration =
                ConvertTime(_playerItem.Duration);
                ...
            }
        }
    }
}

```

```

    }
    }
    ...
}

```

The `ConvertTime` method converts a `CMTIME` object to a `TimeSpan` value.

Windows

On Windows, the `MediaPlayerElement.MediaPlayer.NaturalDuration` property is a `TimeSpan` value that becomes valid when the `MediaPlayerElement.MediaPlayer.MediaOpened` event has been raised. The `MauiVideoPlayer` class uses the `MediaOpened` event handler to obtain the `NaturalDuration` property value:

C#

```

using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        bool _isMediaPlayerAttached;
        ...

        void OnMediaPlayerMediaOpened(MediaPlayer sender, object args)
        {
            MainThread.BeginInvokeOnMainThread(() =>
            {
                ((IVideoController)_video).Duration =
                _mediaPlayerElement.MediaPlayer.NaturalDuration;
            });
        }
        ...
    }
}

```

The `OnMediaPlayer` event handler then calls the `MainThread.BeginInvokeOnMainThread` method to set the `Duration` property on the `Video` object, by casting it to

`IVideoController`, on the main thread. This is necessary because the `MediaPlayerElement.MediaPlayer.MediaOpened` event is handled on a background thread. For more information about running code on the main thread, see [Create a thread on the .NET MAUI UI thread](#).

Position

The `Video` control also needs a `Position` property that increases from zero to `Duration` as the video plays. The `Video` class implements this property as a bindable property with public `get` and `set` accessors:

```
C#

namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        public static readonly BindableProperty PositionProperty =
            BindableProperty.Create(nameof(Position), typeof(TimeSpan),
            typeof(Video), new TimeSpan(),
            propertyChanged: (bindable, oldValue, newValue) =>
            ((Video)bindable).SetTimeToEnd());

        public TimeSpan Position
        {
            get { return (TimeSpan)GetValue(PositionProperty); }
            set { SetValue(PositionProperty, value); }
        }
        ...
    }
}
```

The `get` accessor returns the current position of the video as its playing. The `set` accessor responds to user manipulation of the positioning bar by moving the video position forwards or backwards.

ⓘ Note

The property-changed event handler for the `Position` bindable property calls a method named `SetTimeToEnd`, which is described in [Calculating time to end](#).

On Android, iOS and Mac Catalyst, the property that obtains the current position only has a `get` accessor. Instead, a `Seek` method is available to set the position. This seems to be a more sensible approach than using a single `Position` property, which has an

inherent problem. As a video plays, a `Position` property must be continually updated to reflect the new position. But you don't want most changes of the `Position` property to cause the video player to move to a new position in the video. If that happens, the video player would respond by seeking to the last value of the `Position` property, and the video wouldn't advance.

Despite the difficulties of implementing a `Position` property with `get` and `set` accessors, this approach is used because it can utilize data binding. The `Position` property of the `Video` control can be bound to a [Slider](#) that's used both to display the position and to seek a new position. However, several precautions are necessary when implementing the `Position` property, to avoid feedback loops.

Android

On Android, the `VideoView.CurrentPosition` property indicates the current position of the video. The `MauiVideoPlayer` class sets the `Position` property in the `UpdateStatus` method at the same time as it sets the `Duration` property:

C#

```
using Android.Content;
using Android.Views;
using Android.Widget;
using AndroidX.CoordinatorLayout.Widget;
using VideoDemos.Controls;
using Color = Android.Graphics.Color;
using Uri = Android.Net.Uri;

namespace VideoDemos.Platforms.Android
{
    public class MauiVideoPlayer : CoordinatorLayout
    {
        VideoView _videoView;
        Video _video;
        ...

        public void UpdateStatus()
        {
            ...
            TimeSpan timeSpan =
            TimeSpan.FromMilliseconds(_videoView.CurrentPosition);
            _video.Position = timeSpan;
        }

        public void UpdatePosition()
        {
            if (Math.Abs(_videoView.CurrentPosition -
            _video.Position.TotalMilliseconds) > 1000)
```

```

        {
            _videoView.SeekTo((int)_video.Position.TotalMilliseconds);
        }
    }
    ...
}
}

```

Every time the `Position` property is set by the `UpdateStatus` method, the `Position` property fires a `PropertyChanged` event, which causes the property mapper for the handler to call the `UpdatePosition` method. The `UpdatePosition` method should do nothing for most of the property changes. Otherwise, with every change in the video's position it would be moved to same position it just reached. To avoid this feedback loop, the `UpdatePosition` only calls the `Seek` method on the `VideoView` object when the difference between the `Position` property and the current position of the `VideoView` is greater than one second.

iOS and Mac Catalyst

On iOS and Mac Catalyst, the `AVPlayerItem.CurrentTime` property indicates the current position of the video. The `MauiVideoPlayer` class sets the `Position` property in the `UpdateStatus` method at the same time as it sets the `Duration` property:

```

C#

using AVFoundation;
using AVKit;
using CoreMedia;
using Foundation;
using System.Diagnostics;
using UIKit;
using VideoDemos.Controls;

namespace VideoDemos.Platforms.MaciOS
{
    public class MauiVideoPlayer : UIView
    {
        AVPlayer _player;
        AVPlayerItem _playerItem;
        Video _video;
        ...

        TimeSpan ConvertTime(CMTime cmTime)
        {
            return TimeSpan.FromSeconds(Double.IsNaN(cmTime.Seconds) ? 0 :
cmTime.Seconds);
        }
    }
}

```

```

    public void UpdateStatus()
    {
        ...
        if (_playerItem != null)
        {
            ...
            _video.Position = ConvertTime(_playerItem.CurrentTime);
        }
    }

    public void UpdatePosition()
    {
        TimeSpan controlPosition = ConvertTime(_player.CurrentTime);
        if (Math.Abs((controlPosition - _video.Position).TotalSeconds) >
1)
        {
            _player.Seek(CMTime.FromSeconds(_video.Position.TotalSeconds, 1));
        }
        ...
    }
}

```

Every time the `Position` property is set by the `UpdateStatus` method, the `Position` property fires a `PropertyChanged` event, which causes the property mapper for the handler to call the `UpdatePosition` method. The `UpdatePosition` method should do nothing for most of the property changes. Otherwise, with every change in the video's position it would be moved to same position it just reached. To avoid this feedback loop, the `UpdatePosition` only calls the `Seek` method on the `AVPlayer` object when the difference between the `Position` property and the current position of the `AVPlayer` is greater than one second.

Windows

On Windows, the `MediaPlayerElement.MedeaPlayer.Position` property indicates the current position of the video. The `MauiVideoPlayer` class sets the `Position` property in the `UpdateStatus` method at the same time as it sets the `Duration` property:

C#

```

using Microsoft.UI.Xaml.Controls;
using VideoDemos.Controls;
using Windows.Media.Core;
using Windows.Media.Playback;
using Windows.Storage;
using Grid = Microsoft.UI.Xaml.Controls.Grid;

```

```

namespace VideoDemos.Platforms.Windows
{
    public class MauiVideoPlayer : Grid, IDisposable
    {
        MediaPlayerElement _mediaPlayerElement;
        Video _video;
        bool _isMediaPlayerAttached;
        ...

        public void UpdateStatus()
        {
            if (_isMediaPlayerAttached)
            {
                ...
                _video.Position = _mediaPlayerElement.MediaPlayer.Position;
            }
        }

        public void UpdatePosition()
        {
            if (_isMediaPlayerAttached)
            {
                if (Math.Abs((_mediaPlayerElement.MediaPlayer.Position -
                _video.Position).TotalSeconds) > 1)
                {
                    _mediaPlayerElement.MediaPlayer.Position =
                _video.Position;
                }
            }
        }
        ...
    }
}

```

Every time the `Position` property is set by the `UpdateStatus` method, the `Position` property fires a `PropertyChanged` event, which causes the property mapper for the handler to call the `UpdatePosition` method. The `UpdatePosition` method should do nothing for most of the property changes. Otherwise, with every change in the video's position it would be moved to same position it just reached. To avoid this feedback loop, the `UpdatePosition` only sets the `MediaPlayerElement.MediaPlayer.Position` property when the difference between the `Position` property and the current position of the `MediaPlayerElement` is greater than one second.

Calculating time to end

Sometimes video players show the time remaining in the video. This value begins at the video's duration when the video begins, and decreases down to zero when the video ends.

The `Video` class includes a read-only `TimeToEnd` property that's calculated based on changes to the `Duration` and `Position` properties:

C#

```
namespace VideoDemos.Controls
{
    public class Video : View, IVideoController
    {
        ...
        private static readonly BindablePropertyKey TimeToEndPropertyKey =
            BindableProperty.CreateReadOnly(nameof(TimeToEnd),
            typeof(TimeSpan), typeof(Video), new TimeSpan());

        public static readonly BindableProperty TimeToEndProperty =
            TimeToEndPropertyKey.BindableProperty;

        public TimeSpan TimeToEnd
        {
            get { return (TimeSpan)GetValue(TimeToEndProperty); }
            private set { SetValue(TimeToEndPropertyKey, value); }
        }

        void SetTimeToEnd()
        {
            TimeToEnd = Duration - Position;
        }
        ...
    }
}
```

The `SetTimeToEnd` method is called from the property-changed event handlers of the `Duration` and `Position` properties.

Custom positioning bar

A custom positioning bar can be implemented by creating a class that derives from `Slider`, which contains `Duration` and `Position` properties of type `TimeSpan`:

C#

```
namespace VideoDemos.Controls
{
    public class PositionSlider : Slider
    {
        public static readonly BindableProperty DurationProperty =
            BindableProperty.Create(nameof(Duration), typeof(TimeSpan),
            typeof(PositionSlider), new TimeSpan(1),
            propertyChanged: (bindable, oldValue, newValue) =>
```



```

        {
            double seconds = ((TimeSpan)newValue).TotalSeconds;
            ((Slider)bindable).Maximum = seconds <= 0 ? 1 : seconds;
        });

        public static readonly BindableProperty PositionProperty =
            BindableProperty.Create(nameof(Position), typeof(TimeSpan),
            typeof(PositionSlider), new TimeSpan(0),
            defaultBindingMode: BindingMode.TwoWay,
            propertyChanged: (bindable, oldValue, newValue) =>
            {
                double seconds = ((TimeSpan)newValue).TotalSeconds;
                ((Slider)bindable).Value = seconds;
            });

        public TimeSpan Duration
        {
            get { return (TimeSpan)GetValue(DurationProperty); }
            set { SetValue(DurationProperty, value); }
        }

        public TimeSpan Position
        {
            get { return (TimeSpan)GetValue(PositionProperty); }
            set { SetValue (PositionProperty, value); }
        }

        public PositionSlider()
        {
            PropertyChanged += (sender, args) =>
            {
                if (args.PropertyName == "Value")
                {
                    TimeSpan newPosition = TimeSpan.FromSeconds(Value);
                    if (Math.Abs(newPosition.TotalSeconds -
                    Position.TotalSeconds) / Duration.TotalSeconds > 0.01)
                        Position = newPosition;
                }
            }
        }
    }
}

```

The property-changed event handler for the `Duration` property sets the `Maximum` property of the `Slider` to the `TotalSeconds` property of the `TimeSpan` value. Similarly, the property-changed event handler for the `Position` property sets the `Value` property of the `Slider`. This is the mechanism by which the `Slider` tracks the position of `PositionSlider`.

The `PositionSlider` is updated from the underlying `Slider` in only one scenario, which is when the user manipulates the `Slider` to indicate that the video should be advanced or

reversed to a new position. This is detected in the `PropertyChanged` handler in the `PositionSlider` constructor. This event handler checks for a change in the `Value` property, and if it's different from the `Position` property, then the `Position` property is set from the `Value` property.

Register the handler

A custom control and its handler must be registered with an app, before it can be consumed. This should occur in the `CreateMauiApp` method in the `MauiProgram` class in your app project, which is the cross-platform entry point for the app:

C#

```
using VideoDemos.Controls;
using VideoDemos.Handlers;

namespace VideoDemos;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
            })
            .ConfigureMauiHandlers(handlers =>
            {
                handlers.AddHandler(typeof(Video), typeof(VideoHandler));
            });

        return builder.Build();
    }
}
```

The handler is registered with the `ConfigureMauiHandlers` and `AddHandler` method. The first argument to the `AddHandler` method is the cross-platform control type, with the second argument being its handler type.

Consume the cross-platform control

After registering the handler with your app, the cross-platform control can be consumed.

Play a web video

The `Video` control can play a video from a URL, as shown in the following example:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:controls="clr-namespace:VideoDemos.Controls"
  x:Class="VideoDemos.Views.PlayWebVideoPage"
  Unloaded="OnContentPageUnloaded"
  Title="Play web video">
  <controls:Video x:Name="video"

  Source="https://archive.org/download/BigBuckBunny_328/BigBuckBunny_512kb.mp4"
  />
</ContentPage>
```

In this example, the `VideoSourceConverter` class converts the string that represents the URI to a `UriVideoSource`. The video then begins loading and starts playing once a sufficient quantity of data has been downloaded and buffered. On each platform, the transport controls fade out if they're not used but can be restored by tapping on the video.

Play a video resource

Video files that are embedded in the *Resources\Raw* folder of the app, with a **MauiAsset** build action, can be played by the `Video` control:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:controls="clr-namespace:VideoDemos.Controls"
  x:Class="VideoDemos.Views.PlayVideoResourcePage"
  Unloaded="OnContentPageUnloaded"
  Title="Play video resource">
  <controls:Video x:Name="video"
    Source="video.mp4" />
</ContentPage>
```

In this example, the `VideoSourceConverter` class converts the string that represents the filename of the video to a `ResourceVideoSource`. For each platform, the video begins playing almost immediately after the video source is set because the file is in the app package and doesn't need to be downloaded. On each platform, the transport controls fade out if they're not used but can be restored by tapping on the video.

Play a video file from the device's library

Video files that are stored on the device can be retrieved and then played by the `Video` control:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-namespace:VideoDemos.Controls"
             x:Class="VideoDemos.Views.PlayLibraryVideoPage"
             Unloaded="OnContentPageUnloaded"
             Title="Play library video">
    <Grid RowDefinitions="*,Auto">
        <controls:Video x:Name="video" />
        <Button Grid.Row="1"
                Text="Show Video Library"
                Margin="10"
                HorizontalOptions="Center"
                Clicked="OnShowVideoLibraryClicked" />
    </Grid>
</ContentPage>
```

When the `Button` is tapped its `Clicked` event handler is executed, which is shown in the following code example:

C#

```
async void OnShowVideoLibraryClicked(object sender, EventArgs e)
{
    Button button = sender as Button;
    button.IsEnabled = false;

    var pickedVideo = await MediaPicker.PickVideoAsync();
    if (!string.IsNullOrEmpty(pickedVideo?.FileName))
    {
        video.Source = new FileVideoSource
        {
            File = pickedVideo.FullPath
        };
    }
}
```

```
button.IsEnabled = true;  
}
```

The `Clicked` event handler uses .NET MAUI's `MediaPicker` class to let the user pick a video file from the device. The picked video file is then encapsulated as a `FileVideoSource` object and set as the `Source` property of the `Video` control. For more information about the `MediaPicker` class, see [Media picker](#). For each platform, the video begins playing almost immediately after the video source is set because the file is on the device and doesn't need to be downloaded. On each platform, the transport controls fade out if they're not used but can be restored by tapping on the video.

Configure the Video control

You can prevent a video from automatically starting by setting the `AutoPlay` property to `false`:

XAML

```
<controls:Video x:Name="video"  
Source="https://archive.org/download/BigBuckBunny_328/BigBuckBunny_512kb.mp4"  
AutoPlay="False" />
```

You can suppress the transport controls by setting the `AreTransportControlsEnabled` property to `false`:

XAML

```
<controls:Video x:Name="video"  
Source="https://archive.org/download/BigBuckBunny_328/BigBuckBunny_512kb.mp4"  
AreTransportControlsEnabled="False" />
```

If you set `AutoPlay` and `AreTransportControlsEnabled` to `false`, the video won't begin playing and there will be no way to start it playing. In this scenario you'd need to call the `Play` method from the code-behind file, or create your own transport controls.

In addition, you can set a video to loop by setting the `IsLooping` property to `true`:

XAML

```

<controls:Video x:Name="video"

Source="https://archive.org/download/BigBuckBunny_328/BigBuckBunny_512kb.mp4
"

IsLooping="true" />

```

If you set the `IsLooping` property to `true` this ensures that the `Video` control automatically sets the video position to the start after reaching its end.

Use custom transport controls

The following XAML example shows custom transport controls that play, pause, and stop the video:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
xmlns:controls="clr-namespace:VideoDemos.Controls"
x:Class="VideoDemos.Views.CustomTransportPage"
Unloaded="OnContentPageUnloaded"
Title="Custom transport controls">
    <Grid RowDefinitions="*,Auto">
        <controls:Video x:Name="video"
            AutoPlay="False"
            AreTransportControlsEnabled="False"

Source="https://archive.org/download/BigBuckBunny_328/BigBuckBunny_512kb.mp4
" />

        <ActivityIndicator Color="Gray"
            IsVisible="False">
            <ActivityIndicator.Triggers>
                <DataTrigger TargetType="ActivityIndicator"
                    Binding="{Binding Source={x:Reference video},
                        Path=Status}"
                    Value="{x:Static
controls:VideoStatus.NotReady}">
                    <Setter Property="IsVisible"
                        Value="True" />
                    <Setter Property="IsRunning"
                        Value="True" />
                </DataTrigger>
            </ActivityIndicator.Triggers>
        </ActivityIndicator>
        <Grid Grid.Row="1"
            Margin="0,10"
            ColumnDefinitions="0.5*,0.5*"
            BindingContext="{x:Reference video}">
            <Button Text="⏮️⏪⏩⏭; Play"
                HorizontalOptions="Center"

```

```

        Clicked="OnPlayPauseButtonClicked">
    <Button.Triggers>
        <DataTrigger TargetType="Button"
            Binding="{Binding Status}"
            Value="{x:Static
controls:VideoStatus.Playing}">
            <Setter Property="Text"
                Value="⏸; Pause" />
        </DataTrigger>
        <DataTrigger TargetType="Button"
            Binding="{Binding Status}"
            Value="{x:Static
controls:VideoStatus.NotReady}">
            <Setter Property="IsEnabled"
                Value="False" />
        </DataTrigger>
    </Button.Triggers>
</Button>
<Button Grid.Column="1"
    Text="⏹; Stop"
    HorizontalOptions="Center"
    Clicked="OnStopButtonClicked">
    <Button.Triggers>
        <DataTrigger TargetType="Button"
            Binding="{Binding Status}"
            Value="{x:Static
controls:VideoStatus.NotReady}">
            <Setter Property="IsEnabled"
                Value="False" />
        </DataTrigger>
    </Button.Triggers>
</Button>
</Grid>
</Grid>
</ContentPage>

```

In this example, the `Video` control sets the `AreTransportControlsEnabled` property to `false` and defines a `Button` that plays and pauses the video, and a `Button` that stop video playback. Button appearance is defined using unicode characters and their text equivalents, to create buttons that consist of an icon and text:



When the video is playing, the play button is updated to a pause button:



The UI also includes an [ActivityIndicator](#) that's displayed while the video is loading. Data triggers are used to enable and disable the [ActivityIndicator](#) and the buttons, and to switch the first button between play and pause. For more information about data triggers, see [Data triggers](#).

The code-behind file defines the event handlers for the button `Clicked` events:

C#

```
public partial class CustomTransportPage : ContentPage
{
    ...
    void OnPlayPauseButtonClicked(object sender, EventArgs args)
    {
        if (video.Status == VideoStatus.Playing)
        {
            video.Pause();
        }
        else if (video.Status == VideoStatus.Paused)
        {
            video.Play();
        }
    }

    void OnStopButtonClicked(object sender, EventArgs args)
    {
        video.Stop();
    }
    ...
}
```

Custom positioning bar

The following example shows a custom positioning bar, `PositionSlider`, being consumed in XAML:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:controls="clr-namespace:VideoDemos.Controls"
             x:Class="VideoDemos.Views.CustomPositionBarPage"
             Unloaded="OnContentPageUnloaded"
             Title="Custom position bar">
    <Grid RowDefinitions="*,Auto,Auto">
        <controls:Video x:Name="video"
                       AreTransportControlsEnabled="False"
                       Source="{StaticResource ElephantsDream}" />
        ...
    </Grid>
</ContentPage>
```



```

<Grid Grid.Row="1"
      Margin="10,0"
      ColumnDefinitions="0.25*,0.25*,0.25*,0.25*"
      BindingContext="{x:Reference video}">
  <Label Text="{Binding Path=Position,
                      StringFormat='{0:hh\\:mm\\:ss}'}"
        HorizontalOptions="Center"
        VerticalOptions="Center" />

  ...
  <Label Grid.Column="3"
        Text="{Binding Path=TimeToEnd,
                      StringFormat='{0:hh\\:mm\\:ss}'}"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
</Grid>
<controls:PositionSlider Grid.Row="2"
                        Margin="10,0,10,10"
                        BindingContext="{x:Reference video}"
                        Duration="{Binding Duration}"
                        Position="{Binding Position}">
  <controls:PositionSlider.Triggers>
    <DataTrigger TargetType="controls:PositionSlider"
                  Binding="{Binding Status}"
                  Value="{x:Static
controls:VideoStatus.NotReady}">
      <Setter Property="IsEnabled"
              Value="False" />
    </DataTrigger>
  </controls:PositionSlider.Triggers>
</controls:PositionSlider>
</Grid>
</ContentPage>

```

The `Position` property of the `Video` object is bound to the `Position` property of the `PositionSlider`, without performance issues, because the `Video.Position` property is changed by the `MauiVideoPlayer.UpdateStatus` method on each platform, which is only called 10 times a second. In addition, two `Label` objects display the `Position` and `TimeToEnd` properties values from the `Video` object.

Control handler disconnection

Each platform's handler implementation overrides the `DisconnectHandler` implementation, which is used to perform native view cleanup such as unsubscribing from events and disposing objects. By default, handlers automatically disconnect from their controls when possible, such as when navigating backwards in an app.

In some scenarios you might want to control when a handler disconnects from its control, which can be achieved with the `HandlerProperties.DisconnectPolicy` attached

property. This property requires a [HandlerDisconnectPolicy](#) argument, with the enumeration defining the following values:

- `Automatic`, which indicates that the handler will be disconnected automatically. This is the default value of the [HandlerProperties.DisconnectPolicy](#) attached property.
- `Manual`, which indicates that the handler will have to be disconnected manually by invoking the [DisconnectHandler\(\)](#) implementation.

The following example shows setting the [HandlerProperties.DisconnectPolicy](#) attached property:

XAML

```
<controls:Video x:Name="video"
    HandlerProperties.DisconnectPolicy="Manual"
    Source="video.mp4"
    AutoPlay="False" />
```

The equivalent C# code is:

C#

```
Video video = new Video
{
    Source = "video.mp4",
    AutoPlay = false
};
HandlerProperties.SetDisconnectPolicy(video,
HandlerDisconnectPolicy.Manual);
```

When setting the [HandlerProperties.DisconnectPolicy](#) attached property to `Manual` you must invoke the handler's [DisconnectHandler](#) implementation yourself from a suitable location in your app's lifecycle. This can be achieved by invoking `video.Handler?.DisconnectHandler();`.

In addition, there's a [DisconnectHandlers](#) extension method that disconnects handlers from a given [IView](#):

C#

```
video.DisconnectHandlers();
```

When disconnecting, the [DisconnectHandlers](#) method will propagate down the control tree until it completes or arrives at a control that has set a manual policy.

Customize controls with handlers

Article • 05/03/2024

 [Browse the sample](#)

Handlers can be customized to augment the appearance and behavior of a cross-platform control beyond the customization that's possible through the control's API. This customization, which modifies the native views for the cross-platform control, is achieved by modifying the mapper for a handler with one of the following methods:

- [PrependToMapping](#), which modifies the mapper for a handler before the .NET MAUI control mappings have been applied.
- [ModifyMapping](#), which modifies an existing mapping.
- [AppendToMapping](#), which modifies the mapper for a handler after the .NET MAUI control mappings have been applied.

Each of these methods has an identical signature that requires two arguments:

- A `string`-based key. When modifying one of the mappings provided by .NET MAUI, the key used by .NET MAUI must be specified. The key values used by .NET MAUI control mappings are based on interface and property names, for example `nameof(IEntry.IsPassword)`. The interfaces, and their properties, that abstract each cross-platform control can be found [here](#) . This is the key format that should be used if you want your handler customization to run every time a property changes. Otherwise, the key can be an arbitrary value that doesn't have to correspond to the name of a property exposed by a type. For example, `MyCustomization` can be specified as a key, with any native view modification being performed as the customization. However, a consequence of this key format is that your handler customization will only be run when the mapper for the handler is first modified.
- An [Action](#) that represents the method that performs the handler customization. The [Action](#) specifies two arguments:
 - A `handler` argument that provides an instance of the handler being customized.
 - A `view` argument that provides an instance of the cross-platform control that the handler implements.

Important

Handler customizations are global and aren't scoped to a specific control instance. Handler customization is allowed to happen anywhere in your app. Once a handler is customized, it affects all controls of that type, everywhere in your app.

Each handler class exposes the native view for the cross-platform control via its [PlatformView](#) property. This property can be accessed to set native view properties, invoke native view methods, and subscribe to native view events. In addition, the cross-platform control implemented by the handler is exposed via its [VirtualView](#) property.

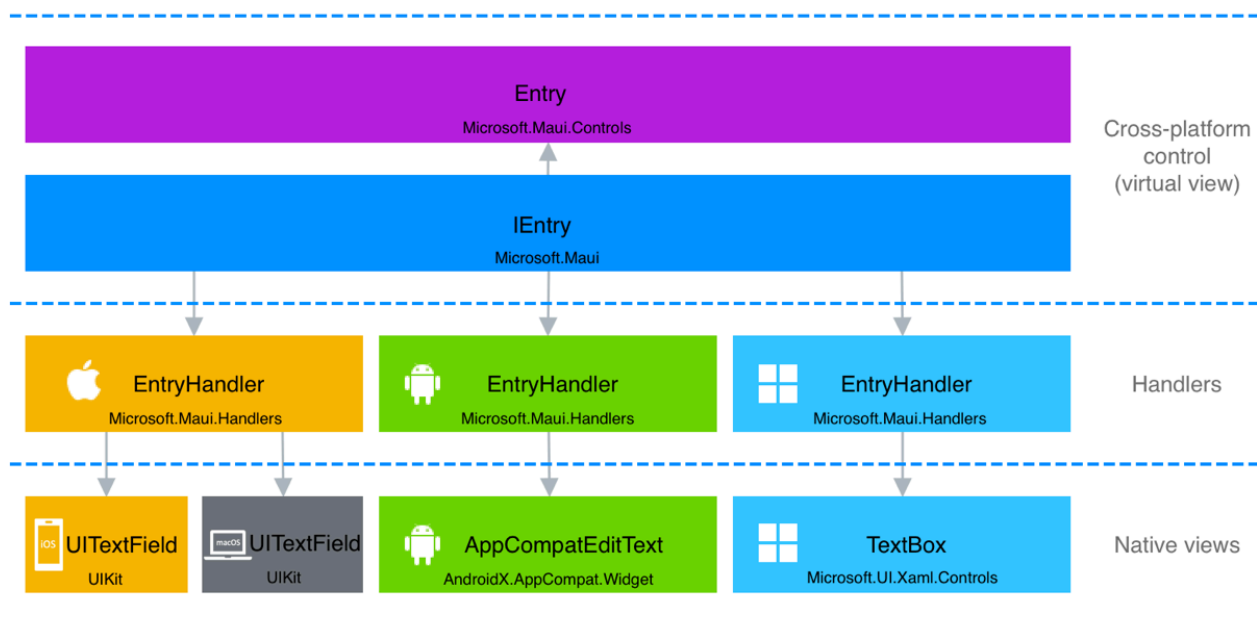
Handlers can be customized per platform by using conditional compilation, to multi-target code based on the platform. Alternatively, you can use partial classes to organize your code into platform-specific folders and files. For more information about conditional compilation, see [Conditional compilation](#).

Customize a control

The .NET MAUI [Entry](#) view is a single-line text input control, that implements the [IEntry](#) interface. The [EntryHandler](#) maps the [Entry](#) view to the following native views for each platform:

- **iOS/Mac Catalyst:** `UITextField`
- **Android:** `AppCompatActivity`
- **Windows:** `TextBox`

The following diagrams shows how the [Entry](#) view is mapped to its native views via the [EntryHandler](#):



The [Entry](#) property mapper, in the [EntryHandler](#) class, maps the cross-platform control properties to the native view API. This ensures that when a property is set on an [Entry](#), the underlying native view is updated as required.

The property mapper can be modified to customize [Entry](#) on each platform:

C#

```
namespace CustomizeHandlersDemo.Views;

public partial class CustomizeEntryPage : ContentPage
{
    public CustomizeEntryPage()
    {
        InitializeComponent();
        ModifyEntry();
    }

    void ModifyEntry()
    {
        Microsoft.Maui.Handlers.EntryHandler.Mapper.AppendToMapping("MyCustomization", (handler, view) =>
        {
            #if ANDROID
                handler.PlatformView.SetSelectAllOnFocus(true);
            #elif IOS || MACCATALYST
                handler.PlatformView.EditingDidBegin += (s, e) =>
                {
                    handler.PlatformView.PerformSelector(new
ObjCRuntime.Selector("selectAll"), null, 0.0f);
                };
            #elif WINDOWS
                handler.PlatformView.GotFocus += (s, e) =>
                {
                    handler.PlatformView.SelectAll();
                };
            #endif
        }));
    }
}
```

In this example, the [Entry](#) customization occurs in a page class. Therefore, all [Entry](#) controls on Android, iOS, and Windows will be customized once an instance of the `CustomizeEntryPage` is created. Customization is performed by accessing the handlers [PlatformView](#) property, which provides access to the native view that maps to the cross-platform control on each platform. Native code then customizes the handler by selecting all of the text in the [Entry](#) when it gains focus.

For more information about mappers, see [Mappers](#).

Customize a specific control instance

Handlers are global, and customizing a handler for a control will result in all controls of the same type being customized in your app. However, handlers for specific control

instances can be customized by subclassing the control, and then by modifying the handler for the base control type only when the control is of the subclassed type. For example, to customize a specific [Entry](#) control on a page that contains multiple [Entry](#) controls, you should first subclass the [Entry](#) control:

```
C#

namespace CustomizeHandlersDemo.Controls
{
    internal class MyEntry : Entry
    {
    }
}
```

You can then customize the `EntryHandler`, via its property mapper, to perform the desired modification only to `MyEntry` instances:

```
C#

Microsoft.Maui.Handlers.EntryHandler.Mapper.AppendToMapping("MyCustomization", (handler, view) =>
{
    if (view is MyEntry)
    {
        #if ANDROID
            handler.PlatformView.SetSelectAllOnFocus(true);
        #elif IOS || MACCATALYST
            handler.PlatformView.EditingDidBegin += (s, e) =>
            {
                handler.PlatformView.PerformSelector(new
ObjCRuntime.Selector("selectAll"), null, 0.0f);
            };
        #elif WINDOWS
            handler.PlatformView.GotFocus += (s, e) =>
            {
                handler.PlatformView.SelectAll();
            };
        #endif
    }
});
```

If the handler customization is performed in your `App` class, any `MyEntry` instances in the app will be customized as per the handler modification.

Customize a control using the handler lifecycle

All handler-based .NET MAUI controls support [HandlerChanging](#) and [HandlerChanged](#) events. The [HandlerChanged](#) event is raised when the native view that implements the cross-platform control is available and initialized. The [HandlerChanging](#) event is raised when the control's handler is about to be removed from the cross-platform control. For more information about handler lifecycle events, see [Handler lifecycle](#).

The handler lifecycle can be used to perform handler customization. For example, to subscribe to, and unsubscribe from, native view events you must register event handlers for the [HandlerChanged](#) and [HandlerChanging](#) events on the cross-platform control being customized:

XAML

```
<Entry HandlerChanged="OnEntryHandlerChanged"
        HandlerChanging="OnEntryHandlerChanging" />
```

Handlers can be customized per platform by using conditional compilation, or by using partial classes to organize your code into platform-specific folders and files. Each approach will be discussed in turn, by customizing an [Entry](#) so that all of its text is selected when it gains focus.

Conditional compilation

The code-behind file containing the event handlers for the [HandlerChanged](#) and [HandlerChanging](#) events is shown in the following example, which uses conditional compilation:

C#

```
#if ANDROID
using AndroidX.AppCompat.Widget;
#elif IOS || MACCATALYST
using UIKit;
#elif WINDOWS
using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml;
#endif

namespace CustomizeHandlersDemo.Views;

public partial class CustomizeEntryHandlerLifecyclePage : ContentPage
{
    public CustomizeEntryHandlerLifecyclePage()
    {
        InitializeComponent();
    }
}
```



```

    void OnEntryHandlerChanged(object sender, EventArgs e)
    {
        Entry entry = sender as Entry;
#if ANDROID
        (entry.Handler.PlatformView as
AppCompatActivity).SetSelectAllOnFocus(true);
#elif IOS || MACCATALYST
        (entry.Handler.PlatformView as UITextField).EditingDidBegin +=
OnEditingDidBegin;
#elif WINDOWS
        (entry.Handler.PlatformView as TextBox).GotFocus += OnGotFocus;
#endif
    }

    void OnEntryHandlerChanging(object sender, HandlerChangingEventArgs e)
    {
        if (e.OldHandler != null)
        {
#if IOS || MACCATALYST
            (e.OldHandler.PlatformView as UITextField).EditingDidBegin -=
OnEditingDidBegin;
#elif WINDOWS
            (e.OldHandler.PlatformView as TextBox).GotFocus -= OnGotFocus;
#endif
        }
    }

#if IOS || MACCATALYST
    void OnEditingDidBegin(object sender, EventArgs e)
    {
        var nativeView = sender as UITextField;
        nativeView.PerformSelector(new ObjCRuntime.Selector("selectAll"),
null, 0.0f);
    }
#elif WINDOWS
    void OnGotFocus(object sender, RoutedEventArgs e)
    {
        var nativeView = sender as TextBox;
        nativeView.SelectAll();
    }
#endif
}

```

The [HandlerChanged](#) event is raised after the native view that implements the cross-platform control has been created and initialized. Therefore, its event handler is where native event subscriptions should be performed. This requires casting the [PlatformView](#) property of the handler to the type, or base type, of the native view so that native events can be accessed. In this example, on iOS, Mac Catalyst, and Windows, the `OnEntryHandlerChanged` event subscribes to native view events that are raised when the native views that implement the [Entry](#) gain focus.

The `OnEditingDidBegin` and `OnGotFocus` event handlers access the native view for the [Entry](#) on their respective platforms, and select all text that's in the [Entry](#).

The [HandlerChanging](#) event is raised before the existing handler is removed from the cross-platform control, and before the new handler for the cross-platform control is created. Therefore, its event handler is where native event subscriptions should be removed, and other cleanup should be performed. The [HandlerChangingEventArgs](#) object that accompanies this event has [OldHandler](#) and [NewHandler](#) properties, which will be set to the old and new handlers respectively. In this example, the `OnEntryHandlerChanging` event removes the subscription to the native view events on iOS, Mac Catalyst, and Windows.

Partial classes

Rather than using conditional compilation, it's also possible to use partial classes to organize your control customization code into platform-specific folders and files. With this approach, your customization code is separated into a cross-platform partial class and a platform-specific partial class:

- The cross-platform partial class typically defines members, but doesn't implement them, and is built for all platforms. This class shouldn't be placed in any of the *Platforms* child folders of your project, because doing so would make it a platform-specific class.
- The platform-specific partial class typically implements the members defined in the cross-platform partial class, and is built for a single platform. This class should be placed in the child folder of the *Platforms* folder for your chosen platform.

The following example shows a cross-platform partial class:

```
C#

namespace CustomizeHandlersDemo.Views;

public partial class CustomizeEntryPartialMethodsPage : ContentPage
{
    public CustomizeEntryPartialMethodsPage()
    {
        InitializeComponent();
    }

    partial void ChangedHandler(object sender, EventArgs e);
    partial void ChangingHandler(object sender, HandlerChangingEventArgs e);

    void OnEntryHandlerChanged(object sender, EventArgs e) =>
        ChangedHandler(sender, e);
    void OnEntryHandlerChanging(object sender, HandlerChangingEventArgs e)
```

```
=> ChangingHandler(sender, e);  
}
```

In this example, the two event handlers call partial methods named `ChangedHandler` and `ChangingHandler`, whose signatures are defined in the cross-platform partial class. The partial method implementations are then defined in the platform-specific partial classes, which should be placed in the correct *Platforms* child folders to ensure that the build system only attempts to build native code when building for the specific platform. For example, the following code shows the `CustomizeEntryPartialMethodsPage` class in the *Platforms > Windows* folder of the project:

C#

```
using Microsoft.UI.Xaml;  
using Microsoft.UI.Xaml.Controls;  
  
namespace CustomizeHandlersDemo.Views  
{  
    public partial class CustomizeEntryPartialMethodsPage : ContentPage  
    {  
        partial void ChangedHandler(object sender, EventArgs e)  
        {  
            Entry entry = sender as Entry;  
            (entry.Handler.PlatformView as TextBox).GotFocus += OnGotFocus;  
        }  
  
        partial void ChangingHandler(object sender, HandlerChangingEventArgs  
e)  
        {  
            if (e.OldHandler != null)  
            {  
                (e.OldHandler.PlatformView as TextBox).GotFocus -=  
OnGotFocus;  
            }  
        }  
  
        void OnGotFocus(object sender, RoutedEventArgs e)  
        {  
            var nativeView = sender as TextBox;  
            nativeView.SelectAll();  
        }  
    }  
}
```

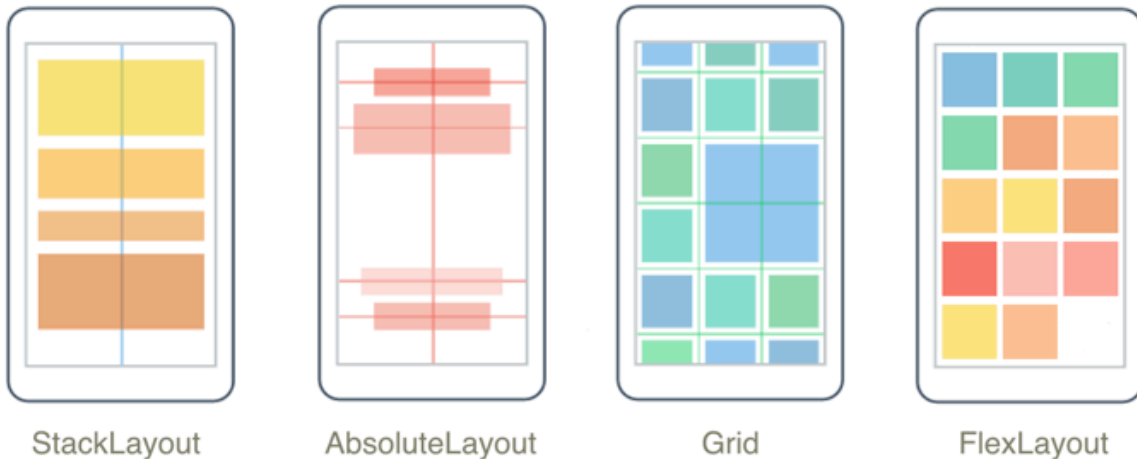
The advantage of this approach is that conditional compilation isn't required, and that the partial methods don't have to be implemented on each platform. If an implementation isn't provided on a platform, then the method and all calls to the

method are removed at compile time. For information about partial methods, see [Partial methods](#).

For information about the organization of the *Platforms* folder in a .NET MAUI project, see [Partial classes and methods](#). For information about how to configure multi-targeting so that you don't have to place platform code into sub-folders of the *Platforms* folder, see [Configure multi-targeting](#).

Layouts

Article • 09/30/2024



.NET Multi-platform App UI (.NET MAUI) layout classes allow you to arrange and group UI controls in your application. Choosing a layout class requires knowledge of how the layout positions its child elements, and how the layout sizes its child elements. In addition, it may be necessary to nest layouts to create your desired layout.

StackLayout

A [StackLayout](#) organizes elements in a one-dimensional stack, either horizontally or vertically. The `Orientation` property specifies the direction of the elements, and the default orientation is `Vertical`. [StackLayout](#) is typically used to arrange a subsection of the UI on a page.

The following XAML shows how to create a vertical [StackLayout](#) containing three [Label](#) objects:

XAML

```
<StackLayout Margin="20,35,20,25">
  <Label Text="The StackLayout has its Margin property set, to control the
  rendering position of the StackLayout." />
  <Label Text="The Padding property can be set to specify the distance
  between the StackLayout and its children." />
  <Label Text="The Spacing property can be set to specify the distance
  between views in the StackLayout." />
</StackLayout>
```

In a [StackLayout](#), if an element's size is not explicitly set, it expands to fill the available width, or height if the `Orientation` property is set to `Horizontal`.

A [StackLayout](#) is often used as a parent layout, which contains other child layouts. However, a [StackLayout](#) should not be used to reproduce a [Grid](#) layout by using a combination of [StackLayout](#) objects. The following code shows an example of this bad practice:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="Details.HomePage"
             Padding="0,20,0,0">
    <StackLayout>
        <StackLayout Orientation="Horizontal">
            <Label Text="Name:" />
            <Entry Placeholder="Enter your name" />
        </StackLayout>
        <StackLayout Orientation="Horizontal">
            <Label Text="Age:" />
            <Entry Placeholder="Enter your age" />
        </StackLayout>
        <StackLayout Orientation="Horizontal">
            <Label Text="Occupation:" />
            <Entry Placeholder="Enter your occupation" />
        </StackLayout>
        <StackLayout Orientation="Horizontal">
            <Label Text="Address:" />
            <Entry Placeholder="Enter your address" />
        </StackLayout>
    </StackLayout>
</ContentPage>
```

This is wasteful because unnecessary layout calculations are performed. Instead, the desired layout can be better achieved by using a [Grid](#).

For more information, see [StackLayout](#).

HorizontalStackLayout

A [HorizontalStackLayout](#) organizes child views in a one-dimensional horizontal stack, and is a more performant alternative to a [StackLayout](#). [HorizontalStackLayout](#) is typically used to arrange a subsection of the UI on a page.

The following XAML shows how to create a [HorizontalStackLayout](#) containing different child views:

XAML

```
<HorizontalStackLayout Margin="20">
    <Rectangle Fill="Red"
        HeightRequest="30"
        WidthRequest="30" />
    <Label Text="Red"
        FontSize="18" />
</HorizontalStackLayout>
```

In a [HorizontalStackLayout](#), if an element's size is not explicitly set, it expands to fill the available height.

For more information, see [HorizontalStackLayout](#).

VerticalStackLayout

A [VerticalStackLayout](#) organizes child views in a one-dimensional vertical stack, and is a more performant alternative to a [StackLayout](#). [VerticalStackLayout](#) is typically used to arrange a subsection of the UI on a page.

The following XAML shows how to create a [VerticalStackLayout](#) containing three [Label](#) objects:

XAML

```
<VerticalStackLayout Margin="20,35,20,25">
    <Label Text="The VerticalStackLayout has its Margin property set, to
control the rendering position of the VerticalStackLayout." />
    <Label Text="The Padding property can be set to specify the distance
between the VerticalStackLayout and its children." />
    <Label Text="The Spacing property can be set to specify the distance
between views in the VerticalStackLayout." />
</VerticalStackLayout>
```

In a [VerticalStackLayout](#), if an element's size is not explicitly set, it expands to fill the available width.

For more information, see [VerticalStackLayout](#).

Grid

A [Grid](#) is used for displaying elements in rows and columns, which can have proportional or absolute sizes. A grid's rows and columns are specified with the `RowDefinitions` and `ColumnDefinitions` properties.

To position elements in specific [Grid](#) cells, use the `Grid.Column` and `Grid.Row` attached properties. To make elements span across multiple rows and columns, use the `Grid.RowSpan` and `Grid.ColumnSpan` attached properties.

ⓘ Note

A [Grid](#) layout should not be confused with tables, and is not intended to present tabular data.

The following XAML shows how to create a [Grid](#) with two rows and two columns:

XAML

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition Height="50" />
    <RowDefinition Height="50" />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="Auto" />
    <ColumnDefinition />
  </Grid.ColumnDefinitions>
  <Label Text="Column 0, Row 0"
    WidthRequest="200" />
  <Label Grid.Column="1"
    Text="Column 1, Row 0" />
  <Label Grid.Row="1"
    Text="Column 0, Row 1" />
  <Label Grid.Column="1"
    Grid.Row="1"
    Text="Column 1, Row 1" />
</Grid>
```

In this example, sizing works as follows:

- Each row has an explicit height of 50 device-independent units.
- The width of the first column is set to `Auto`, and is therefore as wide as required for its children. In this case, it's 200 device-independent units wide to accommodate the width of the first [Label](#).

Space can be distributed within a column or row by using auto sizing, which lets columns and rows size to fit their content. This is achieved by setting the height of a `RowDefinition`, or the width of a `ColumnDefinition`, to `Auto`. Proportional sizing can also be used to distribute available space among the rows and columns of the grid by weighted proportions. This is achieved by setting the height of a `RowDefinition`, or the width of a `ColumnDefinition`, to a value that uses the `*` operator.

⊗ Caution

Try to ensure that as few rows and columns as possible are set to `Auto` size. Each auto-sized row or column will cause the layout engine to perform additional layout calculations. Instead, use fixed size rows and columns if possible. Alternatively, set rows and columns to occupy a proportional amount of space with the `GridUnitType.Star` enumeration value.

For more information, see [Grid](#).

FlexLayout

A [FlexLayout](#) is similar to a [StackLayout](#) in that it displays child elements either horizontally or vertically in a stack. However, a [FlexLayout](#) can also wrap its children if there are too many to fit in a single row or column, and also enables more granular control of the size, orientation, and alignment of its child elements.

The following XAML shows how to create a [FlexLayout](#) that displays its views in a single column:

XAML

```
<FlexLayout Direction="Column"
            AlignItems="Center"
            JustifyContent="SpaceEvenly">
    <Label Text="FlexLayout in Action" />
    <Button Text="Button" />
    <Label Text="Another Label" />
</FlexLayout>
```

In this example, layout works as follows:

- The `Direction` property is set to `Column`, which causes the children of the [FlexLayout](#) to be arranged in a single column of items.
- The `AlignItems` property is set to `Center`, which causes each item to be horizontally centered.
- The `JustifyContent` property is set to `SpaceEvenly`, which allocates all leftover vertical space equally between all the items, and above the first item, and below the last item.

For more information, see [FlexLayout](#).

AbsoluteLayout

An [AbsoluteLayout](#) is used to position and size elements using explicit values, or values relative to the size of the layout. The position is specified by the upper-left corner of the child relative to the upper-left corner of the [AbsoluteLayout](#).

An [AbsoluteLayout](#) should be regarded as a special-purpose layout to be used only when you can impose a size on children, or when the element's size doesn't affect the positioning of other children. A standard use of this layout is to create an overlay, which covers the page with other controls, perhaps to protect the user from interacting with the normal controls on the page.

Important

The `HorizontalOptions` and `VerticalOptions` properties have no effect on children of an [AbsoluteLayout](#).

Within an [AbsoluteLayout](#), the `AbsoluteLayout.LayoutBounds` attached property is used to specify the horizontal position, vertical position, width and height of an element. In addition, the `AbsoluteLayout.LayoutFlags` attached property specifies how the layout bounds will be interpreted.

The following XAML shows how to arrange elements in an [AbsoluteLayout](#):

XAML

```
<AbsoluteLayout Margin="40">
  <BoxView Color="Red"
    AbsoluteLayout.LayoutFlags="PositionProportional"
    AbsoluteLayout.LayoutBounds="0.5, 0, 100, 100"
    Rotation="30" />
  <BoxView Color="Green"
    AbsoluteLayout.LayoutFlags="PositionProportional"
    AbsoluteLayout.LayoutBounds="0.5, 0, 100, 100"
    Rotation="60" />
  <BoxView Color="Blue"
    AbsoluteLayout.LayoutFlags="PositionProportional"
    AbsoluteLayout.LayoutBounds="0.5, 0, 100, 100" />
</AbsoluteLayout>
```

In this example, layout works as follows:

- Each [BoxView](#) is given an explicit size of 100x100, and is displayed in the same position, horizontally centered.

- The red [BoxView](#) is rotated 30 degrees, and the green [BoxView](#) is rotated 60 degrees.
- On each [BoxView](#), the `AbsoluteLayout.LayoutFlags` attached property is set to `PositionProportional`, indicating that the position is proportional to the remaining space after width and height are accounted for.

⊗ Caution

Avoid using the `AbsoluteLayout.AutoSize` property whenever possible, as it will cause the layout engine to perform additional layout calculations.

For more information, see [AbsoluteLayout](#).

BindableLayout

A [BindableLayout](#) enables any layout class that derives from the [Layout](#) class to generate its content by binding to a collection of items, with the option to set the appearance of each item with a [DataTemplate](#).

A bindable layout is populated with data by setting its `ItemsSource` property to any collection that implements `IEnumerable`, and attaching it to a [Layout](#)-derived class. The appearance of each item in the bindable layout can be defined by setting the `BindableLayout.ItemTemplate` attached property to a [DataTemplate](#).

The following XAML shows how to bind a [StackLayout](#) to a collection of items, and define their appearance with a [DataTemplate](#):

XAML

```
<StackLayout BindableLayout.ItemsSource="{Binding User.TopFollowers}"
    Orientation="Horizontal">
    <BindableLayout.ItemTemplate>
        <DataTemplate>
            <Image Source="{Binding}"
                Aspect="AspectFill"
                WidthRequest="44"
                HeightRequest="44" />
        </DataTemplate>
    </BindableLayout.ItemTemplate>
</StackLayout>
```

Bindable layouts should only be used when the collection of items to be displayed is small, and scrolling and selection isn't required.

For more information, see [BindableLayout](#).

Custom layouts

In .NET MAUI, the layout classes derive from the abstract [Layout](#) class. This class delegates cross-platform layout and measurement to a layout manager class. Each layout manager class implements the [ILayoutManager](#) interface, which specifies that [Measure](#) and [ArrangeChildren](#) implementations must be provided:

- The [Measure](#) implementation calls [IView.Measure](#) on each view in the layout, and returns the total size of the layout given the constraints.
- The [ArrangeChildren](#) implementation determines where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view with its appropriate bounds. The return value is the actual size of the layout.

.NET MAUI's layouts have pre-defined layout managers to handle their layout. However, sometimes it's necessary to organize page content using a layout that isn't provided by .NET MAUI. This can be achieved by writing your own custom layout. For more information, see [Custom layouts](#).

Input transparency

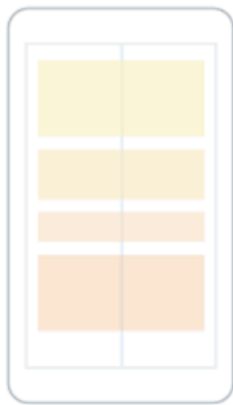
Each visual element has an `InputTransparent` bindable property that's used to define whether the element can receive input. Its default value is `false`, ensuring that the element can receive input. When this property is `true` on an element, the element won't receive any input. Instead, input will be passed to any elements that are visually behind the element.

The [Layout](#) class, from which all layouts derive, has a `CascadeInputTransparent` bindable property that controls whether child elements inherit the input transparency of the layout. Its default value is `true`, ensuring that setting the `InputTransparent` property to `true` on a layout class will result in all elements within the layout not receiving any input.

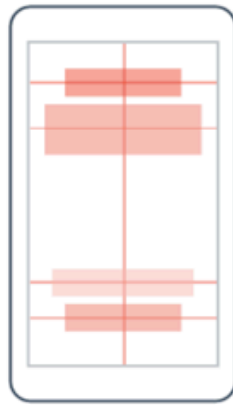
AbsoluteLayout

Article • 09/30/2024

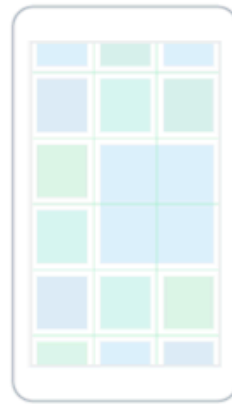
 [Browse the sample](#)



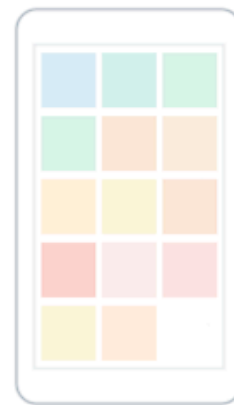
StackLayout



AbsoluteLayout



Grid



FlexLayout

The .NET Multi-platform App UI (.NET MAUI) [AbsoluteLayout](#) is used to position and size children using explicit values. The position is specified by the upper-left corner of the child relative to the upper-left corner of the [AbsoluteLayout](#), in device-independent units. [AbsoluteLayout](#) also implements a proportional positioning and sizing feature. In addition, unlike some other layout classes, [AbsoluteLayout](#) is able to position children so that they overlap.

An [AbsoluteLayout](#) should be regarded as a special-purpose layout to be used only when you can impose a size on children, or when the element's size doesn't affect the positioning of other children.

The [AbsoluteLayout](#) class defines the following properties:

- `LayoutBounds`, of type `Rect`, which is an attached property that represents the position and size of a child. The default value of this property is `(0,0,AutoSize,AutoSize)`.
- `LayoutFlags`, of type `AbsoluteLayoutFlags`, which is an attached property that indicates whether properties of the layout bounds used to position and size the child are interpreted proportionally. The default value of this property is `AbsoluteLayoutFlags.None`.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings and styled. For more information about attached properties, see [.NET MAUI Attached Properties](#).

Position and size children

The position and size of children in an [AbsoluteLayout](#) is defined by setting the `AbsoluteLayout.LayoutParams` attached property of each child, using absolute values or proportional values. Absolute and proportional values can be mixed for children when the position should scale, but the size should stay fixed, or vice versa. For information about absolute values, see [Absolute positioning and sizing](#). For information about proportional values, see [Proportional positioning and sizing](#).

The `AbsoluteLayout.LayoutParams` attached property can be set using two formats, regardless of whether absolute or proportional values are used:

- `x, y`. With this format, the `x` and `y` values indicate the position of the upper-left corner of the child relative to its parent. The child is unconstrained and sizes itself.
- `x, y, width, height`. With this format, the `x` and `y` values indicate the position of the upper-left corner of the child relative to its parent, while the `width` and `height` values indicate the child's size.

To specify that a child sizes itself horizontally or vertically, or both, set the `width` and/or `height` values to the `AbsoluteLayout.AutoSize` property. However, overuse of this property can harm application performance, as it causes the layout engine to perform additional layout calculations.

Important

The `HorizontalOptions` and `VerticalOptions` properties have no effect on children of an [AbsoluteLayout](#).

Absolute positioning and sizing

By default, an [AbsoluteLayout](#) positions and sizes children using absolute values, specified in device-independent units, which explicitly define where children should be placed in the layout. This is achieved by adding children to an [AbsoluteLayout](#) and setting the `AbsoluteLayout.LayoutParams` attached property on each child to absolute position and/or size values.

Warning

Using absolute values for positioning and sizing children can be problematic, because different devices have different screen sizes and resolutions. Therefore, the

coordinates for the center of the screen on one device may be offset on other devices.

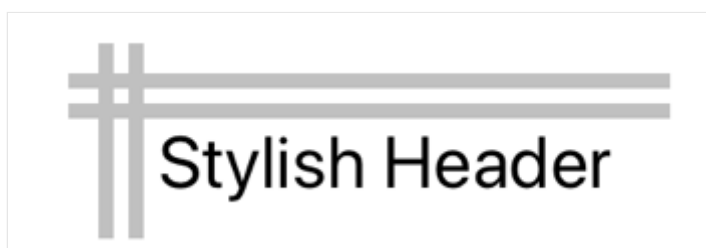
The following XAML shows an [AbsoluteLayout](#) whose children are positioned using absolute values:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="AbsoluteLayoutDemos.Views.XAML.StylishHeaderDemoPage"
             Title="Stylish header demo">
    <AbsoluteLayout Margin="20">
        <BoxView Color="Silver"
                 AbsoluteLayout.LayoutBounds="0, 10, 200, 5" />
        <BoxView Color="Silver"
                 AbsoluteLayout.LayoutBounds="0, 20, 200, 5" />
        <BoxView Color="Silver"
                 AbsoluteLayout.LayoutBounds="10, 0, 5, 65" />
        <BoxView Color="Silver"
                 AbsoluteLayout.LayoutBounds="20, 0, 5, 65" />
        <Label Text="Stylish Header"
               FontSize="24"
               AbsoluteLayout.LayoutBounds="30, 25" />
    </AbsoluteLayout>
</ContentPage>
```

In this example, the position of each [BoxView](#) object is defined using the first two absolute values that are specified in the `AbsoluteLayout.LayoutBounds` attached property. The size of each [BoxView](#) is defined using the third and fourth values. The position of the [Label](#) object is defined using the two absolute values that are specified in the `AbsoluteLayout.LayoutBounds` attached property. Size values are not specified for the [Label](#), and so it's unconstrained and sizes itself. In all cases, the absolute values represent device-independent units.

The following screenshot shows the resulting layout:



The equivalent C# code is shown below:

C#

```
public class StylishHeaderDemoPage : ContentPage
{
    public StylishHeaderDemoPage()
    {
        AbsoluteLayout absoluteLayout = new AbsoluteLayout
        {
            Margin = new Thickness(20)
        };

        absoluteLayout.Add(new BoxView
        {
            Color = Colors.Silver
        }, new Rect(0, 10, 200, 5));
        absoluteLayout.Add(new BoxView
        {
            Color = Colors.Silver
        }, new Rect(0, 20, 200, 5));
        absoluteLayout.Add(new BoxView
        {
            Color = Colors.Silver
        }, new Rect(10, 0, 5, 65));
        absoluteLayout.Add(new BoxView
        {
            Color = Colors.Silver
        }, new Rect(20, 0, 5, 65));

        absoluteLayout.Add(new Label
        {
            Text = "Stylish Header",
            FontSize = 24
        }, new Point(30, 25));

        Title = "Stylish header demo";
        Content = absoluteLayout;
    }
}
```

In this example, the position and size of each `BoxView` is defined using a `Rect` object. The position of the `Label` is defined using a `Point` object. The C# example uses the following `Add` extension methods to add children to the `AbsoluteLayout`:

C#

```
using Microsoft.Maui.Layouts;

namespace Microsoft.Maui.Controls
{
    public static class AbsoluteLayoutExtensions
    {
        public static void Add(this AbsoluteLayout absoluteLayout, IView
```



```

view, Rect bounds, AbsoluteLayoutFlags flags = AbsoluteLayoutFlags.None)
{
    if (view == null)
        throw new ArgumentNullException(nameof(view));
    if (bounds.IsEmpty)
        throw new ArgumentNullException(nameof(bounds));

    absoluteLayout.Add(view);
    absoluteLayout.SetLayoutBounds(view, bounds);
    absoluteLayout.SetLayoutFlags(view, flags);
}

public static void Add(this AbsoluteLayout absoluteLayout, IView
view, Point position)
{
    if (view == null)
        throw new ArgumentNullException(nameof(view));
    if (position.IsEmpty)
        throw new ArgumentNullException(nameof(position));

    absoluteLayout.Add(view);
    absoluteLayout.SetLayoutBounds(view, new Rect(position.X,
position.Y, AbsoluteLayout.AutoSize, AbsoluteLayout.AutoSize));
}
}
}

```

In C#, it's also possible to set the position and size of a child of an [AbsoluteLayout](#) after it has been added to the layout, using the `AbsoluteLayout.SetLayoutBounds` method. The first argument to this method is the child, and the second is a `Rect` object.

⚠ Note

An [AbsoluteLayout](#) that uses absolute values can position and size children so that they don't fit within the bounds of the layout.

Proportional positioning and sizing

An [AbsoluteLayout](#) can position and size children using proportional values. This is achieved by adding children to the [AbsoluteLayout](#) and by setting the `AbsoluteLayout.LayoutBounds` attached property on each child to proportional position and/or size values in the range 0-1. Position and size values are made proportional by setting the `AbsoluteLayout.LayoutFlags` attached property on each child.

The `AbsoluteLayout.LayoutFlags` attached property, of type `AbsoluteLayoutFlags`, allows you to set a flag that indicates that the layout bounds position and size values for a

child are proportional to the size of the [AbsoluteLayout](#). When laying out a child, [AbsoluteLayout](#) scales the position and size values appropriately, to any device size.

The `AbsoluteLayoutFlags` enumeration defines the following members:

- `None`, indicates that values will be interpreted as absolute. This is the default value of the `AbsoluteLayout.LayoutFlags` attached property.
- `XProportional`, indicates that the `x` value will be interpreted as proportional, while treating all other values as absolute.
- `YProportional`, indicates that the `y` value will be interpreted as proportional, while treating all other values as absolute.
- `WidthProportional`, indicates that the `width` value will be interpreted as proportional, while treating all other values as absolute.
- `HeightProportional`, indicates that the `height` value will be interpreted as proportional, while treating all other values as absolute.
- `PositionProportional`, indicates that the `x` and `y` values will be interpreted as proportional, while the size values are interpreted as absolute.
- `SizeProportional`, indicates that the `width` and `height` values will be interpreted as proportional, while the position values are interpreted as absolute.
- `All`, indicates that all values will be interpreted as proportional.

Tip

The `AbsoluteLayoutFlags` enumeration is a `Flags` enumeration, which means that enumeration members can be combined. This is accomplished in XAML with a comma-separated list, and in C# with the bitwise OR operator.

For example, if you use the `SizeProportional` flag and set the width of a child to 0.25 and the height to 0.1, the child will be one-quarter of the width of the [AbsoluteLayout](#) and one-tenth the height. The `PositionProportional` flag is similar. A position of (0,0) puts the child in the upper-left corner, while a position of (1,1) puts the child in the lower-right corner, and a position of (0.5,0.5) centers the child within the [AbsoluteLayout](#).

The following XAML shows an [AbsoluteLayout](#) whose children are positioned using proportional values:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="AbsoluteLayoutDemos.Views.XAML.ProportionalDemoPage"
```

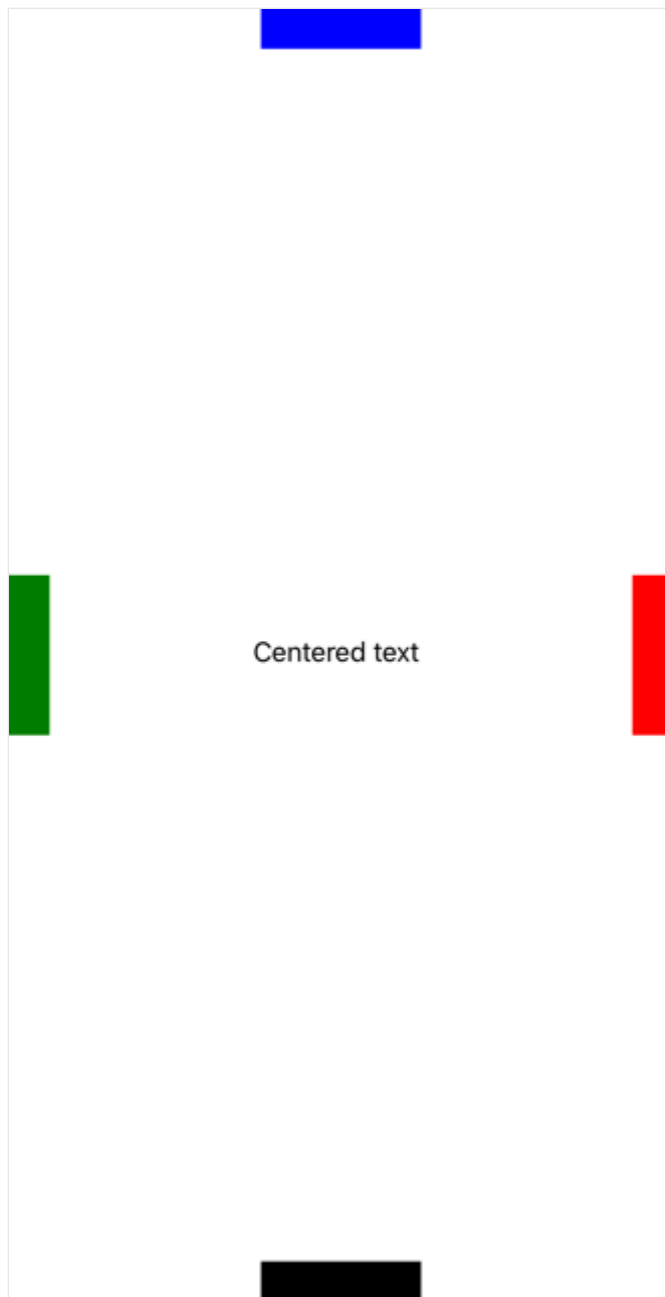
```

        Title="Proportional demo">
<AbsoluteLayout>
    <BoxView Color="Blue"
        AbsoluteLayout.LayoutBounds="0.5,0,100,25"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
    <BoxView Color="Green"
        AbsoluteLayout.LayoutBounds="0,0.5,25,100"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
    <BoxView Color="Red"
        AbsoluteLayout.LayoutBounds="1,0.5,25,100"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
    <BoxView Color="Black"
        AbsoluteLayout.LayoutBounds="0.5,1,100,25"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
    <Label Text="Centered text"
        AbsoluteLayout.LayoutBounds="0.5,0.5,110,25"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
</AbsoluteLayout>
</ContentPage>

```

In this example, each child is positioned using proportional values but sized using absolute values. This is accomplished by setting the `AbsoluteLayout.LayoutFlags` attached property of each child to `PositionProportional`. The first two values that are specified in the `AbsoluteLayout.LayoutBounds` attached property, for each child, define the position using proportional values. The size of each child is defined with the third and fourth absolute values, using device-independent units.

The following screenshot shows the resulting layout:



The equivalent C# code is shown below:

C#

```
public class ProportionalDemoPage : ContentPage
{
    public ProportionalDemoPage()
    {
        BoxView blue = new BoxView { Color = Colors.Blue };
        AbsoluteLayout.SetLayoutBounds(blue, new Rect(0.5, 0, 100, 25));
        AbsoluteLayout.SetLayoutFlags(blue,
        AbsoluteLayoutFlags.PositionProportional);

        BoxView green = new BoxView { Color = Colors.Green };
        AbsoluteLayout.SetLayoutBounds(green, new Rect(0, 0.5, 25, 100));
        AbsoluteLayout.SetLayoutFlags(green,
        AbsoluteLayoutFlags.PositionProportional);

        BoxView red = new BoxView { Color = Colors.Red };
    }
}
```

```

        AbsoluteLayout.SetLayoutBounds(red, new Rect(1, 0.5, 25, 100));
        AbsoluteLayout.SetLayoutFlags(red,
AbsoluteLayoutFlags.PositionProportional);

        BoxView black = new BoxView { Color = Colors.Black };
        AbsoluteLayout.SetLayoutBounds(black, new Rect(0.5, 1, 100, 25));
        AbsoluteLayout.SetLayoutFlags(black,
AbsoluteLayoutFlags.PositionProportional);

        Label label = new Label { Text = "Centered text" };
        AbsoluteLayout.SetLayoutBounds(label, new Rect(0.5, 0.5, 110, 25));
        AbsoluteLayout.SetLayoutFlags(label,
AbsoluteLayoutFlags.PositionProportional);

        Title = "Proportional demo";
        Content = new AbsoluteLayout
        {
            Children = { blue, green, red, black, label }
        };
    }
}

```

In this example, the position and size of each child is set with the `AbsoluteLayout.SetLayoutBounds` method. The first argument to the method is the child, and the second is a `Rect` object. The position of each child is set with proportional values, while the size of each child is set with absolute values, using device-independent units.

ⓘ Note

An [AbsoluteLayout](#) that uses proportional values can position and size children so that they don't fit within the bounds of the layout by using values outside the 0-1 range.

BindableLayout

Article • 09/30/2024

 [Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) bindable layouts enable any layout class that derives from the [Layout](#) class to generate its content by binding to a collection of items, with the option to set the appearance of each item with a [DataTemplate](#).

Bindable layouts are provided by the [BindableLayout](#) class, which exposes the following attached properties:

- `ItemsSource` – specifies the collection of `IEnumerable` items to be displayed by the layout.
- `ItemTemplate` – specifies the [DataTemplate](#) to apply to each item in the collection of items displayed by the layout.
- `ItemTemplateSelector` – specifies the [DataTemplateSelector](#) that will be used to choose a [DataTemplate](#) for an item at runtime.

ⓘ Note

The `ItemTemplate` property takes precedence when both the `ItemTemplate` and `ItemTemplateSelector` properties are set.

In addition, the [BindableLayout](#) class exposes the following bindable properties:

- `EmptyView` – specifies the `string` or view that will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The default value is `null`.
- `EmptyViewTemplate` – specifies the [DataTemplate](#) that will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The default value is `null`.

ⓘ Note

The `EmptyViewTemplate` property takes precedence when both the `EmptyView` and `EmptyViewTemplate` properties are set.

All of these properties can be attached to the [AbsoluteLayout](#), [FlexLayout](#), [Grid](#), [HorizontalStackLayout](#), [StackLayout](#), and [VerticalStackLayout](#) classes, which all derive from the [Layout](#) class.

When the `BindableLayout.ItemsSource` property is set to a collection of items and attached to a [Layout](#)-derived class, each item in the collection is added to the [Layout](#)-derived class for display. The [Layout](#)-derived class will then update its child views when the underlying collection changes.

Bindable layouts should only be used when the collection of items to be displayed is small, and scrolling and selection isn't required. While scrolling can be provided by wrapping a bindable layout in a [ScrollView](#), this is not recommended as bindable layouts lack UI virtualization. When scrolling is required, a scrollable view that includes UI virtualization, such as [ListView](#) or [CollectionView](#), should be used. Failure to observe this recommendation can lead to performance issues.

Important

While it's technically possible to attach a bindable layout to any layout class that derives from the [Layout](#) class, it's not always practical to do so, particularly for the [AbsoluteLayout](#) and [Grid](#) classes. For example, consider the scenario of wanting to display a collection of data in a [Grid](#) using a bindable layout, where each item in the collection is an object containing multiple properties. Each row in the [Grid](#) should display an object from the collection, with each column in the [Grid](#) displaying one of the object's properties. Because the [DataTemplate](#) for the bindable layout can only contain a single object, it's necessary for that object to be a layout class containing multiple views that each display one of the object's properties in a specific [Grid](#) column. While this scenario can be realised with bindable layouts, it results in a parent [Grid](#) containing a child [Grid](#) for each item in the bound collection, which is a highly inefficient and problematic use of the [Grid](#) layout.

Populate a bindable layout with data

A bindable layout is populated with data by setting its `ItemsSource` property to any collection that implements `IEnumerable`, and attaching it to a [Layout](#)-derived class:

XAML

```
<Grid BindableLayout.ItemsSource="{Binding Items}" />
```

The equivalent C# code is:

```
C#

IEnumerable<string> items = ...;
Grid grid = new Grid();
BindableLayout.SetItemsSource(grid, items);
```

When the `BindableLayout.ItemsSource` attached property is set on a layout, but the `BindableLayout.ItemTemplate` attached property isn't set, every item in the `IEnumerable` collection will be displayed by a `Label` that's created by the `BindableLayout` class.

Define item appearance

The appearance of each item in the bindable layout can be defined by setting the `BindableLayout.ItemTemplate` attached property to a `DataTemplate`:

XAML

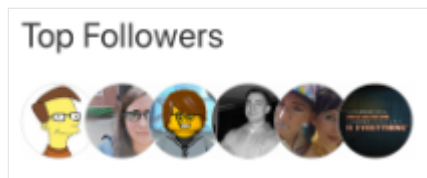
```
<StackLayout BindableLayout.ItemsSource="{Binding User.TopFollowers}"
    Orientation="Horizontal"
    ...>
    <BindableLayout.ItemTemplate>
        <DataTemplate>
            <Image Source="{Binding}"
                Aspect="AspectFill"
                WidthRequest="44"
                HeightRequest="44"
                ... />
        </DataTemplate>
    </BindableLayout.ItemTemplate>
</StackLayout>
```

The equivalent C# code is:

```
C#

DataTemplate imageTemplate = ...;
StackLayout stackLayout = new StackLayout();
BindableLayout.SetItemsSource(stackLayout, viewModel.User.TopFollowers);
BindableLayout.SetItemTemplate(stackLayout, imageTemplate);
```

In this example, every item in the `TopFollowers` collection will be displayed by an `Image` view defined in the `DataTemplate`:



For more information about data templates, see [Data templates](#).

Choose item appearance at runtime

The appearance of each item in the bindable layout can be chosen at runtime, based on the item value, by setting the `BindableLayout.ItemTemplateSelector` attached property to a [DataTemplateSelector](#):

XAML

```
<FlexLayout BindableLayout.ItemsSource="{Binding User.FavoriteTech}"
            BindableLayout.ItemTemplateSelector="{StaticResource
            TechItemTemplateSelector}"
            ... />
```

The equivalent C# code is:

C#

```
DataTemplateSelector dataTemplateSelector = new TechItemTemplateSelector {
    ... };
FlexLayout flexLayout = new FlexLayout();
BindableLayout.SetItemsSource(flexLayout, viewModel.User.FavoriteTech);
BindableLayout.SetItemTemplateSelector(flexLayout, dataTemplateSelector);
```

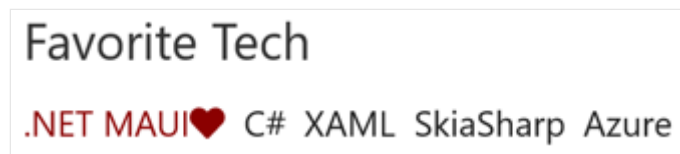
The following example shows the `TechItemTemplateSelector` class:

C#

```
public class TechItemTemplateSelector : DataTemplateSelector
{
    public DataTemplate DefaultTemplate { get; set; }
    public DataTemplate MAUITemplate { get; set; }

    protected override DataTemplate OnSelectTemplate(object item,
        BindableObject container)
    {
        return (string)item == ".NET MAUI" ? MAUITemplate : DefaultTemplate;
    }
}
```

The `TechItemTemplateSelector` class defines `DefaultTemplate` and `MAUITemplate` `DataTemplate` properties that are set to different data templates. The `OnSelectTemplate` method returns the `MAUITemplate`, which displays an item in dark red with a heart next to it, when the item is equal to ".NET MAUI". When the item isn't equal to ".NET MAUI", the `OnSelectTemplate` method returns the `DefaultTemplate`, which displays an item using the default color of a `Label`:



For more information about data template selectors, see [Create a DataTemplateSelector](#).

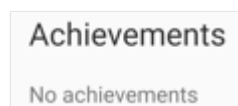
Display a string when data is unavailable

The `EmptyView` property can be set to a string, which will be displayed by a `Label` when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The following XAML shows an example of this scenario:

```
XAML

<StackLayout BindableLayout.ItemsSource="{Binding
UserWithoutAchievements.Achievements}"
            BindableLayout.EmptyView="No achievements">
    ...
</StackLayout>
```

The result is that when the data bound collection is `null`, the string set as the `EmptyView` property value is displayed:



Display views when data is unavailable

The `EmptyView` property can be set to a view, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. This can be a single view, or a view that contains multiple child views. The following XAML example shows the `EmptyView` property set to a view that contains multiple child views:

XAML

```
<StackLayout BindableLayout.ItemsSource="{Binding
UserWithoutAchievements.Achievements}">
    <BindableLayout.EmptyView>
        <StackLayout>
            <Label Text="None."
                FontAttributes="Italic"
                FontSize="{StaticResource smallTextSize}" />
            <Label Text="Try harder and return later?"
                FontAttributes="Italic"
                FontSize="{StaticResource smallTextSize}" />
        </StackLayout>
    </BindableLayout.EmptyView>
    ...
</StackLayout>
```

The result is that when the data bound collection is `null`, the `StackLayout` and its child views are displayed.

Achievements

None.

Try harder and return later?

Similarly, the `EmptyViewTemplate` can be set to a `DataTemplate`, which will be displayed when the `ItemsSource` property is `null`, or when the collection specified by the `ItemsSource` property is `null` or empty. The `DataTemplate` can contain a single view, or a view that contains multiple child views. In addition, the `BindingContext` of the `EmptyViewTemplate` will be inherited from the `BindingContext` of the `BindableLayout`. The following XAML example shows the `EmptyViewTemplate` property set to a `DataTemplate` that contains a single view:

XAML

```
<StackLayout BindableLayout.ItemsSource="{Binding
UserWithoutAchievements.Achievements}">
    <BindableLayout.EmptyViewTemplate>
        <DataTemplate>
            <Label Text="{Binding Source={x:Reference usernameLabel},
                Path=Text, StringFormat='{0} has no achievements.'}" />
        </DataTemplate>
    </BindableLayout.EmptyViewTemplate>
    ...
</StackLayout>
```

The result is that when the data bound collection is `null`, the `Label` in the `DataTemplate` is displayed:

Achievements

davidortinau has no achievements.

ⓘ Note

The `EmptyViewTemplate` property can't be set via a [DataTemplateSelector](#).

Choose an EmptyView at runtime

Views that will be displayed as an `EmptyView` when data is unavailable, can be defined as `ContentView` objects in a `ResourceDictionary`. The `EmptyView` property can then be set to a specific `ContentView`, based on some business logic, at runtime. The following XAML shows an example of this scenario:

XAML

```
<ContentPage ...>
  <ContentPage.Resources>
    ...
    <ContentView x:Key="BasicEmptyView">
      <StackLayout>
        <Label Text="No achievements."
              FontSize="14" />
      </StackLayout>
    </ContentView>
    <ContentView x:Key="AdvancedEmptyView">
      <StackLayout>
        <Label Text="None."
              FontAttributes="Italic"
              FontSize="14" />
        <Label Text="Try harder and return later?"
              FontAttributes="Italic"
              FontSize="14" />
      </StackLayout>
    </ContentView>
  </ContentPage.Resources>

  <StackLayout>
    ...
    <Switch Toggled="OnEmptyViewSwitchToggled" />

    <StackLayout x:Name="stackLayout"
                  BindableLayout.ItemsSource="{Binding
UserWithoutAchievements.Achievements}">
      ...
    </StackLayout>
  </StackLayout>
</ContentPage>
```

```
</StackLayout>
</ContentPage>
```

The XAML defines two [ContentView](#) objects in the page-level [ResourceDictionary](#), with the [Switch](#) object controlling which [ContentView](#) object will be set as the `EmptyView` property value. When the [Switch](#) is toggled, the `OnEmptyViewSwitchToggled` event handler executes the `ToggleEmptyView` method:

C#

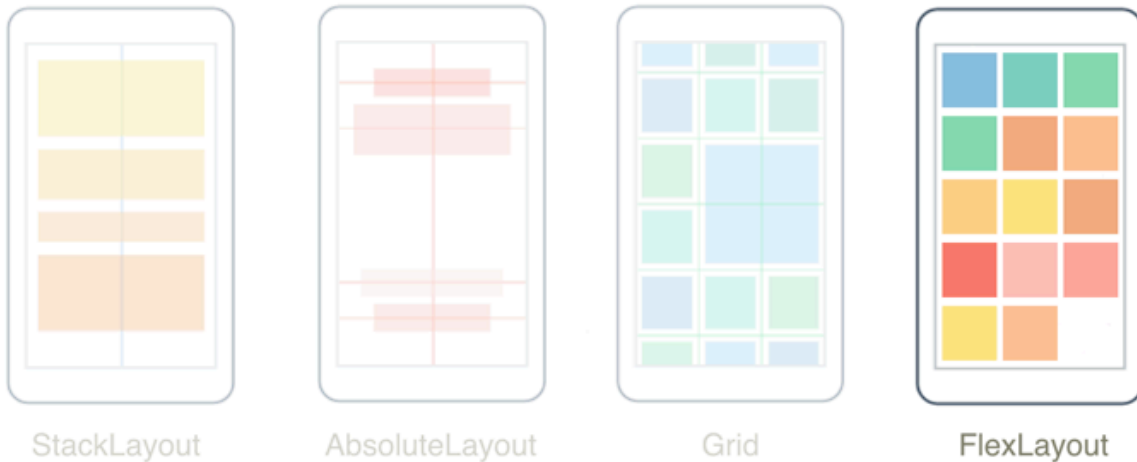
```
void ToggleEmptyView(bool isToggled)
{
    object view = isToggled ? Resources["BasicEmptyView"] :
Resources["AdvancedEmptyView"];
    BindableLayout.SetEmptyView(stackLayout, view);
}
```

The `ToggleEmptyView` method sets the `EmptyView` property of the [StackLayout](#) object to one of the two [ContentView](#) objects stored in the [ResourceDictionary](#), based on the value of the `Switch.IsToggled` property. Then, when the data bound collection is `null`, the [ContentView](#) object set as the `EmptyView` property is displayed.

FlexLayout

Article • 09/30/2024

 [Browse the sample](#)



The .NET Multi-platform App UI (.NET MAUI) [FlexLayout](#) is a layout that can arrange its children horizontally and vertically in a stack, and can also wrap its children if there are too many to fit in a single row or column. In addition, [FlexLayout](#) can control orientation and alignment, and adapt to different screen sizes. [FlexLayout](#) is based on the Cascading Style Sheets (CSS) [Flexible Box Layout Module](#) [↗].

The [FlexLayout](#) class defines the following properties:

- `AlignContent`, of type `FlexAlignContent`, which determines how the layout engine will distribute space between and around children that have been laid out on multiple lines. The default value of this property is `Stretch`. For more information, see [AlignContent](#).
- `AlignItems`, of type `FlexAlignItems`, which indicates how the layout engine will distribute space between and around children along the cross axis. The default value of this property is `Stretch`. For more information, see [AlignItems](#).
- `Direction`, of type `FlexDirection`, which defines the direction and main axis of children. The default value of this property is `Row`. For more information, see [Direction](#).
- `JustifyContent`, of type `FlexJustify`, which specifies how space is distributed between and around children along the main axis. The default value of this property is `Start`. For more information, see [JustifyContent](#).
- `Position`, of type `FlexPosition`, which determines whether the position of children are relative to each other, or by using fixed values. The default value of this property is `Relative`.

- `Wrap`, of type `FlexWrap`, which controls whether children are laid out in a single line or in multiple lines. The default value of this property is `NoWrap`. For more information, see [Wrap](#).
- `AlignSelf`, of type `FlexAlignSelf`, which is an attached property that indicates how the layout engine will distribute space between and around children for a specific child along the cross axis. The default value of this property is `Auto`. For more information, see [AlignSelf](#).
- `Basis`, of type `FlexBasis`, which is an attached property that defines the initial main size of the child before free space is distributed according to other property values. The default value of this property is `Auto`. For more information, see [Basis](#).
- `Grow`, of type `float`, which is an attached property that specifies the amount of available space the child should use on the main axis. The default value of this property is 0.0. A validation callback ensures that when the property is set, its value is greater than or equal to 0. For more information, see [Grow](#).
- `Order`, of type `int`, which is an attached property that determines whether the child should be laid out before or after other children in the container. The default value of this property is 0. For more information, see [Order](#).
- `Shrink`, of type `float`, which is an attached property that controls how a child should shrink so that all children can fit inside the container. The default value of this property is 1.0. A validation callback ensures that when the property is set, its value is greater than or equal to 0. For more information, see [Shrink](#).

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings and styled.

Important

When items in a [FlexLayout](#) are arranged in a column, the [FlexLayout](#) has a vertical *main axis* and a horizontal *cross axis*. When items in a [FlexLayout](#) are arranged in a row, the [FlexLayout](#) has a horizontal *main axis* and a vertical *cross axis*.

[FlexLayout](#) and its children can be partially styled using Cascading Style Sheets (CSS). For more information, see [Styling apps using Cascading Style Sheets \(CSS\)](#).

Orientation and alignment

The `Direction`, `Wrap`, `JustifyContent`, `AlignItems`, `AlignContent`, and `Position` bindable properties can be set on a [FlexLayout](#) to control orientation and alignment of all children.

Direction

The `Direction` property, of type `FlexDirection`, defines the direction and main axis of children. The `FlexDirection` enumeration defines the following members:

- `Column`, which indicates that children should be stacked vertically.
- `ColumnReverse` (or "column-reverse" in XAML), which indicates that children should be stacked vertically in reverse order.
- `Row`, which indicates that children should be stacked horizontally. This is the default value of the `Direction` property.
- `RowReverse` (or "row-reverse" in XAML), which indicates that children should be stacked horizontally in reverse order.

When the `Direction` property is set to `Column`, or `ColumnReverse`, the main-axis will be the y-axis and items will be stacked vertically. When the `Direction` property is set to `Row`, or `RowReverse`, the main-axis will be the x-axis and children will be stacked horizontally.

ⓘ Note

In XAML, you can specify the value of this property using the enumeration member names in lowercase, uppercase, or mixed case, or you can use the two additional strings shown in parentheses.

Wrap

The `Wrap` property, of type `FlexWrap`, controls whether children are laid out in a single line or in multiple lines. The `FlexWrap` enumeration defines the following members:

- `NoWrap`, which indicates that children are laid out in a single line. This is the default value of the `Wrap` property.
- `Wrap`, which indicates that items are laid out in multiple lines if needed.
- `Reverse` (or "wrap-reverse" in XAML), which indicates that items are laid out in multiple lines if needed, in reverse order.

When the `Wrap` property is set to `NoWrap` and the main axis is constrained, and the main axis is not wide or tall enough to fit all the children, the [FlexLayout](#) attempts to make the items smaller. You can control the shrink factor of children with the `Shrink` attached bindable property.

When the `Wrap` property is set to `Wrap` or `WrapReverse`, the `AlignContent` property can be used to specify how the lines should be distributed.

JustifyContent

The `JustifyContent` property, of type `FlexJustify`, specifies how space is distributed between and around children along the main axis. The `FlexJustify` enumeration defines the following members:

- `Start` (or "flex-start" in XAML), which indicates that children should be aligned at the start. This is the default value of the `JustifyContent` property.
- `Center`, which indicates that children should be aligned around the center.
- `End` (or "flex-end" in XAML), which indicates that children should be aligned at the end.
- `SpaceBetween` (or "space-between" in XAML), which indicates that children should be evenly distributed, with the first child being at the start and the last child being at the end.
- `SpaceAround` (or "space-around" in XAML), which indicates that children should be evenly distributed, with the first and last children having a half-size space.
- `SpaceEvenly`, which indicates that children should be evenly distributed, with all children having equal space around them.

AlignItems

The `AlignItems` property, of type `FlexAlignItems`, indicates how the layout engine will distribute space between and around children along the cross axis. The `FlexAlignItems` enumeration defines the following members:

- `Stretch`, which indicates that children should be stretched out. This is the default value of the `AlignItems` property.
- `Center`, which indicates that children should be aligned around the center.
- `Start` (or "flex-start" in XAML), which indicates that children should be aligned at the start.
- `End` (or "flex-end" in XAML), which indicates that children should be aligned at the end.

This is one of two properties that indicates how children are aligned on the cross axis. Within each row, children are stretched or aligned on the start, center, or end of each item.

For any individual child, the `AlignItems` setting can be overridden with the `AlignSelf` attached bindable property.

AlignContent

The `AlignContent` property, of type `FlexAlignContent`, determines how the layout engine will distribute space between and around children that have been laid out on multiple lines. The `FlexAlignContent` enumeration defines the following members:

- `Stretch`, which indicates that children should be stretched out. This is the default value of the `AlignContent` property.
- `Center`, which indicates that children should be aligned around the center.
- `Start` (or "flex-start" in XAML), which indicates that children should be aligned at the start.
- `End` (or "flex-end" in XAML), which indicates that children should be aligned at the end.
- `SpaceBetween` (or "space-between" in XAML), which indicates that children should be evenly distributed, with the first child being at the start and the last child being at the end.
- `SpaceAround` (or "space-around" in XAML), which indicates that children should be evenly distributed, with the first and last children having a half-size space.
- `SpaceEvenly`, which indicates that children should be evenly distributed, with all children having equal space around them.

The `AlignContent` property has no effect when there is only one row or column.

Child alignment and sizing

The `AlignSelf`, `Order`, `Basis`, `Grow`, and `Shrink` attached bindable properties can be set on children of the [FlexLayout](#) to control child orientation, alignment, and sizing.

AlignSelf

The `AlignSelf` property, of type `FlexAlignSelf`, indicates how the layout engine will distribute space between and around children for a specific child along the cross axis. The `FlexAlignSelf` enumeration defines the following members:

- `Auto`, which indicates that a child should be aligned according to the alignment value of its parent. This is the default value of the `AlignSelf` property.
- `Stretch`, which indicates that a child should be stretched out.

- `Center`, which indicates that a child should be aligned around the center.
- `Start` (or "flex-start" in XAML), which indicates that a child should be aligned at the start.
- `End` (or "flex-end" in XAML), which indicates that a child should be aligned at the end.

For any individual child of the `FlexLayout`, this property overrides the `AlignItems` property set on the `FlexLayout`. The default setting of `Auto` means to use the `AlignItems` setting.

In XAML, this property is set on a child without any reference to its `FlexLayout` parent:

XAML

```
<Label FlexLayout.AlignSelf="Center"
... />
```

The equivalent C# code is:

C#

```
Label label = new Label();
FlexLayout.SetAlignSelf(label, FlexAlignSelf.Center);
```

Order

The `Order` property, of type `int`, enables you to change the order that children of the `FlexLayout` are arranged. The default value of this property is 0.

Usually, children are arranged in the order in which they are added to the `FlexLayout`. However, this order can be overridden by setting this property to a non-zero integer value on one or more children. The `FlexLayout` then arranges its children based on their `Order` property values. Children with the same `Order` property values are arranged in the order in which they are added to the `FlexLayout`.

Basis

The `Basis` property, of type `FlexBasis`, defines the initial size of the child on the main axis before free space is distributed according to other property values. The value specified by this property is the size along the main axis of the parent `FlexLayout`. Therefore, this property indicates the width of a child when children are arranged in

rows, or the height of a child when children are arranged in columns. This property is called *basis* because it specifies a size that is the basis of all subsequent layout.

The `FlexBasis` type is a structure that enables size to be specified in device-independent units, or as a percentage of the size of the `FlexLayout`. The default value of the `Basis` property is `Auto`, which means that the child's requested width or height is used.

In XAML, you can use a number for a size in device-independent units:

XAML

```
<Label FlexLayout.Basis="40"  
... />
```

The equivalent C# code is:

C#

```
FlexLayout.SetBasis(label, 40);
```

In XAML, a percentage can be specified as follows:

XAML

```
<Label FlexLayout.Basis="25%"  
... />
```

The equivalent C# code is:

C#

```
FlexLayout.SetBasis(label, new FlexBasis(0.25f, true));
```

The first argument to the `FlexBasis` constructor is a fractional `float` value that must be in the range of 0 to 1. The second argument indicates that the size is relative, rather than absolute.

Grow

The `Grow` property, of type `float`, specifies the amount of available space the child should use on the main axis. The default value of this property is 0.0, and its value must be greater than or equal to 0.

The `Grow` property is used when the `Wrap` property is set to `NoWrap` and a row of children has a total width less than the width of the `FlexLayout`, or a column of children has a shorter height than the `FlexLayout`. The `Grow` property indicates how to apportion the leftover space among the children. If a single child is given a positive `Grow` value, then that child takes up all the remaining space. Alternatively, the remaining space can also be allocated among two or more children.

Shrink

The `Shrink` property, of type `float`, controls how a child should shrink so that all children can fit inside the container. The default value of this property is 1.0, and its value must be greater than or equal to 0.

The `Shrink` property is used when the `Wrap` property is set to `NoWrap` and the aggregate width of a row of children is greater than the width of the `FlexLayout`, or the aggregate height of a single column of children is greater than the height of the `FlexLayout`. Normally the `FlexLayout` will display these children by constricting their sizes. The `Shrink` property can indicate which children are given priority in being displayed at their full sizes.

💡 Tip

The `Grow` and `Shrink` values can both be set to accommodate situations where the aggregate child sizes might sometimes be less than or sometimes greater than the size of the `FlexLayout`.

Examples

The following examples demonstrate common uses of `FlexLayout`.

Stack

A `FlexLayout` can substitute for a `StackLayout`:

XAML

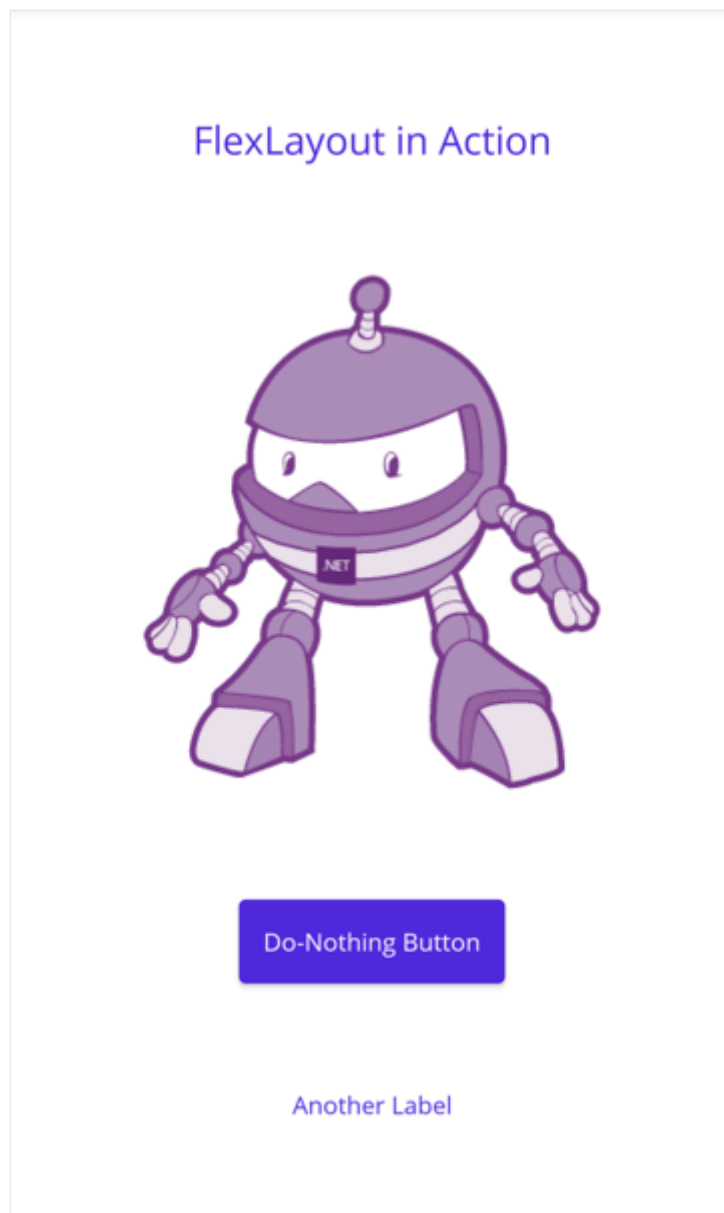
```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  x:Class="FlexLayoutDemos.Views.SimpleStackPage"
  Title="Simple Stack">
  <FlexLayout Direction="Column"
```

```

        AlignItems="Center"
        JustifyContent="SpaceEvenly">
<Label Text="FlexLayout in Action"
        FontSize="18" />
<Image Source="dotnet_bot_branded.png"
        HeightRequest="300" />
<Button Text="Do-Nothing Button" />
<Label Text="Another Label" />
    </FlexLayout>
</ContentPage>

```

In this example, the `Direction` property is set to `Column`, which causes the children of the `FlexLayout` to be arranged in a single column. The `AlignItems` property is set to `Center`, which causes each child to be horizontally centered. The `JustifyContent` property is set to `SpaceEvenly` which allocates all leftover vertical space equally between all the children, above the first child and below the last child:



The `AlignSelf` attached property can be used to override the `AlignItems` property for a specific child.

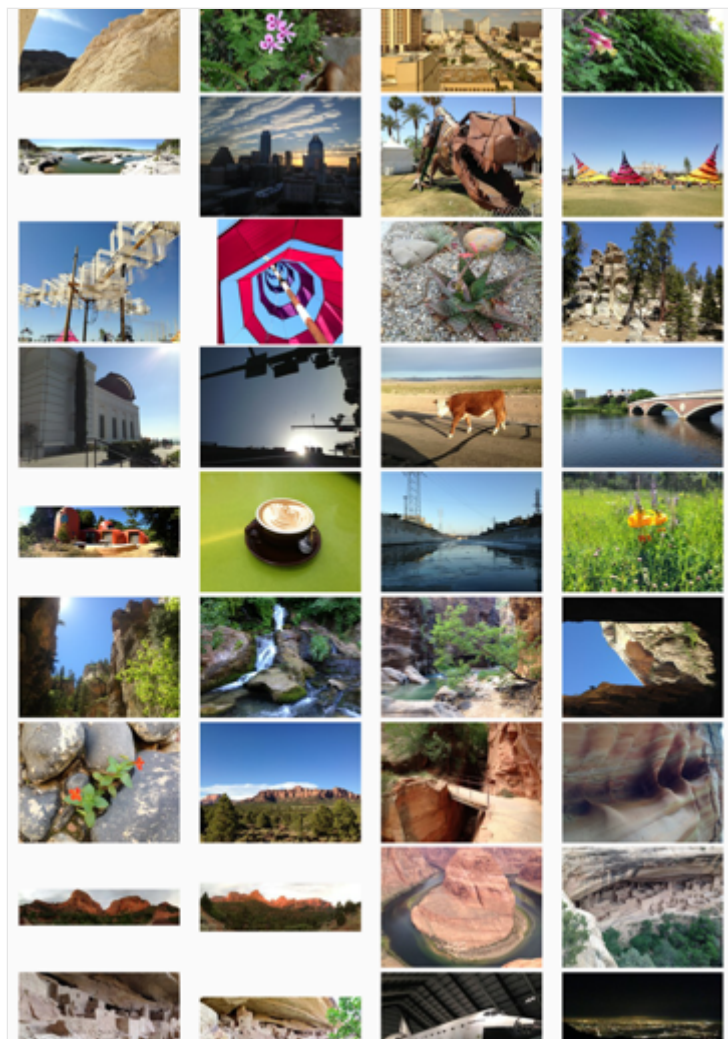
Wrap items

A `FlexLayout` can wrap its children to additional rows or columns:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="FlexLayoutDemos.Views.PhotoWrappingPage"
             Title="Photo Wrapping">
    <Grid>
        <ScrollView>
            <FlexLayout x:Name="flexLayout"
                        Wrap="Wrap"
                        JustifyContent="SpaceAround" />
        </ScrollView>
        ...
    </Grid>
</ContentPage>
```

In this example, the `Direction` property of the `FlexLayout` is not set, so it has the default setting of `Row`, meaning that the children are arranged in rows and the main axis is horizontal. The `Wrap` property is set to `Wrap`, which causes children to wrap to the next row if there are too many children to fit on a row. The `JustifyContent` property is set to `SpaceAround` which allocates all leftover space on the main axis so that each child is surrounded by the same amount of space:



The code-behind file for this example retrieves a collection of photos and adds them to the [FlexLayout](#).

In addition, the [FlexLayout](#) is a child of a [ScrollView](#). Therefore, if there are too many rows to fit on the page, then the [ScrollView](#) has a default `Orientation` property of `Vertical` and allows vertical scrolling.

Page layout

There is a standard layout in web design called the [holy grail](#) because it's a layout format that is very desirable, but often hard to realize with perfection. The layout consists of a header at the top of the page and a footer at the bottom, both extending to the full width of the page. Occupying the center of the page is the main content, but often with a columnar menu to the left of the content and supplementary information (sometimes called an *aside* area) at the right. This layout can be realized with a [FlexLayout](#).

The following example shows an implementation of this layout using a [FlexLayout](#) nested in another:


```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="FlexLayoutDemos.Views.HolyGrailLayoutPage"
              Title="Holy Grail Layout">

    <FlexLayout Direction="Column">

        <!-- Header -->
        <Label Text="HEADER"
              FontSize="18"
              BackgroundColor="Aqua"
              HorizontalTextAlignment="Center" />

        <!-- Body -->
        <FlexLayout FlexLayout.Grow="1">

            <!-- Content -->
            <Label Text="CONTENT"
                  FontSize="18"
                  BackgroundColor="Gray"
                  HorizontalTextAlignment="Center"
                  VerticalTextAlignment="Center"
                  FlexLayout.Grow="1" />

            <!-- Navigation items-->
            <BoxView FlexLayout.Basis="50"
                    FlexLayout.Order="-1"
                    Color="Blue" />

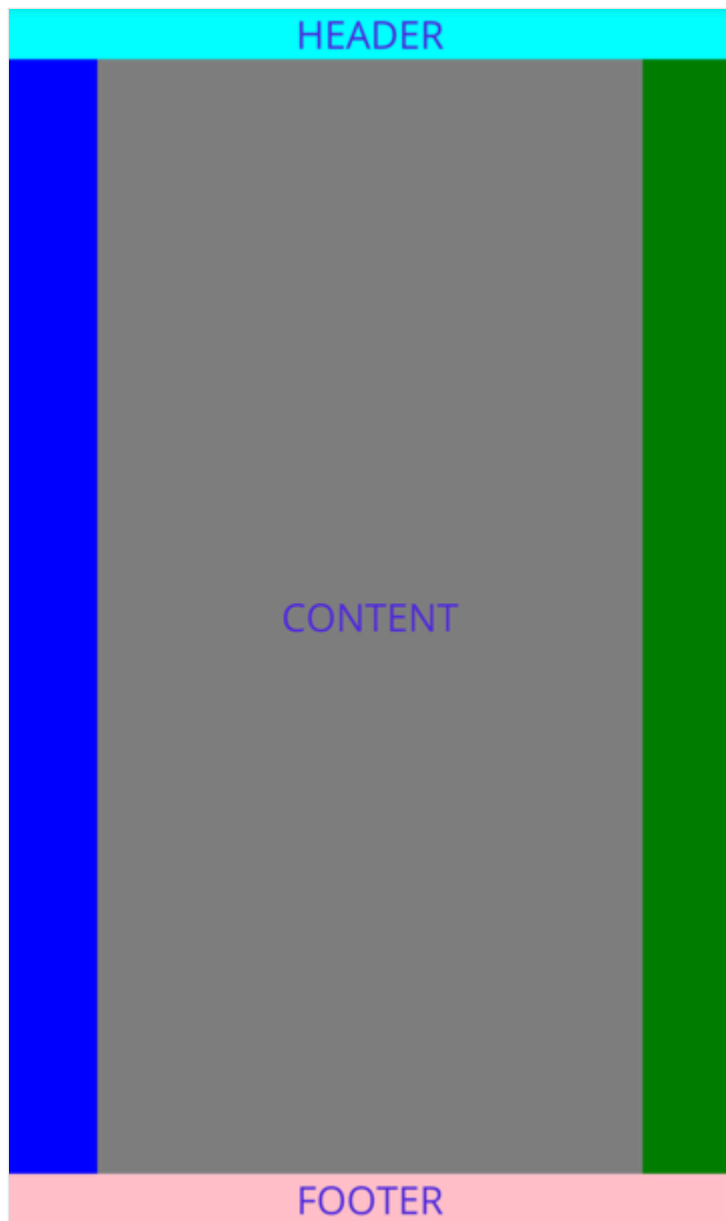
            <!-- Aside items -->
            <BoxView FlexLayout.Basis="50"
                    Color="Green" />

        </FlexLayout>

        <!-- Footer -->
        <Label Text="FOOTER"
              FontSize="18"
              BackgroundColor="Pink"
              HorizontalTextAlignment="Center" />
    </FlexLayout>
</ContentPage>

```

The navigation and aside areas are rendered with a [BoxView](#) on the left and right. The first [FlexLayout](#) has a vertical main axis and contains three children arranged in a column. These are the header, the body of the page, and the footer. The nested [FlexLayout](#) has a horizontal main axis with three children arranged in a row:



In this example, the `Order` property is set on the first `BoxView` to a value less than its siblings to cause it to appear as the first item in the row. The `Basis` property is set on both `BoxView` objects to give them a width of 50 device-independent units. The `Grow` property is set on the nested `FlexLayout` to indicate that this `FlexLayout` should occupy all of the unused vertical space within the outer `FlexLayout`. In addition, the `Grow` property is set on the `Label` representing the content, to indicate that this content is to occupy all of the unused horizontal space within the nested `FlexLayout`.

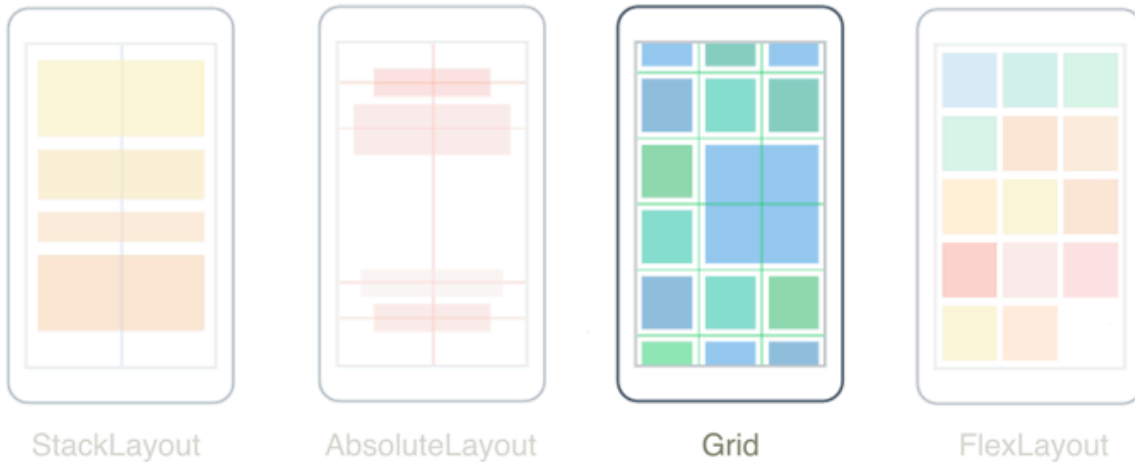
ⓘ Note

There's also a `Shrink` property that you can use when the size of children exceeds the size of the `FlexLayout` but wrapping is not desired.

Grid

Article • 08/30/2024

 [Browse the sample](#)



The .NET Multi-platform App UI (.NET MAUI) [Grid](#), is a layout that organizes its children into rows and columns, which can have proportional or absolute sizes. By default, a [Grid](#) contains one row and one column. In addition, a [Grid](#) can be used as a parent layout that contains other child layouts.

The [Grid](#) should not be confused with tables, and is not intended to present tabular data. Unlike HTML tables, a [Grid](#) is intended for laying out content. For displaying tabular data, consider using a [ListView](#) or [CollectionView](#).

The [Grid](#) class defines the following properties:

- `Column`, of type `int`, which is an attached property that indicates the column alignment of a view within a parent [Grid](#). The default value of this property is 0. A validation callback ensures that when the property is set, its value is greater than or equal to 0.
- `ColumnDefinitions`, of type `ColumnDefinitionCollection`, is a list of `ColumnDefinition` objects that define the width of the grid columns.
- `ColumnSpacing`, of type `double`, indicates the distance between grid columns. The default value of this property is 0.
- `ColumnSpan`, of type `int`, which is an attached property that indicates the total number of columns that a view spans within a parent [Grid](#). The default value of this property is 1. A validation callback ensures that when the property is set, its value is greater than or equal to 1.
- `Row`, of type `int`, which is an attached property that indicates the row alignment of a view within a parent [Grid](#). The default value of this property is 0. A validation

callback ensures that when the property is set, its value is greater than or equal to 0.

- `RowDefinitions`, of type `RowDefinitionCollection`, is a list of `RowDefinition` objects that define the height of the grid rows.
- `RowSpacing`, of type `double`, indicates the distance between grid rows. The default value of this property is 0.
- `RowSpan`, of type `int`, which is an attached property that indicates the total number of rows that a view spans within a parent `Grid`. The default value of this property is 1. A validation callback ensures that when the property is set, its value is greater than or equal to 1.

These properties are backed by `BindableProperty` objects, which means that the properties can be targets of data bindings and styled.

Rows and columns

By default, a `Grid` contains one row and one column:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="GridTutorial.MainPage">
    <Grid Margin="20,35,20,20">
        <Label Text="By default, a Grid contains one row and one column." />
    </Grid>
</ContentPage>
```

In this example, the `Grid` contains a single child `Label` that's automatically positioned in a single location:

By default, a Grid contains one row and one column.

The layout behavior of a `Grid` can be defined with the `RowDefinitions` and `ColumnDefinitions` properties, which are collections of `RowDefinition` and `ColumnDefinition` objects, respectively. These collections define the row and column characteristics of a `Grid`, and should contain one `RowDefinition` object for each row in the `Grid`, and one `ColumnDefinition` object for each column in the `Grid`.

The `RowDefinition` class defines a `Height` property, of type `GridLength`, and the `ColumnDefinition` class defines a `Width` property, of type `GridLength`. The `GridLength`

struct specifies a row height or a column width in terms of the `GridUnitType` enumeration, which has three members:

- `Absolute` – the row height or column width is a value in device-independent units (a number in XAML).
- `Auto` – the row height or column width is autosized based on the cell contents (`Auto` in XAML).
- `Star` – leftover row height or column width is allocated proportionally (a number followed by `*` in XAML).

A `Grid` row with a `Height` property of `Auto` constrains the height of views in that row in the same way as a vertical `StackLayout`. Similarly, a column with a `Width` property of `Auto` works much like a horizontal `StackLayout`.

⊗ Caution

Try to ensure that as few rows and columns as possible are set to `Auto` size. Each auto-sized row or column will cause the layout engine to perform additional layout calculations. Instead, use fixed size rows and columns if possible. Alternatively, set rows and columns to occupy a proportional amount of space with the `GridUnitType.Star` enumeration value.

The following XAML shows how to create a `Grid` with three rows and two columns:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="GridDemos.Views.XAML.BasicGridPage"
              Title="Basic Grid demo">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="2*" />
            <RowDefinition Height="*" />
            <RowDefinition Height="100" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>
        ...
    </Grid>
</ContentPage>
```

In this example, the [Grid](#) has an overall height that is the height of the page. The [Grid](#) knows that the height of the third row is 100 device-independent units. It subtracts that height from its own height, and allocates the remaining height proportionally between the first and second rows based on the number before the star. In this example, the height of the first row is twice that of the second row.

The two `ColumnDefinition` objects both set the `Width` to `*`, which is the same as `1*`, meaning that the width of the screen is divided equally beneath the two columns.

Important

The default value of the `RowDefinition.Height` property is `*`. Similarly, the default value of the `ColumnDefinition.Width` property is `*`. Therefore, it's not necessary to set these properties in cases where these defaults are acceptable.

Child views can be positioned in specific [Grid](#) cells with the `Grid.Column` and `Grid.Row` attached properties. In addition, to make child views span across multiple rows and columns, use the `Grid.RowSpan` and `Grid.ColumnSpan` attached properties.

The following XAML shows the same [Grid](#) definition, and also positions child views in specific [Grid](#) cells:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="GridDemos.Views.XAML.BasicGridPage"
             Title="Basic Grid demo">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="2*" />
            <RowDefinition />
            <RowDefinition Height="100" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition />
            <ColumnDefinition />
        </Grid.ColumnDefinitions>
        <BoxView Color="Green" />
        <Label Text="Row 0, Column 0"
              HorizontalOptions="Center"
              VerticalOptions="Center" />
        <BoxView Grid.Column="1"
              Color="Blue" />
        <Label Grid.Column="1"
              Text="Row 0, Column 1"
              HorizontalOptions="Center"
```

```

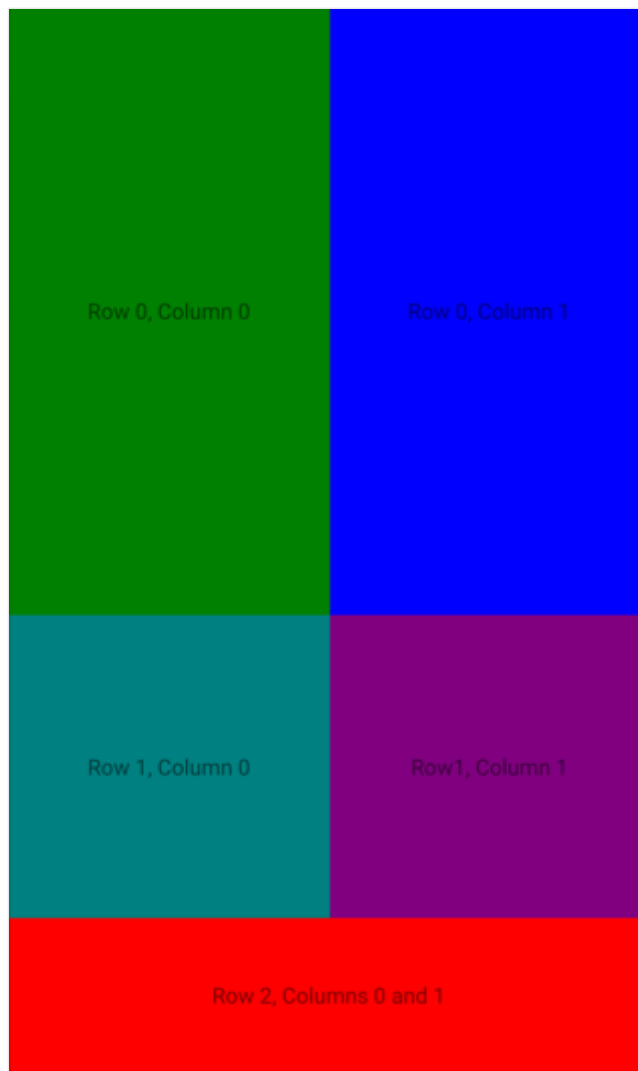
        VerticalOptions="Center" />
<BoxView Grid.Row="1"
        Color="Teal" />
<Label Grid.Row="1"
        Text="Row 1, Column 0"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
<BoxView Grid.Row="1"
        Grid.Column="1"
        Color="Purple" />
<Label Grid.Row="1"
        Grid.Column="1"
        Text="Row1, Column 1"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
<BoxView Grid.Row="2"
        Grid.ColumnSpan="2"
        Color="Red" />
<Label Grid.Row="2"
        Grid.ColumnSpan="2"
        Text="Row 2, Columns 0 and 1"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
</Grid>
</ContentPage>

```

⚠ Note

The `Grid.Row` and `Grid.Column` properties are both indexed from 0, and so `Grid.Row="2"` refers to the third row while `Grid.Column="1"` refers to the second column. In addition, both of these properties have a default value of 0, and so don't need to be set on child views that occupy the first row or first column of a [Grid](#).

In this example, all three `Grid` rows are occupied by `BoxView` and `Label` views. The third row is 100 device-independent units high, with the first two rows occupying the remaining space (the first row is twice as high as the second row). The two columns are equal in width and divide the `Grid` in half. The `BoxView` in the third row spans both columns:



In addition, child views in a [Grid](#) can share cells. The order that the children appear in the XAML is the order that the children are placed in the [Grid](#). In the previous example, the [Label](#) objects are only visible because they are rendered on top of the [BoxView](#) objects. The [Label](#) objects would not be visible if the [BoxView](#) objects were rendered on top of them.

The equivalent C# code is:

C#

```
public class BasicGridPage : ContentPage
{
    public BasicGridPage()
    {
        Grid grid = new Grid
        {
            RowDefinitions =
            {
                new RowDefinition { Height = new GridLength(2,
GridUnitType.Star) },
                new RowDefinition(),
                new RowDefinition { Height = new GridLength(100) }
            },

```



```

        ColumnDefinitions =
        {
            new ColumnDefinition(),
            new ColumnDefinition()
        }
    };

    // Row 0
    // The BoxView and Label are in row 0 and column 0, and so only need
to be added to the
    // Grid to obtain the default row and column settings.
    grid.Add(new BoxView
    {
        Color = Colors.Green
    });
    grid.Add(new Label
    {
        Text = "Row 0, Column 0",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.Center
    });

    // This BoxView and Label are in row 0 and column 1, which are
specified as arguments
    // to the Add method.
    grid.Add(new BoxView
    {
        Color = Colors.Blue
    }, 1, 0);
    grid.Add(new Label
    {
        Text = "Row 0, Column 1",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.Center
    }, 1, 0);

    // Row 1
    // This BoxView and Label are in row 1 and column 0, which are
specified as arguments
    // to the Add method overload.
    grid.Add(new BoxView
    {
        Color = Colors.Teal
    }, 0, 1);
    grid.Add(new Label
    {
        Text = "Row 1, Column 0",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.Center
    }, 0, 1);

    // This BoxView and Label are in row 1 and column 1, which are
specified as arguments
    // to the Add method overload.
    grid.Add(new BoxView

```

```

{
    Color = Colors.Purple
}, 1, 1);
grid.Add(new Label
{
    Text = "Row1, Column 1",
    HorizontalOptions = LayoutOptions.Center,
    VerticalOptions = LayoutOptions.Center
}, 1, 1);

// Row 2
// Alternatively, the BoxView and Label can be positioned in cells
with the Grid.SetRow
// and Grid.SetColumn methods. Here, the Grid.SetColumnSpan method
is used to span two columns.
BoxView boxView = new BoxView { Color = Colors.Red };
Grid.SetRow(boxView, 2);
Grid.SetColumnSpan(boxView, 2);
Label label = new Label
{
    Text = "Row 2, Column 0 and 1",
    HorizontalOptions = LayoutOptions.Center,
    VerticalOptions = LayoutOptions.Center
};
Grid.SetRow(label, 2);
Grid.SetColumnSpan(label, 2);

grid.Add(boxView);
grid.Add(label);

Title = "Basic Grid demo";
Content = grid;
}
}

```

In code, to specify the height of a `RowDefinition` object, and the width of a `ColumnDefinition` object, you use values of the `GridLength` structure, often in combination with the `GridUnitType` enumeration.

ⓘ Note

`Grid` also defines an `AddWithSpan` extension method that adds a view to the `Grid` at the specified row and column with the specified row and column spans.

Simplify row and column definitions

In XAML, the row and column characteristics of a `Grid` can be specified using a simplified syntax that avoids having to define `RowDefinition` and `ColumnDefinition` objects for

each row and column. Instead, the `RowDefinitions` and `ColumnDefinitions` properties can be set to strings containing comma-delimited `GridUnitType` values, from which type converters built into .NET MAUI create `RowDefinition` and `ColumnDefinition` objects:

XAML

```
<Grid RowDefinitions="1*, Auto, 25, 14, 20"
      ColumnDefinitions="*, 2*, Auto, 300">
    ...
</Grid>
```

In this example, the `Grid` has five rows and four columns. The third, fourth, and fifth rows are set to absolute heights, with the second row auto-sizing to its content. The remaining height is then allocated to the first row.

The fourth column is set to an absolute width, with the third column auto-sizing to its content. The remaining width is allocated proportionally between the first and second columns based on the number before the star. In this example, the width of the second column is twice that of the first column (because `*` is identical to `1*`).

Space between rows and columns

By default, `Grid` rows and columns have no space between them. This can be changed by setting the `RowSpacing` and `ColumnSpacing` properties, respectively:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="GridDemos.Views.XAML.GridSpacingPage"
             Title="Grid spacing demo">
    <Grid RowSpacing="6"
          ColumnSpacing="6">
        ...
    </Grid>
</ContentPage>
```

This example creates a `Grid` whose rows and columns are separated by 6 device-independent units of space:



💡 Tip

The `RowSpacing` and `ColumnSpacing` properties can be set to negative values to make cell contents overlap.

The equivalent C# code is:

C#

```
public class GridSpacingPage : ContentPage
{
    public GridSpacingPage()
    {
        Grid grid = new Grid
        {
            RowSpacing = 6,
            ColumnSpacing = 6,
            ...
        };
        ...

        Content = grid;
    }
}
```

```
}  
}
```

Alignment

Child views in a [Grid](#) can be positioned within their cells by the `HorizontalOptions` and `VerticalOptions` properties. These properties can be set to the following fields from the `LayoutOptions` struct:

- `Start`
- `Center`
- `End`
- `Fill`

The following XAML creates a [Grid](#) with nine equal-size cells, and places a [Label](#) in each cell with a different alignment:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="GridDemos.Views.XAML.GridAlignmentPage"  
             Title="Grid alignment demo">  
    <Grid>  
        <Grid.RowDefinitions>  
            <RowDefinition />  
            <RowDefinition />  
            <RowDefinition />  
        </Grid.RowDefinitions>  
        <Grid.ColumnDefinitions>  
            <ColumnDefinition />  
            <ColumnDefinition />  
            <ColumnDefinition />  
        </Grid.ColumnDefinitions>  
  
        <BoxView Color="AliceBlue" />  
        <Label Text="Upper left"  
              HorizontalOptions="Start"  
              VerticalOptions="Start" />  
        <BoxView Grid.Column="1"  
              Color="LightSkyBlue" />  
        <Label Grid.Column="1"  
              Text="Upper center"  
              HorizontalOptions="Center"  
              VerticalOptions="Start" />  
        <BoxView Grid.Column="2"  
              Color="CadetBlue" />  
        <Label Grid.Column="2"
```

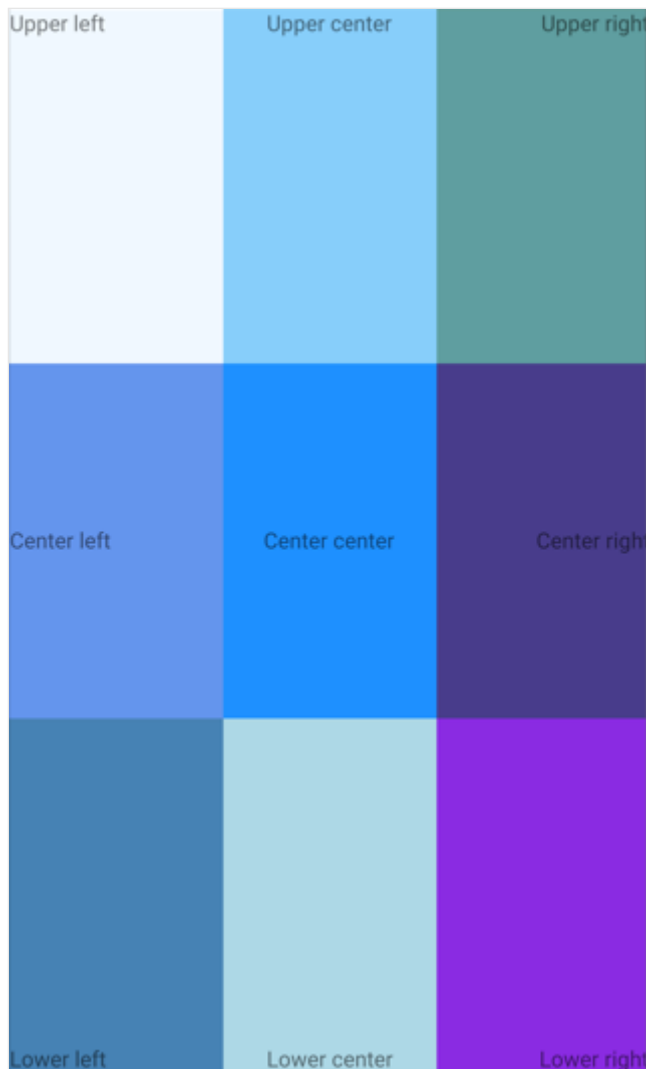
```

        Text="Upper right"
        HorizontalOptions="End"
        VerticalOptions="Start" />
<BoxView Grid.Row="1"
        Color="CornflowerBlue" />
<Label Grid.Row="1"
        Text="Center left"
        HorizontalOptions="Start"
        VerticalOptions="Center" />
<BoxView Grid.Row="1"
        Grid.Column="1"
        Color="DodgerBlue" />
<Label Grid.Row="1"
        Grid.Column="1"
        Text="Center center"
        HorizontalOptions="Center"
        VerticalOptions="Center" />
<BoxView Grid.Row="1"
        Grid.Column="2"
        Color="DarkSlateBlue" />
<Label Grid.Row="1"
        Grid.Column="2"
        Text="Center right"
        HorizontalOptions="End"
        VerticalOptions="Center" />
<BoxView Grid.Row="2"
        Color="SteelBlue" />
<Label Grid.Row="2"
        Text="Lower left"
        HorizontalOptions="Start"
        VerticalOptions="End" />
<BoxView Grid.Row="2"
        Grid.Column="1"
        Color="LightBlue" />
<Label Grid.Row="2"
        Grid.Column="1"
        Text="Lower center"
        HorizontalOptions="Center"
        VerticalOptions="End" />
<BoxView Grid.Row="2"
        Grid.Column="2"
        Color="BlueViolet" />
<Label Grid.Row="2"
        Grid.Column="2"
        Text="Lower right"
        HorizontalOptions="End"
        VerticalOptions="End" />
</Grid>
</ContentPage>

```

In this example, the [Label](#) objects in each row are all identically aligned vertically, but use different horizontal alignments. Alternatively, this can be thought of as the [Label](#)

objects in each column being identically aligned horizontally, but using different vertical alignments:



The equivalent C# code is:

C#

```
public class GridAlignmentPage : ContentPage
{
    public GridAlignmentPage()
    {
        Grid grid = new Grid
        {
            RowDefinitions =
            {
                new RowDefinition(),
                new RowDefinition(),
                new RowDefinition()
            },
            ColumnDefinitions =
            {
                new ColumnDefinition(),
                new ColumnDefinition(),
                new ColumnDefinition()
            }
        }
    }
}
```

```

    }
};

// Row 0
grid.Add(new BoxView
{
    Color = Colors.AliceBlue
});
grid.Add(new Label
{
    Text = "Upper left",
    HorizontalOptions = LayoutOptions.Start,
    VerticalOptions = LayoutOptions.Start
});

grid.Add(new BoxView
{
    Color = Colors.LightSkyBlue
}, 1, 0);
grid.Add(new Label
{
    Text = "Upper center",
    HorizontalOptions = LayoutOptions.Center,
    VerticalOptions = LayoutOptions.Start
}, 1, 0);

grid.Add(new BoxView
{
    Color = Colors.CadetBlue
}, 2, 0);
grid.Add(new Label
{
    Text = "Upper right",
    HorizontalOptions = LayoutOptions.End,
    VerticalOptions = LayoutOptions.Start
}, 2, 0);

// Row 1
grid.Add(new BoxView
{
    Color = Colors.CornflowerBlue
}, 0, 1);
grid.Add(new Label
{
    Text = "Center left",
    HorizontalOptions = LayoutOptions.Start,
    VerticalOptions = LayoutOptions.Center
}, 0, 1);

grid.Add(new BoxView
{
    Color = Colors.DodgerBlue
}, 1, 1);
grid.Add(new Label
{

```



```

        Text = "Center center",
        HorizontalOptions = LayoutOptions.Center,
        VerticalOptions = LayoutOptions.Center
    }, 1, 1);

grid.Add(new BoxView
{
    Color = Colors.DarkSlateBlue
}, 2, 1);
grid.Add(new Label
{
    Text = "Center right",
    HorizontalOptions = LayoutOptions.End,
    VerticalOptions = LayoutOptions.Center
}, 2, 1);

// Row 2
grid.Add(new BoxView
{
    Color = Colors.SteelBlue
}, 0, 2);
grid.Add(new Label
{
    Text = "Lower left",
    HorizontalOptions = LayoutOptions.Start,
    VerticalOptions = LayoutOptions.End
}, 0, 2);

grid.Add(new BoxView
{
    Color = Colors.LightBlue
}, 1, 2);
grid.Add(new Label
{
    Text = "Lower center",
    HorizontalOptions = LayoutOptions.Center,
    VerticalOptions = LayoutOptions.End
}, 1, 2);

grid.Add(new BoxView
{
    Color = Colors.BlueViolet
}, 2, 2);
grid.Add(new Label
{
    Text = "Lower right",
    HorizontalOptions = LayoutOptions.End,
    VerticalOptions = LayoutOptions.End
}, 2, 2);

Title = "Grid alignment demo";
Content = grid;
    }
}

```

Nested Grid objects

A [Grid](#) can be used as a parent layout that contains nested child [Grid](#) objects, or other child layouts. When nesting [Grid](#) objects, the `Grid.Row`, `Grid.Column`, `Grid.RowSpan`, and `Grid.ColumnSpan` attached properties always refer to the position of views within their parent [Grid](#).

The following XAML shows an example of nesting [Grid](#) objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:converters="clr-namespace:GridDemos.Converters"
             x:Class="GridDemos.Views.XAML.ColorSlidersGridPage"
             Title="Nested Grids demo">

    <ContentPage.Resources>
        <converters:DoubleToIntConverter x:Key="doubleToInt" />

        <Style TargetType="Label">
            <Setter Property="HorizontalTextAlignment"
                    Value="Center" />
        </Style>
    </ContentPage.Resources>

    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="500" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>

        <BoxView x:Name="boxView"
                Color="Black" />

        <Grid Grid.Row="1"
              Margin="20">
            <Grid.RowDefinitions>
                <RowDefinition />
                <RowDefinition />
                <RowDefinition />
                <RowDefinition />
                <RowDefinition />
                <RowDefinition />
            </Grid.RowDefinitions>
            <Slider x:Name="redSlider"
                    ValueChanged="OnSliderValueChanged" />
            <Label Grid.Row="1"
                   Text="{Binding Source={x:Reference redSlider},
                                Path=Value,
                                Converter={StaticResource doubleToInt},
                                ConverterParameter=255,
                                StringFormat='Red = {0}'}" />
        </Grid>
    </Grid>
</ContentPage>
```

```

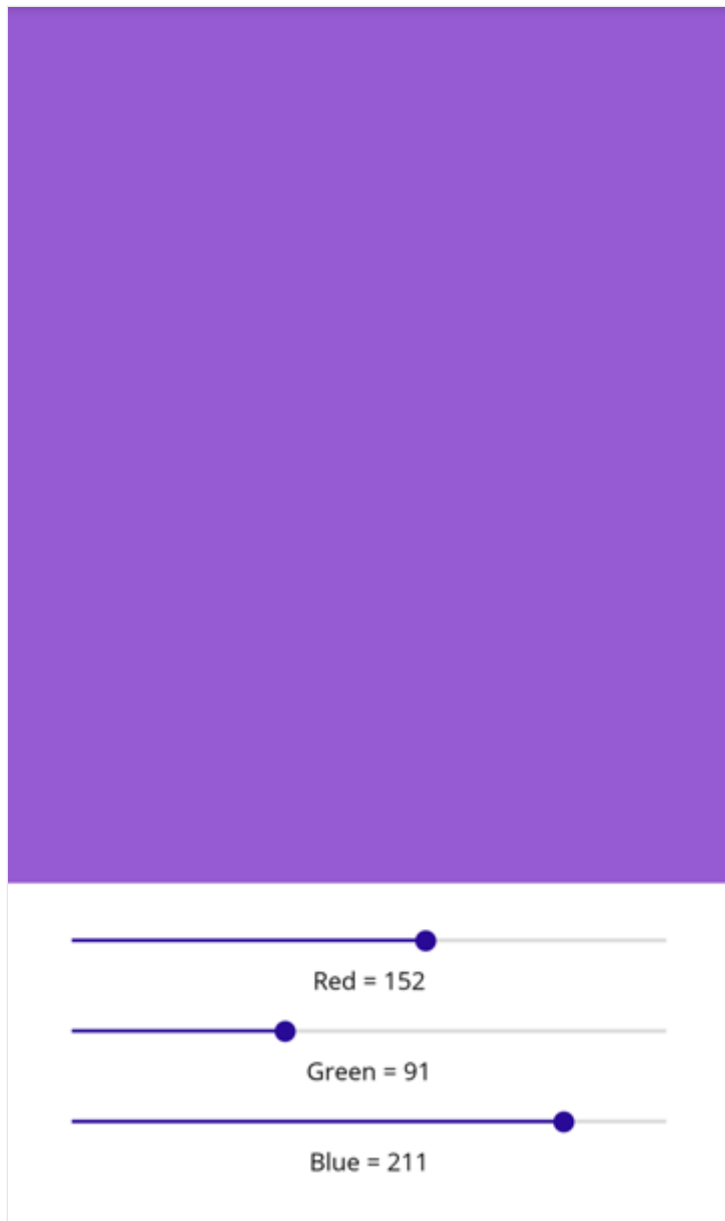
        <Slider x:Name="greenSlider"
            Grid.Row="2"
            ValueChanged="OnSliderValueChanged" />
        <Label Grid.Row="3"
            Text="{Binding Source={x:Reference greenSlider},
                Path=Value,
                Converter={StaticResource doubleToInt},
                ConverterParameter=255,
                StringFormat='Green = {0}'}" />

        <Slider x:Name="blueSlider"
            Grid.Row="4"
            ValueChanged="OnSliderValueChanged" />
        <Label Grid.Row="5"
            Text="{Binding Source={x:Reference blueSlider},
                Path=Value,
                Converter={StaticResource doubleToInt},
                ConverterParameter=255,
                StringFormat='Blue = {0}'}" />

    </Grid>
</Grid>
</ContentPage>

```

In this example, the root `Grid` contains a `BoxView` in its first row, and a child `Grid` in its second row. The child `Grid` contains `Slider` objects that manipulate the color displayed by the `BoxView`, and `Label` objects that display the value of each `Slider`:



❗ Important

The deeper you nest [Grid](#) objects and other layouts, the more layout calculations will be performed which may impact performance. For more information, see [Choose the correct layout](#).

The equivalent C# code is:

C#

```
public class ColorSlidersGridPage : ContentPage
{
    BoxView boxView;
    Slider redSlider;
    Slider greenSlider;
    Slider blueSlider;

    public ColorSlidersGridPage()
```

```

{
    // Create an implicit style for the Labels
    Style labelStyle = new Style(typeof(Label))
    {
        Setters =
        {
            new Setter { Property =
Label.HorizontalTextAlignmentProperty, Value = TextAlignment.Center }
        }
    };
    Resources.Add(labelStyle);

    // Root page layout
    Grid rootGrid = new Grid
    {
        RowDefinitions =
        {
            new RowDefinition { HeightRequest = 500 },
            new RowDefinition()
        }
    };

    boxView = new BoxView { Color = Colors.Black };
    rootGrid.Add(boxView);

    // Child page layout
    Grid childGrid = new Grid
    {
        Margin = new Thickness(20),
        RowDefinitions =
        {
            new RowDefinition(),
            new RowDefinition(),
            new RowDefinition(),
            new RowDefinition(),
            new RowDefinition(),
            new RowDefinition()
        }
    };

    DoubleToIntConverter doubleToInt = new DoubleToIntConverter();

    redSlider = new Slider();
    redSlider.ValueChanged += OnSliderValueChanged;
    childGrid.Add(redSlider);

    Label redLabel = new Label();
    redLabel.SetBinding(Label.TextProperty, new Binding("Value",
converter: doubleToInt, converterParameter: "255", stringFormat: "Red =
{0}"), source: redSlider));
    Grid.SetRow(redLabel, 1);
    childGrid.Add(redLabel);

    greenSlider = new Slider();
    greenSlider.ValueChanged += OnSliderValueChanged;

```

```

        Grid.SetRow(greenSlider, 2);
        childGrid.Add(greenSlider);

        Label greenLabel = new Label();
        greenLabel.SetBinding(Label.TextProperty, new Binding("Value",
converter: doubleToInt, converterParameter: "255", stringFormat: "Green =
{0}"), source: greenSlider));
        Grid.SetRow(greenLabel, 3);
        childGrid.Add(greenLabel);

        blueSlider = new Slider();
        blueSlider.ValueChanged += OnSliderValueChanged;
        Grid.SetRow(blueSlider, 4);
        childGrid.Add(blueSlider);

        Label blueLabel = new Label();
        blueLabel.SetBinding(Label.TextProperty, new Binding("Value",
converter: doubleToInt, converterParameter: "255", stringFormat: "Blue =
{0}"), source: blueSlider));
        Grid.SetRow(blueLabel, 5);
        childGrid.Add(blueLabel);

        // Place the child Grid in the root Grid
        rootGrid.Add(childGrid, 0, 1);

        Title = "Nested Grids demo";
        Content = rootGrid;
    }

    void OnSliderValueChanged(object sender, ValueChangedEventArgs e)
    {
        boxView.Color = new Color(redSlider.Value, greenSlider.Value,
blueSlider.Value);
    }
}

```

HorizontalStackLayout

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) [HorizontalStackLayout](#) organizes child views in a one-dimensional horizontal stack, and is a more performant alternative to a [StackLayout](#). In addition, a [HorizontalStackLayout](#) can be used as a parent layout that contains other child layouts.

The [HorizontalStackLayout](#) defines the following properties:

- `Spacing`, of type `double`, indicates the amount of space between each child view. The default value of this property is 0.

This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings and styled.

The following XAML shows how to create a [HorizontalStackLayout](#) that contains different child views:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.HorizontalStackLayoutPage">
    <HorizontalStackLayout Margin="20">
        <Rectangle Fill="Red"
                   HeightRequest="30"
                   WidthRequest="30" />
        <Label Text="Red"
               FontSize="18" />
    </HorizontalStackLayout>
</ContentPage>
```

This example creates a [HorizontalStackLayout](#) containing a [Rectangle](#) and a [Label](#) object. By default, there is no space between the child views:



ⓘ Note

The value of the `Margin` property represents the distance between an element and its adjacent elements. For more information, see [Position controls](#).

Space between child views

The spacing between child views in a [HorizontalStackLayout](#) can be changed by setting the `Spacing` property to a `double` value:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.HorizontalStackLayoutPage">
    <HorizontalStackLayout Margin="20"
                          Spacing="10">
        <Rectangle Fill="Red"
                  HeightRequest="30"
                  WidthRequest="30" />
        <Label Text="Red"
              FontSize="18" />
    </HorizontalStackLayout>
</ContentPage>
```

This example creates a [HorizontalStackLayout](#) containing a [Rectangle](#) and a [Label](#) object, that have ten device-independent units of space between them:



💡 Tip

The `Spacing` property can be set to negative values to make child views overlap.

Position and size child views

The size and position of child views within a [HorizontalStackLayout](#) depends upon the values of the child views' [HeightRequest](#) and [WidthRequest](#) properties, and the values of their `VerticalOptions` properties. In a [HorizontalStackLayout](#), child views expand to fill the available height when their size isn't explicitly set.

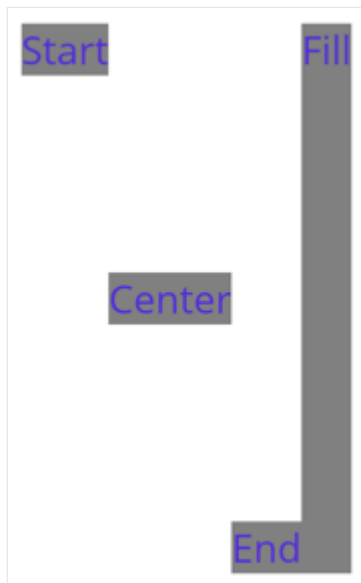
The `VerticalOptions` properties of a [HorizontalStackLayout](#), and its child views, can be set to fields from the `LayoutOptions` struct, which encapsulates an *alignment* layout preference. This layout preference determines the position and size of a child view within its parent layout.

The following XAML example sets alignment preferences on each child view in the [HorizontalStackLayout](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.HorizontalStackLayoutPage">
    <HorizontalStackLayout Margin="20"
                          HeightRequest="200">
        <Label Text="Start"
               BackgroundColor="Gray"
               VerticalOptions="Start" />
        <Label Text="Center"
               BackgroundColor="Gray"
               VerticalOptions="Center" />
        <Label Text="End"
               BackgroundColor="Gray"
               VerticalOptions="End" />
        <Label Text="Fill"
               BackgroundColor="Gray"
               VerticalOptions="Fill" />
    </HorizontalStackLayout>
</ContentPage>
```

In this example, alignment preferences are set on the [Label](#) objects to control their position within the [HorizontalStackLayout](#). The `Start`, `Center`, `End`, and `Fill` fields are used to define the alignment of the [Label](#) objects within the parent [HorizontalStackLayout](#):



A [HorizontalStackLayout](#) only respects the alignment preferences on child views that are in the opposite direction to the orientation of the layout. Therefore, the [Label](#) child views within the [HorizontalStackLayout](#) set their `VerticalOptions` properties to one of the alignment fields:

- `Start`, which positions the `Label` at the start of the `HorizontalStackLayout`.
- `Center`, which vertically centers the `Label` in the `HorizontalStackLayout`.
- `End`, which positions the `Label` at the end of the `HorizontalStackLayout`.
- `Fill`, which ensures that the `Label` fills the height of the `HorizontalStackLayout`.

For more information about alignment, see [Align and position .NET MAUI controls](#).

Nest `HorizontalStackLayout` objects

A `HorizontalStackLayout` can be used as a parent layout that contains other nested child layouts.

The following XAML shows an example of nesting `VerticalStackLayout` objects in a `HorizontalStackLayout`:

XAML

```
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.HorizontalStackLayoutPage">
    <HorizontalStackLayout Margin="20"
                          Spacing="6">
        <Label Text="Primary colors:" />
        <VerticalStackLayout Spacing="6">
            <Rectangle Fill="Red"
                      WidthRequest="30"
                      HeightRequest="30" />
            <Rectangle Fill="Yellow"
                      WidthRequest="30"
                      HeightRequest="30" />
            <Rectangle Fill="Blue"
                      WidthRequest="30"
                      HeightRequest="30" />
        </VerticalStackLayout>
        <Label Text="Secondary colors:" />
        <VerticalStackLayout Spacing="6">
            <Rectangle Fill="Green"
                      WidthRequest="30"
                      HeightRequest="30" />
            <Rectangle Fill="Orange"
                      WidthRequest="30"
                      HeightRequest="30" />
            <Rectangle Fill="Purple"
                      WidthRequest="30"
                      HeightRequest="30" />
        </VerticalStackLayout>
    </HorizontalStackLayout>
</ContentPage>
```

In this example, the parent [HorizontalStackLayout](#) contains two nested [VerticalStackLayout](#) objects:



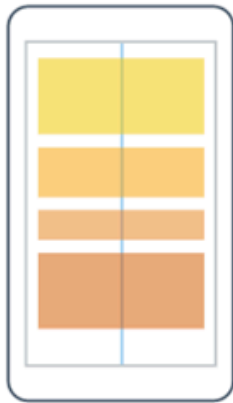
📘 Important

The deeper you nest layout objects the more layout calculations will be performed, which may impact performance. For more information, see [Choose the correct layout](#).

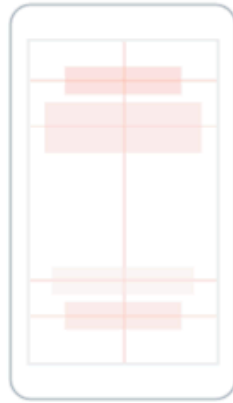
StackLayout

Article • 09/30/2024

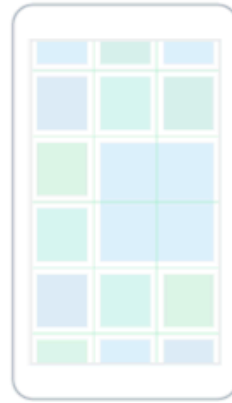
 [Browse the sample](#)



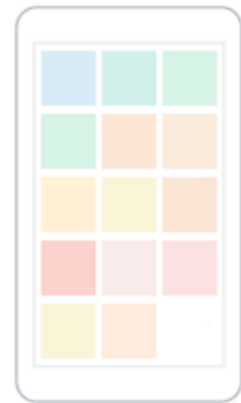
StackLayout



AbsoluteLayout



Grid



FlexLayout

The .NET Multi-platform App UI (.NET MAUI) [StackLayout](#) organizes child views in a one-dimensional stack, either horizontally or vertically. By default, a [StackLayout](#) is oriented vertically. In addition, a [StackLayout](#) can be used as a parent layout that contains other child layouts.

The [StackLayout](#) class defines the following properties:

- **Orientation**, of type `StackOrientation`, represents the direction in which child views are positioned. The default value of this property is `Vertical`.
- **Spacing**, of type `double`, indicates the amount of space between each child view. The default value of this property is 0.

These properties are backed by [BindableProperty](#) objects, which means that the properties can be targets of data bindings and styled.

Vertical orientation

The following XAML shows how to create a vertically oriented [StackLayout](#) that contains different child views:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="StackLayoutDemos.Views.XAML.VerticalStackLayoutPage"
              Title="Vertical StackLayout demo">
    <StackLayout Margin="20">
```

```

<Label Text="Primary colors" />
<BoxView Color="Red"
          HeightRequest="40" />
<BoxView Color="Yellow"
          HeightRequest="40" />
<BoxView Color="Blue"
          HeightRequest="40" />
<Label Text="Secondary colors" />
<BoxView Color="Green"
          HeightRequest="40" />
<BoxView Color="Orange"
          HeightRequest="40" />
<BoxView Color="Purple"
          HeightRequest="40" />
</StackLayout>
</ContentPage>

```

This example creates a vertical [StackLayout](#) containing [Label](#) and [BoxView](#) objects. By default, there's no space between the child views:



The equivalent C# code is:

```

C#

public class VerticalStackLayoutPage : ContentPage
{
    public VerticalStackLayoutPage()
    {
        Title = "Vertical StackLayout demo";

        StackLayout stackLayout = new StackLayout { Margin = new
Thickness(20) };

        stackLayout.Add(new Label { Text = "Primary colors" });
        stackLayout.Add(new BoxView { Color = Colors.Red, HeightRequest = 40
});
        stackLayout.Add(new BoxView { Color = Colors.Yellow, HeightRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Blue, HeightRequest =

```

```

40 });
    stackLayout.Add(new Label { Text = "Secondary colors" });
    stackLayout.Add(new BoxView { Color = Colors.Green, HeightRequest =
40 });
    stackLayout.Add(new BoxView { Color = Colors.Orange, HeightRequest =
40 });
    stackLayout.Add(new BoxView { Color = Colors.Purple, HeightRequest =
40 });

    Content = stackLayout;
}
}

```

ⓘ Note

The value of the `Margin` property represents the distance between an element and its adjacent elements. For more information, see [Position controls](#).

Horizontal orientation

The following XAML shows how to create a horizontally oriented [StackLayout](#) by setting its `Orientation` property to `Horizontal`:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="StackLayoutDemos.Views.XAML.HorizontalStackLayoutPage"
    Title="Horizontal StackLayout demo">
    <StackLayout Margin="20"
        Orientation="Horizontal"
        HorizontalOptions="Center">
        <BoxView Color="Red"
            WidthRequest="40" />
        <BoxView Color="Yellow"
            WidthRequest="40" />
        <BoxView Color="Blue"
            WidthRequest="40" />
        <BoxView Color="Green"
            WidthRequest="40" />
        <BoxView Color="Orange"
            WidthRequest="40" />
        <BoxView Color="Purple"
            WidthRequest="40" />
    </StackLayout>
</ContentPage>

```

This example creates a horizontal [StackLayout](#) containing [BoxView](#) objects, with no space between the child views:



The equivalent C# code is:

C#

```
public class HorizontalStackLayoutPage : ContentPage
{
    public HorizontalStackLayoutPage()
    {
        Title = "Horizontal StackLayout demo";

        StackLayout stackLayout = new StackLayout
        {
            Margin = new Thickness(20),
            Orientation = StackOrientation.Horizontal,
            HorizontalOptions = LayoutOptions.Center
        };

        stackLayout.Add(new BoxView { Color = Colors.Red, WidthRequest = 40
    });
        stackLayout.Add(new BoxView { Color = Colors.Yellow, WidthRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Blue, WidthRequest = 40
    });
        stackLayout.Add(new BoxView { Color = Colors.Green, WidthRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Orange, WidthRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Purple, WidthRequest =
40 });

        Content = stackLayout;
    }
}
```

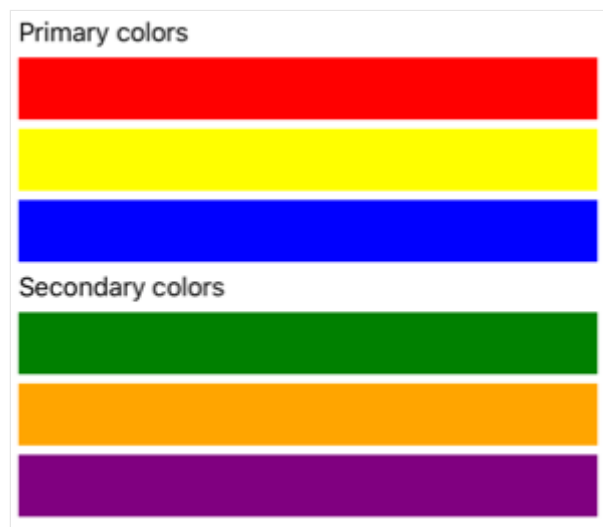
Space between child views

The spacing between child views in a [StackLayout](#) can be changed by setting the `Spacing` property to a `double` value:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="StackLayoutDemos.Views.XAML.StackLayoutSpacingPage"
              Title="StackLayout Spacing demo">
    <StackLayout Margin="20"
                 Spacing="6">
        <Label Text="Primary colors" />
        <BoxView Color="Red"
                  HeightRequest="40" />
        <BoxView Color="Yellow"
                  HeightRequest="40" />
        <BoxView Color="Blue"
                  HeightRequest="40" />
        <Label Text="Secondary colors" />
        <BoxView Color="Green"
                  HeightRequest="40" />
        <BoxView Color="Orange"
                  HeightRequest="40" />
        <BoxView Color="Purple"
                  HeightRequest="40" />
    </StackLayout>
</ContentPage>
```

This example creates a vertical [StackLayout](#) containing [Label](#) and [BoxView](#) objects that have six device-independent units of vertical space between them:



💡 Tip

The `Spacing` property can be set to negative values to make child views overlap.

The equivalent C# code is:

C#

```
public class StackLayoutSpacingPage : ContentPage
{
    public StackLayoutSpacingPage()
    {
        Title = "StackLayout Spacing demo";

        StackLayout stackLayout = new StackLayout
        {
            Margin = new Thickness(20),
            Spacing = 6
        };

        stackLayout.Add(new Label { Text = "Primary colors" });
        stackLayout.Add(new BoxView { Color = Colors.Red, HeightRequest = 40
});
        stackLayout.Add(new BoxView { Color = Colors.Yellow, HeightRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Blue, HeightRequest =
40 });
        stackLayout.Add(new Label { Text = "Secondary colors" });
        stackLayout.Add(new BoxView { Color = Colors.Green, HeightRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Orange, HeightRequest =
40 });
        stackLayout.Add(new BoxView { Color = Colors.Purple, HeightRequest =
40 });

        Content = stackLayout;
    }
}
```

Position and size of child views

The size and position of child views within a [StackLayout](#) depends upon the values of the child views' [HeightRequest](#) and [WidthRequest](#) properties, and the values of their [HorizontalOptions](#) and [VerticalOptions](#) properties. In a vertical [StackLayout](#), child views expand to fill the available width when their size isn't explicitly set. Similarly, in a horizontal [StackLayout](#), child views expand to fill the available height when their size isn't explicitly set.

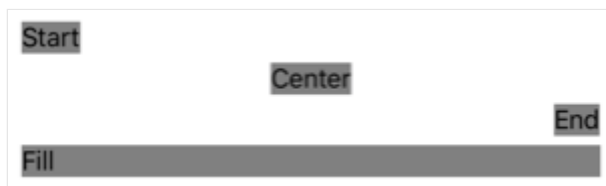
The [HorizontalOptions](#) and [VerticalOptions](#) properties of a [StackLayout](#), and its child views, can be set to fields from the [LayoutOptions](#) struct, which encapsulates an *alignment* layout preference. This layout preference determines the position and size of a child view within its parent layout.

The following XAML example sets alignment preferences on each child view in the [StackLayout](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.XAML.AlignmentPage"
             Title="Alignment demo">
    <StackLayout Margin="20"
                 Spacing="6">
        <Label Text="Start"
               BackgroundColor="Gray"
               HorizontalOptions="Start" />
        <Label Text="Center"
               BackgroundColor="Gray"
               HorizontalOptions="Center" />
        <Label Text="End"
               BackgroundColor="Gray"
               HorizontalOptions="End" />
        <Label Text="Fill"
               BackgroundColor="Gray"
               HorizontalOptions="Fill" />
    </StackLayout>
</ContentPage>
```

In this example, alignment preferences are set on the [Label](#) objects to control their position within the [StackLayout](#). The `Start`, `Center`, `End`, and `Fill` fields are used to define the alignment of the [Label](#) objects within the parent [StackLayout](#):



A [StackLayout](#) only respects the alignment preferences on child views that are in the opposite direction to the [StackLayout](#) orientation. Therefore, the [Label](#) child views within the vertically oriented [StackLayout](#) set their `HorizontalOptions` properties to one of the alignment fields:

- `Start`, which positions the [Label](#) on the left-hand side of the [StackLayout](#).
- `Center`, which centers the [Label](#) in the [StackLayout](#).
- `End`, which positions the [Label](#) on the right-hand side of the [StackLayout](#).
- `Fill`, which ensures that the [Label](#) fills the width of the [StackLayout](#).

The equivalent C# code is:

C#

```
public class AlignmentPage : ContentPage
{
    public AlignmentPage()
    {
        Title = "Alignment demo";

        StackLayout stackLayout = new StackLayout
        {
            Margin = new Thickness(20),
            Spacing = 6
        };

        stackLayout.Add(new Label { Text = "Start", BackgroundColor =
Colors.Gray, HorizontalOptions = LayoutOptions.Start });
        stackLayout.Add(new Label { Text = "Center", BackgroundColor =
Colors.Gray, HorizontalOptions = LayoutOptions.Center });
        stackLayout.Add(new Label { Text = "End", BackgroundColor =
Colors.Gray, HorizontalOptions = LayoutOptions.End });
        stackLayout.Add(new Label { Text = "Fill", BackgroundColor =
Colors.Gray, HorizontalOptions = LayoutOptions.Fill });

        Content = stackLayout;
    }
}
```

For more information about alignment, see [Align views in layouts](#).

Nested StackLayout objects

A [StackLayout](#) can be used as a parent layout that contains nested child [StackLayout](#) objects, or other child layouts.

The following XAML shows an example of nesting [StackLayout](#) objects:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="StackLayoutDemos.Views.XAML.CombinedStackLayoutPage"
    Title="Combined StackLayouts demo">
    <StackLayout Margin="20">
        ...
        <Frame BorderColor="Black"
            Padding="5">
            <StackLayout Orientation="Horizontal"
                Spacing="15">
                <BoxView Color="Red"
                    WidthRequest="40" />
            </StackLayout>
        </Frame>
    </StackLayout>
</ContentPage>
```

```

        <Label Text="Red"
              FontSize="18"
              VerticalOptions="Center" />
    </StackLayout>
</Frame>
<Frame BorderColor="Black"
        Padding="5">
    <StackLayout Orientation="Horizontal"
                  Spacing="15">
        <BoxView Color="Yellow"
                  WidthRequest="40" />
        <Label Text="Yellow"
                FontSize="18"
                VerticalOptions="Center" />
    </StackLayout>
</Frame>
<Frame BorderColor="Black"
        Padding="5">
    <StackLayout Orientation="Horizontal"
                  Spacing="15">
        <BoxView Color="Blue"
                  WidthRequest="40" />
        <Label Text="Blue"
                FontSize="18"
                VerticalOptions="Center" />
    </StackLayout>
</Frame>
...
</StackLayout>
</ContentPage>

```

In this example, the parent `StackLayout` contains nested `StackLayout` objects inside `Frame` objects. The parent `StackLayout` is oriented vertically, while the child `StackLayout` objects are oriented horizontally:



Important

The deeper you nest [StackLayout](#) objects and other layouts, the more layout calculations will be performed which may impact performance. For more information, see [Choose the correct layout](#).

The equivalent C# code is:

C#

```
public class CombinedStackLayoutPage : ContentPage
{
    public CombinedStackLayoutPage()
    {
        Title = "Combined StackLayouts demo";

        Frame frame1 = new Frame
        {
            BorderColor = Colors.Black,
            Padding = new Thickness(5)
        };
        StackLayout frame1StackLayout = new StackLayout
        {
            Orientation = StackOrientation.Horizontal,
            Spacing = 15
        };
        frame1StackLayout.Add(new BoxView { Color = Colors.Red, WidthRequest
= 40 });
        frame1StackLayout.Add(new Label { Text = "Red", FontSize = 22,
VerticalOptions = LayoutOptions.Center });
        frame1.Content = frame1StackLayout;

        Frame frame2 = new Frame
        {
            BorderColor = Colors.Black,
            Padding = new Thickness(5)
        };
        StackLayout frame2StackLayout = new StackLayout
        {
            Orientation = StackOrientation.Horizontal,
            Spacing = 15
        };
        frame2StackLayout.Add(new BoxView { Color = Colors.Yellow,
WidthRequest = 40 });
        frame2StackLayout.Add(new Label { Text = "Yellow", FontSize = 22,
VerticalOptions = LayoutOptions.Center });
        frame2.Content = frame2StackLayout;

        Frame frame3 = new Frame
        {
            BorderColor = Colors.Black,
```

```

        Padding = new Thickness(5)
    };
    StackLayout frame3StackLayout = new StackLayout
    {
        Orientation = StackOrientation.Horizontal,
        Spacing = 15
    };
    frame3StackLayout.Add(new BoxView { Color = Colors.Blue,
WidthRequest = 40 });
    frame3StackLayout.Add(new Label { Text = "Blue", FontSize = 22,
VerticalOptions = LayoutOptions.Center });
    frame3.Content = frame3StackLayout;

    ...

    StackLayout stackLayout = new StackLayout { Margin = new
Thickness(20) };
    stackLayout.Add(new Label { Text = "Primary colors" });
    stackLayout.Add(frame1);
    stackLayout.Add(frame2);
    stackLayout.Add(frame3);
    stackLayout.Add(new Label { Text = "Secondary colors" });
    stackLayout.Add(frame4);
    stackLayout.Add(frame5);
    stackLayout.Add(frame6);

    Content = stackLayout;
}
}

```

VerticalStackLayout

Article • 09/30/2024

The .NET Multi-platform App UI (.NET MAUI) [VerticalStackLayout](#) organizes child views in a one-dimensional vertical stack, and is a more performant alternative to a [StackLayout](#). In addition, a [VerticalStackLayout](#) can be used as a parent layout that contains other child layouts.

The [VerticalStackLayout](#) defines the following properties:

- `Spacing`, of type `double`, indicates the amount of space between each child view. The default value of this property is 0.

This property is backed by a [BindableProperty](#) object, which means that it can be the target of data bindings and styled.

The following XAML shows how to create a [VerticalStackLayout](#) that contains different child views:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.VerticalStackLayoutPage">
    <VerticalStackLayout Margin="20">
        <Label Text="Primary colors" />
        <Rectangle Fill="Red"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Yellow"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Blue"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Label Text="Secondary colors" />
        <Rectangle Fill="Green"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Orange"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Purple"
                   HeightRequest="30"
                   WidthRequest="300" />
    </VerticalStackLayout>
</ContentPage>
```

This example creates a [VerticalStackLayout](#) containing [Label](#) and [Rectangle](#) objects. By default, there is no space between the child views:



ⓘ Note

The value of the `Margin` property represents the distance between an element and its adjacent elements. For more information, see [Position controls](#).

Space between child views

The spacing between child views in a [VerticalStackLayout](#) can be changed by setting the `Spacing` property to a `double` value:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.VerticalStackLayoutPage">
    <VerticalStackLayout Margin="20"
                       Spacing="10">
        <Label Text="Primary colors" />
        <Rectangle Fill="Red"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Yellow"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Blue"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Label Text="Secondary colors" />
        <Rectangle Fill="Green"
                   HeightRequest="30"
                   WidthRequest="300" />
        <Rectangle Fill="Orange" />
    </VerticalStackLayout>
</ContentPage>
```



```

        HeightRequest="30"
        WidthRequest="300" />
    <Rectangle Fill="Purple"
        HeightRequest="30"
        WidthRequest="300" />
</VerticalStackLayout>
</ContentPage>

```

This example creates a [VerticalStackLayout](#) containing [Label](#) and [Rectangle](#) objects that have ten device-independent units of space between the child views:



💡 Tip

The `Spacing` property can be set to negative values to make child views overlap.

Position and size child views

The size and position of child views within a [VerticalStackLayout](#) depends upon the values of the child views' [HeightRequest](#) and [WidthRequest](#) properties, and the values of their `HorizontalOptions` properties. In a [VerticalStackLayout](#), child views expand to fill the available width when their size isn't explicitly set.

The `HorizontalOptions` properties of a [VerticalStackLayout](#), and its child views, can be set to fields from the `LayoutOptions` struct, which encapsulates an *alignment* layout preference. This layout preference determines the position and size of a child view within its parent layout.

💡 Tip

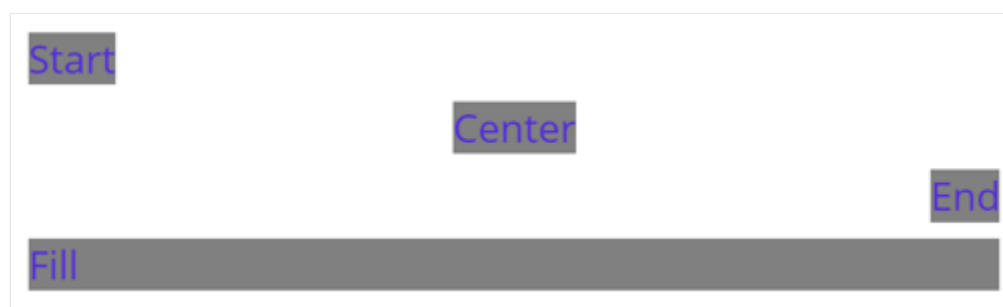
Don't set the `HorizontalOptions` property of a [VerticalStackLayout](#) unless you need to. The default value of `LayoutOptions.Fill` allows for the best layout optimization. Changing this property has a cost and consumes memory, even when setting it back to its default value.

The following XAML example sets alignment preferences on each child view in the [VerticalStackLayout](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.VerticalStackLayoutPage">
    <VerticalStackLayout Margin="20"
                        Spacing="6">
        <Label Text="Start"
              BackgroundColor="Gray"
              HorizontalOptions="Start" />
        <Label Text="Center"
              BackgroundColor="Gray"
              HorizontalOptions="Center" />
        <Label Text="End"
              BackgroundColor="Gray"
              HorizontalOptions="End" />
        <Label Text="Fill"
              BackgroundColor="Gray"
              HorizontalOptions="Fill" />
    </VerticalStackLayout>
</ContentPage>
```

In this example, alignment preferences are set on the [Label](#) objects to control their position within the [VerticalStackLayout](#). The `Start`, `Center`, `End`, and `Fill` fields are used to define the alignment of the [Label](#) objects within the parent [VerticalStackLayout](#):



A [VerticalStackLayout](#) only respects the alignment preferences on child views that are in the opposite direction to the orientation of the layout. Therefore, the [Label](#) child views within the [VerticalStackLayout](#) set their `HorizontalOptions` properties to one of the alignment fields:

- `Start`, which positions the `Label` on the left-hand side of the `VerticalStackLayout`.
- `Center`, which centers the `Label` in the `VerticalStackLayout`.
- `End`, which positions the `Label` on the right-hand side of the `VerticalStackLayout`.
- `Fill`, which ensures that the `Label` fills the width of the `VerticalStackLayout`.

Nest `VerticalStackLayout` objects

A `VerticalStackLayout` can be used as a parent layout that contains other nested child layouts.

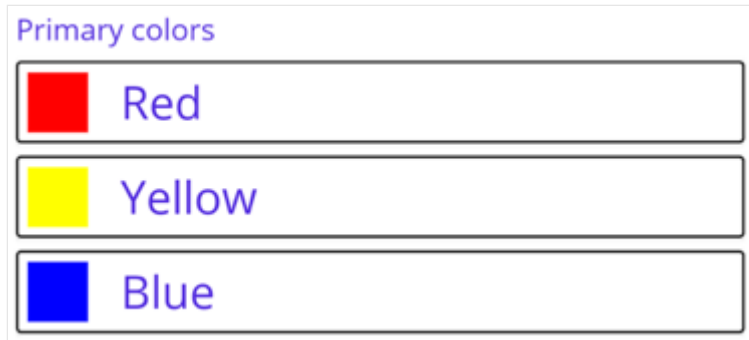
The following XAML shows an example of nesting `HorizontalStackLayout` objects in a `VerticalStackLayout`:

XAML

```
<ContentPage xmlns="http://xamarin.com/schemas/2014/forms"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="StackLayoutDemos.Views.VerticalStackLayoutPage">
    <VerticalStackLayout Margin="20"
                       Spacing="6">
        <Label Text="Primary colors" />
        <Frame BorderColor="Black"
              Padding="5">
            <HorizontalStackLayout Spacing="15">
                <Rectangle Fill="Red"
                          HeightRequest="30"
                          WidthRequest="30" />
                <Label Text="Red"
                      FontSize="18" />
            </HorizontalStackLayout>
        </Frame>
        <Frame BorderColor="Black"
              Padding="5">
            <HorizontalStackLayout Spacing="15">
                <Rectangle Fill="Yellow"
                          HeightRequest="30"
                          WidthRequest="30" />
                <Label Text="Yellow"
                      FontSize="18" />
            </HorizontalStackLayout>
        </Frame>
        <Frame BorderColor="Black"
              Padding="5">
            <HorizontalStackLayout Spacing="15">
                <Rectangle Fill="Blue"
                          HeightRequest="30"
                          WidthRequest="30" />
                <Label Text="Blue"
                      FontSize="18" />
            </HorizontalStackLayout>
        </Frame>
    </VerticalStackLayout>
</ContentPage>
```

```
</Frame>
</VerticalStackLayout>
</ContentPage>
```

In this example, the parent `VerticalStackLayout` contains nested `HorizontalStackLayout` objects inside `Frame` objects:



📌 Important

The deeper you nest layout objects the more layout calculations will be performed, which may impact performance. For more information, see [Choose the correct layout](#).

Custom layouts

Article • 03/04/2024



[Browse the sample](#)

.NET Multi-platform App UI (.NET MAUI) defines multiple layouts classes that each arrange their children in a different way. A layout can be thought of as list of views with rules and properties that define how to arrange those views within the layout. Examples of layouts include [Grid](#), [AbsoluteLayout](#), and [VerticalStackLayout](#).

.NET MAUI layout classes derive from the abstract [Layout](#) class. This class delegates cross-platform layout and measurement to a layout manager class. The [Layout](#) class also contains an overridable [CreateLayoutManager\(\)](#) method that derived layouts can use to specify the layout manager.

Each layout manager class implements the [ILayoutManager](#) interface, which specifies that [Measure](#) and [ArrangeChildren](#) implementations must be provided:

- The [Measure](#) implementation calls [IView.Measure](#) on each view in the layout, and returns the total size of the layout given the constraints.
- The [ArrangeChildren](#) implementation determines where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view with its appropriate bounds. The return value is the actual size of the layout.

.NET MAUI's layouts have pre-defined layout managers to handle their layout. However, sometimes it's necessary to organize page content using a layout that isn't provided by .NET MAUI. This can be achieved by producing your own custom layout, which requires you to have an understanding of how .NET MAUI's cross-platform layout process works.

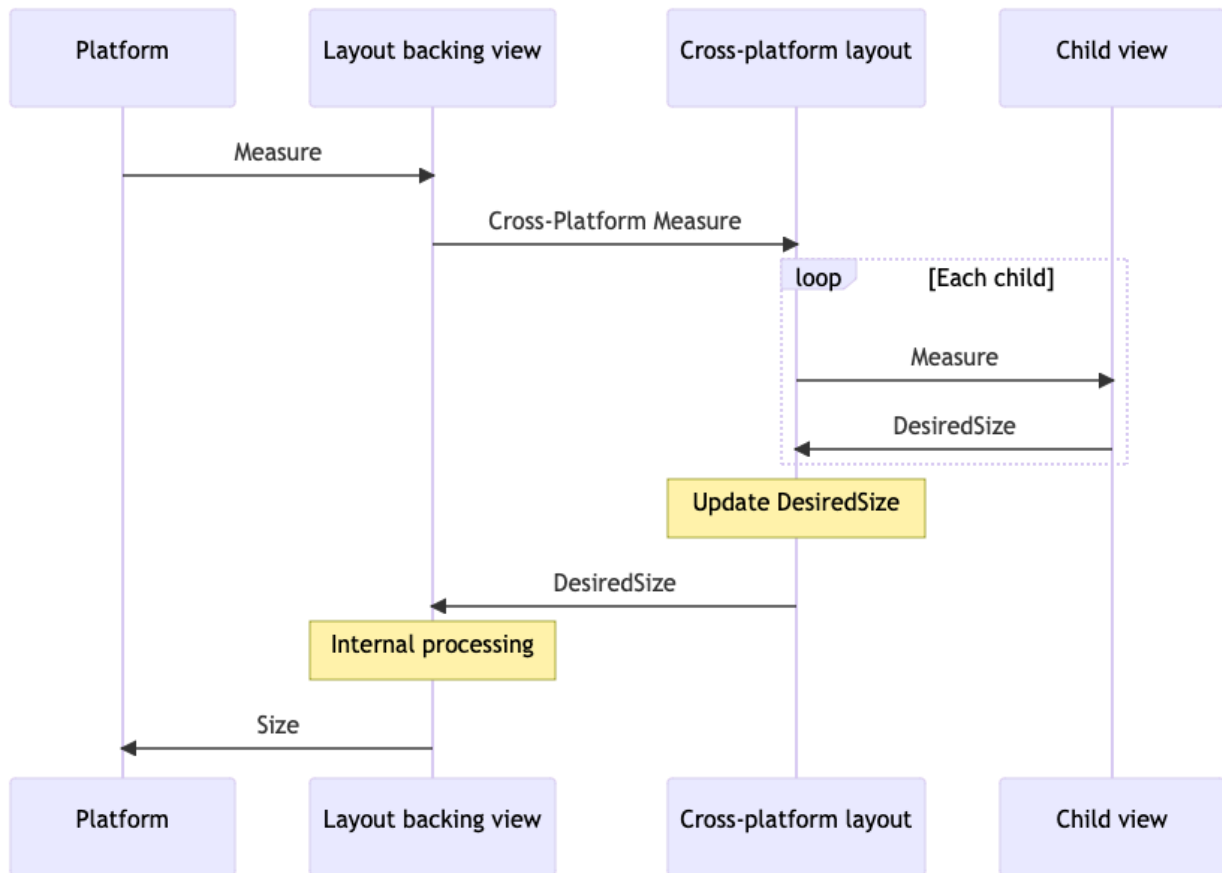
Layout process

.NET MAUI's cross-platform layout process builds on top of the native layout process on each platform. Generally, the layout process is initiated by the native layout system. The cross-platform process runs when a layout or content control initiates it as a result of being measured or arranged by the native layout system.

ⓘ Note

Each platform handles layout slightly differently. However, .NET MAUI's cross-platform layout process aims to be as platform-agnostic as possible.

The following diagram shows the process when a native layout system initiates layout measurement:

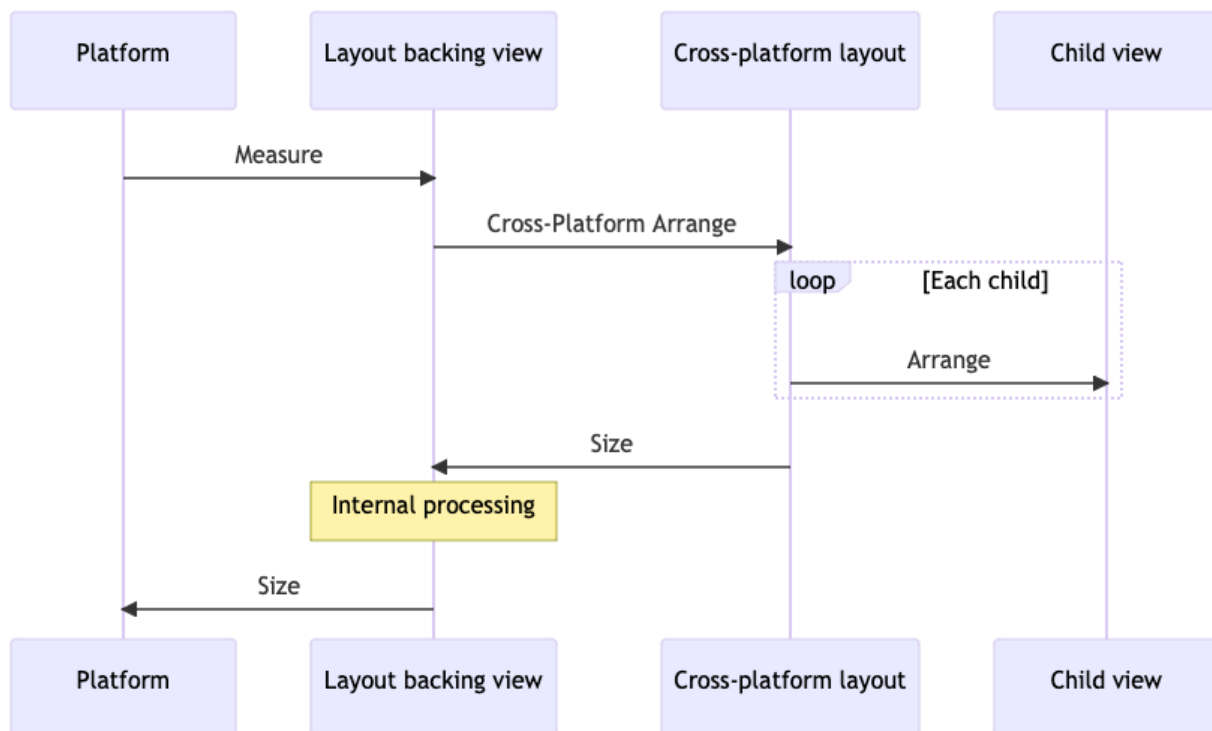


All .NET MAUI layouts have a single backing view on each platform:

- On Android, this backing view is `LayoutViewGroup`.
- On iOS and Mac Catalyst, this backing view is `LayoutView`.
- On Windows, this backing view is `LayoutPanel1`.

When the native layout system for a platform requests the measurement of one of these backing views, the backing view calls the `Layout.CrossPlatformMeasure` method. This is the point at which control is passed from the native layout system to .NET MAUI's layout system. `Layout.CrossPlatformMeasure` calls the layout managers' `Measure` method. This method is responsible for measuring child views by calling `IView.Measure` on each view in the layout. The view measures its native control, and updates its `DesiredSize` property based on that measurement. This value is returned to the backing view as the result of the `CrossPlatformMeasure` method. The backing view performs whatever internal processing it needs to do, and returns its measured size to the platform.

The following diagram shows the process when a native layout system initiates layout arrangement:



When the native layout system for a platform requests the arrangement, or layout, of one of these backing views, the backing view calls the [Layout.CrossPlatformArrange](#) method. This is the point at which control is passed from the native layout system to .NET MAUI's layout system. [Layout.CrossPlatformArrange](#) calls the layout managers' [ArrangeChildren](#) method. This method is responsible for determining where each view should be placed within the bounds of the layout, and calls [Arrange](#) on each view to set its location. The size of the layout is returned to the backing view as the result of the `CrossPlatformArrange` method. The backing view performs whatever internal processing it needs to do, and returns the actual size to the platform.

ⓘ Note

[ILayoutManager.Measure](#) may be called multiple times before [ArrangeChildren](#) is called, because a platform may need to perform some speculative measurements before arranging views.

Custom layout approaches

There are two main approaches to creating a custom layout:

1. Create a custom layout type, which is usually a subclass of an existing layout type or of [Layout](#), and override [CreateLayoutManager\(\)](#) in your custom layout type. Then, provide an [ILayoutManager](#) implementation that contains your custom layout logic. For more information, see [Create a custom layout type](#).

2. Modify the behavior of an existing layout type by creating a type that implements [ILayoutManagerFactory](#). Then, use this layout manager factory to replace .NET MAUI's default layout manager for the existing layout with your own [ILayoutManager](#) implementation that contains your custom layout logic. For more information, see [Modify the behavior of an existing layout](#).

Create a custom layout type

The process for creating a custom layout type is to:

1. Create a class that subclasses an existing layout type or the [Layout](#) class, and override [CreateLayoutManager\(\)](#) in your custom layout type. For more information, see [Subclass a layout](#).
2. Create a layout manager class that derives from an existing layout manager, or that implements the [ILayoutManager](#) interface directly. In your layout manager class, you should:
 - a. Override, or implement, the [Measure](#) method to calculate the total size of the layout given its constraints.
 - b. Override, or implement, the [ArrangeChildren](#) method to size and position all the children within the layout.

For more information, see [Create a layout manager](#).

3. Consume your custom layout type by adding it to a [Page](#), and by adding children to the layout. For more information, see [Consume the layout type](#).

An orientation-sensitive `HorizontalWrapLayout` is used to demonstrate this process. `HorizontalWrapLayout` is similar to a [HorizontalStackLayout](#) in that it arranges its children horizontally across the page. However, it wraps the display of children to a new row when it encounters the right edge of its container

ⓘ Note

The [sample](#) defines additional custom layouts that can be used to understand how to produce a custom layout.

Subclass a layout

To create a custom layout type you must first subclass an existing layout type, or the [Layout](#) class. Then, override [CreateLayoutManager\(\)](#) in your layout type and return a

new instance of the layout manager for your layout type:

```
C#  
  
using Microsoft.Maui.Layouts;  
  
public class HorizontalWrapLayout : HorizontalStackLayout  
{  
    protected override ILayoutManager CreateLayoutManager()  
    {  
        return new HorizontalWrapLayoutManager(this);  
    }  
}
```

`HorizontalWrapLayout` derives from `HorizontalStackLayout` to use its layout functionality. .NET MAUI layouts delegate cross-platform layout and measurement to a layout manager class. Therefore, the `CreateLayoutManager()` override returns a new instance of the `HorizontalWrapLayoutManager` class, which is the layout manager that's discussed in the next section.

Create a layout manager

A layout manager class is used to perform cross-platform layout and measurement for your custom layout type. It should derive from an existing layout manager, or it should directly implement the `ILayoutManager` interface. `HorizontalWrapLayoutManager` derives from `HorizontalStackLayoutManager` so that it can use its underlying functionality and access members in its inheritance hierarchy:

```
C#  
  
using Microsoft.Maui.Layouts;  
using HorizontalStackLayoutManager =  
    Microsoft.Maui.Layouts.HorizontalStackLayoutManager;  
  
public class HorizontalWrapLayoutManager : HorizontalStackLayoutManager  
{  
    HorizontalWrapLayout _layout;  
  
    public HorizontalWrapLayoutManager(HorizontalWrapLayout  
horizontalWrapLayout) : base(horizontalWrapLayout)  
    {  
        _layout = horizontalWrapLayout;  
    }  
  
    public override Size Measure(double widthConstraint, double  
heightConstraint)  
    {  
    }  
}
```

```

    public override Size ArrangeChildren(Rect bounds)
    {
    }
}

```

The `HorizontalWrapLayoutManager` constructor stores an instance of the `HorizontalWrapLayout` type in a field, so that it can be accessed throughout the layout manager. The layout manager also overrides the `Measure` and `ArrangeChildren` methods from the `HorizontalStackLayoutManager` class. These methods are where you'll define the logic to implement your custom layout.

Measure the layout size

The purpose of the `ILayoutManager.Measure` implementation is to calculate the total size of the layout. It should do this by calling `IView.Measure` on each child in the layout. It should then use this data to calculate and return the total size of the layout given its constraints.

The following example shows the `Measure` implementation for the `HorizontalWrapLayoutManager` class:

C#

```

public override Size Measure(double widthConstraint, double
heightConstraint)
{
    var padding = _layout.Padding;

    widthConstraint -= padding.HorizontalThickness;

    double currentRowWidth = 0;
    double currentRowHeight = 0;
    double totalWidth = 0;
    double totalHeight = 0;

    for (int n = 0; n < _layout.Count; n++)
    {
        var child = _layout[n];
        if (child.Visibility == Visibility.Collapsed)
        {
            continue;
        }

        var measure = child.Measure(double.PositiveInfinity,
heightConstraint);

        // Will adding this IView put us past the edge?

```

```

    if (currentRowWidth + measure.Width > widthConstraint)
    {
        // Keep track of the width so far
        totalWidth = Math.Max(totalWidth, currentRowWidth);
        totalHeight += currentRowHeight;

        // Account for spacing
        totalHeight += _layout.Spacing;

        // Start over at 0
        currentRowWidth = 0;
        currentRowHeight = measure.Height;
    }
    currentRowWidth += measure.Width;
    currentRowHeight = Math.Max(currentRowHeight, measure.Height);

    if (n < _layout.Count - 1)
    {
        currentRowWidth += _layout.Spacing;
    }
}

// Account for the last row
totalWidth = Math.Max(totalWidth, currentRowWidth);
totalHeight += currentRowHeight;

// Account for padding
totalWidth += padding.HorizontalThickness;
totalHeight += padding.VerticalThickness;

// Ensure that the total size of the layout fits within its constraints
var finalWidth = ResolveConstraints(widthConstraint, Stack.Width,
totalWidth, Stack.MinimumWidth, Stack.MaximumWidth);
var finalHeight = ResolveConstraints(heightConstraint, Stack.Height,
totalHeight, Stack.MinimumHeight, Stack.MaximumHeight);

return new Size(finalWidth, finalHeight);
}

```

The [Measure\(Double, Double\)](#) method enumerates through all of the visible children in the layout, invoking the [IView.Measure](#) method on each child. It then returns the total size of the layout, taking into account the constraints and the values of the [Padding](#) and [Spacing](#) properties. The [ResolveConstraints](#) method is called to ensure that the total size of the layout fits within its constraints.

Important

When enumerating children in the [ILayoutManager.Measure](#) implementation, skip any child whose [Visibility](#) property is set to [Collapsed](#). This ensures that the custom layout won't leave space for invisible children.

Arrange children in the layout

The purpose of the [ArrangeChildren](#) implementation is to size and position all of the children within the layout. To determine where each child should be placed within the bounds of the layout, it should call [Arrange](#) on each child with its appropriate bounds. It should then return a value that represents the actual size of the layout.

Warning

Failure to invoke the [ArrangeChildren](#) method on each child in the layout will result in the child never receiving a correct size or position, and hence the child won't become visible on the page.

The following example shows the [ArrangeChildren](#) implementation for the `HorizontalWrapLayoutManager` class:

C#

```
public override Size ArrangeChildren(Rect bounds)
{
    var padding = Stack.Padding;
    double top = padding.Top + bounds.Top;
    double left = padding.Left + bounds.Left;

    double currentRowTop = top;
    double currentX = left;
    double currentRowHeight = 0;

    double maxStackWidth = currentX;

    for (int n = 0; n < _layout.Count; n++)
    {
        var child = _layout[n];
        if (child.Visibility == Visibility.Collapsed)
        {
            continue;
        }

        if (currentX + child.DesiredSize.Width > bounds.Right)
        {
            // Keep track of our maximum width so far
            maxStackWidth = Math.Max(maxStackWidth, currentX);
        }
    }
}
```

```

        // Move down to the next row
        currentX = left;
        currentRowTop += currentRowHeight + _layout.Spacing;
        currentRowHeight = 0;
    }

    var destination = new Rect(currentX, currentRowTop,
child.DesiredSize.Width, child.DesiredSize.Height);
    child.Arrange(destination);

    currentX += destination.Width + _layout.Spacing;
    currentRowHeight = Math.Max(currentRowHeight, destination.Height);
}

var actual = new Size(maxStackWidth, currentRowTop + currentRowHeight);

// Adjust the size if the layout is set to fill its container
return actual.AdjustForFill(bounds, Stack);
}

```

The `ArrangeChildren` method enumerates through all the visible children in the layout to size and position them within the layout. It does this by invoking `Arrange` on each child with appropriate bounds, that take into account the `Padding` and `Spacing` of the underlying layout. It then returns the actual size of the layout. The `AdjustForFill` method is called to ensure that the size takes into account whether the the layout has its `HorizontalLayoutAlignment` and `VerticalLayoutAlignment` properties set to `LayoutOptions.Fill`.

❗ Important

When enumerating children in the `ArrangeChildren` implementation, skip any child whose `Visibility` property is set to `Collapsed`. This ensures that the custom layout won't leave space for invisible children.

Consume the layout type

The `HorizontalWrapLayout` class can be consumed by placing it in a `Page` derived type:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:layouts="clr-namespace:CustomLayoutDemos.Layouts"
    x:Class="CustomLayoutDemos.Views.HorizontalWrapLayoutPage"
    Title="Horizontal wrap layout">
    <ScrollView Margin="20">

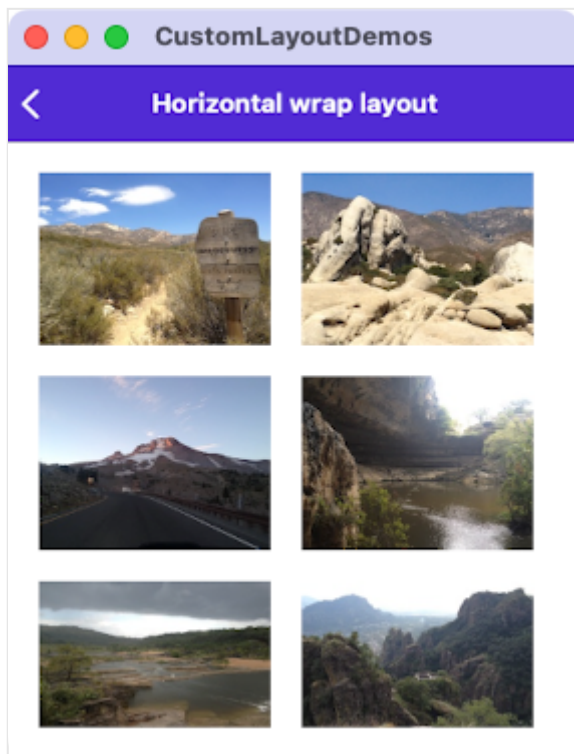
```

```

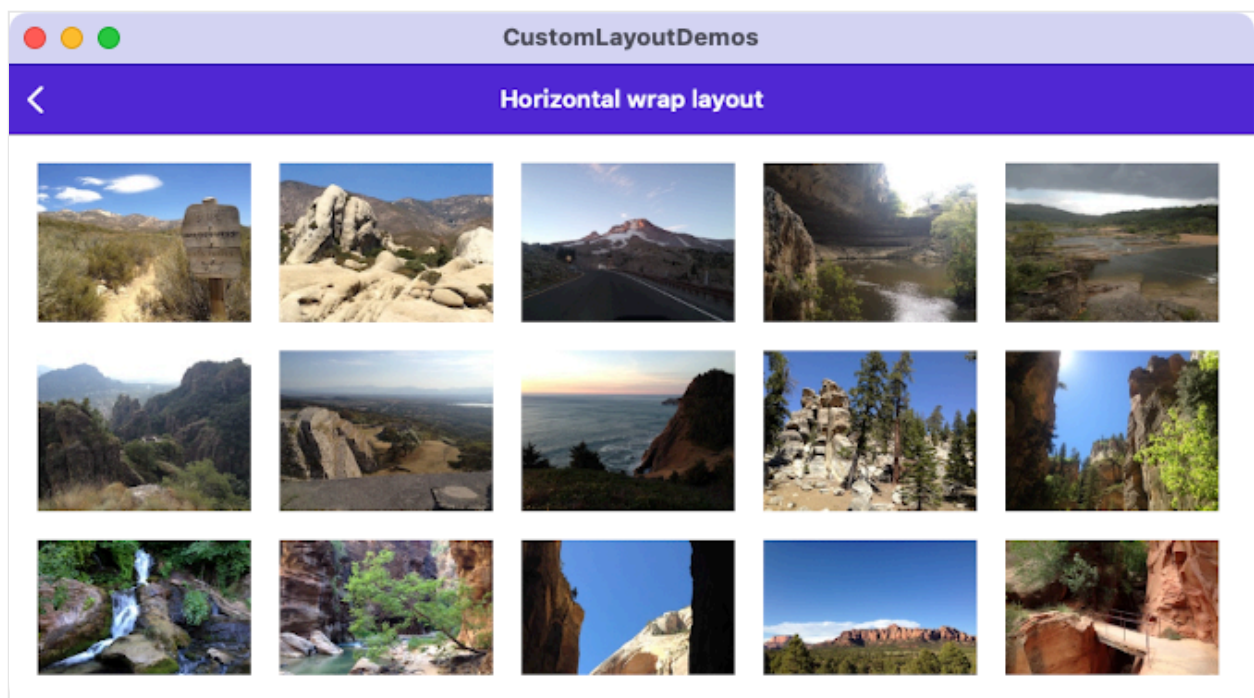
<layouts:HorizontalWrapLayout Spacing="20">
  <Image Source="img_0074.jpg"
    WidthRequest="150" />
  <Image Source="img_0078.jpg"
    WidthRequest="150" />
  <Image Source="img_0308.jpg"
    WidthRequest="150" />
  <Image Source="img_0437.jpg"
    WidthRequest="150" />
  <Image Source="img_0475.jpg"
    WidthRequest="150" />
  <Image Source="img_0613.jpg"
    WidthRequest="150" />
  <!-- More images go here -->
</layouts:HorizontalWrapLayout>
</ScrollView>
</ContentPage>

```

Controls can be added to the `HorizontalWrapLayout` as required. In this example, when the page containing the `HorizontalWrapLayout` appears, the `Image` controls are displayed:



The number of columns in each row depends on the image size, the width of the page, and the number of pixels per device-independent unit:



ⓘ Note

Scrolling is supported by wrapping the `HorizontalWrapLayout` in a [ScrollView](#).

Modify the behavior of an existing layout

In some scenarios you may want to change the behavior of an existing layout type without having to create a custom layout type. For these scenarios you can create a type that implements [ILayoutManagerFactory](#) and use it to replace .NET MAUI's default layout manager for the existing layout with your own [ILayoutManager](#) implementation. This enables you to define a new layout manager for an existing layout, such as providing a custom layout manager for [Grid](#). This can be useful for scenarios where you want to add a new behavior to a layout but don't want to update the type of an existing widely-used layout in your app.

The process for modifying the behavior of an existing layout, with a layout manager factory, is to:

1. Create a layout manager that derives from one of .NET MAUI's layout manager types. For more information, see [Create a custom layout manager](#).
2. Create a type that implements [ILayoutManagerFactory](#). For more information, see [Create a layout manager factory](#).
3. Register your layout manager factory with the app's service provider. For more information, see [Register the layout manager factory](#).

Create a custom layout manager

A layout manager is used to perform cross-platform layout and measurement for a layout. To change the behavior of an existing layout you should create a custom layout manager that derives from the layout manager for the layout:

```
C#

using Microsoft.Maui.Layouts;

public class CustomGridLayoutManager : GridLayoutManager
{
    public CustomGridLayoutManager(IGridLayout layout) : base(layout)
    {
    }

    public override Size Measure(double widthConstraint, double
heightConstraint)
    {
        EnsureRows();
        return base.Measure(widthConstraint, heightConstraint);
    }

    void EnsureRows()
    {
        if (Grid is not Grid grid)
        {
            return;
        }

        // Find the maximum row value from the child views
        int maxRow = 0;
        foreach (var child in grid)
        {
            maxRow = Math.Max(grid.GetRow(child), maxRow);
        }

        // Add more rows if we need them
        for (int n = grid.RowDefinitions.Count; n <= maxRow; n++)
        {
            grid.RowDefinitions.Add(new RowDefinition(GridLength.Star));
        }
    }
}
```

In this example, `CustomGridLayoutManager` derives from .NET MAUI's `GridLayoutManager` class, and overrides its `Measure` method. This custom layout manager ensures that at runtime the `RowDefinitions` for the `Grid` includes enough rows to account for each `Grid.Row` attached property set in a child view. Without this modification, the `RowDefinitions` for the `Grid` would need to be specified at design time.

Important

When modifying the behavior of an existing layout manager, don't forget to ensure that you call the `base.Measure` method from your [Measure](#) implementation.

Create a layout manager factory

The custom layout manager should be created in a layout manager factory. This is achieved by creating a type that implements the [ILayoutManagerFactory](#) interface:

```
C#  
  
using Microsoft.Maui.Layouts;  
  
public class CustomLayoutManagerFactory : ILayoutManagerFactory  
{  
    public ILayoutManager CreateLayoutManager(Layout layout)  
    {  
        if (layout is Grid)  
        {  
            return new CustomGridLayoutManager(layout as IGridLayout);  
        }  
        return null;  
    }  
}
```

In this example, a `CustomGridLayoutManager` instance is returned if the layout is a [Grid](#).

Register the layout manager factory

The layout manager factory should be registered with your app's service provider in your `MauiProgram` class:

```
C#  
  
public static class MauiProgram  
{  
    public static MauiApp CreateMauiApp()  
    {  
        var builder = MauiApp.CreateBuilder();  
        builder  
            .UseMauiApp<App>()  
            .ConfigureFonts(fonts =>  
            {  
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");  
                fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");  
            }  
        );  
    }  
}
```

```

    });

    // Setup a custom layout manager so the default manager for the Grid
    can be replaced.
    builder.Services.Add(new
    ServiceDescriptor(typeof(ILayoutManagerFactory), new
    CustomLayoutManagerFactory()));

    return builder.Build();
}
}

```

Then, when the app renders a [Grid](#) it will use the custom layout manager to ensure that at runtime the [RowDefinitions](#) for the [Grid](#) includes enough rows to account for each `Grid.Row` attached property set in child views.

The following example shows a [Grid](#) that sets the `Grid.Row` attached property in child views, but doesn't set the [RowDefinitions](#) property:

XAML

```

<Grid>
    <Label Text="This Grid demonstrates replacing the LayoutManager for an
    existing layout type." />
    <Label Grid.Row="1"
        Text="In this case, it's a LayoutManager for Grid which
    automatically adds enough rows to accommodate the rows specified in the
    child views' attached properties." />
    <Label Grid.Row="2"
        Text="Notice that the Grid doesn't explicitly specify a
    RowDefinitions collection." />
    <Label Grid.Row="3"
        Text="In MauiProgram.cs, an instance of an ILayoutManagerFactory
    has been added that replaces the default GridLayoutManager. The custom
    manager will automatically add the necessary RowDefinitions at runtime." />
    <Label Grid.Row="5"
        Text="We can even skip some rows, and it will add the intervening
    ones for us (notice the gap between the previous label and this one)." />
</Grid>

```

The layout manager factory uses the custom layout manager to ensure that the [Grid](#) in this example displays correctly, despite the [RowDefinitions](#) property not being set:

This page demonstrates replacing the `LayoutManager` for an existing layout type.

In this case, it's a `LayoutManager` for `Grid` which automatically adds enough rows to accommodate the rows specified in the child views' attached properties.

Notice that the `Grid` doesn't explicitly specify a `RowDefinitions` collection.

In `MauiProgram.cs`, an instance of an `LayoutManagerFactory` has been added that replaces the default `GridLayoutManager`. The custom manager will automatically add the necessary `RowDefinitions` at runtime.

We can even skip some rows, and it will add the intervening ones for us (notice the gap between the previous label and this one).

ContentPage

Article • 10/04/2024



ContentPage



FlyoutPage



NavigationPage



TabbedPage

The .NET Multi-platform App UI (.NET MAUI) [ContentPage](#) displays a single view, which is often a layout such as [Grid](#) or [StackLayout](#), and is the most common page type.

[ContentPage](#) defines the following properties:

- [Content](#) property, of type [View](#), which defines the view that represents the page's content.
- [HideSoftInputOnTapped](#), of type `bool`, which indicates whether tapping anywhere on the page will cause the soft input keyboard to hide if it's visible on Android, iOS, and Mac Catalyst.

These properties are backed by [BindableProperty](#) objects, which means that they can be the target of data bindings, and styled.

In addition, [ContentPage](#) inherits `Title`, `IconImageSource`, `BackgroundImageSource`, `IsBusy`, and `Padding` bindable properties from the [Page](#) class.

ⓘ Note

The `Content` property is the content property of the [ContentPage](#) class, and therefore does not need to be explicitly set from XAML.

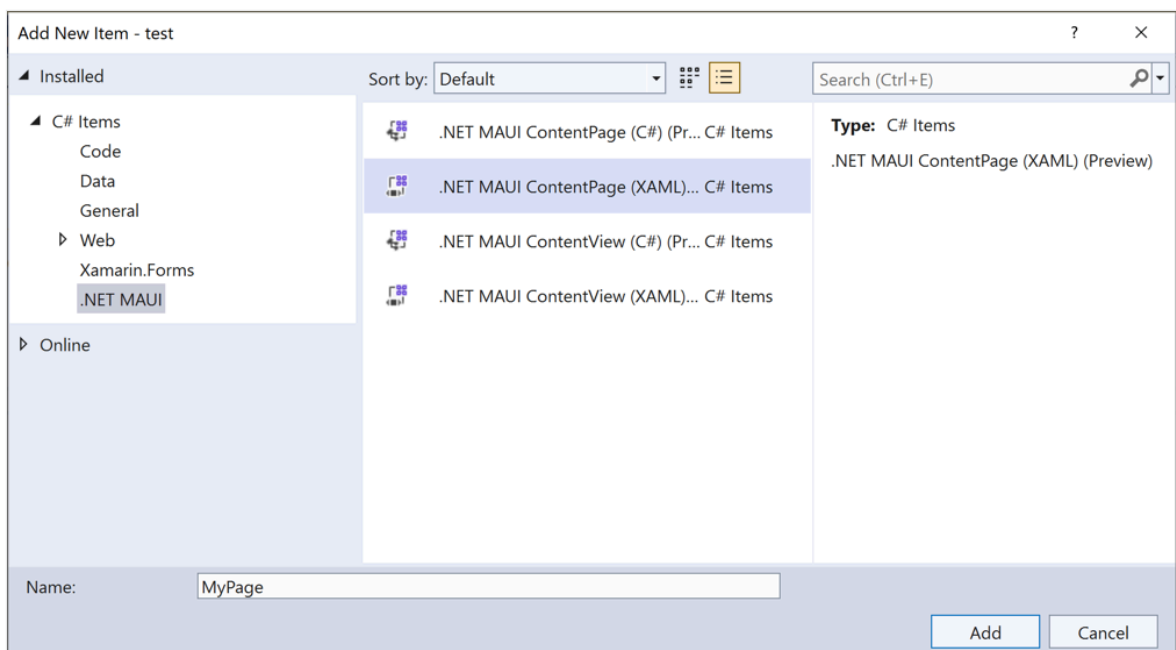
.NET MAUI apps typically contain multiple pages that derive from [ContentPage](#), and navigation between these pages can be performed. For more information about page navigation, see [NavigationPage](#).

A [ContentPage](#) can be templated with a control template. For more information, see [Control templates](#).

Create a ContentPage

To add a [ContentPage](#) to a .NET MAUI app:

1. In **Solution Explorer** right-click on your project or folder in your project, and select **New Item....**
2. In the **Add New Item** dialog, expand **Installed > C# Items**, select **.NET MAUI**, and select the **.NET MAUI ContentPage (XAML)** item template, enter a suitable page name, and click the **Add** button:



Visual Studio then creates a new [ContentPage](#)-derived page, which will be similar to the following example:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MyMauiApp.MyPage"
              Title="MyPage"
              BackgroundColor="White">
    <StackLayout>
        <Label Text="Welcome to .NET MAUI!"
               VerticalOptions="Center"
               HorizontalOptions="Center" />
        <!-- Other views go here -->
    </StackLayout>
</ContentPage>
```

The child of a [ContentPage](#) is typically a layout, such as [Grid](#) or [StackLayout](#), with the layout typically containing multiple views. However, the child of the [ContentPage](#) can be a view that displays a collection, such as [CollectionView](#).

ⓘ **Note**

The value of the `Title` property will be shown on the navigation bar, when the app performs navigation using a [NavigationPage](#). For more information, see [NavigationPage](#).

FlyoutPage

Article • 09/30/2024

 [Browse the sample](#)



ContentPage



FlyoutPage



NavigationPage



TabbedPage

The .NET Multi-platform App UI (.NET MAUI) [FlyoutPage](#) is a page that manages two related pages of information – a flyout page that presents items, and a detail page that presents details about items on the flyout page.

A [FlyoutPage](#) has two layout behaviors:

- In a popover layout, the detail page covers or partially covers the flyout page. Selecting an item on the flyout page will navigate to the corresponding detail page. Apps running on phones always use this layout behavior.
- In a split layout, the flyout page is displayed on the left and the detail page is on the right. Apps running on tablets or the desktop can use this layout behavior, with Windows using it by default.

For more information about layout behavior, see [Layout behavior](#).

[FlyoutPage](#) defines the following properties:

- `Detail`, of type [Page](#), defines the detail page displayed for the selected item in the flyout page.
- `Flyout`, of type [Page](#), defines the flyout page.
- `FlyoutLayoutBehavior`, of type [FlyoutLayoutBehavior](#), indicates the layout behavior of flyout and detail pages.
- `IsGestureEnabled`, of type `bool`, determines whether a swipe gesture will switch between flyout and detail pages. The default value of this property is `true`.
- `IsPresented`, of type `bool`, determines whether the flyout or detail page is displayed. The default value of this property is `false`, which displays the detail

page. It should be set to `true` to display the flyout page.

The `IsGestureEnabled`, `IsPresented`, and `FlyoutLayoutBehavior` properties are backed by `BindableProperty` objects, which means that they can be targets of data bindings, and styled.

`FlyoutPage` also defines an `IsPresentedChanged` event, that's raised when the `IsPresented` property changes value.

⚠ Warning

`FlyoutPage` is incompatible with .NET MAUI Shell apps, and an exception will be thrown if you attempt to use `FlyoutPage` in a Shell app. For more information about Shell apps, see [Shell](#).

Create a FlyoutPage

To create a flyout page, create a `FlyoutPage` object and set its `Flyout` and `Detail` properties. The `Flyout` property should be set to `ContentPage` object, and the `Detail` property should be set to a `TabbedPage`, `NavigationPage`, or `ContentPage` object. This will help to ensure a consistent user experience across all platforms.

📌 Important

A `FlyoutPage` is designed to be the root page of an app, and using it as a child page in other page types could result in unexpected and inconsistent behavior.

The following example shows a `FlyoutPage` that sets the `Flyout` and `Detail` properties:

XAML

```
<FlyoutPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:FlyoutPageNavigation"
             x:Class="FlyoutPageNavigation.MainPage">
    <FlyoutPage.Flyout>
        <local:FlyoutMenuPage x:Name="flyoutPage" />
    </FlyoutPage.Flyout>
    <FlyoutPage.Detail>
        <NavigationPage>
            <x:Arguments>
                <local:ContactsPage />
            </x:Arguments>
        </NavigationPage>
    </FlyoutPage.Detail>
</FlyoutPage>
```



```
        </NavigationPage>
    </FlyoutPage.Detail>
</FlyoutPage>
```

In this example, the `Flyout` property is set to a `ContentPage` object, and the `Detail` property is set to a `NavigationPage` containing a `ContentPage` object.

The following example shows the definition of the `FlyoutMenuPage` object, which is of type `ContentPage`:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:FlyoutPageNavigation"
             x:Class="FlyoutPageNavigation.FlyoutMenuPage"
             Padding="0,40,0,0"
             IconImageSource="hamburger.png"
             Title="Personal Organiser">
    <CollectionView x:Name="collectionView"
                   x:FieldModifier="public"
                   SelectionMode="Single">
        <CollectionView.ItemsSource>
            <x:Array Type="{x:Type local:FlyoutPageItem}">
                <local:FlyoutPageItem Title="Contacts"
                                       IconSource="contacts.png"
                                       TargetType="{x:Type
local:ContactsPage}" />
                <local:FlyoutPageItem Title="TodoList"
                                       IconSource="todo.png"
                                       TargetType="{x:Type
local:TodoListPage}" />
                <local:FlyoutPageItem Title="Reminders"
                                       IconSource="reminders.png"
                                       TargetType="{x:Type
local:ReminderPage}" />
            </x:Array>
        </CollectionView.ItemsSource>
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Grid Padding="5,10">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="30"/>
                        <ColumnDefinition Width="*" />
                    </Grid.ColumnDefinitions>
                    <Image Source="{Binding IconSource}" />
                    <Label Grid.Column="1"
                           Margin="20,0"
                           Text="{Binding Title}"
                           FontSize="20"
                           FontAttributes="Bold"
                           VerticalOptions="Center" />
                </Grid>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</ContentPage>
```

```

        </Grid>
    </DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</ContentPage>

```

In this example, the flyout page consists of a [CollectionView](#) that's populated with data by setting its `ItemsSource` property to an array of `FlyoutPageItem` objects. The following example shows the definition of the `FlyoutPageItem` class:

```

C#

public class FlyoutPageItem
{
    public string Title { get; set; }
    public string IconSource { get; set; }
    public Type TargetType { get; set; }
}

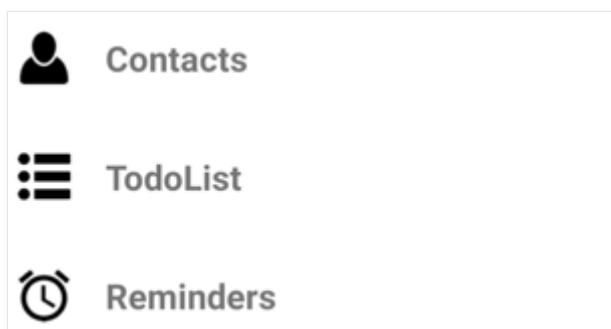
```

A [DataTemplate](#) is assigned to the `CollectionView.ItemTemplate` property, to display each `FlyoutPageItem`. The [DataTemplate](#) contains a [Grid](#) that consists of an [Image](#) and a [Label](#). The [Image](#) displays the `IconSource` property value, and the [Label](#) displays the `Title` property value, for each `FlyoutPageItem`. In addition, the flyout page has its `Title` and `IconImageSource` properties set. The icon will appear on the detail page, provided that the detail page has a title bar.

ⓘ Note

The `Flyout` page must have its `Title` property set, or an exception will occur.

The following screenshot shows the resulting flyout:



Create and display the detail page

The `FlyoutMenuPage` object contains a `CollectionView` that's referenced from the `MainPage` class. This allows the `MainPage` class to register a handler for the `SelectionChanged` event. This enables the `MainPage` object to set the `Detail` property to the page that represents the selected `CollectionView` item. The following example shows the event handler:

C#

```
public partial class MainPage : FlyoutPage
{
    public MainPage()
    {
        ...
        flyoutPage.collectionView.SelectionChanged += OnSelectionChanged;
    }

    void OnSelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        var item = e.CurrentSelection.FirstOrDefault() as FlyoutPageItem;
        if (item != null)
        {
            Detail = new
            NavigationPage((Page)Activator.CreateInstance(item.TargetType));
            if (!((IFlyoutPageController)this).ShouldShowSplitMode)
                IsPresented = false;
        }
    }
}
```

In this example, the `OnSelectionChanged` event handler retrieves the `CurrentSelection` from the `CollectionView` object and sets the detail page to an instance of the page type stored in the `TargetType` property of the `FlyoutPageItem`. The detail page is displayed by setting the `FlyoutPage.IsPresented` property to `false`, provided that the `FlyoutPage` isn't using a split layout. When the `FlyoutPage` is using a split layout, the flyout and detail pages are both displayed and so it's not necessary to set the `FlyoutPage.IsPresented` property.

Layout behavior

How the `FlyoutPage` displays the flyout and detail pages depends on the form factor of the device the app is running on, the orientation of the device, and the value of the `FlyoutLayoutBehavior` property. This property should be set to a value of the `FlyoutLayoutBehavior` enumeration, which defines the following members:

- `Default` – pages are displayed using the platform default.

- `Popover` – the detail page covers, or partially covers the flyout page.
- `Split` – the flyout page is displayed on the left and the detail page is on the right.
- `SplitOnLandscape` – a split screen is used when the device is in landscape orientation.
- `SplitOnPortrait` – a split screen is used when the device is in portrait orientation.

The following example shows how to set the `FlyoutLayoutBehavior` property on a `FlyoutPage`:

XAML

```
<FlyoutPage ...  
    FlyoutLayoutBehavior="Split">  
    ...  
</FlyoutPage>
```

Important

The value of the `FlyoutLayoutBehavior` property only affects apps running on tablets or the desktop. Apps running on phones always have the `Popover` behavior.

NavigationPage

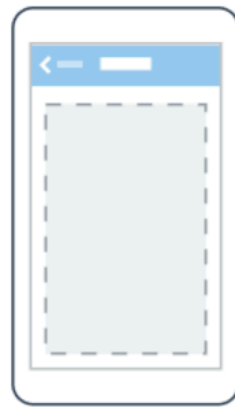
Article • 09/30/2024



ContentPage



FlyoutPage



NavigationPage



TabbedPage

The .NET Multi-platform App UI (.NET MAUI) [NavigationPage](#) provides a hierarchical navigation experience where you're able to navigate through pages, forwards and backwards, as desired. [NavigationPage](#) provides navigation as a last-in, first-out (LIFO) stack of [Page](#) objects.

[NavigationPage](#) defines the following properties:

- `BarBackground`, of type [Brush](#), specifies the background of the navigation bar as a [Brush](#).
- `BarBackgroundColor`, of type [Color](#), specifies the background color of the navigation bar.
- `BackButtonTitle`, of type `string`, represents the text to use for the back button. This is an attached property.
- `BarTextColor`, of type [Color](#), specifies the color of the text on the navigation bar.
- `CurrentPage`, of type [Page](#), represents the page that's on top of the navigation stack. This is a read-only property.
- `HasNavigationBar`, of type `bool`, represents whether a navigation bar is present on the [NavigationPage](#). The default value of this property is `true`. This is an attached property.
- `HasBackButton`, of type `bool`, represents whether the navigation bar includes a back button. The default value of this property is `true`. This is an attached property.
- `IconColor`, of type [Color](#), defines the background color of the icon in the navigation bar. This is an attached property.
- `RootPage`, of type [Page](#), represents the root page of the navigation stack. This is a read-only property.

- `TitleIconImageSource`, of type [ImageSource](#), defines the icon that represents the title on the navigation bar. This is an attached property.
- `TitleView`, of type [View](#), defines the view that can be displayed in the navigation bar. This is an attached property.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The [NavigationPage](#) class also defines three events:

- `Pushed` is raised when a page is pushed onto the navigation stack.
- `Popped` is raised when a page is popped from the navigation stack.
- `PoppedToRoot` is raised when the last non-root page is popped from the navigation stack.

All three events receive `NavigationEventArgs` objects that define a read-only [Page](#) property, which retrieves the page that was popped from the navigation stack, or the newly visible page on the stack.

Warning

[NavigationPage](#) is incompatible with .NET MAUI Shell apps, and an exception will be thrown if you attempt to use [NavigationPage](#) in a Shell app. For more information about Shell apps, see [Shell](#).

Perform modeless navigation

.NET MAUI supports modeless page navigation. A modeless page stays on screen and remains available until you navigate to another page.

A [NavigationPage](#) is typically used to navigate through a stack of [ContentPage](#) objects. When one page navigates to another, the new page is pushed on the stack and becomes the active page:



When the second page returns back to the first page, a page is popped from the stack, and the new topmost page then becomes active:



A [NavigationPage](#) consists of a navigation bar, with the active page being displayed below the navigation bar. The following diagram shows the main components of the navigation bar:



An optional icon can be displayed between the back button and the title.

Navigation methods are exposed by the `Navigation` property on any [Page](#) derived types. These methods provide the ability to push pages onto the navigation stack, to pop pages from the stack, and to manipulate the stack.

💡 Tip

It's recommended that a [NavigationPage](#) should only be populated with [ContentPage](#) objects.

Create the root page

An app that is structured around multiple pages always has a *root* page, which is the first page added to the navigation stack. This is accomplished by creating a [NavigationPage](#) object whose constructor argument is the root page of the app, and setting the resulting object as the root page of a [Window](#):

```
C#  
  
public partial class App : Application  
{  
    public App()  
    {  
        InitializeComponent();  
    }  
}
```

```
protected override Window CreateWindow(IActivationState?
activationState)
{
    return new Window(new NavigationPage(new MainPage()));
}
```

ⓘ Note

The `RootPage` property of a [NavigationPage](#) provides access to the first page in the navigation stack.

Push pages to the navigation stack

A page can be navigated to by calling the `PushAsync` method on the `Navigation` property of the current page:

C#

```
await Navigation.PushAsync(new DetailsPage());
```

In this example, the `DetailsPage` object is pushed onto the navigation stack, where it becomes the active page.

ⓘ Note

The `PushAsync` method has an override that includes a `bool` argument that specifies whether to display a page transition during navigation. The `PushAsync` method that lacks the `bool` argument enables the page transition by default.

Pop pages from the navigation stack

The active page can be popped from the navigation stack by pressing the *Back* button on a device, regardless of whether this is a physical button on the device or an on-screen button.

To programmatically return to the previous page, the `PopAsync` method should be called on the `Navigation` property of the current page:

C#


```
await Navigation.PopAsync();
```

In this example, the current page is removed from the navigation stack, with the new topmost page becoming the active page.

ⓘ Note

The `PopAsync` method has an override that includes a `bool` argument that specifies whether to display a page transition during navigation. The `PopAsync` method that lacks the `bool` argument enables the page transition by default.

In addition, the `Navigation` property of each page also exposes a `PopToRootAsync` method that pops all but the root page off the navigation stack, therefore making the app's root page the active page.

Manipulate the navigation stack

The `Navigation` property of a `Page` exposes a `NavigationStack` property from which the pages in the navigation stack can be obtained. While .NET MAUI maintains access to the navigation stack, the `Navigation` property provides the `InsertPageBefore` and `RemovePage` methods for manipulating the stack by inserting pages or removing them.

The `InsertPageBefore` method inserts a specified page in the navigation stack before an existing specified page, as shown in the following diagram:



The `RemovePage` method removes the specified page from the navigation stack, as shown in the following diagram:



Together, these methods enable a custom navigation experience, such as replacing a login page with a new page following a successful login.

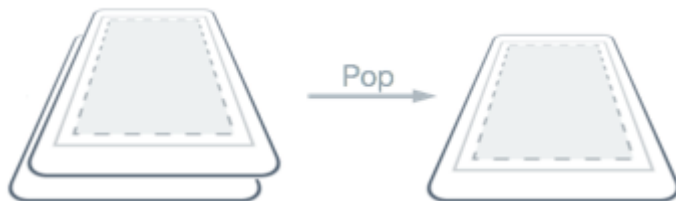
Perform modal navigation

.NET MAUI supports modal page navigation. A modal page encourages users to complete a self-contained task that cannot be navigated away from until the task is completed or cancelled.

A modal page can be any of the page types supported by .NET MAUI. To display a page modally, the app should push it onto the modal stack, where it will become the active page:



To return to the previous page the app should pop the current page from the modal stack, and the new topmost page becomes the active page:



Modal navigation methods are exposed by the `Navigation` property on any [Page](#) derived types. These methods provide the ability to push pages onto the modal stack, and pop pages from the modal stack. The `Navigation` property also exposes a `ModalStack` property from which pages in the modal stack can be obtained. However, there is no concept of performing modal stack manipulation, or popping to the root page in modal navigation. This is because these operations are not universally supported on the underlying platforms.

ⓘ Note

A [NavigationPage](#) object is not required for performing modal page navigation.

Push pages to the modal stack

A page can be modally navigated to by calling the `PushModalAsync` method on the `Navigation` property of the current page:

C#

```
await Navigation.PushModalAsync(new DetailsPage());
```

In this example, the `DetailsPage` object is pushed onto the modal stack, where it becomes the active page.

ⓘ Note

The `PushModalAsync` method has an override that includes a `bool` argument that specifies whether to display a page transition during navigation. The `PushModalAsync` method that lacks the `bool` argument enables the page transition by default.

Pop pages from the modal stack

The active page can be popped from the modal stack by pressing the *Back* button on a device, regardless of whether this is a physical button on the device or an on-screen button.

To programmatically return to the original page, the `PopModalAsync` method should be called on the `Navigation` property of the current page:

C#

```
await Navigation.PopModalAsync();
```

In this example, the current page is removed from the modal stack, with the new topmost page becoming the active page.

ⓘ Note

The `PopModalAsync` method has an override that includes a `bool` argument that specifies whether to display a page transition during navigation. The `PopModalAsync` method that lacks the `bool` argument enables the page transition by default.

Disable the back button

On Android, you can always return to the previous page by pressing the standard *Back* button on the device. If the modal page requires a self-contained task to be completed before leaving the page, the app must disable the *Back* button. This can be accomplished by overriding the `Page.OnBackButtonPressed` method on the modal page.

Pass data during navigation

Sometimes it's necessary for a page to pass data to another page during navigation. Two standard techniques for accomplishing this are passing data through a page constructor, and by setting the new page's `BindingContext` to the data.

Pass data through a page constructor

The simplest technique for passing data to another page during navigation is through a page constructor argument:

C#

```
Contact contact = new Contact
{
    Name = "Jane Doe",
    Age = 30,
    Occupation = "Developer",
    Country = "USA"
};
...
await Navigation.PushModalAsync(new DetailsPage(contact));
```

In this example, a `Contact` object is passed as a constructor argument to `DetailPage`. The `contact` object can then be displayed by `DetailsPage`.

Pass data through a BindingContext

An alternative approach for passing data to another page during navigation is by setting the new page's `BindingContext` to the data:

C#

```
Contact contact = new Contact
{
    Name = "Jane Doe",
    Age = 30,
```

```

        Occupation = "Developer",
        Country = "USA"
    };

    await Navigation.PushAsync(new DetailsPage
    {
        BindingContext = contact
    });

```

The advantage of passing navigation data via a page's `BindingContext` is that the new page can use data binding to display the data:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="MyMauiApp.DetailsPage"
             Title="Details">
    <StackLayout>
        <Label Text="{Binding Name}" />
        <Label Text="{Binding Occupation}" />
    </StackLayout>
</ContentPage>

```

For more information about data binding, see [Data binding](#).

Display views in the navigation bar

Any .NET MAUI [View](#) can be displayed in the navigation bar of a [NavigationPage](#). This is accomplished by setting the `NavigationPage.TitleView` attached property to a [View](#). This attached property can be set on any [Page](#), and when the [Page](#) is pushed onto a [NavigationPage](#), the [NavigationPage](#) will respect the value of the property.

The following example shows how to set the `NavigationPage.TitleView` attached property:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             x:Class="NavigationPageTitleView.TitleViewPage">
    <NavigationPage.TitleView>
        <Slider HeightRequest="44"
                WidthRequest="300" />
    </NavigationPage.TitleView>
    ...
</ContentPage>

```

The equivalent C# code is:

C#

```
Slider titleView = new Slider { HeightRequest = 44, WidthRequest = 300 };  
NavigationPage.SetTitleView(this, titleView);
```

In this example, a [Slider](#) is displayed in the navigation bar of the [NavigationPage](#), to control zooming.

Important

Many views won't appear in the navigation bar unless the size of the view is specified with the [WidthRequest](#) and [HeightRequest](#) properties.

Because the [Layout](#) class derives from the [View](#) class, the `TitleView` attached property can be set to display a layout class that contains multiple views. However, this can result in clipping if the view displayed in the navigation bar is larger than the default size of the navigation bar. However, on Android, the height of the navigation bar can be changed by setting the `NavigationPage.BarHeight` bindable property to a `double` representing the new height.

Alternatively, an extended navigation bar can be suggested by placing some of the content in the navigation bar, and some in a view at the top of the page content that you color match to the navigation bar. In addition, on iOS the separator line and shadow that's at the bottom of the navigation bar can be removed by setting the `NavigationPage.HideNavigationBarSeparator` bindable property to `true`.

Tip

The `BackButtonTitle`, `Title`, `TitleIconImageSource`, and `TitleView` properties can all define values that occupy space on the navigation bar. While the navigation bar size varies by platform and screen size, setting all of these properties will result in conflicts due to the limited space available. Instead of attempting to use a combination of these properties, you may find that you can better achieve your desired navigation bar design by only setting the `TitleView` property.

TabbedPage

Article • 09/30/2024



ContentPage



FlyoutPage



NavigationPage



TabbedPage

The .NET Multi-platform App UI (.NET MAUI) [TabbedPage](#) maintains a collection of children of type [Page](#), only one of which is fully visible at a time. Each child is identified by a series of tabs across the top or bottom of the page. Typically, each child will be a [ContentPage](#) and when its tab is selected the page content is displayed.

[TabbedPage](#) defines the following properties:

- `BarBackground`, of type [Brush](#), defines the background of the tab bar.
- `BarBackgroundColor`, of type [Color](#), defines the background color of the tab bar.
- `BarTextColor`, of type [Color](#), represents the color of the text on the tab bar.
- `SelectedTabColor`, of type [Color](#), indicates the color of a tab when it's selected.
- `UnselectedTabColor`, of type [Color](#), represents the color of a tab when it's unselected.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

The title of a tab is defined by the [Page.Title](#) property of the child page, and the tab icon is defined by the [Page.IconImageSource](#) property of the child page.

In a [TabbedPage](#), each [Page](#) object is created when the [TabbedPage](#) is constructed. This can lead to a poor user experience, particularly if the [TabbedPage](#) is the root page of your app. However, .NET MAUI Shell enables pages accessed through a tab bar to be created on demand, in response to navigation. For more information about Shell apps, see [Shell](#).

 **Warning**

[TabbedPage](#) is incompatible with .NET MAUI Shell apps, and an exception will be thrown if you attempt to use [TabbedPage](#) in a Shell app.

Create a TabbedPage

Two approaches can be used to create a [TabbedPage](#):

- Populate the [TabbedPage](#) with a collection of child [Page](#) objects, such as a collection of [ContentPage](#) objects. For more information, see [Populate a TabbedPage with a Page collection](#).
- Assign a collection to the `ItemsSource` property and assign a [DataTemplate](#) to the `ItemTemplate` property to return pages for objects in the collection. For more information, see [Populate a TabbedPage with a DataTemplate](#).

Important

A [TabbedPage](#) should only be populated with [NavigationPage](#) and [ContentPage](#) objects.

Regardless of the approach taken, the location of the tab bar in a [TabbedPage](#) is platform-dependent:

- On iOS, the list of tabs appears at the bottom of the screen, and the page content is above. Each tab consists of a title and an icon. In portrait orientation, tab bar icons appear above tab titles. In landscape orientation, icons and titles appear side by side. In addition, a regular or compact tab bar may be displayed, depending on the device and orientation. If there are more than five tabs, a **More** tab will appear, which can be used to access the additional tabs.
- On Android, the list of tabs appears at the top of the screen, and the page content is below. Each tab consists of a title and an icon. However, the tabs can be moved to the bottom of the screen with a platform-specific. If there are more than five tabs, and the tab list is at the bottom of the screen, a *More* tab will appear that can be used to access the additional tabs. For information about moving the tabs to the bottom of the screen, see [TabbedPage toolbar placement on Android](#).
- On Windows, the list of tabs appears at the top of the screen, and the page content is below. Each tab consists of a title.

Populate a TabbedPage with a Page collection

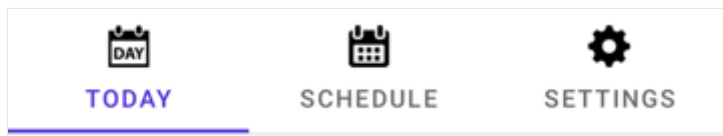
A [TabbedPage](#) can be populated with a collection of child [Page](#) objects, which will typically be [ContentPage](#) objects. This is achieved by adding [ContentPage](#) objects as children of the [TabbedPage](#):

XAML

```
<TabbedPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:TabbedPageWithNavigationPage"
             x:Class="TabbedPageWithNavigationPage.MainPage">
    <local:TodayPage />
    <local:SchedulePage />
    <local:SettingsPage />
</TabbedPage>
```

[Page](#) objects that are added as child elements of [TabbedPage](#) are added to the `Children` collection. The `Children` property of the `MultiPage<T>` class, from which [TabbedPage](#) derives, is the [ContentProperty](#) of `MultiPage<T>`. Therefore, in XAML it's not necessary to explicitly assign the [Page](#) objects to the `Children` property.

The following screenshot shows the appearance of the resulting tab bar on the [TabbedPage](#):



The page content for a tab appears when the tab is selected.

Populate a [TabbedPage](#) with a [DataTemplate](#)

[TabbedPage](#) inherits `ItemsSource`, `ItemTemplate`, and `SelectedItem` bindable properties from the `MultiPage<T>` class. These properties enable you to generate [TabbedPage](#) children dynamically, by setting the `ItemsSource` property to an `IEnumerable` collection of objects with public properties suitable for data bindings, and by setting the `ItemTemplate` property to a [DataTemplate](#) with a page type as the root element.

The following example shows generating [TabbedPage](#) children dynamically:

XAML

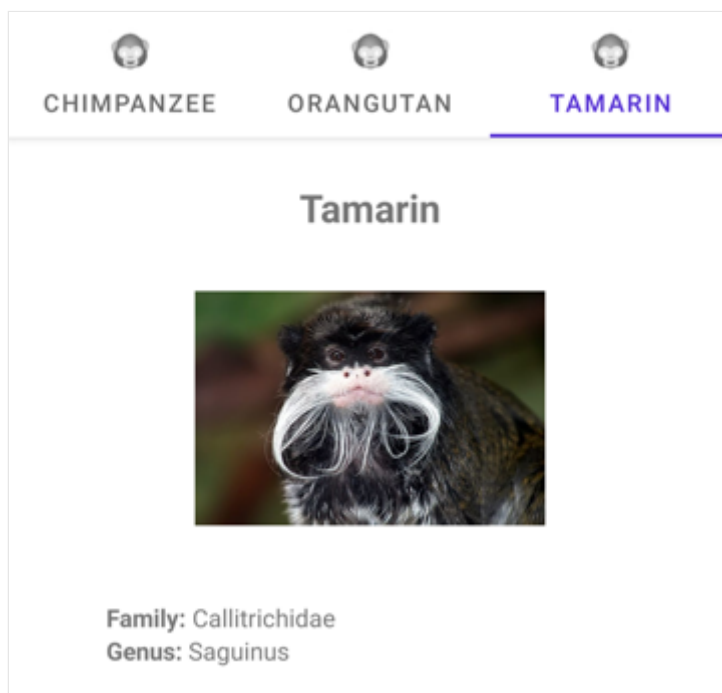
```
<TabbedPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:TabbedPageDemo"
             x:Class="TabbedPageDemo.MainPage">
```

```

        ItemsSource="{x:Static local:MonkeyDataModel.All}">
    <TabbedPage.ItemTemplate>
        <DataTemplate>
            <ContentPage Title="{Binding Name}"
                IconImageSource="monkeyicon.png">
                <StackLayout Padding="5, 25">
                    <Label Text="{Binding Name}"
                        FontAttributes="Bold"
                        FontSize="18"
                        HorizontalOptions="Center" />
                    <Image Source="{Binding PhotoUrl}"
                        HorizontalOptions="Center"
                        WidthRequest="200"
                        HeightRequest="200" />
                    <StackLayout Padding="50, 10">
                        <StackLayout Orientation="Horizontal">
                            <Label Text="Family: "
                                FontAttributes="Bold" />
                            <Label Text="{Binding Family}" />
                        </StackLayout>
                        ...
                    </StackLayout>
                </StackLayout>
            </ContentPage>
        </DataTemplate>
    </TabbedPage.ItemTemplate>
</TabbedPage>

```

In this example, each tab consists of a [ContentPage](#) object that uses [Image](#) and [Label](#) objects to display data for the tab:



Navigate within a tab

Navigation can be performed within a tab, provided that the [ContentPage](#) object is wrapped in a [NavigationPage](#) object:

XAML

```
<TabbedPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:TabbedPageWithNavigationPage"
             x:Class="TabbedPageWithNavigationPage.MainPage">
    <local:TodayPage />
    <NavigationPage Title="Schedule"
                    IconImageSource="schedule.png">
        <x:Arguments>
            <local:SchedulePage />
        </x:Arguments>
    </NavigationPage>
</TabbedPage>
```

In this example, the [TabbedPage](#) is populated with two [Page](#) objects. The first child is a [ContentPage](#) object, and the second child is a [NavigationPage](#) object containing a [ContentPage](#) object.

When a [ContentPage](#) is wrapped in a [NavigationPage](#), forwards page navigation can be performed by calling the `PushAsync` method on the `Navigation` property of the [ContentPage](#) object:

C#

```
await Navigation.PushAsync(new UpcomingAppointmentsPage());
```

For more information about performing navigation using the [NavigationPage](#) class, see [NavigationPage](#).

Warning

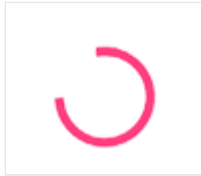
While a [NavigationPage](#) can be placed in a [TabbedPage](#), it's not recommended to place a [TabbedPage](#) into a [NavigationPage](#).

ActivityIndicator

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [ActivityIndicator](#) displays an animation to show that the application is engaged in a lengthy activity. Unlike [ProgressBar](#), [ActivityIndicator](#) gives no indication of progress.

The appearance of an [ActivityIndicator](#) is platform-dependent, and the following screenshot shows an [ActivityIndicator](#) on Android:



[ActivityIndicator](#) defines the following properties:

- `Color` is a [Color](#) value that defines the color of the [ActivityIndicator](#).
- `IsRunning` is a `bool` value that indicates whether the [ActivityIndicator](#) should be visible and animating, or hidden. The default value of this property is `false`, which indicates that the [ActivityIndicator](#) isn't visible.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

Create an ActivityIndicator

To indicate a lengthy activity, create an [ActivityIndicator](#) object and sets its properties to define its appearance.

The following XAML example shows how to display an [ActivityIndicator](#):

XAML

```
<ActivityIndicator IsRunning="true" />
```

The equivalent C# code is:

C#

```
ActivityIndicator activityIndicator = new ActivityIndicator { IsRunning = true };
```

The following XAML example shows how to change the color of an [ActivityIndicator](#):

XAML

```
<ActivityIndicator IsRunning="true"
    Color="Orange" />
```

The equivalent C# code is:

C#

```
ActivityIndicator activityIndicator = new ActivityIndicator
{
    IsRunning = true,
    Color = Colors.Orange
};
```

Host a Blazor web app in a .NET MAUI app using BlazorWebView

Article • 11/13/2024

The .NET Multi-platform App UI (.NET MAUI) [BlazorWebView](#) is a control that enables you to host a Blazor web app in your .NET MAUI app. These apps, known as Blazor Hybrid apps, enable a Blazor web app to be integrated with platform features and UI controls. The [BlazorWebView](#) control can be added to any page of a .NET MAUI app, and pointed to the root of the Blazor app. The [Razor components](#) run natively in the .NET process and render web UI to an embedded web view control. In .NET MAUI, Blazor Hybrid apps can run on all the platforms supported by .NET MAUI.

[BlazorWebView](#) defines the following properties:

- [HostPage](#), of type `string?`, which defines the root page of the Blazor web app.
- [RootComponents](#), of type `RootComponentsCollection`, which specifies the collection of root components that can be added to the control.
- [StartPath](#), of type `string`, which defines the path for initial navigation within the Blazor navigation context when the Blazor component is finished loading.

The [RootComponent](#) class defines the following properties:

- [Selector](#), of type `string?`, which defines the CSS selector string that specifies where in the document the component should be placed.
- [ComponentType](#), of type `Type?`, which defines the type of the root component.
- [Parameters](#), of type `IDictionary<string, object?>?`, which represents an optional dictionary of parameters to pass to the root component.

In addition, [BlazorWebView](#) defines the following events:

- [BlazorWebViewInitializing](#), with an accompanying `BlazorWebViewInitializingEventArgs` object, which is raised before the [BlazorWebView](#) is initialized. This event enables customization of the [BlazorWebView](#) configuration.
- [BlazorWebViewInitialized](#), with an accompanying `BlazorWebViewInitializedEventArgs` object, which is raised after the [BlazorWebView](#) is initialized but before any component has been rendered. This event enables retrieval of the platform-specific web view instance.
- [UrlLoading](#), with an accompanying `UrlLoadingEventArgs` object, is raised when a hyperlink is clicked within a [BlazorWebView](#). This event enables customization of

whether a hyperlink is opened in the [BlazorWebView](#), in an external app, or whether the URL loading attempt is cancelled.

Existing [Razor components](#) can be used in a .NET MAUI Blazor app by moving the code into the app, or by referencing an existing class library or package that contains the component. For more information, see [Reuse Razor components in ASP.NET Core Blazor Hybrid](#).

Browser developer tools can be used to inspect .NET MAUI Blazor apps. For more information, see [Use browser developer tools with ASP.NET Core Blazor Hybrid](#).

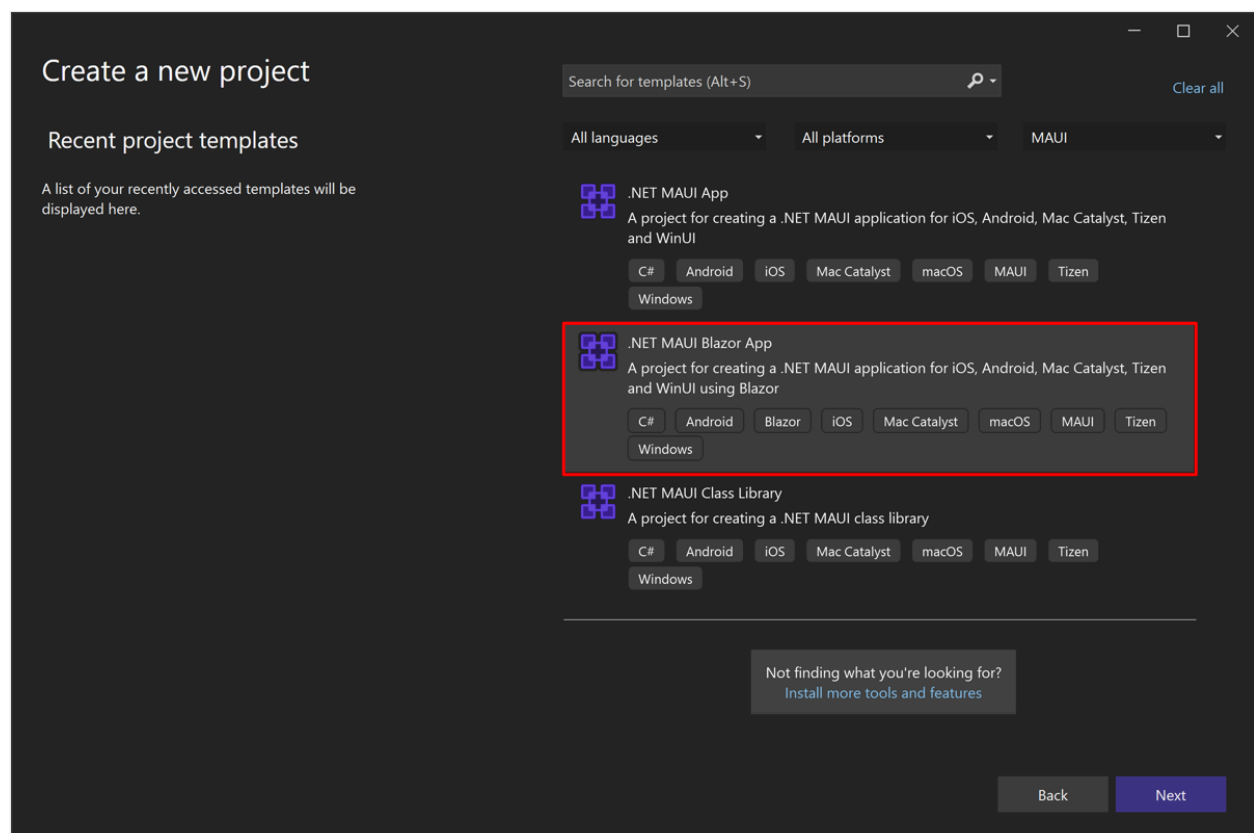
ⓘ Note

While Visual Studio installs all the required tooling to develop .NET MAUI Blazor apps, end users of .NET MAUI Blazor apps on Windows must install the [WebView2](#) runtime.

For more information about Blazor Hybrid apps, see [ASP.NET Core Blazor Hybrid](#).

Create a .NET MAUI Blazor app

A .NET MAUI Blazor app can be created in Visual Studio by the **.NET MAUI Blazor app** template:



This project template creates a multi-targeted .NET MAUI Blazor app that can be deployed to Android, iOS, macOS, and Windows. For step-by-step instructions on creating a .NET MAUI Blazor app, see [Build a .NET MAUI Blazor app](#).

The [BlazorWebView](#) created by the project template is defined in *MainPage.xaml*, and points to the root of the Blazor app:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:BlazorWebViewDemo"
             x:Class="BlazorWebViewDemo.MainPage"
             BackgroundColor="{DynamicResource PageBackgroundColor}">

    <BlazorWebView HostPage="wwwroot/index.html">
        <BlazorWebView.RootComponents>
            <RootComponent Selector="#app" ComponentType="{x:Type
local:Main}" />
        </BlazorWebView.RootComponents>
    </BlazorWebView>

</ContentPage>
```

The root [Razor component](#) for the app is in *Main.razor*, which Razor compiles into a type named `Main` in the application's root namespace. The rest of the [Razor components](#) are in the *Pages* and *Shared* project folders, and are identical to the components used in the default Blazor web template. Static web assets for the app are in the *wwwroot* folder.

Add a BlazorWebView to an existing app

The process to add a [BlazorWebView](#) to an existing .NET MAUI app is as follows:

1. Add the Razor SDK, `Microsoft.NET.Sdk.Razor` to your project by editing its first line of the CSProj project file:

XML

```
<Project Sdk="Microsoft.NET.Sdk.Razor">
```

The Razor SDK is required to build and package projects containing Razor files for Blazor projects.

2. Add the root [Razor component](#) for the app to the project.
3. Add your [Razor components](#) to project folders named *Pages* and *Shared*.

4. Add your static web assets to a project folder named *wwwroot*.
5. Add any optional *_Imports.razor* files to your project.
6. Add a [BlazorWebView](#) to a page in your .NET MAUI app, and point it to the root of the Blazor app:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:MyBlazorApp"
             x:Class="MyBlazorApp.MainPage">

    <BlazorWebView HostPage="wwwroot/index.html">
        <BlazorWebView.RootComponents>
            <RootComponent Selector="#app" ComponentType="{x:Type
local:Main}" />
        </BlazorWebView.RootComponents>
    </BlazorWebView>

</ContentPage>
```

7. Modify the `CreateMauiApp` method of your `MauiProgram` class to register the [BlazorWebView](#) control for use in your app. To do this, on the `IServiceCollection` object, call the `AddMauiBlazorWebView` method to add component web view services to the services collection:

C#

```
public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf",
"OpenSansRegular");
            });

        builder.Services.AddMauiBlazorWebView();
#if DEBUG
        builder.Services.AddBlazorWebViewDeveloperTools();
#endif
        // Register any app services on the IServiceCollection object
        // e.g. builder.Services.AddSingleton<WeatherForecastService>
        ();
    }
}
```

```
        return builder.Build();
    }
}
```

This code also enables developer tools on the underlying WebView controls, when the app is running in debug configuration.

Access scoped services from native UI

[BlazorWebView](#) has a [TryDispatchAsync](#) method that can call a specified `Action<ServiceProvider>` asynchronously and pass in the scoped services available in Razor components. This enables code from the native UI to access scoped services such as [NavigationManager](#):

C#

```
private async void OnMyMauiButtonClicked(object sender, EventArgs e)
{
    var wasDispatchCalled = await blazorWebView.TryDispatchAsync(sp =>
    {
        var navMan = sp.GetRequiredService<NavigationManager>();
        navMan.CallSomeNavigationApi(...);
    });

    if (!wasDispatchCalled)
    {
        // Consider what to do if it the dispatch fails - that's up to your
        app to decide.
    }
}
```

Diagnosing issues

[BlazorWebView](#) has built-in logging that can help you diagnose issues in your Blazor Hybrid app. There are two steps to enable this logging:

1. Enable [BlazorWebView](#) and related components to log diagnostic information.
2. Configure a logger to write the log output to where you can view it.

For more information about logging, see [Logging in C# and .NET](#).

Enable BlazorWebView logging

All logging configuration can be performed as part of service registration in the dependency injection system. To enable maximum logging for [BlazorWebView](#) and related components under the [Microsoft.AspNetCore.Components.WebView](#) namespace, add the following code to where your app's services are registered:

```
C#

services.AddLogging(logging =>
{
    logging.AddFilter("Microsoft.AspNetCore.Components.WebView",
LogLevel.Trace);
});
```

Alternatively, to enable maximum logging for every component that uses [Microsoft.Extensions.Logging](#), you could use the following code:

```
C#

services.AddLogging(logging =>
{
    logging.SetMinimumLevel(LogLevel.Trace);
});
```

Configure logging output and viewing the output

After configuring components to write log information you need to configure where the loggers should write the logs to, and then view the log output.

The **Debug** logging providers write the output using `Debug` statements, and the output can be viewed from Visual Studio.

To configure the **Debug** logging provider, first add a reference in your project to the [Microsoft.Extensions.Logging.Debug](#) [NuGet package](#). Then, register the provider inside the call to [AddLogging](#) that you added in the previous step by calling the [AddDebug](#) extension method:

```
C#

services.AddLogging(logging =>
{
    logging.AddFilter("Microsoft.AspNetCore.Components.WebView",
LogLevel.Trace);
    logging.AddDebug();
});
```

When you run the app from Visual Studio (with debugging enabled), you can view the debug output in Visual Studio's **Output** window.

Play inline video on iOS

To play inline video in a Blazor hybrid app on iOS, in a [BlazorWebView](#), you should:

- Set the [UrlLoadingStrategy](#) property to `OpenInWebView`. This can be accomplished in the event handler for the [UrlLoading](#) event:

```
C#  
  
private void BlazorUrlLoading(object? sender, UrlLoadingEventArgs e)  
{  
    #if IOS  
        e.UrlLoadingStrategy = UrlLoadingStrategy.OpenInWebView;  
    #endif  
}
```

- Ensure that the `AllowsInlineMediaPlayback` property in a `Configuration` object is set to `true`. This can be accomplished in the event handler for the [BlazorWebViewInitializing](#) event:

```
C#  
  
private void BlazorWebViewInitializing(object? sender,  
    BlazorWebViewInitializingEventArgs e)  
{  
    #if IOS  
        e.Configuration.AllowsInlineMediaPlayback = true;  
    #endif  
}
```

Disposal deadlocks on Android

By default, [BlazorWebView](#) fires and forgets the async disposal of the underlying `WebViewManager`. This reduces the potential for disposal deadlocks to occur on Android.

Warning

This fire-and-forget default behavior means that disposal can return before all objects are disposed, which can cause behavioral changes in your app. The items

that are disposed are partially Blazor's own internal types, but also app-defined types such as scoped services used within the [BlazorWebView](#) portion of your app.

To opt out of this behavior, you should configure your app to block on dispose via an [AppContext](#) switch in the `CreateMauiApp` method in your `MauiProgram` class:

C#

```
AppContext.SetSwitch("BlazorWebView.AndroidFireAndForgetAsync", false);
```

If your app is configured to block on dispose via this switch, [BlazorWebView](#) performs async-over-sync disposal, which means that it blocks the thread until the async disposal is complete. However, this can cause deadlocks if the disposal needs to run code on the same thread (because the thread is blocked while waiting).

Host content using the legacy behavior on iOS and Mac Catalyst

On iOS and Mac Catalyst 18, the default behavior for hosting content in a [BlazorWebView](#) has changed to `localhost`. The internal `0.0.0.1` address used to host content no longer works and results in the [BlazorWebView](#) not loading any content and rendering as an empty rectangle.

To opt into using the `0.0.0.1` address, add the following code to the `CreateMauiApp` method in `MauiProgram.cs`:

C#

```
// Set this switch to use the LEGACY behavior of always using 0.0.0.1 to  
// host BlazorWebView  
AppContext.SetSwitch("BlazorWebView.AppHostAddressAlways0000", true);
```

Border

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [Border](#) is a container control that draws a border, background, or both, around another control. A [Border](#) can only contain one child object. If you want to put a border around multiple objects, wrap them in a container object such as a layout. For more information about layouts, see [Layouts](#).

[Border](#) defines the following properties:

- `Content`, of type `IView`, represents the content to display in the border. This property is the [ContentProperty](#) of the [Border](#) class, and therefore does not need to be explicitly set from XAML.
- `Padding`, of type `Thickness`, represents the distance between the border and its child element.
- `StrokeShape`, of type `IShape`, describes the shape of the border. This property has a type converter applied to it that can convert a string to its equivalent `IShape`. Its default value is [Rectangle](#). Therefore, a [Border](#) will be rectangular by default.
- `Stroke`, of type [Brush](#), indicates the brush used to paint the border.
- `StrokeThickness`, of type `double`, indicates the width of the border. The default value of this property is 1.0.
- `StrokeDashArray`, of type `DoubleCollection`, which represents a collection of `double` values that indicate the pattern of dashes and gaps that make up the border.
- `StrokeDashOffset`, of type `double`, specifies the distance within the dash pattern where a dash begins. The default value of this property is 0.0.
- `StrokeLineCap`, of type `PenLineCap`, describes the shape at the start and end of its line. The default value of this property is `PenLineCap.Flat`.
- `StrokeLineJoin`, of type `PenLineJoin`, specifies the type of join that is used at the vertices of the stroke shape. The default value of this property is `PenLineJoin.Miter`.
- `StrokeMiterLimit`, of type `double`, specifies the limit on the ratio of the miter length to half the stroke thickness. The default value of this property is 10.0.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

Important

When creating a border using a shape, such as a [Rectangle](#) or [Polygon](#), only closed shapes should be used. Therefore, open shapes such as [Line](#) are unsupported.

For more information about the properties that control the shape and stroke of the border, see [Shapes](#).

Create a Border

To draw a border, create a [Border](#) object and set its properties to define its appearance. Then, set its child to the control to which the border should be added.

The following XAML example shows how to draw a border around a [Label](#):

XAML

```
<Border Stroke="#C49B33"
        StrokeThickness="4"
        StrokeShape="RoundRectangle 40,0,0,40"
        Background="#2B0B98"
        Padding="16,8"
        HorizontalOptions="Center">
    <Label Text=".NET MAUI"
          TextColor="White"
          FontSize="18"
          FontAttributes="Bold" />
</Border>
```

Alternatively, the `StrokeShape` property value can be specified using property tag syntax:

XAML

```
<Border Stroke="#C49B33"
        StrokeThickness="4"
        Background="#2B0B98"
        Padding="16,8"
        HorizontalOptions="Center">
    <Border.StrokeShape>
        <RoundRectangle CornerRadius="40,0,0,40" />
    </Border.StrokeShape>
    <Label Text=".NET MAUI"
          TextColor="White"
          FontSize="18"
          FontAttributes="Bold" />
</Border>
```

The equivalent C# code is:

C#

```
using Microsoft.Maui.Controls.Shapes;
using GradientStop = Microsoft.Maui.Controls.GradientStop;
...

Border border = new Border
{
    Stroke = Color.FromArgb("#C49B33"),
    Background = Color.FromArgb("#2B0B98"),
    StrokeThickness = 4,
    Padding = new Thickness(16, 8),
    HorizontalOptions = LayoutOptions.Center,
    StrokeShape = new RoundedRectangle
    {
        CornerRadius = new CornerRadius(40, 0, 0, 40)
    },
    Content = new Label
    {
        Text = ".NET MAUI",
        TextColor = Colors.White,
        FontSize = 18,
        FontAttributes = FontAttributes.Bold
    }
};
```

In this example, a border with rounded top-left and bottom-right corners is drawn around a [Label](#). The border shape is defined as a [RoundedRectangle](#) object, whose `CornerRadius` property is set to a `Thickness` value that enables independent control of each corner of the rectangle:



Because the `Stroke` property is of type [Brush](#), borders can also be drawn using gradients:

XAML

```
<Border StrokeThickness="4"
        StrokeShape="RoundRectangle 40,0,0,40"
        Background="#2B0B98"
        Padding="16,8"
        HorizontalOptions="Center">
    <Border.Stroke>
        <LinearGradientBrush EndPoint="0,1">
            <GradientStop Color="Orange"
                Offset="0.1" />
        </LinearGradientBrush>
    </Border.Stroke>
</Border>
```



```

        <GradientStop Color="Brown"
                    Offset="1.0" />
    </LinearGradientBrush>
</Border.Stroke>
<Label Text=".NET MAUI"
        TextColor="White"
        FontSize="18"
        FontAttributes="Bold" />
</Border>

```

The equivalent C# code is:

```

C#

using Microsoft.Maui.Controls.Shapes;
using GradientStop = Microsoft.Maui.Controls.GradientStop;
...

Border gradientBorder = new Border
{
    StrokeThickness = 4,
    Background = Color.FromArgb("#2B0B98"),
    Padding = new Thickness(16, 8),
    HorizontalOptions = LayoutOptions.Center,
    StrokeShape = new RoundRectangle
    {
        CornerRadius = new CornerRadius(40, 0, 0, 40)
    },
    Stroke = new LinearGradientBrush
    {
        EndPoint = new Point(0, 1),
        GradientStops = new GradientStopCollection
        {
            new GradientStop { Color = Colors.Orange, Offset = 0.1f },
            new GradientStop { Color = Colors.Brown, Offset = 1.0f }
        },
    },
    Content = new Label
    {
        Text = ".NET MAUI",
        TextColor = Colors.White,
        FontSize = 18,
        FontAttributes = FontAttributes.Bold
    }
};

```

In this example, a border that uses a linear gradient is drawn around a [Label](#):



Define the border shape with a string

In XAML, the value of the `StrokeShape` property can be defined using property-tag syntax, or as a `string`. Valid `string` values for the `StrokeShape` property are:

- `Ellipse`
- `Line`, followed by one or two x- and y-coordinate pairs. For example, `Line 10 20` draws a line from (10,20) to (0,0), and `Line 10 20, 100 120` draws a line from (10,20) to (100,120).
- `Path`, followed by path markup syntax data. For example, `Path M 10,100 L 100,100 100,50Z` will draw a triangular border. For more information about path markup syntax, see [Path markup syntax](#).
- `Polygon`, followed by a collection of x- and y-coordinate pairs. For example, `Polygon 40 10, 70 80, 10 50`.
- `Polyline`, followed by a collection x- and y-coordinate pairs. For example, `Polyline 0,0 10,30 15,0 18,60 23,30 35,30 40,0 43,60 48,30 100,30`.
- `Rectangle`
- `RoundRectangle`, optionally followed by a corner radius. For example, `RoundRectangle 40` Or `RoundRectangle 40,0,0,40`.

❗ Important

While `Line` is a valid `string` value for the `StrokeShape` property, its use is not supported.

`string`-based x- and y-coordinate pairs can be delimited by a single comma and/or one or more spaces. For example, "40,10 70,80" and "40 10, 70 80" are both valid. Coordinate pairs will be converted to `Point` objects that define `X` and `Y` properties, of type `double`.

BoxView

Article • 08/30/2024

The .NET Multi-platform App UI (.NET MAUI) [BoxView](#) draws a simple rectangle or square, of a specified width, height, and color.

[BoxView](#) defines the following properties:

- `Color`, of type [Color](#), which defines the color of the [BoxView](#).
- `CornerRadius`, of type `CornerRadius`, which defines the corner radius of the [BoxView](#). This property can be set to a single `double` uniform corner radius value, or a `CornerRadius` structure defined by four `double` values that are applied to the top left, top right, bottom left, and bottom right of the [BoxView](#).

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

ⓘ Note

Although [BoxView](#) can mimic simple graphics, a better alternative is to use .NET MAUI Shapes or [.NET MAUI Graphics](#).

Create a BoxView

To draw a rectangle or square, create a [BoxView](#) object and set its `Color`, [WidthRequest](#), and [HeightRequest](#) properties. Optionally, you can also set its `CornerRadius` property.

The following XAML example shows how to create a [BoxView](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:BasicBoxView"
              x:Class="BasicBoxView.MainPage">
    <BoxView Color="CornflowerBlue"
              CornerRadius="10"
              WidthRequest="160"
              HeightRequest="160"
              VerticalOptions="Center"
              HorizontalOptions="Center" />
</ContentPage>
```

In this example, a cornflower blue [BoxView](#) is displayed in the center of the page:



The [WidthRequest](#) and [HeightRequest](#) properties are measured in device-independent units.

ⓘ **Note**

A [BoxView](#) can also be a child of an [AbsoluteLayout](#). In this case, both the location and size of the [BoxView](#) are set using the [LayoutBounds](#) attached bindable property.

A [BoxView](#) can also be sized to resemble a line of a specific width and thickness.

Button

Article • 10/29/2024

The .NET Multi-platform App UI (.NET MAUI) [Button](#) displays text and responds to a tap or click that directs the app to carry out a task. A [Button](#) usually displays a short text string indicating a command, but it can also display a bitmap image, or a combination of text and an image. When the [Button](#) is pressed with a finger or clicked with a mouse it initiates that command.

[Button](#) defines the following properties:

- `BorderColor`, of type [Color](#), describes the border color of the button.
- `BorderWidth`, of type `double`, defines the width of the button's border.
- `CharacterSpacing`, of type `double`, defines the spacing between characters of the button's text.
- `Command`, of type [ICommand](#), defines the command that's executed when the button is tapped.
- `CommandParameter`, of type `object`, is the parameter that's passed to `Command`.
- `ContentLayout`, of type `ButtonContentLayout`, defines the object that controls the position of the button image and the spacing between the button's image and text.
- `CornerRadius`, of type `int`, describes the corner radius of the button's border.
- `FontAttributes`, of type `FontAttributes`, determines text style.
- `FontAutoScalingEnabled`, of type `bool`, defines whether the button text will reflect scaling preferences set in the operating system. The default value of this property is `true`.
- `FontFamily`, of type `string`, defines the font family.
- `FontSize`, of type `double`, defines the font size.
- `ImageSource`, of type [ImageSource](#), specifies a bitmap image to display as the content of the button.
- `LineBreakMode`, of type `LineBreakMode`, determines how text should be handled when it can't fit on one line.
- `Padding`, of type `Thickness`, determines the button's padding.
- `Text`, of type `string`, defines the text displayed as the content of the button.
- `TextColor`, of type [Color](#), describes the color of the button's text.
- `TextTransform`, of type `TextTransform`, defines the casing of the button's text.

These properties are backed by [BindableProperty](#) objects, which means that they can be targets of data bindings, and styled.

❗ Note

While [Button](#) defines an [ImageSource](#) property, that allows you to display a image on the [Button](#), this property is intended to be used when displaying a small icon next to the [Button](#) text.

In addition, [Button](#) defines `Clicked`, `Pressed`, and `Released` events. The `Clicked` event is raised when a [Button](#) tap with a finger or mouse pointer is released from the button's surface. The `Pressed` event is raised when a finger presses on a [Button](#), or a mouse button is pressed with the pointer positioned over the [Button](#). The `Released` event is raised when the finger or mouse button is released. Generally, a `Clicked` event is also raised at the same time as the `Released` event, but if the finger or mouse pointer slides away from the surface of the [Button](#) before being released, the `Clicked` event might not occur.

📌 Important

A [Button](#) must have its `IsEnabled` property set to `true` for it to respond to taps.

Create a Button

To create a button, create a [Button](#) object and handle its `Clicked` event.

The following XAML example show how to create a [Button](#):

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="ButtonDemos.BasicButtonClickPage"
    Title="Basic Button Click">
    <StackLayout>
        <Button Text="Click to Rotate Text!"
            VerticalOptions="Center"
            HorizontalOptions="Center"
            Clicked="OnButtonClicked" />
        <Label x:Name="label"
            Text="Click the Button above"
            FontSize="18"
            VerticalOptions="Center"
            HorizontalOptions="Center" />
    </StackLayout>
</ContentPage>
```

The `Text` property specifies the text that appears in the `Button`. The `Clicked` event is set to an event handler named `OnButtonClicked`. This handler is located in the code-behind file:

C#

```
public partial class BasicButtonClickPage : ContentPage
{
    public BasicButtonClickPage ()
    {
        InitializeComponent ();

        async void OnButtonClicked(object sender, EventArgs args)
        {
            await label.RelRotateTo(360, 1000);
        }
    }
}
```

In this example, when the `Button` is tapped, the `OnButtonClicked` method executes. The `sender` argument is the `Button` object responsible for this event. You can use this to access the `Button` object, or to distinguish between multiple `Button` objects sharing the same `Clicked` event. The `Clicked` handler calls an animation function that rotates the `Label` 360 degrees in 1000 milliseconds:



The equivalent C# code to create a `Button` is:

C#

```
Button button = new Button
{
    Text = "Click to Rotate Text!",
    VerticalOptions = LayoutOptions.Center,
    HorizontalOptions = LayoutOptions.Center
};
button.Clicked += async (sender, args) => await label.RelRotateTo(360,
1000);
```

Use the command interface

An app can respond to [Button](#) taps without handling the `Clicked` event. The [Button](#) implements an alternative notification mechanism called the *command* or *commanding* interface. This consists of two properties:

- `Command` of type [ICommand](#), an interface defined in the [System.Windows.Input](#) namespace.
- `CommandParameter` property of type [Object](#).

This approach is particularly suitable in connection with data-binding, and particularly when implementing the Model-View-ViewModel (MVVM) pattern. In an MVVM application, the viewmodel defines properties of type [ICommand](#) that are then connected to [Button](#) objects with data bindings. .NET MAUI also defines `Command` and `Command<T>` classes that implement the [ICommand](#) interface and assist the viewmodel in defining properties of type [ICommand](#). For more information about commanding, see [Commanding](#).

The following example shows a very simple viewmodel class that defines a property of type `double` named `Number`, and two properties of type [ICommand](#) named `MultiplyBy2Command` and `DivideBy2Command`:

C#

```
public class CommandDemoViewModel : INotifyPropertyChanged
{
    double number = 1;

    public event PropertyChangedEventHandler PropertyChanged;

    public ICommand MultiplyBy2Command { get; private set; }
    public ICommand DivideBy2Command { get; private set; }

    public CommandDemoViewModel()
    {
        MultiplyBy2Command = new Command(() => Number *= 2);
        DivideBy2Command = new Command(() => Number /= 2);
    }

    public double Number
    {
        get
        {
            return number;
        }
        set
        {
            if (number != value)
            {
                number = value;
                PropertyChanged?.Invoke(this, new
```



```

PropertyChangedEventArgs("Number"));
    }
}
}
}

```

In this example, the two `ICommand` properties are initialized in the class's constructor with two objects of type `Command`. The `Command` constructors include a little function (called the `execute` constructor argument) that either doubles or halves the value of the `Number` property.

The following XAML example consumes the `CommandDemoViewModel` class:

XAML

```

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:ButtonDemos"
    x:Class="ButtonDemos.BasicButtonCommandPage"
    Title="Basic Button Command">
    <ContentPage.BindingContext>
        <local:CommandDemoViewModel />
    </ContentPage.BindingContext>

    <StackLayout>
        <Label Text="{Binding Number, StringFormat='Value is now {0}}'"
            FontSize="18"
            VerticalOptions="Center"
            HorizontalOptions="Center" />
        <Button Text="Multiply by 2"
            VerticalOptions="Center"
            HorizontalOptions="Center"
            Command="{Binding MultiplyBy2Command}" />
        <Button Text="Divide by 2"
            VerticalOptions="Center"
            HorizontalOptions="Center"
            Command="{Binding DivideBy2Command}" />
    </StackLayout>
</ContentPage>

```

In this example, the `Label` element and two `Button` objects contain bindings to the three properties in the `CommandDemoViewModel` class. As the two `Button` objects are tapped, the commands are executed, and the number changes value. The advantage of this approach over `Clicked` handlers is that all the logic involving the functionality of this page is located in the viewmodel rather than the code-behind file, achieving a better separation of the user interface from the business logic.

It's also possible for the `Command` objects to control the enabling and disabling of the `Button` objects. For example, suppose you want to limit the range of number values between 2^{10} and 2^{-10} . You can add another function to the constructor (called the `canExecute` argument) that returns `true` if the `Button` should be enabled:

C#

```
public class CommandDemoViewModel : INotifyPropertyChanged
{
    ...
    public CommandDemoViewModel()
    {
        MultiplyBy2Command = new Command(
            execute: () =>
            {
                Number *= 2;
                ((Command)MultiplyBy2Command).ChangeCanExecute();
                ((Command)DivideBy2Command).ChangeCanExecute();
            },
            canExecute: () => Number < Math.Pow(2, 10));

        DivideBy2Command = new Command(
            execute: () =>
            {
                Number /= 2;
                ((Command)MultiplyBy2Command).ChangeCanExecute();
                ((Command)DivideBy2Command).ChangeCanExecute();
            },
            canExecute: () => Number > Math.Pow(2, -10));
    }
    ...
}
```

In this example, the calls to the `ChangeCanExecute` method of `Command` are required so that the `Command` method can call the `canExecute` method and determine whether the `Button` should be disabled or not. With this code change, as the number reaches the limit, the `Button` is disabled.

It's also possible for two or more `Button` elements to be bound to the same `ICommand` property. The `Button` elements can be distinguished using the `CommandParameter` property of `Button`. In this case, you'll want to use the generic `Command<T>` class. The `CommandParameter` object is then passed as an argument to the `execute` and `canExecute` methods. For more information, see [Commanding](#).

Press and release the button

The `Pressed` event is raised when a finger presses on a `Button`, or a mouse button is pressed with the pointer positioned over the `Button`. The `Released` event is raised when the finger or mouse button is released. Generally, a `Clicked` event is also raised at the same time as the `Released` event, but if the finger or mouse pointer slides away from the surface of the `Button` before being released, the `Clicked` event might not occur.

The following XAML example shows a `Label` and a `Button` with handlers attached for the `Pressed` and `Released` events:

XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="ButtonDemos.PressAndReleaseButtonPage"
              Title="Press and Release Button">
    <StackLayout>
        <Button Text="Press to Rotate Text!"
                VerticalOptions="Center"
                HorizontalOptions="Center"
                Pressed="OnButtonPressed"
                Released="OnButtonReleased" />
        <Label x:Name="label"
                Text="Press and hold the Button above"
                FontSize="18"
                VerticalOptions="Center"
                HorizontalOptions="Center" />
    </StackLayout>
</ContentPage>
```

The code-behind file animates the `Label` when a `Pressed` event occurs, but suspends the rotation when a `Released` event occurs:

C#

```
public partial class PressAndReleaseButtonPage : ContentPage
{
    IDispatcherTimer timer;
    Stopwatch stopwatch = new Stopwatch();

    public PressAndReleaseButtonPage()
    {
        InitializeComponent();

        timer = Dispatcher.CreateTimer();
        timer.Interval = TimeSpan.FromMilliseconds(16);
        timer.Tick += (s, e) =>
        {
            label.Rotation = 360 * (stopwatch.Elapsed.TotalSeconds % 1);
        };
    }
}
```

```

    }

    void OnButtonPressed(object sender, EventArgs args)
    {
        stopwatch.Start();
        timer.Start();
    }

    void OnButtonReleased(object sender, EventArgs args)
    {
        stopwatch.Stop();
        timer.Stop();
    }
}

```

The result is that the [Label](#) only rotates while a finger is in contact with the [Button](#), and stops when the finger is released.

Button visual states

[Button](#) has a `Pressed` [VisualState](#) that can be used to initiate a visual change to the [Button](#) when pressed, provided that it's enabled.

The following XAML example shows how to define a visual state for the `Pressed` state:

XAML

```

<Button Text="Click me!"
    ...>
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroupList>
            <VisualStateGroup x:Name="CommonStates">
                <VisualState x:Name="Normal">
                    <VisualState.Setters>
                        <Setter Property="Scale"
                            Value="1" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState x:Name="Pressed">
                    <VisualState.Setters>
                        <Setter Property="Scale"
                            Value="0.8" />
                    </VisualState.Setters>
                </VisualState>
                <VisualState x:Name="PointerOver" />
            </VisualStateGroup>
        </VisualStateGroupList>
    </VisualStateManager.VisualStateGroups>
</Button>

```

In this example, the `Pressed VisualState` specifies that when the `Button` is pressed, its `Scale` property will be changed from its default value of 1 to 0.8. The `Normal VisualState` specifies that when the `Button` is in a normal state, its `Scale` property will be set to 1. Therefore, the overall effect is that when the `Button` is pressed, it's rescaled to be slightly smaller, and when the `Button` is released, it's rescaled to its default size.

❗ Important

For a `Button` to return to its `Normal` state the `VisualStateManager` must also define a `PointerOver` state. If you use the styles `ResourceDictionary` created by the .NET MAUI app project template, you'll already have an implicit `Button` style that defines the `PointerOver` state.

For more information about visual states, see [Visual states](#).

Use bitmaps with buttons

The `Button` class defines an `ImageSource` property that allows you to display a small bitmap image on the `Button`, either alone or in combination with text. You can also specify how the text and image are arranged. The `ImageSource` property is of type `ImageSource`, which means that the bitmaps can be loaded from a file, embedded resource, URI, or stream.

Bitmaps aren't scaled to fit a `Button`. The best size is usually between 32 and 64 device-independent units, depending on how large you want the bitmap to be.

You can specify how the `Text` and `ImageSource` properties are arranged on the `Button` using the `ContentLayout` property of `Button`. This property is of type `ButtonContentLayout`, and its constructor has two arguments:

- A member of the `ImagePosition` enumeration: `Left`, `Top`, `Right`, or `Bottom` indicating how the bitmap appears relative to the text.
- A `double` value for the spacing between the bitmap and the text.

In XAML, you can create a `Button` and set the `ContentLayout` property by specifying only the enumeration member, or the spacing, or both in any order separated by commas:

XAML

```
<Button Text="Button text"
        ImageSource="button.png"/>
```

```
ContentLayout="Right, 20" />
```

The equivalent C# code is:

C#

```
Button button = new Button
{
    Text = "Button text",
    ImageSource = new FileImageSource
    {
        File = "button.png"
    },
    ContentLayout = new
    Button.ButtonContentLayout(Button.ButtonContentLayout.ImagePosition.Right,
    20)
};
```

ⓘ Note

If a [Button](#) contains text and an image it might not be possible to fit all the content inside the button, and so you should size your image manually to achieve your desired layout.

Disable a Button

Sometimes an app enters a state where a [Button](#) click is not a valid operation. In such cases, the [Button](#) can be disabled by setting its `IsEnabled` property to `false`.